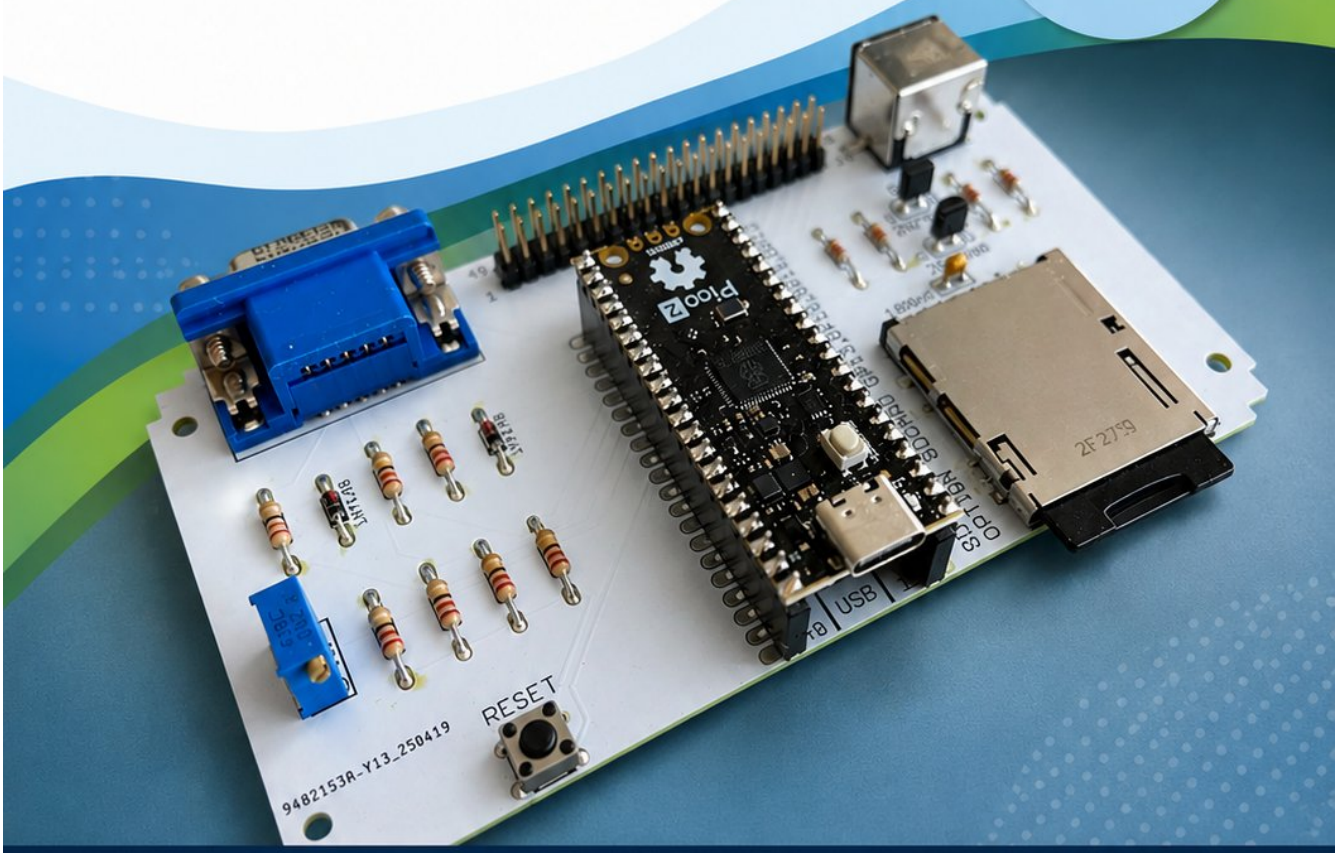
>_
Programmieren leicht gemacht!

# Mein erstes MMBasic Programm

## Schritt für Schritt in die Welt von MMBasic auf dem PicoMite und Colour Maximite

Für Einsteiger ohne Vorkenntnisse geeignet!




Erste Schritte mit MMBasic

Programme eingeben & starten

Textausgabe und Befehle

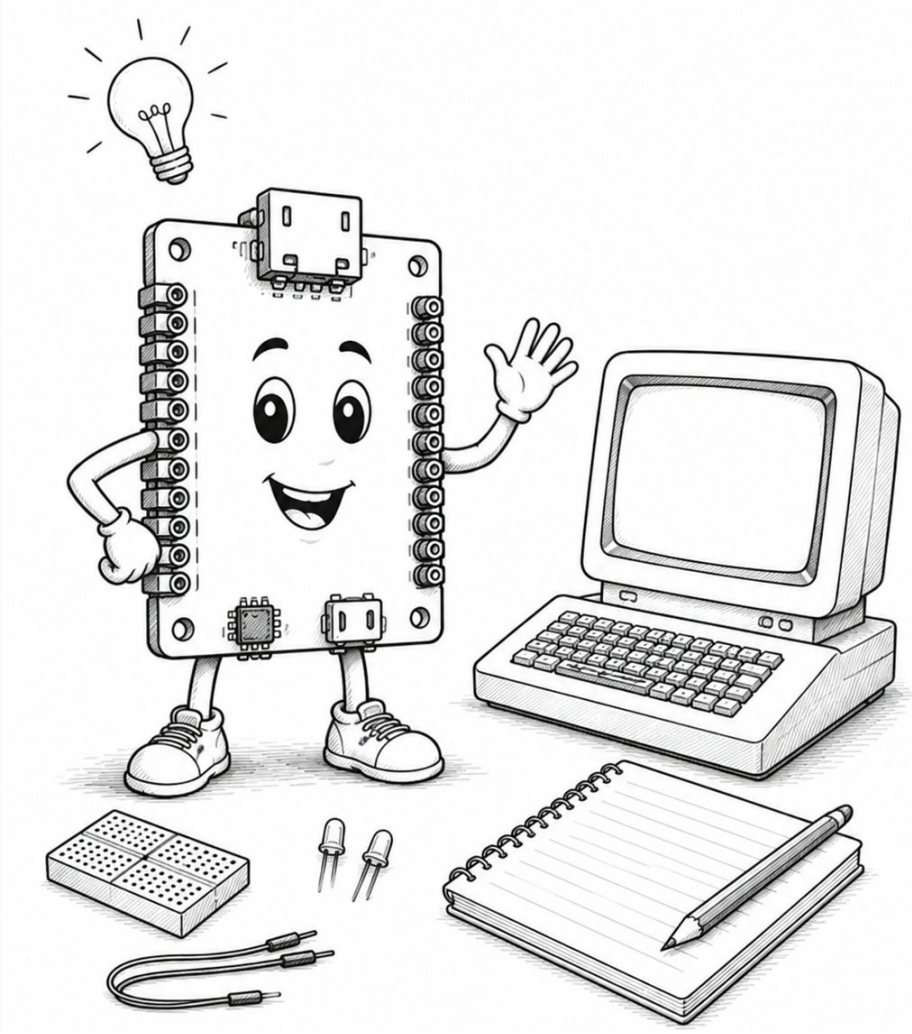
Beispiele & Übungen zum Mitmachen

# Mein erstes MMBasic Programm

**Schritt für Schritt in die Welt von MMBasic**

für PicoMite und Colour Maximite

Manfred Becker



Version 0.23
2026-11-10

# Impressum

Autor: Manfred Becker

GitHub-Projektseite:
https://github.com/ManiBecker/MeinErstesMMBasicProgramm

Webseite zum Tutorial:
https://mikrocontroller.bplaced.net/wordpress/mein-erstes-mmbasic-programm-vorwort/

Dieses Tutorial entsteht Schritt für Schritt öffentlich auf GitHub. Dort stehen auch die BASIC-Programme und weitere Projektdateien zum Download bereit.

Die aktuellste Version des PDFs zum Download, begleitende Informationen, Neuigkeiten und zusätzliche Beiträge zum Projekt finden sich auf der Webseite zum Tutorial.

Mein besonderer Dank gilt Geoff Graham, dem Entwickler von MMBasic, sowie Peter Mather, der MMBasic für den Colour Maximite 2 und die PicoMite-Familie entscheidend weiterentwickelt hat.

Ohne ihre jahrelange Arbeit und die frei verfügbaren Handbücher wäre dieses Tutorial nicht möglich.

**Hinweis zur Entstehung des Tutorials**

Dieses Tutorial wurde vom Autor erstellt und wird kontinuierlich weiterentwickelt.

Bei der Ausarbeitung der Texte, der sprachlichen Überarbeitung sowie der Strukturierung einzelner Kapitel kamen Werkzeuge auf Basis generativer Künstlicher Intelligenz (KI) unterstützend zum Einsatz.

Alle fachlichen Inhalte, Programmbeispiele und technischen Aussagen wurden vom Autor ausgewählt, geprüft und für die Veröffentlichung freigegeben.

**Hinweis zu den Illustrationen**

Viele der Illustrationen in diesem Tutorial wurden mit Unterstützung generativer Künstlicher Intelligenz (KI) erstellt.

Die Bildideen, die fachlichen Inhalte sowie die Auswahl der endgültigen Illustrationen stammen vom Autor. Die KI wurde ausschließlich als Werkzeug zur Erstellung der Abbildungen eingesetzt.

Ziel war es, technische Sachverhalte anschaulich zu vermitteln und den Einstieg in MMBasic zu erleichtern.

# Vorwort von Geoff Graham

MMBasic entstand Ende 2010 als Firmware für einen Computer, den ich entwickelt hatte und der den Namen **Maximite** trug. Dieser Computer sollte als Selbstbauprojekt in der australischen Elektronikzeitschrift **Silicon Chip** veröffentlicht werden. Als Grundlage diente der PIC32, ein damals neuer 32-Bit-Mikrocontroller von Microchip.

Ich wollte einen eigenständigen Computer schaffen, der nach dem Einschalten direkt mit einer BASIC-Eingabeaufforderung startet – ganz so wie die Heimcomputer der 1970er- und 1980er-Jahre, beispielsweise der Apple II, der Tandy TRS-80 oder der Commodore 64.

Damals gab es keinen wirklich brauchbaren BASIC-Interpreter, der frei verfügbar war. Also beschloss ich, meinen eigenen zu schreiben. So entstand auch der Name MMBasic: der BASIC-Interpreter für den Maximite-Computer.

Die erste Version von MMBasic war noch recht einfach gehalten, da sie die BASIC-Dialekte der 1970er-Jahre nachbilden sollte, die ebenfalls vergleichsweise schlicht waren. Doch zu meiner großen Überraschung wurde der Maximite mit MMBasic ein riesiger Erfolg. Viele Tausend begeisterte Hobbyelektroniker bauten den Computer nach.

Dieser Erfolg motivierte mich, MMBasic kontinuierlich zu verbessern und zu erweitern. Aus dem ursprünglich einfachen Interpreter entwickelte sich nach und nach eine ausgereifte Programmiersprache, mit der sich auch große und komplexe Programme erstellen lassen.

Bis 2019 führte ich die Entwicklung von MMBasic selbst fort. Dann übernahm Peter Mather die Herausforderung, den Interpreter weiter zu verfeinern und die Unterstützung für neue Geräte und Bildschirme auszubauen.

Ein weiterer großer Entwicklungsschritt folgte 2021 mit der Veröffentlichung des Raspberry Pi Pico. Peter Mather brachte damals die erste Version von MMBasic heraus, die auf diesem neuen Mikrocontroller lief.

Der Raspberry Pi Pico und seine Nachfolger zeichnen sich unter anderem dadurch aus, dass sie über reichlich Flash-Speicher für die Firmware verfügen. Peter nutzte diese Möglichkeiten im Laufe der Zeit, um MMBasic um eine Vielzahl neuer Funktionen zu erweitern. Dazu gehören unter anderem HDMI-Grafikausgabe, WLAN- und Internetanbindung, Compiler-Technologie und vieles mehr. So entwickelte sich MMBasic zu der leistungsfähigen Programmiersprache, die wir heute kennen.

Von Anfang an war ich davon überzeugt, dass eine gute Dokumentation entscheidend für den Erfolg eines Projekts ist. Schließlich nützt es wenig, eine bemerkenswerte neue Software oder ein faszinierendes Gerät zu entwickeln, wenn niemand weiß, wie man damit umgeht – etwas, das bei Hobbyprojekten leider nur allzu häufig vorkommt.

Deshalb gab es für MMBasic von Beginn an ein vollständiges Benutzerhandbuch, das sowohl ausführlich als auch gut verständlich sein sollte.

Umso mehr habe ich mich gefreut, von Manfreds neuem Projekt zu erfahren: diesem umfassenden MMBasic-Tutorial.

Es bietet einen einsteigerfreundlichen Zugang zu MMBasic. Angefangen beim allerersten BASIC-Programm führt es die Leser Schritt für Schritt an immer anspruchsvollere Themen heran – darunter Grafik, Sound, Dateizugriffe, Interrupts und vieles mehr.

Und das Beste daran: Es ist leicht verständlich und unterhaltsam geschrieben.

Also macht es euch bequem und entdeckt die Welt von MMBasic!

**Geoff Graham**

**2026**

# Vorwort des Autors

Meine erste Begegnung mit MMBasic liegt inzwischen mehr als zehn Jahre zurück. Damals hatte mein jüngerer Bruder Uwe eine Portierung von MMBasic auf das STM32F746-Discovery-Board realisiert. Ein BASIC-Interpreter auf einem kleinen Einplatinencomputer – das war genau meine Welt: Technik und Programmierung ohne viel Ballast. Einschalten, loslegen und einfach Spaß am Programmieren haben.

Später stieß ich auf der Webseite *Geoff's Projects* von Geoff Graham, dem Entwickler von MMBasic und dem ursprünglichen Colour Maximite, auf weitere Rechner, auf denen MMBasic zu Hause war. Besonders der PicoMite und die Maximite-Rechner weckten mein Interesse. Mit Tastatur und Monitor wurden daraus kleine, eigenständige Computer, bei denen man nach dem Einschalten praktisch sofort mit dem Programmieren beginnen konnte.

Lange Zeit war vor allem der Colour Maximite 2 – kurz CMM2 – ein echter Wunschcomputer für mich. Leider war er kaum noch zu bekommen. Irgendwann entdeckte ich schließlich ein Exemplar bei einer eBay-Auktion und konnte nicht widerstehen.

Als der CMM2 dann endlich vor mir stand, war es um mich geschehen. Ich mochte diesen Computer von der ersten Minute an. Er verkörperte genau das, was mich schon immer an kleinen Computersystemen begeistert hat: einschalten, programmieren, ausprobieren – und unmittelbar sehen, was passiert.

Später fand ich heraus, dass sich die Platinen für viele dieser Systeme bei Leiterplattenherstellern wie JLCPCB fertigen und teilweise sogar bestücken lassen. Damit begannen meine eigenen Projekte mit den Reference Designs der PicoMites und Maximites. Besonders bei Sammelbestellungen ließen sich die kleinen Computer erstaunlich günstig aufbauen, und überzählige Sets konnte ich an andere Interessierte weitergeben.

Dabei begegnete mir immer wieder dieselbe Frage:

*Gibt es die Dokumentation eigentlich auch auf Deutsch?*

Und noch etwas fehlte: ein deutschsprachiges Tutorial, das Einsteiger Schritt für Schritt an MMBasic heranführt. Nicht nur eine Befehlsreferenz, sondern eine Anleitung, mit der man einfach anfangen, Programme ausprobieren und nach und nach die Möglichkeiten von MMBasic entdecken kann.

Also setzte ich mich eines Tages hin und unterhielt mich mit meiner KI – ChatGPT – darüber, wie ein solches Tutorial aussehen könnte. Aus ersten Ideen entstand eine Struktur, und kurze Zeit später waren bereits die ersten Kapitel geschrieben. Viele weitere sollten folgen.

Was zunächst als überschaubares Tutorial begann, wuchs mit jedem neuen Thema, jedem Beispielprogramm und vielen praktischen Tests weiter. Aus dem Tutorial wurde schließlich das Buch, das Sie nun in den Händen halten.

ChatGPT hat mich bei seiner Entstehung als Werkzeug begleitet – beim Strukturieren von Ideen, beim Formulieren und bei der Überarbeitung von Texten und Programmen. Entscheidend blieb dabei aber immer die praktische Überprüfung: Die Beispiele wurden getestet, technische Angaben

mit der Dokumentation abgeglichen und vieles direkt auf den unterschiedlichen MMBasic-Systemen ausprobiert.

Ich hoffe, dass Ihnen dieses Buch den Einstieg in MMBasic erleichtert und vielleicht etwas von der Begeisterung vermittelt, die mich selbst an diesen kleinen Computern fasziniert:

**Einschalten, loslegen und Spaß am Programmieren haben.**

Wenn Ihnen das Buch beim Lesen und Experimentieren genauso viel Freude bereitet, wie mir seine Entstehung bereitet hat, dann hat es sein Ziel erreicht.

Manfred Becker

# Danksagung

Ein Buch wie dieses entsteht nicht ganz allein. Deshalb möchte ich mich bei einigen Menschen bedanken, ohne die es dieses Projekt in dieser Form vermutlich nicht geben würde.

Mein besonderer Dank gilt meinem Bruder **Uwe**. Durch seine Portierung von MMBasic auf das STM32F746-Discovery-Board hatte ich vor vielen Jahren meine erste Begegnung mit MMBasic. Damals konnte noch niemand ahnen, dass daraus irgendwann einmal dieses Buch entstehen würde.

Ein ganz besonderer Dank geht natürlich an **Geoff Graham**, den Schöpfer von MMBasic. Mit MMBasic hat er eine Programmiersprache geschaffen, die etwas bewahrt, das mich schon an den Heimcomputern früherer Jahre begeistert hat: Man schaltet den Computer ein und kann praktisch sofort mit dem Programmieren beginnen. Gleichzeitig ist MMBasic eine moderne und erstaunlich leistungsfähige BASIC-Implementierung, mit der sich weit mehr realisieren lässt als nur die ersten kleinen Programme.

Ebenso möchte ich **Peter Mather** danken. Seine kontinuierliche Weiterentwicklung von MMBasic und insbesondere des PicoMite hat eine beeindruckende Familie kleiner Computersysteme hervorgebracht. Viele der Möglichkeiten, die in diesem Buch vorgestellt werden, wären ohne seine Arbeit nicht vorhanden.

Mein Dank gilt außerdem all den Entwicklern, Platinen-Designern und Enthusiasten, die ihre Projekte, Reference Designs, Programme und Erfahrungen mit anderen teilen. Gerade diese Offenheit macht für mich einen großen Teil der MMBasic-Community aus.

Bedanken möchte ich mich auch bei den vielen Menschen in den Foren und Communities, die Fragen gestellt, Fehler gefunden, Hinweise gegeben oder einfach ihre eigenen Erfahrungen geteilt haben. Besonders hervorheben möchte ich dabei das Forum des Vereins zum Erhalt klassischer Computer e. V. (VzEkC), in dem ich selbst Mitglied bin und in dem die Entstehung dieses Buches mit vielen hilfreichen Rückmeldungen, Anregungen und Diskussionen begleitet wurde.

Manche dieser Rückmeldungen haben unmittelbar dazu geführt, dass ich etwas noch einmal ausprobiert, genauer erklärt oder in diesem Buch ergänzt habe. Gerade dieser Austausch zeigt für mich, wie wertvoll eine aktive Community rund um klassische Computer, moderne Nachbauten und Projekte wie MMBasic sein kann.

Auch **ChatGPT** war während der Entstehung dieses Buches ein hilfreicher Begleiter. Die KI half mir dabei, Ideen zu strukturieren, Formulierungen zu überarbeiten, Beispiele zu diskutieren und immer wieder neue Ansätze zu finden. Die technischen Inhalte und Programme habe ich dabei nicht einfach übernommen, sondern anhand der Dokumentation und auf realer Hardware überprüft. Gerade diese Zusammenarbeit aus eigenen Erfahrungen, praktischen Tests und KI-Unterstützung hat die Arbeit an diesem Buch für mich zu einem spannenden Experiment gemacht.

Und schließlich möchte ich mich bei allen bedanken, die dieses Buch lesen, die Beispielprogramme ausprobieren und vielleicht selbst anfangen, mit MMBasic zu experimentieren.

Denn genau darum geht es:

**Einschalten, loslegen und Spaß am Programmieren haben.**

# Inhaltsverzeichnis


Mein erstes MMBasic Programm . . . 2
- Impressum . . . 3
- Vorwort von Geoff Graham . . . 4
- Vorwort des Autors . . . 6
- Danksagung . . . 8
- Willkommen zu MMBasic . . . 12

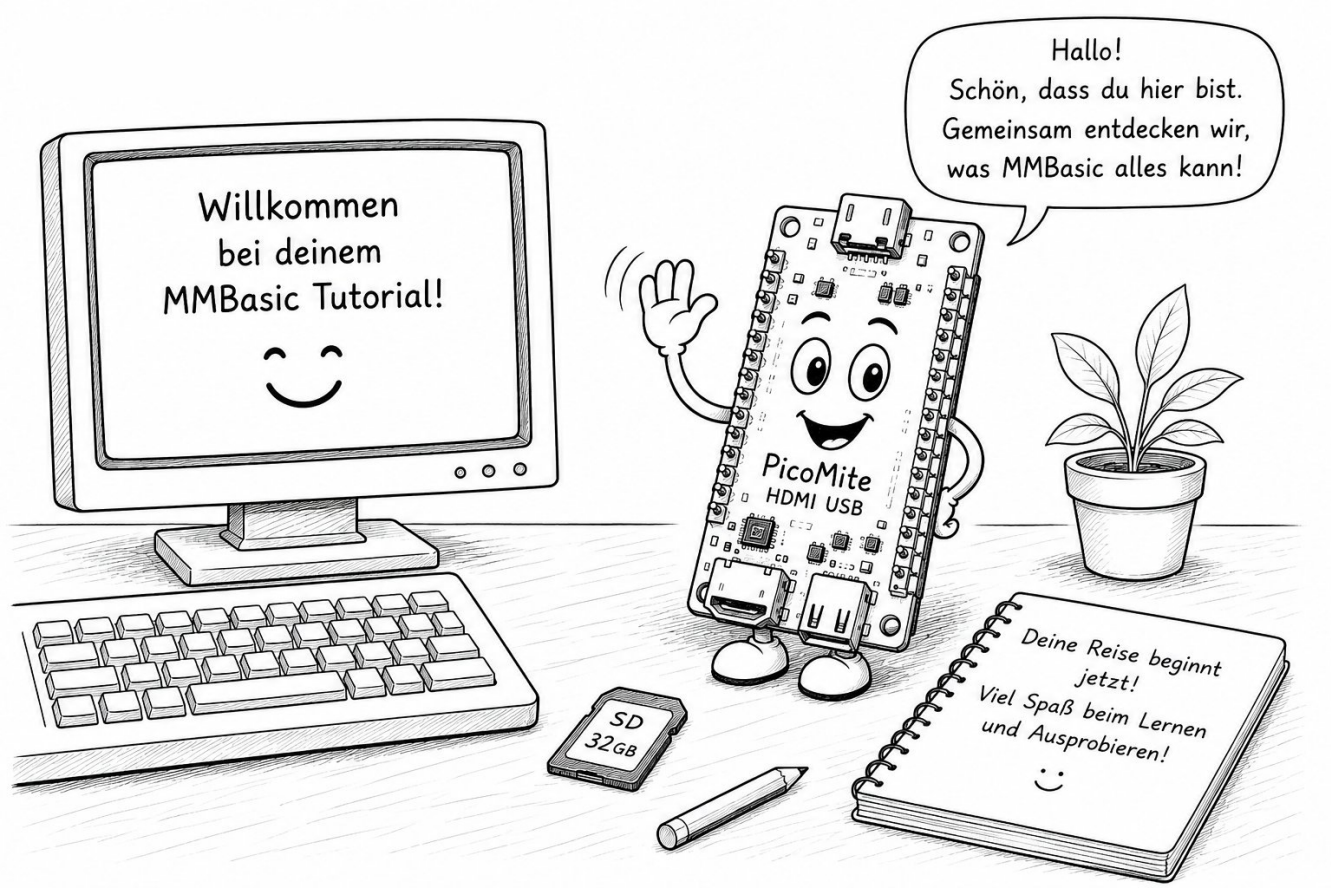
Teil I – Einstieg in MMBasic . . . 19
- 1. Hallo Welt! . . . 20
- 2. Mit MMBasic rechnen . . . 23
- 3. Variablen . . . 28
- 4. Benutzereingaben mit INPUT . . . 38
- 5. Entscheidungen mit IF . . . 48
- 6. Schleifen mit FOR … NEXT . . . 56
- 7. Zufallszahlen . . . 65
- 8. Unser erstes Spiel: Zahlenraten . . . 70
- 9. Zahlenraten Version 2 . . . 76

Teil II – Größere Programme entwickeln . . . 84
- 10. Programme strukturieren mit SUB und FUNCTION . . . 85
- 11. Arrays . . . 96
- 12. Mit Texten arbeiten . . . 111
- 13. Little Professor . . . 127
- 14. Little Professor Version 2 . . . 134
- 15. Little Professor Version 3 . . . 142
- 16. Dateien speichern und laden . . . 149

Teil III – Grafik und Spiele . . . 177
- 17. Die verschiedenen Grafikmodi . . . 178
- 18. Farben und Schriftarten . . . 191
- 19. Die ersten Grafikbefehle . . . 204
- 20. Rechtecke, Kreise und Texte . . . 216
- 21. Treffe die Scheibe . . . 226
- 22. Treffe die Scheibe – Version 2 . . . 232
- 23. Treffe die Scheibe – Version 3 . . . 239
- 24. Eine analoge Bahnhofsuhr . . . 250
- 25. Geometrische Muster und Spirographen . . . 262
- 26. Animationen und bewegte Grafiken . . . 271
- 27. Flackerfreie Grafik mit Framebuffer . . . 288
- 28. Layerbuffer und bewegliche Objekte . . . 297
- 29. Pong – das Spiel der Spiele . . . 305

Teil IV – Hardware und Elektronik . . . 331
30. Digitale Ein- und Ausgänge . . . 332
31. Eine LED zum Blinken bringen . . . 339
32. Taster und Schalter . . . 351
33. Eine Ampelsteuerung . . . 363
34. Analoge Eingänge . . . 374
35. PWM und LED-Dimmer . . . 387
36. Töne und Piezo-Summer . . . 397
37. Senso - Das elektronische Gedächtnisspiel . . . 408
Teil V – Werkzeuge und Entwicklung . . . 422
38. Der eingebaute Editor . . . 423
39. Programmieren mit MMEdit . . . 435
Teil VI – Dateien und Projekte . . . 448
40. Programme speichern, laden und Libraries verwenden . . . 449
41. Dateiverwaltung auf der SD-Karte . . . 462
42. Programme verketten mit CHAIN . . . 475
43. Nützliche Systeminformationen . . . 489
Teil VII – Fortgeschrittene Techniken . . . 499
44. Wichtige OPTION-Befehle . . . 500
45. Ereignisgesteuerte Programmierung mit Interrupts . . . 516
46. Datum und Uhrzeit . . . 529
47. Serielle Kommunikation . . . 543
48. WS2812 RGB-LEDs . . . 567
49. Spielcontroller und Gamepads . . . 579
50. Sprites und BLIT . . . 595
51. Von der Idee zum fertigen Spiel . . . 618
Epilog . . . 645
52. Wie geht es jetzt weiter? . . . 646
Anhang . . . 651
Anhang A: MMBasic-Befehlsreferenz . . . 652
Anhang B: MMBasic-Funktionsreferenz . . . 671
Stichwortverzeichnis . . . 678

# Willkommen zu MMBasic




Willkommen zu diesem MMBasic-Tutorial.

Dieses Buch richtet sich an alle, die MMBasic kennenlernen und damit eigene Programme entwickeln möchten.

Dabei spielt es keine Rolle, ob du bereits Programmiererfahrung besitzt oder zum ersten Mal eine Programmiersprache verwendest.

## Was ist MMBasic?

MMBasic ist eine moderne BASIC-Programmiersprache für Mikrocontroller und Computersysteme.

Sie verbindet die Einfachheit klassischer BASIC-Dialekte mit den Möglichkeiten moderner Hardware.

MMBasic läuft unter anderem auf:

- PicoMite
- PicoMite VGA
- PicoMite HDMI
- Colour Maximite 2

Mit MMBasic lassen sich nicht nur Konsolenprogramme entwickeln, sondern auch:

- Grafikanwendungen
- Spiele
- Hardwareprojekte
- Mess- und Steueranwendungen
- Netzwerkanwendungen

# Die Entwickler hinter MMBasic

MMBasic wurde von Geoff Graham entwickelt. Die Sprache entstand ursprünglich für den Maximite, einen kleinen Computer, der nach dem Einschalten direkt mit einer BASIC-Eingabeaufforderung startet.

Peter Mather entwickelte MMBasic für weitere leistungsfähige Plattformen weiter. Dazu gehören unter anderem der Colour Maximite 2 sowie die PicoMite-Familie für den Raspberry Pi Pico und Pico 2.

Ohne ihre langjährige Entwicklungsarbeit, ihre ausführlichen Handbücher und die kontinuierliche Pflege der Firmware wäre dieses Buch nicht möglich.

# Welche MMBasic-Systeme gibt es?

MMBasic ist keine Programmiersprache für nur einen bestimmten Computer.

Sie läuft heute auf verschiedenen Plattformen – vom einfachen Raspberry Pi Pico bis hin zu leistungsfähigen Komplettsystemen mit Grafik, Sound und Massenspeicher.

Wenn du gerade erst mit MMBasic beginnst, hilft dir die folgende Übersicht bei der Auswahl der passenden Plattform.

| Plattform | Benötigte Hardware | Grafik | GPIO | Empfehlung |
|---|---|---|---|---|
| MMBasic for Windows (MMB4W) | Windows-PC | Ja | Nein | Ideal zum Erlernen der Programmiersprache. |
| MMBasic for Linux (MMB4L) | Linux-PC oder Raspberry Pi | Ja | Nein | Ideal für Linux-Anwender. |
| MMBasic for MacOS (MMB4M) | Mac oder MacBook | Ja | Nein | Ideal für Mac-Anwender. |
| DOS MMBasic | DOS-PC oder DOSBox | Text | Nein | Gut für ältere PCs oder DOSBox. |
| Raspberry Pi Pico + Terminal | Pico + USB-Kabel | Text | Ja | Perfekt für erste Hardware-Experimente. |

| Plattform | Benötigte Hardware | Grafik | GPIO | Empfehlung |
|---|---|---|---|---|
| PicoMite VGA oder HDMI | PicoMite-System | Ja | Ja | Empfohlene Plattform für dieses Tutorial. |
| Colour Maximite 2 | Colour Maximite 2 | Ja | Ja | Leistungsfähiges Komplettsystem. |

Dieses Tutorial verwendet überwiegend den PicoMite als Referenzsystem. Die meisten Programme laufen jedoch auch auf anderen MMBasic-Systemen mit wenigen oder gar keinen Änderungen.

Wenn du zunächst nur die Programmiersprache kennenlernen möchtest, kannst du mit MMB4W, MMB4L, MMB4M oder DOS MMBasic direkt auf deinem Computer beginnen. Für die Hardwareprojekte in den späteren Kapiteln benötigst du allerdings ein MMBasic-System mit Ein- und Ausgängen, beispielsweise einen PicoMite.

# Welche Plattform ist die richtige für dich?

Wenn du einfach nur die Programmiersprache MMBasic kennenlernen möchtest, kannst du mit MMB4W, MMB4L, MMB4M oder DOS MMBasic direkt auf deinem Computer beginnen.

Möchtest du dagegen auch LEDs, Taster, Sensoren und andere elektronische Bauteile anschließen, empfiehlt sich ein PicoMite-System. Alle Hardwarebeispiele in diesem Tutorial wurden auf einem PicoMite entwickelt und getestet.

Besitzt du einen Colour Maximite 2, kannst du den größten Teil der Programme ebenfalls ohne Änderungen oder mit nur geringfügigen Anpassungen ausführen.

# Verwendete MMBasic-Firmware

Die Programme und Beispiele in diesem Tutorial wurden überwiegend mit der **MMBasic-Firmware Version 7.0** erstellt und überprüft. Mit dieser Firmwaregeneration wurden zahlreiche Funktionen der PicoMite-Familie und des Colour Maximite 2 vereinheitlicht.

Eine wichtige Neuerung betrifft die Kompatibilität der Grafikbefehle. Auf dem Colour Maximite 2 kann mit `OPTION PICOMITE VGA` beziehungsweise `OPTION PICOMITE HDMI` ein entsprechender PicoMite-Kompatibilitätsmodus aktiviert werden.

Dadurch lassen sich viele Programme, die ursprünglich für den PicoMite VGA oder PicoMite HDMI geschrieben wurden, auch auf dem Colour Maximite 2 ausführen, ohne dass umfangreiche Anpassungen erforderlich sind.

Allerdings sind nicht alle Funktionen der verschiedenen Systeme identisch. Insbesondere bei hardwarespezifischen Befehlen und Einstellungen können weiterhin Unterschiede bestehen.

# Warum dieses Tutorial?

Die offiziellen MMBasic-Handbücher von Geoff Graham und Peter Mather dokumentieren

ausführlich die Befehle, Funktionen und Möglichkeiten der Programmiersprache. Sie sind unverzichtbare Nachschlagewerke für alle, die mit MMBasic arbeiten.

**Dieses Tutorial soll diese Handbücher nicht ersetzen, sondern sinnvoll ergänzen.**

Hier steht das praktische Lernen im Vordergrund. Wir entwickeln gemeinsam Programme und Projekte und lernen dabei Schritt für Schritt die Programmiersprache kennen.

Wir beginnen mit einfachen Ausgaben auf dem Bildschirm und arbeiten uns nach und nach zu anspruchsvolleren Themen vor:

- Grafikanwendungen
- Spielen
- Hardwareprojekten
- elektronischen Experimenten

Dabei geht es nicht nur darum, einzelne Befehle kennenzulernen, sondern vor allem zu verstehen, wie aus vielen kleinen Bausteinen größere Programme entstehen.

Während die offiziellen Handbücher als umfassende Referenz dienen, begleitet dich dieses Tutorial auf deinem Weg vom ersten kleinen BASIC-Programm bis hin zu eigenen Projekten.

# Lernen durch praktische Beispiele

Viele Beispiele dieses Tutorials stammen aus realen Projekten und eigenen Erfahrungen.

Dazu gehören unter anderem:

- Little Professor
- Pong
- Ampelsteuerung
- Hardwareexperimente mit LEDs und Tastern
- Senso – Das elektronische Gedächtnisspiel

Jedes Projekt baut auf bereits gelerntem Wissen auf.

Dadurch entsteht Schritt für Schritt ein tieferes Verständnis der Sprache.

# Aufbau des Buches

Dieses Tutorial begleitet dich Schritt für Schritt beim Erlernen von MMBasic.

Wir beginnen mit einfachen Befehlen und kleinen Programmen.

Später beschäftigen wir uns unter anderem mit:

- Variablen

- Schleifen
- Unterprogrammen
- Grafikausgabe
- Spielen
- Dateiverarbeitung
- Hardwareprojekten
- Entwicklungswerkzeugen

Die einzelnen Kapitel bauen weitgehend aufeinander auf.

Deshalb empfiehlt es sich insbesondere für Einsteiger, das Tutorial der Reihe nach durchzuarbeiten.

# Wie arbeitet man mit MMBasic?

Eine Besonderheit von MMBasic ist, dass nach dem Einschalten sofort eine Eingabeaufforderung erscheint.

Beispielsweise:

```
>
```

An dieser Eingabeaufforderung können BASIC-Befehle direkt eingegeben und sofort ausgeführt werden.

Beispiel:

```
PRINT "Hallo Welt"
```

Nach dem Drücken der Eingabetaste erscheint unmittelbar:

```
Hallo Welt
```

Viele Beispiele der ersten Kapitel können deshalb direkt am Prompt ausprobiert werden.

Und genau das ist einer der großen Vorteile von MMBasic: Du kannst neue Befehle sofort ausprobieren und erhältst unmittelbar eine Rückmeldung. So lernst du die Sprache besonders schnell kennen.

# Befehle und Programme

Es gibt einen wichtigen Unterschied zwischen einzelnen Befehlen und vollständigen Programmen.

Einzelne Befehle können direkt am Prompt eingegeben werden.

Beispiel:

```
PRINT 2+3
```

Ausgabe:

```
5
```

Sobald ein Programm aus mehreren Zeilen besteht, wird jedoch ein Editor benötigt.

# Der eingebaute Editor

MMBasic enthält bereits einen komfortablen Vollbild-Editor.

Dieser wird mit dem Befehl:

```
EDIT
```

gestartet.

Im Editor können Programme erstellt, gespeichert und ausgeführt werden.

Den Editor lernen wir später ausführlich kennen.

# Programme speichern und laden

Programme können dauerhaft gespeichert und später wieder geladen werden.

Typische Befehle dafür sind:

```
SAVE "MEINPROG.BAS"

LOAD "MEINPROG.BAS"
```

Was diese Befehle genau machen und welche weiteren Möglichkeiten die Dateiverwaltung bietet, lernen wir in einem späteren Kapitel.

# Programmieren mit MMEdit

Alternativ kann auch ein PC verwendet werden.

Mit dem kostenlosen Programm MMEdit lassen sich Programme komfortabel auf dem Computer erstellen und anschließend zum PicoMite oder Colour Maximite übertragen.

Dazu wird lediglich eine USB-Verbindung benötigt.

MMEdit wird später ausführlich vorgestellt.

# Keine Angst vor Fehlern

Programmieren lernt man nicht durch Lesen allein.

Deshalb solltest du die Beispiele ausprobieren, verändern und auch bewusst Fehler machen.

Fehlermeldungen gehören zum Programmieren dazu.

Jeder erfahrene Entwickler produziert täglich Fehler.

Entscheidend ist, daraus zu lernen.

# Das Ziel dieses Buches

Wenn du dieses Tutorial vollständig durcharbeitest, wirst du nicht nur einzelne BASIC-Befehle kennen.

Du wirst in der Lage sein:

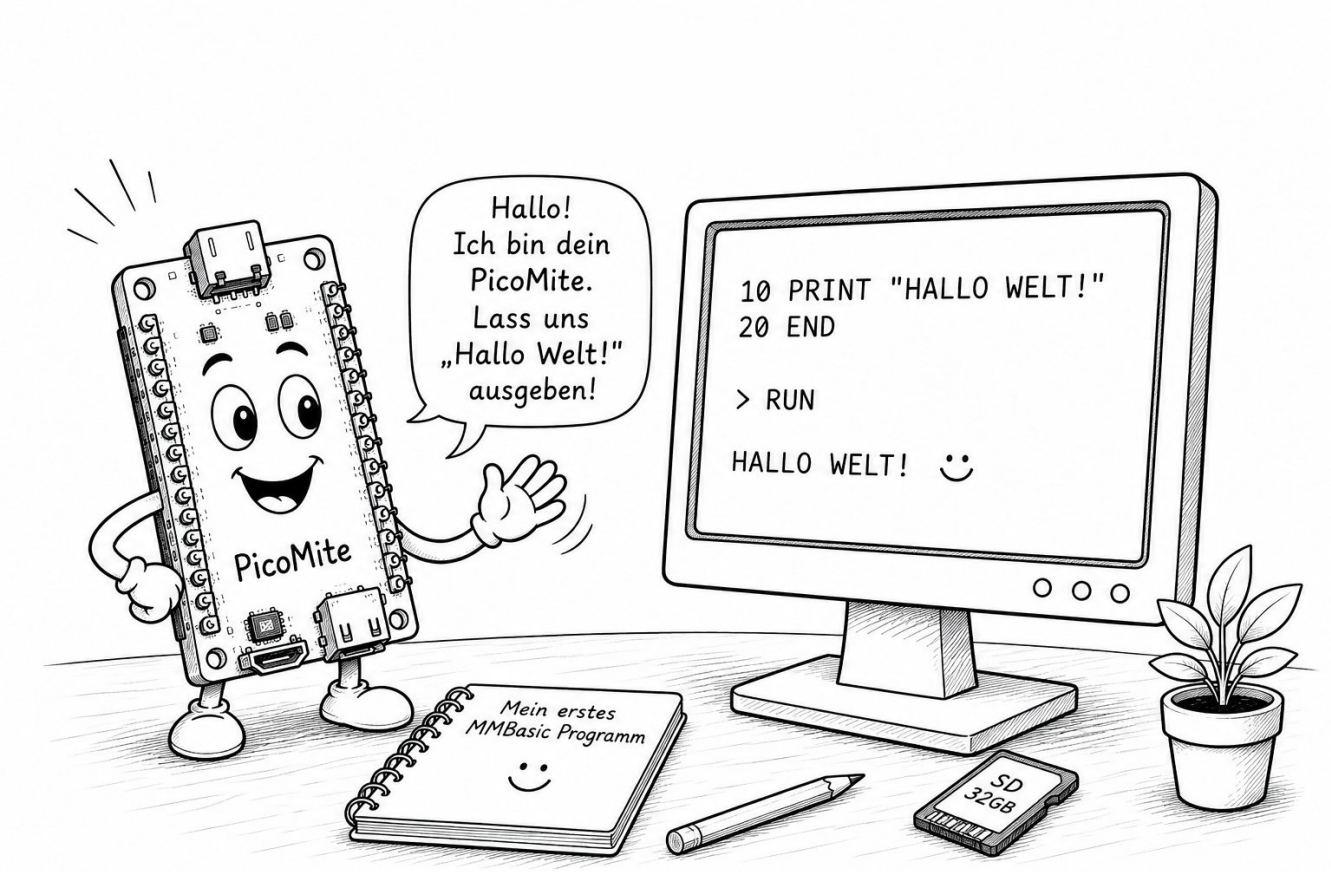
- eigene Programme zu schreiben
- Grafiken zu erzeugen
- Hardware anzusteuern
- Spiele zu entwickeln
- größere Projekte zu strukturieren

Und genau darum geht es.

Viel Spaß beim Programmieren mit MMBasic!

# Teil I – Einstieg in MMBasic

# Kapitel 1. Hallo Welt!




## 1.1. Das lernst du

In diesem Kapitel lernst du:

- wie ein MMBasic-Programm aussieht
- wie Programme gestartet werden
- wie Text auf dem Bildschirm ausgegeben wird

Am Ende des Kapitels hast du dein erstes eigenes MMBasic-Programm geschrieben und ausgeführt.

## 1.2. Unser erstes Programm

Gib folgendes Programm ein:

```
PRINT "Hallo Welt!"
```

Starte das Programm anschließend.

Auf dem Bildschirm erscheint:

```
Hallo Welt!
```

Herzlichen Glückwunsch!

Du hast gerade dein erstes MMBasic-Programm geschrieben.

## 1.3. Schritt für Schritt erklärt

Schauen wir uns das Programm genauer an:

```
PRINT "Hallo Welt!"
```

Die Anweisung `PRINT` gibt Text auf dem Bildschirm aus.

Alles, was zwischen den Anführungszeichen steht, wird unverändert angezeigt.

Du kannst jeden beliebigen Text ausgeben lassen.

Zum Beispiel:

```
PRINT "Hallo MMBasic!"
```

oder

```
PRINT "Ich programmiere einen PicoMite."
```

## 1.4. Experimentiere!

Ändere den Text zwischen den Anführungszeichen.

Probiere verschiedene Meldungen aus.

Beispiele:

```
PRINT "Hallo Colour Maximite!"
```

```
PRINT "Heute lerne ich MMBasic."
```

Was passiert, wenn du den Text änderst?

## 1.5. Probier’s selbst!

Versuche folgende Aufgaben:

1. Gib deinen Namen aus.
2. Gib deinen Wohnort aus.
3. Gib eine Begrüßung für einen Freund aus.
4. Gib drei verschiedene Meldungen nacheinander aus.

Beispiel:

```
PRINT "Hallo Welt!"
PRINT "Willkommen bei MMBasic!"
PRINT "Viel Spaß beim Programmieren!"
```

## 1.6. Zusammenfassung

In diesem Kapitel hast du die Anweisung `PRINT` kennengelernt.

Mit `PRINT` können Texte auf dem Bildschirm ausgegeben werden.

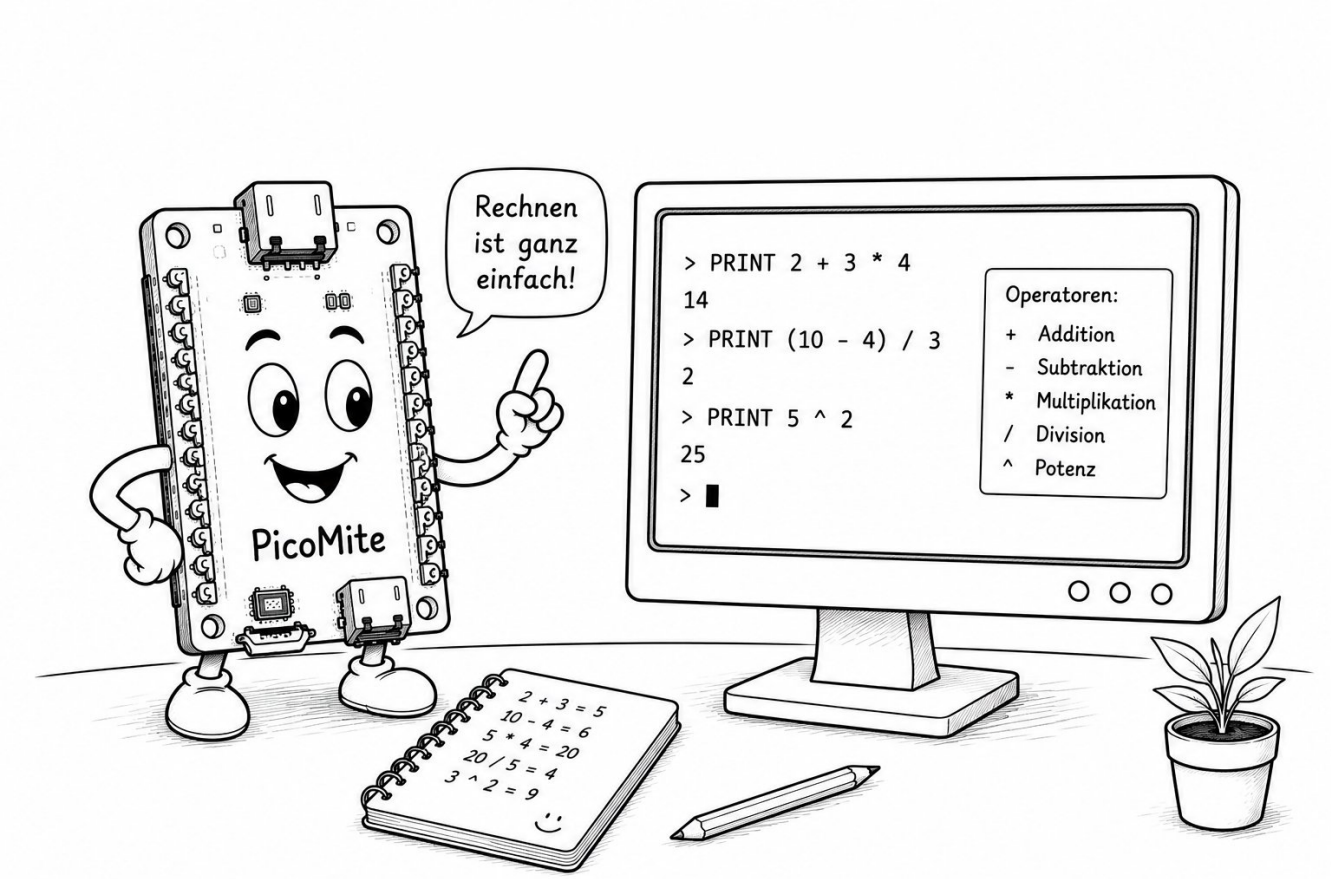
Im nächsten Kapitel lernst du, wie MMBasic Berechnungen ausführt und wie du verschiedene Rechenoperationen verwendest.

## 1.7. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 01_hallo_welt.bas

# Kapitel 2. Mit MMBasic rechnen




## 2.1. Das lernst du

In diesem Kapitel lernst du:

- wie MMBasic Berechnungen ausführt
- die vier Grundrechenarten
- die Verwendung von Klammern
- einige nützliche mathematische Funktionen

Am Ende des Kapitels kannst du MMBasic für einfache und komplexere Berechnungen einsetzen.

## 2.2. MMBasic kann rechnen

Im letzten Kapitel haben wir mit `PRINT` Text auf dem Bildschirm ausgegeben.

Mit `PRINT` können aber nicht nur Texte, sondern auch Berechnungsergebnisse ausgegeben werden.

Gib einmal folgende Zeile ein:

```
PRINT 1+2
```

MMBasic antwortet:

```
3
```

Die Berechnung wird sofort ausgeführt.

## 2.3. Die vier Grundrechenarten

Addition:

```
PRINT 5+3
```

Ergebnis:

```
8
```

Subtraktion:

```
PRINT 10-4
```

Ergebnis:

```
6
```

Multiplikation:

```
PRINT 6*7
```

Ergebnis:

```
42
```

Division:

```
PRINT 20/4
```

Ergebnis:

```
5
```

## 2.4. Mehrere Rechenarten kombinieren

Natürlich können auch mehrere Rechenarten miteinander kombiniert werden.

```
PRINT 2+3*4
```

Ergebnis:

```
14
```

Vielleicht hast du 20 erwartet?

MMBasic beachtet die bekannten Rechenregeln aus der Mathematik:

Multiplikation und Division werden vor Addition und Subtraktion ausgeführt.

## 2.5. Klammern verwenden

Mit Klammern kann die Reihenfolge verändert werden.

```
PRINT (2+3)*4
```

Ergebnis:

```
20
```

Klammern helfen dabei, Berechnungen übersichtlicher zu gestalten und Missverständnisse zu vermeiden.

## 2.6. Potenzen berechnen

Mit dem Zeichen ^ können Potenzen berechnet werden.

```
PRINT 2^8
```

Ergebnis:

```
256
```

Ein weiteres Beispiel:

```
PRINT 10^3
```

Ergebnis:

```
1000
```

## 2.7. Die Quadratwurzel

Die Funktion `SQR()` berechnet die Quadratwurzel einer Zahl.

```
PRINT SQR(25)
```

Ergebnis:

```
5
```

Weiteres Beispiel:

```
PRINT SQR(144)
```

Ergebnis:

```
12
```

## 2.8. Der Betrag einer Zahl

Die Funktion `ABS()` liefert den Betrag einer Zahl.

```
PRINT ABS(-10)
```

Ergebnis:

```
10
```

```
PRINT ABS(10)
```

Ergebnis:

```
10
```

## 2.9. Experimentiere!

Probiere einige eigene Berechnungen aus:

```
PRINT 123+456
PRINT 50*20
PRINT (10+20)*3
PRINT 3^4
PRINT SQR(81)
```

Was passiert?

## 2.10. Probier’s selbst!

Versuche folgende Aufgaben:

1. Berechne 15 + 27.
2. Berechne 100 - 37.
3. Berechne 12 * 12.
4. Berechne die Quadratwurzel von 225.
5. Berechne (5 + 7) * 8.

## 2.11. Zusammenfassung

In diesem Kapitel hast du gelernt, wie MMBasic Berechnungen ausführt.

Du kennst nun:

- die vier Grundrechenarten
- Klammern
- Potenzen
- die Funktionen `SQR()` und `ABS()`

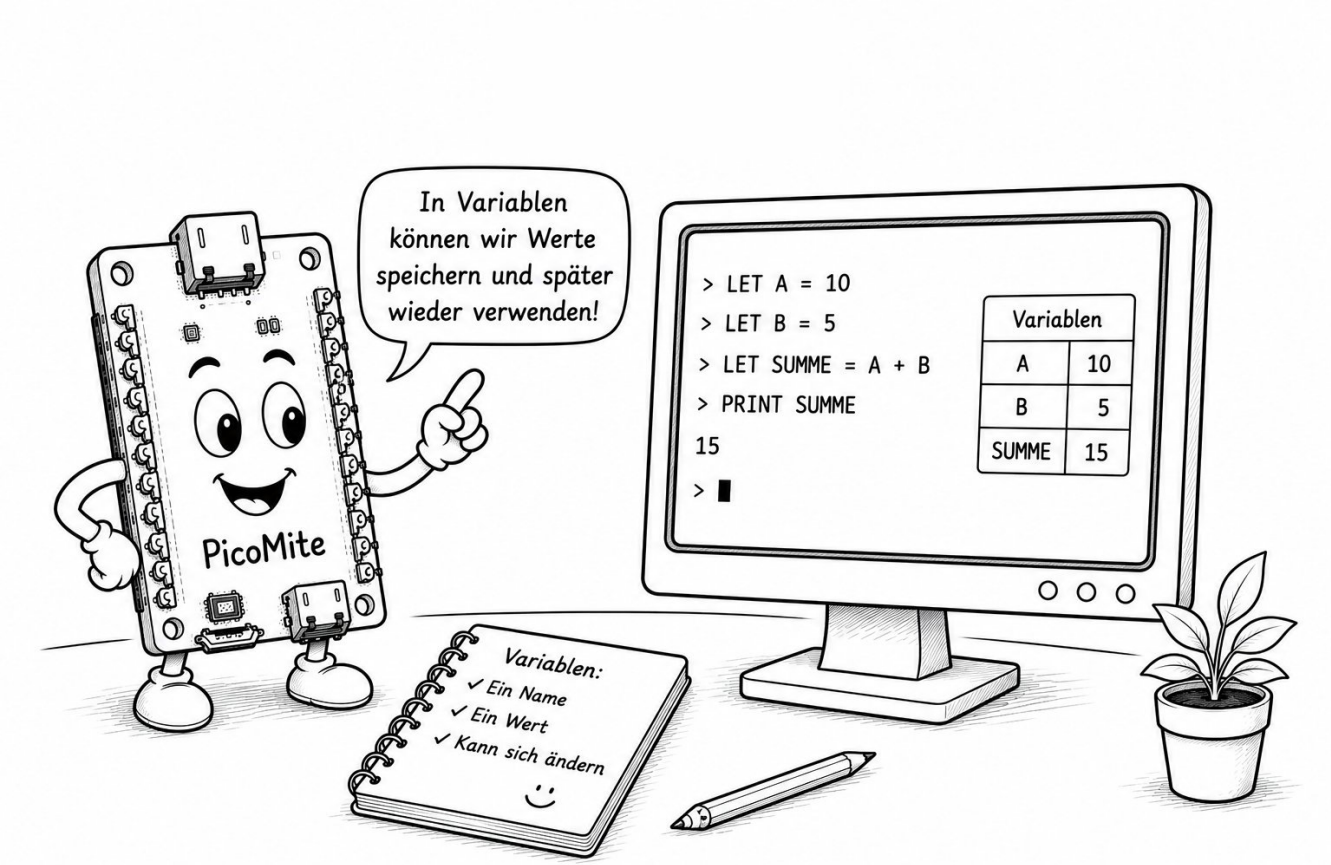
Im nächsten Kapitel lernen wir Variablen kennen. Damit können wir Werte speichern und später wiederverwenden.

## 2.12. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 02_rechenarten.bas

# Kapitel 3. Variablen




## 3.1. Das lernst du

In diesem Kapitel lernst du:

- was Variablen sind
- wie Werte gespeichert werden
- wie Variablen benannt werden
- welche grundlegenden Datentypen es gibt
- wie Variablen mit `DIM` deklariert werden
- wie `OPTION EXPLICIT` Tippfehler bei Variablennamen erkennen hilft
- wie Variablen in Berechnungen verwendet werden
- wie Variablen ihren Inhalt ändern können

Am Ende des Kapitels kannst du eigene Werte speichern und weiterverarbeiten.

## 3.2. Was ist eine Variable?

Eine Variable ist ein Speicherplatz für einen Wert.

Du kannst dir eine Variable wie eine beschriftete Schachtel vorstellen.

In diese Schachtel kannst du einen Wert hineinlegen und später wieder darauf zugreifen.

## 3.3. Unsere erste Variable

Gib folgende Zeilen ein:

```
A=10
PRINT A
```

Ausgabe:

```
10
```

MMBasic speichert den Wert 10 in der Variablen `A`.

Mit `PRINT` kann der gespeicherte Wert angezeigt werden.

## 3.4. Mehrere Variablen verwenden

Natürlich können mehrere Variablen gleichzeitig verwendet werden.

```
A=10
B=20

PRINT A
PRINT B
```

Ausgabe:

```
10
20
```

## 3.5. Aussagekräftige Variablennamen

Für die ersten Beispiele sind kurze Variablennamen wie `A` und `B` praktisch. In größeren Programmen ist es jedoch leichter zu verstehen, was eine Variable enthält, wenn ihr Name den Inhalt beschreibt.

Statt

```
A=60
B=100
```

kannst du zum Beispiel schreiben:

```
ALTER=60
PUNKTE=100
```

MMBasic erlaubt dazu längere Variablennamen.

Ein Variablenname beginnt mit einem Buchstaben. Danach können weitere Buchstaben, Ziffern, der Unterstrich _ und der Punkt `.` verwendet werden.

Unterstrich und Punkt können helfen, längere Variablennamen übersichtlicher zu gestalten. Zum Beispiel:

```
MAX_PUNKTE=100
SPIELER.NAME$="Manfred"
SPIELER.PUNKTE=50
```

`MAX_PUNKTE` besteht aus zwei Wörtern, die durch einen Unterstrich getrennt werden. Der Punkt kann verwendet werden, um zusammengehörige Namen zu strukturieren. `SPIELER.NAME$` und `SPIELER.PUNKTE` lassen beispielsweise sofort erkennen, dass beide Werte zu einem Spieler gehören.

Verwende _ und . nur dann, wenn sie einen Variablennamen wirklich verständlicher machen. Entscheidend ist nicht, möglichst lange Namen zu verwenden, sondern Namen, deren Bedeutung beim Lesen des Programms sofort erkennbar ist.

### 3.5.1. Schlüsselwörter dürfen keine Variablennamen sein

Bei der Wahl eines Variablennamens gibt es noch eine wichtige Regel: Schlüsselwörter von MMBasic dürfen nicht als Variablennamen verwendet werden.

Schlüsselwörter sind Wörter, die in MMBasic bereits eine feste Bedeutung haben. Dazu gehören beispielsweise Befehle und Anweisungen wie `PRINT`, `INPUT`, `IF` oder `FOR`.

Folgende Variablennamen wären deshalb nicht zulässig:

```
PRINT=10
INPUT=20
FOR=5
```

Verwende stattdessen einen Namen, der den Inhalt der Variablen beschreibt:

```
ANZAHL=10
EINGABE=20
ZAHL=5
```

Da MMBasic bei Variablennamen nicht zwischen Groß- und Kleinschreibung unterscheidet, lässt sich diese Regel nicht durch eine andere Schreibweise umgehen. `PRINT`, `Print` und `print` werden gleichermaßen als Schlüsselwort erkannt.

# 3.6. Zahlen oder Text? – Ein erster Blick auf Datentypen

Bis jetzt haben unsere Variablen Zahlen enthalten:

```
A
B
ALTER
PUNKTE
```

MMBasic kann aber auch Texte in Variablen speichern. Dazu verwenden wir eine String-Variable. Eine String-Variable erkennst du am Dollarzeichen `$` am Ende ihres Namens:

```
NAME$="Manfred"
WOHNORT$="Heidelberg"

PRINT NAME$
PRINT WOHNORT$
```

Das Dollarzeichen `$` zeigt an, dass die Variable einen Text enthält. Solche Texte werden in der Programmierung als **Strings** bezeichnet.

Eine Variable besitzt also einen bestimmten **Datentyp**. Der Datentyp legt fest, welche Art von Wert in einer Variablen gespeichert werden kann.

Für den Anfang genügt uns die Unterscheidung zwischen Zahlen und Texten:

| Beispiel | Datentyp | Typische Verwendung |
|---|---|---|
| `ALTER` | Zahl | Alter, Punkte, Messwerte oder Berechnungen |
| `NAME$` | String | Namen, Texte oder andere Zeichenfolgen |

## 3.6.1. Es gibt noch weitere Datentypen

MMBasic kann Zahlen noch genauer unterscheiden. Dafür gibt es besondere Typkennzeichen, die an einen Variablennamen angehängt werden können:

| Zeichen | Datentyp | Beispiel | Bedeutung |
|---|---|---|---|
| `$` | String | `NAME$` | Text bzw. Zeichenfolge |
| `%` | Integer | `PUNKTE%` | Ganze Zahl |

| Zeichen | Datentyp | Beispiel | Bedeutung |
|---|---|---|---|
| ! | Float | `TEMPERATUR!` | Zahl mit Nachkommastellen |

Du musst dir diese Datentypen jetzt noch nicht im Detail merken. Wichtig ist zunächst, dass Variablen unterschiedliche Arten von Werten speichern können.

### 3.6.2. Groß- und Kleinschreibung bei Variablennamen

Bei Variablennamen unterscheidet MMBasic nicht zwischen Groß- und Kleinschreibung. Die folgenden Schreibweisen bezeichnen deshalb dieselbe Variable:

```
ALTER=25
PRINT alter
PRINT Alter
```

Ausgabe:

```
25
25
```

Das Gleiche gilt auch für String-Variablen:

```
NAME$="Manfred"
PRINT name$
```

MMBasic erkennt `NAME$` und `name$` als dieselbe Variable.

Verwende trotzdem eine einheitliche Schreibweise. In diesem Buch schreiben wir Variablennamen in den Beispielen überwiegend in Großbuchstaben. So lassen sie sich im Programmtext leicht erkennen.

### 3.6.3. Ein Name – ein Datentyp

Bei den verschiedenen Datentypen gibt es noch eine wichtige Regel: Ein Variablenname kann in MMBasic nicht gleichzeitig für unterschiedliche Datentypen verwendet werden.

Es ist also nicht zulässig, beispielsweise diese beiden Variablen im selben Programm zu verwenden:

```
WERT%=10
WERT$="zehn"
```

Obwohl `%` eine Integer-Variable und `$` eine String-Variable kennzeichnet, verwenden beide denselben Basisnamen `WERT`.

Für einen Variablennamen musst du dich für einen Datentyp entscheiden.

Wenn du sowohl einen Zahlenwert als auch einen dazugehörigen Text speichern möchtest, verwende unterschiedliche Namen:

```
WERT%=10
WERTTEXT$="zehn"
```

## 3.7. Variablen ausdrücklich deklarieren

Bisher haben wir Variablen einfach verwendet:

```
ALTER=60
NAME$="Manfred"
```

MMBasic legt diese Variablen automatisch an, sobald sie zum ersten Mal verwendet werden. Das ist für unsere ersten Programme sehr praktisch.

Variablen können aber auch ausdrücklich mit `DIM` deklariert werden:

```
DIM ALTER
DIM NAME$

ALTER=60
NAME$="Manfred"
```

Bei größeren Programmen hat diese Schreibweise einen wichtigen Vorteil: Die verwendeten Variablen werden ausdrücklich festgelegt. In Verbindung mit `OPTION EXPLICIT` kann MMBasic dadurch auch versehentlich falsch geschriebene Variablennamen erkennen.

### 3.7.1. Tippfehler mit OPTION EXPLICIT erkennen

Ein Tippfehler in einem Variablennamen kann schwer zu entdecken sein:

```
PUNKTE=100
PRINT PUNKET
```

MMBasic kann `PUNKET` als neue Variable betrachten, obwohl eigentlich `PUNKTE` gemeint war.

Mit `OPTION EXPLICIT` kann festgelegt werden, dass Variablen vor ihrer Verwendung deklariert werden müssen:

```
OPTION EXPLICIT
```

```
DIM PUNKTE
PUNKTE=100

PRINT PUNKTE
```

Ein versehentlich verwendeter Variablenname, der nicht vorher deklariert wurde, führt dann zu einer Fehlermeldung.

Für unsere ersten kleinen Programme verzichten wir zunächst auf `OPTION EXPLICIT`. Bei größeren Programmen ist es jedoch eine gute Möglichkeit, Tippfehler in Variablennamen frühzeitig zu entdecken.

## 3.8. Mit Variablen rechnen

Variablen können in Berechnungen verwendet werden.

```
A=10
B=20

PRINT A+B
```

Ausgabe:

```
30
```

Weitere Beispiele:

```
PRINT A-B
PRINT A*B
PRINT B/A
```

## 3.9. Variablen verändern

Der Inhalt einer Variablen kann jederzeit geändert werden.

```
A=10
PRINT A

A=25
PRINT A
```

Ausgabe:

```
10
25
```

Der alte Wert wird dabei überschrieben.

## 3.10. Der Inhalt einer Variablen kann berechnet werden

Besonders interessant wird es, wenn eine Variable mit ihrem eigenen Inhalt verändert wird.

```
A=10
A=A+1

PRINT A
```

Ausgabe:

```
11
```

Schauen wir uns die zweite Zeile genauer an:

```
A=A+1
```

Zuerst berechnet MMBasic die rechte Seite:

```
10+1
```

Danach wird das Ergebnis wieder in `A` gespeichert.

## 3.11. Zähler erstellen

Auf diese Weise kann ein einfacher Zähler aufgebaut werden.

```
A=0

A=A+1
PRINT A

A=A+1
PRINT A

A=A+1
```

```
PRINT A
```

Ausgabe:

```
1
2
3
```

## 3.12. Experimentiere!

Probiere folgende Beispiele aus:

```
A=100
B=50

PRINT A+B
PRINT A-B
PRINT A*B
```

Was passiert?

Probiere auch einmal eine String-Variable aus:

```
NAME$="MMBasic"
PRINT NAME$
```

## 3.13. Probier’s selbst!

Versuche folgende Aufgaben:

1. Speichere dein Alter in einer Variablen.
2. Speichere dein Geburtsjahr in einer Variablen.
3. Speichere deinen Namen in einer String-Variablen.
4. Addiere zwei Zahlen mit Hilfe von Variablen.
5. Erhöhe eine Variable dreimal um 1.

## 3.14. Zusammenfassung

In diesem Kapitel hast du Variablen und erste Datentypen kennengelernt.

Du kannst nun:

- Werte in Variablen speichern

- aussagekräftige Variablennamen verwenden
- die wichtigsten Regeln für Variablennamen beachten
- Zahlen und Texte in Variablen unterscheiden
- die Typkennzeichen `%`, `!` und `$` einordnen
- Variablen mit `DIM` ausdrücklich deklarieren
- mit `OPTION EXPLICIT` nicht deklarierte Variablen erkennen
- Werte verändern und mit Variablen rechnen

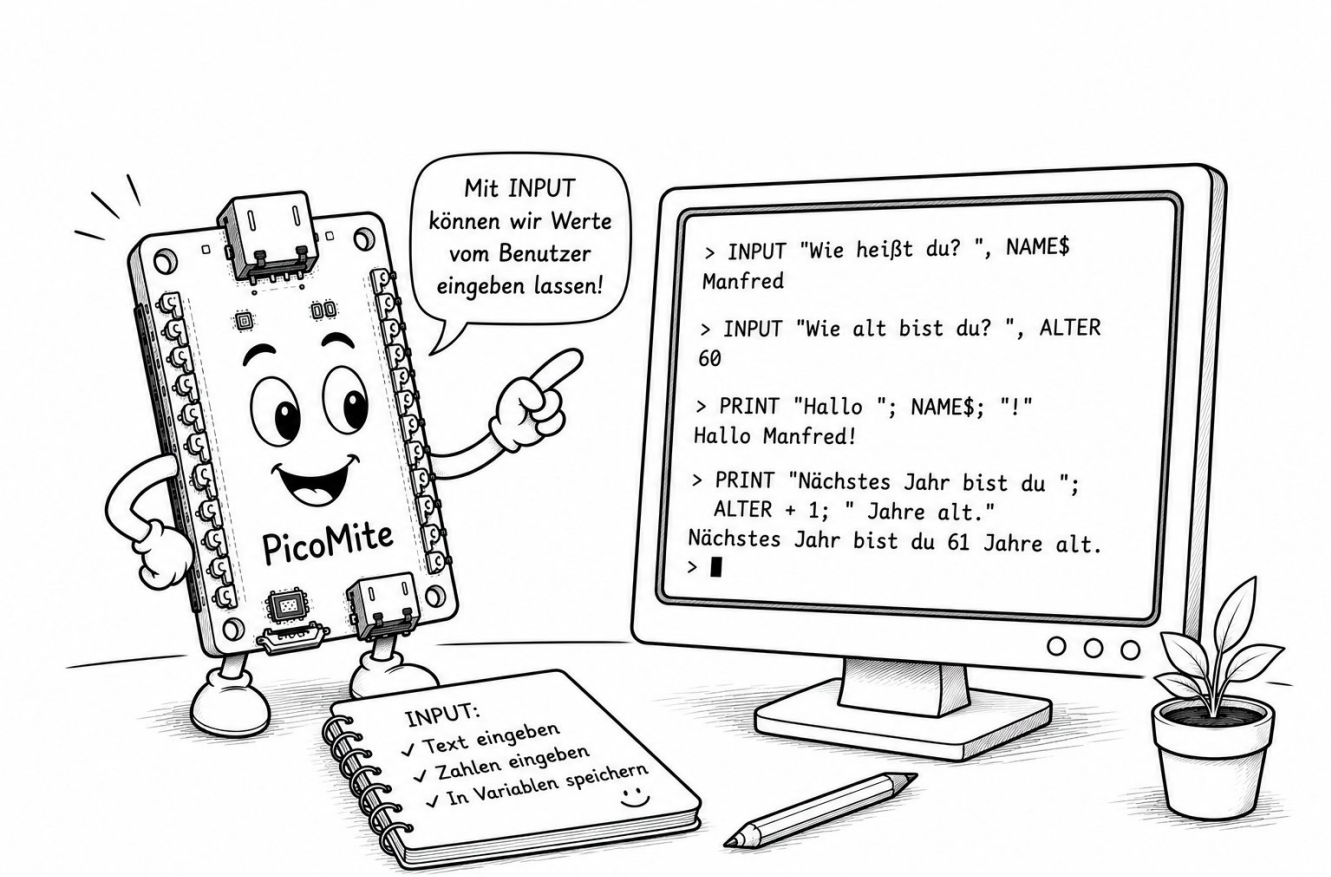
Im nächsten Kapitel lernen wir Benutzereingaben mit `INPUT` kennen.

## 3.15. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 03_variablen.bas

# Kapitel 4. Benutzereingaben mit INPUT




## 4.1. Das lernst du

In diesem Kapitel lernst du:

- wie Daten vom Benutzer eingegeben werden
- wie die Anweisung `INPUT` verwendet wird
- wie Eingaben in Variablen gespeichert werden
- wie Zahlen und Texte eingegeben werden
- warum eine Eingabe zum Datentyp der Variablen passen muss
- wie mit `VAL()` Text in eine Zahl umgewandelt wird
- wie mit `STR$()` eine Zahl in Text umgewandelt wird
- wie Eingaben weiterverarbeitet werden

Am Ende des Kapitels kannst du eigene interaktive Programme schreiben.

## 4.2. Warum Benutzereingaben wichtig sind

Bisher standen alle Werte fest im Programm.

Zum Beispiel:

```
A=10
B=20
PRINT A+B
```

Das Ergebnis ist immer gleich.

Interessanter wird es, wenn der Benutzer die Werte selbst eingeben kann.

# 4.3. Unsere erste Eingabe

Die Anweisung `INPUT` wartet auf eine Eingabe des Benutzers.

Gib folgendes ein:

```
INPUT A
PRINT A
```

MMBasic zeigt ein Fragezeichen an:

```
?
```

Gib nun beispielsweise die Zahl `42` ein und bestätige mit der Eingabetaste.

Auf dem Bildschirm sieht das so aus:

```
? 42
42
```

Die erste `42` ist deine Eingabe. Die zweite wird anschließend mit `PRINT` ausgegeben.

# 4.4. Eingaben in Variablen speichern

Die eingegebene Zahl wird in der Variablen `A` gespeichert.

```
INPUT A
PRINT A
```

Wenn du beispielsweise `100` eingibst, enthält `A` anschließend den Wert 100.

## 4.4.1. Die Eingabe muss zum Datentyp passen

Im vorherigen Kapitel hast du gelernt, dass Variablen unterschiedliche Datentypen besitzen können. Das spielt auch bei `INPUT` eine wichtige Rolle.

Bei

```
INPUT A
```

erwartet MMBasic eine Zahl. Gibst du beispielsweise `42` ein, kann dieser Wert problemlos in `A` gespeichert werden.

Was passiert aber, wenn du stattdessen einen Text eingibst?

```
? Hallo
```

MMBasic kann den Text `Hallo` nicht als Zahl in der numerischen Variablen `A` speichern. Die Eingabe schlägt deshalb fehl.

Wenn du Text erfassen möchtest, musst du eine String-Variable verwenden:

```
INPUT NAME$
```

Dann ist beispielsweise diese Eingabe möglich:

```
? Manfred
```

Der Datentyp der Variablen bestimmt, welche Art von Eingabe `INPUT` erwartet.

- Eine numerische Variable wie `A` erwartet eine Zahl.
- Eine String-Variable wie `NAME$` erwartet Text.

Die Eingabe muss also zum Datentyp der Variablen passen.

## 4.5. Text und Zahlen umwandeln

Manchmal liegt eine Zahl nicht als numerischer Wert, sondern als Text vor. Zum Beispiel kann der String `"42"` in einer String-Variablen gespeichert sein:

```
EINGABE$="42"
```

Mit `VAL()` kann ein solcher Text in einen Zahlenwert umgewandelt werden:

```
EINGABE$="42"
ZAHL=VAL(EINGABE$)

PRINT ZAHL
```

```
PRINT ZAHL+8
```

Ausgabe:

```
42
50
```

`VAL()` liest den Zahlenwert am Anfang eines Strings und liefert ihn als numerischen Wert zurück.

Die umgekehrte Richtung ist ebenfalls möglich. Mit `STR$()` kann ein Zahlenwert in einen String umgewandelt werden:

```
ZAHL=42
TEXT$=STR$(ZAHL)

PRINT TEXT$
```

Ausgabe:

```
42
```

Damit stehen uns zwei wichtige Funktionen zur Umwandlung zur Verfügung:

| **Funktion** | **Umwandlung** | **Beispiel** |
|---|---|---|
| `VAL()` | String → Zahl | `ZAHL=VAL("42")` |
| `STR$()` | Zahl → String | `TEXT$=STR$(42)` |

`VAL()` und `STR$()` werden dir in späteren Programmen noch häufiger begegnen. Für den Moment genügt es, sich die beiden Richtungen zu merken:

`VAL()` macht aus einem Zahlentext eine Zahl, `STR$()` aus einer Zahl einen String.

# 4.6. Mit Eingaben rechnen

Eingegebene Werte können sofort weiterverarbeitet werden.

```
INPUT A
PRINT A*2
```

Beispiel:

```
? 21
```

```
42
```

## 4.7. Zwei Werte eingeben

Natürlich können auch mehrere Werte eingegeben werden.

```
INPUT A, B

PRINT A+B
```

Beispiel:

```
? 10, 20
30
```

Da ein Benutzer nicht unbedingt erkennen kann, in welcher Form die Eingabe nach dem ?-Prompt erfolgen soll, kann dem `INPUT`-Befehl auch ein Fragetext mitgegeben werden.

Die vollständige Syntax von `INPUT` lautet:

```
INPUT ["prompt$(;|,)] var1 [, var2[, var3[, etc]]]
```

Auf den ersten Blick sieht diese Schreibweise etwas kompliziert aus. Sie lässt sich aber leicht entschlüsseln.

Die eckigen Klammern `[` und `]` kennzeichnen **optionale Bestandteile**. Alles, was in eckigen Klammern steht, kann also angegeben werden, muss aber nicht.

Der senkrechte Strich `|` bedeutet **oder**. Die Schreibweise

```
(;|,)
```

bedeutet daher: An dieser Stelle kann entweder ein Semikolon (`;`) oder ein Komma (`,`) stehen.

Die einzelnen Bestandteile der Syntax bedeuten:

| **Bestandteil** | **Bedeutung** |
|---|---|
| `INPUT` | Die MMBasic-Anweisung zum Einlesen von Benutzereingaben. |
| `prompt$` | Ein optionaler Fragetext, der vor der Eingabe angezeigt wird. |
| `;` | Schließt den Fragetext ab. MMBasic zeigt danach zusätzlich ein Fragezeichen ? an. |
| `,` | Kann anstelle des Semikolons verwendet werden. Das zusätzliche Fragezeichen wird dann nicht angezeigt. |

| Bestandteil | Bedeutung |
|---|---|
| `var1` | Die erste Variable, in der eine Eingabe gespeichert wird. Diese Variable ist erforderlich. |
| `var2`, `var3`, … | Weitere optionale Variablen. Dadurch können mit einer einzigen `INPUT`-Anweisung mehrere Werte eingelesen werden. |

Die Schreibweise

```
[, var2[, var3[, etc]]]
```

bedeutet, dass nach der ersten Variablen weitere Variablen folgen dürfen. Sie werden jeweils durch ein Komma voneinander getrennt.

Beginnen wir mit der einfachsten Form:

```
INPUT A
```

Hier gibt es keinen Fragetext und nur eine Variable. MMBasic zeigt automatisch das Fragezeichen ? als Eingabeaufforderung an.

Mit einem Fragetext kann daraus zum Beispiel werden:

```
INPUT "Alter"; A
```

Das Semikolon (`;`) nach dem Fragetext bewirkt, dass MMBasic zusätzlich ein Fragezeichen anzeigt:

```
Alter? 60
```

Sollen mehrere Werte eingelesen werden, können mehrere Variablen angegeben werden:

```
INPUT "Bitte gib zwei Zahlen ein"; A, B
```

Die beiden Eingaben werden durch ein Komma getrennt:

```
Bitte gib zwei Zahlen ein? 10,20
```

Danach enthält `A` den Wert `10` und `B` den Wert `20`.

Anstelle des Semikolons kann nach dem Fragetext auch ein Komma verwendet werden:

```
INPUT "Alter: ", A
```

In diesem Fall wird das zusätzliche Fragezeichen unterdrückt:

```
Alter: 60
```

Die etwas kompliziert aussehende Syntax

```
INPUT ["prompt$(;|,)] var1 [, var2[, var3[, etc]]]
```

beschreibt somit lediglich verschiedene mögliche Formen derselben Anweisung:

```
INPUT A
INPUT A, B
INPUT "Alter"; A
INPUT "Zwei Zahlen"; A, B
INPUT "Alter: ", A
```

Die Zeichen [ und ] sowie der senkrechte Strich | gehören nur zur **Beschreibung der Syntax**. Sie werden nicht in das BASIC-Programm eingegeben.

Merke:

- [⋯] bedeutet: Dieser Teil ist optional.
- | bedeutet: Wähle eine der angegebenen Möglichkeiten.
- `prompt$` ist ein optionaler Fragetext.
- `var1` ist erforderlich.
- `var2`, `var3` und weitere Variablen sind optional.
- Mehrere Eingabewerte werden durch Kommas voneinander getrennt.

Normalerweise wird der Fragetext mit einem Semikolon (`;`) abgeschlossen. In diesem Fall wird hinter dem Fragetext ein Fragezeichen angezeigt.

```
INPUT "Bitte gib zwei Zahlen ein, getrennt durch ein Komma: "; A, B
PRINT "Die Summe der zwei Zahlen ist: "; A+B
```

Beispiel:

```
Bitte gib zwei Zahlen ein, getrennt durch ein Komma: ? 10,20
Die Summe der zwei Zahlen ist: 30
```

Wenn der Fragetext mit einem Komma (`,`) abgeschlossen wird, wird das Fragezeichen unterdrückt.

```
INPUT "Bitte gib zwei Zahlen ein, getrennt durch ein Komma: ", A, B
```

```
PRINT "Die Summe der zwei Zahlen ist: "; A+B
```

Beispiel:

```
Bitte gib zwei Zahlen ein, getrennt durch ein Komma: 10,20
Die Summe der zwei Zahlen ist: 30
```

# 4.8. Eine persönliche Begrüßung

Im vorherigen Kapitel hast du bereits String-Variablen für Texte kennengelernt. Auch mit `INPUT` können solche Texte eingegeben werden.

Für den Namen verwenden wir die String-Variable `NAME$`:

```
INPUT "Wie heisst du"; NAME$
PRINT "Hallo "; NAME$; ", schön, dass du da bist."
```

Beispiel:

```
Wie heisst du? Manfred
Hallo Manfred, schön, dass du da bist.
```

Das `$` am Ende von `NAME$` zeigt, wie du bereits weißt, dass diese Variable einen String enthält.

# 4.9. Aussagekräftige Namen bei Eingaben

Gerade bei Benutzereingaben sind aussagekräftige Variablennamen hilfreich.

Statt

```
INPUT A
INPUT B$
```

kannst du zum Beispiel schreiben:

```
INPUT "Wie alt bist du"; ALTER
INPUT "Wo wohnst du"; WOHNORT$
```

Beispiel:

```
Wie alt bist du? 60
Wo wohnst du? Heidelberg
```

Schon beim Lesen des Programms ist damit klar, welche Information in einer Variablen gespeichert wird.

## 4.10. Experimentiere!

Probiere folgende Änderungen aus:

- Gib dein Alter ein.
- Addiere zwei eingegebene Zahlen.
- Gib deinen Wohnort ein.
- Erweitere die Begrüßung um weitere Texte.

## 4.11. Probier’s selbst!

Versuche folgende Aufgaben:

1. Frage den Benutzer nach seinem Namen.
2. Frage den Benutzer nach seinem Alter.
3. Addiere zwei eingegebene Zahlen.
4. Frage nach dem Wohnort.
5. Gib eine persönliche Begrüßung aus.

## 4.12. Zusammenfassung

In diesem Kapitel hast du die Anweisung `INPUT` kennengelernt.

Du kannst nun:

- Zahlen eingeben
- Texte eingeben
- Eingaben in Variablen speichern
- mehrere Werte eingeben
- den passenden Datentyp für eine Eingabe verwenden
- mit `VAL()` einen Text in eine Zahl umwandeln
- mit `STR$()` eine Zahl in einen Text umwandeln
- Eingaben weiterverarbeiten

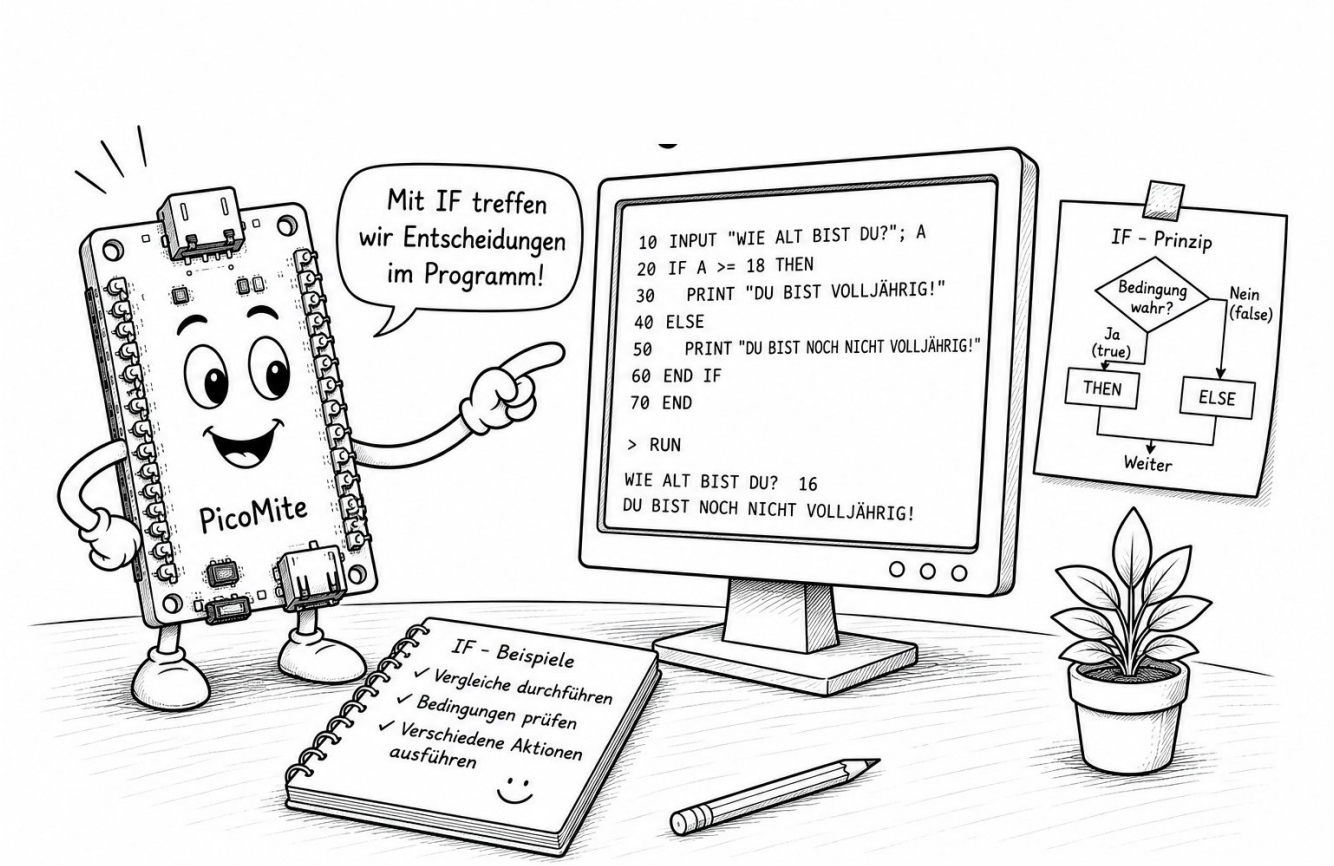
Im nächsten Kapitel lernen wir Entscheidungen mit `IF ⋯ THEN` kennen.

## 4.13. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 04_persoenliche_begruessung.bas

# Kapitel 5. Entscheidungen mit IF




## 5.1. Das lernst du

In diesem Kapitel lernst du:

- den integrierten Editor von MMBasic kennen
- Programme zu speichern und auszuführen
- Entscheidungen mit `IF` zu treffen
- `ELSE` und `ELSEIF` zu verwenden
- Bedingungen mit Vergleichsoperatoren zu formulieren

Am Ende des Kapitels kannst du Programme schreiben, die auf unterschiedliche Situationen reagieren.

## 5.2. Programme werden größer

Bisher haben wir alle Beispiele direkt an der MMBasic-Eingabeaufforderung eingegeben.

Das war ideal, um einzelne Befehle auszuprobieren und ihre Wirkung sofort zu sehen.

Ab diesem Kapitel werden unsere Programme jedoch umfangreicher. Sie bestehen aus mehreren zusammengehörenden Zeilen und enthalten Programmstrukturen wie `IF`, `FOR` oder `DO···LOOP`.

Solche Programme schreibt man am einfachsten im integrierten Editor von MMBasic.

Keine Sorge – der Editor ist bewusst einfach gehalten und schnell zu erlernen.

## 5.3. Den Editor starten

Den Editor startest du mit dem Befehl:

```
EDIT
```

Nun kannst du dein Programm Zeile für Zeile eingeben.

Während der Eingabe kannst du den Cursor mit den Pfeiltasten bewegen und den Text jederzeit ändern.

Falls sich bereits ein anderes Programm im Speicher befindet, kannst du es vorher mit

```
NEW
```

löschen.

MMBasic fragt anschließend nach, ob das Programm wirklich gelöscht werden soll.

## 5.4. Programme speichern

Nachdem du dein Programm eingegeben hast, speicherst du es mit **F1**.

Das Programm bleibt auch nach dem Ausschalten des PicoMite erhalten.

Mit **F2** verlässt du anschließend den Editor und kehrst zur MMBasic-Eingabeaufforderung zurück.

Später in diesem Buch lernst du außerdem, wie Programme als Dateien im Dateisystem oder auf einer SD-Karte gespeichert und wieder geladen werden können.

## 5.5. Programme starten

Ein im Speicher befindliches Programm wird mit dem Befehl

```
RUN
```

gestartet.

Möchtest du das Programm später ändern, öffnest du den Editor einfach erneut mit

```
EDIT
```

## 5.6. Warum Entscheidungen wichtig sind

Bis jetzt wurden alle Programmzeilen nacheinander ausgeführt.

Oft soll ein Programm jedoch unterschiedlich reagieren.

Zum Beispiel:

- Ist eine Zahl größer als 100?
- Wurde ein Taster gedrückt?
- Ist eine Datei vorhanden?
- Hat der Benutzer die richtige Antwort eingegeben?

Für solche Entscheidungen gibt es den `IF`-Befehl.

## 5.7. Unsere erste Entscheidung

Das folgende Programm überprüft, ob die Variable `A` den Wert 10 besitzt.

```
A = 10

IF A = 10 THEN
  PRINT "A hat den Wert 10."
ENDIF
```

Die Bedingung hinter `IF` wird geprüft.

Ist sie wahr, werden die Anweisungen zwischen `THEN` und `ENDIF` ausgeführt.

Ist sie falsch, werden diese Zeilen übersprungen.

## 5.8. Was bedeutet IF?

`IF` bedeutet auf Deutsch **wenn**.

Die allgemeine Schreibweise lautet:

```
IF Bedingung THEN

  Anweisungen

ENDIF
```

Das Programm kann dadurch unterschiedliche Wege einschlagen.

# 5.9. ELSE - der andere Fall

Mit `ELSE` kann festgelegt werden, was passieren soll, wenn die Bedingung nicht erfüllt ist.

```
A = 5

IF A = 10 THEN
  PRINT "A ist 10."
ELSE
  PRINT "A ist nicht 10."
ENDIF
```

Da `A` den Wert `5` besitzt, erscheint:

```
A ist nicht 10.
```

## 5.9.1. Wo sind eigentlich die Zeilennummern?

Wenn du bereits mit älteren BASIC-Dialekten gearbeitet hast, ist dir vielleicht aufgefallen, dass unsere Programme keine Zeilennummern besitzen.

Bei klassischen BASIC-Versionen sah ein Programm häufig etwa so aus:

```
10 INPUT "Wie alt bist du"; ALTER
20 IF ALTER>=18 THEN 50
30 PRINT "Du bist noch nicht volljährig."
40 GOTO 60
50 PRINT "Du bist volljährig."
60 END
```

MMBasic unterstützt solche Zeilennummern weiterhin. Sie sind jedoch optional und für neue Programme nicht erforderlich.

Statt mit Zeilennummern und `GOTO` können wir Entscheidungen strukturiert schreiben:

```
INPUT "Wie alt bist du"; ALTER

IF ALTER>=18 THEN
  PRINT "Du bist volljährig."
ELSE
  PRINT "Du bist noch nicht volljährig."
ENDIF
```

Diese Schreibweise macht den Programmablauf unmittelbar sichtbar und kommt ohne Sprünge zu bestimmten Zeilennummern aus.

Anders als bei vielen klassischen BASIC-Systemen können nummerierte Programmzeilen nicht einfach am MMBasic-Prompt eingegeben werden. Eine dort eingegebene Zeile wie `10 PRINT "Hallo"` wird unmittelbar ausgeführt und nicht als neue Programmzeile gespeichert.

Zeilennummern können jedoch innerhalb eines Programms verwendet werden. MMBasic unterstützt dabei auch die klassische Schreibweise `IF ⋯ THEN 100`. Für neue Programme empfiehlt das MMBasic-Handbuch die eindeutigere Schreibweise `IF ⋯ THEN GOTO 100` oder die Verwendung eines Labels.

In diesem Buch verwenden wir für neue Programme keine Zeilennummern.

## 5.10. Vergleichsoperatoren

Für Bedingungen werden Vergleichsoperatoren verwendet.

| Operator | Bedeutung |
|---|---|
| = | gleich |
| <> | ungleich |
| < | kleiner |
| > | größer |
| <= | kleiner oder gleich |
| >= | größer oder gleich |

## 5.11. Unser erstes Quiz

Mit `IF` können Programme auf Benutzereingaben reagieren.

Das folgende Programm stellt eine einfache Rechenaufgabe.

```
PRINT "Wieviel ist 2+3?"
INPUT ANTWORT

IF ANTWORT = 5 THEN
  PRINT "Richtig!"
ELSE
  PRINT "Leider falsch."
ENDIF
```

Beispiel:

```
Wieviel ist 2+3?
? 5

Richtig!
```

Obwohl das Programm nur aus wenigen Zeilen besteht, trifft es bereits eine Entscheidung.

Genau dieses Prinzip wird später in Spielen, Menüs oder Messprogrammen verwendet.

## 5.12. Mehrere Entscheidungen

Ein Programm kann beliebig viele Entscheidungen enthalten.

Im folgenden Beispiel wird für jede Schulnote eine eigene `IF`-Anweisung verwendet.

```
INPUT "Schulnote"; NOTE

IF NOTE = 1 THEN
  PRINT "Sehr gut"
ENDIF

IF NOTE = 2 THEN
  PRINT "Gut"
ENDIF

IF NOTE = 3 THEN
  PRINT "Befriedigend"
ENDIF
```

Das funktioniert zwar, ist bei vielen Möglichkeiten jedoch etwas umständlich.

## 5.13. ELSEIF verwenden

Mehrere Möglichkeiten lassen sich deutlich übersichtlicher mit `ELSEIF` formulieren.

```
INPUT "Schulnote"; NOTE

IF NOTE = 1 THEN
  PRINT "Sehr gut"
ELSEIF NOTE = 2 THEN
  PRINT "Gut"
ELSEIF NOTE = 3 THEN
  PRINT "Befriedigend"
ELSE
  PRINT "Unbekannte Note"
ENDIF
```

Beispiel:

```
Schulnote? 2

Gut
```

Sobald eine Bedingung erfüllt ist, werden die restlichen `ELSEIF`-Bedingungen nicht mehr geprüft.

Programme werden dadurch übersichtlicher und leichter zu erweitern.

## 5.14. Verschachtelte Entscheidungen

Auch innerhalb eines `IF`-Blocks kann sich eine weitere Entscheidung befinden.

```
INPUT "Alter"; ALTER

IF ALTER >= 18 THEN

  PRINT "Volljährig"

  IF ALTER >= 67 THEN
    PRINT "Rentenalter erreicht"
  ENDIF

ENDIF
```

Solche verschachtelten Entscheidungen kommen in größeren Programmen häufig vor.

## 5.15. Häufige Fehler

Beim Arbeiten mit `IF` passieren Anfängern häufig dieselben Fehler.

Achte besonders auf folgende Punkte:

- Nach der Bedingung muss `THEN` stehen.
- Jeder `IF`-Block benötigt ein passendes `ENDIF`.
- Für Vergleiche wird = verwendet.
- Die Anweisungen zwischen `THEN` und `ENDIF` sollten eingerückt werden. Dadurch werden Programme wesentlich übersichtlicher.

## 5.16. Probier’s selbst!

Versuche folgende Aufgaben:

1. Schreibe ein Programm, das prüft, ob eine Zahl größer als 100 ist.

2. Gib aus, ob eine Zahl gerade oder ungerade ist.
3. Vergleiche zwei Zahlen miteinander.
4. Erweitere das Quiz um weitere Rechenaufgaben.
5. Gib abhängig vom Alter unterschiedliche Texte aus.

## 5.17. Zusammenfassung

In diesem Kapitel hast du gelernt,

- den Editor von MMBasic zu verwenden,
- Programme zu speichern und auszuführen,
- Entscheidungen mit `IF` zu treffen,
- `ELSE` und `ELSEIF` einzusetzen,
- Bedingungen mit Vergleichsoperatoren zu formulieren,
- Programme durch Einrückungen übersichtlich zu gestalten.

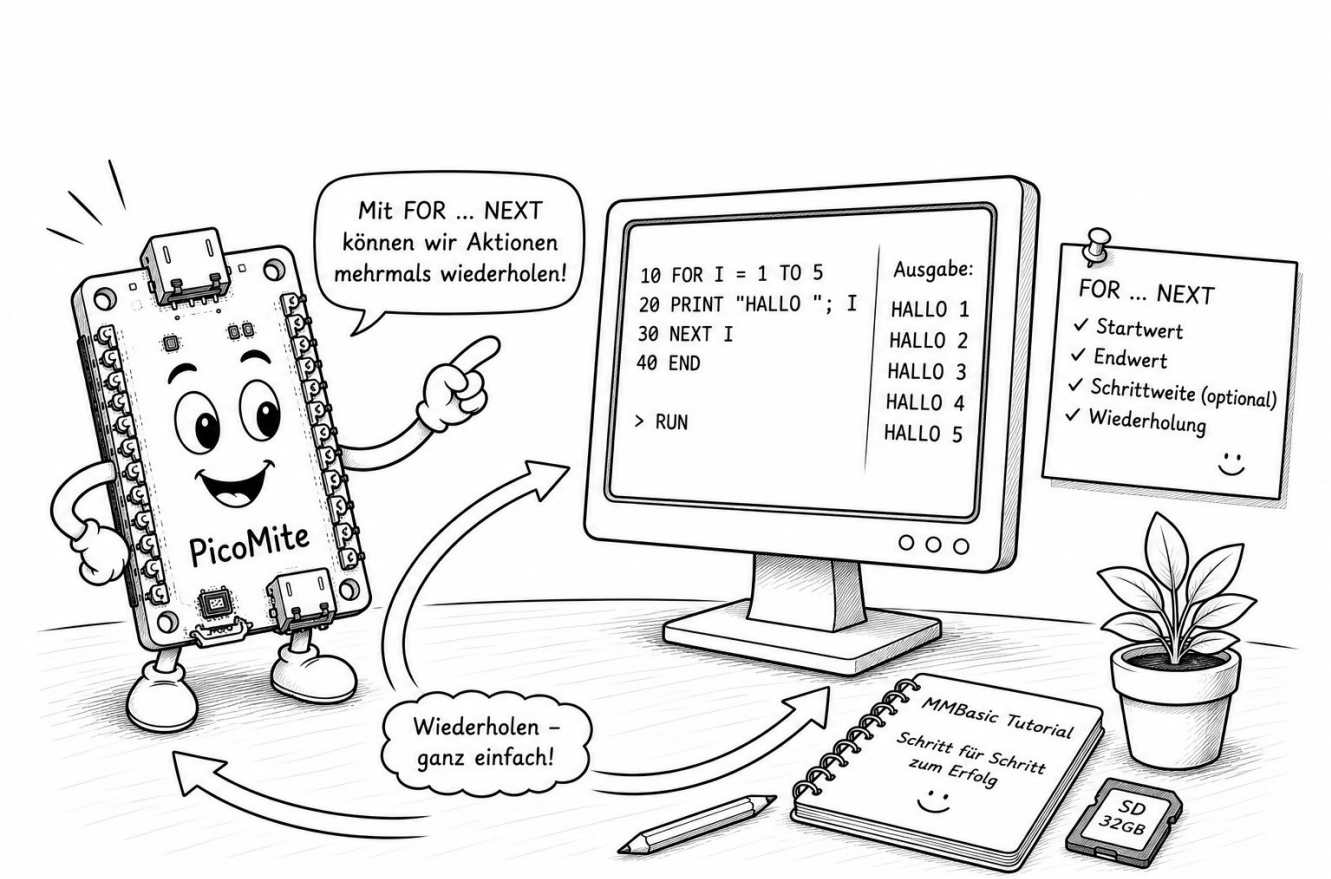
Im nächsten Kapitel lernst du Schleifen kennen. Damit können Programmteile beliebig oft wiederholt werden.

## 5.18. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 05_rechenquiz.bas

# Kapitel 6. Schleifen mit FOR … NEXT




## 6.1. Das lernst du

In diesem Kapitel lernst du:

- wie Schleifen funktionieren
- wie die Anweisung FOR verwendet wird
- wie Programme Anweisungen mehrfach ausführen können
- wie Zählvariablen verwendet werden

Am Ende des Kapitels kannst du Programme schreiben, die Aufgaben automatisch wiederholen.

## 6.2. Warum Schleifen wichtig sind

Stell dir vor, du möchtest zehnmal "Hallo Welt!" ausgeben.

Ohne Schleife müsstest du schreiben:

```
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
```

```
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
PRINT "Hallo Welt!"
```

Das funktioniert, ist aber sehr umständlich. Mit einer Schleife geht es deutlich einfacher.

## 6.3. Unsere erste Schleife

```
FOR I=1 TO 10
  PRINT "Hallo Welt!"
NEXT I
```

Ausgabe:

```
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
Hallo Welt!
```

## 6.4. Was passiert hier?

Schauen wir uns die Schleife genauer an:

```
FOR I=1 TO 10
```

MMBasic erzeugt eine Zählvariable mit dem Namen `I`.

Der Wert beginnt bei 1.

Nach jedem Durchlauf wird der Wert automatisch erhöht.

Sobald der Wert 10 erreicht hat, endet die Schleife.

```
1
2
```

```
3
4
5
6
7
8
9
10
```

## 6.5. Die Zählvariable ausgeben

Die aktuelle Nummer kann jederzeit ausgegeben werden.

```
FOR I=1 TO 10
PRINT I
NEXT I
```

Ausgabe:

```
1
2
3
4
5
6
7
8
9
10
```

## 6.6. Mit der Zählvariable rechnen

Die Zählvariable kann auch in Berechnungen verwendet werden.

```
FOR I=1 TO 10
PRINT I*I
NEXT I
```

Ausgabe:

```
1
4
9
16
25
```

```
36
49
64
81
100
```

## 6.7. Das kleine Einmaleins

Mit einer Schleife lässt sich schnell ein kleines Einmaleins erzeugen.

```
FOR I=1 TO 10
PRINT I;" x 5 = ";I*5
NEXT I
```

Ausgabe:

```
1 x 5 = 5
2 x 5 = 10
3 x 5 = 15
...
10 x 5 = 50
```

## 6.8. Rückwärts zählen

Schleifen können auch rückwärts laufen.

Dazu wird `STEP` verwendet.

```
FOR I=10 TO 1 STEP -1
PRINT I
NEXT I
```

Ausgabe:

```
10
9
8
7
6
5
4
3
2
1
```

## 6.9. Größere Schritte

Mit STEP können auch andere Schrittweiten verwendet werden.

```
FOR I=0 TO 20 STEP 2
PRINT I
NEXT I
```

Ausgabe:

```
0
2
4
6
8
10
12
14
16
18
20
```

## 6.10. Unser erster Countdown

```
FOR I=10 TO 1 STEP -1
PRINT I
NEXT I

PRINT "Start!"
```

Ausgabe:

```
10
9
8
7
6
5
4
3
2
1
Start!
```

Die Schleifenvariable wird von `FOR … NEXT` automatisch verwaltet. Verändere ihren Wert deshalb innerhalb der Schleife möglichst nicht selbst.

Vermeide beispielsweise:

```
FOR I=1 TO 10
  PRINT I
  I=I+1
NEXT I
```

Hier wird `I` innerhalb der Schleife verändert und anschließend durch `NEXT` noch einmal erhöht. Dadurch ist der Ablauf schwieriger nachzuvollziehen.

Wenn eine Schleife unter bestimmten Bedingungen vorzeitig beendet werden soll, verwende stattdessen `EXIT FOR`.

## 6.11. Schleifen und Entscheidungen

Schleifen und IF lassen sich hervorragend kombinieren.

Im folgenden Beispiel kommt der Operator `MOD` zum Einsatz. Er berechnet den **Rest einer Division**.

Beispiele:

```
7 MOD 2 = 1
8 MOD 2 = 0
9 MOD 2 = 1
10 MOD 2 = 0
```

Alle geraden Zahlen ergeben bei der Division durch 2 den Rest 0.

Das können wir nutzen, um nur gerade Zahlen auszugeben.

```
FOR I=1 TO 10

  IF I MOD 2 = 0 THEN
    PRINT I
  ENDIF

NEXT I
```

Ausgabe:

```
2
4
6
```

```
8
10
```

## 6.12. Eine mathematische Funktion berechnen

Schleifen werden häufig verwendet, um mathematische Funktionen für viele Werte zu berechnen.

Betrachten wir die Funktion:

```
y = 0.5*x^2 + x + 0.5
```

Wir möchten die Funktion für alle Werte von 0 bis 10 berechnen.

```
FOR X=0 TO 10

  Y=0.5*X^2+X+0.5

  PRINT "X=";X;"  Y=";Y

NEXT X
```

Ausgabe:

```
X=0   Y=0.5
X=1   Y=2
X=2   Y=4.5
X=3   Y=8
X=4   Y=12.5
X=5   Y=18
X=6   Y=24.5
X=7   Y=32
X=8   Y=40.5
X=9   Y=50
X=10  Y=60.5
```

Mit einer Schleife können auf diese Weise auch Tabellen für komplexere Funktionen erzeugt werden.

Frühe Heimcomputer und Taschenrechner wurden häufig genau für solche Berechnungen eingesetzt.

Mit einer Schleife lassen sich auch Tabellen für mathematische Funktionen erzeugen.

Solche Wertetabellen wurden früher häufig verwendet, um Funktionen zu untersuchen oder später als Graph darzustellen.

## 6.13. Verschachtelte Schleifen

Eine `FOR … NEXT`-Schleife kann auch eine weitere Schleife enthalten. Man spricht dann von **verschachtelten Schleifen**.

```
FOR ZEILE=1 TO 3
  FOR SPALTE=1 TO 5
    PRINT "*";
  NEXT SPALTE
  PRINT
NEXT ZEILE
```

Ausgabe:

```
*****
*****
*****
```

Bei jedem Durchlauf der äußeren Schleife wird die innere Schleife vollständig durchlaufen. In unserem Beispiel läuft `ZEILE` dreimal von 1 bis 3. Für jede Zeile läuft `SPALTE` fünfmal von 1 bis 5.

## 6.14. Eine Schleife vorzeitig verlassen

Normalerweise wird eine `FOR … NEXT`-Schleife so lange ausgeführt, bis ihr Endwert erreicht ist. Manchmal soll die Schleife jedoch schon vorher beendet werden.

Dafür gibt es `EXIT FOR`.

```
FOR I=1 TO 10
  PRINT I

  IF I=5 THEN
    EXIT FOR
  ENDIF
NEXT I
```

Ausgabe:

```
1
2
3
4
5
```

Sobald `I` den Wert 5 erreicht, wird `EXIT FOR` ausgeführt. Das Programm verlässt die Schleife und

setzt die Ausführung hinter `NEXT I` fort.

## 6.15. Experimentiere!

Probiere folgende Änderungen aus:

Gib die Zahlen von 1 bis 20 aus. Zähle von 100 bis 0 herunter. Gib alle Vielfachen von 3 aus. Berechne die Quadratzahlen von 1 bis 20.

## 6.16. Probier’s selbst!

Versuche folgende Aufgaben:

1. Gib die Zahlen von 1 bis 50 aus.
2. Gib nur die geraden Zahlen von 1 bis 20 aus.
3. Erstelle einen Countdown von 20 bis 1.
4. Erzeuge das Einmaleins der Zahl 7.
5. Berechne die Quadratzahlen von 1 bis 15.

## 6.17. Zusammenfassung

In diesem Kapitel hast du die FOR-Schleife kennengelernt.

Du kannst nun:

Anweisungen mehrfach ausführen vorwärts zählen rückwärts zählen Schrittweiten mit STEP verwenden Schleifen mit IF kombinieren

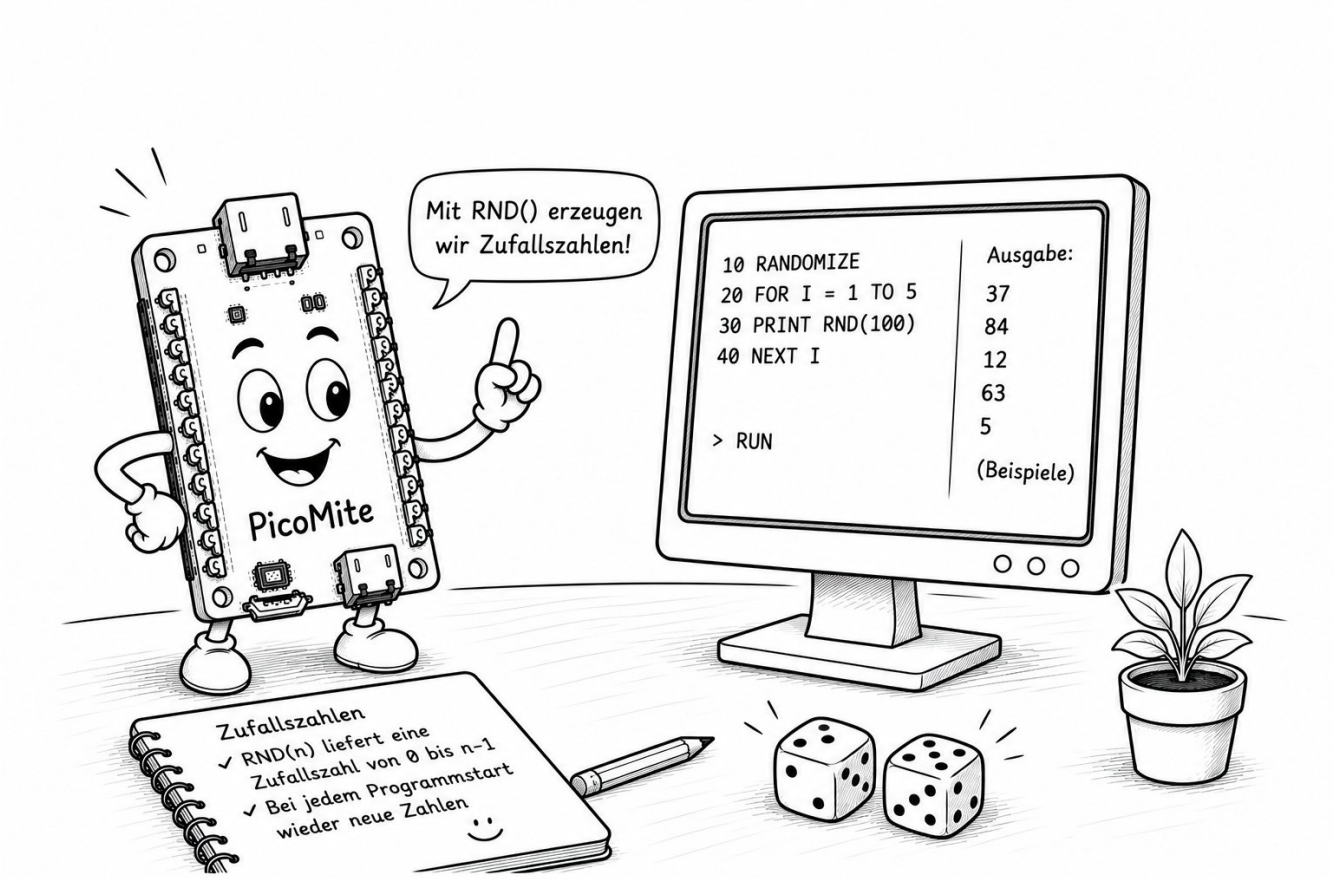
Im nächsten Kapitel schreiben wir unser erstes größeres Programm.

## 6.18. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 06_schleifen.bas

# Kapitel 7. Zufallszahlen




## 7.1. Das lernst du

In diesem Kapitel lernst du:

- wie Zufallszahlen erzeugt werden
- wie die Funktion `RND` verwendet wird
- wie Zufallszahlen auf einen gewünschten Bereich angepasst werden
- warum der RP2040 Pseudozufallszahlen erzeugt
- wie mit `RANDOMIZE` unterschiedliche Zufallsfolgen erzeugt werden
- warum `RANDOMIZE` auf dem RP2350 nicht benötigt wird

Am Ende des Kapitels kannst du Würfel, Münzwürfe und kleine Spiele programmieren.

## 7.2. Warum Zufallszahlen wichtig sind

Viele Programme benötigen Zufallszahlen.

Zum Beispiel:

- Spiele
- Quizprogramme

- Simulationen
- Würfel
- Lottozahlen

MMBasic stellt dafür die Funktion `RND` bereit.

## 7.3. Eine Zufallszahl erzeugen

Gib folgendes Programm ein:

```
PRINT RND
```

Bei jeder Ausführung erscheint eine andere Zahl.

Zum Beispiel:

```
0.38142
```

oder

```
0.92471
```

## 7.4. Zufallszahlen mehrfach erzeugen

Mit einer Schleife können mehrere Zufallszahlen ausgegeben werden.

```
FOR I=1 TO 10
  PRINT RND
NEXT I
```

## 7.5. Einen Würfel simulieren

Ein Würfel besitzt die Werte 1 bis 6.

```
PRINT INT(RND*6)+1
```

Mögliche Ausgabe:

```
4
```

## 7.6. Mehrfach würfeln

```
FOR I=1 TO 10
  PRINT INT(RND*6)+1
NEXT I
```

## 7.7. Münze werfen

```
IF INT(RND*2)=0 THEN
  PRINT "Kopf"
ELSE
  PRINT "Zahl"
ENDIF
```

## 7.8. Lottozahlen erzeugen

```
FOR I=1 TO 6
  PRINT INT(RND*49)+1
NEXT I
```

Die erzeugten Lottozahlen können mehrfach auftreten. Für eine richtige Lottoziehung müssen wir verhindern, dass dieselbe Zahl zweimal gezogen wird. Wie das funktioniert, sehen wir später, sobald wir Arrays kennengelernt haben.

## 7.9. Ein Ratespiel

```
GEHEIM=INT(RND*10)+1

PRINT "Rate eine Zahl von 1 bis 10"

INPUT TIPP

IF TIPP=GEHEIM THEN
  PRINT "Richtig!"
ELSE
  PRINT "Leider falsch."
ENDIF
```

## 7.10. RANDOMIZE – unterschiedliche Zufallsfolgen

Auf einem RP2040 erzeugt `RND` sogenannte **Pseudozufallszahlen**. Dabei wird eine Zahlenfolge berechnet, die zufällig erscheint.

Nach dem Einschalten beginnt der Zufallszahlengenerator jedoch mit demselben Startwert. Dadurch kann bei jedem Programmstart dieselbe Folge von Zufallszahlen entstehen.

Mit `RANDOMIZE` kann ein anderer Startwert festgelegt werden. Als Startwert eignet sich beispielsweise `TIMER`:

```
RANDOMIZE TIMER

FOR I=1 TO 5
  PRINT RND
NEXT I
```

Da sich der Wert von `TIMER` verändert, beginnt der Zufallszahlengenerator nicht immer mit derselben Zahlenfolge.

Auf dem RP2350 ist `RANDOMIZE` nicht erforderlich. Dieser Mikrocontroller besitzt einen Hardware-Zufallszahlengenerator und erzeugt mit `RND` echte Zufallszahlen. Der Befehl `RANDOMIZE` hat auf dem RP2350 deshalb keine Wirkung.

Der RP2040 besitzt diesen Hardware-Zufallszahlengenerator nicht. Dort ist `RANDOMIZE` sinnvoll, wenn bei jedem Programmstart eine andere Folge von Pseudozufallszahlen gewünscht ist.

MMBasic bietet mit `MATH(RAND)` außerdem einen Zufallszahlengenerator auf Basis des Mersenne-Twister-Algorithmus. Dieser kann mit `MATH RANDOMIZE` initialisiert werden.

Für unsere ersten Programme verwenden wir zunächst `RND`.

## 7.11. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Zufallszahlen mit `RND` zu erzeugen
- Zufallszahlen auf einen gewünschten Zahlenbereich anzupassen
- Würfel zu simulieren
- Münzen zu werfen
- kleine Spiele mit Zufallszahlen zu programmieren
- warum der RP2040 Pseudozufallszahlen erzeugt
- mit `RANDOMIZE` unterschiedliche Zufallsfolgen auf dem RP2040 zu erzeugen
- warum `RANDOMIZE` auf dem RP2350 nicht benötigt wird

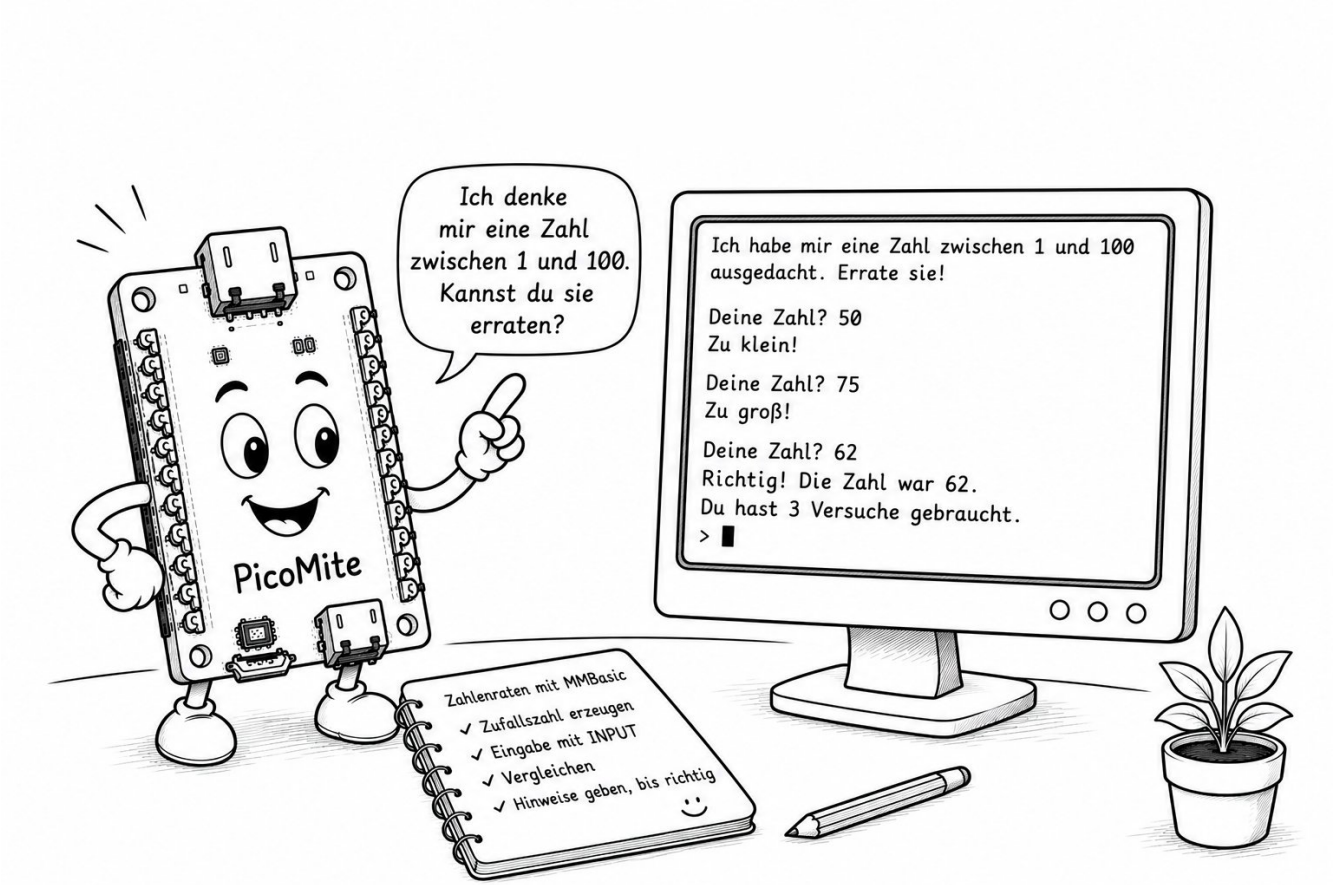
Im nächsten Kapitel schreiben wir unser erstes größeres Programm.

## 7.12. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 07_wuerfel.bas

# Kapitel 8. Unser erstes Spiel: Zahlenraten




## 8.1. Das lernst du

In diesem Kapitel lernst du:

- wie mehrere Befehle zu einem vollständigen Programm kombiniert werden
- wie Zufallszahlen verwendet werden
- wie Benutzereingaben verarbeitet werden
- wie Programme auf richtige und falsche Antworten reagieren

Am Ende des Kapitels hast du dein erstes kleines Spiel programmiert.

## 8.2. Programme als Dateien speichern

Bis jetzt haben wir immer mit dem Programm gearbeitet, das sich im PicoMite befindet. Bei größeren Projekten ist es jedoch praktisch, mehrere Programme als Dateien zu speichern.

Mit dem Befehl `SAVE` speicherst du das aktuelle Programm unter einem Namen:

```
SAVE "ZAHLENRATEN.BAS"
```

Mit `FILES` kannst du dir alle gespeicherten Dateien anzeigen lassen:

```
FILES
```

Um ein gespeichertes Programm später wieder zu laden, verwendest du `LOAD`:

```
LOAD "ZAHLENRATEN.BAS"
```

Du kannst ein gespeichertes Programm auch direkt zum Bearbeiten öffnen:

```
EDIT "ZAHLENRATEN.BAS"
```

Damit kannst du beliebig viele Programme aufbewahren und jederzeit weiterentwickeln.

## 8.3. Die Spielidee

Der Computer denkt sich eine Zahl aus.

Der Spieler versucht diese Zahl zu erraten.

Danach verrät das Programm, ob der Tipp richtig war.

## 8.4. Das komplette Programm

```
GEHEIM=INT(RND*10)+1

PRINT "Ich habe mir eine Zahl von 1 bis 10 ausgedacht."

INPUT TIPP

IF TIPP=GEHEIM THEN
  PRINT "Richtig!"
ELSE
  PRINT "Leider falsch."
ENDIF
```

## 8.5. Das Programm verstehen

Die erste Zeile erzeugt eine Zufallszahl:

```
GEHEIM=INT(RND*10)+1
```

Die Funktion `RND` liefert eine Zufallszahl.

Durch Multiplikation mit 10 und anschließendes Abrunden mit `INT()` entsteht eine ganze Zahl

zwischen 0 und 9.

Durch Addition von 1 wird daraus eine Zahl zwischen 1 und 10.

Danach fordert das Programm den Benutzer zur Eingabe eines Tipps auf:

```
INPUT TIPP
```

Anschließend werden Tipp und Zufallszahl verglichen:

```
IF TIPP=GEHEIM THEN
  ...
ENDIF
```

Stimmen beide Werte überein, gewinnt der Spieler.

## 8.6. Das Spiel testen

Starte das Programm mehrmals.

Du wirst feststellen, dass sich die geheime Zahl bei jedem Programmlauf ändert.

Das liegt daran, dass jedes Mal eine neue Zufallszahl erzeugt wird.

## 8.7. Mehrere Spielrunden

Mit einer Schleife können mehrere Spielrunden hintereinander gespielt werden.

```
FOR RUNDE=1 TO 5

  GEHEIM=INT(RND*10)+1

  PRINT
  PRINT "Runde ";RUNDE

  INPUT TIPP

  IF TIPP=GEHEIM THEN
    PRINT "Richtig!"
  ELSE
    PRINT "Leider falsch."
  ENDIF

NEXT RUNDE
```

Das Spiel wird nun fünfmal hintereinander ausgeführt.

## 8.8. Punkte sammeln

Mit einer zusätzlichen Variablen können wir die Anzahl der Treffer zählen.

```
PUNKTE=0

FOR RUNDE=1 TO 5

  GEHEIM=INT(RND*10)+1

  PRINT
  PRINT "Runde ";RUNDE

  INPUT TIPP

  IF TIPP=GEHEIM THEN
    PRINT "Richtig!"
    PUNKTE=PUNKTE+1
  ELSE
    PRINT "Leider falsch."
  ENDIF

NEXT RUNDE

PRINT
PRINT "Du hast ";PUNKTE;" Punkte erreicht."
```

## 8.9. Das Spiel verbessern

Das Spiel funktioniert bereits erstaunlich gut.

Allerdings hat der Spieler pro Runde nur einen einzigen Versuch.

Außerdem erfährt er nicht, ob sein Tipp zu groß oder zu klein war.

Diese Verbesserungen werden wir in einem späteren Kapitel umsetzen.

## 8.10. Ideen für Erweiterungen

Vielleicht möchtest du das Spiel bereits jetzt erweitern.

Zum Beispiel:

- Zahlen von 1 bis 20 verwenden
- Zahlen von 1 bis 100 verwenden
- zehn Spielrunden durchführen
- zwei Punkte für einen Treffer vergeben

- den Namen des Spielers abfragen

## 8.11. Experimentiere!

Probiere folgende Änderungen aus:

- Erhöhe den Zahlenbereich auf 20.
- Erhöhe den Zahlenbereich auf 100.
- Spiele zehn Runden statt fünf.
- Vergib zwei Punkte für jeden Treffer.

Welche Auswirkungen haben die Änderungen?

## 8.12. Probier’s selbst!

Versuche folgende Aufgaben:

1. Erweitere das Spiel auf zehn Runden.
2. Zähle die Treffer.
3. Gib die Gesamtpunktzahl aus.
4. Verwende Zahlen von 1 bis 50.
5. Frage den Namen des Spielers ab und begrüße ihn persönlich.

## 8.13. Zusammenfassung

In diesem Kapitel hast du dein erstes vollständiges Spiel programmiert.

Dabei wurden viele der bisher gelernten Befehle verwendet:

- PRINT
- INPUT
- Variablen
- IF
- FOR
- RND

Du hast gesehen, wie aus einzelnen Befehlen ein vollständiges Programm entsteht.

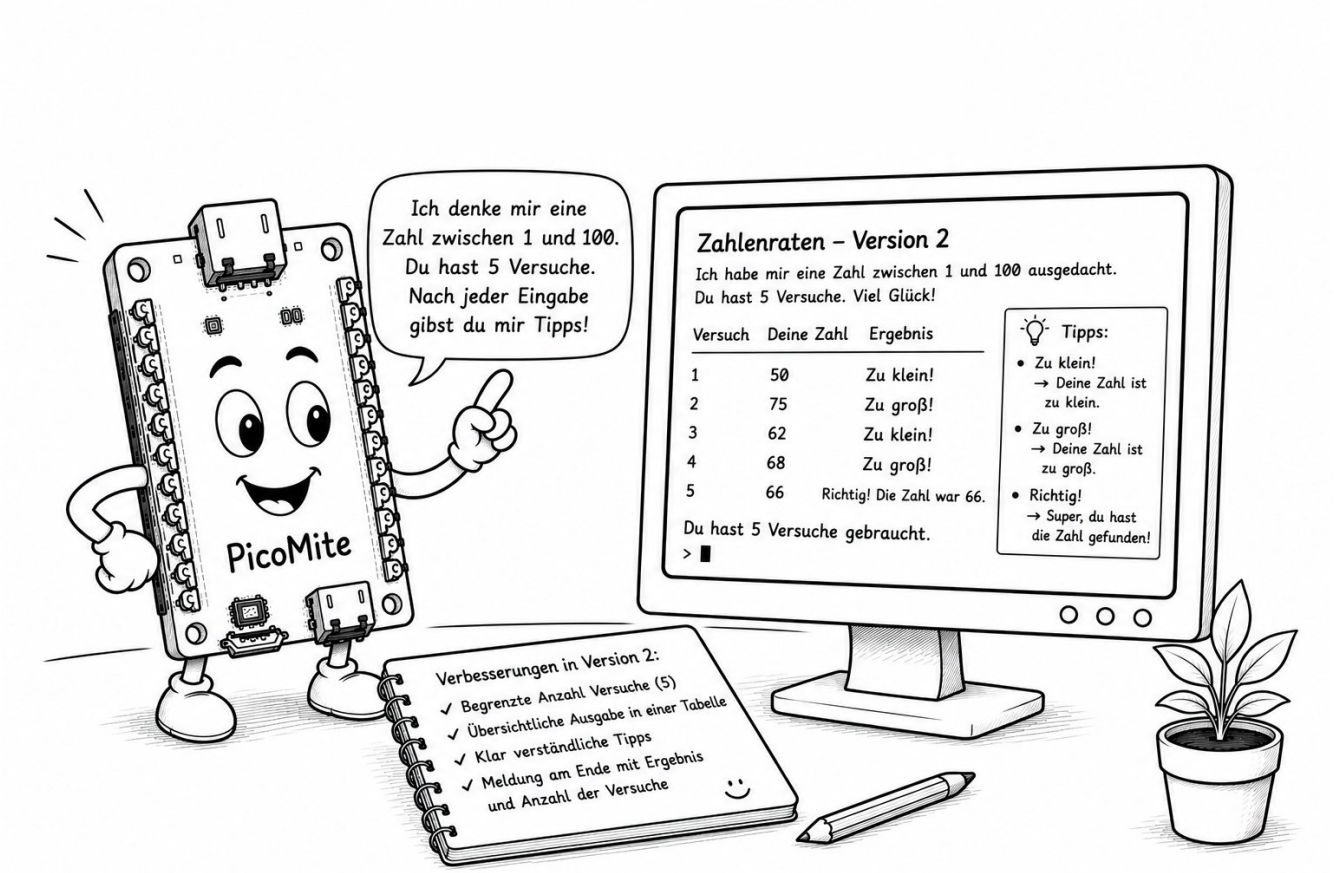
Im nächsten Kapitel werden wir das Spiel weiter verbessern und dem Spieler Hinweise geben, ob sein Tipp zu groß oder zu klein war.

## 8.14. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 08_zahlenraten.bas

# Kapitel 9. Zahlenraten Version 2




## 9.1. Das lernst du

In diesem Kapitel lernst du:

- wann eine `DO ⋯ LOOP`-Schleife verwendet wird
- wie eine Schleife mit `UNTIL` oder `WHILE` gesteuert wird
- warum die Position der Bedingung eine Rolle spielt
- wie ein Spiel mehrere Versuche erlauben kann
- wie Hinweise ausgegeben werden
- wie die Anzahl der Versuche gezählt wird

Am Ende des Kapitels hast du ein deutlich verbessertes Zahlenratespiel programmiert.

## 9.2. Das Problem

In unserem ersten Zahlenratespiel hatte der Spieler nur einen einzigen Versuch.

War die Eingabe falsch, war das Spiel sofort beendet.

Das ist nicht besonders spannend.

Wir möchten dem Spieler deshalb mehrere Versuche erlauben.

## 9.3. Die neue Spielidee

Der Computer denkt sich wieder eine Zahl aus.

Der Spieler darf so lange raten, bis er die richtige Zahl gefunden hat.

Zusätzlich erhält er hilfreiche Hinweise:

- Zu klein!
- Zu groß!
- Richtig!

## 9.4. Eine neue Schleife

In Kapitel 6 haben wir die Schleife `FOR ⋯ NEXT` kennengelernt.

Sie eignet sich besonders dann, wenn bereits vorher feststeht, wie oft etwas wiederholt werden soll:

```
FOR I=1 TO 10
  PRINT I
NEXT I
```

Beim Zahlenratespiel wissen wir dagegen nicht, wie viele Versuche der Spieler benötigen wird.

Für solche Aufgaben eignet sich die Schleife `DO ⋯ LOOP`.

Die einfachste Form sieht so aus:

```
DO

  PRINT "Hallo Welt!"

LOOP
```

Diese Schleife besitzt keine Schleifenbedingung und läuft deshalb als Endlosschleife immer weiter.

Eine Schleifenbedingung legt fest, wie lange eine Schleife ausgeführt wird.

Eine Schleife kann mit `CTRL+C` abgebrochen werden.

`DO ⋯ LOOP` eignet sich besonders für Wiederholungen, bei denen die Anzahl der Durchläufe vorher nicht bekannt ist.

## 9.5. Die Schleife beenden

Natürlich soll eine Schleife normalerweise nicht endlos laufen.

Mit `UNTIL` können wir festlegen, wann die Schleife beendet werden soll.

```
DO

  INPUT A

LOOP UNTIL A=10
```

Das Programm fordert so lange eine Zahl an, bis die Zahl 10 eingegeben wird.

`UNTIL` bedeutet sinngemäß „bis“. Die Schleife läuft also:

```
Wiederhole ...
bis A=10
```

Bei `LOOP UNTIL` wird die Bedingung erst am Ende der Schleife geprüft.

Die Anweisungen zwischen `DO` und `LOOP` werden deshalb mindestens einmal ausgeführt.

## 9.6. Wiederholen mit WHILE

Neben `UNTIL` können wir eine `DO ⋯ LOOP`-Schleife auch mit `WHILE` steuern.

`WHILE` bedeutet sinngemäß „so lange“.

```
A=1

DO

  PRINT A
  A=A+1

LOOP WHILE A<=5
```

Die Ausgabe lautet:

```
1
2
3
4
```

```
5
```

Die Schleife wird wiederholt, so lange die Bedingung `A⇐5` erfüllt ist.

Damit unterscheiden sich `UNTIL` und `WHILE` vor allem in der Bedeutung:

| | |
|---|---|
| `UNTIL` | wiederholen, bis die Bedingung wahr wird |
| `WHILE` | wiederholen, so lange die Bedingung wahr ist |

Bei `UNTIL` beschreibt die Bedingung also, wann die Schleife *enden* soll. Bei `WHILE` beschreibt sie dagegen, unter welcher Bedingung die Schleife *weiterlaufen* soll.

### 9.6.1. Die Bedingung kann auch am Anfang stehen

Die Bedingung einer `DO ⋯ LOOP`-Schleife kann auch bereits bei `DO` angegeben werden:

```
A=1

DO WHILE A<=5

  PRINT A
  A=A+1

LOOP
```

Hier wird die Bedingung geprüft, bevor die Anweisungen innerhalb der Schleife ausgeführt werden.

Ist die Bedingung bereits beim ersten Test nicht erfüllt, wird die Schleife kein einziges Mal durchlaufen.

Bei einer Bedingung hinter `LOOP` wird der Schleifeninhalt dagegen mindestens einmal ausgeführt.

Auch `UNTIL` kann direkt hinter `DO` verwendet werden:

```
DO UNTIL BEDINGUNG
  ...
LOOP
```

Damit kann die Bedingung sowohl am Anfang als auch am Ende einer `DO ⋯ LOOP` -Schleife geprüft werden.

Welche Form besser geeignet ist, hängt davon ab, ob der Schleifeninhalt auf jeden Fall mindestens einmal ausgeführt werden soll.

## 9.7. Das neue Zahlenratespiel

Nach so viel Schleifentheorie wird es Zeit, wieder zu programmieren.

Jetzt kennen wir die wichtigsten Grundlagen der `DO ⋯ LOOP`-Schleife und können sie in unserem Zahlenratespiel einsetzen.

Da wir vorher nicht wissen, wie viele Versuche der Spieler benötigen wird, verwenden wir eine Schleife mit `LOOP UNTIL`. Das Spiel fragt dadurch so lange nach einem neuen Tipp, bis die geheime Zahl gefunden wurde.

Unser verbessertes Zahlenratespiel sieht damit so aus:

```
GEHEIM=INT(RND*100)+1

PRINT "Ich habe mir eine Zahl von 1 bis 100 ausgedacht."

DO

  INPUT TIPP

  IF TIPP<GEHEIM THEN
    PRINT "Zu klein!"
  ELSEIF TIPP>GEHEIM THEN
    PRINT "Zu gross!"
  ELSE
    PRINT "Richtig!"
  ENDIF

LOOP UNTIL TIPP=GEHEIM
```

## 9.8. Das Programm verstehen

Wie bisher wird zunächst eine Zufallszahl erzeugt.

```
GEHEIM=INT(RND*100)+1
```

Danach beginnt die Schleife:

```
DO
...
LOOP UNTIL TIPP=GEHEIM
```

Die Schleife wird so lange wiederholt, bis der Spieler die richtige Zahl eingegeben hat.

## 9.9. Die Anzahl der Versuche zählen

Wie viele Versuche benötigt der Spieler?

Das können wir mit einer Variablen zählen.

```
VERSUCHE=0

DO

  VERSUCHE=VERSUCHE+1

  INPUT TIPP

  IF TIPP<GEHEIM THEN
    PRINT "Zu klein!"
  ELSEIF TIPP>GEHEIM THEN
    PRINT "Zu gross!"
  ELSE
    PRINT "Richtig!"
  ENDIF

LOOP UNTIL TIPP=GEHEIM

PRINT
PRINT "Du hast ";VERSUCHE;" Versuche benoetigt."
```

## 9.10. Das Spiel ausprobieren

Starte das Programm mehrfach.

Wie viele Versuche benötigst du?

Schaffst du es in weniger als fünf Versuchen?

## 9.11. Eine persönliche Begrüßung

Das Spiel kann den Spieler auch persönlich begrüßen.

```
PRINT "Wie heisst du?"
INPUT NAME$

PRINT
PRINT "Hallo ";NAME$

GEHEIM=INT(RND*100)+1
VERSUCHE=0
```

```
DO

  VERSUCHE=VERSUCHE+1

  INPUT TIPP

  IF TIPP<GEHEIM THEN
    PRINT "Zu klein!"
  ELSEIF TIPP>GEHEIM THEN
    PRINT "Zu gross!"
  ELSE
    PRINT "Richtig!"
  ENDIF

LOOP UNTIL TIPP=GEHEIM

PRINT
PRINT "Du hast ";VERSUCHE;" Versuche benoetigt."
```

Viele Programme entstehen nicht auf einmal.

Oft beginnt man mit einer einfachen Version und erweitert diese Schritt für Schritt.

Genau so ist auch unser Zahlenratespiel entstanden.

## 9.12. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende Zahlen von 1 bis 50.
- Verwende Zahlen von 1 bis 1000.
- Begrenze die Anzahl der Versuche.
- Vergib Punkte für besonders gute Ergebnisse.

## 9.13. Probier’s selbst!

Versuche folgende Aufgaben:

1. Begrenze die Anzahl der Versuche auf 10.
2. Gib nach dem Spiel die geheime Zahl aus.
3. Vergib Punkte abhängig von der Anzahl der Versuche.
4. Frage nach jedem Spiel, ob noch einmal gespielt werden soll.
5. Erweitere das Spiel um einen Highscore.

## 9.14. Zusammenfassung

In diesem Kapitel hast du unser erstes Spiel verbessert.

Du hast gelernt:

- `DO ⋯ LOOP` für Wiederholungen mit unbekannter Anzahl von Durchläufen zu verwenden
- Schleifen mit `UNTIL` zu wiederholen, bis eine Bedingung erfüllt ist
- Schleifen mit `WHILE` zu wiederholen, so lange eine Bedingung erfüllt ist
- Bedingungen am Anfang oder am Ende einer `DO ⋯ LOOP`-Schleife zu prüfen
- Hinweise auszugeben
- Versuche zu zählen
- Programme schrittweise zu erweitern

Im nächsten Kapitel lernen wir Unterprogramme kennen. Damit können größere Programme übersichtlicher aufgebaut werden.

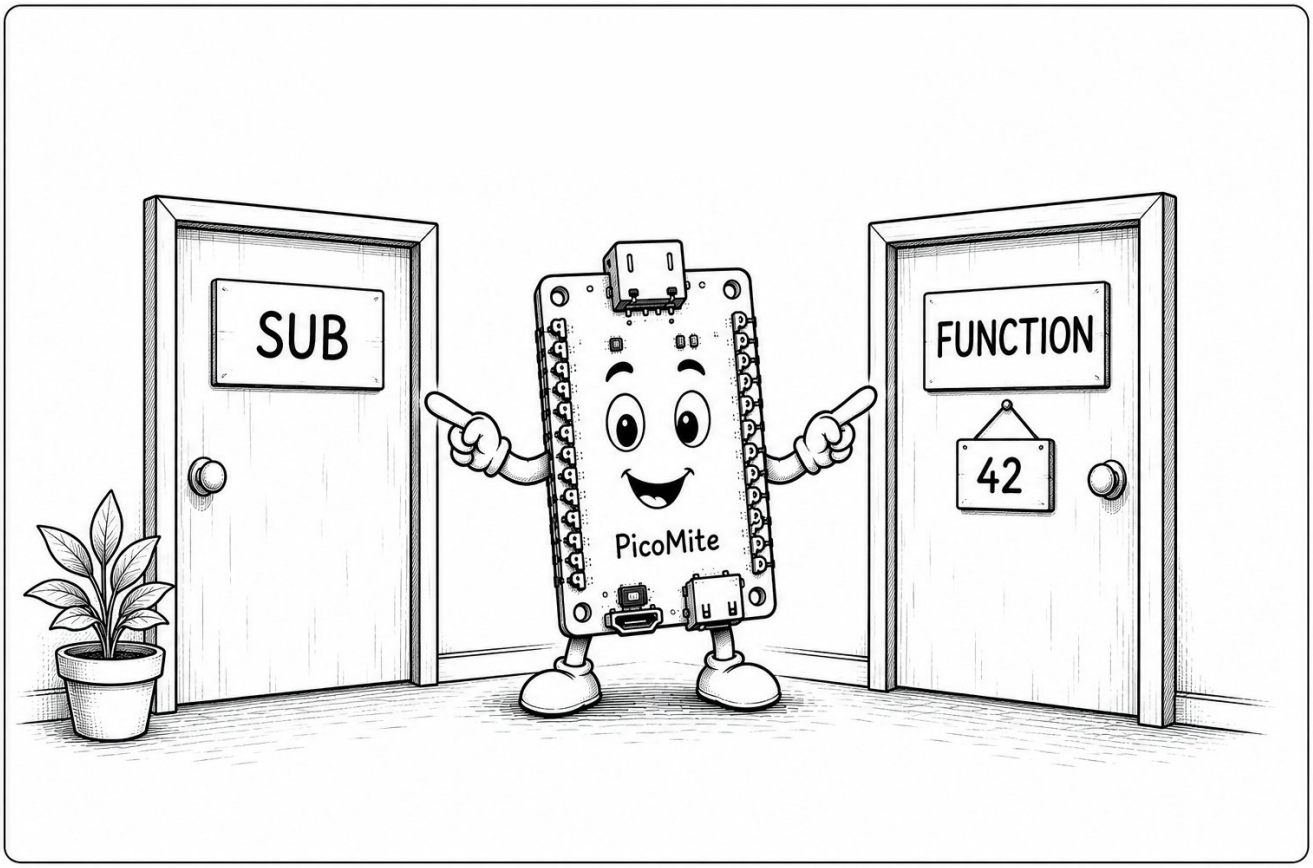
## 9.15. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 09_zahlenraten_v2.bas

# Teil II – Größere Programme entwickeln

# Kapitel 10. Programme strukturieren mit SUB und FUNCTION




## 10.1. Das lernst du

In diesem Kapitel lernst du:

- Programme in kleinere Bausteine aufzuteilen
- eigene Unterprogramme mit `SUB` zu erstellen
- eigene Funktionen mit `FUNCTION` zu schreiben
- Parameter an Unterprogramme und Funktionen zu übergeben
- Werte aus Funktionen zurückzugeben
- wiederkehrende Aufgaben in Unterprogrammen zusammenzufassen
- Berechnungen in eigenen Funktionen zu kapseln
- `SUB` und `FUNCTION` gemeinsam in einem Programm zu verwenden

Am Ende des Kapitels kannst du eigene Befehle und Funktionen entwickeln.

## 10.2. Warum Programme strukturieren?

Unsere Programme werden langsam größer.

Das Zahlenratespiel aus dem letzten Kapitel besteht bereits aus vielen Zeilen Code.

Bei größeren Programmen wird es zunehmend schwieriger, den Überblick zu behalten.

Deshalb teilen Programmierer ihre Programme in kleinere, überschaubare Bausteine auf.

MMBasic stellt dafür SUB und FUNCTION zur Verfügung.

## 10.3. Unsere erste SUB

Eine `SUB` fasst mehrere Anweisungen zu einem Unterprogramm zusammen.

```
SUB Begruessung

  PRINT "Hallo Welt!"

END SUB

Begruessung
```

Ausgabe:

```
Hallo Welt!
```

Die Anweisungen zwischen `SUB` und `END SUB` werden erst ausgeführt, wenn die `SUB` aufgerufen wird.

## 10.4. Eine SUB mehrfach aufrufen

Eine SUB kann beliebig oft verwendet werden.

```
SUB Begruessung

  PRINT "Hallo Welt!"

END SUB

Begruessung
Begruessung
Begruessung
```

Ausgabe:

```
Hallo Welt!
Hallo Welt!
Hallo Welt!
```

## 10.5. Eine SUB mit Parametern

Oft soll ein Unterprogramm unterschiedliche Werte verarbeiten.

Dazu können Parameter verwendet werden.

```
SUB Begruessung(NAME$)

  PRINT "Hallo ";NAME$

END SUB

Begruessung("Manfred")
Begruessung("Geoff")
Begruessung("Peter")
```

Ausgabe:

```
Hallo Manfred
Hallo Geoff
Hallo Peter
```

## 10.6. Warum FUNCTION?

Eine SUB führt Anweisungen aus.

Manchmal möchten wir jedoch einen Wert berechnen und zurückgeben.

Dafür gibt es `FUNCTION`.

## 10.7. Unsere erste FUNCTION

```
FUNCTION Quadrat(X)

  Quadrat=X*X

END FUNCTION

PRINT Quadrat(5)
```

Ausgabe:

```
25
```

Die Funktion wird mit `END FUNCTION` abgeschlossen. Sie erhält einen Wert als Parameter und liefert

das Quadrat dieses Wertes zurück.

## 10.8. Weitere Funktionen

```
FUNCTION Verdoppeln(X)

  Verdoppeln=X*2

END FUNCTION

PRINT Verdoppeln(10)
```

Ausgabe:

```
20
```

## 10.9. Funktionen in Berechnungen verwenden

Der Rückgabewert einer Funktion kann direkt in Berechnungen verwendet werden.

```
FUNCTION Quadrat(X)

  Quadrat=X*X

END FUNCTION

PRINT Quadrat(5)+Quadrat(3)
```

Ausgabe:

```
34
```

SUB- und FUNCTION-Definitionen können in MMBasic an beliebiger Stelle im Programm stehen. Sie müssen also nicht vor ihrem ersten Aufruf definiert werden.

Du kannst beispielsweise das Hauptprogramm am Anfang schreiben und die benötigten Unterprogramme und Funktionen anschließend folgen lassen:

```
PRINT "Das Quadrat von 5 ist ";Quadrat(5)
Begruessung

SUB Begruessung
  PRINT "Willkommen bei MMBasic!"
END SUB
```

```
FUNCTION Quadrat(X)
  Quadrat=X*X
END FUNCTION
```

MMBasic kennt die im Programm definierten Unterprogramme und Funktionen auch dann, wenn ihre Definition erst nach dem Aufruf folgt.

## 10.10. Eine Würfelfunktion

Mit Funktionen lassen sich eigene BASIC-Befehle erstellen.

```
FUNCTION Wuerfel()

  Wuerfel=INT(RND*6)+1

END FUNCTION

PRINT Wuerfel()
PRINT Wuerfel()
PRINT Wuerfel()
```

Mögliche Ausgabe:

```
4
1
6
```

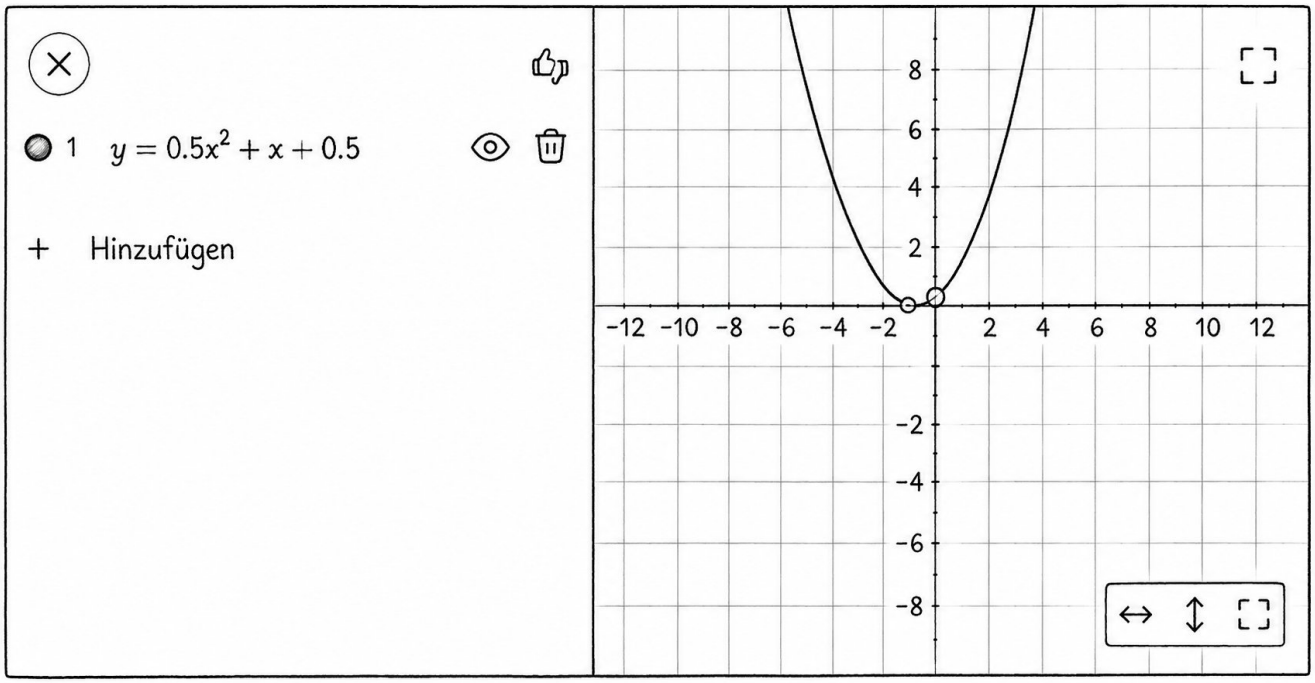
Bei jedem Aufruf wird eine neue Zufallszahl erzeugt.

## 10.11. Eine mathematische Funktion

Eigene Funktionen eignen sich hervorragend für mathematische Berechnungen.

```
FUNCTION F(X)

  F=0.5*X^2+X+0.5

END FUNCTION

FOR X=0 TO 10

  PRINT X;"  ";F(X)

NEXT X
```

Die Funktion berechnet dabei: y = 0.5x² + x + 0.5




# 10.12. Das Zahlenratespiel verbessern

Im letzten Kapitel wurde die geheime Zahl direkt erzeugt:

```
GEHEIM=INT(RND*100)+1
```

Mit einer Funktion wird der Programmcode besser lesbar.

```
FUNCTION NeueZahl()
  NeueZahl=INT(RND*100)+1
END FUNCTION
GEHEIM=NeueZahl()
```

Der Programmierer erkennt sofort, was die Zeile bewirkt.

# 10.13. Wann verwendet man SUB?

Eine SUB eignet sich für Programmteile, die Aufgaben ausführen.

Beispiele:

- Begrüßungen ausgeben
- Menüs anzeigen

- Bildschirmausgaben erzeugen
- Daten einlesen

## 10.14. Wann verwendet man FUNCTION?

Eine FUNCTION eignet sich für Programmteile, die einen Wert berechnen.

Beispiele:

- Flächen berechnen
- Temperaturen umrechnen
- Zufallszahlen erzeugen
- mathematische Funktionen berechnen

## 10.15. Eine SUB für wiederkehrende Aufgaben

Besonders nützlich sind Unterprogramme, wenn dieselbe Aufgabe an mehreren Stellen eines Programms benötigt wird.

Das folgende Unterprogramm zeichnet eine Trennlinie mit einer frei wählbaren Länge:

```
SUB Linie(ANZAHL)

  FOR I=1 TO ANZAHL
    PRINT "-";
  NEXT I
  PRINT

END SUB

PRINT "MMBasic Tutorial"
Linie(20)
PRINT "Kapitel 10"
Linie(20)
```

Ausgabe:

```
MMBasic Tutorial
--------------------
Kapitel 10
--------------------
```

Wenn wir später das Aussehen der Linie ändern möchten, müssen wir dafür nur noch die `SUB` ändern.

## 10.16. Eine Funktion zur Temperaturumrechnung

Eine Funktion kann eine Berechnung kapseln, die an verschiedenen Stellen eines Programms benötigt wird.

```
FUNCTION Fahrenheit(C)

  Fahrenheit=C*9/5+32

END FUNCTION

PRINT "0 Grad Celsius = ";Fahrenheit(0);" Grad Fahrenheit"
PRINT "20 Grad Celsius = ";Fahrenheit(20);" Grad Fahrenheit"
PRINT "30 Grad Celsius = ";Fahrenheit(30);" Grad Fahrenheit"
```

Ausgabe:

```
0 Grad Celsius = 32 Grad Fahrenheit
20 Grad Celsius = 68 Grad Fahrenheit
30 Grad Celsius = 86 Grad Fahrenheit
```

## 10.17. Werte auf einen Bereich begrenzen

In vielen Programmen darf ein Wert einen bestimmten Bereich nicht verlassen. Dafür können wir eine eigene Funktion schreiben:

```
FUNCTION Begrenzen(WERT,MINIMUM,MAXIMUM)

  IF WERT<MINIMUM THEN WERT=MINIMUM
  IF WERT>MAXIMUM THEN WERT=MAXIMUM

  Begrenzen=WERT

END FUNCTION

PRINT Begrenzen(120,0,100)
PRINT Begrenzen(50,0,100)
PRINT Begrenzen(-10,0,100)
```

Ausgabe:

```
100
50
0
```

## 10.18. SUB und FUNCTION gemeinsam verwenden

In einem größeren Programm werden Unterprogramme und Funktionen häufig gemeinsam eingesetzt.

```
FUNCTION Wuerfel()

  Wuerfel=INT(RND*6)+1

END FUNCTION

SUB WurfAnzeigen(WURF)

  PRINT "Du hast eine ";WURF;" gewuerfelt."

END SUB

FOR I=1 TO 3
  WURF=Wuerfel()
  WurfAnzeigen(WURF)
NEXT I
```

Eine `FUNCTION` erzeugt hier den Würfelwert und gibt ihn zurück. Die `SUB` übernimmt die Ausgabe.

Dadurch erhält jeder Programmteil eine klar abgegrenzte Aufgabe.

## 10.19. Beispiel: Den Body-Mass-Index berechnen

Ein praktisches Beispiel für eine Funktion ist die Berechnung des Body-Mass-Index (BMI).

Der BMI wird aus dem Körpergewicht in Kilogramm und der Körpergröße in Metern berechnet:

```
BMI = Gewicht / (Größe * Größe)
```

Dafür können wir eine eigene Funktion schreiben:

```
FUNCTION BMI(GEWICHT,GROESSE)

  BMI=GEWICHT/(GROESSE*GROESSE)

END FUNCTION

PRINT BMI(80,1.80)
```

Ausgabe:

```
24.69136
```

Unsere Funktion erwartet zwei Werte:

- `GEWICHT` ist das Körpergewicht in Kilogramm.
- `GROESSE` ist die Körpergröße in Metern.

Die Funktion berechnet daraus den BMI und gibt das Ergebnis zurück.

Natürlich können die Werte auch vom Benutzer eingegeben werden:

```
INPUT "Gewicht in kg"; GEWICHT
INPUT "Koerpergroesse in m"; GROESSE

PRINT "Dein BMI ist ";BMI(GEWICHT,GROESSE)

FUNCTION BMI(GEWICHT,GROESSE)

  BMI=GEWICHT/(GROESSE*GROESSE)

END FUNCTION
```

Damit übernimmt das Hauptprogramm die Eingabe und Ausgabe, während die Funktion ausschließlich für die Berechnung zuständig ist.

Genau darin liegt ein wichtiger Vorteil von Funktionen: Eine Berechnung wird an einer Stelle definiert und kann anschließend überall im Programm verwendet werden.

## 10.20. Experimentiere!

Probiere folgende Änderungen aus:

- Schreibe eine Funktion zur Berechnung des Dreifachen einer Zahl.
- Schreibe eine Funktion für die Berechnung eines Kreises.
- Erweitere die Würfelfunktion.
- Erstelle eine SUB für eine persönliche Begrüßung.

## 10.21. Probier’s selbst!

Versuche folgende Aufgaben:

1. Schreibe eine Funktion zur Berechnung des Würfels einer Zahl.
2. Schreibe eine Funktion zur Berechnung des Umfangs eines Quadrats.

3. Erstelle eine SUB, die fünfmal "MMBasic" ausgibt.
4. Verwende eine Funktion innerhalb einer FOR-Schleife.
5. Baue die Funktion NeueZahl() in dein Zahlenratespiel ein.

## 10.22. Zusammenfassung

In diesem Kapitel hast du gelernt, Programme in kleinere Bausteine aufzuteilen und dadurch übersichtlicher zu strukturieren.

Du kannst nun:

- eigene Unterprogramme mit `SUB … END SUB` erstellen
- eigene Funktionen mit `FUNCTION … END FUNCTION` schreiben
- Parameter an Unterprogramme und Funktionen übergeben
- Werte aus Funktionen zurückgeben
- wiederkehrende Aufgaben in Unterprogrammen zusammenfassen
- Berechnungen in eigenen Funktionen kapseln
- `SUB` und `FUNCTION` gemeinsam in einem Programm verwenden

Mit diesen Werkzeugen lassen sich auch größere Programme übersichtlich aufbauen und einzelne Programmteile wiederverwenden.

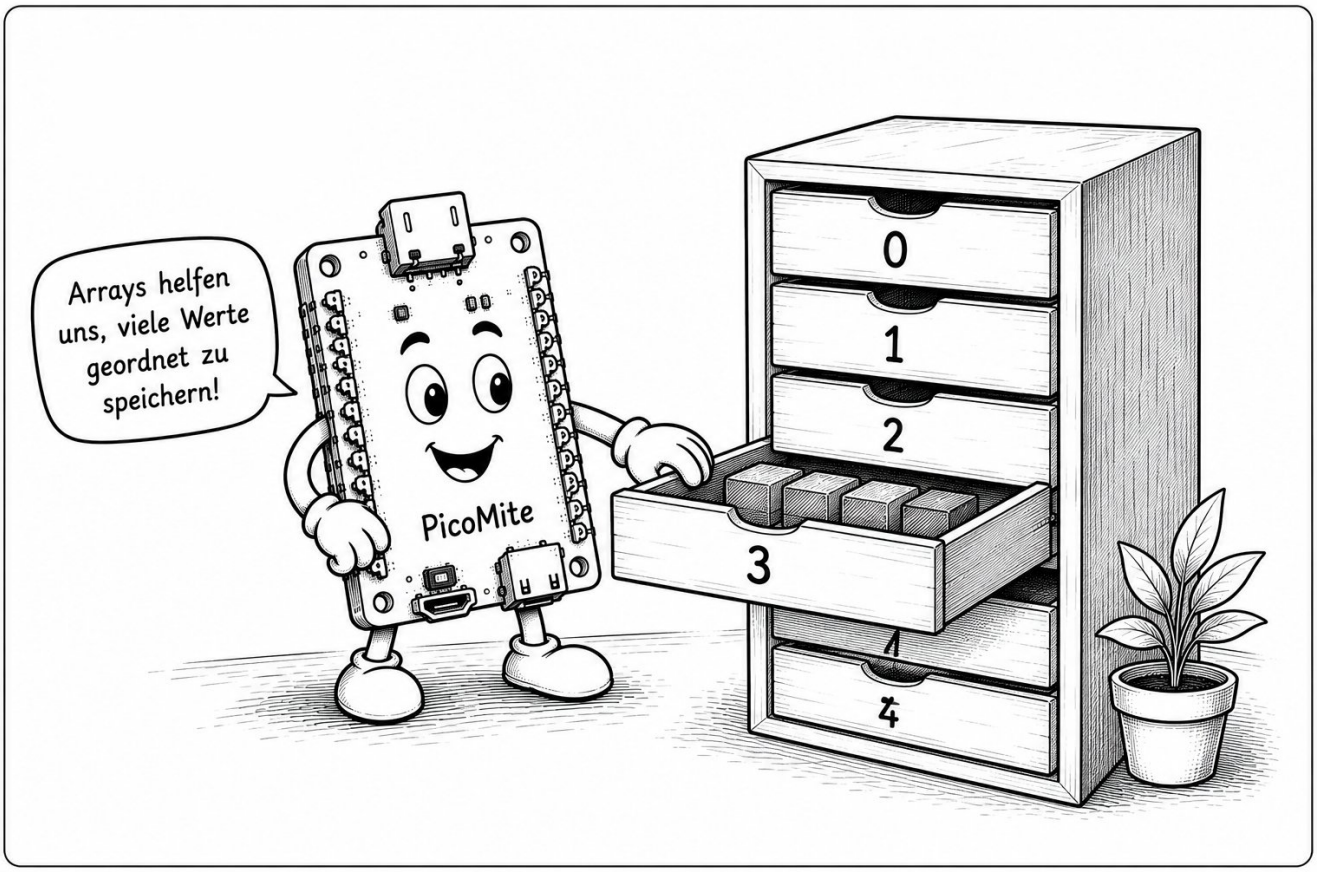
Im nächsten Kapitel werden wir unser Wissen nutzen, um ein weiteres Spiel oder ein größeres Praxisprojekt zu entwickeln.

## 10.23. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 10_strukturiertes_programm.bas

# Kapitel 11. Arrays




## 11.1. Das lernst du

In diesem Kapitel lernst du:

- was ein Array ist
- wie Arrays mit `DIM` angelegt werden
- wie Werte in Arrays gespeichert und wieder ausgelesen werden
- wie Arrays mit `FOR`-Schleifen verarbeitet werden
- wie Zahlen- und Textarrays verwendet werden
- wie ein Array Zustände speichern kann
- wie mehrdimensionale Arrays aufgebaut sind
- wie Daten in Zeilen und Spalten organisiert werden

Am Ende des Kapitels kannst du viele zusammengehörige Werte übersichtlich speichern und verarbeiten.

## 11.2. Das Problem

In unserem Zahlenratespiel könnten mehrere Spieler gegeneinander antreten.

Für jeden Spieler möchten wir die Anzahl der benötigten Versuche speichern.

Ohne Arrays könnte das so aussehen:

```
VERSUCH0=5
VERSUCH1=8
VERSUCH2=3
VERSUCH3=6
VERSUCH4=4
```

Das funktioniert.

Aber was passiert bei 20 Spielern?

Oder 100 Spielern?

Es muss einen besseren Weg geben.

## 11.3. Die Lösung: Arrays

Ein Array ist eine Sammlung von Variablen mit demselben Namen.

Jedes Element besitzt eine Nummer, den sogenannten Index.

Bevor ein Array verwendet werden kann, muss es mit `DIM` angelegt werden.

```
DIM VERSUCH(4)
```

MMBasic nummeriert Array-Elemente standardmäßig ab 0.

Das Array

```
DIM VERSUCH(4)
```

enthält die Elemente

```
VERSUCH(0)
VERSUCH(1)
VERSUCH(2)
VERSUCH(3)
VERSUCH(4)
```

also insgesamt fünf Speicherplätze.

## 11.4. Werte speichern

Werte werden wie bei normalen Variablen gespeichert.

```
DIM VERSUCH(4)

VERSUCH(0)=5
VERSUCH(1)=8
VERSUCH(2)=3
VERSUCH(3)=6
VERSUCH(4)=4
```

## 11.5. Werte ausgeben

Auf einzelne Elemente kann direkt zugegriffen werden.

```
PRINT VERSUCH(0)
PRINT VERSUCH(1)
PRINT VERSUCH(2)
```

Ausgabe:

```
5
8
3
```

## 11.6. Arrays und FOR-Schleifen

Arrays und FOR-Schleifen sind ein perfektes Team.

```
DIM VERSUCH(4)

FOR I=0 TO 4

  Print "Gib den";i+1;". von 5 Zahlenwerten ein";

  INPUT VERSUCH(I)

NEXT I
```

Die eingegebenen Werte werden automatisch in den Array-Elementen gespeichert.

## 11.7. Alle Werte ausgeben

```
FOR I=0 TO 4

  PRINT VERSUCH(I)

NEXT I
```

## 11.8. Den Durchschnitt berechnen

Arrays eignen sich hervorragend für Berechnungen.

```
DIM VERSUCH(4)

SUMME=0

FOR I=0 TO 4

  INPUT VERSUCH(I)

  SUMME=SUMME+VERSUCH(I)

NEXT I

PRINT
PRINT "Durchschnitt:"
PRINT SUMME/5
```

## 11.9. Den besten Spieler finden

Nun suchen wir die kleinste Anzahl von Versuchen.

```
DIM VERSUCH(4)

FOR I=0 TO 4

  INPUT VERSUCH(I)

NEXT I

BEST=VERSUCH(0)

FOR I=1 TO 4

  IF VERSUCH(I)<BEST THEN
    BEST=VERSUCH(I)
```

```
  ENDIF

NEXT I

PRINT
PRINT "Bester Spieler:"
PRINT BEST;" Versuche"
```

## 11.10. Arrays initialisieren

Arrays können bereits beim Anlegen mit Werten gefüllt werden.

```
DIM NAME$(2)=("Manfred","Geoff","Peter")
```

Anschließend können die Werte direkt verwendet werden.

```
PRINT NAME$(0)
PRINT NAME$(1)
PRINT NAME$(2)
```

Ausgabe:

```
Manfred
Geoff
Peter
```

## 11.11. Textarrays

Natürlich können auch Textarrays verwendet werden.

```
DIM SPIELER$(4)

SPIELER$(0)="Manfred"
SPIELER$(1)="Geoff"
SPIELER$(2)="Peter"
SPIELER$(3)="Tom"
SPIELER$(4)="Sarah"
```

Ausgabe:

```
FOR I=0 TO 4

  PRINT SPIELER$(I)
```

```
NEXT I
```

Ausgabe:

```
Manfred
Geoff
Peter
Tom
Sarah
```

## 11.12. Würfelergebnisse speichern

In Kapitel 7 haben wir einen Würfel simuliert.

Mit einem Array können wir mehrere Würfe speichern.

```
DIM WURF(9)

FOR I=0 TO 9

  WURF(I)=INT(RND*6)+1

NEXT I

FOR I=0 TO 9

  PRINT WURF(I)

NEXT I
```

Mögliche Ausgabe:

```
4
2
6
1
5
3
2
4
6
1
```

## 11.13. Eine Wertetabelle speichern

In Kapitel 6 haben wir eine mathematische Funktion berechnet.

Mit Arrays können wir die Ergebnisse speichern.

```
DIM Y(10)

FOR X=0 TO 10

  Y(X)=0.5*X^2+X+0.5

NEXT X

FOR X=0 TO 10

  PRINT X;"  ";Y(X)

NEXT X
```

## 11.14. Lottozahlen ohne Wiederholung

In Kapitel 7 haben wir bereits sechs Lottozahlen erzeugt. Damals konnten die Zahlen allerdings noch mehrfach auftreten.

Inzwischen kennen wir Arrays und können dieses Problem lösen.

Für jede der 49 möglichen Lottozahlen merken wir uns in einem Array, ob sie bereits gezogen wurde. Dazu verwenden wir die beiden Konstanten `TRUE` und `FALSE`.

Mit `CONST` können wir einen Namen mit einem festen Wert verbinden. Anders als bei einer Variablen kann dieser Wert während des Programms nicht mehr verändert werden.

```
CONST TRUE=1
CONST FALSE=0
```

Damit steht `TRUE` für den Wert 1 und `FALSE` für den Wert 0. Im Programm können wir dadurch beispielsweise schreiben:

```
GEZOGEN(22)=TRUE
```

Das ist leichter verständlich als:

```
GEZOGEN(22)=1
```

`TRUE` und `FALSE` sind hier lediglich Namen für die Zahlenwerte 1 und 0.

Wenn wir schreiben:

```
GEZOGEN(22)=TRUE
```

wird im Array nicht das Wort `TRUE` gespeichert. Da wir zuvor

```
CONST TRUE=1
CONST FALSE=0
```

definiert haben, speichert MMBasic tatsächlich den Zahlenwert `1` im Array-Element `GEZOGEN(22)`.

Entsprechend bedeutet:

```
GEZOGEN(22)=FALSE
```

dass dort der Zahlenwert `0` gespeichert wird.

Die Konstanten `TRUE` und `FALSE` machen das Programm lediglich leichter lesbar.

Mit diesem Wissen können wir nun unsere Lottoziehung programmieren. Das Array `GEZOGEN()` merkt sich dabei für jede Zahl von 1 bis 49, ob sie bereits gezogen wurde:

```
CONST TRUE=1
CONST FALSE=0

DIM GEZOGEN(49)

DO

  ' Array fuer eine neue Ziehung loeschen
  FOR ZAHL=1 TO 49
    GEZOGEN(ZAHL)=FALSE
  NEXT ZAHL

  ' Sechs verschiedene Lottozahlen ziehen
  FOR I=1 TO 6

    DO
      ZAHL=INT(RND*49)+1
    LOOP WHILE GEZOGEN(ZAHL)=TRUE

    GEZOGEN(ZAHL)=TRUE

  NEXT I
```

```
  ' Zusatzzahl ziehen
  DO
    ZUSATZZAHL=INT(RND*49)+1
  LOOP WHILE GEZOGEN(ZUSATZZAHL)=TRUE

  ' Lottozahlen sortiert ausgeben
  PRINT "Ziehung der Lottozahlen: ";

  ERSTE=TRUE

  FOR ZAHL=1 TO 49

    IF GEZOGEN(ZAHL)=TRUE THEN

      IF ERSTE=FALSE THEN PRINT ", ";

      PRINT ZAHL;
      ERSTE=FALSE

    ENDIF

  NEXT ZAHL

  PRINT ". Zusatzzahl: ";ZUSATZZAHL

  ' Weitere Ziehung?
  INPUT "Noch einmal ziehen (J/N)"; NOCHMAL$

LOOP WHILE UCASE$(NOCHMAL$)="J"
```

Eine mögliche Ausgabe ist:

```
Ziehung der Lottozahlen: 4, 6, 22, 33, 34, 40. Zusatzzahl: 3
Noch einmal ziehen (J/N)? J

Ziehung der Lottozahlen: 2, 9, 14, 21, 37, 48. Zusatzzahl: 16
Noch einmal ziehen (J/N)? N
```

### 11.14.1. Das Array als Merkspeicher

Das Array `GEZOGEN()` dient als Gedächtnis für unsere Lottoziehung.

Zu Beginn einer neuen Ziehung setzen wir alle Elemente auf `FALSE`:

```
FOR ZAHL=1 TO 49
  GEZOGEN(ZAHL)=FALSE
NEXT ZAHL
```

Damit ist zunächst keine der 49 möglichen Zahlen als gezogen markiert.

Wird beispielsweise die Zahl 22 gezogen, setzen wir das entsprechende Array-Element auf `TRUE`:

```
GEZOGEN(22)=TRUE
```

Bei jeder weiteren Ziehung prüfen wir, ob die erzeugte Zahl bereits als gezogen markiert ist:

```
DO
  ZAHL=INT(RND*49)+1
LOOP WHILE GEZOGEN(ZAHL)=TRUE
```

Solange die gezogene Zahl bereits im Array mit `TRUE` markiert ist, wird eine neue Zufallszahl erzeugt. Erst bei einer noch nicht gezogenen Zahl wird die Schleife beendet.

Auf diese Weise erhalten wir sechs verschiedene Lottozahlen.

### 11.14.2. Die Zusatzzahl

Auch die Zusatzzahl darf keine der sechs zuvor gezogenen Lottozahlen sein. Deshalb verwenden wir bei ihrer Ermittlung dieselbe Prüfung:

```
DO
  ZUSATZZAHL=INT(RND*49)+1
LOOP WHILE GEZOGEN(ZUSATZZAHL)=TRUE
```

Da die Zusatzzahl anschließend nicht mehr für weitere Ziehungen geprüft werden muss, brauchen wir sie nicht zusätzlich im Array zu markieren.

### 11.14.3. Sortieren ganz ohne Sortieralgorithmus

Für die Ausgabe benötigen wir keinen eigenen Sortieralgorithmus.

Das Array `GEZOGEN()` besitzt für jede mögliche Lottozahl von 1 bis 49 ein Element. Deshalb können wir das Array einfach in dieser Reihenfolge durchlaufen:

```
FOR ZAHL=1 TO 49
  IF GEZOGEN(ZAHL)=TRUE THEN
    PRINT ZAHL
  ENDIF
NEXT ZAHL
```

Da wir bei 1 beginnen und bei 49 aufhören, werden die gezogenen Zahlen automatisch in aufsteigender Reihenfolge ausgegeben.

Für die gewünschte Ausgabe in einer einzigen Zeile benötigen wir noch die Variable `ERSTE`. Sie

merkt sich, ob bereits eine Lottozahl ausgegeben wurde:

```
ERSTE=TRUE

FOR ZAHL=1 TO 49

  IF GEZOGEN(ZAHL)=TRUE THEN

    IF ERSTE=FALSE THEN PRINT ", ";

    PRINT ZAHL;
    ERSTE=FALSE

  ENDIF

NEXT ZAHL
```

Vor der ersten Lottozahl wird dadurch kein Komma ausgegeben. Vor allen folgenden Zahlen dagegen schon.

### 11.14.4. Eine weitere Ziehung

Nach jeder Ziehung fragen wir den Benutzer, ob eine weitere Ziehung durchgeführt werden soll:

```
INPUT "Noch einmal ziehen (J/N)"; NOCHMAL$
```

Die gesamte Lottoziehung befindet sich deshalb in einer weiteren `DO ⋯ LOOP`-Schleife:

```
DO

  ' Lottoziehung

  INPUT "Noch einmal ziehen (J/N)"; NOCHMAL$

LOOP WHILE UCASE$(NOCHMAL$)="J"
```

Mit `UCASE$()` wird die Eingabe vor dem Vergleich in Großbuchstaben umgewandelt. Deshalb werden sowohl `J` als auch `j` akzeptiert.

Wird etwas anderes eingegeben, endet die Schleife und damit das Programm.

Dieses Beispiel zeigt einen weiteren Vorteil von Arrays: Sie können nicht nur Werte speichern, sondern auch Zustände festhalten.

`GEZOGEN(22)=TRUE` lässt sich fast wie ein Satz lesen:

„Die Zahl 22 wurde bereits gezogen.“

> Die Konstanten `TRUE` und `FALSE` machen solche Abfragen leichter lesbar als die direkte Verwendung der Zahlen 1 und 0.

## 11.15. Mehrdimensionale Arrays

Ein Array kann mehrere Dimensionen besitzen.

Stell dir eine Tabelle mit Zeilen und Spalten vor.

In einer kleinen Notenverwaltung stehen die Zeilen für die Schüler und die Spalten für die einzelnen Fächer.

```
DIM SCHUELER$(4)
SCHUELER$(0)="Anna"
SCHUELER$(1)="Ben"
SCHUELER$(2)="Clara"
SCHUELER$(3)="David"
SCHUELER$(4)="Emma"

DIM FACH$(2)
FACH$(0)="Deutsch"
FACH$(1)="Englisch"
FACH$(2)="Mathe"

DIM NOTE(4,2)

' Zufällige Noten vergeben
FOR S=0 TO 4
  FOR F=0 TO 2
    NOTE(S,F)=INT(RND*6)+1
  NEXT F
NEXT S

' Tabelle ausgeben
PRINT "Schüler",
FOR F=0 TO 2
  PRINT FACH$(F),
NEXT F
PRINT

FOR S=0 TO 4
  PRINT SCHUELER$(S),
  FOR F=0 TO 2
    PRINT NOTE(S,F),
  NEXT F
  PRINT
NEXT S
```

Die erste Dimension (`S`) beschreibt den Schüler.

Die zweite Dimension (F) beschreibt das Fach.

Der Zugriff

```
NOTE(2,1)=1
```

bedeutet:

- Schüler Nummer 2
- Fach Nummer 1 (Englisch)
- Note 1

## 11.16. Beispiel: Ein Spielfeld im Array

Ein zweidimensionales Array eignet sich nicht nur für Tabellen. Auch ein Spielfeld kann als Array gespeichert werden.

Dabei beschreibt die erste Dimension die Zeile und die zweite Dimension die Spalte.

Wir legen ein Spielfeld mit 5 Zeilen und 10 Spalten an:

```
DIM FELD$(4,9)
```

Zunächst füllen wir alle Felder mit einem Punkt:

```
FOR ZEILE=0 TO 4
  FOR SPALTE=0 TO 9
    FELD$(ZEILE,SPALTE)="."
  NEXT SPALTE
NEXT ZEILE
```

Nun setzen wir an einigen Positionen Hindernisse und eine Spielfigur:

```
FELD$(1,3)="#"
FELD$(2,6)="#"
FELD$(3,4)="#"
FELD$(2,2)="X"
```

Anschließend geben wir das gesamte Spielfeld aus:

```
FOR ZEILE=0 TO 4
  FOR SPALTE=0 TO 9
    PRINT FELD$(ZEILE,SPALTE);
  NEXT SPALTE
```

```
  PRINT
NEXT ZEILE
```

Ausgabe:

```
..........
...#......
..X...#...
....#.....
..........
```

Jedes Element des Arrays entspricht genau einer Position auf dem Spielfeld.

Der Zugriff

```
FELD$(2,2)
```

bezeichnet beispielsweise das Feld in Zeile 2 und Spalte 2. Dort befindet sich in unserem Beispiel die Spielfigur `X`.

Zweidimensionale Arrays werden häufig verwendet, wenn Daten in Zeilen und Spalten angeordnet sind. Neben Tabellen können damit beispielsweise Spielfelder, Labyrinthe oder Karten gespeichert werden.

## 11.17. Experimentiere!

Probiere folgende Änderungen aus:

- Speichere zehn Würfelergebnisse.
- Speichere die Punkte von zehn Spielern.
- Berechne einen Durchschnitt.
- Vergrößere ein Array.
- Verändere das Spielfeld und füge weitere Hindernisse hinzu.
- Verschiebe die Spielfigur `X` auf eine andere Position.

## 11.18. Probier’s selbst!

Versuche folgende Aufgaben:

1. Speichere die Alter von fünf Personen.
2. Gib alle Werte mit einer FOR-Schleife aus.
3. Berechne die Summe aller Werte.
4. Ermittle den größten Wert.

5. Speichere fünf Namen in einem Textarray.

## 11.19. Zusammenfassung

In diesem Kapitel hast du Arrays kennengelernt.

Du kannst nun:

- Arrays mit `DIM` anlegen
- Werte in Arrays speichern und wieder auslesen
- Arrays mit `FOR`-Schleifen verarbeiten
- Zahlen- und Textarrays verwenden
- Arrays als Merkspeicher für Zustände verwenden
- mehrdimensionale Arrays anlegen
- Daten in Zeilen und Spalten organisieren

Arrays gehören zu den wichtigsten Werkzeugen der Programmierung und werden in vielen späteren Projekten verwendet.

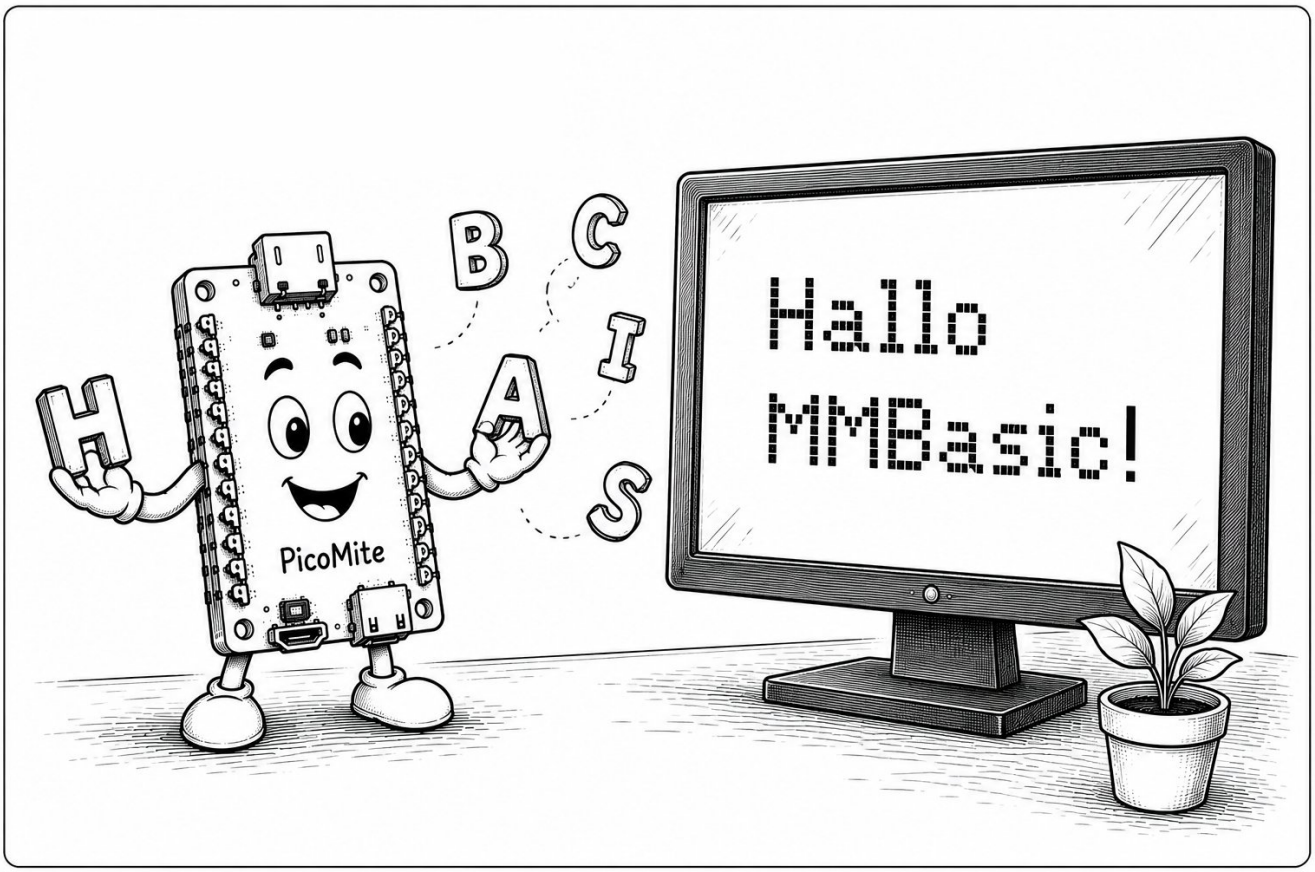
Im nächsten Kapitel beschäftigen wir uns mit Zeichenketten und lernen, Texte zu bearbeiten und auszuwerten.

## 11.20. Programmbeispiele

Die vollständigen BASIC-Programme findest du hier:

- 11_array_einfach.bas
- 11_lottozahlen.bas

# Kapitel 12. Mit Texten arbeiten




## 12.1. Das lernst du

In diesem Kapitel lernst du:

- Texte in Variablen zu speichern
- Texte einzugeben und auszugeben
- die Länge eines Textes zu bestimmen
- Teile eines Textes auszulesen
- Texte in Groß- und Kleinbuchstaben umzuwandeln
- in Texten zu suchen
- Zahlen und Texte ineinander umzuwandeln
- Datum und Uhrzeit zu verwenden
- mehrere Stringfunktionen in einem Programm miteinander zu kombinieren

Am Ende des Kapitels kannst du Texte in deinen Programmen verarbeiten und auswerten und die verschiedenen Möglichkeiten der Stringverarbeitung in einem kleinen Spiel praktisch einsetzen.

## 12.2. Zahlen und Texte

Bisher haben wir hauptsächlich mit Zahlen gearbeitet.

Zum Beispiel:

```
ALTER=60
PUNKTE=125
```

Programme arbeiten aber nicht nur mit Zahlen.

Oft müssen auch Texte verarbeitet werden.

Zum Beispiel:

- Namen
- Adressen
- Dateinamen
- Antworten auf Fragen
- Meldungen

Texte werden in MMBasic in String-Variablen gespeichert.

String-Variablen erkennt man am Dollarzeichen am Ende des Namens.

```
NAME$="Manfred"
ORT$="Schwetzingen"
```

# 12.3. Wie lang darf ein Text sein?

Normale Zeichenketten können in MMBasic maximal 255 Zeichen enthalten.

Für die meisten Programme ist das vollkommen ausreichend.

Beispiele:

- Namen
- Dateinamen
- Benutzereingaben
- Meldungen
- kurze Texte

```
NAME$="Manfred"
```

Für längere Texte und größere Datenmengen stellt MMBasic sogenannte Long Strings zur Verfügung.

Wie Long Strings verwendet werden, sehen wir später im Kapitel zur seriellen Kommunikation.

## 12.4. Texte ausgeben

Texte können wie Zahlen ausgegeben werden.

```
NAME$="Manfred"

PRINT NAME$
```

Ausgabe:

```
Manfred
```

## 12.5. Texte eingeben

Mit INPUT können auch Texte eingegeben werden.

```
INPUT NAME$

PRINT "Hallo ";NAME$
```

Beispiel:

```
? Manfred

Hallo Manfred
```

## 12.6. Texte verbinden

Mehrere Texte können in einer Ausgabe kombiniert werden.

```
VORNAME$="Manfred"
NACHNAME$="Becker"

PRINT VORNAME$;" ";NACHNAME$
```

Ausgabe:

```
Manfred Becker
```

## 12.7. Die Länge eines Textes bestimmen

Mit `LEN()` kann die Anzahl der Zeichen ermittelt werden.

```
NAME$="Manfred"

PRINT LEN(NAME$)
```

Ausgabe:

```
7
```

Jeder Buchstabe zählt als ein Zeichen.

Auch Leerzeichen werden mitgezählt.

## 12.8. Das erste Zeichen ermitteln

Mit `LEFT$()` können Zeichen vom Anfang eines Textes gelesen werden.

```
NAME$="Manfred"

PRINT LEFT$(NAME$,1)
```

Ausgabe:

```
M
```

## 12.9. Mehrere Zeichen vom Anfang lesen

```
NAME$="Manfred"

PRINT LEFT$(NAME$,3)
```

Ausgabe:

```
Man
```

## 12.10. Zeichen vom Ende lesen

Mit `RIGHT$()` können Zeichen vom Ende eines Textes gelesen werden.

```
NAME$="Manfred"

PRINT RIGHT$(NAME$,3)
```

Ausgabe:

```
red
```

## 12.11. Zeichen aus der Mitte lesen

Mit `MID$()` können Zeichen aus der Mitte eines Textes gelesen werden.

```
NAME$="Manfred"

PRINT MID$(NAME$,3,2)
```

Ausgabe:

```
nf
```

Die Zahl 3 gibt die Startposition an.

Die Zahl 2 gibt die Anzahl der Zeichen an.

## 12.12. Groß- und Kleinbuchstaben

Mit `UCASE$()` wird ein Text in Großbuchstaben umgewandelt.

```
NAME$="Manfred"

PRINT UCASE$(NAME$)
```

Ausgabe:

```
MANFRED
```

Mit `LCASE$()` wird ein Text in Kleinbuchstaben umgewandelt.

```
NAME$="Manfred"

PRINT LCASE$(NAME$)
```

Ausgabe:

```
manfred
```

## 12.13. Texte vergleichen

Texte können mit IF verglichen werden.

```
INPUT ANTWORT$

IF ANTWORT$="JA" THEN
  PRINT "Richtig"
ENDIF
```

Dabei muss die Schreibweise exakt übereinstimmen.

Die Eingabe

```
ja
```

ist nicht dasselbe wie

```
JA
```

## 12.14. Groß- und Kleinschreibung ignorieren

Oft soll die Schreibweise keine Rolle spielen.

Mit UCASE$() lässt sich das einfach lösen.

```
INPUT ANTWORT$

IF UCASE$(ANTWORT$)="JA" THEN
  PRINT "Richtig"
ENDIF
```

Jetzt werden beispielsweise

```
JA
Ja
ja
jA
```

alle akzeptiert.

## 12.15. In einem Text suchen

Mit `INSTR()` kann nach einem Zeichen oder einer Zeichenfolge gesucht werden.

```
TEXT$="MMBasic macht Spass"

PRINT INSTR(TEXT$,"macht")
```

Ausgabe:

```
9
```

Die Funktion liefert die Position zurück, an der der gesuchte Text beginnt.

Wird der Text nicht gefunden, liefert INSTR() den Wert 0.

```
TEXT$="MMBasic macht Spass"

IF INSTR(TEXT$,"Basic")>0 THEN
  PRINT "Text gefunden"
ELSE
  PRINT "Text nicht gefunden"
ENDIF
```

## 12.16. Zahlen und Texte umwandeln

Mit `VAL()` wird ein Text in eine Zahl umgewandelt.

```
ZAHL=VAL("123")

PRINT ZAHL+1
```

Ausgabe:

```
124
```

Mit `STR$()` wird umgekehrt eine Zahl in einen Text umgewandelt.

```
TEXT$=STR$(123)
```

```
PRINT "Die Zahl lautet ";TEXT$
```

## 12.17. Datum und Uhrzeit

MMBasic stellt das aktuelle Datum und die aktuelle Uhrzeit als Zeichenketten zur Verfügung. Dafür stehen `DATE$` und `TIME$` bereit.

```
PRINT DATE$
PRINT TIME$
```

Die genaue Darstellung hängt von der Konfiguration des Systems ab.

Da Datum und Uhrzeit Zeichenketten sind, können sie mit den bekannten String-Funktionen verarbeitet werden.

```
PRINT "Heute ist ";DATE$
PRINT "Es ist ";TIME$
```

## 12.18. Begrüßung abhängig von der Uhrzeit

Mit LEFT$() lässt sich die Stunde aus der Uhrzeit auslesen.

Da LEFT$() einen Text zurückliefert, wird dieser mit VAL() in eine Zahl umgewandelt.

```
STUNDE=VAL(LEFT$(TIME$,2))

IF STUNDE<12 THEN
  PRINT "Guten Morgen!"
ELSEIF STUNDE<18 THEN
  PRINT "Guten Tag!"
ELSE
  PRINT "Guten Abend!"
ENDIF
```

## 12.19. Ein kleines Praxisprogramm

Das folgende Programm verwendet viele der neuen Funktionen.

```
INPUT NAME$

PRINT
PRINT "Hallo ";NAME$
PRINT
```

```
PRINT "Heute ist ";DATE$
PRINT "Aktuelle Uhrzeit: ";TIME$
PRINT

PRINT "Dein Name hat ";LEN(NAME$);" Zeichen."

PRINT "Er beginnt mit ";LEFT$(NAME$,1)

PRINT "Er endet mit ";RIGHT$(NAME$,1)

IF VAL(LEFT$(TIME$,2))<12 THEN
  PRINT "Guten Morgen!"
ELSEIF VAL(LEFT$(TIME$,2))<18 THEN
  PRINT "Guten Tag!"
ELSE
  PRINT "Guten Abend!"
ENDIF
```

# 12.20. Praxisbeispiel: Hangman

Wir haben inzwischen verschiedene Möglichkeiten kennengelernt, Zeichenketten zu untersuchen und zu verändern.

Zeit, diese Funktionen in einem kleinen Spiel miteinander zu kombinieren.

Beim klassischen Hangman-Spiel muss ein unbekanntes Wort Buchstabe für Buchstabe erraten werden. Zu Beginn sehen wir nur, wie viele Buchstaben das gesuchte Wort besitzt:

```
_ _ _ _ _ _ _
```

Nach jeder Eingabe werden alle richtig geratenen Buchstaben aufgedeckt.

Für unser erstes Spiel verwenden wir das Wort `MMBASIC`.

## 12.20.1. Das gesuchte Wort

Das gesuchte Wort speichern wir in einer Zeichenkette:

```
WORT$="MMBASIC"
```

Mit `LEN()` können wir feststellen, aus wie vielen Zeichen das Wort besteht:

```
PRINT LEN(WORT$)
```

Die Ausgabe lautet:

```
7
```

Diese Information benötigen wir später, um alle Buchstaben des Wortes nacheinander zu untersuchen.

### 12.20.2. Bereits geratene Buchstaben speichern

Wir benötigen außerdem eine Zeichenkette, in der wir uns die bereits eingegebenen Buchstaben merken:

```
GERATEN$=""
```

Gibt der Spieler beispielsweise den Buchstaben `A` ein, hängen wir ihn an diese Zeichenkette an:

```
GERATEN$=GERATEN$+"A"
```

Nach mehreren Versuchen könnte `GERATEN$` beispielsweise so aussehen:

```
AMBS
```

Damit weiß unser Programm, welche Buchstaben bereits geraten wurden.

### 12.20.3. Einen einzelnen Buchstaben untersuchen

Mit `MID$()` können wir einen einzelnen Buchstaben aus dem gesuchten Wort herausholen:

```
PRINT MID$(WORT$,3,1)
```

Beim Wort `MMBASIC` erhalten wir:

```
B
```

Die Zahl `3` gibt die Position an und die Zahl `1`, dass genau ein Zeichen gelesen werden soll.

Mit einer `FOR`-Schleife können wir deshalb das gesamte Wort Zeichen für Zeichen durchlaufen:

```
FOR I=1 TO LEN(WORT$)
  PRINT MID$(WORT$,I,1)
NEXT I
```

### 12.20.4. Welche Buchstaben dürfen wir anzeigen?

Jetzt verbinden wir `MID$()` mit `INSTR()`.

Für jeden Buchstaben des gesuchten Wortes prüfen wir, ob er bereits in `GERATEN$` vorkommt:

```
FOR I=1 TO LEN(WORT$)

  BUCHSTABE$=MID$(WORT$,I,1)

  IF INSTR(GERATEN$,BUCHSTABE$)>0 THEN
    PRINT BUCHSTABE$;" ";
  ELSE
    PRINT "_ ";
  ENDIF

NEXT I

PRINT
```

Enthält `GERATEN$` beispielsweise

```
GERATEN$="AMB"
```

erscheint:

```
M M B A _ _ _
```

Damit haben wir bereits den wichtigsten Teil unseres Hangman-Spiels.

### 12.20.5. Groß- und Kleinschreibung

Der Spieler soll sowohl `m` als auch `M` eingeben dürfen.

Mit `UCASE$()` wandeln wir die Eingabe deshalb in Großbuchstaben um:

```
INPUT "Buchstabe"; EINGABE$

EINGABE$=UCASE$(EINGABE$)
```

Für das Spiel macht es dadurch keinen Unterschied, ob der Spieler `a` oder `A` eingibt.

### 12.20.6. Doppelte Eingaben erkennen

Bevor wir einen Buchstaben zu `GERATEN$` hinzufügen, können wir mit `INSTR()` prüfen, ob er dort bereits vorhanden ist:

```
IF INSTR(GERATEN$,EINGABE$)>0 THEN
  PRINT "Diesen Buchstaben hast du bereits versucht."
ELSE
  GERATEN$=GERATEN$+EINGABE$
ENDIF
```

Auch hier zeigt sich, wie praktisch Zeichenketten sein können: `GERATEN$` dient gleichzeitig als Speicher für alle bisher eingegebenen Buchstaben.

### 12.20.7. Das vollständige Spiel

Nun setzen wir die einzelnen Teile zusammen.

Wie bei unserer Lottoziehung verwenden wir wieder die Konstanten `TRUE` und `FALSE`. Die Variable `FERTIG` merkt sich damit, ob alle Buchstaben des gesuchten Wortes bereits gefunden wurden.

Der Spieler hat acht Fehlversuche. Nach jedem Versuch wird das Wort erneut angezeigt.

```
CONST TRUE=1
CONST FALSE=0

WORT$="MMBASIC"
GERATEN$=""
FEHLER=0
MAXFEHLER=8

DO

  PRINT

  ' Wort anzeigen
  FERTIG=TRUE

  FOR I=1 TO LEN(WORT$)

    BUCHSTABE$=MID$(WORT$,I,1)

    IF INSTR(GERATEN$,BUCHSTABE$)>0 THEN
      PRINT BUCHSTABE$;" ";
    ELSE
      PRINT "_ ";
      FERTIG=FALSE
    ENDIF

  NEXT I

  PRINT
  PRINT
```

```
  ' Wurde das ganze Wort erraten?
  IF FERTIG=TRUE THEN

    PRINT "Geschafft!"
    PRINT "Du hast das Wort ";WORT$;" erraten."

  ELSE

    PRINT "Fehlversuche: ";FEHLER;" von ";MAXFEHLER
    INPUT "Buchstabe"; EINGABE$

    EINGABE$=UCASE$(EINGABE$)

    ' Nur das erste eingegebene Zeichen verwenden
    EINGABE$=LEFT$(EINGABE$,1)

    IF INSTR(GERATEN$,EINGABE$)>0 THEN

      PRINT "Diesen Buchstaben hast du bereits versucht."

    ELSE

      GERATEN$=GERATEN$+EINGABE$

      IF INSTR(WORT$,EINGABE$)=0 THEN
        FEHLER=FEHLER+1
        PRINT "Leider falsch!"
      ENDIF

    ENDIF

  ENDIF

LOOP UNTIL FERTIG=TRUE OR FEHLER=MAXFEHLER

PRINT

IF FERTIG=FALSE THEN
  PRINT "Leider verloren!"
  PRINT "Das gesuchte Wort war ";WORT$;"."
ENDIF
```

Eine mögliche Runde könnte so aussehen:

```
_ _ _ _ _ _ _

Fehlversuche: 0 von 8
Buchstabe? A

_ _ _ A _ _ _
```

```
Fehlversuche: 0 von 8
Buchstabe? X
Leider falsch!

_ _ _ A _ _ _

Fehlversuche: 1 von 8
Buchstabe? M

M M _ A _ _ _

Fehlversuche: 1 von 8
Buchstabe? B

M M B A _ _ _

...

M M B A S I C

Geschafft!
Du hast das Wort MMBASIC erraten.
```

### 12.20.8. Viele Stringfunktionen in einem Spiel

Unser kleines Hangman-Spiel verwendet gleich mehrere der Stringfunktionen, die wir in diesem Kapitel kennengelernt haben:

| Funktion | Aufgabe im Spiel |
|---|---|
| `LEN()` | ermittelt die Länge des gesuchten Wortes |
| `MID$()` | liest einzelne Buchstaben aus dem Wort |
| `INSTR()` | sucht nach bereits geratenen oder richtigen Buchstaben |
| `UCASE$()` | wandelt die Eingabe in Großbuchstaben um |
| `LEFT$()` | verwendet nur das erste eingegebene Zeichen |
| `+` | fügt einen neuen Buchstaben an `GERATEN$` an |

Aus einzelnen Befehlen zur Textverarbeitung ist damit ein vollständiges kleines Spiel entstanden.

## 12.21. Experimentiere!

Probiere folgende Änderungen aus:

- Gib die ersten drei Zeichen eines Namens aus.
- Gib die letzten drei Zeichen aus.
- Wandle einen Namen in Großbuchstaben um.

- Wandle einen Namen in Kleinbuchstaben um.
- Suche nach einem Wort in einem Text.
- Ändere das gesuchte Wort im Hangman-Spiel.
- Verändere die Anzahl der erlaubten Fehlversuche.
- Lass das Hangman-Spiel zusätzlich alle bisher geratenen Buchstaben anzeigen.

## 12.22. Probier’s selbst!

Versuche folgende Aufgaben:

1. Frage den Namen des Benutzers ab.
2. Gib die Anzahl der Zeichen aus.
3. Gib den ersten Buchstaben aus.
4. Gib den letzten Buchstaben aus.
5. Gib den Namen komplett in Großbuchstaben aus.
6. Suche nach einem bestimmten Wort in einem Satz.
7. Gib Datum und Uhrzeit aus.
8. Erweitere das Hangman-Spiel so, dass der Spieler nach Spielende gefragt wird, ob er noch einmal spielen möchte.
9. Lege mehrere mögliche Wörter in einem String-Array ab und wähle für jede neue Runde zufällig eines davon aus.

## 12.23. Zusammenfassung

In diesem Kapitel hast du gelernt, mit Texten und Zeichenketten zu arbeiten.

Du kennst nun:

- String-Variablen
- `LEN()`
- `LEFT$()`
- `MID$()`
- `RIGHT$()`
- `UCASE$()`
- `LCASE$()`
- `INSTR()`
- `VAL()`
- `STR$()`
- `DATE$`

- `TIME$`

Du kannst damit Texte speichern, verändern, durchsuchen und auswerten.

Im Hangman-Spiel hast du außerdem gesehen, wie mehrere dieser Funktionen zusammen eingesetzt werden können. Dabei haben wir einen String sogar als Speicher für die bereits geratenen Buchstaben verwendet.

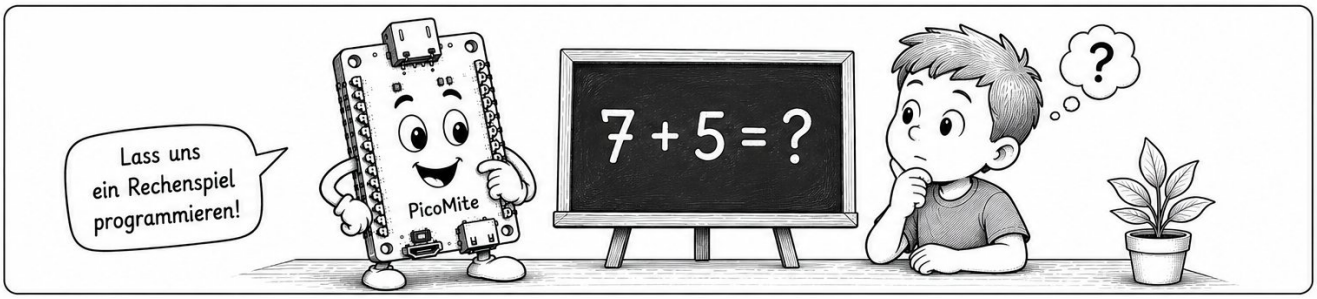
Im nächsten Kapitel werden wir das bisher Gelernte in einem größeren Praxisprojekt einsetzen.

## 12.24. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 12_stringfunktionen.bas
- 12_hangman.bas

# Kapitel 13. Little Professor




## 13.1. Das lernst du

In diesem Kapitel verwendest du viele der bisher gelernten Befehle, um ein vollständiges Programm zu entwickeln.

Dabei kommen unter anderem zum Einsatz:

- Variablen
- Zufallszahlen
- Benutzereingaben
- Entscheidungen mit IF
- Schleifen mit FOR
- Punktzählung

Am Ende des Kapitels besitzt du deinen eigenen kleinen Mathematiktrainer.

## 13.2. Was ist ein Little Professor?

Der Little Professor war ein Lerncomputer von Texas Instruments.

Er stellte Rechenaufgaben und überprüfte die Antworten des Benutzers.

Das Gerät erfreute sich besonders bei Schülern großer Beliebtheit, da es das Kopfrechnen auf spielerische Weise trainierte.

In diesem Kapitel entwickeln wir eine einfache Version mit MMBasic.

## 13.3. Unsere erste Rechenaufgabe

Beginnen wir mit einer einzigen Aufgabe.

```
A=INT(RND*10)
B=INT(RND*10)

PRINT A;" + ";B;" = ?"
```

```
INPUT ANTWORT

IF ANTWORT=A+B THEN
  PRINT "Richtig!"
ELSE
  PRINT "Leider falsch!"
ENDIF
```

## 13.4. Das Programm verstehen

Die Variablen `A` und `B` erhalten Zufallszahlen zwischen 0 und 9.

```
A=INT(RND*10)
B=INT(RND*10)
```

Anschließend wird die Aufgabe ausgegeben.

```
PRINT A;" + ";B;" = ?"
```

Mit `INPUT` wird die Antwort eingelesen.

Danach vergleicht die IF-Anweisung die Eingabe mit dem korrekten Ergebnis.

## 13.5. Mehrere Aufgaben hintereinander

Ein echter Little Professor stellt natürlich nicht nur eine einzige Aufgabe.

Mit einer FOR-Schleife können wir mehrere Aufgaben erzeugen.

```
FOR RUNDE=1 TO 10

  A=INT(RND*10)
  B=INT(RND*10)

  PRINT
  PRINT "Aufgabe ";RUNDE
  PRINT A;" + ";B;" = ?"

  INPUT ANTWORT

  IF ANTWORT=A+B THEN
    PRINT "Richtig!"
  ELSE
    PRINT "Leider falsch!"
  ENDIF
```

```
NEXT RUNDE
```

Jetzt werden zehn Aufgaben hintereinander gestellt.

## 13.6. Punkte zählen

Natürlich möchten wir wissen, wie viele Aufgaben richtig gelöst wurden.

Dafür verwenden wir eine Variable mit dem Namen `PUNKTE`.

```
PUNKTE=0
```

Für jede richtige Antwort erhöhen wir den Punktestand um eins.

```
PUNKTE=PUNKTE+1
```

## 13.7. Die richtige Lösung anzeigen

Wird eine Aufgabe falsch beantwortet, kann das Programm die richtige Lösung anzeigen.

```
IF ANTWORT=A+B THEN
  PRINT "Richtig!"
  PUNKTE=PUNKTE+1
ELSE
  PRINT "Leider falsch!"
  PRINT "Richtig waere ";A+B
ENDIF
```

Dadurch kann der Spieler aus seinen Fehlern lernen.

## 13.8. Das komplette Programm

```
PUNKTE=0

FOR RUNDE=1 TO 10

  A=INT(RND*10)
  B=INT(RND*10)

  PRINT
  PRINT "Aufgabe ";RUNDE
  PRINT A;" + ";B;" = ?"
```

```
  INPUT ANTWORT

  IF ANTWORT=A+B THEN
    PRINT "Richtig!"
    PUNKTE=PUNKTE+1
  ELSE
    PRINT "Leider falsch!"
    PRINT "Richtig waere ";A+B
  ENDIF

NEXT RUNDE

PRINT
PRINT "Du hast ";PUNKTE;" von 10 Punkten erreicht."
```

## 13.9. Eine Beispielrunde

Eine mögliche Programmausführung könnte so aussehen:

```
Aufgabe 1
7 + 5 = ?
12
Richtig!

Aufgabe 2
3 + 8 = ?
12
Leider falsch!
Richtig waere 11

Aufgabe 3
6 + 1 = ?
7
Richtig!
```

## 13.10. Das Ergebnis bewerten

Am Ende können wir das Ergebnis bewerten.

```
PRINT

IF PUNKTE=10 THEN
  PRINT "Perfekt!"
ELSEIF PUNKTE>=8 THEN
  PRINT "Sehr gut!"
ELSEIF PUNKTE>=5 THEN
  PRINT "Gut gemacht!"
```

```
ELSE
  PRINT "Weiter ueben!"
ENDIF
```

Dadurch erhält der Spieler eine direkte Rückmeldung über seine Leistung.

# 13.11. Effizient mit dem Editor arbeiten

Unser Little-Professor-Programm ist inzwischen deutlich größer geworden. Bei umfangreicheren Programmen lohnt es sich, die wichtigsten Funktionen des Editors kennenzulernen. Dadurch kannst du Änderungen schneller durchführen und behältst auch bei vielen Programmzeilen den Überblick.

## 13.11.1. Cursor bewegen

Mit den Pfeiltasten bewegst du den Cursor wie gewohnt durch den Quelltext. Die Tasten **Pos1 (Home)** und **Ende (End)** springen an den Anfang bzw. das Ende der aktuellen Zeile. Mit **Bild** und **Bild** kannst du schneller durch längere Programme blättern.

## 13.11.2. Zeilen einfügen und ändern

Du kannst den Cursor an eine beliebige Stelle setzen und dort sofort mit dem Schreiben beginnen. Bestehende Zeilen lassen sich jederzeit ändern oder ergänzen. So musst du ein Programm nicht von vorne beginnen, wenn du eine Idee verbessern möchtest.

## 13.11.3. Kopieren und Einfügen

Wenn sich Programmteile wiederholen, musst du sie nicht erneut eintippen. Markiere den gewünschten Bereich und kopiere ihn mit den üblichen Tastenkombinationen **Strg+C** und **Strg+V**. Das spart Zeit und vermeidet Tippfehler.

## 13.11.4. Suchen

In längeren Programmen kann es schwierig werden, eine bestimmte Variable oder eine IF-Anweisung wiederzufinden. Die Suchfunktion des Editors hilft dir dabei, gesuchte Begriffe schnell zu finden.

Mit diesen wenigen Funktionen lässt sich bereits sehr komfortabel arbeiten. Weitere Möglichkeiten des Editors lernst du später im Kapitel »Der eingebaute Editor« kennen.

**Mit der Maus arbeiten**

Der integrierte Editor unterstützt auch eine USB-Maus. Damit kannst du den Cursor bequem positionieren, Text markieren und durch größere Programme navigieren.

Damit die Maus verwendet werden kann, muss sie zunächst aktiviert werden:

```
OPTION MOUSE ON
```

Diese Einstellung muss nur einmal vorgenommen werden und bleibt dauerhaft gespeichert.

Natürlich funktioniert der Editor auch vollständig ohne Maus – alle Funktionen lassen sich ebenso mit der Tastatur bedienen.

## 13.12. Verbesserungsmöglichkeiten

Unser Little Professor funktioniert bereits erstaunlich gut.

Es gibt jedoch viele Möglichkeiten zur Erweiterung:

- größere Zahlenbereiche
- Subtraktionsaufgaben
- Multiplikationsaufgaben
- Division
- verschiedene Schwierigkeitsstufen
- mehrere Spieler

In den nächsten Kapiteln werden wir einige dieser Ideen umsetzen.

## 13.13. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende Zahlen bis 20.
- Verwende Zahlen bis 100.
- Stelle 20 Aufgaben.
- Vergib zwei Punkte pro richtiger Antwort.
- Ändere die Bewertung am Ende.

## 13.14. Probier’s selbst!

Versuche folgende Aufgaben:

1. Ergänze Subtraktionsaufgaben.
2. Ergänze Multiplikationsaufgaben.
3. Frage den Namen des Spielers ab.
4. Begrüße den Spieler persönlich.
5. Gib die Anzahl der falschen Antworten aus.

6. Zeige am Ende die erreichte Prozentzahl an.

## 13.15. Zusammenfassung

In diesem Kapitel hast du ein vollständiges Lernprogramm entwickelt.

Dabei wurden viele der bisher gelernten Befehle verwendet:

- PRINT
- INPUT
- IF … THEN … ELSE
- FOR … NEXT
- RND
- Variablen

Du hast gesehen, wie aus vielen kleinen Bausteinen ein nützliches Programm entsteht.

Im nächsten Kapitel werden wir den Little Professor weiter verbessern und um zusätzliche Funktionen erweitern.

## 13.16. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 13_little_professor.bas

# Kapitel 14. Little Professor Version 2

## 14.1. Das lernst du

In diesem Kapitel verbessern wir unseren Little Professor aus dem letzten Kapitel.

Dabei lernst du:

- zufällig verschiedene Aufgabentypen auszuwählen
- Addition, Subtraktion, Multiplikation und Division zu kombinieren
- SUB zur Strukturierung eines Programms einzusetzen
- Funktionen und Variablen sinnvoll zusammenzuarbeiten
- den Spieler persönlich anzusprechen

Am Ende des Kapitels besitzt du einen deutlich leistungsfähigeren Mathematiktrainer.

## 14.2. Unser Programm wächst

Der Little Professor aus dem letzten Kapitel konnte bereits Additionsaufgaben erzeugen.

Nun möchten wir weitere Rechenarten hinzufügen:

- Addition
- Subtraktion
- Multiplikation
- Division

Außerdem soll der Spieler mit seinem Namen begrüßt werden.

## 14.3. Der Spielername

Zu Beginn fragen wir den Namen des Spielers ab.

```
INPUT "Wie heisst du ";NAME$

PRINT
PRINT "Hallo ";NAME$
```

```
PRINT
```

## 14.4. Zufällig eine Rechenart auswählen

Für jede Aufgabe soll eine Rechenart zufällig ausgewählt werden.

Dazu verwenden wir eine Zufallszahl zwischen 0 und 3.

```
OP=INT(RND*4)
```

Die Werte bedeuten:

```
0 = Addition
1 = Subtraktion
2 = Multiplikation
3 = Division
```

## 14.5. Eine Aufgabe erzeugen

Damit unser Hauptprogramm übersichtlich bleibt, erzeugen wir die Aufgabe in einer SUB.

```
SUB NeueAufgabe

  OP=INT(RND*4)

END SUB
```

Später wird diese SUB alle benötigten Werte erzeugen.

## 14.6. Additionsaufgaben

```
A=INT(RND*10)
B=INT(RND*10)

ERG=A+B

PRINT A;" + ";B;" = ?"
```

## 14.7. Subtraktionsaufgaben

Bei Subtraktionsaufgaben möchten wir negative Ergebnisse vermeiden.

```
A=INT(RND*10)+10
B=INT(RND*10)

ERG=A-B

PRINT A;" - ";B;" = ?"
```

## 14.8. Multiplikationsaufgaben

```
A=INT(RND*10)
B=INT(RND*10)

ERG=A*B

PRINT A;" * ";B;" = ?"
```

## 14.9. Divisionsaufgaben

Divisionen sollen immer ganzzahlig lösbar sein.

Deshalb erzeugen wir zuerst das Ergebnis.

```
ERG=INT(RND*10)+1

B=INT(RND*10)+1

A=ERG*B

PRINT A;" / ";B;" = ?"
```

Dadurch entstehen Aufgaben wie:

```
24 / 6 = ?
```

Die Lösung ist immer eine ganze Zahl.

## 14.10. Die SUB vervollständigen

Nun fassen wir alle Aufgabentypen zusammen.

```
SUB NeueAufgabe

  OP=INT(RND*4)
```

```
  IF OP=0 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A+B

    PRINT A;" + ";B;" = ?"

  ELSEIF OP=1 THEN

    A=INT(RND*10)+10
    B=INT(RND*10)

    ERG=A-B

    PRINT A;" - ";B;" = ?"

  ELSEIF OP=2 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A*B

    PRINT A;" * ";B;" = ?"

  ELSE

    ERG=INT(RND*10)+1

    B=INT(RND*10)+1

    A=ERG*B

    PRINT A;" / ";B;" = ?"

  ENDIF

END SUB
```

## 14.11. Das komplette Programm

```
INPUT "Wie heisst du ";NAME$

PRINT
PRINT "Hallo ";NAME$
PRINT
```

```
PUNKTE=0

FOR RUNDE=1 TO 10

  PRINT
  PRINT "Aufgabe ";RUNDE

  NeueAufgabe

  INPUT ANTWORT

  IF ANTWORT=ERG THEN

    PRINT "Richtig!"

    PUNKTE=PUNKTE+1

  ELSE

    PRINT "Leider falsch!"

    PRINT "Richtig waere ";ERG

  ENDIF

NEXT RUNDE

PRINT
PRINT NAME$;", du hast ";PUNKTE;" von 10 Punkten erreicht."

PROZENT=PUNKTE*100/10

IF PROZENT=100 THEN

  PRINT "Perfekt!"

ELSEIF PROZENT>=90 THEN

  PRINT "Sehr gut!"

ELSEIF PROZENT>=80 THEN

  PRINT "Gut gemacht!"

ELSEIF PROZENT>=60 THEN

  PRINT "Das war ordentlich."

ELSE
```

```
  PRINT "Weiter ueben!"

ENDIF



SUB NeueAufgabe

  OP=INT(RND*4)

  IF OP=0 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A+B

    PRINT A;" + ";B;" = ?"

  ELSEIF OP=1 THEN

    A=INT(RND*10)+10
    B=INT(RND*10)

    ERG=A-B

    PRINT A;" - ";B;" = ?"

  ELSEIF OP=2 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A*B

    PRINT A;" * ";B;" = ?"

  ELSE

    ERG=INT(RND*10)+1

    B=INT(RND*10)+1

    A=ERG*B

    PRINT A;" / ";B;" = ?"

  ENDIF

END SUB
```

## 14.12. Beispielausgabe

```
Wie heisst du? Manfred

Hallo Manfred


Aufgabe 1
7 + 5 = ?
12
Richtig!

Aufgabe 2
24 / 6 = ?
4
Richtig!

Aufgabe 3
8 * 7 = ?
55
Leider falsch!
Richtig waere 56
```

## 14.13. Warum SUB hier sinnvoll ist

Die Aufgabe wird in jedem Durchlauf neu erzeugt.

Würden wir den gesamten Code direkt in die Schleife schreiben, würde das Hauptprogramm schnell unübersichtlich werden.

Durch die SUB bleibt der Ablauf klar erkennbar:

```
NeueAufgabe

INPUT ANTWORT

Antwort prüfen
```

## 14.14. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende Zahlen bis 20.
- Stelle 20 Aufgaben.
- Verwende nur Multiplikationsaufgaben.

- Verwende nur Divisionsaufgaben.
- Ändere die Bewertung am Ende.

## 14.15. Probier’s selbst!

Versuche folgende Aufgaben:

1. Füge eine Schwierigkeitsstufe hinzu.
2. Lasse den Spieler die Anzahl der Aufgaben wählen.
3. Vergib Bonuspunkte für besonders schnelle Lösungen.
4. Speichere die Anzahl richtiger und falscher Antworten getrennt.
5. Zeige am Ende die erreichte Prozentzahl an.

## 14.16. Zusammenfassung

In diesem Kapitel hast du den Little Professor erweitert.

Du hast gelernt:

- verschiedene Rechenarten zu kombinieren
- SUB zur Strukturierung einzusetzen
- Zufallszahlen für unterschiedliche Aufgaben zu verwenden
- den Spieler persönlich anzusprechen
- Ergebnisse zu bewerten

Das Programm ist nun deutlich leistungsfähiger und ähnelt bereits einem echten Lerncomputer.

Im nächsten Kapitel werden wir weitere Verbesserungen einbauen und erstmals Arrays verwenden, um Statistiken über die gelösten Aufgaben zu speichern.

## 14.17. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 14_little_professor_v2.bas
- 14_little_professor_v2_final.bas

# Kapitel 15. Little Professor Version 3

## 15.1. Das lernst du

In diesem Kapitel erweitern wir unseren Little Professor erneut.

Dabei lernst du:

- Arrays in einem größeren Projekt einzusetzen
- Antworten und Lösungen zu speichern
- Ergebnisse auszuwerten
- einfache Statistiken zu erstellen
- dem Spieler gezielt Rückmeldungen zu geben

Am Ende des Kapitels besitzt du einen Mathematiktrainer, der nicht nur Punkte zählt, sondern auch die Fehler des Spielers analysiert.

## 15.2. Warum noch eine Version?

Unser Little Professor kann bereits:

- verschiedene Rechenarten erzeugen
- Antworten prüfen
- Punkte zählen
- Ergebnisse bewerten

Am Ende kennen wir jedoch nur die erreichte Punktzahl.

Wir wissen nicht:

- Welche Aufgaben falsch beantwortet wurden?
- Welche Antwort der Spieler eingegeben hat?
- Bei welcher Rechenart die meisten Fehler auftraten?

Das möchten wir nun ändern.

## 15.3. Die Aufgaben speichern

Für jede Aufgabe speichern wir:

- die Aufgabe selbst
- die richtige Lösung
- die Antwort des Spielers
- die Rechenart

Dafür legen wir einige Arrays an.

```
DIM AUFGABE$(9)
DIM LOESUNG(9)
DIM EINGABE(9)
DIM TYP(9)
```

Da wir zehn Aufgaben stellen, benötigen wir die Elemente 0 bis 9.

## 15.4. Die Arrays füllen

Nach jeder Aufgabe speichern wir die Daten.

```
AUFGABE$(RUNDE-1)=TEXT$
LOESUNG(RUNDE-1)=ERG
EINGABE(RUNDE-1)=ANTWORT
TYP(RUNDE-1)=OP
```

Dadurch stehen die Informationen auch nach dem Spiel noch zur Verfügung.

## 15.5. Eine Aufgabe als Text speichern

Zusätzlich erzeugen wir einen Text, der die Aufgabe beschreibt.

Bei einer Addition:

```
TEXT$=STR$(A)+" + "+STR$(B)
```

Bei einer Multiplikation:

```
TEXT$=STR$(A)+" * "+STR$(B)
```

Dieser Text wird später für die Auswertung benötigt.

## 15.6. Das komplette Programm

```
INPUT "Wie heisst du ";NAME$

DIM AUFGABE$(9)
DIM LOESUNG(9)
DIM EINGABE(9)
DIM TYP(9)

PUNKTE=0

FOR RUNDE=1 TO 10

  NeueAufgabe

  PRINT
  PRINT "Aufgabe ";RUNDE
  PRINT TEXT$;" = ?"

  INPUT ANTWORT

  AUFGABE$(RUNDE-1)=TEXT$
  LOESUNG(RUNDE-1)=ERG
  EINGABE(RUNDE-1)=ANTWORT
  TYP(RUNDE-1)=OP

  IF ANTWORT=ERG THEN

    PRINT "Richtig!"

    PUNKTE=PUNKTE+1

  ELSE

    PRINT "Leider falsch!"

    PRINT "Richtig waere ";ERG

  ENDIF

NEXT RUNDE

PRINT
PRINT NAME$;", du hast ";PUNKTE;" von 10 Punkten erreicht."



PRINT
PRINT "Auswertung"
PRINT "-----------"
```

```
FOR I=0 TO 9

  IF EINGABE(I)=LOESUNG(I) THEN

    PRINT AUFGABE$(I);" = ";LOESUNG(I);"  OK"

  ELSE

    PRINT AUFGABE$(I);" = ";LOESUNG(I)
    PRINT "Deine Antwort: ";EINGABE(I)

  ENDIF

NEXT I



SUB NeueAufgabe

  OP=INT(RND*4)

  IF OP=0 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A+B

    TEXT$=STR$(A)+" + "+STR$(B)

  ELSEIF OP=1 THEN

    A=INT(RND*10)+10
    B=INT(RND*10)

    ERG=A-B

    TEXT$=STR$(A)+" - "+STR$(B)

  ELSEIF OP=2 THEN

    A=INT(RND*10)
    B=INT(RND*10)

    ERG=A*B

    TEXT$=STR$(A)+" * "+STR$(B)

  ELSE
```

```
    ERG=INT(RND*10)+1

    B=INT(RND*10)+1

    A=ERG*B

    TEXT$=STR$(A)+" / "+STR$(B)

  ENDIF

END SUB
```

## 15.7. Die Auswertung verstehen

Während des Spiels werden alle Antworten gespeichert.

Nach der letzten Aufgabe durchläuft eine FOR-Schleife die Arrays.

```
FOR I=0 TO 9
  ...
NEXT I
```

Dabei werden die gespeicherten Informationen ausgewertet.

## 15.8. Beispielausgabe

```
Manfred, du hast 7 von 10 Punkten erreicht.


Auswertung
-----------

7 + 5 = 12  OK

8 * 7 = 56
Deine Antwort: 55

24 / 6 = 4  OK

15 - 9 = 6
Deine Antwort: 8
```

Jetzt erkennt der Spieler sofort, welche Aufgaben falsch beantwortet wurden.

## 15.9. Fehler nach Rechenart zählen

Mit den gespeicherten Aufgabentypen können wir weitere Statistiken erstellen.

```
ADD=0
SUB=0
MUL=0
DIV=0

FOR I=0 TO 9

  IF EINGABE(I)<>LOESUNG(I) THEN

    IF TYP(I)=0 THEN ADD=ADD+1
    IF TYP(I)=1 THEN SUB=SUB+1
    IF TYP(I)=2 THEN MUL=MUL+1
    IF TYP(I)=3 THEN DIV=DIV+1

  ENDIF

NEXT I
```

Danach können wir die Statistik ausgeben:

```
PRINT
PRINT "Fehlerstatistik"

PRINT "Additionen:      ";ADD
PRINT "Subtraktionen:   ";SUB
PRINT "Multiplikation:  ";MUL
PRINT "Divisionen:      ";DIV
```

## 15.10. Warum Arrays hier wichtig sind

Ohne Arrays würden die Antworten nach jeder Aufgabe verloren gehen.

Durch die Arrays können wir:

- Aufgaben speichern
- Antworten speichern
- Lösungen speichern
- Statistiken berechnen
- Fehler analysieren

Genau für solche Aufgaben werden Arrays in der Praxis verwendet.

## 15.11. Experimentiere!

Probiere folgende Änderungen aus:

- Erhöhe die Anzahl der Aufgaben auf 20.
- Speichere zusätzlich die benötigte Zeit.
- Gib nur die falsch beantworteten Aufgaben aus.
- Zähle richtige Antworten je Rechenart.

## 15.12. Probier’s selbst!

Versuche folgende Aufgaben:

1. Speichere den Namen des Spielers.
2. Gib die erreichte Prozentzahl aus.
3. Zeige nur die falschen Antworten an.
4. Ermittle die schwierigste Rechenart.
5. Vergib eine Schulnote.

## 15.13. Zusammenfassung

In diesem Kapitel hast du Arrays in einem echten Projekt eingesetzt.

Du hast gelernt:

- Daten in Arrays zu speichern
- Informationen später auszuwerten
- Statistiken zu berechnen
- Fehler gezielt zu analysieren

Der Little Professor ist damit von einem einfachen Übungsprogramm zu einem echten Lernsystem geworden.

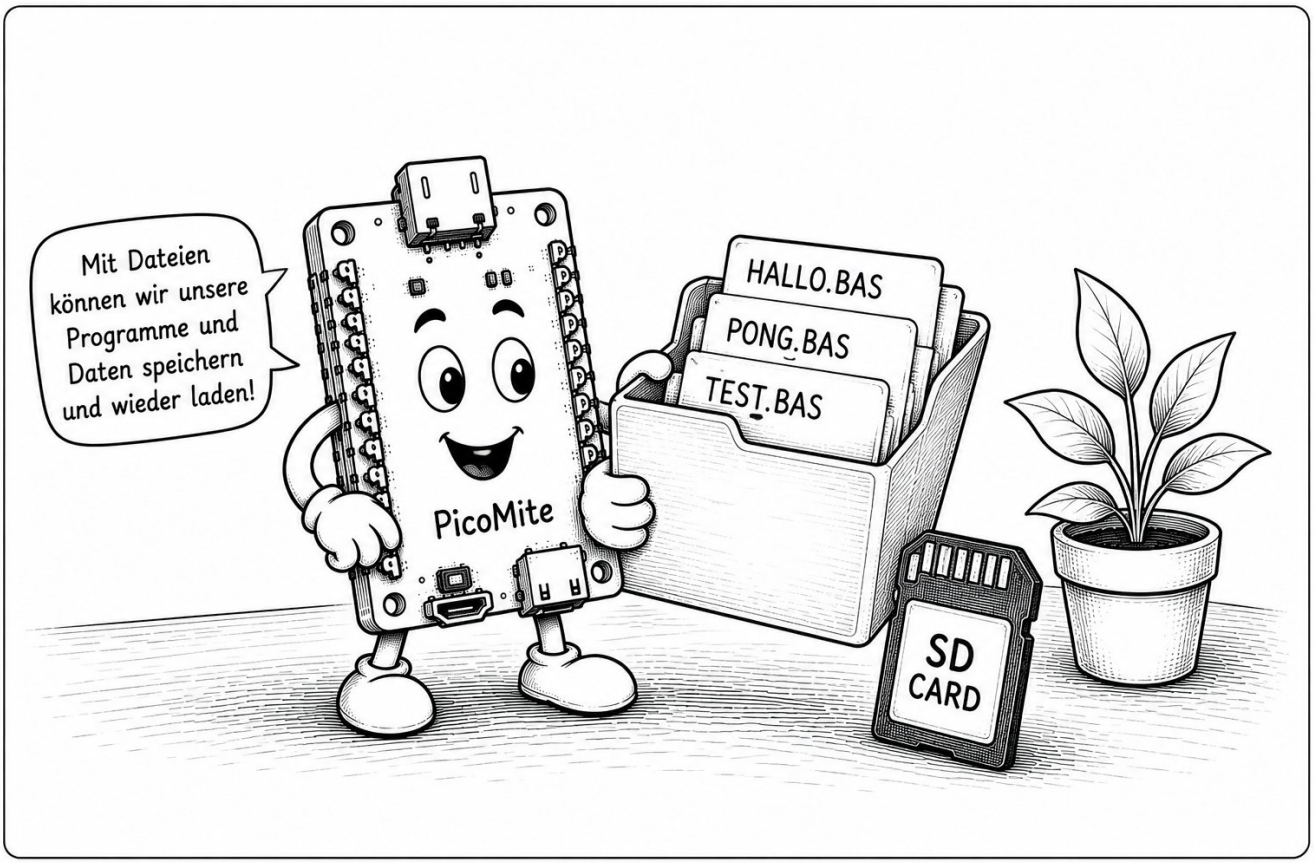
Im nächsten Kapitel werden wir lernen Dateien anzulegen, Daten in Dateien zu speichern und diese wieder auszulesen.

## 15.14. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 15_little_professor_v3.bas
- 15_little_professor_v3_final.bas

# Kapitel 16. Dateien speichern und laden




## 16.1. Das lernst du

In diesem Kapitel lernst du:

- Dateien anzulegen
- Daten in Dateien zu speichern
- Dateien wieder einzulesen
- Textdateien zu verwenden
- zu prüfen, ob eine Datei bereits existiert
- mehrere Datensätze in einer Datei zu speichern und auszuwerten
- einzelne Datensätze aus einer Datei zu entfernen
- Highscores, Einstellungen und andere Programmdaten dauerhaft zu speichern

Am Ende des Kapitels kannst du Programme schreiben, die ihre Daten dauerhaft speichern, beim nächsten Start wieder einlesen und verwalten.

## 16.2. Das Problem

Bisher gingen alle Daten verloren, sobald ein Programm beendet wurde.

Nehmen wir unseren Little Professor als Beispiel.

Ein Spieler erreicht 9 von 10 Punkten.

Nach dem nächsten Programmstart ist dieses Ergebnis vergessen.

Es wäre viel besser, wenn wir die Ergebnisse dauerhaft speichern könnten.

Genau dafür gibt es Dateien.

## 16.3. Was ist eine Datei?

Eine Datei ist ein Bereich auf einem Speichermedium, in dem Informationen dauerhaft gespeichert werden.

Typische Beispiele sind:

- Texte
- Bilder
- Programme
- Tabellen
- Spielstände

Auch unsere BASIC-Programme können Dateien lesen und schreiben.

## 16.4. Eine Datei öffnen

Bevor eine Datei verwendet werden kann, muss sie geöffnet werden.

Dies geschieht mit dem Befehl `OPEN`.

```
OPEN "test.txt" FOR OUTPUT AS #1
```

Dabei bedeutet:

| Teil | Bedeutung |
|---|---|
| test.txt | Dateiname |
| OUTPUT | Datei zum Schreiben öffnen |
| #1 | Dateinummer |

Die Dateinummer wird später verwendet, um auf die geöffnete Datei zuzugreifen.

## 16.5. Eine Datei schreiben

Mit `PRINT #` können Daten in eine Datei geschrieben werden.

```
OPEN "test.txt" FOR OUTPUT AS #1

PRINT #1,"Hallo Welt"

CLOSE #1
```

Nach dem Ausführen enthält die Datei:

```
Hallo Welt
```

## 16.6. Dateien immer schließen

Nach der Arbeit mit einer Datei sollte diese mit `CLOSE` geschlossen werden.

```
CLOSE #1
```

Dadurch werden alle Daten sicher gespeichert.

## 16.7. Eine Datei lesen

Zum Lesen wird die Datei im Modus `INPUT` geöffnet.

```
OPEN "test.txt" FOR INPUT AS #1

LINE INPUT #1,A$

PRINT A$

CLOSE #1
```

Ausgabe:

```
Hallo Welt
```

## 16.8. LINE INPUT

`LINE INPUT #` liest eine komplette Zeile aus einer Datei.

```
LINE INPUT #1,TEXT$
```

Dies ist besonders praktisch bei Textdateien.

## 16.9. Mehrere Zeilen speichern

Natürlich können auch mehrere Zeilen geschrieben werden.

```
OPEN "namen.txt" FOR OUTPUT AS #1

PRINT #1,"Manfred"
PRINT #1,"Geoff"
PRINT #1,"Peter"

CLOSE #1
```

Die Datei enthält anschließend:

```
Manfred
Geoff
Peter
```

## 16.10. Daten anhängen

Normalerweise überschreibt OUTPUT eine vorhandene Datei.

Soll der Inhalt erhalten bleiben, verwendet man `APPEND`.

```
OPEN "namen.txt" FOR APPEND AS #1

PRINT #1,"Sarah"

CLOSE #1
```

Die neue Zeile wird an das Ende der Datei angehängt.

## 16.11. Der Unterschied zwischen OUTPUT und APPEND

```
OPEN "daten.txt" FOR OUTPUT AS #1
```

öffnet eine Datei zum Schreiben und löscht einen eventuell vorhandenen Inhalt.

```
OPEN "daten.txt" FOR APPEND AS #1
```

öffnet die Datei zum Anhängen neuer Daten.

Vorhandene Daten bleiben erhalten.

## 16.12. Mit WRITE speichern

Neben `PRINT #` gibt es auch den Befehl `WRITE #`.

```
OPEN "spieler.txt" FOR OUTPUT AS #1

WRITE #1,"Manfred",9

CLOSE #1
```

Der Inhalt der Datei sieht dann so aus:

```
"Manfred",9
```

WRITE eignet sich besonders gut zum Speichern strukturierter Daten.

## 16.13. Ein Highscore speichern

Nun speichern wir den Namen eines Spielers und seine Punktzahl.

```
INPUT "Name ";NAME$

PUNKTE=8

OPEN "highscore.txt" FOR OUTPUT AS #1

WRITE #1,NAME$,PUNKTE

CLOSE #1

PRINT "Highscore gespeichert."
```

## 16.14. Einen Highscore laden

Mit `READ #` werden die zuvor mit `WRITE #` gespeicherten Werte wieder eingelesen.

```
OPEN "highscore.txt" FOR INPUT AS #1

READ #1, NAME$, PUNKTE

CLOSE #1

PRINT "Name   : "; NAME$
PRINT "Punkte : "; PUNKTE
```

Ausgabe:

```
Name   : Manfred
Punkte : 8
```

## 16.15. Dateinamen und Verzeichnisse

Dateien können auch in Unterverzeichnissen gespeichert werden.

Beispiel:

```
OPEN "daten\spieler.txt" FOR OUTPUT AS #1
```

Es können lange Dateinamen verwendet werden.

```
OPEN "mein_erster_highscore.txt" FOR OUTPUT AS #1
```

## 16.16. Wichtige Betriebsarten

MMBasic kennt verschiedene Betriebsarten beim Öffnen einer Datei.

| **Modus** | **Bedeutung** |
|---|---|
| INPUT | Datei lesen |
| OUTPUT | Datei schreiben (überschreibt vorhandene Datei) |
| APPEND | Daten an bestehende Datei anhängen |
| RANDOM | Datei lesen und schreiben mit direktem Zugriff |

Für die meisten Programme reichen INPUT, OUTPUT und APPEND völlig aus.

## 16.17. Fehlerquellen

Ein häufiger Fehler ist das Lesen einer Datei, die nicht existiert.

```
OPEN "unbekannt.txt" FOR INPUT AS #1
```

In diesem Fall erzeugt MMBasic eine Fehlermeldung.

Ebenso sollte jede geöffnete Datei wieder geschlossen werden.

# 16.18. Praxisbeispiel: Ein Geburtstagskalender

Bisher haben wir einzelne Möglichkeiten kennengelernt, Daten in Dateien zu speichern und wieder einzulesen.

Nun verbinden wir diese Techniken zu einem kleinen vollständigen Programm: einem Geburtstagskalender.

Unser Programm soll:

- neue Geburtstage speichern
- alle gespeicherten Geburtstage anzeigen
- einzelne Einträge löschen
- beim Start auf heutige Geburtstage hinweisen
- das Alter des Geburtstagskindes berechnen

Die Geburtstage sollen auch nach dem Beenden des Programms erhalten bleiben. Dafür speichern wir sie in einer Textdatei.

## 16.18.1. Wie speichern wir einen Geburtstag?

Für jeden Geburtstag benötigen wir vier Informationen:

- Name
- Tag
- Monat
- Geburtsjahr

Wir könnten jede Information in einer eigenen Datei speichern. Wesentlich praktischer ist es jedoch, alle Angaben zu einer Person in einer einzigen Zeile abzulegen.

Als Trennzeichen verwenden wir ein Semikolon:

```
Manfred;27;7;1966
Peter;12;3;1970
Sabine;5;11;1982
```

Jede Zeile bildet damit einen vollständigen Datensatz.

Unser Dateiformat lautet:

```
Name;Tag;Monat;Jahr
```

Das Semikolon benötigen wir später, um die einzelnen Bestandteile wieder voneinander zu trennen.

### 16.18.2. Die Datendatei

Die Geburtstage speichern wir in der Datei `geburtstage.txt`.

Den Dateinamen legen wir in einer String-Variablen ab:

```
DATEI$="geburtstage.txt"
```

Für das spätere Löschen eines Datensatzes benötigen wir außerdem eine temporäre Datei:

```
TEMPDATEI$="geburtstage.tmp"
```

Beim ersten Start unseres Programms existiert `geburtstage.txt` allerdings noch nicht.

Würden wir sofort versuchen, die Datei zum Lesen zu öffnen,

```
OPEN DATEI$ FOR INPUT AS #1
```

würde MMBasic einen Fehler melden.

Mit `MM.INFO(EXISTS FILE …)` können wir deshalb zunächst prüfen, ob die Datei bereits vorhanden ist:

```
IF MM.INFO(EXISTS FILE DATEI$)=FALSE THEN
  OPEN DATEI$ FOR OUTPUT AS #1
  CLOSE #1
ENDIF
```

Existiert die Datei noch nicht, wird sie mit `OPEN … FOR OUTPUT` angelegt und anschließend sofort wieder geschlossen.

Eine neu angelegte Datei ist zunächst leer. Das ist für unser Programm kein Problem. Sobald der erste Geburtstag eingetragen wird, schreiben wir den ersten Datensatz hinein.

### 16.18.3. Einen Geburtstag speichern

Zum Eintragen eines neuen Geburtstags fragen wir die benötigten Informationen mit `INPUT` ab:

```
INPUT "Name"; NAME$
INPUT "Tag"; TAG
INPUT "Monat"; MONAT
INPUT "Jahr"; JAHR
```

Anschließend öffnen wir unsere Datei mit APPEND:

```
OPEN DATEI$ FOR APPEND AS #1
```

APPEND fügt neue Daten am Ende einer bestehenden Datei hinzu. Bereits gespeicherte Geburtstage bleiben dadurch erhalten.

Die vier Werte schreiben wir durch Semikola getrennt in eine Zeile:

```
PRINT #1,NAME$;";";TAG;";";MONAT;";";JAHR
```

Danach schließen wir die Datei wieder:

```
CLOSE #1
```

Aus den Eingaben

```
Name? Manfred
Tag? 27
Monat? 7
Jahr? 1966
```

entsteht damit in der Datei:

```
Manfred;27;7;1966
```

Die vollständige Unterroutine sieht so aus:

```
SUB GeburtstagEintragen

  LOCAL NAME$
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL DUMMY$

  CLS

  PRINT "GEBURTSTAG EINTRAGEN"
  PRINT "===================="
  PRINT

  INPUT "Name"; NAME$
  INPUT "Tag"; TAG
```

```
  INPUT "Monat"; MONAT
  INPUT "Jahr"; JAHR

  OPEN DATEI$ FOR APPEND AS #1

  PRINT #1,NAME$;";";TAG;";";MONAT;";";JAHR

  CLOSE #1

  PRINT
  PRINT "Geburtstag gespeichert."
  PRINT
  INPUT "Weiter mit ENTER"; DUMMY$

END SUB
```

### 16.18.4. Einen Datensatz wieder zerlegen

Beim Einlesen erhalten wir zunächst wieder die vollständige Zeile:

```
Manfred;27;7;1966
```

Unser Programm benötigt aber die einzelnen Werte `Manfred`, `27`, `7` und `1966`.

Hier können wir die Stringfunktionen aus Kapitel 12 wiederverwenden.

Mit `INSTR()` suchen wir zunächst das erste Semikolon:

```
P1=INSTR(ZEILE$,";")
```

Mit `LEFT$()` holen wir alles links davon:

```
NAME$=LEFT$(ZEILE$,P1-1)
```

Damit enthält `NAME$`:

```
Manfred
```

Den bereits verarbeiteten Teil entfernen wir mit `MID$()`:

```
REST$=MID$(ZEILE$,P1+1)
```

In `REST$` steht nun nur noch:

```
27;7;1966
```

Dasselbe Verfahren wiederholen wir für Tag und Monat. Mit `VAL()` wandeln wir die gelesenen Zeichenketten wieder in Zahlen um.

Da wir diese Aufgabe an mehreren Stellen unseres Programms benötigen, schreiben wir dafür eine eigene Unterroutine:

```
SUB DatensatzLesen(ZEILE$,NAME$,TAG,MONAT,JAHR)

  LOCAL P1
  LOCAL P2
  LOCAL P3
  LOCAL REST$

  ' Name abtrennen
  P1=INSTR(ZEILE$,";")

  NAME$=LEFT$(ZEILE$,P1-1)
  REST$=MID$(ZEILE$,P1+1)

  ' Tag abtrennen
  P2=INSTR(REST$,";")

  TAG=VAL(LEFT$(REST$,P2-1))
  REST$=MID$(REST$,P2+1)

  ' Monat abtrennen
  P3=INSTR(REST$,";")

  MONAT=VAL(LEFT$(REST$,P3-1))
  JAHR=VAL(MID$(REST$,P3+1))

END SUB
```

Hier zeigt sich ein Vorteil von Unterprogrammen besonders deutlich: Das Zerlegen eines Datensatzes müssen wir nur einmal programmieren und können es anschließend an verschiedenen Stellen wiederverwenden.

### 16.18.5. Alle Geburtstage anzeigen

Nun können wir unsere Datei öffnen und Zeile für Zeile einlesen.

Da wir vorher nicht wissen müssen, wie viele Geburtstage gespeichert sind, verwenden wir `EOF()`.

`EOF` steht für *End Of File*, also „Ende der Datei“.

```
DO WHILE NOT EOF(#1)

  LINE INPUT #1,ZEILE$

  ...

LOOP
```

Solange das Dateiende noch nicht erreicht wurde, lesen wir mit `LINE INPUT` eine weitere Zeile ein.

Anschließend zerlegen wir den Datensatz:

```
DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR
```

Mit `FORMAT$()` sorgen wir dafür, dass Tag und Monat immer zweistellig ausgegeben werden:

```
PRINT FORMAT$(TAG,"%02g");".";
PRINT FORMAT$(MONAT,"%02g");".";
PRINT FORMAT$(JAHR,"%04g")
```

Aus

```
Manfred;27;7;1966
```

wird dadurch beispielsweise:

```
Manfred  27.07.1966
```

Zusätzlich nummerieren wir die Einträge. Diese Nummern benötigen wir später beim Löschen eines Geburtstags.

Die vollständige Unterroutine lautet:

```
SUB GeburtstageAnzeigen

  LOCAL ZEILE$
  LOCAL NAME$
  LOCAL NR
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL DUMMY$

  CLS
```

```
  PRINT "GESPEICHERTE GEBURTSTAGE"
  PRINT "========================"
  PRINT

  NR=0

  OPEN DATEI$ FOR INPUT AS #1

  DO WHILE NOT EOF(#1)

    LINE INPUT #1,ZEILE$

    NR=NR+1

    DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR

    PRINT NR;"  ";
    PRINT NAME$;"  ";
    PRINT FORMAT$(TAG,"%02g");".";
    PRINT FORMAT$(MONAT,"%02g");".";
    PRINT FORMAT$(JAHR,"%04g")

  LOOP

  CLOSE #1

  IF NR=0 THEN
    PRINT "Keine Geburtstage gespeichert."
  ENDIF

  PRINT
  INPUT "Weiter mit ENTER"; DUMMY$

END SUB
```

### 16.18.6. Wer hat heute Geburtstag?

Ein Geburtstagskalender wird besonders nützlich, wenn er uns automatisch auf heutige Geburtstage aufmerksam macht.

Das aktuelle Datum liefert uns `DATE$`.

Tag, Monat und Jahr können wir mit den bereits bekannten Stringfunktionen herausholen:

```
HEUTETAG=VAL(LEFT$(DATE$,2))
HEUTEMONAT=VAL(MID$(DATE$,4,2))
HEUTEJAHR=VAL(RIGHT$(DATE$,4))
```

Anschließend lesen wir wieder alle gespeicherten Geburtstage ein.

Für jeden Datensatz vergleichen wir Tag und Monat:

```
IF TAG=HEUTETAG AND MONAT=HEUTEMONAT THEN
  ...
ENDIF
```

Das Geburtsjahr müssen wir dabei nicht vergleichen. Stattdessen verwenden wir es, um das heutige Alter zu berechnen:

```
ALTER=HEUTEJAHR-JAHR
```

Hat beispielsweise jemand am heutigen Tag Geburtstag und wurde 1966 geboren, kann unser Programm ausgeben:

```
HEUTE HAT GEBURTSTAG
--------------------

Manfred wird heute 60 Jahre alt.

Alles Gute zum Geburtstag!
```

Die Variable `GEFUNDEN` merkt sich mit `TRUE` und `FALSE`, ob mindestens ein heutiger Geburtstag gefunden wurde.

```
SUB GeburtstageHeute

  LOCAL ZEILE$
  LOCAL NAME$
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL HEUTETAG
  LOCAL HEUTEMONAT
  LOCAL HEUTEJAHR
  LOCAL ALTER
  LOCAL GEFUNDEN

  ' Aktuelles Datum zerlegen
  HEUTETAG=VAL(LEFT$(DATE$,2))
  HEUTEMONAT=VAL(MID$(DATE$,4,2))
  HEUTEJAHR=VAL(RIGHT$(DATE$,4))

  GEFUNDEN=FALSE

  OPEN DATEI$ FOR INPUT AS #1
```

```
  DO WHILE NOT EOF(#1)

    LINE INPUT #1,ZEILE$

    DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR

    IF TAG=HEUTETAG AND MONAT=HEUTEMONAT THEN

      IF GEFUNDEN=FALSE THEN
        PRINT "HEUTE HAT GEBURTSTAG"
        PRINT "--------------------"
        GEFUNDEN=TRUE
      ENDIF

      ALTER=HEUTEJAHR-JAHR

      PRINT
      PRINT NAME$;" wird heute ";ALTER;" Jahre alt."

    ENDIF

  LOOP

  CLOSE #1

  IF GEFUNDEN=TRUE THEN
    PRINT
    PRINT "Alles Gute zum Geburtstag!"
    PRINT
  ENDIF

END SUB
```

### 16.18.7. Einen Datensatz löschen

Das Löschen eines einzelnen Datensatzes ist etwas aufwendiger.

Wir können nicht einfach sagen:

```
Lösche Zeile 3 aus geburtstage.txt
```

Stattdessen verwenden wir eine zweite, temporäre Datei.

Das Prinzip sieht so aus:

```
geburtstage.txt
       |
       | einlesen
```

```
       v
+--------------------+
| Manfred;27;7;1966   | ---> übernehmen
| Peter;12;3;1970     | ---> nicht übernehmen
| Sabine;5;11;1982    | ---> übernehmen
+--------------------+
       |
       v
geburtstage.tmp
```

Wir lesen also alle Datensätze der ursprünglichen Datei ein. Alle Einträge außer dem zu löschenden schreiben wir in `geburtstage.tmp`.

Zunächst zeigen wir dafür alle Geburtstage nummeriert an:

```
1  Manfred  27.07.1966
2  Peter    12.03.1970
3  Sabine   05.11.1982
```

Der Benutzer wählt anschließend die Nummer des zu löschenden Eintrags:

```
Welcher Eintrag soll geloescht werden? 2
```

Beim Kopieren erhöhen wir für jede gelesene Zeile `NR`:

```
NR=NR+1

IF NR<>LOESCHEN THEN
  PRINT #2,ZEILE$
ENDIF
```

Nur wenn die aktuelle Nummer nicht der ausgewählten Nummer entspricht, wird der Datensatz in die temporäre Datei geschrieben.

Anschließend schließen wir beide Dateien:

```
CLOSE #1
CLOSE #2
```

Nun enthält `geburtstage.tmp` alle Datensätze außer dem gelöschten.

Die ursprüngliche Datei kann deshalb mit `KILL` gelöscht werden:

```
KILL DATEI$
```

Danach benennen wir die temporäre Datei in `geburtstage.txt` um:

```
RENAME TEMPDATEI$ AS DATEI$
```

Damit ist der ausgewählte Datensatz verschwunden.

Dieses Verfahren wird häufig verwendet, wenn einzelne Datensätze aus einer einfachen Textdatei entfernt werden sollen: Die gewünschten Datensätze werden in eine neue Datei geschrieben und anschließend ersetzt diese die alte Datei.

Die vollständige Unterroutine sieht so aus:

```
SUB GeburtstagLoeschen

  LOCAL ZEILE$
  LOCAL NAME$
  LOCAL NR
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL LOESCHEN
  LOCAL DUMMY$

  CLS

  PRINT "GEBURTSTAG LOESCHEN"
  PRINT "=================="
  PRINT

  NR=0

  OPEN DATEI$ FOR INPUT AS #1

  DO WHILE NOT EOF(#1)

    LINE INPUT #1,ZEILE$

    NR=NR+1

    DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR

    PRINT NR;"  ";
    PRINT NAME$;"  ";
    PRINT FORMAT$(TAG,"%02g");".";
    PRINT FORMAT$(MONAT,"%02g");".";
    PRINT FORMAT$(JAHR,"%04g")

  LOOP
```

```
CLOSE #1

IF NR=0 THEN

  PRINT "Keine Geburtstage gespeichert."

ELSE

  PRINT
  INPUT "Welcher Eintrag soll geloescht werden"; LOESCHEN

  IF LOESCHEN<1 OR LOESCHEN>NR THEN

    PRINT
    PRINT "Ungueltige Nummer."

  ELSE

    OPEN DATEI$ FOR INPUT AS #1
    OPEN TEMPDATEI$ FOR OUTPUT AS #2

    NR=0

    DO WHILE NOT EOF(#1)

      LINE INPUT #1,ZEILE$

      NR=NR+1

      IF NR<>LOESCHEN THEN
        PRINT #2,ZEILE$
      ENDIF

    LOOP

    CLOSE #1
    CLOSE #2

    KILL DATEI$
    RENAME TEMPDATEI$ AS DATEI$

    PRINT
    PRINT "Eintrag geloescht."

  ENDIF

ENDIF

PRINT
INPUT "Weiter mit ENTER"; DUMMY$
```

```
END SUB
```

### 16.18.8. Das Hauptmenü

Jetzt benötigen wir nur noch ein Menü, mit dem der Benutzer die einzelnen Funktionen aufrufen kann.

Dafür verwenden wir wieder eine `DO ⋯ LOOP`-Schleife und `SELECT CASE`:

```
DO

  CLS

  PRINT "GEBURTSTAGSKALENDER"
  PRINT "==================="
  PRINT

  GeburtstageHeute

  PRINT "1 - Geburtstag eintragen"
  PRINT "2 - Geburtstage anzeigen"
  PRINT "3 - Geburtstag loeschen"
  PRINT "4 - Beenden"
  PRINT

  INPUT "Auswahl"; AUSWAHL

  SELECT CASE AUSWAHL

    CASE 1
      GeburtstagEintragen

    CASE 2
      GeburtstageAnzeigen

    CASE 3
      GeburtstagLoeschen

    CASE 4
      PRINT
      PRINT "Auf Wiedersehen!"

    CASE ELSE
      PRINT
      PRINT "Ungueltige Auswahl."
      PRINT
      INPUT "Weiter mit ENTER"; DUMMY$

  END SELECT
```

```
LOOP UNTIL AUSWAHL=4
```

Beachte den Aufruf

```
GeburtstageHeute
```

direkt nach der Überschrift. Dadurch werden aktuelle Geburtstage bei jeder Anzeige des Hauptmenüs automatisch angezeigt.

### 16.18.9. Das vollständige Programm

Nun können wir alle Teile zu unserem Geburtstagskalender zusammensetzen.

```
CONST TRUE=1
CONST FALSE=0

DATEI$="geburtstage.txt"
TEMPDATEI$="geburtstage.tmp"

' Datendatei beim ersten Programmstart anlegen
IF MM.INFO(EXISTS FILE DATEI$)=FALSE THEN
  OPEN DATEI$ FOR OUTPUT AS #1
  CLOSE #1
ENDIF

DO

  CLS

  PRINT "GEBURTSTAGSKALENDER"
  PRINT "==================="
  PRINT

  GeburtstageHeute

  PRINT "1 - Geburtstag eintragen"
  PRINT "2 - Geburtstage anzeigen"
  PRINT "3 - Geburtstag loeschen"
  PRINT "4 - Beenden"
  PRINT

  INPUT "Auswahl"; AUSWAHL

  SELECT CASE AUSWAHL

    CASE 1
      GeburtstagEintragen
```

```
    CASE 2
      GeburtstageAnzeigen

    CASE 3
      GeburtstagLoeschen

    CASE 4
      PRINT
      PRINT "Auf Wiedersehen!"

    CASE ELSE
      PRINT
      PRINT "Ungueltige Auswahl."
      PRINT
      INPUT "Weiter mit ENTER"; DUMMY$

  END SELECT

LOOP UNTIL AUSWAHL=4

END


' ------------------------------------------------------------------
' Neuen Geburtstag eintragen
' ------------------------------------------------------------------

SUB GeburtstagEintragen

  LOCAL NAME$
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL DUMMY$

  CLS

  PRINT "GEBURTSTAG EINTRAGEN"
  PRINT "===================="
  PRINT

  INPUT "Name"; NAME$
  INPUT "Tag"; TAG
  INPUT "Monat"; MONAT
  INPUT "Jahr"; JAHR

  OPEN DATEI$ FOR APPEND AS #1

  PRINT #1,NAME$;";";TAG;";";MONAT;";";JAHR

  CLOSE #1
```

```
  PRINT
  PRINT "Geburtstag gespeichert."
  PRINT
  INPUT "Weiter mit ENTER"; DUMMY$

END SUB


' ------------------------------------------------------------------
' Alle Geburtstage anzeigen
' ------------------------------------------------------------------

SUB GeburtstageAnzeigen

  LOCAL ZEILE$
  LOCAL NAME$
  LOCAL NR
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL DUMMY$

  CLS

  PRINT "GESPEICHERTE GEBURTSTAGE"
  PRINT "========================"
  PRINT

  NR=0

  OPEN DATEI$ FOR INPUT AS #1

  DO WHILE NOT EOF(#1)

    LINE INPUT #1,ZEILE$

    NR=NR+1

    DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR

    PRINT NR;"  ";
    PRINT NAME$;"  ";
    PRINT FORMAT$(TAG,"%02g");".";
    PRINT FORMAT$(MONAT,"%02g");".";
    PRINT FORMAT$(JAHR,"%04g")

  LOOP

  CLOSE #1
```

```
  IF NR=0 THEN
    PRINT "Keine Geburtstage gespeichert."
  ENDIF

  PRINT
  INPUT "Weiter mit ENTER"; DUMMY$

END SUB


' ------------------------------------------------------------------
' Einen Geburtstag loeschen
' ------------------------------------------------------------------

SUB GeburtstagLoeschen

  LOCAL ZEILE$
  LOCAL NAME$
  LOCAL NR
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL LOESCHEN
  LOCAL DUMMY$

  CLS

  PRINT "GEBURTSTAG LOESCHEN"
  PRINT "=================="
  PRINT

  NR=0

  OPEN DATEI$ FOR INPUT AS #1

  DO WHILE NOT EOF(#1)

    LINE INPUT #1,ZEILE$

    NR=NR+1

    DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR

    PRINT NR;"  ";
    PRINT NAME$;"  ";
    PRINT FORMAT$(TAG,"%02g");".";
    PRINT FORMAT$(MONAT,"%02g");".";
    PRINT FORMAT$(JAHR,"%04g")

  LOOP
```

```
CLOSE #1

IF NR=0 THEN

  PRINT "Keine Geburtstage gespeichert."

ELSE

  PRINT
  INPUT "Welcher Eintrag soll geloescht werden"; LOESCHEN

  IF LOESCHEN<1 OR LOESCHEN>NR THEN

    PRINT
    PRINT "Ungueltige Nummer."

  ELSE

    OPEN DATEI$ FOR INPUT AS #1
    OPEN TEMPDATEI$ FOR OUTPUT AS #2

    NR=0

    DO WHILE NOT EOF(#1)

      LINE INPUT #1,ZEILE$

      NR=NR+1

      IF NR<>LOESCHEN THEN
        PRINT #2,ZEILE$
      ENDIF

    LOOP

    CLOSE #1
    CLOSE #2

    KILL DATEI$
    RENAME TEMPDATEI$ AS DATEI$

    PRINT
    PRINT "Eintrag geloescht."

  ENDIF

ENDIF

PRINT
INPUT "Weiter mit ENTER"; DUMMY$
```

```
END SUB


' ------------------------------------------------------------------
' Heutige Geburtstage anzeigen
' ------------------------------------------------------------------

SUB GeburtstageHeute

  LOCAL ZEILE$
  LOCAL NAME$
  LOCAL TAG
  LOCAL MONAT
  LOCAL JAHR
  LOCAL HEUTETAG
  LOCAL HEUTEMONAT
  LOCAL HEUTEJAHR
  LOCAL ALTER
  LOCAL GEFUNDEN

  ' Aktuelles Datum zerlegen
  HEUTETAG=VAL(LEFT$(DATE$,2))
  HEUTEMONAT=VAL(MID$(DATE$,4,2))
  HEUTEJAHR=VAL(RIGHT$(DATE$,4))

  GEFUNDEN=FALSE

  OPEN DATEI$ FOR INPUT AS #1

  DO WHILE NOT EOF(#1)

    LINE INPUT #1,ZEILE$

    DatensatzLesen ZEILE$,NAME$,TAG,MONAT,JAHR

    IF TAG=HEUTETAG AND MONAT=HEUTEMONAT THEN

      IF GEFUNDEN=FALSE THEN
        PRINT "HEUTE HAT GEBURTSTAG"
        PRINT "--------------------"
        GEFUNDEN=TRUE
      ENDIF

      ALTER=HEUTEJAHR-JAHR

      PRINT
      PRINT NAME$;" wird heute ";ALTER;" Jahre alt."

    ENDIF

  LOOP
```

```
  CLOSE #1

  IF GEFUNDEN=TRUE THEN
    PRINT
    PRINT "Alles Gute zum Geburtstag!"
    PRINT
  ENDIF

END SUB


' ------------------------------------------------------------------
' Einen Datensatz zerlegen
'
' Format:
' Name;Tag;Monat;Jahr
'
' Beispiel:
' Manfred;27;7;1966
' ------------------------------------------------------------------

SUB DatensatzLesen(ZEILE$,NAME$,TAG,MONAT,JAHR)

  LOCAL P1
  LOCAL P2
  LOCAL P3
  LOCAL REST$

  ' Name abtrennen
  P1=INSTR(ZEILE$,";")

  NAME$=LEFT$(ZEILE$,P1-1)
  REST$=MID$(ZEILE$,P1+1)

  ' Tag abtrennen
  P2=INSTR(REST$,";")

  TAG=VAL(LEFT$(REST$,P2-1))
  REST$=MID$(REST$,P2+1)

  ' Monat abtrennen
  P3=INSTR(REST$,";")

  MONAT=VAL(LEFT$(REST$,P3-1))
  JAHR=VAL(MID$(REST$,P3+1))

END SUB
```

## 16.19. Zusammenfassung

In diesem Programm arbeiten viele der bisher kennengelernten Techniken zusammen:

- `OPEN`, `CLOSE`, `PRINT #` und `LINE INPUT #` lesen und schreiben Dateien.
- `EOF()` erkennt das Ende einer Datei.
- `MM.INFO(EXISTS FILE …)` prüft, ob unsere Datendatei vorhanden ist.
- `KILL` und `RENAME` helfen beim Löschen eines Datensatzes.
- `INSTR()`, `LEFT$()`, `MID$()` und `VAL()` zerlegen die gespeicherten Daten.
- `DATE$` liefert das aktuelle Datum.
- `SUB` teilt das Programm in überschaubare Aufgaben auf.
- `LOCAL` hält die Variablen der Unterprogramme voneinander getrennt.
- `DO … LOOP`, `IF` und `SELECT CASE` steuern den Programmablauf.

Aus den einzelnen Befehlen zur Dateiverarbeitung ist damit ein kleines Programm entstanden, das seine Daten dauerhaft speichern und beim nächsten Start wiederverwenden kann.

## 16.20. Experimentiere!

Probiere folgende Änderungen aus:

- Speichere mehrere Namen in einer Datei.
- Speichere mehrere Punktzahlen.
- Verwende APPEND statt OUTPUT.
- Ändere den Dateinamen.

## 16.21. Probier’s selbst!

Versuche folgende Aufgaben:

1. Speichere deinen Namen in einer Datei.
2. Speichere dein Alter in einer Datei.
3. Lies die Datei wieder ein.
4. Erstelle eine kleine Highscore-Datei.
5. Speichere mehrere Spieler untereinander.

In diesem Kapitel hast du gelernt, mit Dateien zu arbeiten und Daten dauerhaft zu speichern.

Du kennst nun unter anderem:

- `OPEN`
- `CLOSE`

- `PRINT #`
- `WRITE #`
- `READ #`
- `LINE INPUT #`
- `EOF()`
- `MM.INFO(EXISTS FILE …)`
- `KILL`
- `RENAME`

Du kannst Textdateien anlegen, Daten hineinschreiben und wieder einlesen. Außerdem hast du gesehen, wie mehrere Datensätze in einer Datei gespeichert, wieder zerlegt und ausgewertet werden können.

Mit dem Geburtstagskalender hast du schließlich ein kleines Programm erstellt, das seine Daten dauerhaft speichert, beim nächsten Start wieder einliest und einzelne Einträge verwalten und löschen kann.

Damit können deine Programme Informationen nicht nur über das Programmende hinaus behalten, sondern diese Daten auch gezielt verwalten.

Im nächsten Kapitel werden wir die grafischen Möglichkeiten des PicoMite kennenlernen und erste Zeichnungen auf dem Bildschirm erzeugen.

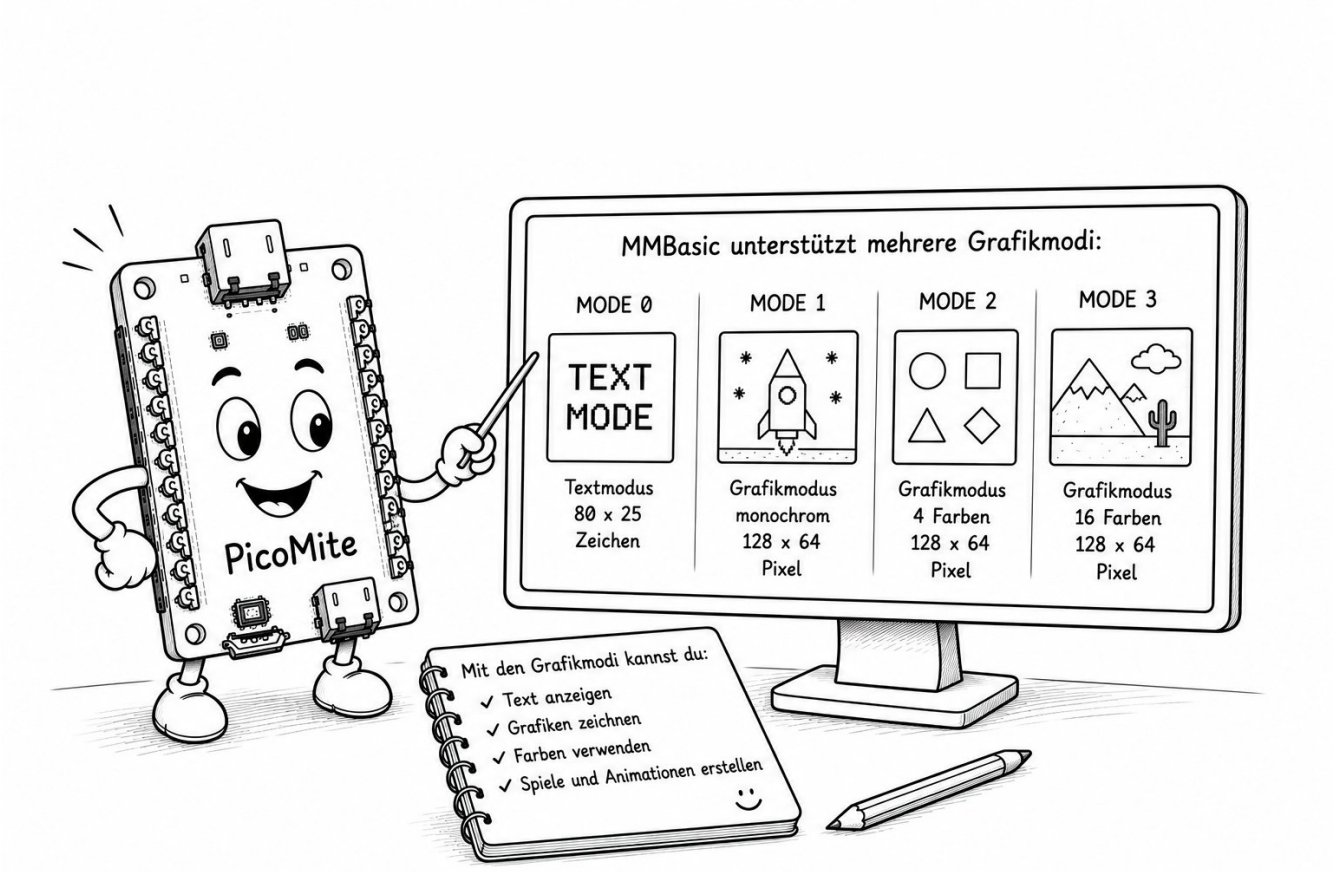
## 16.22. Programmbeispiele

Die vollständigen BASIC-Programme findest du hier:

- 16_highscore_speichern.bas
- 16_geburtstagskalender.bas

# Teil III – Grafik und Spiele

# Kapitel 17. Die verschiedenen Grafikmodi




## 17.1. Das lernst du

In diesem Kapitel lernst du:

- wie MMBasic Grafiken auf dem Bildschirm darstellt
- was Pixel sind
- wie Bildschirmkoordinaten funktionieren
- wie die Bildschirmgröße ermittelt werden kann
- wie Texte mit `PRINT @()` pixelgenau positioniert werden können
- wie Textpositionen abhängig von Bildschirm- und Textgröße berechnet werden
- wie der Grafikmodus festgelegt wird
- welche Informationen MMBasic über den Bildschirm bereitstellt

Am Ende des Kapitels verstehst du die wichtigsten Grundlagen für die Grafikprogrammierung und kannst Bildschirmausgaben unabhängig von festen Koordinaten positionieren.

## 17.2. Eine neue Welt

Bisher haben wir unsere Programme hauptsächlich mit Texten erstellt.

Beispielsweise:

```
PRINT "Hallo Welt"
```

oder:

```
INPUT NAME$
```

Der Bildschirm wurde dabei wie ein klassisches Terminal verwendet.

MMBasic kann jedoch deutlich mehr.

Mit wenigen Befehlen lassen sich:

- Linien zeichnen
- Kreise darstellen
- Texte frei positionieren
- Farben verwenden
- grafische Benutzeroberflächen erstellen

Bevor wir damit beginnen, sollten wir einige Grundlagen kennenlernen.

## 17.3. Was ist ein Pixel?

Jeder Bildschirm besteht aus vielen einzelnen Bildpunkten.

Diese Bildpunkte nennt man Pixel.

Jeder Pixel besitzt eine Position auf dem Bildschirm.

Die Position wird durch zwei Werte beschrieben:

- X = horizontale Position
- Y = vertikale Position

## 17.4. Das Koordinatensystem

Das Koordinatensystem von MMBasic hat seinen Ursprung in der linken oberen Ecke.

Die Position links oben besitzt die Koordinaten:

```
X = 0
Y = 0
```

Die X-Koordinate wächst nach rechts.

Die Y-Koordinate wächst nach unten.

```
(0,0)
  +--------------------------->
  |
  |
  |
  |
  v
```

Wenn später eine Linie oder ein Kreis gezeichnet wird, erfolgt dies immer anhand dieser Koordinaten.

## 17.5. Die Bildschirmgröße

Je nach verwendetem PicoMite-System stehen unterschiedliche Bildschirmauflösungen zur Verfügung. Bei PicoMite-Systemen mit HDMI- oder VGA-Ausgabe wird die gewünschte Auflösung mit dem Befehl `OPTION RESOLUTION` festgelegt.

```
OPTION RESOLUTION 800
```

Je nach Hardware werden unter anderem folgende Bildschirmauflösungen unterstützt:

| **Auflösung** | **Bemerkung** |
|---|---|
| 640 x 480 | Standardauflösung für HDMI und VGA |
| 720 x 400 | Alternative Bildschirmauflösung |
| 800 x 600 | Nur auf RP2350-Systemen verfügbar |
| 848 x 480 | Nur auf RP2350-Systemen verfügbar |
| 1024 x 768 | Nur HDMI |
| 1280 x 720 | Nur HDMI (HD-Auflösung) |

Eine höhere Bildschirmauflösung bietet mehr Platz für Grafiken und Texte. Gleichzeitig steigt jedoch auch der Speicherbedarf der Grafikausgabe. Dadurch stehen einem MMBasic-Programm weniger Programm- und Variablenspeicher zur Verfügung.

Welche Bildschirmauflösung aktuell am System eingestellt ist, kann anhand zweier Variablen ermittelt werden, die MMBasic bereitstellt: `MM.HRES` und `MM.VRES`.

```
PRINT MM.HRES
PRINT MM.VRES
```

Beispiel:

```
640
480
```

Dies bedeutet:

- 640 Pixel Breite
- 480 Pixel Höhe

Die Variablen haben folgende Bedeutung:

| Variable | Beschreibung |
|---|---|
| MM.HRES | Horizontale Auflösung (Breite) |
| MM.VRES | Vertikale Auflösung (Höhe) |

Die Bildschirmauflösung bestimmt also die Größe des Zeichenbereichs. Wie dieser Zeichenbereich anschließend von MMBasic genutzt wird, legen wir im nächsten Abschnitt mit dem Befehl `MODE` fest.

`OPTION RESOLUTION` und `MODE` erfüllen unterschiedliche Aufgaben.

Mit `OPTION RESOLUTION` wird die grundlegende Ausgabeauflösung des VGA- oder HDMI-Systems festgelegt.

Mit `MODE` wählen wir anschließend aus, wie MMBasic diese Bildschirmausgabe für Grafik und Farben verwendet.

Deshalb kann sich die mit `MM.HRES` und `MM.VRES` gemeldete Größe des Zeichenbereichs beim Wechsel des Grafikmodus ändern.

## 17.6. Der Grafikmodus

Bevor wir Grafiken zeichnen können, muss zunächst ein geeigneter Grafikmodus ausgewählt werden.

Dies geschieht mit dem Befehl `MODE`.

```
MODE 2
```

Der Grafikmodus legt fest, wie der Bildschirm verwendet wird. Dazu gehören unter anderem:

- die Bildschirmauflösung
- die Anzahl der darstellbaren Farben
- die verfügbare Bildschirmfläche
- die Größe der Schrift

Welche Grafikmodi verfügbar sind, hängt von der verwendeten Hardware ab. Ein PicoMite mit

HDMI-Ausgabe bietet beispielsweise andere Grafikmodi als ein PicoMite mit VGA-Ausgabe oder ein Colour Maximite 2.

Die meisten Grafikprogramme in diesem Tutorial verwenden `MODE 2`. Dieser Modus eignet sich besonders gut für die gezeigten Beispiele und steht auf den meisten PicoMite-Systemen zur Verfügung.

Die folgende Tabelle gibt einen Überblick über typische Grafikmodi. Nicht jeder Modus steht mit jeder PicoMite-Firmware, Ausgabeart und eingestellten Bildschirmauflösung zur Verfügung.

| MODE | Auflösung | Farben | Verwendung |
|---|---|---|---|
| 1 | 640 x 480 | 2 | Monochrome Darstellung, Standardmodus nach dem Einschalten |
| 2 | 320 x 240 | 16 | Einfacher Farbmodus, ideal für erste Grafikprogramme |
| 3 | 640 x 480 | 16 | Höhere Auflösung bei 16 Farben |
| 4 | 320 x 240 | 32.768 | RGB555-Farbmodus für hochwertige Farbgrafiken |
| 5 | 320 x 240 | 256 | 256 Farben mit frei belegbarer Farbpalette |

## 17.7. Bildschirminformationen anzeigen

Mit dem folgenden Programm kann die aktuelle Bildschirmgröße beim Grafikmodus MODE 2 ausgegeben werden.

```
MODE 2

PRINT "Bildschirminformationen"
PRINT

PRINT "Breite : ";MM.HRES
PRINT "Hoehe  : ";MM.VRES
```

## 17.8. Schriftarten

MMBasic besitzt mehrere eingebaute Schriftarten.

Je nach Schriftart unterscheiden sich Breite und Höhe der Zeichen.

Die aktuelle Schriftgröße kann mit `MM.INFO(FONTWIDTH)` und `MM.INFO(FONTHEIGHT)` abgefragt werden.

```
PRINT MM.INFO(FONTWIDTH)
PRINT MM.INFO(FONTHEIGHT)
```

Beispiel:

```
8
12
```

Das bedeutet:

- 8 Pixel breit
- 12 Pixel hoch

## 17.9. Warum ist das wichtig?

Wenn Texte an einer bestimmten Position ausgegeben werden sollen, muss bekannt sein, wie groß die verwendete Schrift ist.

Dadurch können Texte exakt positioniert werden.

## 17.10. Text pixelgenau positionieren

Bisher haben wir `PRINT` verwendet, um Texte an der aktuellen Schreibposition auszugeben. MMBasic bietet jedoch auch die Möglichkeit, die Ausgabe von `PRINT` direkt an einer bestimmten Pixelposition zu beginnen.

Dazu wird hinter `PRINT` die Funktion `@()` angegeben:

```
PRINT @(x,y) Ausdruck
```

`x` und `y` sind dabei die Pixelkoordinaten auf dem Bildschirm.

Ein Beispiel:

```
MODE 1
CLS

PRINT @(50,50) "Hallo MMBasic!"
PRINT @(100,100) "X=100, Y=100"
PRINT @(200,200) "X=200, Y=200"
```

Die Texte beginnen jeweils an den angegebenen Pixelkoordinaten. Damit können wir das zuvor kennengelernte Koordinatensystem auch für die Textausgabe verwenden.

Wird die Y-Koordinate weggelassen, ändert `@()` nur die horizontale Position. Die aktuelle Zeile bleibt erhalten.

```
PRINT @(100) "Dieser Text beginnt bei X=100"
```

Die Funktion `@()` kann innerhalb einer `PRINT`-Anweisung auch mehrfach verwendet werden.

```
PRINT @(50,50) "Hallo" @(50,80) "MMBasic"
```

Dadurch lassen sich mehrere Texte mit einer einzigen `PRINT`-Anweisung an unterschiedlichen Positionen ausgeben.

### 17.10.1. Verschiedene Ausgabemodi

`@()` kann zusätzlich einen dritten Parameter erhalten:

```
PRINT @(x,y,m) Ausdruck
```

Der Wert `m` bestimmt, wie der Text ausgegeben wird.

| m | Ausgabe |
|---|---|
| 0 | Normale Textausgabe mit Hintergrund |
| 1 | Der Hintergrund wird nicht gezeichnet und bleibt transparent |
| 2 | Invertierte Textausgabe |
| 5 | Die vorhandenen Pixel werden invertiert, der Hintergrund bleibt transparent |

Beispiel:

```
PRINT @(50,50,0) "Normal"
PRINT @(50,80,1) "Transparent"
PRINT @(50,110,2) "Invertiert"
PRINT @(50,140,5) "Pixel invertieren"
```

`PRINT @()` funktioniert sowohl auf der Terminal-Konsole eines angeschlossenen Computers als auch bei VGA- oder HDMI-Ausgabe. Bei einem LCD-Display kann es verwendet werden, wenn das Display mit `OPTION LCDPANEL CONSOLE` als Konsole eingerichtet wurde.

Später lernen wir mit `TEXT` noch einen speziellen Grafikbefehl für Texte kennen. Dieser bietet zusätzliche Möglichkeiten wie Ausrichtung, Skalierung, Farbe und Drehung. Für eine einfache pixelgenaue Textausgabe ist `PRINT @()` jedoch sehr praktisch.

## 17.11. Verschiedene Ausgabegeräte

MMBasic unterstützt verschiedene Grafiksysteme.

Dazu gehören:

- LCD-Displays

- VGA-Monitore
- HDMI-Monitore

Je nach verwendeter Hardware unterscheiden sich:

- Auflösung
- Farbtiefe
- verfügbare Grafikmodi

Die Grafikbefehle selbst funktionieren jedoch weitgehend identisch.

## 17.12. Farben

Farben werden in MMBasic mit der Funktion `RGB()` beschrieben.

Beispiele:

```
RGB(RED)
RGB(BLUE)
RGB(GREEN)
RGB(YELLOW)
```

Oder mit numerischen Farbwerten:

```
RGB(255,0,0)
RGB(0,255,0)
RGB(0,0,255)
```

Die genaue Verwendung von Farben werden wir im nächsten Kapitel kennenlernen.

## 17.13. Ein erstes Informationsprogramm

Das folgende Programm zeigt einige wichtige Informationen über den aktuell verwendeten Bildschirm an.

```
MODE 2
PRINT "Grafiksystem"
PRINT "------------"
PRINT
PRINT "Breite      : ";MM.HRES
PRINT "Hoehe       : ";MM.VRES
PRINT
PRINT "Fontbreite  : ";MM.INFO(FONTWIDTH)
PRINT "Fonthoehe   : ";MM.INFO(FONTHEIGHT)
```

Eine mögliche Ausgabe könnte so aussehen:

```
Grafiksystem
------------

Breite      : 320
Hoehe       : 240

Fontbreite  : 6
Fonthoehe   : 8
```

# 17.14. Praxisbeispiel: Text automatisch zentrieren

Wir kennen nun die Größe unseres Grafikbereichs und wissen, wie Texte mit `PRINT @(x,y)` an einer bestimmten Bildschirmposition ausgegeben werden.

Damit können wir eine kleine Aufgabe lösen, die in vielen Programmen benötigt wird: Wir möchten einen Text genau in der Mitte des Bildschirms ausgeben.

Eine feste Position wie

```
PRINT @(200,100) "MMBasic"
```

funktioniert zwar, ist aber nur für eine bestimmte Bildschirmauflösung und einen bestimmten Text geeignet.

Ändert sich die Auflösung oder die Länge des Textes, stimmt die Position nicht mehr.

Besser ist es, die Position vom Programm berechnen zu lassen.

## 17.14.1. Die Bildschirmmitte berechnen

Die Breite des aktuellen Grafikbereichs erhalten wir mit `MM.HRES`, die Höhe mit `MM.VRES`.

Die Mitte des Bildschirms liegt deshalb zunächst bei:

```
X=MM.HRES\2
Y=MM.VRES\2
```

Wenn wir den Text einfach an dieser Position ausgeben,

```
PRINT @(X,Y) "MMBasic"
```

beginnt der Text allerdings erst in der Bildschirmmitte. Er ist also nicht wirklich zentriert.

Das liegt daran, dass `PRINT @(x,y)` die Position der linken oberen Ecke des Textes angibt.

### 17.14.2. Die Größe des Textes berücksichtigen

Um den Text wirklich zu zentrieren, müssen wir seine Breite kennen.

Mit

```
MM.INFO(FONTWIDTH)
```

erhalten wir die Breite eines Zeichens des aktuell verwendeten Fonts in Pixeln.

Die Anzahl der Zeichen eines Textes kennen wir bereits aus Kapitel 12: `LEN()` liefert die Länge einer Zeichenkette.

Damit können wir die Breite unseres Textes berechnen:

```
BREITE=LEN(TEXT$)*MM.INFO(FONTWIDTH)
```

Für die horizontale Position ziehen wir diese Breite zunächst von der Bildschirmbreite ab und teilen das Ergebnis durch zwei:

```
X=(MM.HRES-BREITE)\2
```

Für die vertikale Position benötigen wir entsprechend die Höhe des aktuellen Fonts:

```
Y=(MM.VRES-MM.INFO(FONTHEIGHT))\2
```

Nun können wir den Text an der berechneten Position ausgeben:

```
PRINT @(X,Y) TEXT$
```

Der Text erscheint damit unabhängig von seiner Länge genau in der Mitte des Grafikbereichs.

### 17.14.3. Das vollständige Programm

Unser kleines Beispiel benötigt nur wenige Zeilen:

```
CLS

TEXT$="Mein erstes MMBasic Programm"

BREITE=LEN(TEXT$)*MM.INFO(FONTWIDTH)

X=(MM.HRES-BREITE)\2
Y=(MM.VRES-MM.INFO(FONTHEIGHT))\2
```

```
PRINT @(X,Y) TEXT$
```

Ändere nun den Inhalt von `TEXT$`:

```
TEXT$="MMBasic"
```

oder beispielsweise:

```
TEXT$="Hallo PicoMite!"
```

Obwohl die Texte unterschiedlich lang sind, berechnet das Programm jedes Mal eine neue Position und gibt den Text wieder zentriert aus.

Versuche nicht, die passenden Koordinaten durch Ausprobieren zu finden.

Wenn die benötigte Position aus bekannten Größen berechnet werden kann, ist das Programm flexibler und funktioniert auch mit anderen Texten oder Bildschirmgrößen.

### 17.14.4. Was haben wir dabei verwendet?

Dieses kleine Beispiel verbindet gleich mehrere Dinge, die wir bereits kennengelernt haben:

- `MM.HRES` liefert die Breite des Grafikbereichs.
- `MM.VRES` liefert die Höhe des Grafikbereichs.
- `MM.INFO(FONTWIDTH)` liefert die Breite eines Zeichens.
- `MM.INFO(FONTHEIGHT)` liefert die Höhe eines Zeichens.
- `LEN()` bestimmt die Anzahl der Zeichen unseres Textes.
- `PRINT @(x,y)` gibt den Text an der berechneten Position aus.

Damit haben wir zum ersten Mal die Eigenschaften des aktuellen Grafikmodus verwendet, um die Position einer Ausgabe automatisch zu berechnen.

Im nächsten Schritt können wir dasselbe Prinzip auf echte Grafikelemente anwenden: Punkte, Linien, Rechtecke und andere Formen lassen sich ebenfalls über ihre Koordinaten gezielt auf dem Bildschirm positionieren.

## 17.15. Experimentiere!

Probiere folgende Änderungen aus:

- Führe das Programm auf verschiedenen Geräten aus.
- Vergleiche HDMI und VGA.
- Ändere die Schriftart und beobachte die Werte.

- Notiere die Bildschirmgröße deines Systems.
- Positioniere einen Text genau in der Bildschirmmitte.
- Gib Texte in allen vier Bildschirmecken aus.
- Ändere den Grafikmodus und beobachte, wie sich `MM.HRES` und `MM.VRES` verändern.
- Berechne die Position so, dass ein Text unabhängig von seiner Länge zentriert erscheint.

## 17.16. Probier’s selbst!

Versuche folgende Aufgaben:

1. Gib die Bildschirmbreite aus.
2. Gib die Bildschirmhöhe aus.
3. Gib die Fontbreite aus.
4. Gib die Fonthöhe aus.
5. Berechne die Bildschirmmitte.
6. Gib mit `PRINT @()` einen Text ungefähr in der Bildschirmmitte aus.

## 17.17. Zusammenfassung

In diesem Kapitel hast du die Grundlagen der Grafikprogrammierung kennengelernt.

Du weißt nun:

- was Pixel sind
- wie das Koordinatensystem funktioniert
- wie die Bildschirmgröße bestimmt wird
- wie Schriftgrößen abgefragt werden können
- wie Texte mit `PRINT @()` pixelgenau positioniert werden können
- wie Textpositionen aus Bildschirmgröße, Schriftgröße und Textlänge berechnet werden
- wie der Grafikmodus festgelegt wird
- wie Farben beschrieben werden

Im Praxisbeispiel hast du diese Informationen miteinander kombiniert und einen Text unabhängig von seiner Länge automatisch in der Bildschirmmitte positioniert.

Damit kennst du die wichtigsten Grundlagen, um grafische Ausgaben nicht nur an festen Koordinaten, sondern abhängig von der aktuellen Bildschirmgröße zu positionieren.

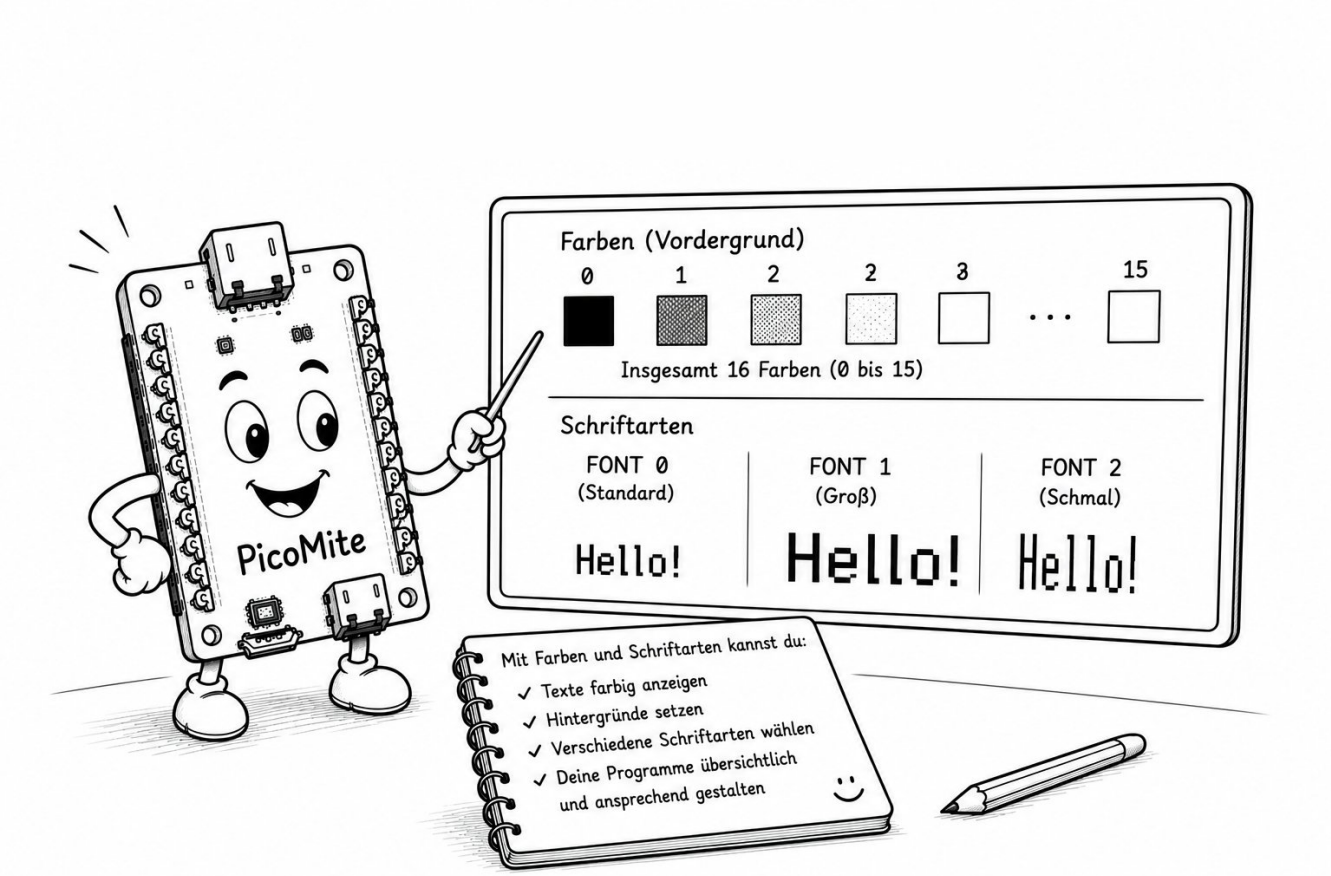
Im nächsten Kapitel beschäftigen wir uns ausführlicher mit Farben und lernen außerdem die verschiedenen Schriftarten von MMBasic kennen.

## 17.18. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 17_grafikmodi.bas

# Kapitel 18. Farben und Schriftarten




## 18.1. Das lernst du

In diesem Kapitel lernst du:

- Farben in MMBasic zu verwenden
- die Funktion `RGB()` einzusetzen
- Vorder- und Hintergrundfarben festzulegen
- verschiedene Schriftarten auszuwählen
- Schriftgrößen zu verändern
- die Breite und Höhe der aktuellen Schrift zu ermitteln
- Texte abhängig von Bildschirm- und Schriftgröße zu positionieren
- farbige Ausgaben übersichtlich in mehreren Bereichen des Bildschirms anzuordnen

Am Ende des Kapitels kannst du farbige Bildschirmausgaben gestalten, verschiedene Schriftarten verwenden und Texte passend zur aktuellen Bildschirm- und Schriftgröße positionieren.

## 18.2. Farben in MMBasic

MMBasic verwendet ein modernes Farbsystem mit 24 Bit Farbtiefe.

Jede Farbe setzt sich aus den Anteilen für:

- Rot
- Grün
- Blau

zusammen.

Daher stammt auch die Bezeichnung RGB.

## 18.3. Farben mit RGB() erzeugen

Die Funktion RGB() erwartet drei Werte zwischen 0 und 255.

```
RGB(Rot, Gruen, Blau)
```

Beispiele:

```
RGB(255,0,0)
RGB(0,255,0)
RGB(0,0,255)
```

erzeugen:

- Rot
- Grün
- Blau

## 18.4. Farbkonstanten verwenden

Häufig verwendete Farben können direkt über Farbkonstanten angegeben werden.

```
RGB(RED)
RGB(GREEN)
RGB(BLUE)
RGB(YELLOW)
RGB(CYAN)
RGB(MAGENTA)
RGB(WHITE)
RGB(BLACK)
```

Dadurch werden Programme leichter lesbar.

## 18.5. Vorder- und Hintergrundfarbe

Mit dem Befehl `COLOUR` werden die Standardfarben festgelegt.

```
COLOUR RGB(YELLOW), RGB(BLUE)
```

Dabei bedeutet:

- Gelb = Vordergrundfarbe
- Blau = Hintergrundfarbe

Alle folgenden Textausgaben verwenden diese Farben.

```
CLS

COLOUR RGB(YELLOW), RGB(BLUE)

PRINT "Hallo MMBasic"
```

## 18.6. Der Bildschirmhintergrund

Mit `CLS` wird der Bildschirm gelöscht.

Ohne Parameter wird die aktuelle Hintergrundfarbe verwendet.

```
CLS
```

Es kann aber auch direkt eine Farbe angegeben werden.

```
CLS RGB(BLACK)
```

## 18.7. Ein kleines Farbexperiment

```
CLS RGB(BLACK)

COLOUR RGB(YELLOW), RGB(BLACK)

PRINT "Gelber Text"

COLOUR RGB(CYAN), RGB(BLACK)

PRINT "Cyanfarbener Text"

COLOUR RGB(RED), RGB(BLACK)
```

```
PRINT "Roter Text"
```

Probiere verschiedene Farbkombinationen aus.

## 18.8. Schriftarten

MMBasic enthält bereits mehrere eingebaute Schriftarten.

Jede Schrift besitzt eine andere Größe.

Die Standard-Schrift ist Schriftart 1.

## 18.9. Schriftart auswählen

Mit `FONT` wird die gewünschte Schriftart ausgewählt.

```
FONT 1
PRINT "Schriftart 1"

FONT 2
PRINT "Schriftart 2"

FONT 3
PRINT "Schriftart 3"
```

## 18.10. Die wichtigsten Schriftarten

| Font | Beschreibung |
|---|---|
| 1 | Standardschrift |
| 2 | Mittlere Schrift |
| 3 | Große Schrift |
| 5 | Sehr große Schrift |
| 6 | Große Ziffern |

## 18.11. Schriftgröße ausprobieren

```
CLS

FONT 1
PRINT "Kleine Schrift"

FONT 2
```

```
PRINT "Mittlere Schrift"

FONT 3
PRINT "Grosse Schrift"

FONT 5
PRINT "Sehr grosse Schrift"
```

## 18.12. Große Zahlen darstellen

Für Messwerte oder Uhren eignet sich Font 6 besonders gut.

```
CLS

FONT 6

PRINT "12:45"
```

## 18.13. Informationen zur aktuellen Schrift

Die Breite und Höhe der aktuellen Schrift können abgefragt werden.

```
PRINT MM.INFO(FONTWIDTH)
PRINT MM.INFO(FONTHEIGHT)
```

Beispiel:

```
16
24
```

Die Werte hängen von der gewählten Schriftart und vom verwendeten Grafikmodus ab.

## 18.14. Ein digitales Display

Das folgende Programm simuliert eine einfache Digitalanzeige.

```
CLS RGB(BLACK)

COLOUR RGB(GREEN), RGB(BLACK)

FONT 6

PRINT TIME$
```

Je nach System wird die aktuelle Uhrzeit in großen Ziffern dargestellt.

## 18.15. Praxisbeispiel: Eine Farb- und Schriftmustertafel

Wir haben nun Farben, Vorder- und Hintergrundfarben sowie verschiedene Schriftarten kennengelernt.

Zum Abschluss erstellen wir eine kleine Farb- und Schriftmustertafel, die mehrere Möglichkeiten auf einer Bildschirmseite zusammenfasst.

Dabei wollen wir die verfügbare Bildschirmbreite möglichst gut ausnutzen: Die vordefinierten Farben erscheinen links, selbst gemischte RGB-Farben rechts. Darunter zeigen wir verschiedene Schriftarten.

Das Beispiel greift damit auch Techniken aus den vorherigen Kapiteln wieder auf. Die Positionen der Texte werden nicht mit festen X-Koordinaten festgelegt, sondern aus der aktuellen Bildschirm- und Schriftgröße berechnet.

### 18.15.1. Zwei Spalten auf dem Bildschirm

Für unsere Mustertafel benötigen wir eine linke und eine rechte Spalte.

Texte in der linken Spalte beginnen immer am linken Bildschirmrand. Dafür schreiben wir eine kleine Unterroutine:

```
Sub PrintLeft(ypos, txt$)
  Print @(0,ypos) txt$
End Sub
```

Die X-Koordinate ist immer `0`. Lediglich die Y-Koordinate und der auszugebende Text werden beim Aufruf übergeben.

Zum Beispiel:

```
PrintLeft(Y, "Vordefinierte Farben:")
```

Interessanter wird die rechte Spalte.

Wir könnten dafür eine feste X-Koordinate verwenden:

```
Print @(200,Y) "Eigene RGB-Farben:"
```

Das würde jedoch nur bei einer bestimmten Bildschirmbreite zuverlässig funktionieren.

Stattdessen soll der Text immer am rechten Bildschirmrand ausgerichtet werden.

### 18.15.2. Die rechte Textposition berechnen

Aus dem vorherigen Kapitel kennen wir bereits `MM.HRES`. Damit erhalten wir die Breite des aktuellen Grafikbereichs.

Die Anzahl der Zeichen eines Textes liefert `LEN()`:

```
Len(txt$)
```

Mit `MM.Info(FONTWIDTH)` erfahren wir, wie breit ein Zeichen des aktuellen Fonts in Pixeln ist.

Die Breite des gesamten Textes können wir deshalb berechnen mit:

```
Len(txt$)*MM.Info(FONTWIDTH)
```

Ziehen wir diese Textbreite von der Bildschirmbreite ab, erhalten wir die X-Koordinate, an der der Text beginnen muss, damit seine rechte Kante am rechten Bildschirmrand liegt.

Da wir diesen berechneten Wert nur für die Ausgabe benötigen, brauchen wir dafür nicht einmal eine zusätzliche Variable. Wir können die Berechnung direkt als X-Koordinate in `PRINT @()` einsetzen:

```
Print @(MM.HRES-Len(txt$)*MM.Info(FONTWIDTH),ypos) txt$
```

Daraus entsteht unsere zweite Unterroutine:

```
Sub PrintRight(ypos, txt$)
  Print @(MM.HRES-Len(txt$)*MM.Info(FONTWIDTH),ypos) txt$
End Sub
```

Ein Aufruf wie

```
PrintRight(Y, "Eigene RGB-Farben:")
```

positioniert den Text damit automatisch am rechten Bildschirmrand.

Die Position hängt nicht von einer fest eingetragenen Bildschirmbreite ab. `MM.HRES` und `MM.Info(FONTWIDTH)` liefern die Werte des aktuell verwendeten Grafikmodus und Fonts.

Dadurch passt sich die Berechnung an die aktuelle Darstellung an.

### 18.15.3. Die Zeilenpositionen berechnen

Auch für die vertikalen Abstände verwenden wir keine fest eingetragenen Pixelwerte.

Zunächst bestimmen wir die Höhe eines Zeichens:

```
DY=MM.Info(FONTHEIGHT)
```

Unsere erste Ausgabezeile berechnen wir mit:

```
Y=MM.Info(FONTHEIGHT)*3
```

Weitere Zeilen können wir anschließend relativ zu dieser Position ausgeben:

```
PrintLeft(Y, "WHITE")
PrintLeft(Y+DY, "RED")
PrintLeft(Y+DY*2, "GREEN")
```

Ändert sich die Höhe des verwendeten Fonts, passen sich damit auch die Zeilenabstände an.

### 18.15.4. Die vordefinierten Farben

In der linken Spalte zeigen wir zunächst einige der vordefinierten Farbkonstanten von MMBasic.

Dazu ändern wir vor jeder Ausgabe die Vordergrundfarbe:

```
Colour RGB(WHITE),RGB(BLACK)
PrintLeft(Y, "WHITE")

Colour RGB(RED),RGB(BLACK)
PrintLeft(Y+DY, "RED")

Colour RGB(GREEN),RGB(BLACK)
PrintLeft(Y+DY*2, "GREEN")

Colour RGB(BLUE),RGB(BLACK)
PrintLeft(Y+DY*3, "BLUE")
```

Auf die gleiche Weise folgen `YELLOW`, `CYAN` und `MAGENTA`.

Der Hintergrund bleibt bei allen Ausgaben schwarz.

### 18.15.5. Eigene RGB-Farben

In der rechten Spalte verwenden wir dagegen selbst zusammengestellte RGB-Farben.

Beispielsweise erzeugen wir Orange mit:

```
Colour RGB(255,128,0),RGB(BLACK)
PrintRight(Y, "ORANGE RGB(255,128,0)")
```

Neben dem Namen der Farbe geben wir auch gleich die verwendeten RGB-Werte aus. Dadurch wird unsere Mustertafel gleichzeitig zu einer kleinen Farbreferenz.

Weitere Beispiele sind:

```
Colour RGB(128,255,0),RGB(BLACK)
PrintRight(Y+DY, "HELLGRUEN RGB(128,255,0)")

Colour RGB(255,128,192),RGB(BLACK)
PrintRight(Y+DY*2, "ROSA RGB(255,128,192)")

Colour RGB(128,128,255),RGB(BLACK)
PrintRight(Y+DY*3, "HELLBLAU RGB(128,128,255)")

Colour RGB(128,128,128),RGB(BLACK)
PrintRight(Y+DY*4, "GRAU RGB(128,128,128)")
```

So können wir die Wirkung verschiedener Rot-, Grün- und Blauanteile direkt miteinander vergleichen.

### 18.15.6. Verschiedene Schriftarten

Unterhalb der Farbtabelle zeigen wir zusätzlich einige der eingebauten Schriftarten.

Zunächst wechseln wir wieder zu weißer Schrift:

```
Colour RGB(WHITE),RGB(BLACK)
```

Dann geben wir Beispiele mit verschiedenen Fonts aus:

```
Font 1
PrintLeft(Y, "Font 1 - Standardschrift")

Y=Y+MM.Info(FONTHEIGHT)

Font 2
PrintLeft(Y, "Font 2")

Y=Y+MM.Info(FONTHEIGHT)

Font 3
```

```
PrintLeft(Y, "Font 3")
```

Beachte, dass `MM.Info(FONTHEIGHT)` jeweils nach dem Wechsel des Fonts ausgewertet wird. Dadurch wird für die nächste Ausgabe die Höhe des gerade aktiven Fonts berücksichtigt.

Am Ende stellen wir wieder `FONT 1` sowie weiße Schrift auf schwarzem Hintergrund ein.

### 18.15.7. Das vollständige Programm

Nun setzen wir alle Teile zu unserer Farb- und Schriftmustertafel zusammen:

```
CLS RGB(BLACK)
Font 1
Colour RGB(WHITE),RGB(BLACK)
Print "MMBasic Farb- und Schriftmustertafel"
Print

' ------------------------------------------------------------
' Positionen fuer die beiden Spalten berechnen
' ------------------------------------------------------------
Y=MM.Info(FONTHEIGHT)*3
DY=MM.Info(FONTHEIGHT)

' ------------------------------------------------------------
' Ueberschriften
' ------------------------------------------------------------
Colour RGB(WHITE),RGB(BLACK)
PrintLeft(Y,"Vordefinierte Farben:")
PrintRight(Y,"Eigene RGB-Farben:")
Y=Y+DY*2

' ------------------------------------------------------------
' Vordefinierte Farben - linke Spalte
' ------------------------------------------------------------
Colour RGB(WHITE),RGB(BLACK)
PrintLeft(Y, "WHITE")

Colour RGB(RED),RGB(BLACK)
PrintLeft(Y+DY, "RED")

Colour RGB(GREEN),RGB(BLACK)
PrintLeft(Y+DY*2, "GREEN")

Colour RGB(BLUE),RGB(BLACK)
PrintLeft(Y+DY*3, "BLUE")

Colour RGB(YELLOW),RGB(BLACK)
PrintLeft(Y+DY*4, "YELLOW")

Colour RGB(CYAN),RGB(BLACK)
```

```
PrintLeft(Y+DY*5, "CYAN")

Colour RGB(MAGENTA),RGB(BLACK)
PrintLeft(Y+DY*6, "MAGENTA")

' ------------------------------------------------------------
' Selbst gemischte Farben - rechte Spalte
' ------------------------------------------------------------
Colour RGB(255,128,0),RGB(BLACK)
PrintRight(Y, "ORANGE RGB(255,128,0)")

Colour RGB(128,255,0),RGB(BLACK)
PrintRight(Y+DY, "HELLGRUEN RGB(128,255,0)")

Colour RGB(255,128,192),RGB(BLACK)
PrintRight(Y+DY*2, "ROSA RGB(255,128,192)")

Colour RGB(128,128,255),RGB(BLACK)
PrintRight(Y+DY*3, "HELLBLAU RGB(128,128,255)")

Colour RGB(128,128,128),RGB(BLACK)
PrintRight(Y+DY*4, "GRAU RGB(128,128,128)")

' ------------------------------------------------------------
' Verschiedene Schriftarten
' ------------------------------------------------------------
Y=Y+DY*8
Colour RGB(WHITE),RGB(BLACK)

Font 1
PrintLeft(Y, "Schriftarten:")

Y=Y+MM.Info(FONTHEIGHT)*2

Font 1
PrintLeft(Y, "Font 1 - Standardschrift")

Y=Y+MM.Info(FONTHEIGHT)

Font 2
PrintLeft(Y, "Font 2")

Y=Y+MM.Info(FONTHEIGHT)

Font 3
PrintLeft(Y, "Font 3")

' Standardschrift und Farben wiederherstellen
Font 1
Colour RGB(WHITE),RGB(BLACK)
```

```
End

Sub PrintLeft(ypos, txt$)
  Print @(0,ypos) txt$
End Sub

Sub PrintRight(ypos, txt$)
  Print @(MM.HRES-Len(txt$)*MM.Info(FONTWIDTH),ypos) txt$
End Sub
```

Die Mustertafel zeigt auf einen Blick vordefinierte Farben, einige selbst gemischte RGB-Farben und verschiedene Schriftarten.

Gleichzeitig haben wir Techniken aus den vorherigen Kapiteln wiederverwendet: Unterprogramme strukturieren den Programmcode, `LEN()` bestimmt die Textlänge und die Systeminformationen `MM.HRES`, `MM.Info(FONTWIDTH)` und `MM.Info(FONTHEIGHT)` helfen uns dabei, die Ausgabe passend zum aktuellen Grafikmodus zu positionieren.

Eine solche Mustertafel kann bei eigenen Programmen sehr nützlich sein. Probiere verschiedene RGB-Werte und Fonts aus und vergleiche direkt auf deinem Bildschirm, welche Kombinationen gut lesbar sind.

# 18.16. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere Farben.
- Teste verschiedene Schriftarten.
- Kombiniere unterschiedliche Vorder- und Hintergrundfarben.
- Zeige das aktuelle Datum an.

# 18.17. Probier’s selbst!

Versuche folgende Aufgaben:

1. Gib deinen Namen in einer großen Schrift aus.
2. Zeige Datum und Uhrzeit gleichzeitig an.
3. Verwende eine andere Farbe für jede Zeile.
4. Erstelle eine einfache digitale Uhr.

# 18.18. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Farben mit `RGB()` zu verwenden

- Vorder- und Hintergrundfarben festzulegen
- eigene Farben aus Rot-, Grün- und Blauanteilen zu mischen
- verschiedene Schriftarten und Schriftgrößen auszuwählen
- Informationen über die aktuelle Schrift abzurufen
- Textpositionen aus Bildschirmgröße, Textlänge und Schriftgröße zu berechnen

Mit der Farb- und Schriftmustertafel hast du diese Möglichkeiten in einem größeren Beispiel miteinander kombiniert. Dabei hast du außerdem gesehen, wie sich Texte unabhängig vom aktuellen Grafikmodus am linken oder rechten Bildschirmrand ausrichten lassen.

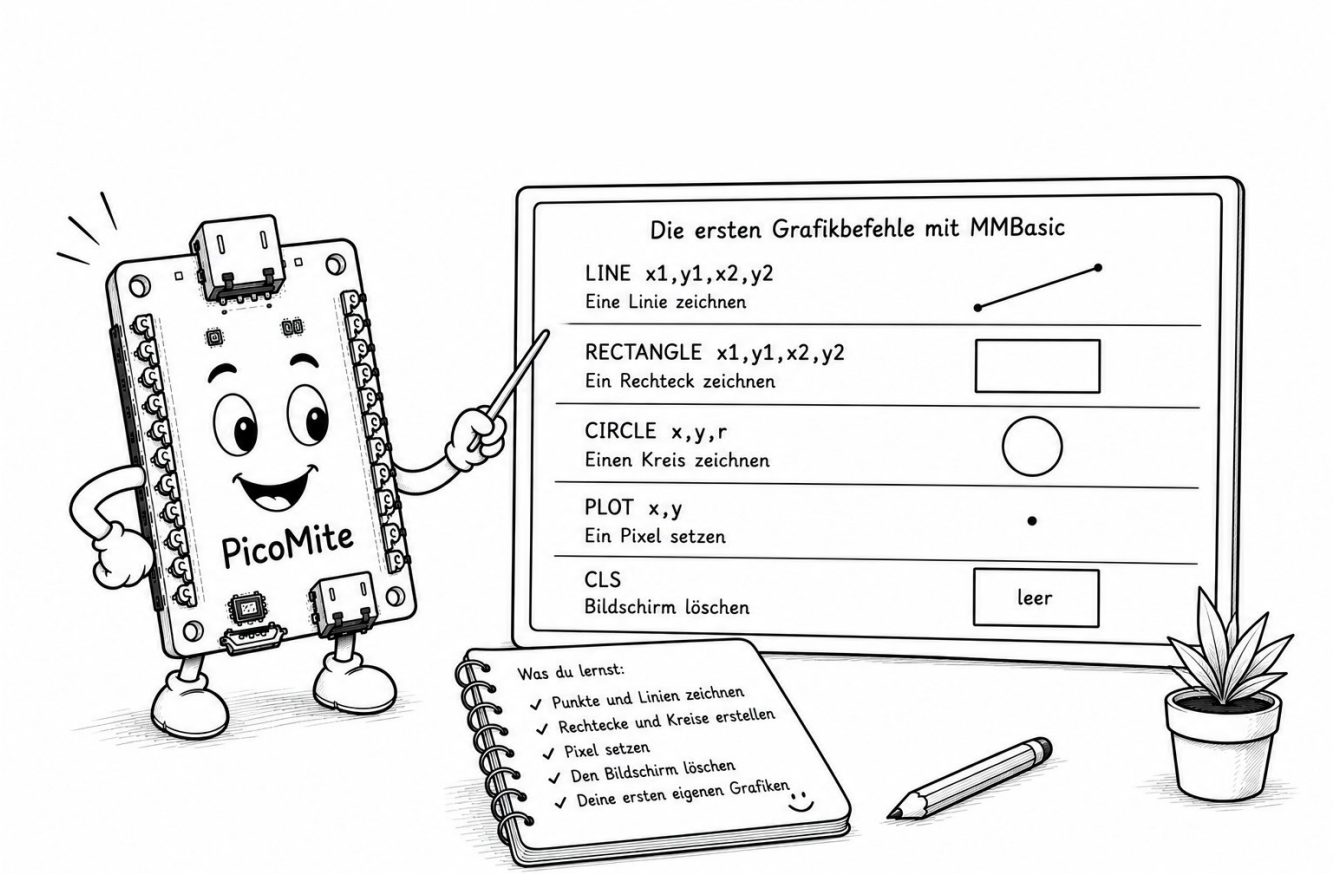
Im nächsten Kapitel beginnen wir mit der eigentlichen Grafikprogrammierung und lernen mit `PIXEL` und `LINE` die ersten Grafikbefehle kennen.

## 18.19. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 18_farben_und_schriften.bas

# Kapitel 19. Die ersten Grafikbefehle




## 19.1. Das lernst du

In diesem Kapitel lernst du:

- den Bildschirm zu löschen
- einzelne Pixel zu setzen
- Linien zu zeichnen
- Koordinaten zu verwenden
- Grafiken mit Schleifen und Zufallszahlen zu erzeugen
- wie das Konzept der Turtle-Grafik funktioniert

Am Ende des Kapitels kannst du eigene einfache Zeichnungen erstellen.

## 19.2. Endlich wird gezeichnet

In den bisherigen Kapiteln haben wir hauptsächlich mit Texten gearbeitet.

Nun beginnen wir mit der eigentlichen Grafikprogrammierung.

Die wichtigsten Befehle für den Einstieg sind:

- CLS

- PIXEL
- LINE

## 19.3. Den Bildschirm löschen

Mit CLS wird der Bildschirm gelöscht.

```
CLS
```

Der gesamte Bildschirm wird dabei mit der aktuellen Hintergrundfarbe gefüllt.

Es kann auch direkt eine Farbe angegeben werden.

```
CLS RGB(BLACK)
```

## 19.4. Ein einzelner Pixel

Der einfachste Grafikbefehl ist `PIXEL`.

```
PIXEL 100,100
```

Dieser Befehl setzt einen einzelnen Bildpunkt an der Position X=100 und Y=100.

## 19.5. Ein farbiger Pixel

Natürlich kann auch eine Farbe angegeben werden.

```
PIXEL 100,100,RGB(RED)
```

## 19.6. Unser erstes Grafikprogramm

```
CLS RGB(BLACK)

PIXEL 100,100,RGB(RED)
```

Das Ergebnis ist ein einzelner roter Punkt auf schwarzem Hintergrund.

## 19.7. Mehrere Pixel zeichnen

```
CLS RGB(BLACK)
```

```
PIXEL 100,100,RGB(RED)
PIXEL 101,100,RGB(RED)
PIXEL 102,100,RGB(RED)
PIXEL 103,100,RGB(RED)
PIXEL 104,100,RGB(RED)
```

Aus einzelnen Pixeln entstehen später Linien, Kreise und Bilder.

## 19.8. Linien zeichnen

Mit `LINE` wird eine Linie zwischen zwei Punkten gezeichnet.

```
LINE X1,Y1,X2,Y2
```

Beispiel:

```
LINE 10,10,200,10
```

## 19.9. Horizontale Linien

```
CLS

LINE 10,50,200,50
```

Die Linie beginnt bei X=10 und endet bei X=200.

Die Y-Koordinate bleibt konstant.

## 19.10. Vertikale Linien

```
CLS

LINE 100,20,100,200
```

Diesmal bleibt die X-Koordinate konstant.

## 19.11. Diagonale Linien

```
CLS

LINE 10,10,200,150
```

Nun werden sowohl X als auch Y verändert.

## 19.12. Linienfarben

Auch für Linien kann eine Farbe angegeben werden.

```
LINE 10,10,200,100,,RGB(RED)
```

## 19.13. Mehrere Linien kombinieren

Mit mehreren Linien können einfache Formen erzeugt werden.

```
CLS

LINE 50,100,250,100
LINE 150,20,150,180
```

Das Ergebnis ist ein Kreuz.

## 19.14. Ein Dreieck zeichnen

```
CLS

LINE 100,50,50,150
LINE 50,150,150,150
LINE 150,150,100,50
```

## 19.15. Ein Haus zeichnen

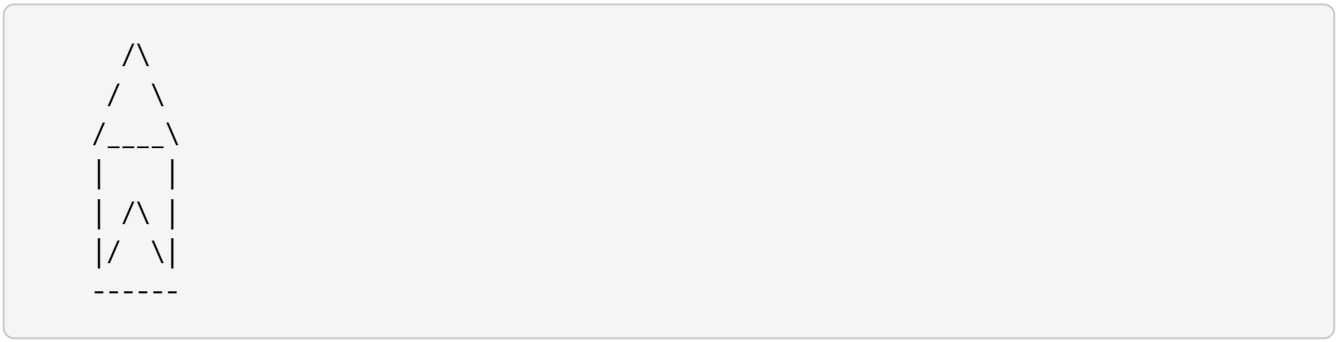
Mit einigen wenigen Linien lässt sich bereits ein kleines Haus darstellen.

```
CLS

LINE 50,150,150,150
LINE 150,150,150,80
LINE 150,80,50,80
LINE 50,80,50,150

LINE 50,80,100,30
LINE 100,30,150,80
```

## 19.16. Das Haus vom Nikolaus

Eine bekannte Zeichenübung ist das Haus vom Nikolaus.

Versuche, es selbst mit LINE zu zeichnen.



## 19.17. Bildschirmmitte berechnen

Mit MM.HRES und MM.VRES lässt sich die Bildschirmmitte berechnen.

```
MITTEX=MM.HRES/2
MITTEY=MM.VRES/2

PIXEL MITTEX,MITTEY,RGB(RED)
```

Dadurch funktioniert das Programm auf unterschiedlichen Bildschirmgrößen.

## 19.18. Ein Fadenkreuz zeichnen

```
CLS

LINE MM.HRES/2,0,MM.HRES/2,MM.VRES

LINE 0,MM.VRES/2,MM.HRES,MM.VRES/2
```

Dieses Programm zeichnet ein Fadenkreuz durch die Bildschirmmitte.

## 19.19. Ein kleines Grafik-Testprogramm

Zum Abschluss verbinden wir die bisher kennengelernten Grafikbefehle mit Schleifen, Zufallszahlen und Farben.

Das folgende Programm zeichnet nacheinander:

- 1000 zufällig verteilte Punkte
- 1000 Kreise mit `CIRCLE`

- 1000 Rechtecke mit `BOX`

Position, Größe und Farbe werden jeweils zufällig bestimmt.

```
CLS RGB(BLACK)

PRINT "1000 zufaellige Punkte"

FOR I=1 TO 1000

  X=INT(RND*MM.HRES)
  Y=INT(RND*MM.VRES)

  FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))

  PIXEL X,Y,FARBE

NEXT I

PAUSE 1000


PRINT "1000 zufaellige Kreise"

FOR I=1 TO 1000

  RADIUS=INT(RND*20)+2

  X=INT(RND*(MM.HRES-2*RADIUS))+RADIUS
  Y=INT(RND*(MM.VRES-2*RADIUS))+RADIUS

  FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))

  CIRCLE X,Y,RADIUS,1,1,FARBE

NEXT I

PAUSE 1000


PRINT "1000 zufaellige Rechtecke"

FOR I=1 TO 1000

  BREITE=INT(RND*50)+5
  HOEHE=INT(RND*50)+5

  X=INT(RND*(MM.HRES-BREITE))
  Y=INT(RND*(MM.VRES-HOEHE))
```

```
  FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))

  BOX X,Y,BREITE,HOEHE,1,FARBE

NEXT I
```

## 19.20. Das Programm verstehen

Für jedes Grafikelement werden zunächst zufällige Bildschirmkoordinaten berechnet.

```
X=INT(RND*MM.HRES)
Y=INT(RND*MM.VRES)
```

Dadurch werden nur Positionen innerhalb der aktuellen Bildschirmauflösung erzeugt.

Auch die Farbe wird zufällig zusammengesetzt:

```
FARBE=RGB(INT(RND*256),INT(RND*256),INT(RND*256))
```

Jeder der drei Farbanteile Rot, Grün und Blau erhält einen zufälligen Wert zwischen 0 und 255.

Bei Kreisen wird zusätzlich ein zufälliger Radius bestimmt:

```
RADIUS=INT(RND*20)+2
```

Die Berechnung der Position berücksichtigt den Radius. Dadurch wird verhindert, dass der Mittelpunkt eines Kreises zu nahe am Bildschirmrand liegt.

Bei den Rechtecken werden Breite und Höhe ebenfalls zufällig erzeugt:

```
BREITE=INT(RND*50)+5
HOEHE=INT(RND*50)+5
```

Die beiden `PAUSE`-Anweisungen sorgen für eine kurze Unterbrechung zwischen den drei Abschnitten.

Da die Grafikelemente nicht gelöscht werden, überlagern sich Punkte, Kreise und Rechtecke zunehmend.

Möchtest du jeden Test auf einem leeren Bildschirm sehen, füge vor dem jeweiligen Abschnitt eine weitere Anweisung ein:

```
CLS RGB(BLACK)
```

## 19.21. Probier’s selbst!

Versuche folgende Aufgaben:

1. Zeichne ein Quadrat aus vier Linien.
2. Zeichne einen Stern.
3. Zeichne ein eigenes Haus.
4. Zeichne ein Spielfeld für Tic-Tac-Toe.
5. Zeichne die Umrisse eines Briefumschlags.

## 19.22. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere Farben.
- Zeichne ein größeres Haus.
- Zeichne mehrere Dreiecke.
- Zeichne ein Gitter aus Linien.

## 19.23. Turtle-Grafik

In den bisherigen Beispielen haben wir Linien gezeichnet, indem wir ihre Anfangs- und Endpunkte angegeben haben.

MMBasic bietet dafür noch eine zweite, besonders anschauliche Möglichkeit: die **Turtle-Grafik**.

Dabei stellst du dir vor, dass sich auf dem Bildschirm eine kleine Schildkröte befindet. Sie kennt ihre aktuelle Position und ihre Blickrichtung.

Anstatt Koordinaten anzugeben, erteilst du ihr einfache Befehle:

- vorwärts laufen
- rückwärts laufen
- nach links drehen
- nach rechts drehen

Während sich die Schildkröte bewegt, zeichnet sie – sofern der Zeichenstift abgesenkt ist – automatisch eine Linie.

Auf diese Weise entstehen Schritt für Schritt geometrische Figuren.

*Geschichte*

Die Turtle-Grafik wurde Ende der 1960er-Jahre von **Seymour Papert** im Zusammenhang mit der Programmiersprache **LOGO** entwickelt.

> Anstatt Koordinaten und mathematische Formeln in den Mittelpunkt zu stellen, sollten Kinder spielerisch lernen, wie Programme aufgebaut sind. Die kleine „Schildkröte“ machte das Ergebnis jedes Befehls sofort sichtbar.
>
> Viele Programmierer kamen auf diese Weise zum ersten Mal mit Computergrafik und Programmierung in Berührung.
>
> Heute gehört die Turtle-Grafik auch zum Funktionsumfang von MMBasic und eignet sich hervorragend für den Einstieg in die Grafikprogrammierung.

# 19.24. Die Schildkröte zurücksetzen

Mit `TURTLE RESET` wird die Turtle in ihre Ausgangsposition zurückgesetzt.

```
CLS
TURTLE RESET
```

Danach befindet sie sich automatisch in der Mitte des Bildschirms und blickt nach oben.

Der Zeichenstift ist abgesenkt, sodass jede Bewegung eine Linie zeichnet.

# 19.25. Ein Quadrat zeichnen

Mit nur zwei Turtle-Befehlen lässt sich bereits ein Quadrat zeichnen.

```
CLS
TURTLE RESET

FOR I = 1 TO 4

  TURTLE FORWARD 100
  TURTLE RIGHT 90

NEXT I
```

Die Schildkröte läuft viermal jeweils 100 Pixel geradeaus und dreht sich anschließend um 90 Grad nach rechts.

Nach der vierten Seite erreicht sie wieder ihren Ausgangspunkt.

# 19.26. Das Programm verstehen

`TURTLE FORWARD` bewegt die Schildkröte in ihrer aktuellen Blickrichtung.

```
TURTLE FORWARD 100
```

Mit `TURTLE RIGHT` oder `TURTLE LEFT` wird die Blickrichtung geändert.

```
TURTLE RIGHT 90
```

Da das Programm diese beiden Befehle viermal wiederholt, entsteht ein Quadrat.

## 19.27. Experimentiere!

Verändere die Seitenlänge:

```
TURTLE FORWARD 150
```

Oder ändere den Drehwinkel:

```
TURTLE RIGHT 60
```

Welche Figur entsteht?

## 19.28. Vielecke zeichnen

Der Drehwinkel bestimmt, welche Figur gezeichnet wird.

Probiere zum Beispiel folgende Winkel aus:

- 120° → Dreieck
- 90° → Quadrat
- 72° → Fünfeck
- 60° → Sechseck
- 170° → Stern- bzw. Rosettenfigur

Je kleiner der Drehwinkel ist, desto mehr Seiten besitzt die Figur.

## 19.29. Eine Spirale

Noch interessanter wird das Ergebnis, wenn die Schrittlänge nach jeder Bewegung etwas größer wird.

```
CLS
TURTLE RESET

FOR I = 1 TO 80

  TURTLE FORWARD I * 3
```

```
  TURTLE RIGHT 20

NEXT I
```

Mit nur wenigen Programmzeilen entsteht eine eindrucksvolle Spiralfigur.

## 19.30. Noch mehr Möglichkeiten

Die Turtle-Grafik von MMBasic bietet weit mehr als nur Vorwärtslaufen und Drehen.

Du kannst unter anderem:

- den Zeichenstift anheben und absenken,
- Farben und Linienstärken ändern,
- Kreise und Kreisbögen zeichnen,
- gefüllte Flächen erzeugen,
- Bézier-Kurven zeichnen,
- die Schildkröte ein- und ausblenden.

Für dieses Tutorial haben wir nur die wichtigsten Turtle-Befehle verwendet.

Eine vollständige Beschreibung aller `TURTLE`-Befehle findest du im MMBasic-Handbuch. Dort werden auch fortgeschrittene Funktionen wie Füllmuster, Bézier-Kurven oder das Arbeiten mit Zustandsstapeln (`PUSH` und `POP`) beschrieben.

## 19.31. Zusammenfassung

In diesem Kapitel hast du die Grundlagen der Grafikprogrammierung mit MMBasic kennengelernt.

Du kennst nun die wichtigsten Grafikbefehle:

- `CLS`
- `PIXEL`
- `LINE`
- `TURTLE`

Außerdem hast du gelernt:

- den Bildschirm zu löschen
- einzelne Pixel zu setzen
- Linien in verschiedenen Farben zu zeichnen
- mit Bildschirmkoordinaten zu arbeiten
- die Bildschirmmitte mit `MM.HRES` und `MM.VRES` zu berechnen

- einfache Zeichnungen aus mehreren Linien zusammenzusetzen
- Grafiken mit Schleifen und Zufallszahlen zu erzeugen
- wie das Konzept der Turtle-Grafik funktioniert
- wie sich mit wenigen Befehlen Quadrate, Vielecke und Spiralen zeichnen lassen

Damit verfügst du bereits über alle Grundlagen, um eigene kleine Grafikprogramme zu entwickeln und geometrische Figuren zu zeichnen.

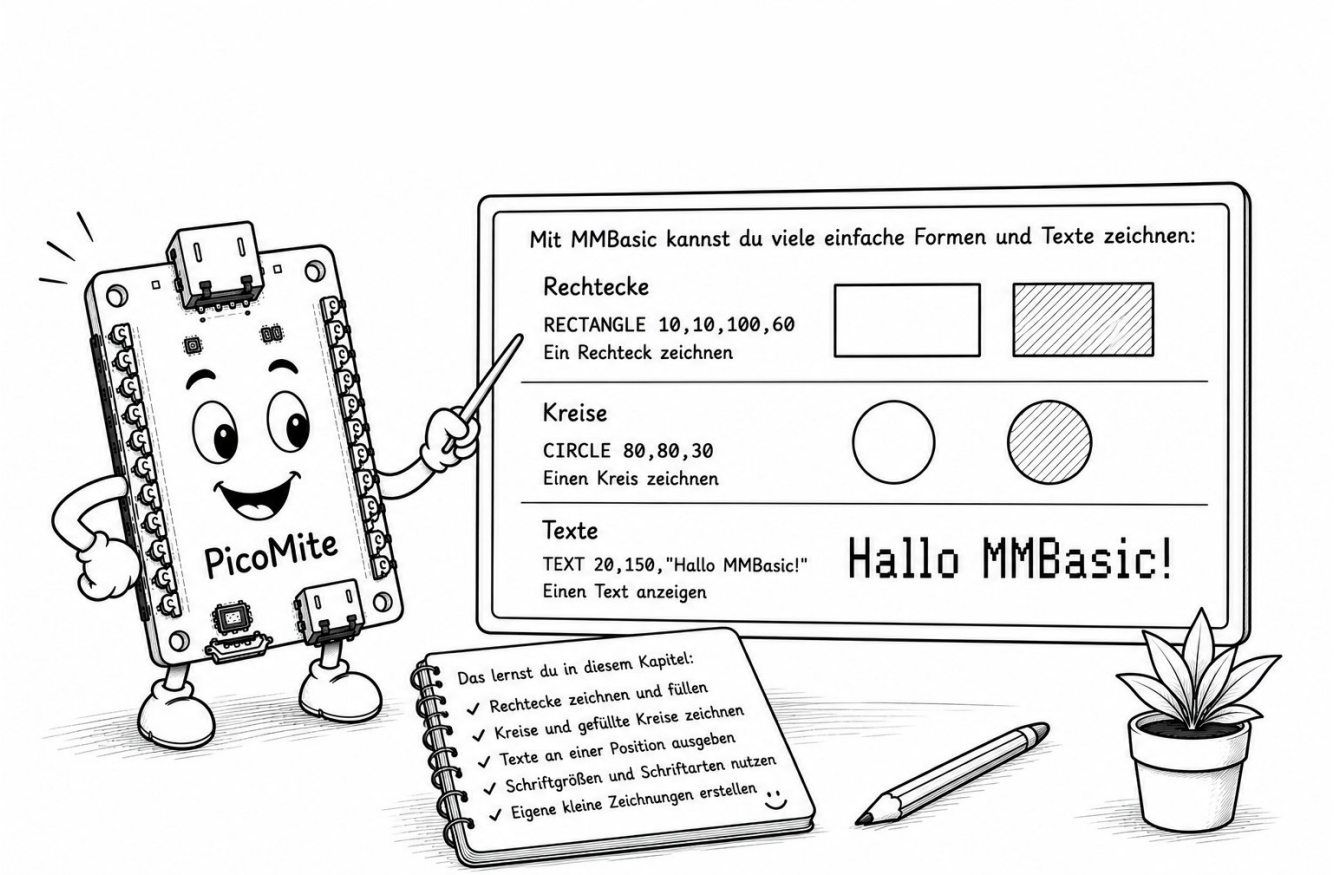
Im nächsten Kapitel erweitern wir diese Möglichkeiten und lernen weitere Grafikbefehle kennen. Damit zeichnen wir Rechtecke, Kreise sowie frei positionierbare Texte und erstellen noch anspruchsvollere Grafiken.

## 19.32. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 19_grafikdemo.bas
- 19_grafikdemo_test_1.bas
- 19_grafikdemo_turtle.bas

# Kapitel 20. Rechtecke, Kreise und Texte




## 20.1. Das lernst du

In diesem Kapitel lernst du:

- Rechtecke mit `BOX` zu zeichnen
- abgerundete Rechtecke mit `RBOX` zu erzeugen
- Kreise und Ellipsen darzustellen
- Kreisbögen mit `ARC` zu zeichnen
- Texte frei auf dem Bildschirm zu positionieren
- Formen mit Farben zu füllen

Am Ende des Kapitels kannst du bereits kleine grafische Bildschirmseiten gestalten.

## 20.2. Mehr als Punkte und Linien

Im letzten Kapitel haben wir einzelne Pixel gesetzt und Linien gezeichnet.

MMBasic bietet weitere Grafikbefehle, mit denen sich häufig benötigte Formen direkt erzeugen lassen:

- `BOX`

- `RBOX`
- `CIRCLE`
- `ARC`
- `TEXT`

Dadurch müssen Rechtecke, Kreise und Texte nicht mühsam aus einzelnen Pixeln zusammengesetzt werden.

## 20.3. Ein Rechteck zeichnen

Mit `BOX` wird ein Rechteck gezeichnet.

Die grundlegende Schreibweise lautet:

```
BOX X,Y,BREITE,HOEHE
```

Ein Beispiel:

```
CLS RGB(BLACK)

BOX 50,40,200,100
```

Das Rechteck beginnt an der Position `X=50`, `Y=40`.

Es ist 200 Pixel breit und 100 Pixel hoch.

## 20.4. Die Parameter von BOX

Der vollständige Befehl lautet:

```
BOX X,Y,BREITE,HOEHE,LINIENBREITE,FARBE,FUELLFARBE
```

Die Parameter haben folgende Bedeutung:

| **Parameter** | **Bedeutung** |
|---|---|
| X | Horizontale Position der linken oberen Ecke |
| Y | Vertikale Position der linken oberen Ecke |
| BREITE | Breite des Rechtecks in Pixeln |
| HOEHE | Höhe des Rechtecks in Pixeln |
| LINIENBREITE | Breite des Rahmens |
| FARBE | Farbe des Rahmens |

| Parameter | Bedeutung |
|---|---|
| FUELLFARBE | Farbe der Fläche |

## 20.5. Ein farbiges Rechteck

```
CLS RGB(BLACK)

BOX 50,40,200,100,3,RGB(YELLOW),RGB(BLUE)
```

Das Rechteck besitzt nun:

- einen drei Pixel breiten gelben Rahmen
- eine blaue Füllung

## 20.6. Ein Rechteck ohne Füllung

Soll das Rechteck nicht gefüllt werden, kann die Füllfarbe weggelassen werden.

```
BOX 50,40,200,100,3,RGB(YELLOW)
```

Der Hintergrund innerhalb des Rechtecks bleibt unverändert.

## 20.7. Mehrere Rechtecke zeichnen

```
CLS RGB(BLACK)

BOX 20,20,100,60,2,RGB(RED)
BOX 50,50,100,60,2,RGB(GREEN)
BOX 80,80,100,60,2,RGB(BLUE)
```

Da sich die Rechtecke überlagern, entsteht bereits eine kleine grafische Komposition.

## 20.8. Abgerundete Rechtecke

Mit `RBOX` werden Rechtecke mit abgerundeten Ecken gezeichnet.

```
RBOX X,Y,BREITE,HOEHE,RADIUS,FARBE,FUELLFARBE
```

Beispiel:

```
CLS RGB(BLACK)
```

```
RBOX 50,50,220,80,15,RGB(WHITE),RGB(BLUE)
```

Der Wert 15 bestimmt den Radius der abgerundeten Ecken.

# 20.9. Ein grafischer Schalter

Abgerundete Rechtecke eignen sich gut für Schaltflächen.

```
CLS RGB(BLACK)

RBOX 100,80,200,70,12,RGB(WHITE),RGB(GREEN)

TEXT 200,115,"START","CM"
```

Der Text wird in der Mitte der Schaltfläche ausgegeben.

# 20.10. Einen Kreis zeichnen

Mit `CIRCLE` wird ein Kreis gezeichnet.

Die grundlegende Schreibweise lautet:

```
CIRCLE X,Y,RADIUS
```

Dabei geben `X` und `Y` den Mittelpunkt des Kreises an.

Beispiel:

```
CLS RGB(BLACK)

CIRCLE 160,120,50
```

Der Kreis besitzt:

- Mittelpunkt `X=160`, `Y=120`
- Radius 50 Pixel

# 20.11. Einen farbigen Kreis zeichnen

```
CLS RGB(BLACK)

CIRCLE 160,120,50,3,1,RGB(YELLOW),RGB(BLUE)
```

Dabei bedeuten die zusätzlichen Parameter:

| Parameter | Bedeutung |
|---|---|
| 3 | Breite der Kreislinie |
| 1 | Seitenverhältnis |
| RGB(YELLOW) | Farbe der Kreislinie |
| RGB(BLUE) | Füllfarbe |

# 20.12. Ellipsen zeichnen

Der Parameter für das Seitenverhältnis bestimmt, ob ein Kreis oder eine Ellipse gezeichnet wird.

Ein Wert von 1 erzeugt einen Kreis.

```
CIRCLE 160,120,60,2,1,RGB(WHITE)
```

Ein Wert kleiner als 1 macht die Form schmaler.

```
CIRCLE 160,120,60,2,0.5,RGB(YELLOW)
```

Dadurch entsteht eine Ellipse.

# 20.13. Ein einfaches Ziel

Mit mehreren Kreisen lassen sich Zielscheiben erzeugen.

```
CLS RGB(BLACK)

X=MM.HRES/2
Y=MM.VRES/2

CIRCLE X,Y,100,2,1,RGB(WHITE)
CIRCLE X,Y,75,2,1,RGB(YELLOW)
CIRCLE X,Y,50,2,1,RGB(RED)
CIRCLE X,Y,25,2,1,RGB(GREEN)
```

Die Position wird aus der Bildschirmgröße berechnet.

Dadurch liegt die Zielscheibe unabhängig von der Auflösung in der Bildschirmmitte.

# 20.14. Kreisbögen zeichnen

Mit `ARC` können Teile eines Kreises gezeichnet werden.

Die grundlegende Schreibweise lautet:

```
ARC X,Y,INNENRADIUS,AUSSENRADIUS,STARTWINKEL,ENDWINKEL,FARBE
```

Beispiel:

```
CLS RGB(BLACK)

ARC 160,120,45,55,0,180,RGB(YELLOW)
```

Der Winkel `0` befindet sich bei MMBasic an der Zwölf-Uhr-Position.

Die Winkel werden in Grad angegeben.

## 20.15. Mehrere Kreisbögen kombinieren

```
CLS RGB(BLACK)

X=MM.HRES/2
Y=MM.VRES/2

ARC X,Y,40,50,0,90,RGB(RED)
ARC X,Y,40,50,90,180,RGB(GREEN)
ARC X,Y,40,50,180,270,RGB(BLUE)
ARC X,Y,40,50,270,360,RGB(YELLOW)
```

Damit entsteht ein Ring aus vier verschiedenfarbigen Abschnitten.

## 20.16. Texte frei positionieren

In Kapitel 17 haben wir mit `PRINT @()` bereits Texte pixelgenau auf dem Bildschirm positioniert. Für die Grafikprogrammierung stellt MMBasic zusätzlich den Befehl `TEXT` zur Verfügung.

Auch `TEXT` arbeitet mit Pixelkoordinaten. Darüber hinaus bietet der Befehl weitere Möglichkeiten, beispielsweise die Ausrichtung, Skalierung, Farbwahl und Drehung des Textes. Damit eignet sich `TEXT` besonders für Texte, die Teil einer Grafik sind.

Die grundlegende Schreibweise lautet:

```
TEXT X,Y,"Text"
```

Beispiel:

```
CLS RGB(BLACK)
```

```
TEXT 50,40,"Hallo MMBasic"
```

## 20.17. Text ausrichten

Der Text kann mit Ausrichtungsparametern relativ zu seiner Position ausgerichtet werden.

```
TEXT X,Y,"Text","AUSRICHTUNG"
```

Für die horizontale Ausrichtung stehen zur Verfügung:

| **Buchstabe** | **Bedeutung** |
|---|---|
| L | Linksbündig |
| C | Zentriert |
| R | Rechtsbündig |

Für die vertikale Ausrichtung stehen zur Verfügung:

| **Buchstabe** | **Bedeutung** |
|---|---|
| T | Oben |
| M | Mittig |
| B | Unten |

## 20.18. Text in der Bildschirmmitte

```
CLS RGB(BLACK)

TEXT MM.HRES/2,MM.VRES/2,"Hallo MMBasic","CM"
```

`C` richtet den Text horizontal zentriert aus.

`M` richtet ihn vertikal mittig aus.

## 20.19. Schriftart und Skalierung

Mit weiteren Parametern können Schriftart und Skalierung festgelegt werden.

```
TEXT 100,80,"MMBasic","LT",2,2
```

Dabei bedeutet:

- Schriftart 2

- Skalierungsfaktor 2

## 20.20. Farbiger Text

Auch Vorder- und Hintergrundfarbe können direkt angegeben werden.

```
CLS RGB(BLACK)

TEXT 100,80,"MMBasic","LT",2,2,RGB(YELLOW),RGB(BLUE)
```

## 20.21. Transparenter Hintergrund

Auf unterstützten Bildschirmen kann der Hintergrund eines Textes transparent bleiben.

Dazu wird als Hintergrundfarbe -1 angegeben.

```
TEXT 100,80,"MMBasic","LT",2,2,RGB(YELLOW),-1
```

Der bereits vorhandene Bildschirminhalt bleibt zwischen den Buchstaben sichtbar.

Transparenter Text wird nicht von jeder Display-Konfiguration unterstützt.

Auf VGA- und HDMI-Systemen steht diese Möglichkeit in der Regel zur Verfügung.

## 20.22. Text drehen

Ein dritter Buchstabe in der Ausrichtung legt die Drehung bzw. Orientierung des Textes fest.

| **Buchstabe** | **Ausrichtung** |
|---|---|
| N | Normal |
| V | Buchstaben senkrecht untereinander |
| I | Um 180 Grad gedreht |
| U | Um 90 Grad gegen den Uhrzeigersinn gedreht |
| D | Um 90 Grad im Uhrzeigersinn gedreht |

Beispiel:

```
CLS RGB(BLACK)

TEXT 20,MM.VRES/2,"MMBasic","LMV",2
```

Der Text wird am linken Bildschirmrand senkrecht dargestellt.

# 20.23. Eine einfache Informationskarte

Nun kombinieren wir Rechteck und Text.

```
CLS RGB(BLACK)

X=MM.HRES/2-150
Y=MM.VRES/2-70

RBOX X,Y,300,140,15,RGB(WHITE),RGB(BLUE)

TEXT MM.HRES/2,Y+35,"MMBasic","CM",2,1,RGB(YELLOW),RGB(BLUE)

TEXT MM.HRES/2,Y+75,DATE$,"CM",1,1,RGB(WHITE),RGB(BLUE)

TEXT MM.HRES/2,Y+105,TIME$,"CM",1,1,RGB(CYAN),RGB(BLUE)
```

Das Ergebnis ist eine kleine Informationskarte mit:

- Überschrift
- Datum
- Uhrzeit

# 20.24. Ein einfaches Gesicht

Mit Kreisen, Bögen und Linien lässt sich ein Gesicht zeichnen.

```
CLS RGB(BLACK)

X=MM.HRES/2
Y=MM.VRES/2

CIRCLE X,Y,100,3,1,RGB(YELLOW)

CIRCLE X-35,Y-30,10,1,1,RGB(WHITE),RGB(WHITE)
CIRCLE X+35,Y-30,10,1,1,RGB(WHITE),RGB(WHITE)

ARC X,Y+10,45,50,120,240,RGB(YELLOW)
```

Probiere aus, wie sich das Gesicht verändert, wenn du die Positionen oder Winkel veränderst.

# 20.25. Experimentiere!

Probiere folgende Änderungen aus:

- Zeichne Rechtecke mit verschiedenen Rahmenbreiten.

- Verwende unterschiedliche Füllfarben.
- Erzeuge Kreise mit verschiedenen Radien.
- Zeichne Ellipsen mit unterschiedlichen Seitenverhältnissen.
- Positioniere Texte an verschiedenen Bildschirmrändern.
- Drehe einen Text in alle verfügbaren Richtungen.

## 20.26. Probier’s selbst!

Versuche folgende Aufgaben:

1. Zeichne eine Ampel aus einem Rechteck und drei Kreisen.
2. Gestalte eine Schaltfläche mit `RBOX` und `TEXT`.
3. Zeichne eine Zielscheibe.
4. Erzeuge eine digitale Uhr mit Rahmen.
5. Zeichne ein einfaches Gesicht.
6. Erstelle eine kleine Visitenkarte mit deinem Namen.

## 20.27. Zusammenfassung

In diesem Kapitel hast du weitere wichtige Grafikbefehle kennengelernt.

Du kannst nun:

- Rechtecke mit `BOX` zeichnen
- abgerundete Rechtecke mit `RBOX` erzeugen
- Kreise und Ellipsen mit `CIRCLE` darstellen
- Kreisbögen mit `ARC` zeichnen
- Texte mit `TEXT` frei positionieren
- Formen mit Farben füllen

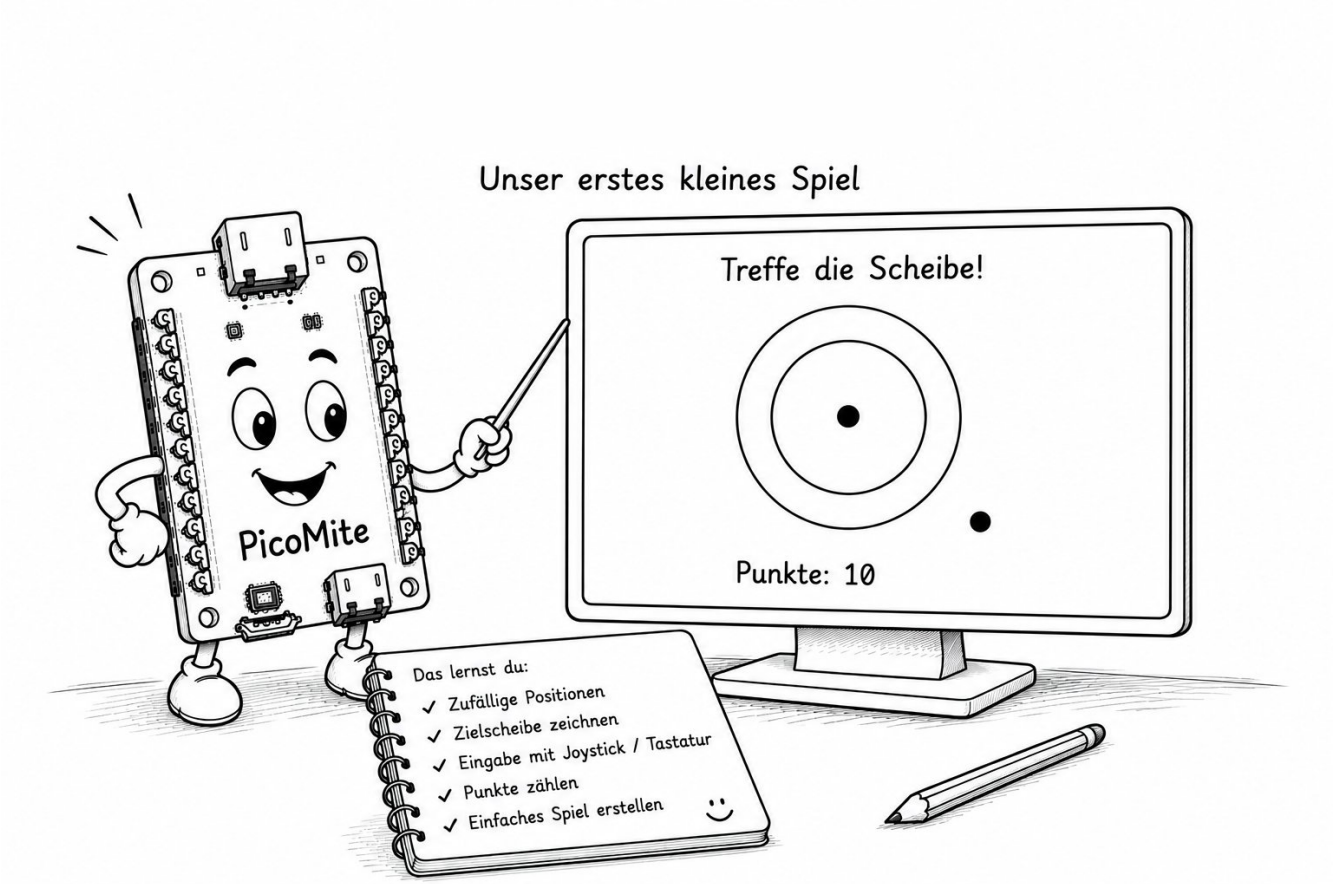
Im nächsten Kapitel verbinden wir die Grafikbefehle zu einem vollständigen kleinen Grafikprojekt.

## 20.28. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 20_formen_demo.bas

# Kapitel 21. Treffe die Scheibe




## 21.1. Das lernst du

In diesem Kapitel lernst du:

- grafische Objekte zu kombinieren
- Bildschirmpositionen zu berechnen
- Entfernungen zu bestimmen
- Funktionen in einem echten Spiel einzusetzen
- ein vollständiges Grafikspiel zu programmieren

Am Ende des Kapitels hast du dein erstes eigenes Grafikspiel entwickelt.

## 21.2. Unser erstes Grafikspiel

In den letzten Kapiteln haben wir gelernt:

- Linien zu zeichnen
- Kreise darzustellen
- Texte zu positionieren
- Farben zu verwenden

Nun wird es Zeit, alles miteinander zu verbinden.

Wir programmieren ein kleines Geschicklichkeitsspiel.

Der Computer versteckt eine Zielscheibe an einer zufälligen Position auf dem Bildschirm.

Deine Aufgabe ist es, die Position möglichst genau zu erraten.

## 21.3. So funktioniert das Spiel

Nach dem Start erscheint eine Zielscheibe.

Anschließend gibst du einen Schuss ab.

Dazu werden die Koordinaten eingegeben:

```
X ?
Y ?
```

Das Programm berechnet anschließend die Entfernung zum Mittelpunkt der Zielscheibe.

Je näher dein Treffer liegt, desto mehr Punkte erhältst du.

## 21.4. Die Zielscheibe zeichnen

Zunächst benötigen wir eine Funktion zum Zeichnen der Zielscheibe.

```
SUB ZeichneScheibe(X,Y)

  CIRCLE X,Y,60,2,1,RGB(WHITE)

  CIRCLE X,Y,40,2,1,RGB(WHITE)

  CIRCLE X,Y,20,2,1,RGB(WHITE)

END SUB
```

Die Zielscheibe besteht aus drei konzentrischen Kreisen.

## 21.5. Die Entfernung berechnen

Um die Entfernung zwischen zwei Punkten zu bestimmen, verwenden wir die Formel:

```
Abstand =
Quadratwurzel(
(X1-X2)^2 +
```

```
(Y1-Y2)^2 )
```

MMBasic besitzt dafür die Funktion SQR().

## 21.6. Das komplette Spiel

```
MODE 2

CLS RGB(BLACK)

TX=INT(RND*(MM.HRES-120))+60
TY=INT(RND*(MM.VRES-120))+60

ZeichneScheibe TX,TY

PRINT
PRINT "Treffe die Scheibe!"
PRINT

INPUT "X ";SX
INPUT "Y ";SY

DIST=SQR((SX-TX)^2+(SY-TY)^2)

CIRCLE SX,SY,4,1,1,RGB(RED),RGB(RED)

PRINT
PRINT "Abstand: ";INT(DIST);" Pixel"

IF DIST<10 THEN

  PRINT "Volltreffer!"

ELSEIF DIST<25 THEN

  PRINT "Sehr gut!"

ELSEIF DIST<50 THEN

  PRINT "Gut getroffen!"

ELSE

  PRINT "Daneben!"

ENDIF
```

```
SUB ZeichneScheibe(X,Y)

  CIRCLE X,Y,60,2,1,RGB(WHITE)

  CIRCLE X,Y,40,2,1,RGB(WHITE)

  CIRCLE X,Y,20,2,1,RGB(WHITE)

END SUB
```

## 21.7. Das Spiel ausprobieren

Nach dem Programmstart erscheint beispielsweise:

```
Treffe die Scheibe!

X ? 120
Y ? 100
```

Anschließend wird dein Schuss als roter Punkt dargestellt.

## 21.8. Die Bildschirmmitte verwenden

Statt einer zufälligen Position kann die Zielscheibe auch in der Mitte erscheinen.

```
TX=MM.HRES/2
TY=MM.VRES/2
```

Dadurch lässt sich das Spiel leichter testen.

## 21.9. Mehrere Schüsse erlauben

Mit einer Schleife können mehrere Versuche durchgeführt werden.

```
FOR RUNDE=1 TO 5

  ...

NEXT RUNDE
```

Dadurch entsteht ein deutlich spannenderes Spiel.

## 21.10. Punkte vergeben

Die Entfernung kann direkt in Punkte umgerechnet werden.

```
PUNKTE=100-DIST

IF PUNKTE<0 THEN
  PUNKTE=0
ENDIF
```

Ein perfekter Treffer bringt 100 Punkte.

## 21.11. Eine farbige Zielscheibe

Mit Füllfarben wird die Zielscheibe attraktiver.

```
CIRCLE X,Y,60,1,1,RGB(WHITE),RGB(BLUE)

CIRCLE X,Y,40,1,1,RGB(WHITE),RGB(RED)

CIRCLE X,Y,20,1,1,RGB(WHITE),RGB(YELLOW)
```

Jetzt erinnert die Scheibe an eine echte Zielscheibe.

## 21.12. Highscore speichern

Mit dem Wissen aus Kapitel 16 könntest du die höchste Punktzahl sogar dauerhaft speichern.

```
OPEN "highscore.txt" FOR OUTPUT AS #1

PRINT #1,PUNKTE

CLOSE #1
```

## 21.13. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere Farben.
- Vergrößere die Zielscheibe.
- Erlaube mehrere Schüsse.
- Vergib Punkte.
- Speichere einen Highscore.

## 21.14. Probier’s selbst!

Versuche folgende Erweiterungen:

1. Führe zehn Schüsse durch.
2. Berechne die Gesamtpunktzahl.
3. Speichere den besten Spieler.
4. Erzeuge unterschiedlich große Zielscheiben.
5. Lasse die Zielscheibe nach jedem Treffer an einer neuen Position erscheinen.

## 21.15. Zusammenfassung

In diesem Kapitel hast du dein erstes Grafikspiel programmiert.

Dabei kamen viele Techniken zusammen:

- Zufallszahlen
- Grafik
- Funktionen
- Benutzereingaben
- mathematische Berechnungen

Du hast damit einen wichtigen Meilenstein erreicht:

Nicht mehr einzelne Befehle stehen im Mittelpunkt, sondern ein vollständiges Programm.

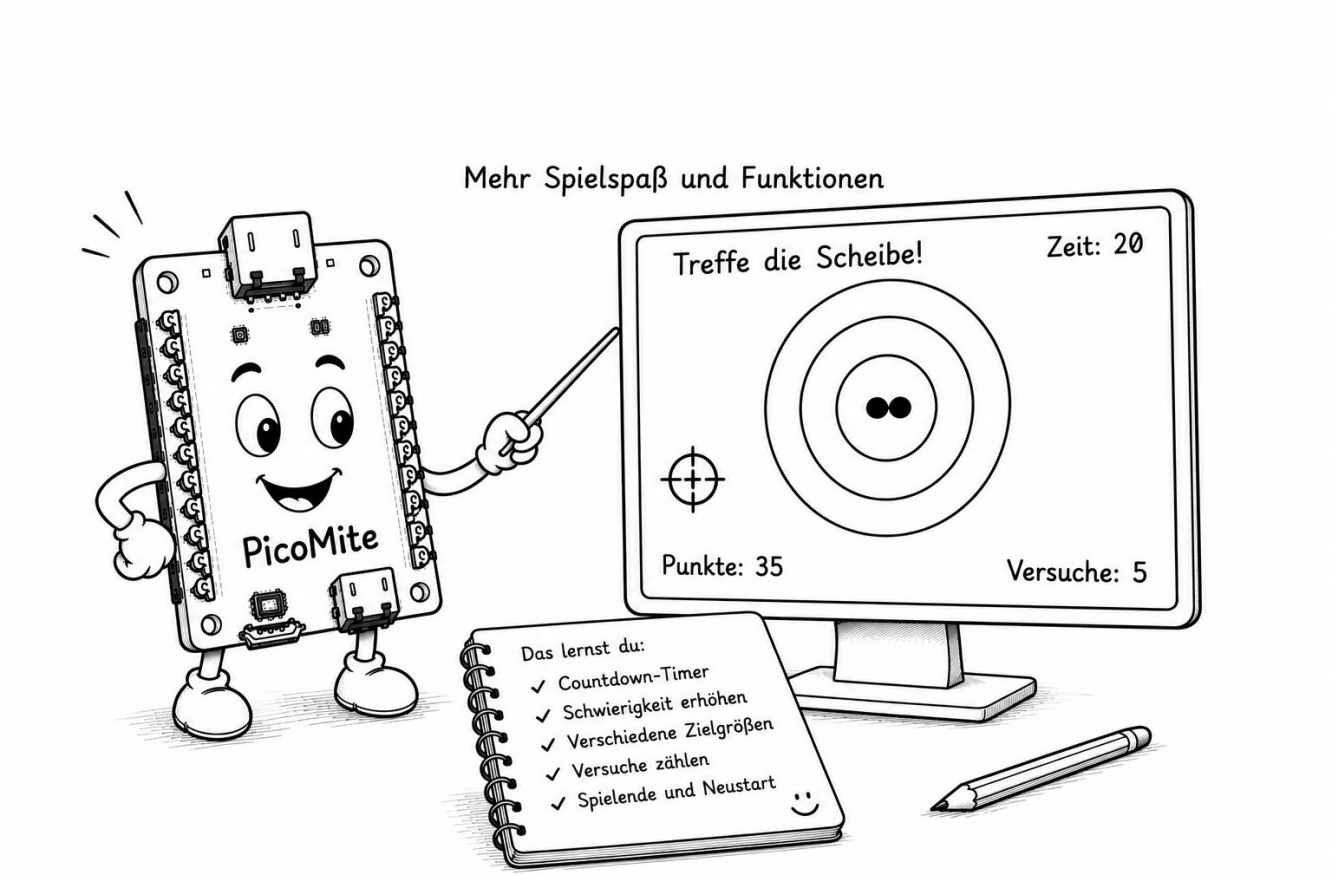
Im nächsten Kapitel werden wir dieses Spiel weiter ausbauen und beispielsweise mehrere Schüsse, Highscores und Schwierigkeitsstufen hinzufügen.

## 21.16. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 21_treffe_die_scheibe.bas

# Kapitel 22. Treffe die Scheibe – Version 2




## 22.1. Das lernst du

In diesem Kapitel lernst du:

- mehrere Spielrunden durchzuführen
- Punkte zu vergeben
- Ergebnisse in Arrays zu speichern
- Statistiken auszuwerten
- Highscores zu speichern

Am Ende des Kapitels besitzt du ein kleines Grafikspiel mit Punktesystem.

## 22.2. Mehr Spielspaß

In der ersten Version durfte der Spieler nur einen einzigen Schuss abgeben.

Das ist zwar ein guter Anfang, aber noch kein richtiges Spiel.

In dieser Version erhält der Spieler zehn Schüsse.

Nach jedem Schuss werden über ein Punktesystem Punkte vergeben.

Am Ende wird die Gesamtpunktzahl berechnet.

## 22.3. Schwierigkeitsstufen

Zu Beginn darf der Spieler den Schwierigkeitsgrad wählen.

```
PRINT "1 = Leicht"
PRINT "2 = Mittel"
PRINT "3 = Schwer"

INPUT "Schwierigkeit ";LEVEL
```

Je höher die Schwierigkeit, desto kleiner wird die Zielscheibe.

```
IF LEVEL=1 THEN
  RADIUS=60
ELSEIF LEVEL=2 THEN
  RADIUS=40
ELSE
  RADIUS=25
ENDIF
```

## 22.4. Ergebnisse speichern

Für jeden Schuss speichern wir die Entfernung.

```
DIM TREFFER(9)
```

Damit können wir die Ergebnisse später auswerten.

## 22.5. Das komplette Spiel

```
MODE 2

CLS RGB(BLACK)

INPUT "Name ";NAME$

PRINT
PRINT "1 = Leicht"
PRINT "2 = Mittel"
PRINT "3 = Schwer"

INPUT "Schwierigkeit ";LEVEL
```

```
IF LEVEL=1 THEN
  RADIUS=60
ELSEIF LEVEL=2 THEN
  RADIUS=40
ELSE
  RADIUS=25
ENDIF

DIM TREFFER(9)

GESAMTPUNKTE=0

FOR RUNDE=1 TO 10

  CLS RGB(BLACK)

  TX=INT(RND*(MM.HRES-2*RADIUS))+RADIUS
  TY=INT(RND*(MM.VRES-2*RADIUS))+RADIUS

  ZeichneScheibe TX,TY,RADIUS

  PRINT
  PRINT "Schuss ";RUNDE;" von 10"

  INPUT "X ";SX
  INPUT "Y ";SY

  DIST=SQR((SX-TX)^2+(SY-TY)^2)

  TREFFER(RUNDE-1)=DIST

  CIRCLE SX,SY,4,1,1,RGB(RED),RGB(RED)

  PUNKTE=100-INT(DIST)

  IF PUNKTE<0 THEN
    PUNKTE=0
  ENDIF

  GESAMTPUNKTE=GESAMTPUNKTE+PUNKTE

  PRINT
  PRINT "Entfernung : ";INT(DIST);" Pixel"
  PRINT "Punkte     : ";PUNKTE

  PAUSE 1500

NEXT RUNDE

CLS RGB(BLACK)
```

```
PRINT
PRINT "====================="
PRINT "   SPIEL BEENDET"
PRINT "====================="
PRINT

PRINT NAME$
PRINT

PRINT "Gesamtpunkte : ";GESAMTPUNKTE

BEST=TREFFER(0)

FOR I=1 TO 9

  IF TREFFER(I)<BEST THEN
    BEST=TREFFER(I)
  ENDIF

NEXT I

PRINT
PRINT "Bester Schuss : ";INT(BEST);" Pixel"

PRINT
PRINT "Alle Schuesse"

FOR I=0 TO 9

  PRINT I+1;". ";INT(TREFFER(I));" Pixel"

NEXT I

OPEN "highscore.txt" FOR OUTPUT AS #1

WRITE #1,NAME$,GESAMTPUNKTE

CLOSE #1

PRINT
PRINT "Highscore gespeichert"

END



SUB ZeichneScheibe(X,Y,R)

  CIRCLE X,Y,R,2,1,RGB(WHITE)

  CIRCLE X,Y,R*0.66,2,1,RGB(WHITE)
```

```
  CIRCLE X,Y,R*0.33,2,1,RGB(WHITE)

END SUB
```

## 22.6. Das Punktesystem

Die Entfernung wird in Punkte umgerechnet.

```
PUNKTE=100-INT(DIST)
```

Ein perfekter Treffer bringt 100 Punkte.

Je weiter der Schuss vom Mittelpunkt entfernt liegt, desto weniger Punkte gibt es.

## 22.7. Die Zielscheibe anpassen

Der Radius der Zielscheibe wird über die Schwierigkeit festgelegt.

| **Stufe** | **Radius** |
|---|---|
| Leicht | 60 |
| Mittel | 40 |
| Schwer | 25 |

Dadurch wird das Spiel deutlich anspruchsvoller.

## 22.8. Die besten Schüsse ermitteln

Alle Entfernungen werden im Array `TREFFER()` gespeichert.

Am Ende wird der beste Schuss gesucht.

```
BEST=TREFFER(0)

FOR I=1 TO 9

  IF TREFFER(I)<BEST THEN
    BEST=TREFFER(I)
  ENDIF

NEXT I
```

Der kleinste Abstand entspricht dem besten Treffer.

## 22.9. Highscores speichern

Mit dem Wissen aus Kapitel 16 speichern wir den Highscore in einer Datei.

```
OPEN "highscore.txt" FOR OUTPUT AS #1

WRITE #1,NAME$,GESAMTPUNKTE

CLOSE #1
```

Die Datei enthält anschließend:

```
"Manfred",742
```

## 22.10. Beispielausgabe

```
====================
   SPIEL BEENDET
====================

Manfred

Gesamtpunkte : 742

Bester Schuss : 3 Pixel

Alle Schuesse

1. 12 Pixel
2. 45 Pixel
3. 3 Pixel
4. 27 Pixel
...
```

## 22.11. Experimentiere!

Probiere folgende Änderungen aus:

- Erhöhe die Anzahl der Schüsse auf 20.
- Verwende andere Farben.
- Vergib Bonuspunkte für Volltreffer.
- Speichere mehrere Highscores.

## 22.12. Probier’s selbst!

Versuche folgende Erweiterungen:

1. Zeige die aktuelle Runde grafisch an.
2. Berechne die durchschnittliche Entfernung.
3. Speichere Datum und Uhrzeit mit dem Highscore.
4. Führe eine Bestenliste mit mehreren Spielern.

## 22.13. Zusammenfassung

In diesem Kapitel hast du das Spiel deutlich erweitert.

Du hast gelernt:

- Ergebnisse in Arrays zu speichern
- Punktesysteme zu entwickeln
- Statistiken auszuwerten
- Highscores zu speichern

Das Spiel fühlt sich nun deutlich mehr wie ein echtes Computerspiel an.

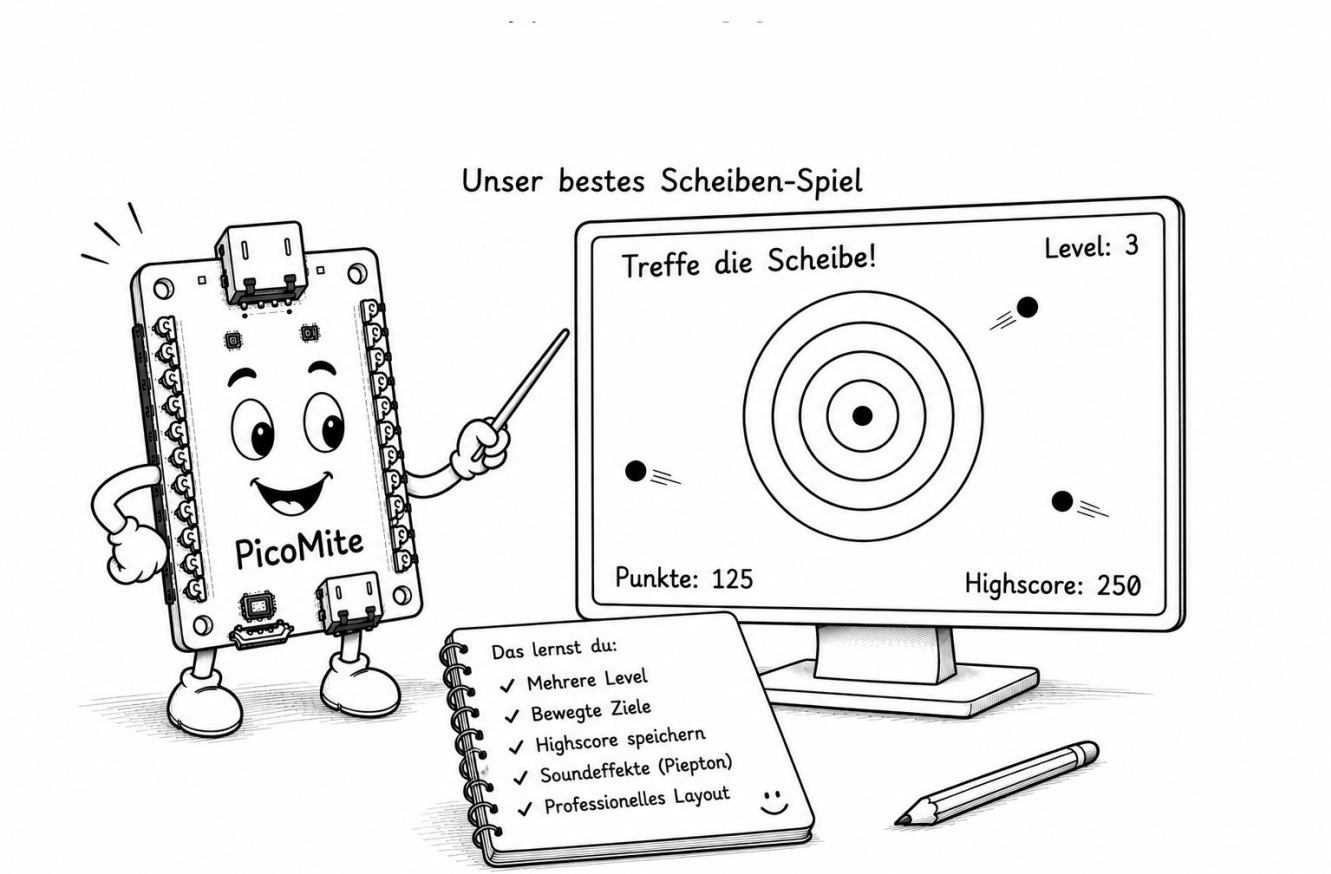
Im nächsten Kapitel werden wir die Steuerung weiter verbessern und die Zielscheibe direkt mit der Maus anvisieren.

## 22.14. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 22_treffe_die_scheibe_v2.bas

# Kapitel 23. Treffe die Scheibe – Version 3




## 23.1. Das lernst du

In diesem Kapitel lernst du:

- die Maus in einem MMBasic-Programm abzufragen
- Mauskoordinaten für ein Grafikspiel zu verwenden
- einen Mausklick zu erkennen
- den Abstand zwischen Mausklick und Ziel zu berechnen
- das Spiel ohne die Eingabe von X- und Y-Koordinaten zu steuern

Am Ende des Kapitels kannst du die Zielscheibe direkt mit der Maus anvisieren.

## 23.2. Direkte Steuerung mit der Maus

In den bisherigen Versionen musste der Spieler die Koordinaten seines Schusses über die Tastatur eingeben:

```
X ? 120
Y ? 85
```

Das war nützlich, um das Koordinatensystem kennenzulernen.

Für ein Grafikspiel ist es jedoch viel natürlicher, direkt mit der Maus auf den Bildschirm zu klicken.

In Version 3 ersetzen wir deshalb die Koordinateneingabe durch eine Maussteuerung.

## 23.3. Informationen der Maus abfragen

MMBasic stellt über die Funktion `MOUSE()` verschiedene Informationen über die angeschlossene Maus bereit.

Die aktuelle horizontale Position erhältst du mit `MOUSE(X)`:

```
MOUSE(X)
```

Die aktuelle vertikale Position erhältst du mit `MOUSE(Y)`:

```
MOUSE(Y)
```

Der Zustand der linken Maustaste kann mit `MOUSE(L)`

```
MOUSE(L)
```

abgefragt werden.

Ein Wert ungleich 0 bedeutet, dass die linke Maustaste gedrückt ist.

Die Maus muss vom verwendeten PicoMite-System unterstützt und korrekt angeschlossen beziehungsweise konfiguriert sein.

Das folgende Programm ist insbesondere für PicoMite-Systeme mit VGA- oder HDMI-Ausgabe und Mausunterstützung vorgesehen.

## 23.4. Die aktuelle Mausposition anzeigen

Mit einem kleinen Testprogramm kannst du zunächst die Mauskoordinaten beobachten.

```
DO

  LOCATE 1,1

  PRINT "X: ";MOUSE(X);"   "
  PRINT "Y: ";MOUSE(Y);"   "

LOOP
```

Bewege die Maus über den Bildschirm.

Die angezeigten Werte verändern sich entsprechend der aktuellen Position.

Das Programm kann mit `Strg+C` beendet werden.

## 23.5. Auf einen Mausklick warten

Das folgende Beispiel wartet, bis die linke Maustaste gedrückt wird:

```
DO
LOOP UNTIL MOUSE(L)<>0

PRINT "Maus wurde geklickt."
```

Die Schleife wird so lange wiederholt, bis ein Mausklick erkannt wird.

## 23.6. Die Position eines Mausklicks speichern

Beim Mausklick speichern wir die aktuellen Koordinaten:

```
DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
SY=MOUSE(Y)
```

Die Variablen `SX` und `SY` enthalten anschließend die Position des Schusses.

## 23.7. Auf das Loslassen der Maustaste warten

Eine Maustaste bleibt meist für mehrere Programmdurchläufe gedrückt.

Damit ein einzelner Klick nicht mehrfach erkannt wird, warten wir anschließend auf das Loslassen:

```
DO
LOOP UNTIL MOUSE(L)=0
```

Ein vollständiger Mausklick besteht damit aus zwei Schritten:

```
DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
```

```
SY=MOUSE(Y)

DO
LOOP UNTIL MOUSE(L)=0
```

## 23.8. Der neue Spielablauf

Das Spiel besteht wieder aus zehn Runden.

In jeder Runde geschieht Folgendes:

1. Der Bildschirm wird gelöscht.
2. Eine Zielscheibe wird an einer zufälligen Position gezeichnet.
3. Das Programm wartet auf einen Mausklick.
4. Der Klick wird als Trefferpunkt dargestellt.
5. Entfernung und Punktzahl werden berechnet.
6. Das Ergebnis wird gespeichert.

## 23.9. Das komplette Spiel

```
MODE 2

INPUT "Name ";NAME$

PRINT
PRINT "1 = Leicht"
PRINT "2 = Mittel"
PRINT "3 = Schwer"

INPUT "Schwierigkeit ";LEVEL

IF LEVEL=1 THEN
  RADIUS=60
ELSEIF LEVEL=2 THEN
  RADIUS=40
ELSE
  RADIUS=25
ENDIF

DIM TREFFER(9)
DIM PUNKTE(9)

GESAMTPUNKTE=0

FOR RUNDE=1 TO 10
```

```
CLS RGB(BLACK)

TX=INT(RND*(MM.HRES-2*RADIUS))+RADIUS
TY=INT(RND*(MM.VRES-2*RADIUS))+RADIUS

ZeichneScheibe TX,TY,RADIUS

TEXT MM.HRES/2,15,"Schuss "+STR$(RUNDE)+" von 10","CT",1,1,RGB(WHITE),-1

TEXT MM.HRES/2,MM.VRES-20,"Klicke auf die Zielscheibe","CB",1,1,RGB(YELLOW),-1

DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
SY=MOUSE(Y)

DO
LOOP UNTIL MOUSE(L)=0

DIST=SQR((SX-TX)^2+(SY-TY)^2)

TREFFER(RUNDE-1)=DIST

PUNKTE(RUNDE-1)=100-INT(DIST)

IF PUNKTE(RUNDE-1)<0 THEN
  PUNKTE(RUNDE-1)=0
ENDIF

GESAMTPUNKTE=GESAMTPUNKTE+PUNKTE(RUNDE-1)

ZeichneTreffer SX,SY

TEXT MM.HRES/2,MM.VRES-55,                        _
     STR$(INT(DIST))+" Pixel entfernt","CB",   _
     1,1,RGB(WHITE),-1

TEXT MM.HRES/2,MM.VRES-35,                        _
     STR$(PUNKTE(RUNDE-1))+" Punkte","CB",     _
     1,1,RGB(CYAN),-1

IF DIST<=RADIUS*0.33 THEN

  TEXT MM.HRES/2,MM.VRES/2,                       _
       "VOLLTREFFER!","CM",2,2,RGB(YELLOW),-1

ELSEIF DIST<=RADIUS*0.66 THEN

  TEXT MM.HRES/2,MM.VRES/2,                       _
       "SEHR GUT!","CM",2,2,RGB(GREEN),-1
```

```
  ELSEIF DIST<=RADIUS THEN

    TEXT MM.HRES/2,MM.VRES/2,                    _
         "GETROFFEN!","CM",2,2,RGB(CYAN),-1

  ELSE

    TEXT MM.HRES/2,MM.VRES/2,                    _
         "DANEBEN!","CM",2,2,RGB(RED),-1

  ENDIF

  PAUSE 1500

NEXT RUNDE

ZeigeAuswertung

SpeichereErgebnis

END



SUB ZeichneScheibe(X,Y,R)

  CIRCLE X,Y,R,2,1,RGB(WHITE),RGB(BLUE)

  CIRCLE X,Y,R*0.66,2,1,RGB(WHITE),RGB(RED)

  CIRCLE X,Y,R*0.33,2,1,RGB(WHITE),RGB(YELLOW)

  CIRCLE X,Y,3,1,1,RGB(BLACK),RGB(BLACK)

END SUB



SUB ZeichneTreffer(X,Y)

  LINE X-7,Y,X+7,Y,2,RGB(WHITE)

  LINE X,Y-7,X,Y+7,2,RGB(WHITE)

  CIRCLE X,Y,4,1,1,RGB(BLACK),RGB(RED)

END SUB
```

```
SUB ZeigeAuswertung

  CLS RGB(BLACK)

  TEXT MM.HRES/2,20,"SPIEL BEENDET","CT",2,2,RGB(YELLOW),-1

  TEXT MM.HRES/2,65,NAME$,"CT",1,2,RGB(WHITE),-1

  TEXT MM.HRES/2,100,                              _
       "Gesamtpunkte: "+STR$(GESAMTPUNKTE),        _
       "CT",1,2,RGB(CYAN),-1

  BEST=TREFFER(0)
  SUMME=0

  FOR I=0 TO 9

    SUMME=SUMME+TREFFER(I)

    IF TREFFER(I)<BEST THEN
      BEST=TREFFER(I)
    ENDIF

  NEXT I

  DURCHSCHNITT=SUMME/10

  TEXT MM.HRES/2,135,                              _
       "Bester Schuss: "+STR$(INT(BEST))+" Pixel",_
       "CT",1,1,RGB(GREEN),-1

  TEXT MM.HRES/2,160,                              _
       "Durchschnitt: "+                           _
       STR$(INT(DURCHSCHNITT))+" Pixel",           _
       "CT",1,1,RGB(WHITE),-1

  Y=200

  FOR I=0 TO 9

    TEXT MM.HRES/2,Y,                              _
         STR$(I+1)+". Schuss: "+                   _
         STR$(INT(TREFFER(I)))+" Pixel, "+         _
         STR$(PUNKTE(I))+" Punkte",                _
         "CT",1,1,RGB(WHITE),-1

    Y=Y+18

  NEXT I

END SUB
```

```
SUB SpeichereErgebnis

  OPEN "highscore.txt" FOR APPEND AS #1

  WRITE #1,DATE$,TIME$,NAME$,LEVEL,GESAMTPUNKTE

  CLOSE #1

END SUB
```

## 23.10. Das Programm verstehen

Die bisherige Eingabe

```
INPUT "X ";SX
INPUT "Y ";SY
```

wurde durch die Mausabfrage ersetzt:

```
DO
LOOP UNTIL MOUSE(L)<>0

SX=MOUSE(X)
SY=MOUSE(Y)

DO
LOOP UNTIL MOUSE(L)=0
```

Dadurch wird der Schuss genau an der angeklickten Bildschirmposition abgegeben.

## 23.11. Den Trefferpunkt darstellen

Ein kleiner roter Kreis zeigt die genaue Position des Mausklicks.

Zusätzlich zeichnen zwei Linien ein Fadenkreuz:

```
LINE X-7,Y,X+7,Y,2,RGB(WHITE)

LINE X,Y-7,X,Y+7,2,RGB(WHITE)

CIRCLE X,Y,4,1,1,RGB(BLACK),RGB(RED)
```

Dadurch ist der Treffer auch auf einer farbigen Zielscheibe gut sichtbar.

## 23.12. Trefferbereiche auswerten

Die Größe der Zielscheibe hängt von der gewählten Schwierigkeitsstufe ab.

Deshalb werden die Trefferbereiche relativ zum Radius berechnet.

```
IF DIST<=RADIUS*0.33 THEN
  ...
ELSEIF DIST<=RADIUS*0.66 THEN
  ...
ELSEIF DIST<=RADIUS THEN
  ...
ELSE
  ...
ENDIF
```

So funktioniert die Auswertung bei allen drei Schwierigkeitsstufen.

## 23.13. Punkte und Entfernungen speichern

Für jede Runde werden Entfernung und Punktzahl in Arrays gespeichert:

```
TREFFER(RUNDE-1)=DIST
PUNKTE(RUNDE-1)=100-INT(DIST)
```

Nach dem Spiel können diese Werte ausgewertet und angezeigt werden.

## 23.14. Den Durchschnitt berechnen

Die Summe aller Entfernungen wird durch die Anzahl der Schüsse geteilt:

```
SUMME=0

FOR I=0 TO 9
  SUMME=SUMME+TREFFER(I)
NEXT I

DURCHSCHNITT=SUMME/10
```

Je kleiner der Durchschnitt ist, desto genauer hat der Spieler insgesamt gezielt.

## 23.15. Ergebnisse dauerhaft speichern

Version 2 hatte die Highscore-Datei bei jedem Spiel überschrieben.

Version 3 verwendet stattdessen den Modus `APPEND`:

```
OPEN "highscore.txt" FOR APPEND AS #1
```

Dadurch bleibt jedes Spielergebnis erhalten.

Mit `WRITE` werden gespeichert:

- Datum
- Uhrzeit
- Spielername
- Schwierigkeitsstufe
- Gesamtpunktzahl

Ein Eintrag kann beispielsweise so aussehen:

```
"12-07-2026","21:45:30","Manfred",2,784
```

## 23.16. Warum SUB hier besonders hilfreich ist

Das Programm besteht inzwischen aus mehreren klar getrennten Aufgaben:

```
ZeichneScheibe
ZeichneTreffer
ZeigeAuswertung
SpeichereErgebnis
```

Jede dieser Aufgaben wurde in eine eigene `SUB` ausgelagert.

Dadurch bleibt das Hauptprogramm trotz seines größeren Umfangs übersichtlich.

## 23.17. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende fünf statt zehn Schüsse.
- Ändere die Farben der Zielscheibe.
- Verkleinere den Radius der schweren Stufe.
- Vergib 200 statt 100 maximal mögliche Punkte.
- Zeichne einen eigenen Mauszeiger.
- Zeige während des Spiels die Gesamtpunktzahl an.

## 23.18. Probier’s selbst!

Versuche folgende Erweiterungen:

1. Beende das Spiel mit der rechten Maustaste.
2. Zeichne vor jedem Schuss einen Countdown.
3. Begrenze die Zeit für einen Schuss.
4. Vergib Bonuspunkte für schnelle Treffer.
5. Lade den bisherigen Highscore aus der Datei.
6. Zeige eine Bestenliste mit mehreren Spielern an.
7. Lasse die Zielscheibe nach kurzer Zeit ihre Position ändern.

## 23.19. Zusammenfassung

In diesem Kapitel hast du das Grafikspiel auf Maussteuerung umgestellt.

Du hast gelernt:

- Mauskoordinaten abzufragen
- Mausklicks zu erkennen
- Klickpositionen grafisch darzustellen
- Treffer relativ zur Zielgröße auszuwerten
- Ergebnisse in Arrays zu speichern
- Statistiken zu berechnen
- mehrere Highscores dauerhaft zu sichern

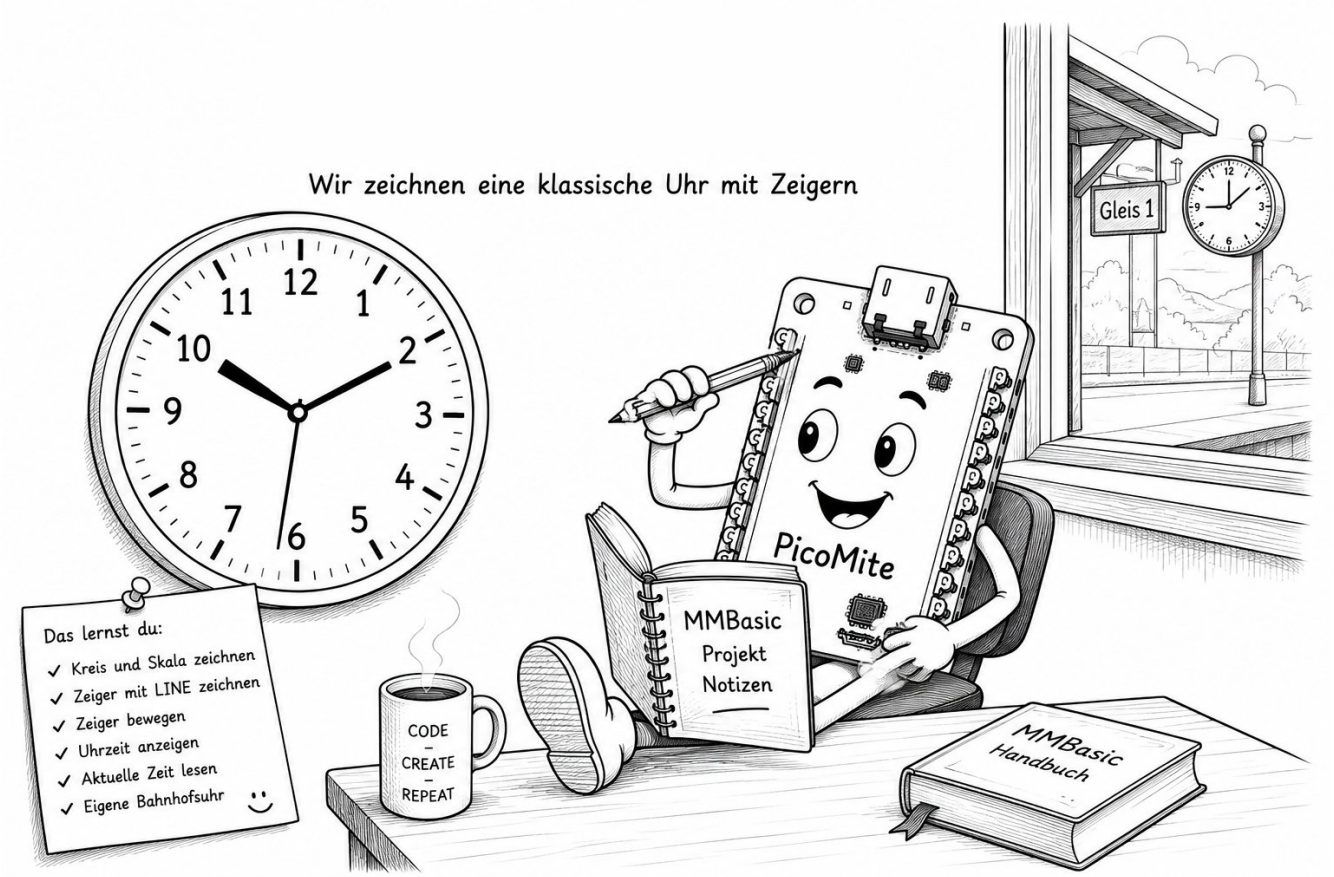
Aus einem einfachen Koordinatenexperiment ist damit Schritt für Schritt ein vollständiges kleines Grafikspiel entstanden.

## 23.20. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 23_treffe_die_scheibe_v3.bas

# Kapitel 24. Eine analoge Bahnhofsuhr




## 24.1. Das lernst du

In diesem Kapitel lernst du:

- eine bildschirmfüllende analoge Uhr zu zeichnen
- die aktuelle Uhrzeit aus `TIME$` auszulesen
- Winkel in Bildschirmkoordinaten umzurechnen
- Stunden-, Minuten- und Sekundenzeiger darzustellen
- ein Zifferblatt an unterschiedliche Bildschirmauflösungen anzupassen
- Texte in eine Grafik einzubauen
- ein größeres Grafikprogramm mit Unterprogrammen zu strukturieren
- ein laufendes Programm mit einem Tastendruck zu beenden

Am Ende des Kapitels besitzt du eine laufende analoge Uhr, die sich am Aussehen klassischer Bahnhofsuhren orientiert und zusätzlich Uhrzeit, Datum, MMBasic-Version und Gerätetyp anzeigt.

## 24.2. Die Idee

Klassische Bahnhofsuhren besitzen ein besonders klares und gut lesbares Design. Typisch sind ein helles Zifferblatt, kräftige Stundenmarkierungen und ein auffälliger Sekundenzeiger. Auf Ziffern

wird häufig verzichtet.

Unsere Uhr greift diese Idee auf, verwendet aber bewusst eigene Farben: Der Stundenzeiger ist grün, der Minutenzeiger blau und der Sekundenzeiger rot. Ein blauer äußerer Rand hebt das Zifferblatt zusätzlich vom schwarzen Bildschirmhintergrund ab.

Die Uhr soll sich automatisch an die aktuelle Bildschirmgröße anpassen und möglichst viel von der verfügbaren Fläche nutzen.

## 24.3. Den Grafikmodus wählen

Nicht alle Grafikmodi stehen auf jeder PicoMite-Variante zur Verfügung. Deshalb prüfen wir zunächst, welche Firmware verwendet wird.

`MM.INFO$(DEVICE)` liefert eine Bezeichnung der verwendeten Geräte- bzw. Firmwarevariante. Beginnt diese mit `WebMite`, verwenden wir `MODE 5`. Andernfalls verwenden wir `MODE 3`:

```
If UCase$(Left$(MM.Info$(DEVICE),7))="WEBMITE" Then
  Mode 5
Else
  Mode 3
EndIf
```

Wir vergleichen nur die ersten sieben Zeichen. Dadurch funktioniert die Abfrage beispielsweise auch dann, wenn `MM.INFO$(DEVICE)` den Text `WebMite RP2350B` zurückliefert.

Damit steht für die Uhr ein geeigneter Grafikmodus zur Verfügung. Anschließend bestimmen wir die Bildschirmmitte und den größtmöglichen Radius.

## 24.4. Die Bildschirmmitte bestimmen

```
mitteX=MM.HRES/2
mitteY=MM.VRES/2
```

Die Variablen `MM.HRES` und `MM.VRES` enthalten die aktuelle Bildschirmbreite und Bildschirmhöhe.

## 24.5. Den größtmöglichen Radius berechnen

Zunächst berechnen wir den Radius aus der Bildschirmbreite:

```
radius=MM.HRES/2-10
```

Danach prüfen wir, ob die Bildschirmhöhe einen kleineren Radius erfordert:

```
If MM.VRES/2-10<radius Then
```

```
  radius=MM.VRES/2-10
EndIf
```

Damit bleibt rund um die Uhr ein kleiner Rand von zehn Pixeln erhalten.

## 24.6. Den Hintergrund vorbereiten

Der Bereich außerhalb des Zifferblatts soll schwarz sein:

```
CLS RGB(BLACK)
```

Dieser Befehl wird nur einmal vor der Hauptschleife ausgeführt. Innerhalb der Schleife wird später lediglich das Zifferblatt neu gezeichnet.

## 24.7. Die Hauptschleife

Die Uhr muss nicht ununterbrochen neu gezeichnet werden. Es genügt, auf eine Änderung von `TIME$` zu reagieren:

```
Do
  If alteZeit$<>Time$ Then
    alteZeit$=Time$
    Berechne Time$
    ZeichneZifferblatt mitteX,mitteY,radius
    ZeichneText mitteX,mitteY,radius
    ZeichneStundenzeiger mitteX,mitteY,radius,stundenWinkel
    ZeichneMinutenzeiger mitteX,mitteY,radius,minutenWinkel
    ZeichneSekundenzeiger mitteX,mitteY,radius,sekundenWinkel
  EndIf
Loop While Inkey$=""
```

Die Uhr wird dadurch nur einmal pro Sekunde aktualisiert. Gleichzeitig prüft `INKEY$`, ob eine Taste gedrückt wurde. Sobald das der Fall ist, endet die Schleife.

## 24.8. Die Uhrzeit berechnen

Die Berechnung der Uhrzeit und der drei Zeigerwinkel ist in ein eigenes Unterprogramm ausgelagert:

```
Sub Berechne(t$)
    stunde=Val(Left$(t$,2))
    minute=Val(Mid$(t$,4,2))
    sekunde=Val(Right$(t$,2))
    stundenWinkel=((stunde Mod 12)*30+minute*0.5)*Pi/180
    minutenWinkel=(minute*6+sekunde*0.1)*Pi/180
```

```
    sekundenWinkel=sekunde*6*Pi/180
End Sub
```

`TIME$` liefert die aktuelle Uhrzeit beispielsweise als:

```
21:37:45
```

Mit `LEFT$()`, `MID$()` und `RIGHT$()` werden Stunde, Minute und Sekunde herausgelöst. `VAL()` wandelt die Zeichenketten anschließend in Zahlen um.

# 24.9. Die Winkel der Zeiger

Eine vollständige Umdrehung besitzt 360 Grad.

Der Sekundenzeiger bewegt sich in 60 Schritten:

```
360 Grad / 60 Sekunden = 6 Grad pro Sekunde
```

Daraus ergibt sich:

```
sekundenWinkel=sekunde*6*Pi/180
```

Auch der Minutenzeiger legt sechs Grad pro Minute zurück. Zusätzlich berücksichtigen wir die Sekunden:

```
minutenWinkel=(minute*6+sekunde*0.1)*Pi/180
```

Der Stundenzeiger bewegt sich pro Stunde um 30 Grad. Durch den Anteil `minute*0.5` wandert er auch zwischen zwei Stundenmarkierungen weiter:

```
stundenWinkel=((stunde Mod 12)*30+minute*0.5)*Pi/180
```

Mit `PI/180` werden die Winkel von Grad in Bogenmaß umgerechnet, denn `SIN()` und `COS()` erwarten Winkel im Bogenmaß.

# 24.10. Das Zifferblatt zeichnen

Das Zifferblatt wird im Unterprogramm `ZeichneZifferblatt` aufgebaut.

Zuerst zeichnen wir zwei gefüllte Kreise:

```
Circle x,y,r,5,1,RGB(BLUE),RGB(WHITE)
```

```
Circle x,y,r-5,5,1,RGB(BLACK),RGB(WHITE)
```

Der erste Kreis erzeugt den blauen Außenrand. Der etwas kleinere zweite Kreis zeichnet den schwarzen Innenrand und das weiße Zifferblatt.

## 24.11. Die 60 Markierungen

Eine Uhr besitzt 60 Markierungen. Der Abstand zwischen zwei Markierungen beträgt:

```
360 Grad / 60 = 6 Grad
```

Mit einer Schleife berechnen wir jede Position:

```
For i=0 To 59
  winkel=i*6*Pi/180
  ...
Next i
```

Jede fünfte Markierung kennzeichnet eine volle Stunde. Diese Markierungen sind länger und dicker:

```
If i Mod 5=0 Then
  innen=r*0.78
  t=3
Else
  innen=r*0.86
  t=1
EndIf
```

Damit unterscheiden sich Stunden- und Minutenmarkierungen nicht nur durch ihre Länge, sondern auch durch ihre Linienstärke.

## 24.12. Positionen auf einem Kreis berechnen

Für jede Markierung benötigen wir einen inneren und einen äußeren Punkt:

```
x1=x+Sin(winkel)*innen
y1=y-Cos(winkel)*innen
x2=x+Sin(winkel)*r*0.92
y2=y-Cos(winkel)*r*0.92
```

Anschließend wird die Markierung gezeichnet:

```
Line x1,y1,x2,y2,t,RGB(BLACK)
```

Das Minuszeichen vor `COS()` ist notwendig, weil die Y-Koordinate auf dem Bildschirm nach unten wächst.

## 24.13. Texte in das Zifferblatt einbauen

Neben der analogen Anzeige enthält die Uhr zusätzliche Informationen. Dafür verwenden wir den bereits bekannten Grafikbefehl `TEXT`.

Das Unterprogramm beginnt mit:

```
Color RGB(BLACK),RGB(WHITE)
```

Damit werden schwarze Schrift und weißer Hintergrund eingestellt.

Zunächst bauen wir aus der MMBasic-Version einen Text auf:

```
version$ = "MMBasic "+Str$(MM.Info(Version))
```

Danach werden MMBasic-Version und Gerätetyp im oberen Bereich des Zifferblatts ausgegeben. Den Gerätetyp liefert `MM.DEVICE$`:

```
Text x-MM.Info(FONTWIDTH)*Len(version$)/2,y-r*0.6,version$
Text x-MM.Info(FONTWIDTH)*Len(MM.DEVICE$)/2,y-r*0.53,MM.DEVICE$
```

`MM.Info(FONTWIDTH)` liefert die Breite eines Zeichens des aktuellen Fonts. Zusammen mit `LEN()` kann der Text so horizontal um die Bildschirmmitte angeordnet werden.

Im unteren Bereich erscheinen die aktuelle Uhrzeit und das Datum:

```
Text x-MM.Info(FONTWIDTH)*4,y+r*0.53,Time$
Text x-MM.Info(FONTWIDTH)*5,y+r*0.6,Date$
```

Damit enthält die Uhr nun sowohl eine analoge als auch eine digitale Anzeige.

## 24.14. Den Stundenzeiger zeichnen

Der Stundenzeiger reicht bis zu 55 Prozent des Radius:

```
x2=x+Sin(winkel)*r*0.55
y2=y-Cos(winkel)*r*0.55
```

```
Line x,y,x2,y2,5,RGB(MIDGREEN)
```

Er wird mit einer Linienstärke von 5 Pixeln in Grün gezeichnet.

Anschließend wird der Mittelpunkt gestaltet:

```
Circle mitteX,mitteY,radius*0.045,4,1,RGB(MIDGREEN),RGB(BLUE)
```

# 24.15. Den Minutenzeiger zeichnen

Der Minutenzeiger ist länger und reicht bis zu 84 Prozent des Radius:

```
x2=x+Sin(winkel)*r*0.84
y2=y-Cos(winkel)*r*0.84
Line x,y,x2,y2,3,RGB(BLUE)
```

Er wird mit einer Linienstärke von 3 Pixeln blau gezeichnet.

Auch hier wird der Mittelpunkt anschließend noch einmal überzeichnet:

```
Circle mitteX,mitteY,radius*0.035,4,1,RGB(BLUE),RGB(RED)
```

Da die Zeiger nacheinander gezeichnet werden, entsteht so eine mehrfarbige Gestaltung des Drehpunkts.

# 24.16. Der rote Sekundenzeiger

Der Sekundenzeiger reicht ebenfalls bis zu 84 Prozent des Radius. Anders als die beiden anderen Zeiger beginnt er nicht genau im Mittelpunkt, sondern besitzt auf der gegenüberliegenden Seite ein kurzes Gegengewicht:

```
x1=x-Sin(winkel)*r*0.14
y1=y+Cos(winkel)*r*0.14
x2=x+Sin(winkel)*r*0.84
y2=y-Cos(winkel)*r*0.84
Line x1,y1,x2,y2,1,RGB(RED)
```

Kurz vor seiner Spitze sitzt außerdem ein kleiner Kreis:

```
xk=x+Sin(winkel)*r*0.70
yk=y-Cos(winkel)*r*0.70
Circle xk,yk,r*0.035,1,1,RGB(RED),RGB(WHITE)
```

Diese Form erinnert an den charakteristischen Sekundenzeiger klassischer Bahnhofsuhren.

## 24.17. Warum muss der Bildschirm nicht jedes Mal gelöscht werden?

In einer früheren, einfacheren Variante könnte man vor jedem Neuzeichnen den gesamten Bildschirm löschen. Das aktuelle Programm geht einen Schritt weiter.

Innerhalb der Hauptschleife wird zuerst das komplette weiße Zifferblatt neu gezeichnet:

```
ZeichneZifferblatt mitteX,mitteY,radius
```

Dadurch werden die alten Zeiger innerhalb des Zifferblatts automatisch übermalt. Danach werden Texte und Zeiger an ihren neuen Positionen gezeichnet.

Der schwarze Bereich außerhalb der Uhr muss deshalb nicht jede Sekunde neu aufgebaut werden. `CLS RGB(BLACK)` wird nur einmal vor dem Start der Schleife ausgeführt.

Je nach Bildschirm, Grafikmodus und Prozessorgeschwindigkeit kann beim vollständigen Neuzeichnen des Zifferblatts ein leichtes Flackern sichtbar sein.

Später lernen wir Framebuffer kennen. Damit kann eine vollständige Grafik zunächst unsichtbar aufgebaut und anschließend in einem Schritt auf den Bildschirm übertragen werden.

## 24.18. Das Programm beenden

Die Hauptschleife läuft nur so lange, wie keine Taste gedrückt wurde:

```
Loop While Inkey$=""
```

Nach einem Tastendruck wird die Schleife verlassen. Anschließend meldet sich das Programm mit:

```
Print
Print "Ready..."
End
```

Damit lässt sich die Uhr jederzeit sauber beenden.

## 24.19. Auflösungsunabhängige Darstellung

Das Programm verwendet keine fest vorgegebenen Mittelpunkt- oder Radiuswerte. Stattdessen werden sie aus `MM.HRES` und `MM.VRES` berechnet.

Dadurch passt sich die Uhr automatisch an unterschiedliche Bildschirmauflösungen an.

## 24.20. Das vollständige Programm

```
Rem ================================================================
Rem Repo:  https://github.com/ManiBecker/MeinErstesMMBasicProgramm
Rem Datei: 24_bahnhofsuhr.bas
Rem Titel: Kapitel 24: Eine analoge Bahnhofsuhr
Rem Buch:  Mein erstes MMBasic Programm
Rem Autor: Manfred Becker
Rem Datum: 09.08.2026
Rem
Rem Beschreibung: Das vollstaendige Programm
Rem
Rem Hardware/Voraussetzungen: PicoMite/ColourMaxiMite
Rem
Rem ================================================================

MODE 3

mitteX=MM.HRES/2
mitteY=MM.VRES/2
radius=MM.HRES/2-10

If MM.VRES/2-10<radius Then
  radius=MM.VRES/2-10
EndIf

CLS RGB(BLACK)

Do
  If alteZeit$<>Time$ Then
    alteZeit$=Time$
    Berechne Time$
    ZeichneZifferblatt mitteX,mitteY,radius
    ZeichneText mitteX,mitteY,radius
    ZeichneStundenzeiger mitteX,mitteY,radius,stundenWinkel
    ZeichneMinutenzeiger mitteX,mitteY,radius,minutenWinkel
    ZeichneSekundenzeiger mitteX,mitteY,radius,sekundenWinkel
  EndIf
Loop While Inkey$=""

Print
Print "Ready..."
End

Sub Berechne(t$)
    stunde=Val(Left$(t$,2))
    minute=Val(Mid$(t$,4,2))
    sekunde=Val(Right$(t$,2))
```

```
    stundenWinkel=((stunde Mod 12)*30+minute*0.5)*Pi/180
    minutenWinkel=(minute*6+sekunde*0.1)*Pi/180
    sekundenWinkel=sekunde*6*Pi/180
End Sub

Sub ZeichneZifferblatt(x,y,r)
  Circle x,y,r,5,1,RGB(BLUE),RGB(WHITE)
  Circle x,y,r-5,5,1,RGB(BLACK),RGB(WHITE)
  For i=0 To 59
    winkel=i*6*Pi/180
    If i Mod 5=0 Then
      innen=r*0.78
      t=3
    Else
      innen=r*0.86
      t=1
    EndIf
    x1=x+Sin(winkel)*innen
    y1=y-Cos(winkel)*innen
    x2=x+Sin(winkel)*r*0.92
    y2=y-Cos(winkel)*r*0.92
    Line x1,y1,x2,y2,t,RGB(BLACK)
  Next i
End Sub

Sub ZeichneText(x,y,r)
    Color RGB(BLACK),RGB(WHITE)
    version$ = "MMBasic "+Str$(MM.Info(Version))
    Text x-MM.Info(FONTWIDTH)*Len(version$)/2,y-r*0.6,version$
    Text x-MM.Info(FONTWIDTH)*Len(MM.DEVICE$)/2,y-r*0.53,MM.DEVICE$
    Text x-MM.Info(FONTWIDTH)*4,y+r*0.53,Time$
    Text x-MM.Info(FONTWIDTH)*5,y+r*0.6,Date$
End Sub

Sub ZeichneStundenzeiger(x,y,r,winkel)
  x2=x+Sin(winkel)*r*0.55
  y2=y-Cos(winkel)*r*0.55
  Line x,y,x2,y2,5,RGB(MIDGREEN)
  Circle mitteX,mitteY,radius*0.045,4,1,RGB(MIDGREEN),RGB(BLUE)
End Sub

Sub ZeichneMinutenzeiger(x,y,r,winkel)
  x2=x+Sin(winkel)*R*0.84
  y2=y-Cos(winkel)*R*0.84
  Line x,y,x2,y2,3,RGB(BLUE)
  Circle mitteX,mitteY,radius*0.035,4,1,RGB(BLUE),RGB(RED)
End Sub

Sub ZeichneSekundenzeiger(x,y,r,winkel)
  x1=x-Sin(winkel)*r*0.14
  y1=y+Cos(winkel)*r*0.14
```

```
  x2=x+Sin(winkel)*r*0.84
  y2=y-Cos(winkel)*r*0.84
  Line x1,y1,x2,y2,1,RGB(RED)
  xk=x+Sin(winkel)*r*0.70
  yk=y-Cos(winkel)*r*0.70
  Circle xk,yk,r*0.035,1,1,RGB(RED),RGB(WHITE)
End Sub
```

## 24.21. Das Programm verstehen

Das Hauptprogramm ist bewusst kurz gehalten. Die eigentliche Arbeit übernehmen mehrere Unterprogramme:

- `Berechne` zerlegt die Uhrzeit und bestimmt die Winkel.
- `ZeichneZifferblatt` zeichnet Rand, Zifferblatt und Markierungen.
- `ZeichneText` ergänzt Version, Gerät, Uhrzeit und Datum.
- `ZeichneStundenzeiger` zeichnet den grünen Stundenzeiger.
- `ZeichneMinutenzeiger` zeichnet den blauen Minutenzeiger.
- `ZeichneSekundenzeiger` zeichnet den roten Sekundenzeiger mit Gegengewicht und Kreis.

Diese Aufteilung macht das Programm übersichtlicher. Jede Aufgabe befindet sich an einer eigenen Stelle und kann unabhängig verändert werden.

## 24.22. Experimentiere!

Probiere folgende Änderungen aus:

- Ändere die Farben der drei Zeiger.
- Vergrößere oder verkleinere den Kreis am Sekundenzeiger.
- Verändere die Länge der Zeiger.
- Ergänze die Ziffern 3, 6, 9 und 12.
- Ändere die Farben des äußeren und inneren Randes.
- Ersetze MMBasic-Version oder Gerätetyp durch einen eigenen Schriftzug.

## 24.23. Probier’s selbst!

Versuche folgende Erweiterungen:

1. Erstelle eine Nachtansicht mit dunklem Zifferblatt.
2. Ergänze eine zweite Zeitzone.
3. Spiele zu jeder vollen Stunde einen Ton ab.
4. Zeige zusätzlich den Wochentag an.

5. Verwende andere Schriftarten oder Schriftgrößen für die Zusatzinformationen.
6. Verwende später einen Framebuffer, um das Flackern zu vermeiden.

## 24.24. Zusammenfassung

In diesem Kapitel hast du eine vollständige analoge Uhr programmiert.

Du hast gelernt:

- eine Grafik an die Bildschirmauflösung anzupassen
- kreisförmig angeordnete Markierungen zu berechnen
- `TIME$` in Stunden, Minuten und Sekunden zu zerlegen
- Winkel mit `SIN()` und `COS()` in Koordinaten umzuwandeln
- Stunden- und Minutenmarkierungen unterschiedlich zu gestalten
- bewegliche Zeiger mit verschiedenen Farben und Linienstärken darzustellen
- zusätzliche Informationen mit `TEXT` in eine Grafik einzubauen
- ein größeres Programm mit mehreren Unterprogrammen zu strukturieren
- eine laufende Schleife mit `INKEY$` zu beenden

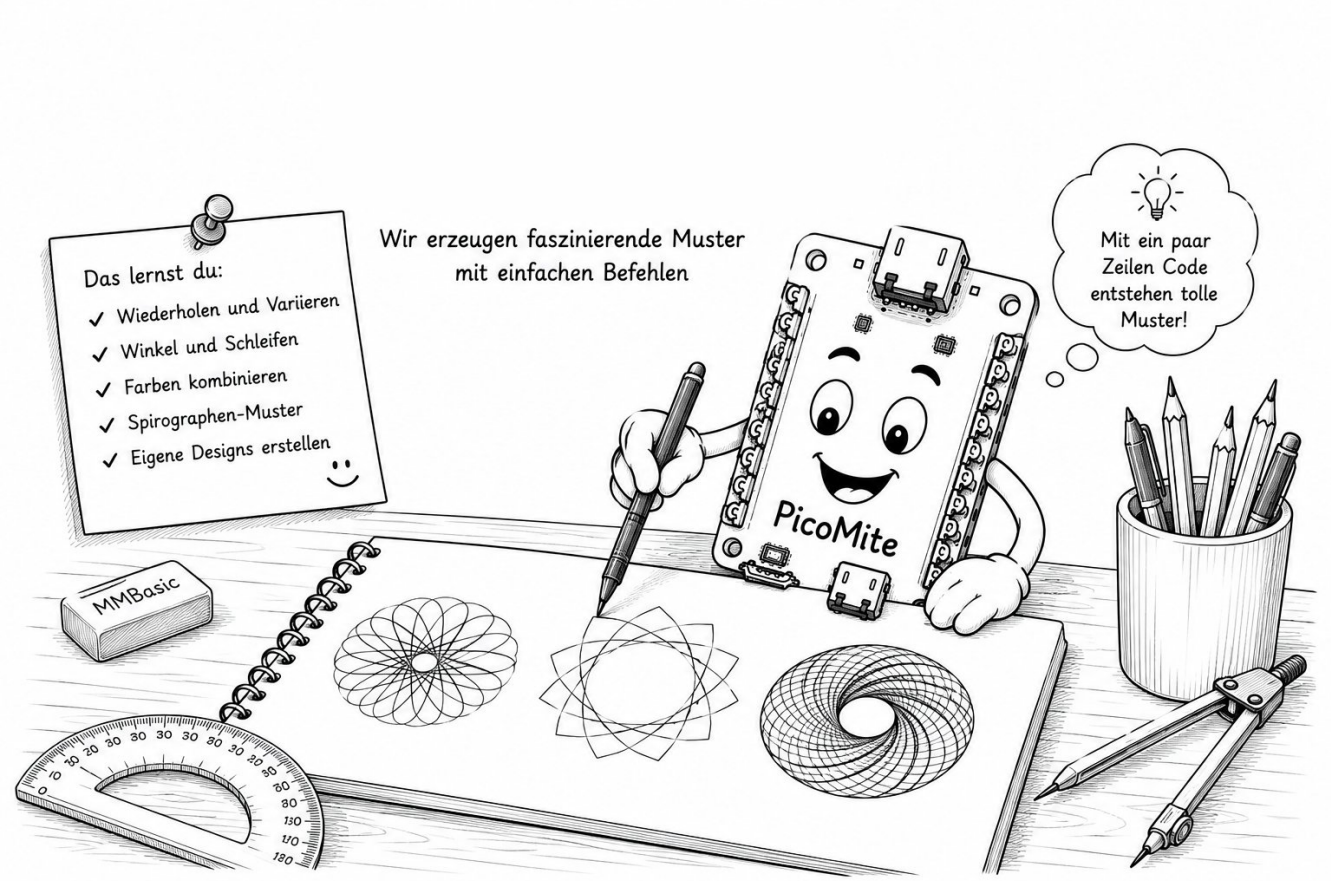
Die Bahnhofsuhr verbindet Grafik, Mathematik, Zeichenketten, Schleifen und Unterprogramme zu einer vollständigen Anwendung.

## 24.25. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 24_bahnhofsuhr.bas

# Kapitel 25. Geometrische Muster und Spirographen




## 25.1. Das lernst du

In diesem Kapitel lernst du:

- Kreise mit Sinus und Cosinus zu berechnen
- Punkte auf einem Kreis zu positionieren
- Sterne zu zeichnen
- Rosetten zu erzeugen
- Spirograph-ähnliche Muster zu erstellen
- mathematische Formeln für Grafiken zu verwenden

Am Ende des Kapitels kannst du faszinierende geometrische Bilder erzeugen.

## 25.2. Mathematik kann schön sein

Viele Menschen verbinden Mathematik mit Formeln und Berechnungen.

Mit Computergrafik wird Mathematik jedoch sichtbar.

Schon wenige Zeilen MMBasic-Code können erstaunliche geometrische Muster erzeugen.

In diesem Kapitel verwenden wir:

- SIN()
- COS()
- PI

um Punkte auf Kreisen zu berechnen.

## 25.3. Punkte auf einem Kreis

Die Position eines Punktes auf einem Kreis kann mit Sinus und Cosinus berechnet werden.

```
X = MITTEX + SIN(WINKEL) * RADIUS
Y = MITTEY - COS(WINKEL) * RADIUS
```

Dabei bedeutet:

- MITTEX = Mittelpunkt X
- MITTEY = Mittelpunkt Y
- WINKEL = Winkel in Bogenmaß
- RADIUS = Radius des Kreises

## 25.4. Ein Kreis aus Punkten

Das folgende Programm zeichnet einen Kreis aus vielen einzelnen Punkten.

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 360

  W=I*PI/180

  X=MX+SIN(W)*100
  Y=MY-COS(W)*100

  PIXEL X,Y,RGB(YELLOW)

NEXT I
```

Die Punkte liegen so dicht beieinander, dass ein Kreis entsteht.

## 25.5. Ein Sonnenstrahlen-Muster

Nun verbinden wir alle Punkte mit dem Mittelpunkt.

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 360 STEP 5

  W=I*PI/180

  X=MX+SIN(W)*150
  Y=MY-COS(W)*150

  LINE MX,MY,X,Y,,,RGB(YELLOW)

NEXT I
```

Das Ergebnis erinnert an Sonnenstrahlen.

## 25.6. Ein Stern

Nun verbinden wir nicht mehr den Mittelpunkt mit dem Kreis, sondern jeweils zwei Punkte auf dem Kreis.

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 4

  W=I*72*PI/180

  X(I)=MX+SIN(W)*120
  Y(I)=MY-COS(W)*120

NEXT I

FOR I=0 TO 4

  J=(I+2) MOD 5

  LINE X(I),Y(I),X(J),Y(J),,,RGB(CYAN)
```

```
NEXT I
```

Es entsteht ein klassischer fünfzackiger Stern.

## 25.7. Ein Stern mit zehn Spitzen

Durch mehr Punkte auf dem Kreis entstehen komplexere Figuren.

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

DIM X(9)
DIM Y(9)

FOR I=0 TO 9

  W=I*36*PI/180

  X(I)=MX+SIN(W)*150
  Y(I)=MY-COS(W)*150

NEXT I

FOR I=0 TO 9

  J=(I+3) MOD 10

  LINE X(I),Y(I),X(J),Y(J),,,RGB(GREEN)

NEXT I
```

Probiere verschiedene Werte für `+3` aus.

## 25.8. Eine Rosette

Rosetten entstehen durch viele Linien zwischen Punkten auf einem Kreis.

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 355 STEP 5

  W1=I*PI/180
```

```
  W2=(I*3)*PI/180

  X1=MX+SIN(W1)*150
  Y1=MY-COS(W1)*150

  X2=MX+SIN(W2)*150
  Y2=MY-COS(W2)*150

  LINE X1,Y1,X2,Y2,,,RGB(YELLOW)

NEXT I
```

Das Ergebnis erinnert an eine Blüte.

## 25.9. Farben ins Spiel bringen

Jetzt erhält jede Linie eine zufällige Farbe.

```
FARBE=RGB( INT(RND*256), INT(RND*256), INT(RND*256) )
```

Setze die Farbe direkt beim Zeichnen der Linie ein.

Dadurch entstehen sehr farbenfrohe Muster.

## 25.10. Ein einfacher Spirograph

Jetzt wird es spannend.

Das folgende Programm erzeugt ein Muster, das an die bekannten Spirograph-Zeichnungen erinnert.

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 1440 STEP 2

  W=I*PI/180

  X=MX+COS(W)*150+COS(W*5)*40

  Y=MY+SIN(W)*150+SIN(W*5)*40

  PIXEL X,Y,RGB(CYAN)

NEXT I
```

Das Muster entsteht durch die Überlagerung zweier Kreisbewegungen.

## 25.11. Das Muster verändern

Die beiden Zahlen:

```
150
40
```

bestimmen die Größe der Kreise.

Die Zahl:

```
5
```

bestimmt das Verhältnis der Drehgeschwindigkeiten.

Schon kleine Änderungen erzeugen völlig andere Bilder.

Probiere beispielsweise:

```
COS(W*7)
SIN(W*7)
```

oder:

```
COS(W*9)
SIN(W*9)
```

aus.

## 25.12. Ein spektakulärer Spirograph

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 3600 STEP 1

  W=I*PI/180

  X=MX+COS(W)*120+COS(W*7)*50

  Y=MY+SIN(W)*120+SIN(W*7)*50
```

```
  FARBE=RGB( INT(RND*256), INT(RND*256), INT(RND*256) )

  PIXEL X,Y,FARBE

NEXT I
```

Auf schnellen PicoMite-Systemen entsteht innerhalb weniger Sekunden ein beeindruckendes Kunstwerk.

## 25.13. Ein Blumenmuster

```
CLS RGB(BLACK)

MX=MM.HRES/2
MY=MM.VRES/2

FOR I=0 TO 3600

  W=I*PI/180

  R=120+40*SIN(W*8)

  X=MX+COS(W)*R
  Y=MY+SIN(W)*R

  PIXEL X,Y,RGB(MAGENTA)

NEXT I
```

Das Ergebnis erinnert an eine Blume.

## 25.14. Was haben wir gelernt?

Alle bisherigen Muster basieren auf derselben Idee:

```
X=MX+SIN(...)
Y=MY+COS(...)
```

Durch geschickte Kombinationen entstehen:

- Sterne
- Rosetten
- Blumen
- Spirographen

- mathematische Kunstwerke

# 25.15. Beispiele geometrischer Muster

Die folgenden Abbildungen zeigen einige der Muster, die mit den in diesem Kapitel vorgestellten Techniken erzeugt werden können.

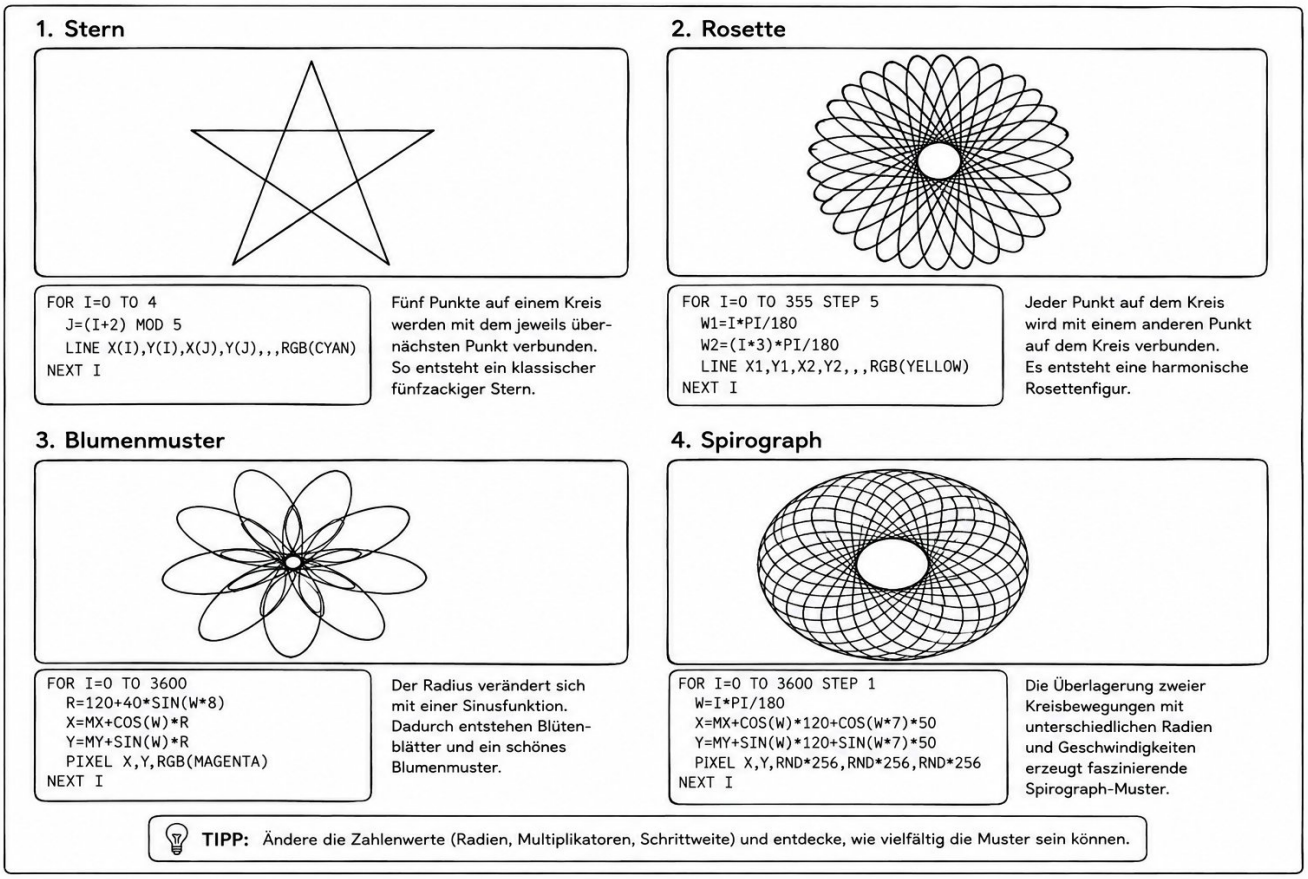



*Abbildung 1. Stern, Rosette, Blumenmuster und Spirograph*

# 25.16. Experimentiere!

Probiere folgende Änderungen aus:

- Ändere die Radien.
- Verwende andere Farben.
- Erhöhe die Anzahl der Linien.
- Verändere die Multiplikatoren 3, 5 oder 7.
- Zeichne mehrere Rosetten gleichzeitig.

# 25.17. Probier’s selbst!

Versuche folgende Aufgaben:

1. Zeichne einen sechszackigen Stern.

2. Erzeuge eine Rosette mit 100 Linien.
3. Verwende zufällige Farben.
4. Kombiniere zwei Blumenmuster.
5. Entwickle deinen eigenen Spirographen.

## 25.18. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Kreispunkte mit Sinus und Cosinus zu berechnen
- geometrische Muster zu erzeugen
- Sterne und Rosetten zu zeichnen
- Spirograph-ähnliche Grafiken zu erstellen

Du hast außerdem gesehen, dass Mathematik nicht nur für Berechnungen nützlich ist, sondern auch faszinierende Bilder erzeugen kann.

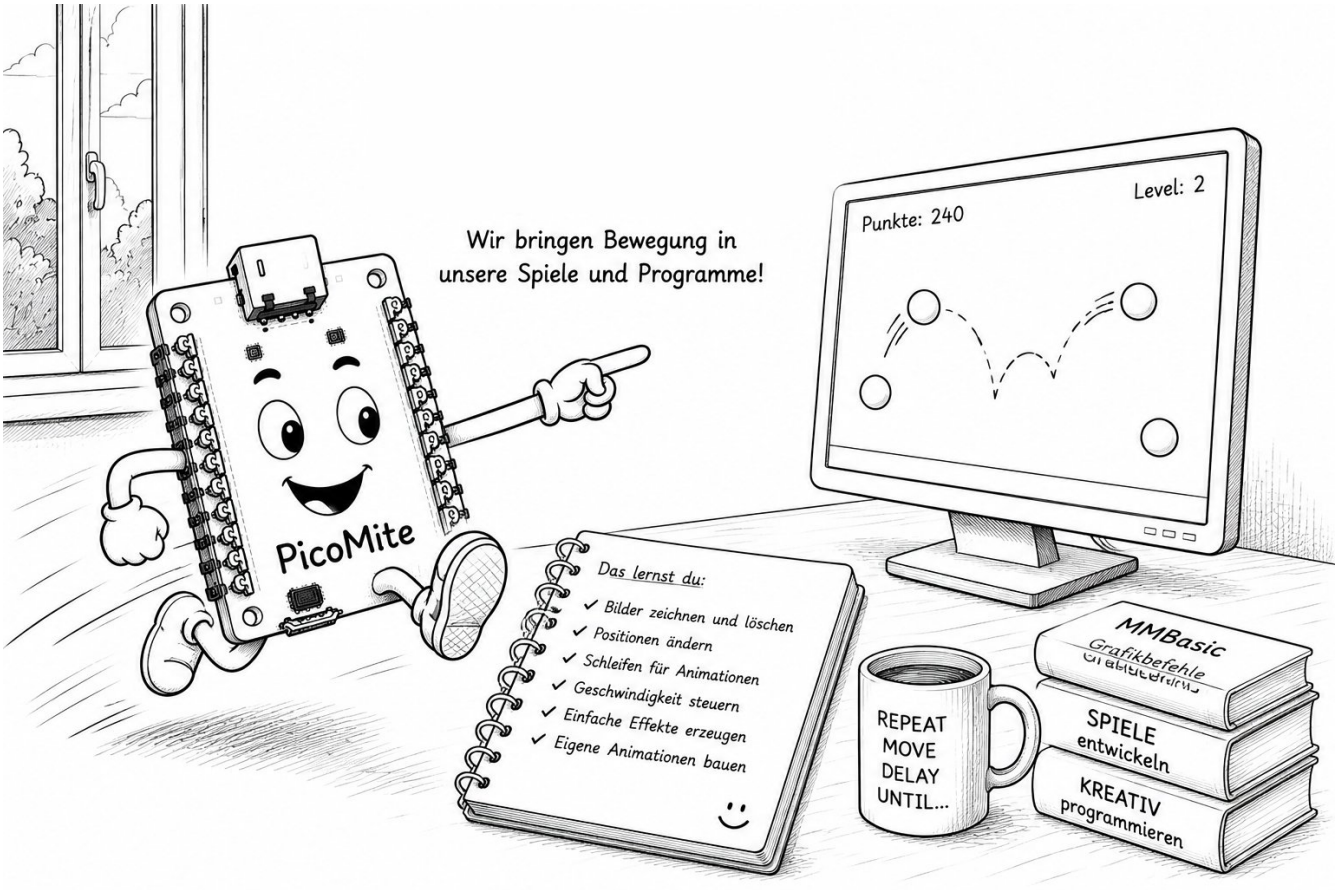
Im nächsten Kapitel werden wir die Grafiken zum Leben erwecken und erste Animationen programmieren.

## 25.19. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 25_spirograph.bas

# Kapitel 26. Animationen und bewegte Grafiken




## 26.1. Das lernst du

In diesem Kapitel lernst du:

- Objekte über den Bildschirm zu bewegen
- Geschwindigkeiten und Bewegungsrichtungen zu verwenden
- Kollisionen mit dem Bildschirmrand zu erkennen
- Animationen mit Schleifen zu erzeugen
- mehrere bewegte Objekte mit Arrays zu verwalten
- zufällige Positionen, Geschwindigkeiten und Farben zu verwenden
- mehrere frühere Positionen für einen Nachzieheffekt zu speichern

Am Ende des Kapitels kannst du eigene kleine Animationen und einfache Bildschirmschoner programmieren.

## 26.2. Von Bildern zu Bewegung

Bisher haben wir Grafiken gezeichnet, die sich nicht verändern.

Eine Animation entsteht dadurch, dass ein Bild immer wieder neu gezeichnet wird. Zwischen zwei Bildern wird die Position eines Objekts verändert.

Der Ablauf sieht dabei vereinfacht so aus:

```
Bild zeichnen
Pause
Bild löschen

Position verändern

Bild neu zeichnen
Pause
Bild löschen

...
```

Werden diese Schritte schnell genug wiederholt, entsteht der Eindruck einer flüssigen Bewegung.

## 26.3. Unser erstes bewegtes Objekt

Beginnen wir mit einem kleinen Ball.

```
MODE 2

X=50
Y=100

DO
  CLS
  CIRCLE X,Y,10,,,RGB(YELLOW),RGB(YELLOW)
  PAUSE 20
  X=X+2
LOOP
```

Der Ball bewegt sich von links nach rechts.

## 26.4. Was passiert hier?

Die Position des Balls wird durch zwei Variablen gespeichert:

```
X
Y
```

`X` bestimmt die horizontale und `Y` die vertikale Position.

Nach jedem Schleifendurchlauf wird `X` erhöht:

```
X=X+2
```

Dadurch verschiebt sich der Ball bei jedem neuen Bild um zwei Pixel nach rechts.

## 26.5. Die Geschwindigkeit ändern

Der Wert `2` bestimmt hier, wie weit der Ball bei jedem Schleifendurchlauf bewegt wird.

```
X=X+5
```

bewegt den Ball deutlich schneller.

```
X=X+1
```

macht ihn langsamer.

Die tatsächliche Geschwindigkeit hängt zusätzlich von der Pause und damit von der Bildwiederholrate ab.

## 26.6. Bewegung in zwei Richtungen

Nun bewegen wir den Ball gleichzeitig nach rechts und nach unten.

```
MODE 2

X=50
Y=50

DO
  CLS
  CIRCLE X,Y,10,,,RGB(CYAN),RGB(CYAN)
  PAUSE 20
  X=X+2
  Y=Y+1
LOOP
```

Jetzt bewegt sich der Ball diagonal über den Bildschirm.

## 26.7. Geschwindigkeiten als Variablen

Statt feste Zahlen zu verwenden, speichern wir die Geschwindigkeit in Variablen.

```
DX=2
DY=1
```

Die Bewegung erfolgt dann über:

```
X=X+DX
Y=Y+DY
```

`DX` steht für die Änderung der X-Position und `DY` für die Änderung der Y-Position. Positive Werte bewegen das Objekt nach rechts beziehungsweise unten, negative Werte nach links beziehungsweise oben.

Das macht das Programm deutlich flexibler, denn die Bewegungsrichtung kann nun einfach durch das Vorzeichen der Geschwindigkeit geändert werden.

## 26.8. Am Bildschirmrand abprallen

Ein Ball soll nicht einfach aus dem sichtbaren Bereich verschwinden.

Trifft er den Rand des Bildschirms, kommt es zu einer Kollisionserkennung: Wir ändern seine Bewegungsrichtung. Für den linken Rand genügt zunächst:

```
IF X<10 THEN DX=-DX
```

Aus `+2` wird dadurch `-2`, aus `-2` entsprechend `+2`. Der Ball bewegt sich anschließend in die Gegenrichtung.

## 26.9. Der erste hüpfende Ball

```
MODE 2

X=100
Y=100
DX=3
DY=2

DO
  CLS
  CIRCLE X,Y,10,,,RGB(YELLOW),RGB(YELLOW)
  PAUSE 15

  X=X+DX
  Y=Y+DY

  IF X<10 THEN X=10: DX=ABS(DX)
```

```
  IF X>MM.HRES-10 THEN X=MM.HRES-10: DX=-ABS(DX)
  IF Y<10 THEN Y=10: DY=ABS(DY)
  IF Y>MM.VRES-10 THEN Y=MM.VRES-10: DY=-ABS(DY)
LOOP
```

Der Ball prallt nun an allen vier Bildschirmrändern ab.

Bei einer Kollision wird nicht nur die Richtung geändert. Die Position wird gleichzeitig auf den zulässigen Randwert gesetzt. Dadurch kann der Ball auch bei größeren Geschwindigkeiten nicht teilweise außerhalb des Bildschirms stehen bleiben.

## 26.10. Warum 10?

Der Ball besitzt einen Radius von 10 Pixeln:

```
CIRCLE X,Y,10
```

`X` und `Y` geben aber den Mittelpunkt des Kreises an. Dieser Mittelpunkt darf deshalb nicht näher als 10 Pixel an einen Bildschirmrand gelangen.

Für den rechten und unteren Rand verwenden wir außerdem die beiden Systemvariablen:

```
MM.HRES
MM.VRES
```

`MM.HRES` enthält die horizontale und `MM.VRES` die vertikale Bildschirmauflösung.

## 26.11. Eine Spur hinterlassen

Entfernen wir das Löschen des Bildschirms:

```
' CLS
```

Dann bleibt jede zuvor gezeichnete Position sichtbar. Auf diese Weise entstehen interessante Muster und Bewegungsspuren.

## 26.12. Ein Farbwechsel bei jeder Kollision

Auch die Farbe eines Objekts kann während einer Animation verändert werden. Eine zufällige Farbe lässt sich zum Beispiel so erzeugen:

```
FARBE=RGB( _
  INT(RND*256), _
  INT(RND*256), _
```

```
  INT(RND*256))
```

Jeder der drei Farbanteile Rot, Grün und Blau erhält dabei einen zufälligen Wert von 0 bis 255.

Wird diese Anweisung innerhalb einer Kollisionsabfrage ausgeführt, erhält der Ball bei jedem Aufprall eine neue Farbe.

# 26.13. Mehrere Bälle gleichzeitig

Für mehrere bewegte Objekte benötigen wir für jedes Objekt eine eigene Position und Geschwindigkeit. Dafür eignen sich Arrays.

```
DIM aX(9)
DIM aY(9)
DIM aDX(9)
DIM aDY(9)
DIM aC(9)
DIM aCB(9)
```

Da die Indizes von `0` bis `9` reichen, können wir damit zehn Bälle verwalten.

Die Namen beginnen hier mit einem kleinen `a`. Das ist keine Vorgabe von MMBasic, sondern dient nur als Hinweis darauf, dass es sich um Arrays handelt.

`aC()` speichert die Randfarbe und `aCB()` die Füllfarbe jedes Balls.

# 26.14. Die Startwerte erzeugen

Jeder Ball erhält eine zufällige Startposition, eine eigene Geschwindigkeit und eigene Farben.

```
FOR I=0 TO 9
  aX(I)=10+INT(RND*(MM.HRES-20))
  aY(I)=10+INT(RND*(MM.VRES-20))

  aDX(I)=INT(RND*5)+1
  aDY(I)=INT(RND*5)+1

  IF RND<0.5 THEN aDX(I)=-aDX(I)
  IF RND<0.5 THEN aDY(I)=-aDY(I)

  aC(I)=RGB(INT(RND*256),INT(RND*256),INT(RND*256))
  aCB(I)=RGB(INT(RND*256),INT(RND*256),INT(RND*256))
NEXT I
```

Die Startposition berücksichtigt bereits den Radius von 10 Pixeln. Dadurch beginnt kein Ball außerhalb des sichtbaren Bereichs.

Mit

```
INT(RND*5)+1
```

entstehen Geschwindigkeiten von 1 bis 5 Pixeln pro Schleifendurchlauf. Anschließend entscheidet eine weitere Zufallszahl, ob das Vorzeichen umgedreht wird:

```
IF RND<0.5 THEN aDX(I)=-aDX(I)
```

Dadurch kann ein Ball von Anfang an sowohl nach rechts als auch nach links fliegen. Dasselbe gilt für die vertikale Richtung.

## 26.15. Zehn hüpfende Bälle

Jetzt setzen wir alles zusammen:

```
MODE 2

DIM aX(9)
DIM aY(9)
DIM aDX(9)
DIM aDY(9)
DIM aC(9)
DIM aCB(9)

FOR I=0 TO 9
  aX(I)=10+INT(RND*(MM.HRES-20))
  aY(I)=10+INT(RND*(MM.VRES-20))

  aDX(I)=INT(RND*5)+1
  aDY(I)=INT(RND*5)+1

  IF RND<0.5 THEN aDX(I)=-aDX(I)
  IF RND<0.5 THEN aDY(I)=-aDY(I)

  aC(I)=RGB(INT(RND*256),INT(RND*256),INT(RND*256))
  aCB(I)=RGB(INT(RND*256),INT(RND*256),INT(RND*256))
NEXT I

DO
  CLS

  FOR I=0 TO 9
    CIRCLE aX(I),aY(I),10,,,aC(I),aCB(I)

    aX(I)=aX(I)+aDX(I)
    aY(I)=aY(I)+aDY(I)
```

```
    IF aX(I)<10 THEN aX(I)=10: aDX(I)=ABS(aDX(I))
    IF aX(I)>MM.HRES-10 THEN aX(I)=MM.HRES-10: aDX(I)=-ABS(aDX(I))
    IF aY(I)<10 THEN aY(I)=10: aDY(I)=ABS(aDY(I))
    IF aY(I)>MM.VRES-10 THEN aY(I)=MM.VRES-10: aDY(I)=-ABS(aDY(I))
  NEXT I

  PAUSE 20
LOOP
```

Nun bewegen sich zehn Bälle in einer Animation unabhängig voneinander. Jeder besitzt eine eigene Position, Geschwindigkeit, Bewegungsrichtung sowie Rand- und Füllfarbe.

## 26.16. Ein kleiner Bildschirmschoner

Das gleiche Prinzip funktioniert nicht nur mit Kreisen. Ersetzen wir die Bälle durch Text, entsteht ein einfacher Bildschirmschoner mit zehn unabhängig bewegten `MMBasic`-Schriftzügen.

Für die Randprüfung gibt es dabei einen wichtigen Unterschied: `X` und `Y` bezeichnen bei `TEXT` die Position des Textes. Deshalb müssen wir seine Breite und Höhe berücksichtigen.

MMBasic stellt mit `MM.FONTWIDTH` und `MM.FONTHEIGHT` die Abmessungen der aktuell verwendeten Schrift in Pixeln bereit. Damit müssen wir die Zeichengröße nicht fest im Programm vorgeben:

```
text$="MMBasic"
textWidth=LEN(text$)*MM.FONTWIDTH
textHeight=MM.FONTHEIGHT
```

Damit können die Startpositionen so gewählt werden, dass der komplette Schriftzug sichtbar ist:

```
aX(I)=INT(RND*(MM.HRES-textWidth+1))
aY(I)=INT(RND*(MM.VRES-textHeight+1))
```

Die Randprüfung sieht dann beispielsweise für die horizontale Bewegung so aus:

```
IF aX(I)<0 THEN aX(I)=0: aDX(I)=ABS(aDX(I))
IF aX(I)>MM.HRES-textWidth THEN _
  aX(I)=MM.HRES-textWidth: aDX(I)=-ABS(aDX(I))
```

Der komplette Bildschirmschoner verwendet wieder die bereits vorhandenen Arrays `aX()`, `aY()`, `aDX()`, `aDY()` und `aC()`.

```
text$="MMBasic"
textWidth=LEN(text$)*MM.FONTWIDTH
textHeight=MM.FONTHEIGHT
```

```
FOR I=0 TO 9
  aX(I)=INT(RND*(MM.HRES-textWidth+1))
  aY(I)=INT(RND*(MM.VRES-textHeight+1))

  aDX(I)=INT(RND*5)+1
  aDY(I)=INT(RND*5)+1

  IF RND<0.5 THEN aDX(I)=-aDX(I)
  IF RND<0.5 THEN aDY(I)=-aDY(I)

  aC(I)=RGB(INT(RND*256),INT(RND*256),INT(RND*256))
NEXT I

DO
  CLS

  FOR I=0 TO 9
    TEXT aX(I),aY(I),text$,,,,aC(I)

    aX(I)=aX(I)+aDX(I)
    aY(I)=aY(I)+aDY(I)

    IF aX(I)<0 THEN aX(I)=0: aDX(I)=ABS(aDX(I))
    IF aX(I)>MM.HRES-textWidth THEN aX(I)=MM.HRES-textWidth: aDX(I)=-ABS(aDX(I))
    IF aY(I)<0 THEN aY(I)=0: aDY(I)=ABS(aDY(I))
    IF aY(I)>MM.VRES-textHeight THEN aY(I)=MM.VRES-textHeight: aDY(I)=-ABS(aDY(I))
  NEXT I

  PAUSE 20
LOOP
```

## 26.17. Ein weiterer Bildschirmschoner

Auch Linien lassen sich animieren. Für jede Linie speichern wir zwei Endpunkte:

```
aX1(I), aY1(I)
aX2(I), aY2(I)
```

Jeder Endpunkt besitzt eine eigene horizontale und vertikale Geschwindigkeit. Dadurch verändern die Linien während der Bewegung ständig ihre Länge und Richtung.

Gezeichnet wird eine Linie mit:

```
LINE aX1(I),aY1(I),aX2(I),aY2(I),1,aC(I)
```

An den Bildschirmrändern wird die Bewegungsrichtung wieder umgekehrt. Im Unterschied zu den Bällen wird beim Aufprall zusätzlich eine neue zufällige Geschwindigkeit zwischen 1 und 5

gewählt:

```
IF aX1(I)<0 THEN aX1(I)=0: aDX1(I)=INT(RND*5)+1
IF aX1(I)>MM.HRES-1 THEN _
  aX1(I)=MM.HRES-1: aDX1(I)=-(INT(RND*5)+1)
```

Da beide Endpunkte unabhängig voneinander bewegt werden, entstehen ständig wechselnde geometrische Muster.

Das vollständige Beispiel findest du ebenfalls in `26_animation.bas`.

# 26.18. Abschlussprojekt: Mystify – ein klassischer Bildschirmschoner

Zum Abschluss dieses Kapitels bauen wir einen Bildschirmschoner nach, der an den bekannten Windows-Bildschirmschoner **Mystify** erinnert. Dieses Beispiel ist bewusst umfangreicher als die bisherigen Programme. Es führt viele Techniken zusammen, die du in diesem Kapitel kennengelernt hast.

Du musst dabei nicht jede einzelne Programmzeile sofort auswendig verstehen. Wichtig ist zunächst zu erkennen, wie die einzelnen Bausteine zusammenspielen.

Unser Mystify-Programm zeichnet zwei vierseitige Figuren. Die vier Eckpunkte jeder Figur bewegen sich unabhängig voneinander über den Bildschirm und prallen an dessen Rändern ab. Zusätzlich bleiben mehrere ältere Positionen der Figur sichtbar. Dadurch entsteht der typische Nachzieheffekt.

## 26.18.1. Die wichtigsten Einstellungen

Das Abschlussprojekt verwendet zunächst `OPTION BASE 0`. Damit ist ausdrücklich festgelegt, dass die Arrayindizes bei `0` beginnen:

```
OPTION BASE 0
```

Anschließend legen wir mit `CONST` fünf Konstanten fest:

```
CONST cFigures   = 2
CONST cTrail     = 5
CONST cSpeed     = 10
CONST cDelay     = 50
CONST cBackColor = RGB(BLACK)
```

`cFigures` bestimmt die Anzahl der gleichzeitig bewegten Figuren. `cTrail` gibt an, wie viele Positionen einer Figur sichtbar bleiben. `cSpeed` legt die maximale Geschwindigkeit eines Eckpunktes fest. Mit `cDelay` bestimmen wir die Pause zwischen zwei Bildern und `cBackColor` enthält die

Hintergrundfarbe.

Das Programm wählt abhängig von der verwendeten PicoMite-Variante MODE 3 oder MODE 5 und löscht anschließend den Bildschirm mit der festgelegten Hintergrundfarbe:

```
If UCase$(Left$(MM.Info$(DEVICE),7))="WEBMITE" Then
  Mode 5
Else
  Mode 3
EndIf
CLS cBackColor
```

Durch die Konstanten können wir das Verhalten des Bildschirmschoners später leicht verändern.

### 26.18.2. Vier bewegte Eckpunkte

Eine Figur besteht aus vier Eckpunkten. Für jeden Punkt benötigen wir eine X- und eine Y-Koordinate. Zusätzlich speichern wir nicht nur die aktuelle Position, sondern auch frühere Positionen:

```
DIM aX1(cFigures-1,cTrail), aY1(cFigures-1,cTrail)
DIM aX2(cFigures-1,cTrail), aY2(cFigures-1,cTrail)
DIM aX3(cFigures-1,cTrail), aY3(cFigures-1,cTrail)
DIM aX4(cFigures-1,cTrail), aY4(cFigures-1,cTrail)
```

Hier verwenden wir zweidimensionale Arrays. Der erste Index bezeichnet die Figur. Weil `cFigures=2` ist und die Indizes bei `0` beginnen, reichen die Figurenindizes von `0` bis `cFigures-1`, also von `0` bis `1`.

Der zweite Index bezeichnet die Position im Nachziehspeicher. Dabei hat er eine besondere Bedeutung:

- Index `0` enthält die aktuelle Position.
- Die Indizes `1` bis `cTrail-1` enthalten die sichtbaren älteren Positionen.
- Index `cTrail` enthält die älteste Position, die im nächsten Bild gelöscht wird.

Bei `cTrail=5` sind damit fünf Positionen sichtbar. Der zusätzliche Arrayplatz mit dem Index `5` wird zum Löschen der ältesten Position benötigt.

Zum Beispiel bezeichnet

```
aX1(0,0)
```

die aktuelle X-Koordinate des ersten Eckpunktes der ersten Figur. `aX1(0,1)` enthält dagegen dessen vorherige Position.

Auch die Farbe wird für jede gespeicherte Position abgelegt:

```
DIM aC(cFigures-1,cTrail)
```

Dadurch kann ein Farbwechsel zusammen mit der Figur durch den Nachzieheffekt wandern.

Die Geschwindigkeiten müssen wir nur für die aktuellen Eckpunkte speichern:

```
DIM aDX1(cFigures-1), aDY1(cFigures-1)
DIM aDX2(cFigures-1), aDY2(cFigures-1)
DIM aDX3(cFigures-1), aDY3(cFigures-1)
DIM aDX4(cFigures-1), aDY4(cFigures-1)
```

Die älteren Positionen werden nur noch gezeichnet. Sie bewegen sich nicht mehr und benötigen deshalb keine eigenen Geschwindigkeiten.

### 26.18.3. Die Startwerte

Für jede Figur werden zunächst zufällige Startpositionen für alle vier Eckpunkte erzeugt. Auch die Geschwindigkeiten und Bewegungsrichtungen werden zufällig gewählt.

Die erste Farbe wird so erzeugt:

```
aC(i,0)=RGB(INT(RND*200)+55, _
            INT(RND*200)+55, _
            INT(RND*200)+55)
```

Durch `+55` werden sehr dunkle Farbanteile vermieden. Anschließend werden alle Plätze des Nachziehspeichers mit derselben Startposition und derselben Farbe gefüllt:

```
FOR j=1 TO cTrail
  aX1(i,j)=aX1(i,0) : aY1(i,j)=aY1(i,0)
  aX2(i,j)=aX2(i,0) : aY2(i,j)=aY2(i,0)
  aX3(i,j)=aX3(i,0) : aY3(i,j)=aY3(i,0)
  aX4(i,j)=aX4(i,0) : aY4(i,j)=aY4(i,0)
  aC(i,j)=aC(i,0)
NEXT j
```

So enthält der Speicher beim Start keine uninitialisierten alten Positionen.

### 26.18.4. Aus vier Punkten wird eine Figur

Die vier Punkte verbinden wir mit vier Linien:

```
LINE aX1(i,j),aY1(i,j),aX2(i,j),aY2(i,j),1,col
```

```
LINE aX2(i,j),aY2(i,j),aX3(i,j),aY3(i,j),1,col
LINE aX3(i,j),aY3(i,j),aX4(i,j),aY4(i,j),1,col
LINE aX4(i,j),aY4(i,j),aX1(i,j),aY1(i,j),1,col
```

Die letzte Linie verbindet den vierten Punkt wieder mit dem ersten und schließt die Figur. Da sich alle vier Eckpunkte unabhängig bewegen, ändert sich ihre Form ständig.

### 26.18.5. Zeichnen und die älteste Figur löschen

Anders als bei den bisherigen Beispielen löschen wir den Bildschirm nicht in jedem Schleifendurchlauf mit `CLS`. Stattdessen zeichnen wir zuerst die älteste gespeicherte Position in der Hintergrundfarbe. Dadurch wird nur diese eine alte Figur entfernt:

```
FOR j=cTrail TO 0 STEP -1
  IF j=cTrail THEN
    col=cBackColor
  ELSE
    col=aC(i,j)
  ENDIF

  ... Figur mit col zeichnen ...
NEXT j
```

Die Schleife läuft vom ältesten zum neuesten Eintrag. Der Index `cTrail` wird in der Hintergrundfarbe gezeichnet und damit gelöscht. Die Indizes `0` bis `cTrail-1` werden dagegen in ihren gespeicherten Farben gezeichnet.

### 26.18.6. Der Nachzieheffekt

Nach dem Zeichnen werden die gespeicherten Positionen um einen Platz nach hinten verschoben:

```
FOR j=cTrail TO 1 STEP -1
  aX1(i,j)=aX1(i,j-1)
  aY1(i,j)=aY1(i,j-1)
  ...
  aC(i,j)=aC(i,j-1)
NEXT j
```

Die Schleife läuft absichtlich rückwärts. Würden wir bei 1 beginnen, würden wir einen Wert überschreiben, bevor er an die nächste Position kopiert wurde.

Dabei wird nicht nur die Position, sondern auch die zugehörige Farbe weitergeschoben. Nach dem Verschieben bleibt Index `0` für die neue aktuelle Position reserviert; die höheren Indizes enthalten immer ältere Zustände der Figur.

Das Grundprinzip lautet damit:

```
älteste Position löschen
aktuelle und ältere Positionen zeichnen
Positionen und Farben im Speicher verschieben
Eckpunkte bewegen
```

### 26.18.7. Bewegung und Randkollision

Die aktuellen Eckpunkte werden genauso bewegt wie die Bälle und Linien aus den vorherigen Beispielen:

```
aX1(i,0)=aX1(i,0)+aDX1(i)
aY1(i,0)=aY1(i,0)+aDY1(i)
```

Beim Erreichen eines Bildschirmrandes wird der Punkt auf den zulässigen Rand gesetzt und erhält eine neue Geschwindigkeit in die entgegengesetzte Richtung:

```
IF aX1(i,0)<0 THEN
  aX1(i,0)=0
  aDX1(i)=INT(RND*cSpeed)+1
ENDIF

IF aX1(i,0)>MM.HRES-1 THEN
  aX1(i,0)=MM.HRES-1
  aDX1(i)=-(INT(RND*cSpeed)+1)
ENDIF
```

Dadurch ändert sich beim Aufprall nicht nur die Bewegungsrichtung, sondern auch die Geschwindigkeit. Das sorgt dafür, dass die Animation weniger regelmäßig wirkt.

### 26.18.8. Zufällige Farbwechsel

Jede aktuelle Figur besitzt eine Farbe in `aC(i,0)`. Gelegentlich wird eine neue Farbe erzeugt:

```
IF INT(RND*100)=0 THEN
  aC(i,0)=RGB(INT(RND*200)+55, _
              INT(RND*200)+55, _
              INT(RND*200)+55)
ENDIF
```

Die Bedingung ist nur selten erfüllt. Deshalb bleibt eine Farbe eine Weile erhalten, bevor sie wechselt. Weil die Farbe zusammen mit den Positionen im Array nach hinten geschoben wird, läuft auch der Farbwechsel durch den Nachzieheffekt.

### 26.18.9. Die Hauptschleife

Die Animation läuft so lange, bis eine Taste gedrückt wird:

```
DO
  ...
  PAUSE cDelay
LOOP WHILE INKEY$=""
```

Mit `cDelay=50` wartet das Programm zwischen zwei Bildern 50 Millisekunden. Der Bildschirmschoner kann jederzeit durch einen Tastendruck beendet werden.

### 26.18.10. Was kommt hier alles zusammen?

Im Mystify-Projekt verwenden wir fast alle Techniken dieses Kapitels gleichzeitig:

- X- und Y-Koordinaten für bewegte Punkte
- getrennte Geschwindigkeiten für beide Richtungen
- positive und negative Bewegungsrichtungen
- Kollisionen mit allen vier Bildschirmrändern
- Zufallszahlen für Positionen, Geschwindigkeiten und Farben
- ein- und zweidimensionale Arrays
- mehrere gleichzeitig bewegte Figuren
- Liniengrafik
- gespeicherte ältere Positionen für einen Nachzieheffekt
- Beenden einer Animation mit `INKEY$`

Das vollständige Programm findest du in `26_animation_2.bas`. Probiere anschließend ruhig andere Werte für `cFigures`, `cTrail`, `cSpeed` und `cDelay` aus. Schon kleine Änderungen erzeugen deutlich andere Muster.

## 26.19. Das DVD-Logo

Vielleicht kennst du den bekannten Bildschirmschoner mit dem DVD-Logo. Ein Schriftzug bewegt sich über den Bildschirm und prallt an den Rändern ab.

Mit dem Wissen aus diesem Kapitel kannst du ihn bereits selbst programmieren. Besonders wichtig ist dabei die Randprüfung aus dem vorherigen Beispiel: Die Breite und Höhe des Logos müssen berücksichtigt werden, damit es vollständig sichtbar bleibt.

## 26.20. Warum flackert das Bild?

Unsere Animationen beginnen jeden Schleifendurchlauf mit:

```
CLS
```

Dadurch wird der Bildschirm ständig gelöscht und anschließend vollständig neu aufgebaut. Dieses Verfahren kann bei einer Animation zu sichtbarem Flackern führen. Je nach Hardware und Umfang der Grafik kann das zu sichtbarem Flackern führen.

MMBasic unterstützt Framebuffer.

Damit kann das nächste Bild zunächst unsichtbar aufgebaut und anschließend in einem Schritt angezeigt werden. Dadurch lassen sich Animationen deutlich flüssiger darstellen.

Dieses Thema werden wir in einem späteren Kapitel behandeln.

## 26.21. Experimentiere!

Probiere folgende Änderungen aus:

- Verändere die Ballgröße.
- Verwende andere oder wechselnde Farben.
- Erhöhe die Anzahl der Bälle.
- Lasse eine Spur sichtbar.
- Verändere die Geschwindigkeiten.
- Ändere den Text des Bildschirmschoners.
- Verwende mehr oder weniger bewegte Linien.

## 26.22. Probier’s selbst!

Versuche folgende Aufgaben:

1. Programmiere einen DVD-Bildschirmschoner.
2. Lasse beim DVD-Bildschirmschoner bei jeder Kollision die Farbe wechseln.
3. Erzeuge 20 statt 10 Bälle.
4. Lasse die Bälle ihre Größe ändern.
5. Zeichne statt Kreisen kleine Sterne.
6. Ergänze eine Anzeige der aktuellen Anzahl von Kollisionen.
7. Verändere beim Linien-Bildschirmschoner die Geschwindigkeit nach jedem Aufprall.

## 26.23. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Objekte durch wiederholtes Zeichnen zu bewegen
- Geschwindigkeiten und Bewegungsrichtungen zu verwenden
- die Größe eines Objekts bei der Randprüfung zu berücksichtigen
- Bildschirmränder zu erkennen und Kollisionen auszuwerten
- mehrere Objekte mit Arrays gleichzeitig zu animieren
- zufällige Positionen, Richtungen, Geschwindigkeiten und Farben zu erzeugen
- einfache Text- und Linien-Bildschirmschoner zu programmieren
- frühere Positionen in zweidimensionalen Arrays für einen Nachzieheffekt zu speichern
- mehrere Animationstechniken in einem größeren Abschlussprojekt zu kombinieren

Damit hast du einen wichtigen Schritt gemacht: Aus statischen Bildern sind erstmals lebendige Programme geworden.

Im nächsten Kapitel werden wir lernen, wie Animationen mit Hilfe von Framebuffern deutlich flüssiger dargestellt werden können.

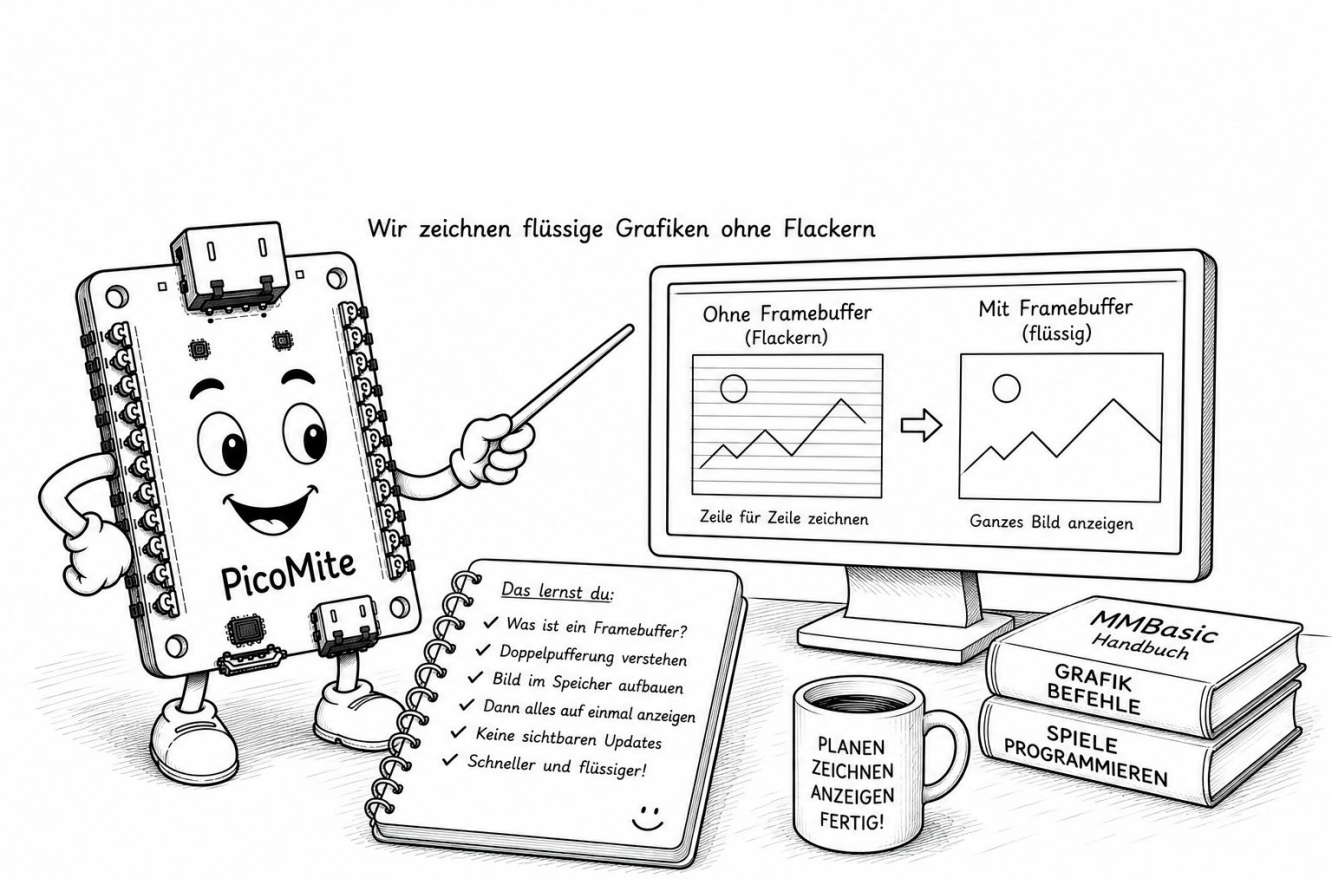
## 26.24. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 26_animation.bas
- 26_animation_2.bas – Abschlussprojekt Mystify

Im Kapitel verwenden die kurzen Einstiegsbeispiele bewusst Endlosschleifen mit `DO ⋯ LOOP`, damit das jeweilige Prinzip möglichst einfach zu erkennen ist. Das vollständige Programm verwendet dagegen begrenzte `FOR ⋯ NEXT`-Schleifen, damit die einzelnen Demonstrationen nacheinander ausgeführt werden können.

# Kapitel 27. Flackerfreie Grafik mit Framebuffer




## 27.1. Das lernst du

In diesem Kapitel lernst du:

- was ein Framebuffer ist
- warum Animationen manchmal flackern
- Grafiken unsichtbar im Speicher aufzubauen
- fertige Bilder auf den Bildschirm zu übertragen
- bestehende Programme zu verbessern
- die Grundlage moderner Grafiktechniken kennenzulernen

Am Ende des Kapitels kannst du Animationen und Grafikanzeigen deutlich professioneller gestalten.

## 27.2. Warum flackern Animationen?

Im vorherigen Kapitel haben wir bewegte Grafiken programmiert.

Dazu wurde der Bildschirm ständig gelöscht und neu aufgebaut.

```
CLS

CIRCLE X,Y,10,,,RGB(YELLOW),RGB(YELLOW)
```

Dieser Vorgang wiederholt sich viele Male pro Sekunde.

Je nach Grafikmodus und Prozessorgeschwindigkeit kann dabei ein leichtes Flackern sichtbar werden.

Besonders bei komplexeren Grafiken fällt dieser Effekt auf.

## 27.3. Die Idee hinter einem Framebuffer

Stell dir vor, du möchtest ein Bild zeichnen.

Ohne Framebuffer zeichnest du direkt auf ein Blatt Papier.

Jeder Strich ist sofort sichtbar.

Der Betrachter sieht jeden einzelnen Arbeitsschritt.

Mit einem Framebuffer verwendest du zunächst ein zweites Blatt Papier. Dieses Verfahren ermöglicht flackerfreie Grafik.

Auf diesem Blatt wird das komplette Bild aufgebaut.

Erst wenn alles fertig ist, wird das Bild gezeigt.

Der Betrachter sieht niemals die einzelnen Zeichenschritte.

## 27.4. Framebuffer und Bildschirm

Normalerweise werden Grafikbefehle direkt auf den Bildschirm ausgegeben.

```
PIXEL
LINE
CIRCLE
BOX
TEXT
```

zeichnen unmittelbar auf dem sichtbaren Bildschirm.

Mit einem Framebuffer können dieselben Befehle stattdessen auf eine unsichtbare Zeichenfläche umgeleitet werden.

## 27.5. Einen Framebuffer erzeugen

Zuerst wird ein Framebuffer mit `FRAMEBUFFER CREATE` angelegt.

```
FRAMEBUFFER CREATE
```

Der PicoMite reserviert nun Speicher für eine zusätzliche Bildfläche.

## 27.6. In den Framebuffer zeichnen

Mit `FRAMEBUFFER WRITE` werden alle Grafikbefehle in den gewünschten Ausgabepuffer umgeleitet.

```
FRAMEBUFFER WRITE F
```

Das `F` steht für:

```
F = Framebuffer
```

Ab jetzt werden alle Grafiken unsichtbar aufgebaut.

## 27.7. Unser erstes Beispiel

Das folgende Programm zeichnet 25000 zufällige Punkte.

```
If UCase$(Left$(MM.Info$(DEVICE),7))="WEBMITE" Then
  Mode 5
Else
  Mode 3
EndIf

CLS RGB(BLACK)

FRAMEBUFFER CREATE

FRAMEBUFFER WRITE F

FOR I=1 TO 25000

  X=INT(RND*MM.HRES)
  Y=INT(RND*MM.VRES)

  PIXEL X,Y,RGB(YELLOW)

NEXT I
```

Was passiert?

Nichts.

Der Bildschirm bleibt schwarz.

## 27.8. Warum bleibt der Bildschirm leer?

Die Punkte wurden tatsächlich gezeichnet.

Allerdings befinden sie sich im Framebuffer.

Der Inhalt wurde noch nicht auf den Bildschirm übertragen.

## 27.9. Das Bild anzeigen

Mit `FRAMEBUFFER COPY` wird der Inhalt des Framebuffers auf den Bildschirm kopiert.

```
FRAMEBUFFER COPY F,N
```

Dabei bedeutet:

| Code | Bedeutung |
|---|---|
| F | Framebuffer |
| N | Normaler Bildschirm |

Erweitern wir das Programm:

```
If UCase$(Left$(MM.Info$(DEVICE),7))="WEBMITE" Then
  Mode 5
Else
  Mode 3
EndIf

CLS RGB(BLACK)

FRAMEBUFFER CREATE

FRAMEBUFFER WRITE F

FOR I=1 TO 25000

  X=INT(RND*MM.HRES)
  Y=INT(RND*MM.VRES)

  PIXEL X,Y,RGB(YELLOW)
```

```
NEXT I

FRAMEBUFFER COPY F,N
```

Nun erscheinen alle Punkte gleichzeitig.

## 27.10. Wieder auf den Bildschirm zeichnen

Nach dem Arbeiten mit dem Framebuffer möchten wir oft wieder direkt auf den Bildschirm zeichnen.

Dazu verwenden wir:

```
FRAMEBUFFER WRITE N
```

Das `N` steht für den normalen Ausgabebildschirm.

## 27.11. Ein komplettes Bild vorbereiten

Der eigentliche Vorteil eines Framebuffers zeigt sich bei komplexen Grafiken.

Das folgende Programm zeichnet mehrere Objekte.

```
FRAMEBUFFER CREATE

FRAMEBUFFER WRITE F

CLS RGB(BLACK)

FOR I=1 TO 100

  X=INT(RND*MM.HRES)
  Y=INT(RND*MM.VRES)

  CIRCLE X,Y,10,,,RGB(CYAN)

NEXT I

TEXT MM.HRES/2,20,     _
     "Framebuffer Demo", _
     "CT"

FRAMEBUFFER COPY F,N
```

Der Betrachter sieht sofort das fertige Bild.

## 27.12. Die Bahnhofsuhr verbessern

Im Kapitel "Eine analoge Bahnhofsuhr" wurde die Uhr jede Sekunde vollständig neu gezeichnet.

Der Ablauf sah vereinfacht so aus:

```
CLS

ZeichneZifferblatt

ZeichneStundenzeiger

ZeichneMinutenzeiger

ZeichneSekundenzeiger
```

Je nach Hardware konnte dabei ein leichtes Flackern sichtbar werden.

Mit einem Framebuffer lässt sich die Uhr zunächst unsichtbar aufbauen.

Erst wenn alle Elemente fertig sind, wird das fertige Bild angezeigt.

## 27.13. Die Uhr mit Framebuffer

Der neue Ablauf lautet:

```
FRAMEBUFFER WRITE F

CLS RGB(WHITE)

ZeichneZifferblatt

ZeichneStundenzeiger

ZeichneMinutenzeiger

ZeichneSekundenzeiger

FRAMEBUFFER COPY F,N
```

Das Ergebnis wirkt deutlich ruhiger. Das komplette Basic Programm befindet sich im Programmverzeichnis:

- Flackerfreie Bahnhofsuhr (27_framebuffer_5.bas)

## 27.14. Ein animierter Ball mit Framebuffer

Auch Animationen profitieren von dieser Technik.

```
DO

  FRAMEBUFFER WRITE F

  CLS RGB(BLACK)

  CIRCLE X,Y,10,,,RGB(YELLOW),RGB(YELLOW)

  FRAMEBUFFER COPY F,N

  X=X+DX
  Y=Y+DY

LOOP
```

Der Ball wird zunächst unsichtbar gezeichnet.

Anschließend erscheint das fertige Bild auf dem Bildschirm.

## 27.15. Den Framebuffer wieder freigeben

Wird der Framebuffer nicht mehr benötigt, sollte der Speicher mit `FRAMEBUFFER CLOSE` freigegeben werden.

```
FRAMEBUFFER CLOSE
```

Dadurch steht der Speicher wieder für Programme und Variablen zur Verfügung.

Im Programmverzeichnis finden sich noch weitere Beispiels von Animationen:

- 10.000 Punkte (27_framebuffer_1.bas)
- Animation eines Balles v1.0 (27_framebuffer_2.bas)
- Animation eines Balles v2.0 (27_framebuffer_3.bas)
- Animation eines Balles v3.0 (27_framebuffer_4.bas)

Der Colour Maximite 2 besitzt ebenfalls Framebuffer-Funktionen.

Dort werden Framebuffer häufig als virtuelle Zeichenflächen verwendet und bieten zusätzliche Möglichkeiten wie große Scrollbereiche und Hintergrundbilder.

Diese Funktionen werden wir in späteren Kapiteln kennenlernen.

## 27.16. Framebuffer und Spiele

Für viele Spiele ist ein Framebuffer die Grundlage einer flüssigen Darstellung.

Typische Anwendungen sind:

- Animationen
- Spiele
- Grafische Menüs
- Uhren
- Messwertanzeigen
- Bildschirmschoner

## 27.17. Ausblick: Layerbuffer

PicoMite HDMI und VGA unterstützen zusätzlich sogenannte Layerbuffer.

Dabei können bewegliche Objekte auf einer separaten Ebene dargestellt werden.

Der Hintergrund muss dadurch nicht ständig neu gezeichnet werden.

Diese Technik ist besonders für Spiele interessant.

Wir werden sie im nächsten Kapitel kennenlernen.

## 27.18. Experimentiere!

Probiere folgende Änderungen aus:

- Zeichne 5000 Punkte.
- Erzeuge zufällige Kreise.
- Zeichne mehrere Texte.
- Verwende unterschiedliche Hintergrundfarben.
- Baue die Bahnhofsuhr auf einen Framebuffer um.

## 27.19. Probier’s selbst!

Versuche folgende Aufgaben:

1. Erweitere den hüpfenden Ball um Farbwechsel.
2. Zeichne mehrere Bälle gleichzeitig.
3. Erzeuge einen einfachen Bildschirmschoner.
4. Baue eine bestehende Grafikdemo auf Framebuffer um.

## 27.20. Zusammenfassung

In diesem Kapitel hast du gelernt:

- was ein Framebuffer ist
- Grafiken unsichtbar aufzubauen
- fertige Bilder auf den Bildschirm zu übertragen
- Flackern zu reduzieren
- Animationen professioneller darzustellen

Framebuffer gehören zu den wichtigsten Werkzeugen moderner Grafikprogrammierung.

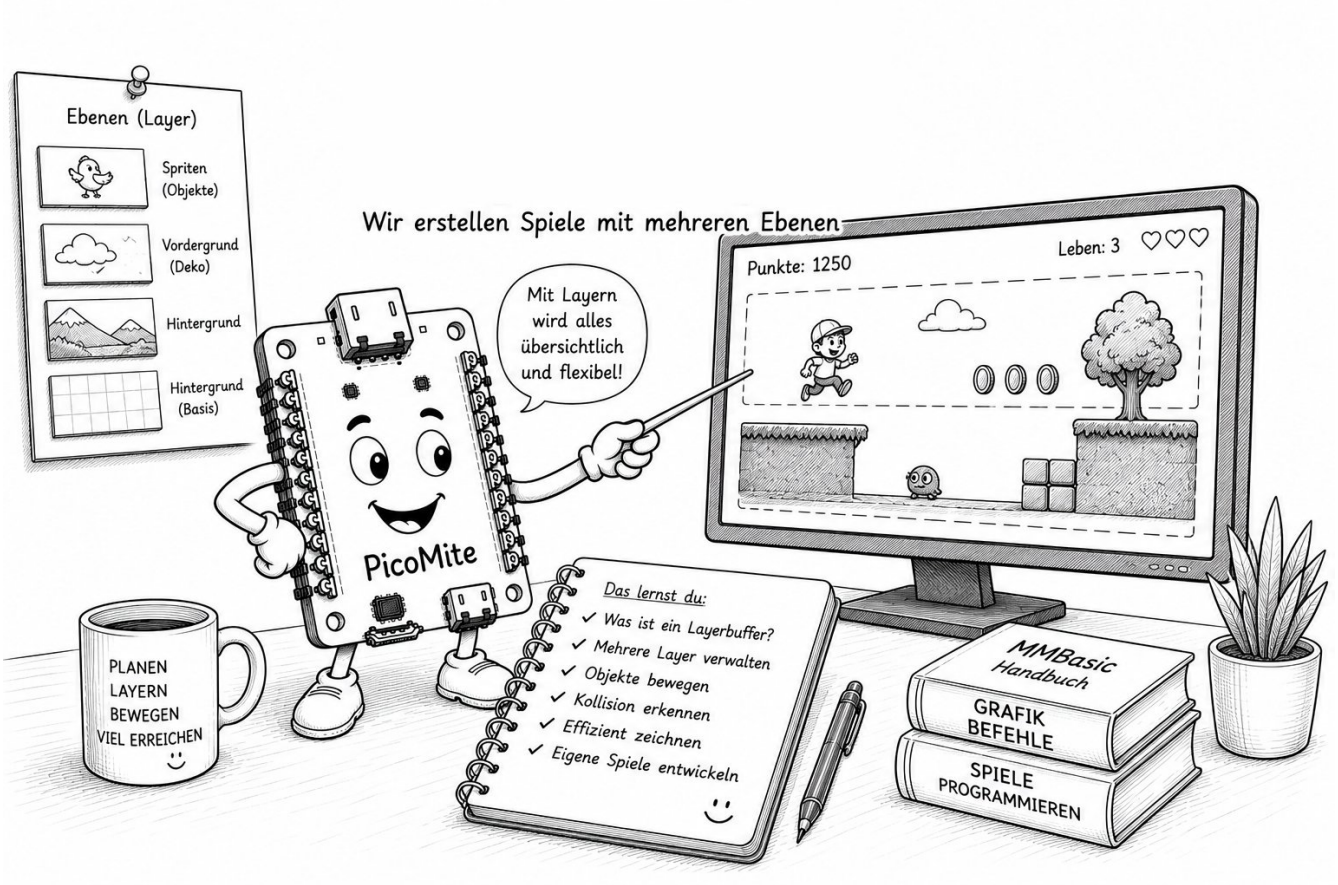
Im nächsten Kapitel werden wir Layerbuffer kennenlernen und bewegliche Objekte noch effizienter darstellen.

## 27.21. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 27_framebuffer_demo.bas
- 27_framebuffer_1.bas
- 27_framebuffer_2.bas
- 27_framebuffer_3.bas
- 27_framebuffer_4.bas
- 27_framebuffer_5.bas

# Kapitel 28. Layerbuffer und bewegliche Objekte




## 28.1. Das lernst du

In diesem Kapitel lernst du:

- was ein Layerbuffer ist
- wie sich Layerbuffer von einem Framebuffer unterscheiden
- bewegliche Objekte auf einer separaten Ebene darzustellen
- statische Hintergründe effizient zu verwenden
- die Grundlage moderner Spieleprogrammierung kennenzulernen

Am Ende des Kapitels kannst du Animationen erstellen, ohne den gesamten Hintergrund ständig neu zeichnen zu müssen.

## 28.2. Das Problem

Im vorherigen Kapitel haben wir einen Framebuffer verwendet.

Der Ablauf war:

```
Hintergrund zeichnen
Objekte zeichnen

Bild anzeigen

Alles löschen

Hintergrund neu zeichnen
Objekte neu zeichnen
```

Das funktioniert gut.

Allerdings wird auch der Hintergrund immer wieder neu aufgebaut.

## 28.3. Ein Beispiel

Stell dir vor, du hast einen Hintergrund mit:

- Sternenhimmel
- Landschaft
- Spielfeld

und darauf bewegt sich nur ein kleiner Ball.

Dann wäre es eigentlich unnötig, den gesamten Hintergrund ständig neu zu zeichnen.

## 28.4. Die Idee eines Layerbuffers

Ein Layerbuffer ist eine zusätzliche Bildebene.

Man kann sich das wie zwei transparente Folien vorstellen.

```
Ebene 1
Hintergrund

Ebene 2
Ball
Spieler
Gegner
```

Der statische Hintergrund bleibt unverändert.

Nur die beweglichen Objekte auf der zweiten Ebene verändern sich.

## 28.5. Framebuffer und Layerbuffer

| Technik | Vorteil | Nachteil |
|---|---|---|
| Framebuffer | Einfach zu verstehen | Komplettes Bild wird neu aufgebaut |
| Layerbuffer | Sehr effizient | Etwas komplexer |

## 28.6. Einen Layerbuffer erzeugen

Zuerst wird ein Layerbuffer mit `FRAMEBUFFER LAYER` angelegt.

```
FRAMEBUFFER LAYER
```

Danach steht eine zusätzliche Zeichenebene zur Verfügung.

## 28.7. Auf den Layerbuffer zeichnen

Mit

```
FRAMEBUFFER WRITE L
```

werden mit `FRAMEBUFFER WRITE L` alle Grafikbefehle auf den Layerbuffer umgeleitet.

Das `L` steht für:

```
L = Layerbuffer
```

## 28.8. Wieder auf den Bildschirm zeichnen

```
FRAMEBUFFER WRITE N
```

stellt mit `FRAMEBUFFER WRITE N` die normale Ausgabe wieder her.

## 28.9. Unser erster Hintergrund

Zunächst zeichnen wir einen statischen Sternenhimmel.

```
CLS RGB(BLACK)

FOR I=1 TO 500

  X=INT(RND*MM.HRES)
```

```
  Y=INT(RND*MM.VRES)

  PIXEL X,Y,RGB(WHITE)

NEXT I
```

Dieser Hintergrund bleibt später unverändert.

## 28.10. Einen Ball auf dem Layerbuffer zeichnen

```
FRAMEBUFFER LAYER

FRAMEBUFFER WRITE L

CIRCLE 100,100,10,,,RGB(YELLOW),RGB(YELLOW)
```

Der Ball befindet sich nun auf der zweiten Ebene.

## 28.11. Hintergrund und Layer

Auf HDMI- und VGA-Systemen wird der Layerbuffer automatisch über den Hintergrund gelegt.

Der Ball erscheint dadurch vor dem Sternenhimmel.

## 28.12. Das Prinzip einer Animation

Der Hintergrund wird nur einmal erzeugt.

Die Bewegung betrifft ausschließlich den Layerbuffer.

```
Hintergrund
einmal zeichnen

Layer
ständig aktualisieren
```

## 28.13. Ein beweglicher Ball

```
CLS RGB(BLACK)

FOR I=1 TO 500

  PIXEL INT(RND*MM.HRES), _
        INT(RND*MM.VRES), _
        RGB(WHITE)
```

```
NEXT I

FRAMEBUFFER LAYER

X=100
Y=100

DX=3
DY=2

DO

  FRAMEBUFFER WRITE L

  CLS

  CIRCLE X,Y,10,,,RGB(YELLOW),RGB(YELLOW)

  X=X+DX
  Y=Y+DY

  IF X<10 THEN DX=-DX
  IF X>MM.HRES-10 THEN DX=-DX

  IF Y<10 THEN DY=-DY
  IF Y>MM.VRES-10 THEN DY=-DY

  PAUSE 20

LOOP
```

## 28.14. Warum wird nur der Layer gelöscht?

Der Befehl

```
CLS
```

wirkt innerhalb von

```
FRAMEBUFFER WRITE L
```

nur auf den Layerbuffer.

Der Hintergrund bleibt erhalten.

## 28.15. Ein Raumschiff über Sternenhimmel

Jetzt ersetzen wir den Ball durch ein kleines Raumschiff.

```
LINE X,Y-10,X-8,Y+8

LINE X,Y-10,X+8,Y+8

LINE X-8,Y+8,X+8,Y+8
```

Das Dreieck bewegt sich nun über den Sternenhimmel.

## 28.16. Mehrere Objekte

Natürlich können auch mehrere Objekte auf dem Layer gezeichnet werden.

```
FOR I=0 TO 9

  CIRCLE X(I),Y(I),8,,,RGB(CYAN)

NEXT I
```

Dadurch entstehen sehr flüssige Animationen.

## 28.17. Warum verwenden Spiele Layer?

Viele Spiele bestehen aus:

- Hintergrund
- Spielfigur
- Gegner
- Geschosse
- Statusanzeigen

Der Hintergrund ändert sich selten.

Die beweglichen Objekte ändern sich ständig.

Layerbuffer ermöglichen genau diese Trennung von Hintergrund und beweglichen Objekten.

## 28.18. Ein einfaches Spielfeld

Stell dir ein Pong-Spiel vor.

```
+------------------+

      0

 |                |

+------------------+
```

Das Spielfeld bleibt unverändert.

Nur Ball und Schläger bewegen sich.

Layerbuffer sind dafür ideal.

## 28.19. HDMI- und VGA-Systeme

Layerbuffer werden automatisch unterstützt in:

- HDMI Mode 2
- HDMI Mode 3
- HDMI Mode 4
- HDMI Mode 5
- VGA Mode 2
- VGA Mode 3 (RP2350)

Je nach Hardware können sogar mehrere Layer gleichzeitig verwendet werden.

## 28.20. Colour Maximite 2

Der Colour Maximite 2 verwendet ein anderes Framebuffer-Konzept.

Dort werden häufig virtuelle Bildschirme verwendet, die größer als die sichtbare Anzeige sein können.

Techniken wie Scrolling und große Karten lassen sich damit sehr elegant umsetzen.

## 28.21. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende mehrere Bälle.
- Zeichne einen Sternenhimmel mit 1000 Sternen.
- Verändere die Ballgröße.

- Erzeuge zufällige Farben.
- Verwende eigene Objekte statt Kreisen.

## 28.22. Probier’s selbst!

Versuche folgende Aufgaben:

1. Programmiere ein kleines Raumschiff.
2. Erzeuge einen Meteoritenschauer.
3. Zeichne mehrere Gegner.
4. Lasse Objekte unterschiedlich schnell fliegen.
5. Erstelle einen einfachen Bildschirmschoner.

## 28.23. Zusammenfassung

In diesem Kapitel hast du gelernt:

- was ein Layerbuffer ist
- wie Layerbuffer funktionieren
- Hintergründe und bewegliche Objekte zu trennen
- Animationen effizienter darzustellen

Layerbuffer gehören zu den wichtigsten Techniken moderner Spieleprogrammierung.

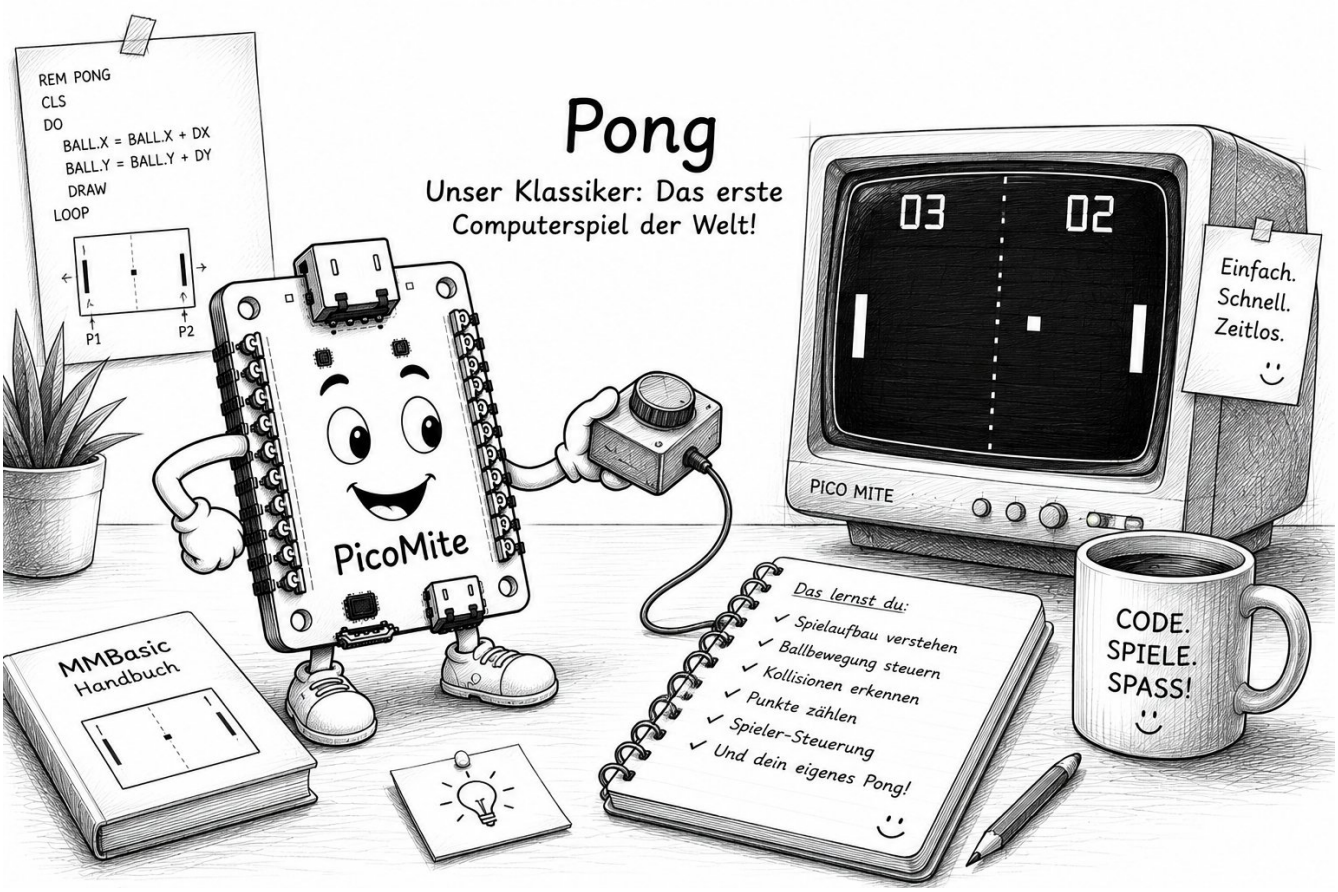
Im nächsten Kapitel werden wir dieses Wissen nutzen und unser erstes echtes Arcade-Spiel entwickeln: Pong.

## 28.24. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 28_layer_animation.bas

# Kapitel 29. Pong – das Spiel der Spiele




## 29.1. Das lernst du

In diesem Kapitel entwickelst du ein vollständiges kleines Arcade-Spiel.

Dabei lernst du:

- einen Ball über den Bildschirm zu bewegen
- einen Schläger mit den Cursortasten zu steuern
- mit `KEYDOWN()` den aktuellen Zustand der Tastatur abzufragen
- einen computergesteuerten Gegner zu programmieren
- Kollisionen mit Wänden und Schlägern zu erkennen
- Punkte zu zählen und anzuzeigen
- den Abprallwinkel des Balls zu verändern
- ein Spiel in mehrere Unterprogramme aufzuteilen
- einen Framebuffer für eine flüssige Darstellung zu verwenden
- eine vollständige Spielschleife aufzubauen

Am Ende des Kapitels besitzt du ein spielbares Pong-Programm für den PicoMite.

## 29.2. Vom Grafikexperiment zum Computerspiel

In den vergangenen Kapiteln haben wir bereits viele Techniken kennengelernt, die wir für ein Spiel benötigen:

- Rechtecke und Kreise zeichnen
- Objekte bewegen
- Bildschirmränder erkennen
- Tastatureingaben verarbeiten
- Unterprogramme verwenden
- Grafiken in einem Framebuffer aufbauen

Nun verbinden wir diese einzelnen Bausteine zu einem vollständigen Spiel.

Pong besteht nur aus wenigen Elementen:

- einem Ball
- zwei Schlägern
- zwei Punkteständen
- einer oberen und unteren Begrenzung

Trotzdem benötigt das Programm fast alle wichtigen Bestandteile eines echten Computerspiels.

## 29.3. Die Spielregeln

Der Spieler steuert den linken Schläger.

Der rechte Schläger wird vom Computer bewegt.

Der Ball fliegt zwischen beiden Seiten hin und her.

Trifft er einen Schläger, wird er zurückgeworfen.

Verlässt der Ball den Bildschirm auf einer Seite, erhält der Gegner einen Punkt.

Wer zuerst zehn Punkte erreicht, gewinnt das Spiel.

## 29.4. Die Steuerung

Der linke Schläger wird mit den Cursortasten gesteuert:

| **Taste** | **Funktion** |
|---|---|
| `UP` | Schläger nach oben bewegen |
| `DOWN` | Schläger nach unten bewegen |
| `ESC` | Spiel beenden |

Für ein Echtzeitspiel ist es wichtig, dass eine Taste nicht nur einmal erkannt wird. Der Schläger soll sich so lange bewegen, wie die entsprechende Cursortaste gedrückt gehalten wird.

Dafür verwenden wir `KEYDOWN()`.

`KEYDOWN(0)` liefert die Anzahl der momentan gedrückten Tasten:

```
KeyDown(0)
```

Mit `KEYDOWN(1)`, `KEYDOWN(2)` und so weiter können anschließend die Tastencodes der gedrückten Tasten abgefragt werden.

Für unsere Steuerung benötigen wir folgende Tastencodes:

| **Code** | **Taste** |
|---|---|
| 128 | Cursor hoch |
| 129 | Cursor runter |
| 27 | ESC |

Im Gegensatz zu `INKEY$` sind wir damit für die Bewegung des Schlägers nicht von der Tastatur-Wiederholrate abhängig.

# 29.5. Die Bestandteile des Spielfelds

Das Spielfeld besteht aus:

- einem schwarzen Hintergrund
- einer gestrichelten Mittellinie
- zwei weißen Schlägern
- einem weißen Ball
- dem Punktestand

Die Bildschirmgröße wird nicht fest vorgegeben.

Stattdessen verwendet das Programm:

```
MM.HRES
MM.VRES
```

Dadurch passt sich das Spielfeld an die aktuelle Auflösung an.

# 29.6. Größen und Positionen festlegen

Zuerst legen wir die Größe der Schläger und des Balls fest.

```
SCHLAEGERBREITE=6
SCHLAEGERHOEHE=50
BALLRADIUS=5
```

Der linke Schläger steht etwas vom linken Bildschirmrand entfernt.

```
SPIELERX=15
```

Der rechte Schläger wird entsprechend am rechten Rand positioniert.

```
COMPUTERX=MM.HRES-15-SCHLAEGERBREITE
```

Beide Schläger beginnen senkrecht in der Bildschirmmitte.

```
SPIELERY=MM.VRES/2-SCHLAEGERHOEHE/2
COMPUTERY=SPIELERY
```

## 29.7. Den Ball beschreiben

Für den Ball benötigen wir vier Variablen:

```
BALLX
BALLY
BALLDX
BALLDY
```

`BALLX` und `BALLY` enthalten die aktuelle Position.

`BALLDX` und `BALLDY` enthalten die Geschwindigkeit in horizontaler und vertikaler Richtung.

Beispielsweise bedeutet:

```
BALLDX=3
BALLDY=2
```

Der Ball bewegt sich:

- drei Pixel nach rechts
- zwei Pixel nach unten

Ein negativer Wert ändert die Bewegungsrichtung.

```
BALLDX=-3
```

bewegt den Ball nach links.

## 29.8. Den Ball in Bewegung setzen

Die Position wird in jedem Schleifendurchlauf verändert.

```
BALLX=BALLX+BALLDX
BALLY=BALLY+BALLDY
```

Wird dieser Vorgang oft genug wiederholt, entsteht eine flüssige Bewegung.

## 29.9. Die obere und untere Wand

Trifft der Ball den oberen oder unteren Rand, muss seine vertikale Bewegungsrichtung umgekehrt werden.

```
IF BALLY-BALLRADIUS<=0 THEN
  BALLY=BALLRADIUS
  BALLDY=-BALLDY
ENDIF
```

Für den unteren Rand gilt:

```
IF BALLY+BALLRADIUS>=MM.VRES-1 THEN
  BALLY=MM.VRES-1-BALLRADIUS
  BALLDY=-BALLDY
ENDIF
```

Die Korrektur von `BALLY` verhindert, dass der Ball außerhalb des sichtbaren Bereichs bleibt.

## 29.10. Rechtecke können Schläger sein

Die Schläger werden mit `BOX` gezeichnet.

```
BOX SPIELERX,SPIELERY,                    _
    SCHLAEGERBREITE,SCHLAEGERHOEHE,       _
    0,RGB(WHITE),RGB(WHITE)
```

Die Rahmenbreite ist `0`, weil wir ein vollständig gefülltes Rechteck benötigen.

Der computergesteuerte Schläger wird auf dieselbe Weise gezeichnet.

## 29.11. Den Ball zeichnen

Der Ball wird als gefüllter Kreis dargestellt.

```
CIRCLE BALLX,BALLY,BALLRADIUS,              _
       1,1,RGB(WHITE),RGB(WHITE)
```

## 29.12. Die Spielschleife

Fast jedes Echtzeitspiel besitzt eine zentrale Schleife.

Diese wird häufig als Spielschleife oder *Game Loop* bezeichnet.

Der vereinfachte Ablauf lautet:

```
Tastatur abfragen

Spieler bewegen
Computer bewegen
Ball bewegen
Kollisionen prüfen
Punkte prüfen

Bild zeichnen

kurz warten

alles wiederholen
```

In MMBasic verwenden wir dafür eine `DO … LOOP`-Schleife.

```
DO

  ' Eingaben verarbeiten
  ' Spielzustand verändern
  ' Bild zeichnen

LOOP
```

## 29.13. Den Spieler bewegen

Die Tastatur wird in jedem Durchlauf der Spielschleife mit einem eigenen Unterprogramm abgefragt:

```
ReadKeyboard
```

Das Unterprogramm untersucht alle momentan gedrückten Tasten:

```
Sub ReadKeyboard
  Local Integer i, key

  For i=1 To KeyDown(0)
    key=KeyDown(i)

    Select Case key
      Case 128                    ' Cursor hoch
        SPIELERY=SPIELERY-SCHLAEGERGESCHWINDIGKEIT

      Case 129                    ' Cursor runter
        SPIELERY=SPIELERY+SCHLAEGERGESCHWINDIGKEIT

      Case 27                     ' ESC
        SPIELEN=0
    End Select
  Next i
End Sub
```

`KEYDOWN(0)` liefert die Anzahl der gleichzeitig gedrückten Tasten. Die Schleife fragt anschließend jede dieser Tasten ab.

Mit `SELECT CASE` reagieren wir auf die für das Spiel interessanten Tastencodes.

Da `KEYDOWN()` den aktuellen Zustand der Tastatur abfragt, bewegt sich der Schläger kontinuierlich, solange eine Cursortaste gedrückt gehalten wird.

Der Spieler darf den Bildschirm jedoch nicht verlassen.

```
IF SPIELERY<0 THEN
  SPIELERY=0
ENDIF

IF SPIELERY>MM.VRES-SCHLAEGERHOEHE THEN
  SPIELERY=MM.VRES-SCHLAEGERHOEHE
ENDIF
```

# 29.14. Der computergesteuerte Gegner

Der computergesteuerte Gegner beobachtet die vertikale Position des Balls.

Befindet sich der Ball oberhalb des Schlägermittelpunkts, bewegt sich der Schläger nach oben.

```
IF BALLY<COMPUTERY+SCHLAEGERHOEHE/2 THEN
  COMPUTERY=COMPUTERY-COMPUTERGESCHWINDIGKEIT
ENDIF
```

Befindet sich der Ball darunter, bewegt er sich nach unten.

```
IF BALLY>COMPUTERY+SCHLAEGERHOEHE/2 THEN
  COMPUTERY=COMPUTERY+COMPUTERGESCHWINDIGKEIT
ENDIF
```

Auch der Computerschläger wird an den Bildschirmgrenzen angehalten.

## 29.15. Warum darf der Computer nicht zu schnell sein?

Würde der Computer seine Position sofort an die Ballposition anpassen, könnte er kaum besiegt werden.

Deshalb besitzt er eine begrenzte Geschwindigkeit.

```
COMPUTERGESCHWINDIGKEIT=2
```

Der Ball kann dadurch schneller sein als der Gegner.

Das Spiel bleibt herausfordernd, aber gewinnbar.

## 29.16. Eine Kollision mit dem Spielerschläger

Eine Kollision findet statt, wenn sich Ball und Schläger überlappen.

Für den linken Schläger prüfen wir zunächst, ob sich der Ball nach links bewegt.

```
IF BALLDX<0 THEN
```

Dann prüfen wir die horizontale Position.

```
IF BALLX-BALLRADIUS<=                    _
   SPIELERX+SCHLAEGERBREITE THEN
```

Schließlich muss sich der Ball auch innerhalb der vertikalen Schlägerfläche befinden.

```
IF BALLY>=SPIELERY AND                   _
```

```
   BALLY<=SPIELERY+SCHLAEGERHOEHE THEN
```

Sind alle Bedingungen erfüllt, wird die horizontale Geschwindigkeit umgekehrt.

```
BALLDX=-BALLDX
```

# 29.17. Den Ball nach der Kollision versetzen

Nach einer Kollision kann sich der Ball bereits ein Stück im Schläger befinden.

Würde nur die Geschwindigkeit umgedreht, könnte er im nächsten Schleifendurchlauf erneut mit dem Schläger kollidieren.

Deshalb setzen wir ihn direkt vor den Schläger.

```
BALLX=SPIELERX+SCHLAEGERBREITE+BALLRADIUS
```

Dadurch wird jede Berührung nur einmal verarbeitet.

# 29.18. Unterschiedliche Abprallwinkel

Würde der Ball immer im gleichen Winkel zurückfliegen, wäre das Spiel schnell langweilig.

Deshalb berücksichtigen wir, an welcher Stelle der Schläger getroffen wurde. So entstehen unterschiedliche Abprallwinkel.

```
TREFFER=(BALLY-(SPIELERY+SCHLAEGERHOEHE/2)) / _
        (SCHLAEGERHOEHE/2)
```

Das Ergebnis liegt ungefähr zwischen -1 und 1.

| **Wert** | **Bedeutung** |
|---|---|
| -1 | Treffer am oberen Ende |
| 0 | Treffer in der Mitte |
| 1 | Treffer am unteren Ende |

Dieser Wert beeinflusst die vertikale Ballgeschwindigkeit.

```
BALLDY=BALLDY+TREFFER*2
```

Ein Treffer am oberen Rand lenkt den Ball nach oben.

Ein Treffer am unteren Rand lenkt ihn nach unten.

## 29.19. Die Ballgeschwindigkeit begrenzen

Durch mehrere Schlägerkontakte könnte die vertikale Geschwindigkeit immer größer werden.

Deshalb begrenzen wir sie.

```
IF BALLDY>5 THEN BALLDY=5
IF BALLDY<-5 THEN BALLDY=-5
```

So bleibt der Ball kontrollierbar.

## 29.20. Die Kollision mit dem Computerschläger

Die Prüfung auf der rechten Seite funktioniert entsprechend.

Da sich der Ball diesmal nach rechts bewegen muss, prüfen wir:

```
IF BALLDX>0 THEN
```

Die horizontale Kollisionsprüfung lautet:

```
IF BALLX+BALLRADIUS>=COMPUTERX THEN
```

Nach einem Treffer wird der Ball links neben den Computerschläger gesetzt.

```
BALLX=COMPUTERX-BALLRADIUS
```

## 29.21. Einen Punkt erkennen

Verlässt der Ball den Bildschirm links, erhält der Computer einen Punkt.

```
IF BALLX<0 THEN
  COMPUTERPUNKTE=COMPUTERPUNKTE+1
  NeuerBall -1
ENDIF
```

Der neue Ball startet anschließend wieder in Richtung des Spielers.

Verlässt er den Bildschirm rechts, erhält der Spieler einen Punkt.

```
IF BALLX>MM.HRES THEN
  SPIELERPUNKTE=SPIELERPUNKTE+1
  NeuerBall 1
```

```
ENDIF
```

Der neue Ball startet nun wieder in Richtung des Computers.

Der Parameter bestimmt, in welche Richtung der neue Ball startet.

## 29.22. Den Ball neu starten

Nach jedem Punkt wird der Ball in die Bildschirmmitte gesetzt.

```
BALLX=MM.HRES/2
BALLY=MM.VRES/2
```

Die horizontale Richtung wird übergeben.

Die vertikale Richtung wird zufällig bestimmt.

```
BALLDY=INT(RND*5)-2
```

Dadurch entstehen Werte zwischen `-2` und `2`.

Damit der Ball nicht vollkommen waagerecht fliegt, ersetzen wir den Wert `0`.

```
IF BALLDY=0 THEN BALLDY=1
```

## 29.23. Die Mittellinie zeichnen

Eine gestrichelte Mittellinie lässt sich mit einer Schleife erzeugen.

```
FOR Y=0 TO MM.VRES STEP 20

  LINE MM.HRES/2,Y,MM.HRES/2,Y+10,1,RGB(GREY)

NEXT Y
```

Die Linie besteht aus zehn Pixel langen Abschnitten mit zehn Pixel großen Lücken.

## 29.24. Den Punktestand anzeigen

Der Punktestand wird oberhalb der Spielfeldmitte ausgegeben.

```
TEXT MM.HRES/2-40,20,STR$(SPIELERPUNKTE),   _
     "CT",2,2,RGB(WHITE),-1
```

```
TEXT MM.HRES/2+40,20,STR$(COMPUTERPUNKTE),  _
     "CT",2,2,RGB(WHITE),-1
```

Mit `STR$()` werden die Zahlen in Texte umgewandelt.

## 29.25. Flackerfreie Darstellung

Das vollständige Spielfeld wird zunächst im Framebuffer aufgebaut.

```
FRAMEBUFFER WRITE F

CLS RGB(BLACK)

ZeichneSpielfeld
ZeichneSchlaeger
ZeichneBall
ZeichnePunkte
```

Erst wenn das komplette Bild fertig ist, wird es auf den Bildschirm übertragen.

```
FRAMEBUFFER COPY F,N,B
```

Der zusätzliche Parameter `B` von `FRAMEBUFFER COPY F,N,B` sorgt dafür, dass die Übertragung während der Bildaustastlücke erfolgt, sofern der verwendete Grafikmodus dies unterstützt.

Dadurch wird das Bild möglichst vollständig in einem Schritt sichtbar.

## 29.26. Das vollständige Programm

```
'------------------------------------------------------
' Pong fuer den PicoMite
'
' Steuerung:
'   Cursor hoch   Schlaeger nach oben
'   Cursor runter Schlaeger nach unten
'   ESC           Spiel beenden
'------------------------------------------------------

MODE 2

CONST SCHLAEGERBREITE=6
CONST SCHLAEGERHOEHE=50
CONST BALLRADIUS=5

CONST SCHLAEGERGESCHWINDIGKEIT=4
```

```
CONST COMPUTERGESCHWINDIGKEIT=2

CONST GEWINNPUNKTE=10

SPIELERX=15
COMPUTERX=MM.HRES-15-SCHLAEGERBREITE

SPIELERY=MM.VRES/2-SCHLAEGERHOEHE/2
COMPUTERY=SPIELERY

SPIELERPUNKTE=0
COMPUTERPUNKTE=0

FRAMEBUFFER CREATE

NeuerBall 1

SPIELEN=1

DO WHILE SPIELEN=1

  ReadKeyboard

  BegrenzeSpieler

  BewegeComputer

  BewegeBall

  PruefeWaende

  PruefeSchlaeger

  PruefePunkt

  ZeichneSpiel

  IF SPIELERPUNKTE>=GEWINNPUNKTE OR          _
     COMPUTERPUNKTE>=GEWINNPUNKTE THEN

    ZeigeSpielende

    SPIELERPUNKTE=0
    COMPUTERPUNKTE=0

    SPIELERY=MM.VRES/2-SCHLAEGERHOEHE/2
    COMPUTERY=SPIELERY

    NeuerBall 1

  ENDIF
```

```
  PAUSE 10

LOOP

FRAMEBUFFER WRITE N
FRAMEBUFFER CLOSE

CLS RGB(BLACK)

PRINT "Pong wurde beendet."

END



SUB BegrenzeSpieler

  IF SPIELERY<0 THEN
    SPIELERY=0
  ENDIF

  IF SPIELERY>MM.VRES-SCHLAEGERHOEHE THEN
    SPIELERY=MM.VRES-SCHLAEGERHOEHE
  ENDIF

END SUB



Sub ReadKeyboard

  Local Integer i, key

  ' KEYDOWN(0) liefert die Anzahl gleichzeitig gedrueckter Tasten.
  For i=1 To KeyDown(0)
    key=KeyDown(i)

    Select Case key
      Case 128                    ' Cursor hoch
        SPIELERY=SPIELERY-SCHLAEGERGESCHWINDIGKEIT

      Case 129                    ' Cursor runter
        SPIELERY=SPIELERY+SCHLAEGERGESCHWINDIGKEIT

      Case 27                     ' ESC
        SPIELEN=0
    End Select
  Next i

End Sub
```

```
SUB BewegeComputer

  MITTE=COMPUTERY+SCHLAEGERHOEHE/2

  IF BALLY<MITTE-3 THEN
    COMPUTERY=COMPUTERY-COMPUTERGESCHWINDIGKEIT
  ELSEIF BALLY>MITTE+3 THEN
    COMPUTERY=COMPUTERY+COMPUTERGESCHWINDIGKEIT
  ENDIF

  IF COMPUTERY<0 THEN
    COMPUTERY=0
  ENDIF

  IF COMPUTERY>MM.VRES-SCHLAEGERHOEHE THEN
    COMPUTERY=MM.VRES-SCHLAEGERHOEHE
  ENDIF

END SUB



SUB BewegeBall

  BALLX=BALLX+BALLDX
  BALLY=BALLY+BALLDY

END SUB



SUB PruefeWaende

  IF BALLY-BALLRADIUS<=0 THEN

    BALLY=BALLRADIUS
    BALLDY=-BALLDY

  ENDIF

  IF BALLY+BALLRADIUS>=MM.VRES-1 THEN

    BALLY=MM.VRES-1-BALLRADIUS
    BALLDY=-BALLDY

  ENDIF

END SUB
```

```
SUB PruefeSchlaeger

  IF BALLDX<0 THEN

    IF BALLX-BALLRADIUS<=                    _
       SPIELERX+SCHLAEGERBREITE THEN

      IF BALLX>=SPIELERX THEN

        IF BALLY>=SPIELERY AND               _
           BALLY<=SPIELERY+SCHLAEGERHOEHE THEN

          BALLX=SPIELERX+                    _
                SCHLAEGERBREITE+BALLRADIUS

          BALLDX=-BALLDX

          TREFFER=(BALLY-                    _
                   (SPIELERY+                _
                    SCHLAEGERHOEHE/2)) /     _
                  (SCHLAEGERHOEHE/2)

          BALLDY=BALLDY+TREFFER*2

          BegrenzeBallgeschwindigkeit

        ENDIF

      ENDIF

    ENDIF

  ENDIF

  IF BALLDX>0 THEN

    IF BALLX+BALLRADIUS>=COMPUTERX THEN

      IF BALLX<=COMPUTERX+SCHLAEGERBREITE THEN

        IF BALLY>=COMPUTERY AND              _
           BALLY<=COMPUTERY+SCHLAEGERHOEHE THEN

          BALLX=COMPUTERX-BALLRADIUS

          BALLDX=-BALLDX

          TREFFER=(BALLY-                    _
```

```
                  (COMPUTERY+                _
                   SCHLAEGERHOEHE/2)) /      _
                 (SCHLAEGERHOEHE/2)

          BALLDY=BALLDY+TREFFER*2

          BegrenzeBallgeschwindigkeit

        ENDIF

      ENDIF

    ENDIF

  ENDIF

END SUB



SUB BegrenzeBallgeschwindigkeit

  IF BALLDY>5 THEN
    BALLDY=5
  ENDIF

  IF BALLDY<-5 THEN
    BALLDY=-5
  ENDIF

END SUB



SUB PruefePunkt

  IF BALLX<0 THEN

    COMPUTERPUNKTE=COMPUTERPUNKTE+1

    ZeigePunkt "Punkt fuer den Computer"

    NeuerBall -1

  ENDIF

  IF BALLX>MM.HRES THEN

    SPIELERPUNKTE=SPIELERPUNKTE+1

    ZeigePunkt "Punkt fuer den Spieler"
```

```
    NeuerBall 1

  ENDIF

END SUB



SUB NeuerBall(RICHTUNG)

  BALLX=MM.HRES/2
  BALLY=MM.VRES/2

  BALLDX=3*RICHTUNG

  BALLDY=INT(RND*5)-2

  IF BALLDY=0 THEN
    BALLDY=1
  ENDIF

  PAUSE 500

END SUB



SUB ZeichneSpiel

  FRAMEBUFFER WRITE F

  CLS RGB(BLACK)

  ZeichneMittellinie

  BOX SPIELERX,SPIELERY,                    _
      SCHLAEGERBREITE,SCHLAEGERHOEHE,       _
      0,RGB(WHITE),RGB(WHITE)

  BOX COMPUTERX,COMPUTERY,                  _
      SCHLAEGERBREITE,SCHLAEGERHOEHE,       _
      0,RGB(WHITE),RGB(WHITE)

  CIRCLE BALLX,BALLY,BALLRADIUS,            _
         1,1,RGB(WHITE),RGB(WHITE)

  TEXT MM.HRES/2-40,20,                     _
       STR$(SPIELERPUNKTE),                 _
       "CT",2,2,RGB(WHITE),-1
```

```
  TEXT MM.HRES/2+40,20,                    _
       STR$(COMPUTERPUNKTE),               _
       "CT",2,2,RGB(WHITE),-1

  FRAMEBUFFER COPY F,N,B

END SUB



SUB ZeichneMittellinie

  FOR Y=0 TO MM.VRES-1 STEP 20

    Y2=Y+10

    IF Y2>MM.VRES-1 THEN
      Y2=MM.VRES-1
    ENDIF

    LINE MM.HRES/2,Y,                      _
         MM.HRES/2,Y2,                    _
         1,RGB(GREY)

  NEXT Y

END SUB



SUB ZeigePunkt(TEXT$)

  FRAMEBUFFER WRITE F

  CLS RGB(BLACK)

  TEXT MM.HRES/2,MM.VRES/2,                _
       TEXT$,"CM",1,2,RGB(YELLOW),-1

  FRAMEBUFFER COPY F,N,B

  PAUSE 800

END SUB



SUB ZeigeSpielende

  FRAMEBUFFER WRITE F
```

```
  CLS RGB(BLACK)

  TEXT MM.HRES/2,MM.VRES/2-50,              _
       "SPIEL BEENDET",                     _
       "CM",2,2,RGB(YELLOW),-1

  IF SPIELERPUNKTE>COMPUTERPUNKTE THEN

    TEXT MM.HRES/2,MM.VRES/2,               _
         "DU HAST GEWONNEN!",               _
         "CM",1,2,RGB(GREEN),-1

  ELSE

    TEXT MM.HRES/2,MM.VRES/2,               _
         "DER COMPUTER GEWINNT",            _
         "CM",1,2,RGB(RED),-1

  ENDIF

  TEXT MM.HRES/2,MM.VRES/2+45,              _
       STR$(SPIELERPUNKTE)+" : "+           _
       STR$(COMPUTERPUNKTE),                _
       "CM",2,2,RGB(WHITE),-1

  TEXT MM.HRES/2,MM.VRES-25,                _
       "Taste fuer ein neues Spiel",        _
       "CB",1,1,RGB(CYAN),-1

  FRAMEBUFFER COPY F,N,B

  DO

    T$=INKEY$

  LOOP UNTIL T$<>""

END SUB
```

# 29.27. Das Hauptprogramm verstehen

Das Hauptprogramm enthält nur den grundsätzlichen Spielablauf.

```
ReadKeyboard
BegrenzeSpieler
BewegeComputer
BewegeBall
PruefeWaende
PruefeSchlaeger
```

```
PruefePunkt
ZeichneSpiel
```

Die Einzelheiten befinden sich in eigenen Unterprogrammen.

Dadurch bleibt die zentrale Spielschleife gut lesbar.

## 29.28. Warum so viele Unterprogramme?

Das Pong-Programm ist deutlich größer als unsere bisherigen Beispiele.

Ohne Unterprogramme würden Tastatursteuerung, Kollisionsprüfung, Grafik und Punkteverwaltung in einer langen Programmliste vermischt.

Die Aufteilung sorgt dafür, dass jedes Unterprogramm genau eine klar erkennbare Aufgabe besitzt.

| **Unterprogramm** | **Aufgabe** |
|---|---|
| `ReadKeyboard` | Liest die Cursortasten und `ESC` mit `KEYDOWN()` ein |
| `BegrenzeSpieler` | Verhindert, dass der Spielerschläger den Bildschirm verlässt |
| `BewegeComputer` | Steuert den gegnerischen Schläger |
| `BewegeBall` | Berechnet die nächste Ballposition |
| `PruefeWaende` | Erkennt Kollisionen mit oberem und unterem Rand |
| `PruefeSchlaeger` | Erkennt Treffer auf beiden Schlägern |
| `PruefePunkt` | Vergibt Punkte, wenn der Ball das Spielfeld verlässt |
| `NeuerBall` | Setzt den Ball nach einem Punkt zurück |
| `ZeichneSpiel` | Baut das vollständige Spielfeld im Framebuffer auf |
| `ZeigePunkt` | Zeigt kurz an, wer einen Punkt erhalten hat |
| `ZeigeSpielende` | Zeigt Gewinner und Endstand an |

## 29.29. Der Computer ist absichtlich nicht perfekt

Der Computer bewegt seinen Schläger zwar in Richtung des Balls, aber nur mit begrenzter Geschwindigkeit.

Außerdem verwendet er einen kleinen Bereich von drei Pixeln, in dem er nicht reagiert.

```
IF BALLY<MITTE-3 THEN
  ...
ELSEIF BALLY>MITTE+3 THEN
  ...
ENDIF
```

Ohne diesen Bereich könnte der Schläger bei kleinen Abweichungen ständig zwischen zwei

Positionen wechseln.

Der Computer wirkt dadurch ruhiger und natürlicher.

## 29.30. Das Spieltempo verändern

Die Geschwindigkeit wird an mehreren Stellen beeinflusst.

Die Pause am Ende der Spielschleife legt die grundsätzliche Bildrate fest.

```
PAUSE 10
```

Ein größerer Wert macht das Spiel langsamer.

```
PAUSE 20
```

Ein kleinerer Wert macht es schneller.

Die horizontale Ballgeschwindigkeit wird in `NeuerBall` festgelegt.

```
BALLDX=3*RICHTUNG
```

Der Spielerschläger verwendet:

```
SCHLAEGERGESCHWINDIGKEIT=4
```

Der Computer verwendet:

```
COMPUTERGESCHWINDIGKEIT=2
```

Diese Werte kannst du unabhängig voneinander verändern.

## 29.31. Eine mögliche Schwierigkeitstabelle

Du kannst später mehrere Schwierigkeitsstufen einbauen.

| **Stufe** | **Ball** | **Spieler** | **Computer** |
|---|---|---|---|
| Leicht | 2 | 5 | 1 |
| Normal | 3 | 4 | 2 |
| Schwer | 4 | 4 | 3 |
| Extrem | 5 | 4 | 4 |

## 29.32. Warum verwenden wir vorerst keine Layerbuffer?

Layerbuffer eignen sich grundsätzlich hervorragend für bewegliche Objekte.

Bei Pong verändert sich jedoch fast der gesamte sichtbare Spielzustand:

- Ball
- linker Schläger
- rechter Schläger
- Punktestand

Das Spielfeld ist außerdem sehr einfach und kann schnell neu aufgebaut werden.

Ein vollständiges Bild im Framebuffer zu zeichnen und anschließend zu kopieren, ist deshalb für dieses erste Spiel übersichtlicher.

Die im vorherigen Kapitel kennengelernten Layerbuffer werden besonders interessant, wenn ein aufwendiger Hintergrund erhalten bleiben soll.

## 29.33. Den zweiten Spieler ergänzen

Das Programm verwendet momentan einen computergesteuerten Gegner.

Du kannst den rechten Schläger auch durch einen zweiten Spieler steuern lassen. Dazu wird die Computersteuerung entfernt und `ReadKeyboard` um weitere Tasten für den zweiten Schläger ergänzt.

Gerade hier zeigt sich ein Vorteil von `KEYDOWN()`: Da `KEYDOWN(0)` die Anzahl der gleichzeitig gedrückten Tasten liefert, können auch beide Spieler ihre Schläger zur selben Zeit bewegen.

Für eine eigene Zwei-Spieler-Version kannst du beispielsweise zwei zusätzliche Buchstabentasten für den rechten Schläger festlegen und deren Tastencodes in `ReadKeyboard` ergänzen.

## 29.34. Farbe ins Spiel bringen

Das klassische Spielfeld verwendet bewusst nur Schwarz und Weiß.

Du kannst das Erscheinungsbild leicht verändern.

Beispielsweise:

```
HINTERGRUND=RGB(0,0,40)
BALLFARBE=RGB(YELLOW)
SPIELERFARBE=RGB(CYAN)
COMPUTERFARBE=RGB(MAGENTA)
```

Diese Variablen können dann in `ZeichneSpiel` verwendet werden.

## 29.35. Geräusche ergänzen

Ein Pong-Spiel wirkt mit Geräuschen noch lebendiger.

Geeignete Ereignisse sind:

- Ball trifft Wand
- Ball trifft Schläger
- Spieler erhält einen Punkt
- Spiel ist beendet

Die genaue Soundausgabe hängt von der verwendeten PicoMite-Hardware und deren Audiokonfiguration ab.

Wir werden Töne und Klänge in einem späteren Kapitel ausführlicher behandeln.

## 29.36. Experimentiere!

Probiere folgende Änderungen aus:

- Verändere die Ballgeschwindigkeit.
- Vergrößere oder verkleinere die Schläger.
- Ändere die Gewinnpunktzahl.
- Verwende farbige Spielelemente.
- Mache den Computer schneller oder langsamer.
- Entferne die Mittellinie.
- Verwende einen quadratischen Ball mit `BOX`.

## 29.37. Probier’s selbst!

Versuche folgende Erweiterungen:

1. Baue mehrere Schwierigkeitsstufen ein.
2. Ergänze einen Startbildschirm.
3. Ermögliche die Auswahl zwischen einem und zwei Spielern.
4. Speichere gewonnene Spiele in einer Datei.
5. Erhöhe die Ballgeschwindigkeit nach jedem Schlägerkontakt.
6. Lasse die Schläger nach jedem Punkt etwas kleiner werden.
7. Ergänze einfache Soundeffekte.
8. Zeige die bisher längste Ballwechselserie an.
9. Beende ein Spiel erst mit zwei Punkten Vorsprung.

10. Entwirf ein eigenes farbiges Spielfeld.

## 29.38. Ideen für eine spätere Version

Unser Pong-Spiel kann noch deutlich erweitert werden.

Mögliche Ideen sind:

- verschiedene Ballgrößen
- mehrere Bälle gleichzeitig
- Hindernisse im Spielfeld
- schräge Schläger
- Spezialeffekte bei Treffern
- Highscores
- Turniermodus
- Maus- oder Joysticksteuerung
- Netzwerkspiel zwischen zwei PicoMites

Schon die aktuelle Version enthält jedoch alle wesentlichen Elemente eines vollständigen Arcade-Spiels.

## 29.39. Zusammenfassung

In diesem Kapitel hast du ein vollständiges Echtzeitspiel entwickelt.

Dabei hast du viele bisher erlernte Techniken miteinander verbunden:

- Variablen
- Konstanten
- Bedingungen
- Schleifen
- Unterprogramme
- Tastatureingaben mit `KEYDOWN()`
- Animationen
- Kollisionserkennung
- computergesteuerte Gegner
- Punkteverwaltung
- Framebuffer
- flackerfreie Grafik

Du hast außerdem den grundlegenden Aufbau eines Spiels kennengelernt:

```
Eingaben lesen
Spielzustand verändern
Kollisionen prüfen
Bild zeichnen
alles wiederholen
```

Dieses Prinzip findet sich nicht nur in Pong, sondern in fast allen Echtzeitspielen wieder.

Aus den ersten einfachen Grafikbefehlen ist damit Schritt für Schritt ein richtiges Computerspiel entstanden.

## 29.40. Rückblick auf die Grafikprogrammierung

Mit einem einzelnen Pixel haben wir begonnen.

Danach entstanden Linien, geometrische Formen, grafische Spiele, eine Bahnhofsuhr, mathematische Kunstwerke und schließlich ein vollständiges Pong-Spiel.

Dabei hast du nicht nur neue Grafikbefehle kennengelernt. Du hast auch erfahren, wie Animationen, Kollisionen, Framebuffer und Spielschleifen zusammenarbeiten.

Die fortgeschrittenen Grafikfunktionen von MMBasic – darunter Bitmaps, BLIT und Sprites – werden wir in einem späteren Teil kennenlernen.

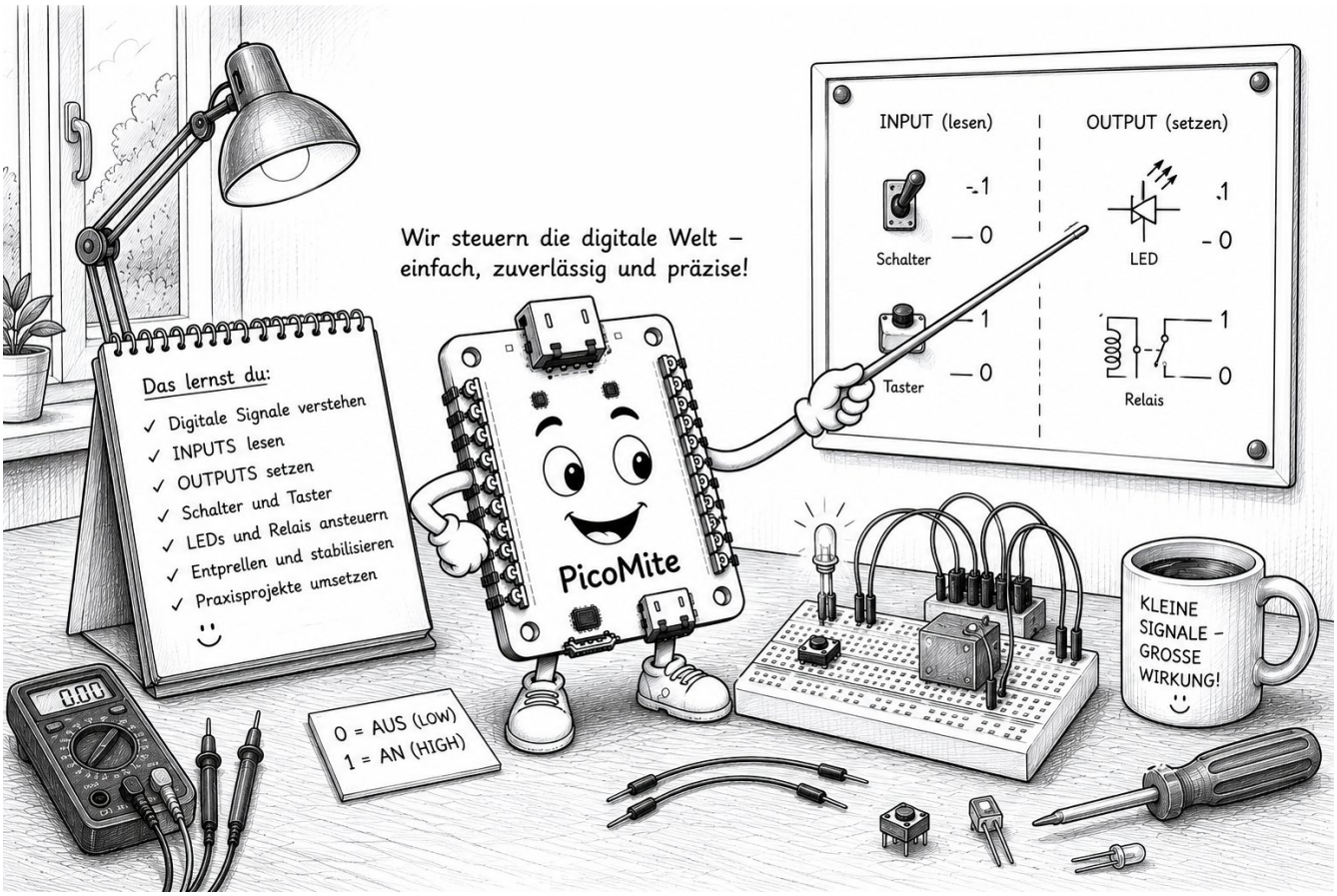
## 29.41. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 29_pong.bas

# Teil IV – Hardware und Elektronik

# Kapitel 30. Digitale Ein- und Ausgänge




## 30.1. Das lernst du

In diesem Kapitel lernst du:

- was digitale Ein- und Ausgänge sind
- wie ein PicoMite Pins steuern kann
- wie Ausgänge ein- und ausgeschaltet werden
- wie Eingänge abgefragt werden
- was die Befehle `SETPIN`, `PIN()` und `GPx` bedeuten
- wie Software mit der Außenwelt kommuniziert

Am Ende des Kapitels kannst du die ersten Hardware-Anschlüsse des PicoMite verwenden.

## 30.2. Vom Bildschirm zur Außenwelt

Bisher liefen unsere Programme ausschließlich innerhalb des Computers.

Sie konnten:

- Texte anzeigen
- Grafiken zeichnen

- Dateien speichern
- Spiele ausführen

Nun soll unser Programm mit der Außenwelt kommunizieren.

Der PicoMite besitzt dazu eine Vielzahl von Anschlüssen.

Diese Anschlüsse nennt man GPIO-Pins: oder einfach:

```
Pins
```

# 30.3. Was ist ein Pin?

Ein Pin ist ein elektrischer Anschluss des Mikrocontrollers.

Er kann beispielsweise:

- eine LED einschalten
- einen Taster abfragen
- einen Summer ansteuern
- einen Sensor auslesen

Je nach Konfiguration arbeitet ein Pin als:

- Eingang
- Ausgang

# 30.4. Digitale Signale

Digitale Ein- und Ausgänge kennen nur zwei Zustände:

| **Wert** | **Bedeutung** |
|---|---|
| 0 | Aus |
| 1 | Ein |

Man spricht häufig auch von `LOW` und `HIGH`:

| **Begriff** | **Bedeutung** |
|---|---|
| LOW | 0 Volt |
| HIGH | 3,3 Volt |

Ein digitaler Ausgang kann also lediglich ein- oder ausgeschaltet werden.

## 30.5. Ein Ausgang

Ein Ausgang liefert ein elektrisches Signal.

Bevor ein Pin verwendet werden kann, muss seine Funktion festgelegt werden.

Dies geschieht mit `SETPIN`:

```
SETPIN GP0,DOUT
```

Dabei bedeutet:

| Teil | Bedeutung |
|---|---|
| GP0 | Pin GP0 |
| DOUT | Digital Output |

Der Pin arbeitet nun als digitaler Ausgang (`DOUT`).

## 30.6. Einen Ausgang einschalten

Nun kann der Ausgang mit `PIN()` eingeschaltet werden.

```
PIN(GP0)=1
```

Der Pin liefert jetzt 3,3 Volt.

## 30.7. Einen Ausgang ausschalten

```
PIN(GP0)=0
```

Der Ausgang wird wieder abgeschaltet.

## 30.8. Das erste Hardwareprogramm

Dieses Beispiel schaltet einen Ausgang ein.

```
SETPIN GP0,DOUT

PIN(GP0)=1
```

Das Programm endet anschließend.

Der Ausgang bleibt jedoch eingeschaltet.

## 30.9. Einen Ausgang blinken lassen

Interessanter wird es mit einer Schleife.

```
SETPIN GP0,DOUT

DO

  PIN(GP0)=1

  PAUSE 500

  PIN(GP0)=0

  PAUSE 500

LOOP
```

Der Ausgang wird nun zweimal pro Sekunde umgeschaltet.

## 30.10. Was macht PAUSE?

`PAUSE` wartet eine bestimmte Anzahl Millisekunden.

```
PAUSE 1000
```

entspricht einer Sekunde.

## 30.11. Ein Eingang

Ein Eingang empfängt Signale von außen.

Dazu wird der Pin als Eingang konfiguriert.

```
SETPIN GP1,DIN
```

Das `DIN` steht für:

```
Digital Input
```

## 30.12. Einen Eingang abfragen

Der Zustand eines Eingangs kann mit `PIN()` gelesen werden.

```
WERT=PIN(GP1)
```

Das Ergebnis ist:

```
0
oder
1
```

## 30.13. Den Zustand anzeigen

```
SETPIN GP1,DIN

DO

  PRINT PIN(GP1)

  PAUSE 200

LOOP
```

Das Programm zeigt den aktuellen Zustand des Eingangs an.

## 30.14. Warum braucht man Eingänge?

Eingänge können beispielsweise mit folgenden Bauteilen verbunden werden:

- Taster
- Schalter
- Bewegungsmelder
- Lichtschranken
- Sensoren

Dadurch kann ein Programm auf Ereignisse reagieren.

## 30.15. Ein digitales Modell

Man kann sich einen digitalen Eingang wie eine Frage vorstellen:

```
Ist etwas aktiv?
Ja oder Nein?
```

oder:

```
Ist der Taster gedrückt?
Ja oder Nein?
```

Mehr Informationen stehen nicht zur Verfügung.

## 30.16. Häufig verwendete Pin-Modi

| Modus | Bedeutung |
|---|---|
| DIN | Digitaler Eingang |
| DOUT | Digitaler Ausgang |
| AIN | Analoger Eingang |
| PWM | Pulsweitenmodulation |
| OFF | Pin deaktivieren |

Weitere Modi werden wir in späteren Kapiteln kennenlernen.

## 30.17. Wichtiger Hinweis

Die GPIO-Pins des PicoMite arbeiten mit:

```
3,3 Volt
```

Höhere Spannungen dürfen nicht direkt angeschlossen werden.

Eine falsche Verdrahtung kann den Mikrocontroller beschädigen.

## 30.18. 3,3 Volt oder 5 Volt?

Heute arbeiten die meisten modernen Mikrocontroller – darunter auch der PicoMite und der Colour Maximite 2 – mit einer Betriebsspannung von **3,3 Volt**.

Das war jedoch nicht immer so. Über viele Jahre waren **5 Volt** der Standard. Zahlreiche klassische Mikrocontroller und Logikbausteine, wie beispielsweise der Z80, der 6502, der 8085 oder viele Bausteine der 74xx- und 74LS-Serie, wurden ausschließlich mit 5 Volt betrieben.

Deshalb findest du auch heute noch viele Schaltungen, Sensoren und Erweiterungsmodule, die für 5 Volt ausgelegt sind.

**Achtung:** Die GPIO-Pins des PicoMite dürfen in der Regel **nicht direkt mit 5-Volt-Signalen verbunden werden**, da sie beschädigt werden können. Prüfe deshalb immer, mit welcher Spannung ein Bauteil arbeitet, bevor du es anschließt.

## 30.19. Noch keine Hardware?

Falls du momentan keine Hardware angeschlossen hast, ist das kein Problem.

In den nächsten Kapiteln werden wir konkrete Schaltungen vorstellen.

Bis dahin genügt es, die grundlegenden Befehle kennenzulernen.

## 30.20. Experimentiere!

Probiere folgende Änderungen aus:

- Verwende andere GPIO-Pins.
- Ändere die Blinkgeschwindigkeit.
- Gib zusätzliche Texte aus.
- Verwende mehrere Ausgänge gleichzeitig.

## 30.21. Probier’s selbst!

Versuche folgende Aufgaben:

1. Schalte zwei Ausgänge abwechselnd ein.
2. Erzeuge ein Lauflicht mit vier Ausgängen.
3. Zeige den Zustand eines Eingangs auf dem Bildschirm an.
4. Baue eine einfache Blinksteuerung mit unterschiedlichen Zeiten.

## 30.22. Zusammenfassung

In diesem Kapitel hast du gelernt:

- was digitale Ein- und Ausgänge sind
- wie Pins konfiguriert werden
- wie Ausgänge geschaltet werden
- wie Eingänge gelesen werden

Damit beginnt ein neuer Abschnitt des Tutorials.

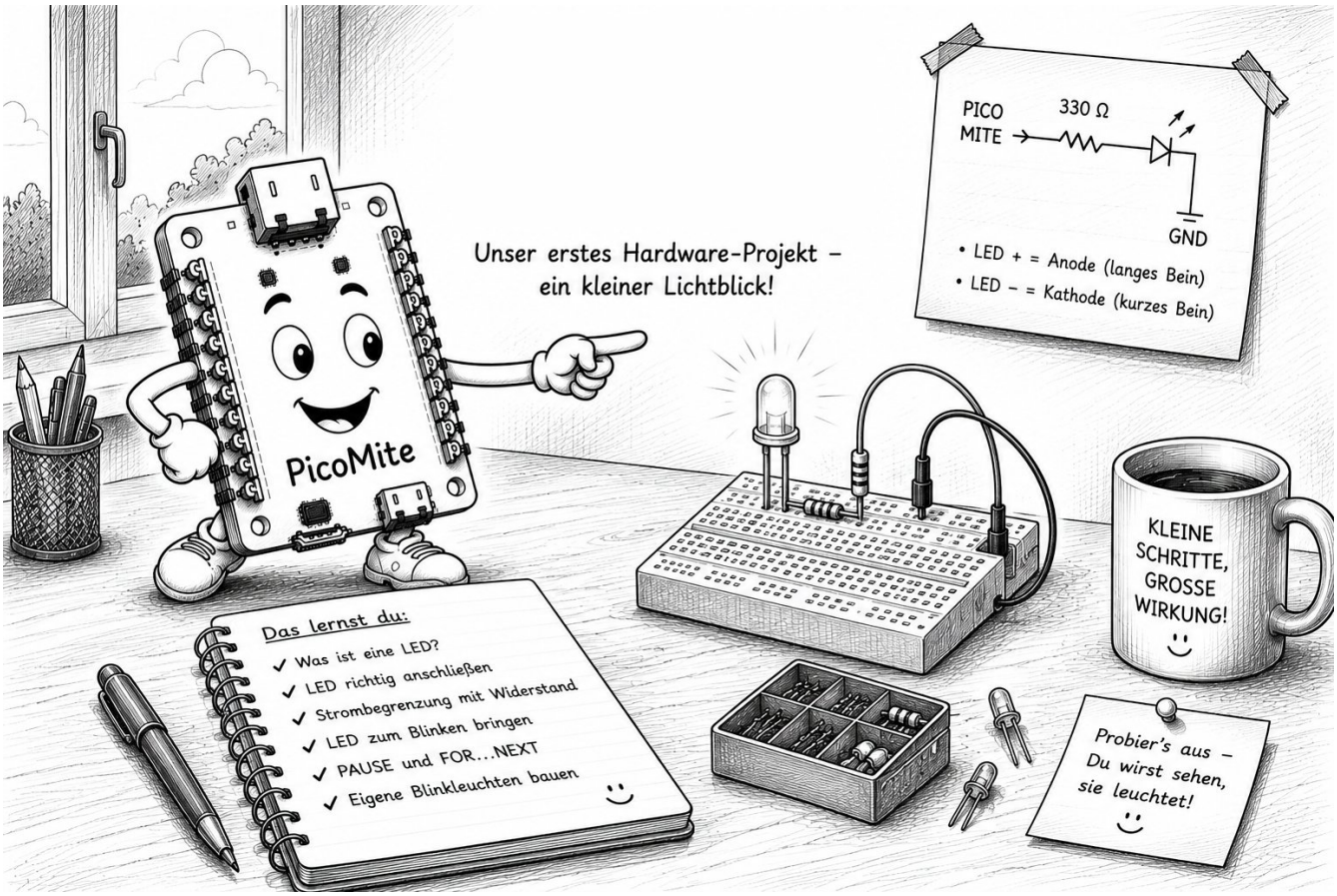
Ab jetzt verlassen wir den Bildschirm und steuern echte Hardware.

## 30.23. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 30_ein_ausgang_test.bas

# Kapitel 31. Eine LED zum Blinken bringen




## 31.1. Das lernst du

In diesem Kapitel lernst du:

- was eine LED ist
- wie eine LED angeschlossen wird
- warum ein Vorwiderstand notwendig ist
- wie GPIO-Ausgänge verwendet werden
- wie ein Blinkprogramm funktioniert
- wie Blinkfrequenzen verändert werden

Am Ende des Kapitels kannst du deine erste externe Schaltung mit MMBasic ansteuern.

## 31.2. Das Mikrocontroller-Hallo-Welt

Bei vielen Programmiersprachen lautet das erste Programm:

```
PRINT "Hallo Welt"
```

Bei Mikrocontrollern gibt es ein anderes klassisches Einstiegsprojekt:

Eine LED zum Blinken bringen.

Der Grund ist einfach:

Zum ersten Mal bewirkt ein Programm etwas in der realen Welt.

Der Mikrocontroller erzeugt ein elektrisches Signal und eine LED beginnt sichtbar zu leuchten.

Dieser Moment begeistert bis heute viele Einsteiger.

## 31.3. PicoMite und Colour Maximite 2

Die Programme in diesem Kapitel funktionieren sowohl auf dem PicoMite als auch auf dem Colour Maximite 2.

Der Unterschied liegt hauptsächlich in der Bezeichnung der Ein- und Ausgänge.

Beim PicoMite werden GPIO-Namen verwendet:

```
SETPIN GP0,DOUT
```

Beim Colour Maximite 2 werden die Pinnummern des 40-poligen I/O-Steckers verwendet:

```
SETPIN 32,DOUT
```

Die Funktionsweise ist ansonsten identisch. Der Colour Maximite 2 stellt insgesamt 28 frei verwendbare Ein- und Ausgänge zur Verfügung.

## 31.4. Die eingebaute LED des PicoMite

Viele Raspberry Pi Pico Boards besitzen bereits eine eingebaute LED.

Diese ist mit dem GPIO-Pin GP25 verbunden.

Dadurch kann das erste Blinkprogramm oft ganz ohne zusätzliche Hardware ausprobiert werden.

## 31.5. Erstes Blinkprogramm

```
SETPIN GP25,DOUT

DO

  PIN(GP25)=1

  PAUSE 500

  PIN(GP25)=0
```

```
  PAUSE 500

LOOP
```

Die LED sollte nun einmal pro Sekunde blinken.

## 31.6. Hinweis zum Pico W

Beim Raspberry Pi Pico W ist die eingebaute LED nicht direkt mit dem Mikrocontroller verbunden.

Sie kann deshalb nicht mit den hier gezeigten MMBasic-Befehlen angesteuert werden.

Verwende in diesem Fall eine externe LED.

## 31.7. Was ist eine LED?

LED steht für:

```
Light Emitting Diode
```

Eine LED ist ein elektronisches Bauteil, das Licht erzeugt, wenn Strom hindurchfließt.

LEDs gibt es in vielen Farben:

- Rot
- Grün
- Gelb
- Blau
- Weiß

Im Gegensatz zu einer Glühlampe benötigt eine LED nur sehr wenig Strom.

## 31.8. Eine LED ist eine Diode

Eine Diode lässt Strom nur in einer Richtung fließen.

Wird sie falsch herum angeschlossen, bleibt sie dunkel.

Deshalb besitzt jede LED zwei Anschlüsse:

| **Anschluss** | **Bedeutung** |
|---|---|
| Anode | Positiver Anschluss |

| Anschluss | Bedeutung |
|---|---|
| Kathode | Negativer Anschluss |

## 31.9. Wie erkennt man die Anschlüsse?

Bei den meisten LEDs gibt es zwei Merkmale:

- Das längere Beinchen ist die Anode (+)
- Das kürzere Beinchen ist die Kathode (-)

Zusätzlich besitzt das Gehäuse häufig eine kleine Abflachung auf der Kathodenseite.

image::bilder/led_polaritaet.jpg[ align=center, title="Anode und Kathode einer LED" ]

## 31.10. Warum benötigen wir einen Vorwiderstand?

Eine LED darf niemals direkt an einen Ausgang angeschlossen werden.

Ohne Vorwiderstand würde zu viel Strom fließen.

Die Folgen können sein:

- Die LED wird beschädigt.
- Der Ausgang wird beschädigt.
- Im schlimmsten Fall wird der Mikrocontroller beschädigt.

Deshalb wird immer ein Widerstand in Reihe geschaltet.

## 31.11. Welchen Widerstand verwenden wir?

Typische Werte liegen zwischen:

```
220 Ω bis 470 Ω
```

Für unsere Beispiele verwenden wir:

```
330 Ω
```

Dieser Wert ist ein guter Kompromiss zwischen Helligkeit und Sicherheit.

## 31.12. LEDs mit eingebautem Vorwiderstand

Neben gewöhnlichen LEDs gibt es auch LEDs mit **eingebautem Vorwiderstand**. Sie werden häufig als "3,3-V-LED", "5-V-LED" oder "12-V-LED" angeboten und

können direkt an die entsprechende Betriebsspannung angeschlossen werden.

Bei diesen LEDs ist der notwendige Vorwiderstand bereits im Gehäuse oder im Anschlusskabel integriert. Ein zusätzlicher Vorwiderstand ist daher normalerweise nicht erforderlich.

Achte deshalb beim Kauf immer auf die Beschreibung der LED. Wird eine LED mit einer festen Betriebsspannung angegeben (z. B. 3,3 V, 5 V oder 12 V), handelt es sich oft um eine Ausführung mit integriertem Vorwiderstand.

Für die Berechnungen und Schaltungen in diesem Buch verwenden wir **normale LEDs ohne eingebauten Vorwiderstand**. Hier muss der Vorwiderstand immer extern berechnet und angeschlossen werden.

## 31.13. Die Schaltung

Für unser erstes Hardwareprojekt benötigen wir:

- MMBasic-System (PicoMite oder Colour Maximite 2)
- LED
- 330-Ohm-Widerstand
- Steckdrähte
- Steckbrett

Die Schaltung sieht so aus:

```
Ausgang ----[330 Ω]----|>|---- GND
```

## 31.14. Schaltplan

image::bilder/led_blinken_schaltplan.jpg[ align=center, title="LED mit Vorwiderstand" ]

## 31.15. Anschluss am PicoMite

Für die ersten Versuche verwenden wir GP0.

```
GP0 ----[330 Ω]----|>|---- GND
```

Programm:

```
SETPIN GP0,DOUT

PIN(GP0)=1
```

## 31.16. Anschluss am Colour Maximite 2

Für die ersten Versuche verwenden wir Pin 32 des I/O-Steckers.

```
Pin 32 ----[330 Ω]----|>|---- GND
```

Programm:

```
SETPIN 32,DOUT

PIN(32)=1
```

## 31.17. Die LED dauerhaft einschalten

PicoMite:

```
SETPIN GP0,DOUT

PIN(GP0)=1
```

Colour Maximite 2:

```
SETPIN 32,DOUT

PIN(32)=1
```

Die LED sollte nun dauerhaft leuchten.

## 31.18. Die LED ausschalten

PicoMite:

```
PIN(GP0)=0
```

Colour Maximite 2:

```
PIN(32)=0
```

## 31.19. Mehr als nur LEDs schalten

Ein GPIO-Ausgang kann nur einen kleinen Strom liefern. Das reicht für LEDs, Taster oder einfache

Piezo-Lautsprecher, nicht jedoch für größere Verbraucher wie Motoren, Relais oder Magnetspulen.

In solchen Fällen verwendet man eine **Treiber- oder Verstärkerschaltung**. Sie sorgt dafür, dass der Mikrocontroller nur ein kleines Steuersignal liefern muss, während die eigentliche Last aus einer separaten Spannungsquelle versorgt wird.

Ein häufig verwendeter Aufbau besteht aus einem NPN-Transistor oder einem MOSFET.

```
                +12 V
                  |
                Verbraucher
                  |
                  |
                 C
GPIO ———1 kΩ———B   NPN-Transistor
                 E
                  |
                 GND
```

Der GPIO-Ausgang schaltet dabei lediglich den Transistor ein oder aus. Dieser übernimmt das Schalten des eigentlichen Verbrauchers.

Mit einer solchen Schaltung lassen sich beispielsweise ansteuern:

- Relais
- Magnetspulen
- Ventile
- Motoren
- Lampen
- Modelleisenbahn-Signale
- größere LED-Leisten

Beim Schalten von Spulen, Relais oder Motoren entsteht beim Ausschalten eine hohe Spannungsspitze. Deshalb wird parallel zur Spule eine sogenannte **Freilaufdiode** angeschlossen, um den Transistor und den Mikrocontroller zu schützen.

Dieses Thema werden wir später noch ausführlicher behandeln.

# 31.20. Das erste Blinkprogramm

PicoMite:

```
SETPIN GP0,DOUT
```

```
DO

  PIN(GP0)=1
  PAUSE 500

  PIN(GP0)=0
  PAUSE 500

LOOP
```

Colour Maximite 2:

```
SETPIN 32,DOUT

DO

  PIN(32)=1
  PAUSE 500

  PIN(32)=0
  PAUSE 500

LOOP
```

## 31.21. Wie funktioniert das Programm?

Zunächst wird der Ausgang konfiguriert:

```
SETPIN ...,DOUT
```

Danach wird der Ausgang eingeschaltet:

```
PIN(...)=1
```

Die LED leuchtet.

Nach einer Pause von einer halben Sekunde wird sie wieder ausgeschaltet:

```
PIN(...)=0
```

Die Schleife beginnt anschließend erneut.

## 31.22. Die Blinkgeschwindigkeit verändern

Schneller:

```
PAUSE 100
```

Langsamer:

```
PAUSE 1000
```

Experimentiere mit verschiedenen Werten.

## 31.23. Mehrfach blinken

```
FOR I=1 TO 10

  PIN(GP0)=1
  PAUSE 200

  PIN(GP0)=0
  PAUSE 200

NEXT I
```

Danach endet das Programm.

## 31.24. Eine Blinkfunktion erstellen

```
SUB Blinke(ANZEIT,AUSZEIT)

  PIN(GP0)=1
  PAUSE ANZEIT

  PIN(GP0)=0
  PAUSE AUSZEIT

END SUB
```

Verwendung:

```
Blinke 200,200
```

## 31.25. Praxisbeispiel: SOS

Mit einer blinkenden LED lassen sich Informationen übertragen.

Der bekannte SOS-Notruf lautet:

```
... --- ...
```

Das bedeutet:

- drei kurze Signale
- drei lange Signale
- drei kurze Signale

## 31.26. SOS senden

```
SETPIN GP0,DOUT

DO

  Kurzsignal
  Kurzsignal
  Kurzsignal

  Langsignal
  Langsignal
  Langsignal

  Kurzsignal
  Kurzsignal
  Kurzsignal

  PAUSE 2000

LOOP
```

Die Unterprogramme:

```
SUB Kurzsignal

  PIN(GP0)=1
  PAUSE 200

  PIN(GP0)=0
  PAUSE 200
```

```
END SUB



SUB Langsignal

  PIN(GP0)=1
  PAUSE 600

  PIN(GP0)=0
  PAUSE 200

END SUB
```

## 31.27. Typische Fehler

Wenn die LED nicht leuchtet, prüfe:

1. Ist die LED richtig herum angeschlossen?
2. Ist der Widerstand eingebaut?
3. Ist GND korrekt verbunden?
4. Wird der richtige Pin verwendet?
5. Wurde `SETPIN` aufgerufen?

## 31.28. Noch keine Hardware vorhanden?

Falls du momentan keine externe LED zur Verfügung hast, kannst du zunächst die eingebaute LED des PicoMite verwenden.

So lassen sich die meisten Beispiele dieses Kapitels ausprobieren.

## 31.29. Experimentiere!

Probiere folgende Änderungen aus:

- andere Blinkgeschwindigkeiten
- längere und kürzere Impulse
- andere Ausgänge
- eigene Blinkmuster

## 31.30. Probier’s selbst!

Versuche folgende Aufgaben:

1. Lasse die LED genau fünfmal blinken.

2. Erzeuge ein Doppelblinken.
3. Sende deinen Anfangsbuchstaben als Morsezeichen.
4. Verwende zwei LEDs mit unterschiedlichen Blinkmustern.
5. Schreibe ein Unterprogramm für beliebige Blinkimpulse.

## 31.31. Zusammenfassung

In diesem Kapitel hast du gelernt:

- was eine LED ist
- warum ein Vorwiderstand notwendig ist
- wie digitale Ausgänge verwendet werden
- wie eine LED ein- und ausgeschaltet wird
- wie Blinkprogramme aufgebaut sind

Zum ersten Mal hat dein MMBasic-Programm direkt mit der realen Welt kommuniziert.

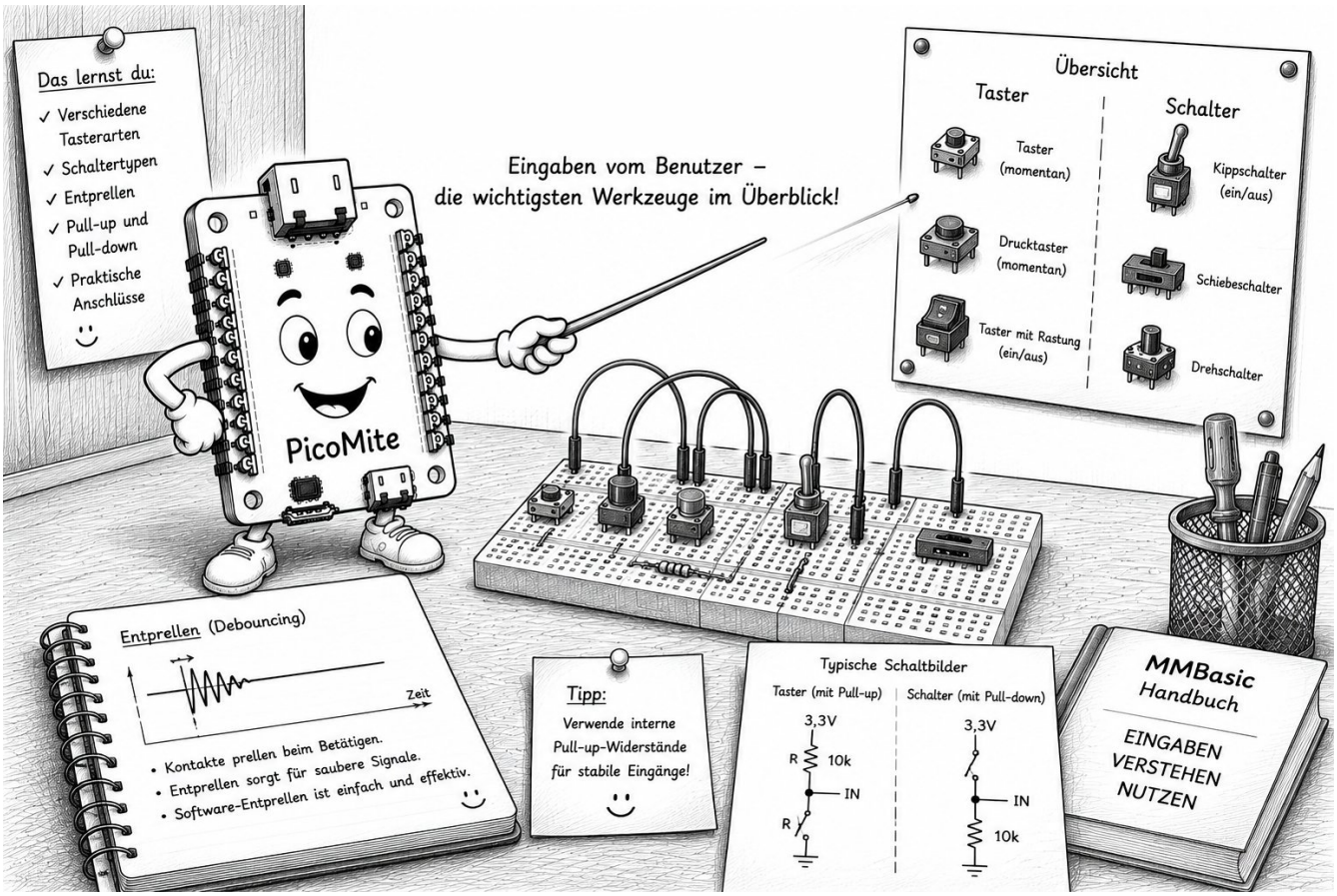
Im nächsten Kapitel werden wir digitale Eingänge kennenlernen und einen Taster abfragen.

## 31.32. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 31_led_blinken.bas

# Kapitel 32. Taster und Schalter




## 32.1. Das lernst du

In diesem Kapitel lernst du:

- was digitale Eingänge sind
- wie Taster und Schalter funktionieren
- wie ein Eingang abgefragt wird
- was Pull-Up- und Pull-Down-Widerstände sind
- warum Taster prellen
- wie man Taster entprellt
- wie Programme auf Benutzereingaben reagieren

Am Ende des Kapitels kannst du eigene Tasterabfragen programmieren und erste interaktive Hardwareprojekte entwickeln.

## 32.2. Vom Blinken zum Reagieren

Im letzten Kapitel konnten wir eine LED ein- und ausschalten.

Der PicoMite beziehungsweise Colour Maximite 2 konnte also Informationen nach außen geben.

Jetzt drehen wir die Richtung um.

Wir möchten, dass unser Programm auf Ereignisse aus seiner Umgebung reagiert.

Dazu benötigen wir Eingänge.

Der einfachste Eingang ist ein Taster.

# 32.3. Digitale Eingänge

Ein digitaler Eingang kennt nur zwei Zustände:

| **Wert** | **Bedeutung** |
|---|---|
| 0 | Aus, LOW, nicht aktiv |
| 1 | Ein, HIGH, aktiv |

Mehr Informationen stehen nicht zur Verfügung.

Ein digitaler Eingang kann also beispielsweise erkennen:

- Taster gedrückt oder nicht gedrückt
- Schalter ein oder aus
- Lichtschranke unterbrochen oder frei
- Bewegungsmelder aktiv oder inaktiv

# 32.4. Taster und Schalter

Obwohl beide Bauteile ähnlich aussehen können, gibt es einen wichtigen Unterschied.

Ein Schalter behält seinen Zustand bei.

```
EIN  -> bleibt EIN
AUS  -> bleibt AUS
```

Ein Taster kehrt nach dem Loslassen automatisch in seine Ausgangsposition zurück.

```
gedrückt     -> EIN
losgelassen  -> AUS
```

# 32.5. Mehr als nur ein Taster

Taster und Schalter gibt es in vielen verschiedenen Bauformen. Je nach Anwendung kommen unterschiedliche Varianten zum Einsatz.

Die wichtigsten Typen sind:

| Bauteil | Verwendung |
|---|---|
| Drucktaster | Schließt den Kontakt nur während des Drückens. Er wird beispielsweise für Klingeln, Tastaturen oder Starttasten verwendet. |
| Mikroschalter | Besitzt einen definierten Schaltpunkt und wird häufig als Endschalter oder Positionssensor eingesetzt. |
| Kippschalter | Bleibt nach dem Betätigen in seiner Stellung. Er eignet sich zum Ein- und Ausschalten von Geräten. |
| Reedkontakt | Wird durch ein Magnetfeld betätigt. Er findet sich beispielsweise in Alarmanlagen, Türkontakten oder Drehzahlmessern. |
| Relais | Ein elektrisch betätigter Schalter. Damit können auch höhere Spannungen oder Ströme geschaltet werden, ohne den Mikrocontroller direkt zu belasten. |

Für unsere ersten Programme genügt ein einfacher Drucktaster. Je nach Projekt können jedoch auch andere Schaltertypen die bessere Wahl sein.

## 32.6. Pull-Up und Pull-Down

Ein Eingang darf niemals "in der Luft hängen".

Andernfalls kann er zufällige Werte liefern.

```
0
1
0
1
1
0
```

obwohl überhaupt nichts passiert.

Dieses Verhalten nennt man einen "floating input".

Deshalb benötigt jeder Eingang einen definierten Grundzustand.

Dafür werden Pull-Up- oder Pull-Down-Widerstände verwendet.

## 32.7. Pull-Up

Beim Pull-Up liegt der Eingang normalerweise auf HIGH.

Wird der Taster gedrückt, wird der Eingang mit GND verbunden.

```
3.3V
 |
```

```
[R]
 |
 +---- Eingang
 |
[Taster]
 |
GND
```

Der Eingang liefert:

| **Taster** | **Wert** |
|---|---|
| nicht gedrückt | 1 |
| gedrückt | 0 |

## 32.8. Pull-Down

Beim Pull-Down liegt der Eingang normalerweise auf LOW.

```
3.3V
 |
[Taster]
 |
 +---- Eingang
 |
[R]
 |
GND
```

Der Eingang liefert:

| **Taster** | **Wert** |
|---|---|
| nicht gedrückt | 0 |
| gedrückt | 1 |

## 32.9. Die interne Pull-Up-Funktion

Die meisten MMBasic-Systeme besitzen interne Pull-Up-Widerstände.

Dadurch wird keine zusätzliche Hardware benötigt.

PicoMite:

```
SETPIN GP1,DIN,PULLUP
```

Colour Maximite 2:

```
SETPIN 31,DIN,PULLUP
```

Der Eingang befindet sich nun automatisch auf HIGH.

## 32.10. Eine einfache Tasterschaltung

PicoMite:

```
GP1 --------+
            |
         [Taster]
            |
           GND
```

Colour Maximite 2:

```
Pin 31 -----+
            |
         [Taster]
            |
           GND
```

## 32.11. Den Eingang abfragen

PicoMite:

```
SETPIN GP1,DIN,PULLUP

PRINT PIN(GP1)
```

Colour Maximite 2:

```
SETPIN 31,DIN,PULLUP

PRINT PIN(31)
```

## 32.12. Das erste Tasterprogramm

PicoMite:

```
SETPIN GP1,DIN,PULLUP

DO

  IF PIN(GP1)=0 THEN
    PRINT "Gedrueckt"
  ENDIF

LOOP
```

Colour Maximite 2:

```
SETPIN 31,DIN,PULLUP

DO

  IF PIN(31)=0 THEN
    PRINT "Gedrueckt"
  ENDIF

LOOP
```

Da wir Pull-Up verwenden, liefert ein gedrückter Taster den Wert 0.

Das wirkt zunächst ungewohnt, ist aber vollkommen normal.

## 32.13. Eine LED mit einem Taster schalten

Nun verbinden wir das neue Wissen mit dem LED-Kapitel.

PicoMite:

```
SETPIN GP0,DOUT
SETPIN GP1,DIN,PULLUP

DO

  IF PIN(GP1)=0 THEN
    PIN(GP0)=1
  ELSE
    PIN(GP0)=0
  ENDIF

LOOP
```

Solange der Taster gedrückt wird, leuchtet die LED.

## 32.14. Ein Schalter mit Speicherfunktion

Manchmal soll ein Tastendruck etwas dauerhaft ein- oder ausschalten.

Beispielsweise:

- Licht ein/aus
- Alarm aktivieren
- Spiel starten

Dafür benötigen wir eine Variable.

```
LICHT=0
```

Bei jedem Tastendruck wird der Zustand umgeschaltet.

```
IF LICHT=0 THEN
  LICHT=1
ELSE
  LICHT=0
ENDIF
```

## 32.15. Warum zählt mein Taster mehrfach?

Betrachten wir folgendes Programm:

```
SETPIN GP1,DIN,PULLUP

ZAHL=0

DO

  IF PIN(GP1)=0 THEN

    ZAHL=ZAHL+1

    PRINT ZAHL

  ENDIF

LOOP
```

Was erwarten wir?

```
Ein Tastendruck
```

```
=
Ein neuer Wert
```

Tatsächlich erhalten wir oft:

```
1
2
3
4
5
```

obwohl wir nur einmal gedrückt haben.

## 32.16. Das Prellen von Kontakten

Der Grund liegt im mechanischen Aufbau eines Tasters.

Beim Drücken schließen die Kontakte nicht sofort sauber.

Sie springen für wenige Millisekunden mehrfach auf und zu.

Ideal:

```
0 -------------------- 1
```

Tatsächlich:

```
0 --1-0-1-0-1------1
```

Der Mikrocontroller erkennt mehrere Tastendrücke.

Dieses Verhalten nennt man **Prellen**.

## 32.17. Eine einfache Entprellung

Die einfachste Lösung besteht darin, nach einem Tastendruck kurz zu warten.

```
IF PIN(GP1)=0 THEN

  ZAHL=ZAHL+1

  PAUSE 50

ENDIF
```

Die Kontakte können sich während dieser Zeit beruhigen.

Für viele Anwendungen genügt das bereits.

## 32.18. Flankenerkennung

Noch besser ist die Erkennung eines Zustandswechsels.

Dazu merken wir uns den vorherigen Zustand.

```
ALT=1

DO

  AKTUELL=PIN(GP1)

  IF AKTUELL=0 AND ALT=1 THEN

    PRINT "Tastendruck"

  ENDIF

  ALT=AKTUELL

LOOP
```

Jetzt wird nur der Übergang von:

```
1 -> 0
```

erkannt.

Der Taster kann beliebig lange gedrückt bleiben.

## 32.19. Praxisprojekt: Reaktionsspiel

Nun entwickeln wir ein kleines Spiel.

Das Programm wartet eine zufällige Zeit.

Danach leuchtet eine LED auf.

Der Spieler muss möglichst schnell den Taster drücken.

Die Reaktionszeit wird mit `TIMER` gemessen.

## 32.20. Aufbau

Benötigt werden:

- eine LED
- ein Taster

PicoMite:

```
GP0 -> LED
GP1 -> Taster
```

Colour Maximite 2:

```
Pin 32 -> LED
Pin 31 -> Taster
```

## 32.21. Programm

```
SETPIN GP0,DOUT
SETPIN GP1,DIN,PULLUP

CLS

PRINT "Reaktionsspiel"
PRINT
PRINT "Warte auf das Signal..."

PAUSE 2000

WARTEZEIT=1000+INT(RND*4000)

PAUSE WARTEZEIT

PIN(GP0)=1

START=TIMER

DO
LOOP UNTIL PIN(GP1)=0

REAKTION=TIMER-START

PIN(GP0)=0

PRINT
PRINT "Deine Reaktionszeit:"
```

```
PRINT REAKTION;" ms"
```

## 32.22. Bewertung

Eine mögliche Bewertung:

| Zeit | Bewertung |
|---|---|
| unter 200 ms | Ausgezeichnet |
| 200 bis 300 ms | Sehr gut |
| 300 bis 400 ms | Gut |
| über 400 ms | Weiter üben |

## 32.23. Typische Anwendungen

Digitale Eingänge werden in vielen Projekten verwendet:

- Taster
- Schalter
- Alarmanlagen
- Lichtschranken
- Türkontakte
- Endschalter
- Bewegungsmelder

Sie gehören zu den wichtigsten Funktionen eines Mikrocontrollers.

## 32.24. Experimentiere!

Probiere folgende Änderungen aus:

- mehrere Taster
- andere Wartezeiten
- mehrere LEDs
- verschiedene Blinkmuster

## 32.25. Probier’s selbst!

Versuche folgende Aufgaben:

1. Baue einen Startknopf für ein Programm.
2. Schalte zwei LEDs mit zwei Tastern.

3. Zähle die Anzahl der Tastendrücke.
4. Messe die Zeit zwischen zwei Tastendrücken.
5. Programmiere einen elektronischen Münzwurf mit Tasterstart.

## 32.26. Zusammenfassung

In diesem Kapitel hast du gelernt:

- wie digitale Eingänge funktionieren
- wie Taster angeschlossen werden
- was Pull-Up-Widerstände sind
- warum Taster prellen
- wie eine einfache Entprellung funktioniert
- wie Programme auf Benutzereingaben reagieren

Damit beherrschst du nun die beiden wichtigsten Hardwarefunktionen eines Mikrocontrollers:

- Ausgänge steuern
- Eingänge auswerten

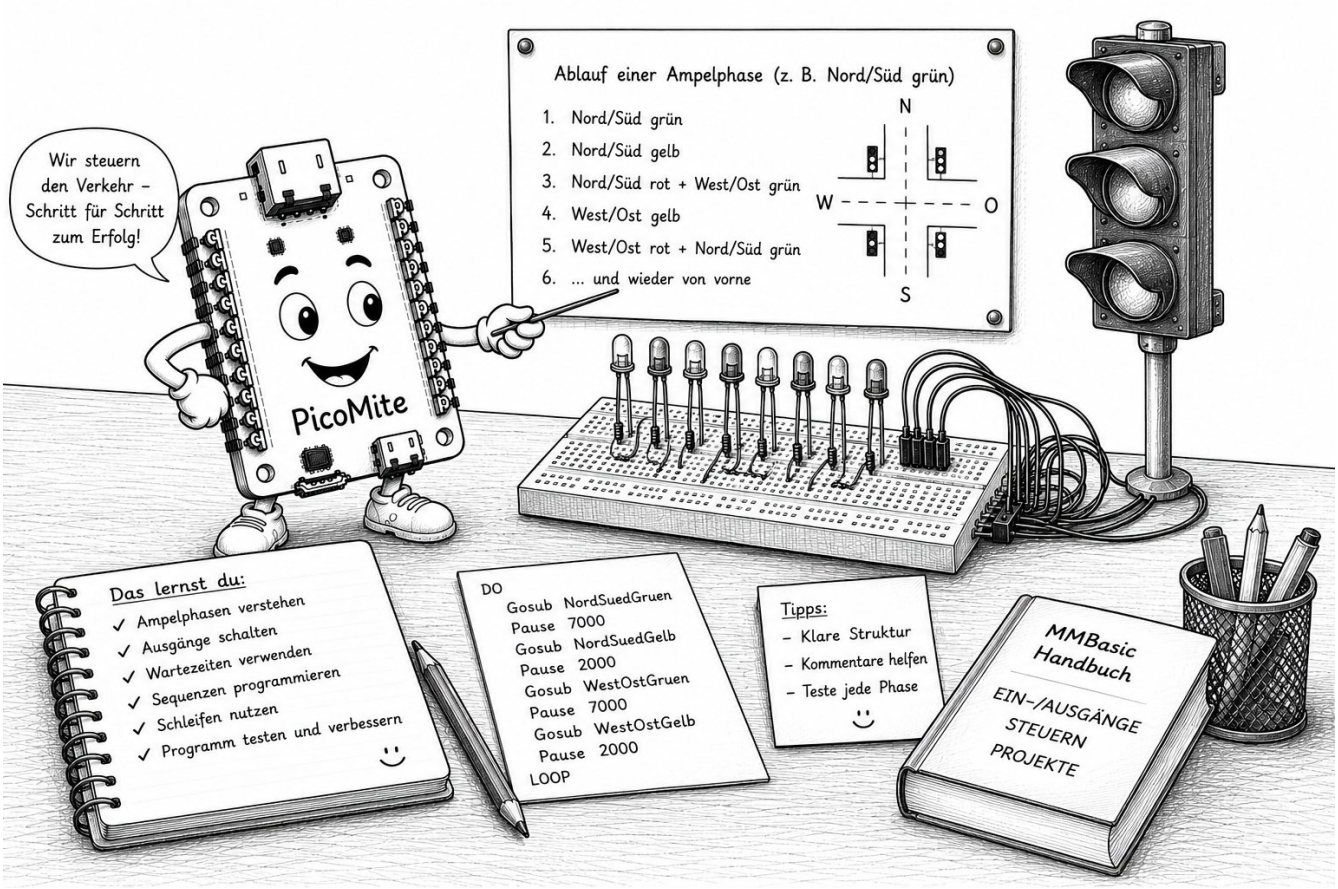
Im nächsten Kapitel werden wir dieses Wissen nutzen und eine Ampelsteuerung entwickeln.

## 32.27. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 32_taster_led.bas

# Kapitel 33. Eine Ampelsteuerung




## 33.1. Das lernst du

In diesem Kapitel lernst du:

- mehrere LEDs gleichzeitig anzusteuern
- Zustände in einem Programm zu verwalten
- den Befehl `SELECT CASE` zu verwenden
- eine Ampelsteuerung zu programmieren
- einen Taster auszuwerten
- eine Fußgängeranforderung umzusetzen

Am Ende des Kapitels kannst du eine einfache Verkehrsampel mit Fußgängeranforderung realisieren.

## 33.2. Warum eine Ampel?

Ampeln gehören zu den bekanntesten Steuerungssystemen überhaupt.

Sie arbeiten nach klaren Regeln:

- Rot bedeutet Halt

- Grün bedeutet Fahrt
- Gelb kündigt einen Zustandswechsel an

Eine Ampel ist deshalb ein ideales Beispiel für die Programmierung eines Mikrocontrollers.

## 33.3. Die Farben einer Verkehrsampel

Eine Fahrzeugampel besitzt normalerweise drei Lampen:

- Rot
- Gelb
- Grün

Die Reihenfolge lautet:

```
Rot
Rot-Gelb
Grün
Gelb
Rot
```

Danach beginnt der Zyklus erneut.

## 33.4. Unsere Hardware

Für das erste Beispiel benötigen wir:

- 1 rote LED
- 1 gelbe LED
- 1 grüne LED
- 3 Widerstände (220 Ω bis 470 Ω)

PicoMite:

| **LED** | **GPIO** |
|---|---|
| Rot | GP0 |
| Gelb | GP1 |
| Grün | GP2 |

Colour Maximite 2:

| **LED** | **Pin** |
|---|---|
| Rot | 32 |

| LED | Pin |
|---|---|
| Gelb | 33 |
| Grün | 34 |

## 33.5. Die Verdrahtung

Jede LED wird wie im vorherigen Kapitel angeschlossen.

```
GPIO ----[330 Ω]----|>|---- GND
```

Die Schaltung wird für jede LED wiederholt.

image::bilder/ampel_drei_leds.jpg[ align=center, title="Drei LEDs als Verkehrsampel" ]

## 33.6. Die erste Ampel

Beginnen wir mit einem einfachen Ablauf.

PicoMite:

```
SETPIN GP0,DOUT
SETPIN GP1,DOUT
SETPIN GP2,DOUT

DO

  PIN(GP0)=1
  PIN(GP1)=0
  PIN(GP2)=0

  PAUSE 3000

  PIN(GP0)=1
  PIN(GP1)=1
  PIN(GP2)=0

  PAUSE 1000

  PIN(GP0)=0
  PIN(GP1)=0
  PIN(GP2)=1

  PAUSE 3000

  PIN(GP0)=0
  PIN(GP1)=1
  PIN(GP2)=0
```

```
  PAUSE 1000

LOOP
```

Die Ampel durchläuft nun kontinuierlich alle Zustände.

## 33.7. Das Problem

Das Programm funktioniert.

Allerdings ist der Ablauf schwer zu lesen.

Wenn wir später weitere Funktionen hinzufügen, wird das schnell unübersichtlich.

## 33.8. Eine Ampel hat Zustände

Eine Ampel befindet sich immer in genau einem Zustand.

Zum Beispiel:

| **Zustand** | **Bedeutung** |
|---|---|
| 1 | Rot |
| 2 | Rot-Gelb |
| 3 | Grün |
| 4 | Gelb |

Wir speichern diesen Zustand in einer Variablen.

```
STATUS=1
```

## 33.9. SELECT CASE

Für solche Aufgaben eignet sich `SELECT CASE` hervorragend.

```
SELECT CASE STATUS

  CASE 1

  CASE 2

  CASE 3

  CASE 4
```

```
END SELECT
```

Je nach Wert von `STATUS` wird mit `CASE` ein anderer Programmteil ausgeführt. `END SELECT` beendet die Auswahlstruktur.

## 33.10. Die Ampel mit Zuständen

```
STATUS=1

DO

  SELECT CASE STATUS

    CASE 1

      PRINT "Rot"

      STATUS=2

    CASE 2

      PRINT "Rot-Gelb"

      STATUS=3

    CASE 3

      PRINT "Gruen"

      STATUS=4

    CASE 4

      PRINT "Gelb"

      STATUS=1

  END SELECT

LOOP
```

Jetzt ist der Ablauf deutlich besser erkennbar.

## 33.11. Warum sind Zustände wichtig?

Viele technische Systeme arbeiten mit Zuständen:

- Ampeln

- Aufzüge
- Waschmaschinen
- Fahrkartenautomaten
- Alarmanlagen

Dieses Programmierprinzip gehört zu den wichtigsten Techniken der Automatisierung.

# 33.12. Die Fußgängerampel

Nun erweitern wir die Anlage.

Zusätzlich benötigen wir:

- eine rote Fußgänger-LED
- eine grüne Fußgänger-LED

PicoMite:

| **LED** | **GPIO** |
|---|---|
| Fußgänger Rot | GP3 |
| Fußgänger Grün | GP4 |

# 33.13. Der Fußgängertaster

Zusätzlich kommt ein Taster hinzu.

PicoMite:

```
GP5 ----+
        |
      Taster
        |
       GND
```

Der Eingang wird mit Pull-Up betrieben.

```
SETPIN GP5,DIN,PULLUP
```

# 33.14. Normalbetrieb

Im Normalfall gilt:

```
Autos: Grün
```

```
Fußgänger: Rot
```

So bleibt der Verkehrsfluss erhalten.

## 33.15. Fußgängeranforderung

Wird der Taster gedrückt:

```
Autos Grün
     
Autos Gelb
     
Autos Rot
     
Fußgänger Grün
     
Fußgänger Rot
     
Autos Rot-Gelb
     
Autos Grün
```

Das entspricht dem Verhalten vieler realer Ampeln.

## 33.16. Eine Anforderung speichern

Der Tastendruck wird in einer Variablen gespeichert.

```
ANFORDERUNG=0
```

Wird der Taster gedrückt:

```
IF PIN(GP5)=0 THEN

  ANFORDERUNG=1

ENDIF
```

Die Information bleibt erhalten.

## 33.17. Warum speichern wir die Anforderung?

Vielleicht drückt ein Fußgänger den Taster genau dann, wenn die Ampel gerade auf Grün steht.

Die Ampel soll den Wunsch trotzdem berücksichtigen.

Deshalb wird die Anforderung gespeichert.

## 33.18. Das vollständige Konzept

Wir verwenden folgende Zustände:

| Status | Bedeutung |
|---|---|
| 1 | Autos Grün |
| 2 | Autos Gelb |
| 3 | Autos Rot |
| 4 | Fußgänger Grün |
| 5 | Fußgänger Rot |
| 6 | Autos Rot-Gelb |

## 33.19. Praxisprojekt: Fußgängerampel

```
SETPIN GP0,DOUT
SETPIN GP1,DOUT
SETPIN GP2,DOUT

SETPIN GP3,DOUT
SETPIN GP4,DOUT

SETPIN GP5,DIN,PULLUP

STATUS=1
ANFORDERUNG=0

DO

  IF PIN(GP5)=0 THEN
    ANFORDERUNG=1
  ENDIF

  SELECT CASE STATUS

    CASE 1

      ' Autos Gruen

      PIN(GP0)=0
      PIN(GP1)=0
      PIN(GP2)=1
```

```
      PIN(GP3)=1
      PIN(GP4)=0

      IF ANFORDERUNG=1 THEN

        PAUSE 1000

        STATUS=2

      ENDIF

    CASE 2

      ' Autos Gelb

      PIN(GP0)=0
      PIN(GP1)=1
      PIN(GP2)=0

      PAUSE 2000

      STATUS=3

    CASE 3

      ' Autos Rot

      PIN(GP0)=1
      PIN(GP1)=0
      PIN(GP2)=0

      PAUSE 1000

      STATUS=4

    CASE 4

      ' Fussgaenger Gruen

      PIN(GP3)=0
      PIN(GP4)=1

      PAUSE 5000

      STATUS=5

    CASE 5

      ' Fussgaenger Rot

      PIN(GP3)=1
```

```
      PIN(GP4)=0

      PAUSE 1000

      STATUS=6

    CASE 6

      ' Autos Rot-Gelb

      PIN(GP0)=1
      PIN(GP1)=1
      PIN(GP2)=0

      PAUSE 1000

      ANFORDERUNG=0

      STATUS=1

  END SELECT

LOOP
```

## 33.20. Was passiert im Programm?

Im Zustand 1 dürfen die Autos fahren.

Erst wenn ein Fußgänger den Taster drückt, wird die Ampel umgeschaltet.

Die Anforderung wird gespeichert.

Anschließend werden alle Zustände automatisch durchlaufen.

Am Ende beginnt der Zyklus erneut.

## 33.21. Erweiterungsmöglichkeiten

Eine echte Verkehrsampel besitzt noch weitere Funktionen:

- Blinkendes Grün
- Gelbblinkbetrieb
- Nachtabschaltung
- Akustische Signale
- Countdown-Anzeigen

Mit dem bisher erlernten Wissen könntest du viele dieser Funktionen bereits selbst ergänzen.

## 33.22. Experimentiere!

Probiere folgende Änderungen aus:

- längere Grünphasen
- kürzere Gelbphasen
- mehrere Fußgängertaster
- blinkendes Fußgänger-Grün

## 33.23. Probier’s selbst!

Versuche folgende Aufgaben:

1. Lasse die Fußgängerampel vor dem Umschalten dreimal blinken.
2. Ergänze einen Summer.
3. Zähle die Anzahl der Fußgängeranforderungen.
4. Baue eine Nachtabschaltung ein.
5. Ergänze einen zweiten Taster für eine weitere Straßenseite.

## 33.24. Zusammenfassung

In diesem Kapitel hast du gelernt:

- mehrere LEDs gleichzeitig anzusteuern
- Zustände in Programmen zu verwenden
- den Befehl `SELECT CASE` einzusetzen
- Taster und LEDs zu kombinieren
- eine einfache Fußgängerampel zu programmieren

Du hast damit dein erstes kleines Automatisierungsprojekt entwickelt.

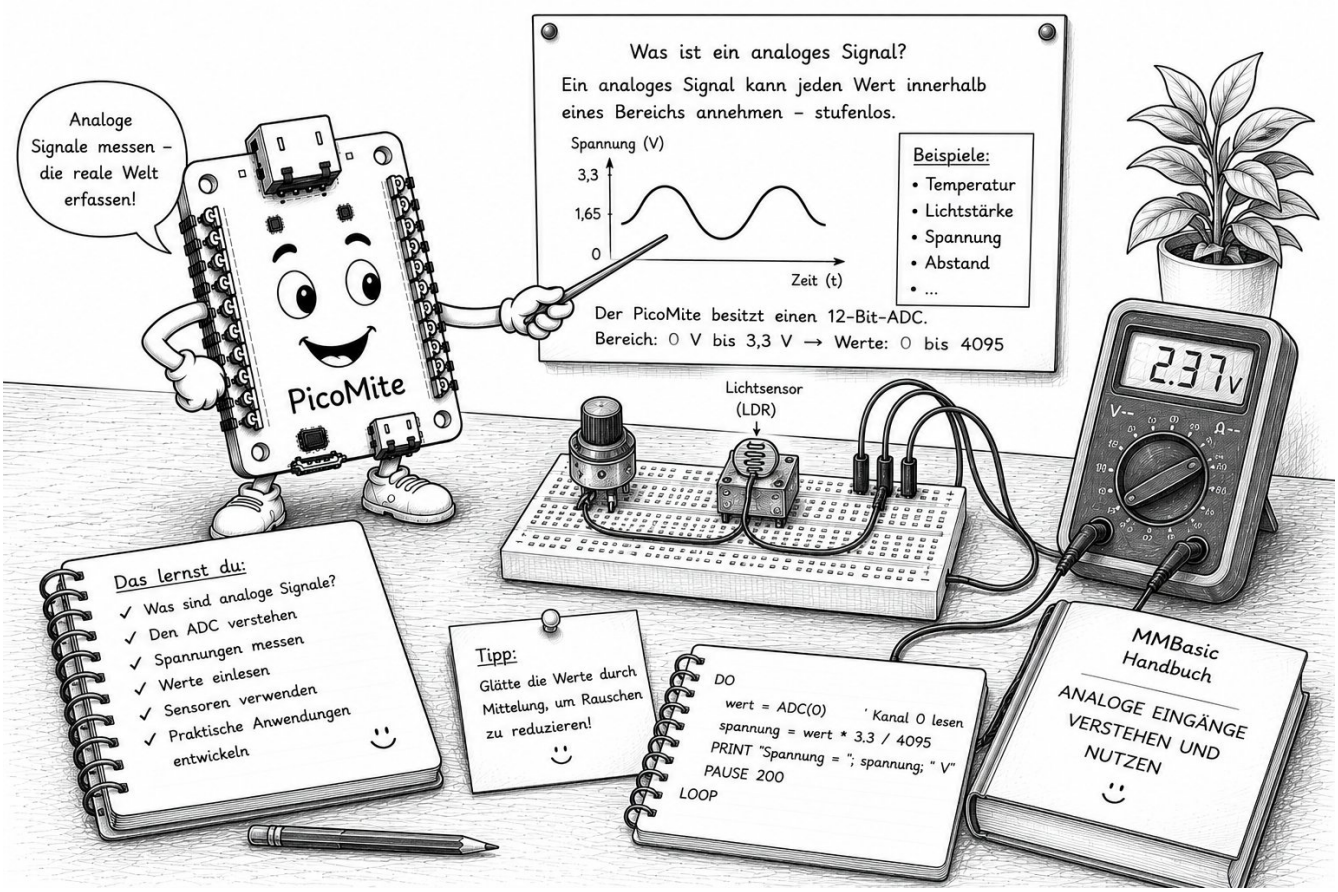
Im nächsten Kapitel werden wir analoge Eingänge kennenlernen und kontinuierliche Messwerte auswerten.

## 33.25. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 33_ampelsteuerung.bas

# Kapitel 34. Analoge Eingänge




## 34.1. Das lernst du

In diesem Kapitel lernst du:

- was ein analoges Signal ist
- den Unterschied zwischen digitalen und analogen Eingängen
- wie ein Analog-Digital-Wandler arbeitet
- wie ein Potentiometer angeschlossen wird
- analoge Spannungen einzulesen
- Messwerte in Prozent umzurechnen
- Messwerte grafisch darzustellen
- mehrere Messungen zu mitteln
- analoge Sensoren auszuwerten

Am Ende des Kapitels kannst du kontinuierliche Messwerte erfassen und für eigene Projekte verwenden.

## 34.2. Von der Ampel zur Messtechnik

Bisher kannten unsere Programme bei Eingängen hauptsächlich zwei Zustände:

- Ein
- Aus

Viele technische Anwendungen benötigen jedoch deutlich mehr Informationen.

Ein Temperaturfühler liefert beispielsweise nicht nur "warm" oder "kalt".

Ein Potentiometer kennt nicht nur "links" oder "rechts".

Stattdessen können sich die Messwerte kontinuierlich verändern.

Genau dafür gibt es analoge Eingänge.

Mit ihnen beginnt die Welt der Mess-, Steuer- und Regeltechnik.

## 34.3. Digitale und analoge Signale

Ein digitaler Eingang kennt nur zwei Zustände:

```
0
1
```

Das genügt zum Beispiel für:

- Taster
- Schalter
- Lichtschranken
- Endschalter

Viele Sensoren liefern jedoch Zwischenwerte.

Beispiele:

- Temperatur
- Helligkeit
- Lautstärke
- Druck
- Feuchtigkeit

Hier reichen nur 0 und 1 nicht mehr aus.

## 34.4. Ein Vergleich

Stell dir einen Lichtschalter vor.

Er kennt nur:

```
Ein
Aus
```

Das entspricht einem digitalen Signal.

Ein Lichtdimmer dagegen kann viele Zwischenstellungen besitzen:

```
0 %
10 %
20 %
...
100 %
```

Das entspricht einem analogen Signal.

## 34.5. Analoge Spannungen

Ein analoger Eingang misst eine Spannung.

Beim PicoMite und Colour Maximite 2 reicht der Messbereich der analogen Eingänge von:

```
0 Volt
bis
3,3 Volt
```

An einem analogen Eingang darf keine Spannung über 3,3 V anliegen. Höhere Spannungen müssen beispielsweise mit einem geeigneten Spannungsteiler reduziert werden.

## 34.6. Der Analog-Digital-Wandler

Ein Mikrocontroller arbeitet intern digital. Eine analoge Spannung muss deshalb zunächst in einen digitalen Wert umgewandelt werden.

Dafür besitzt er einen sogenannten Analog-Digital-Wandler (ADC):

oft auch kurz:

```
A/D-Wandler
```

oder:

```
ADC
```

genannt.

ADC steht für *Analog-to-Digital Converter*.

# 34.7. Was macht ein A/D-Wandler?

Der A/D-Wandler misst die anliegende Spannung und wandelt sie intern in einen digitalen Wert um.

MMBasic nimmt uns bei einem mit `AIN` konfigurierten Eingang die weitere Umrechnung ab: Die Funktion `PIN()` liefert die gemessene Spannung direkt in Volt zurück.

Beispiele:

| **Spannung am Eingang** | **Rückgabewert von** `PIN()` |
|---|---|
| 0,0 V | 0.0 |
| 1,65 V | etwa 1.65 |
| 3,3 V | etwa 3.3 |

Wir müssen den internen ADC-Zahlenwert für normale Spannungsmessungen also nicht selbst auswerten.

## 34.7.1. Zusatzwissen: Rohwert beim PicoMite

Beim PicoMite kann ein ADC-Eingang alternativ mit `ARAW` konfiguriert werden:

```
SETPIN GP40,ARAW
```

In diesem Modus liefert `PIN()` einen einzelnen, unbearbeiteten ADC-Messwert im Bereich:

```
0 bis 4095
```

Für unsere Beispiele verwenden wir jedoch `AIN`, weil wir damit direkt mit Spannungen in Volt arbeiten können.

`ARAW` ist eine PicoMite-Funktion. Beim Colour Maximite 2 verwenden wir für die hier gezeigten Messungen `AIN`.

# 34.8. Das Potentiometer

Für erste Experimente eignet sich ein Potentiometer hervorragend.

Ein Potentiometer ist ein verstellbarer Widerstand. Es besitzt drei Anschlüsse.

```
3,3 V ----+
```

```
           \
            ) Potentiometer
           /
GND  ------+

Mittelanschluss
     |
     +---- Analogeingang
```

## 34.9. Das Potentiometer als Spannungsteiler

Durch Drehen des Knopfes verändert sich die Spannung am Mittelanschluss.

Beispiele:

```
Linksanschlag     0,0 V

Mittelstellung    etwa 1,65 V

Rechtsanschlag    etwa 3,3 V
```

Der Analogeingang kann diese Spannung messen.

## 34.10. Geeignete Potentiometer

Für unsere Versuche eignen sich Potentiometer beispielsweise im Bereich:

```
1 kΩ bis 100 kΩ
```

Sehr häufig verwendet werden:

```
10 kΩ
```

Potentiometer.

## 34.11. Anschluss am PicoMite

Für einen analogen Eingang muss ein Pin verwendet werden, der beim jeweiligen PicoMite als ADC-Eingang zur Verfügung steht.

In unserem Beispielprogramm verwenden wir:

```
GP40
```

Dieser Pin steht beim für das Beispiel verwendeten PicoMite HDMI/USB als ADC-Eingang zur Verfügung.

Beim RP2350 hängt die Lage der ADC-Eingänge von der verwendeten Chipvariante ab:

| **RP2350-Variante** | **ADC-Pins** |
|---|---|
| RP2350A | GP26 bis GP29 |
| RP2350B | GP40 bis GP47 |

Der in diesem Tutorial verwendete PicoMite HDMI/USB besitzt einen RP2350B. Deshalb verwenden wir für unsere Beispiele GP40.

Bei einem PicoMite mit RP2350A muss stattdessen ein geeigneter ADC-Pin aus dem Bereich GP26 bis GP29 verwendet werden.

Welche GPIO-Pins als ADC-Eingänge zur Verfügung stehen, hängt von der verwendeten Hardware ab. Prüfe deshalb im Zweifelsfall die Pinbelegung deines PicoMite.

## 34.12. Anschluss am Colour Maximite 2

Der Colour Maximite 2 besitzt ebenfalls analoge Eingänge.

Dort werden die entsprechenden Pinnummern anstelle von `GP40` verwendet. Die genaue Pinbelegung findest du im Handbuch des Colour Maximite 2.

## 34.13. Einen Analogeingang konfigurieren

Beim PicoMite konfigurieren wir unseren Beispielpin so:

```
SETPIN GP40,AIN
```

Das Kürzel `AIN` bedeutet:

```
Analog Input
```

## 34.14. Die Spannung lesen

Anschließend kann die Spannung mit `PIN()` gelesen werden:

```
WERT=PIN(GP40)
```

Bei `AIN` liefert `PIN()` die gemessene Spannung direkt in Volt.

Der Messbereich reicht von ungefähr:

```
0.0
bis
3.3
```

## 34.15. GPIO-Nummer und Pinnummer

In den bisherigen Beispielen verwenden wir direkt die GPIO-Bezeichnung:

```
SETPIN GP40,AIN
WERT=PIN(GP40)
```

Dabei bezeichnet `GP40` den GPIO-Pin. `PIN()` ist dagegen eine MMBasic-Funktion, mit der der Wert eines Pins gelesen wird.

Bei vielen MMBasic-Befehlen kann die `GPxx`-Schreibweise direkt verwendet werden. Es gibt jedoch auch Situationen, in denen eine numerische Pinnummer benötigt wird.

Mit `MM.INFO(PINNO GP40)` kann die GPIO-Bezeichnung in die entsprechende Pinnummer umgewandelt werden:

```
PRINT MM.INFO(PINNO GP40)
```

Bei größeren Programmen kann es sinnvoll sein, die verwendeten Pins am Anfang des Programms als Konstanten festzulegen:

```
CONST VinPin=MM.INFO(PINNO GP40)

SETPIN VinPin,AIN
Battery=PIN(VinPin)

PRINT Battery
```

Dadurch wird die Hardwarebelegung an einer zentralen Stelle im Programm festgelegt. Soll später ein anderer Pin verwendet werden, muss nur die Definition der Konstanten geändert werden.

Für unsere einfachen Beispiele verwenden wir weiterhin direkt `GP40`:

```
SETPIN GP40,AIN
WERT=PIN(GP40)
```

## 34.16. Unser erstes Messprogramm

```
SETPIN GP40,AIN

DO
  PRINT PIN(GP40)
  PAUSE 200
LOOP WHILE INKEY$=""
```

Drehst du am Potentiometer, ändern sich die angezeigten Spannungswerte.

Beispiel:

```
0.124
0.736
1.648
2.315
3.102
```

Mit einer Taste wird die Schleife beendet.

## 34.17. Was bedeutet der Messwert?

Je größer die Spannung am Analogeingang ist, desto größer ist der von `PIN()` zurückgegebene Spannungswert.

| **Wert** | **Bedeutung** |
|---|---|
| 0.0 | 0 Volt |
| etwa 1.65 | etwa halbe Eingangsspannung |
| etwa 3.3 | maximale Eingangsspannung |

## 34.18. Messwerte auf Prozent umrechnen

Häufig sind Prozentwerte einfacher zu verstehen.

Bei einem Messbereich von 0 bis 3,3 V können wir die Spannung so in Prozent umrechnen:

```
PROZENT=WERT*100/3.3
```

Beispiele:

| **Spannung** | **Prozent** |
|---|---|
| 0,0 V | 0 % |

| Spannung | Prozent |
|---|---|
| 1,65 V | 50 % |
| 3,3 V | 100 % |

## 34.19. Eine Prozentanzeige

```
SETPIN GP40,AIN

DO
  WERT=PIN(GP40)
  PROZENT=WERT*100/3.3
  PRINT INT(PROZENT);" %"
  PAUSE 200
LOOP WHILE INKEY$=""
```

Nun wird die Stellung des Potentiometers als Prozentwert angezeigt.

## 34.20. Eine grafische Balkenanzeige

Messwerte lassen sich oft besser grafisch darstellen.

```
MODE 2

SETPIN GP40,AIN

DO
  WERT=PIN(GP40)
  BREITE=WERT*MM.HRES/3.3

  BOX 20,50,BREITE,40,0,RGB(GREEN),RGB(GREEN)
  TEXT 20,110,STR$(INT(WERT*100/3.3))+" %"

  PAUSE 50
LOOP WHILE INKEY$=""
```

Der Balken wächst und schrumpft nun beim Drehen des Potentiometers.

## 34.21. Schwankende Messwerte

Auch bei einer eigentlich konstanten Eingangsspannung können Messwerte leicht schwanken.

Beispielsweise:

```
1.648
1.651
```

```
1.647
1.652
1.650
```

Das ist bei analogen Messungen normal.

Ursachen können beispielsweise sein:

- elektrisches Rauschen
- kleine Schwankungen der Versorgungsspannung
- Sensorungenauigkeiten

Beim PicoMite führt MMBasic bei `AIN` bereits mehrere Messungen durch. Der Eingang wird zehnmal abgetastet. Die beiden höchsten und die beiden niedrigsten Werte werden verworfen und die verbleibenden sechs Werte gemittelt.

Dadurch ist der von `PIN()` gelieferte Spannungswert bereits geglättet.

## 34.22. Mehrere Messungen mitteln

Falls wir eine noch stärkere Glättung benötigen, können wir zusätzlich mehrere mit `PIN()` gelesene Spannungswerte mitteln.

```
SETPIN GP40,AIN

SUMME=0

FOR I=1 TO 10
  WERT=PIN(GP40)
  PRINT WERT
  SUMME=SUMME+WERT
  PAUSE 200
NEXT I

WERT=SUMME/10
PRINT WERT
```

Das Programm zeigt zunächst die zehn einzelnen Spannungswerte an. Anschließend wird deren Mittelwert berechnet und ausgegeben.

## 34.23. Praxisprojekt: Elektronische Tankanzeige

Nun bauen wir eine einfache Tankanzeige.

Dabei simuliert das Potentiometer den Tankgeber eines Fahrzeugs.

```
Potentiometer
```

```
=
Tankgeber

Spannung
=
Tankinhalt
```

## 34.24. Die Anzeige

```
MODE 2

SETPIN GP40,AIN

DO
  WERT=PIN(GP40)
  PROZENT=WERT*100/3.3

  TEXT MM.HRES/2,20,"Tankanzeige","CT"

  BOX 40,80,200,30

  BOX 40,80,PROZENT*2,30,0,RGB(GREEN),RGB(GREEN)

  TEXT MM.HRES/2,140,STR$(INT(PROZENT))+" %","CT"

  PAUSE 100
LOOP WHILE INKEY$=""
```

Beim Drehen des Potentiometers verändert sich nun der angezeigte Tankfüllstand.

## 34.25. Wo werden analoge Eingänge verwendet?

Analoge Eingänge findet man in vielen Anwendungen:

- Temperaturfühler
- Helligkeitssensoren
- Joysticks
- Potentiometer
- Drucksensoren
- Abstandssensoren
- Feuchtigkeitssensoren

Sie gehören zu den wichtigsten Funktionen moderner Mikrocontroller.

## 34.26. Typische Fehler

Wenn keine sinnvollen Werte erscheinen:

1. Ist das Potentiometer richtig angeschlossen?
2. Wurde ein für Analogmessungen geeigneter ADC-Pin verwendet?
3. Wurde `SETPIN ⋯,AIN` aufgerufen?
4. Sind 3,3 V und GND korrekt verbunden?
5. Liegt die Eingangsspannung wirklich zwischen 0 und 3,3 V?

## 34.27. Experimentiere!

Probiere folgende Änderungen aus:

- andere Potentiometer
- andere Farben
- größere Balkenanzeigen
- Prozentanzeige mit einer Nachkommastelle

## 34.28. Probier’s selbst!

Versuche folgende Aufgaben:

1. Baue eine Temperaturanzeige mit einem geeigneten analogen Sensor.
2. Zeichne einen horizontalen Balken.
3. Zeichne einen vertikalen Balken.
4. Erstelle eine Tankwarnung unter 20 %.
5. Zeige den Messwert zusätzlich als Spannung in Volt an.

## 34.29. Zusammenfassung

In diesem Kapitel hast du gelernt:

- was analoge Signale sind
- wozu ein A/D-Wandler benötigt wird
- wie ein Analogeingang mit `AIN` konfiguriert wird
- dass `PIN()` bei `AIN` die gemessene Spannung direkt in Volt liefert
- wie ein Potentiometer als Spannungsteiler angeschlossen wird
- wie Spannungswerte in Prozent umgerechnet werden
- wie Messwerte grafisch dargestellt und zusätzlich gemittelt werden

Beim PicoMite hast du außerdem `ARAW` als Möglichkeit kennengelernt, den direkten ADC-Rohwert von 0 bis 4095 einzulesen.

Damit kann dein Mikrocontroller nun nicht nur digitale Zustände erkennen, sondern auch kontinuierliche Messwerte erfassen.

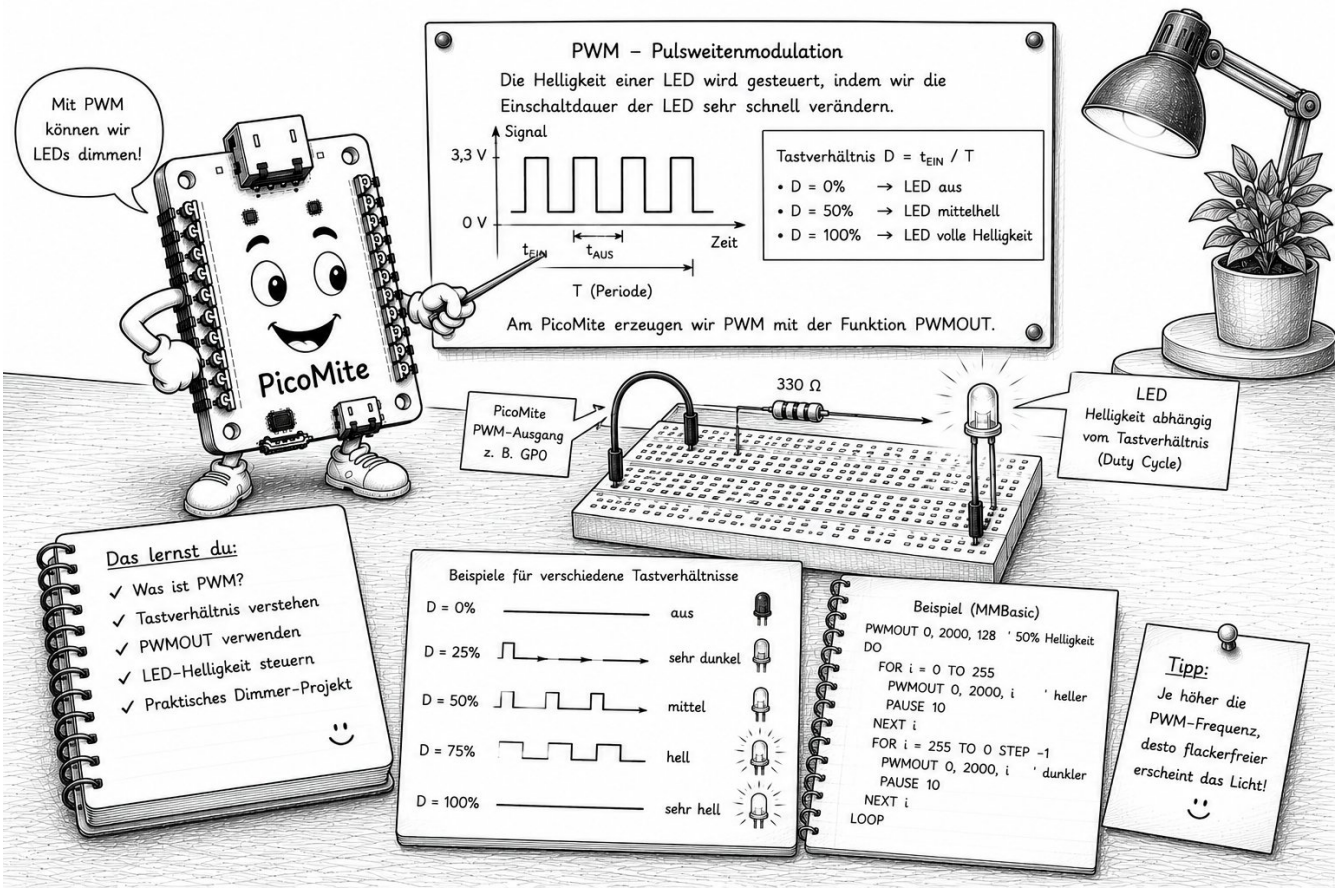
Im nächsten Kapitel werden wir PWM kennenlernen und die Helligkeit einer LED stufenlos steuern.

## 34.30. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 34_adc_einlesen.bas

# Kapitel 35. PWM und LED-Dimmer




## 35.1. Das lernst du

In diesem Kapitel lernst du:

- was PWM (Pulsweitenmodulation) ist
- wie PWM funktioniert
- was ein Tastverhältnis ist
- wie eine LED stufenlos gedimmt werden kann
- wie PWM als analoger Ausgang verwendet wird
- wie ein Potentiometer die Helligkeit einer LED steuern kann

Am Ende des Kapitels kannst du die Helligkeit einer LED stufenlos verändern und eigene PWM-Anwendungen entwickeln.

## 35.2. Warum PWM?

Im letzten Kapitel konnten wir analoge Werte messen.

Mit einem Potentiometer haben wir beispielsweise Werte zwischen:

```
0
```

```
und
1023
```

eingelesen.

Nun möchten wir diese Werte verwenden, um die Helligkeit einer LED zu steuern.

Das klingt zunächst einfach.

Ein Mikrocontroller-Ausgang kennt jedoch normalerweise nur zwei Zustände:

```
EIN
AUS
```

Wie lässt sich daraus eine halbe Helligkeit erzeugen?

Die Lösung heißt:

```
PWM
```

## 35.3. Was bedeutet PWM?

PWM steht für:

```
Pulse Width Modulation
```

auf Deutsch:

```
Pulsweitenmodulation
```

Dabei wird ein Ausgang sehr schnell ein- und ausgeschaltet.

## 35.4. Ein Vergleich

Stell dir eine Lampe vor.

Sie ist immer nur:

```
EIN
oder
AUS
```

Wird sie jedoch sehr schnell ein- und ausgeschaltet, nimmt unser Auge nur noch eine mittlere Helligkeit wahr.

## 35.5. Das Tastverhältnis

Entscheidend ist das Verhältnis zwischen:

- Einschaltzeit
- Ausschaltzeit

Dieses Verhältnis nennt man :

```
Tastverhältnis
```

oder englisch :

```
Duty Cycle
```

## 35.6. Beispiele

100 % Tastverhältnis:

```
1111111111111111
```

Die LED ist während der gesamten Periode eingeschaltet.

50 % Tastverhältnis:

```
1111000011110000
```

Die LED ist etwa gleich lange ein- und ausgeschaltet.

25 % Tastverhältnis:

```
1100000011000000
```

Die LED ist nur während eines Viertels der Zeit eingeschaltet.

Dabei bedeutet:

```
1 = Ausgang eingeschaltet
0 = Ausgang ausgeschaltet
```

## 35.7. Warum sehen wir kein Flackern?

PWM arbeitet normalerweise mit hohen Frequenzen.

Typische Werte sind:

```
100 Hz
500 Hz
1000 Hz
```

Unser Auge kann diese schnellen Änderungen nicht mehr erkennen.

Wir sehen lediglich die resultierende Helligkeit.

## 35.8. PWM beim PicoMite

Der PicoMite besitzt mehrere PWM-Kanäle.

Ein PWM-Signal wird mit dem `PWM`-Befehl:

```
PWM Kanal,Frequenz,Tastverhaeltnis
```

erzeugt.

Beispiel:

```
PWM 0,1000,50
```

Das bedeutet:

| Parameter | Bedeutung |
|---|---|
| 0 | PWM-Kanal |
| 1000 | 1000 Hz Frequenz |
| 50 | 50 % Tastverhältnis |

## 35.9. PWM-Ausgänge konfigurieren

Bevor ein GPIO-Pin verwendet werden kann, muss er einem PWM-Kanal zugeordnet und mit `PWM0A` als PWM-Ausgang konfiguriert werden.

Beispiel:

```
SETPIN GP0,PWM0A
```

Nun wird das PWM-Signal des Kanals 0 auf GP0 ausgegeben.

## 35.10. Erste PWM-Ausgabe

```
SETPIN GP0,PWM0A

PWM 0,1000,50
```

Die LED sollte nun etwa halb so hell leuchten.

## 35.11. Verschiedene Helligkeiten

25 %:

```
PWM 0,1000,25
```

50 %:

```
PWM 0,1000,50
```

75 %:

```
PWM 0,1000,75
```

100 %:

```
PWM 0,1000,100
```

## 35.12. PWM stoppen

Mit `PWM Kanal,OFF` wird die PWM-Ausgabe beendet:

```
PWM 0,OFF
```

Der PWM-Kanal wird abgeschaltet.

## 35.13. Der Colour Maximite 2

Auch der Colour Maximite 2 unterstützt PWM.

Dort werden PWM-Controller verwendet.

Beispiel:

```
PWM 1,1000,50
```

Dadurch wird auf dem ersten PWM-Ausgang ein Signal mit:

- 1000 Hz
- 50 % Tastverhältnis

erzeugt.

Die Grundidee ist auf beiden Plattformen identisch.

## 35.14. Eine LED langsam heller werden lassen

Nun erzeugen wir einen einfachen Aufblendeffekt.

```
SETPIN GP0,PWM0A

FOR HELLIGKEIT=0 TO 100

  PWM 0,1000,HELLIGKEIT

  PAUSE 20

NEXT HELLIGKEIT
```

Die LED wird langsam heller.

## 35.15. Eine LED langsam dunkler werden lassen

```
SETPIN GP0,PWM0A

FOR HELLIGKEIT=100 TO 0 STEP -1

  PWM 0,1000,HELLIGKEIT

  PAUSE 20

NEXT HELLIGKEIT
```

Die LED wird langsam dunkler.

## 35.16. Atemlicht

Kombinieren wir beide Effekte.

```
SETPIN GP0,PWM0A

DO

  FOR H=0 TO 100

    PWM 0,1000,H

    PAUSE 10

  NEXT H

  FOR H=100 TO 0 STEP -1

    PWM 0,1000,H

    PAUSE 10

  NEXT H

LOOP WHILE INKEY$=""
```

Die LED wirkt nun, als würde sie atmen.

## 35.17. PWM und die mittlere Ausgangsspannung

PWM wird häufig verwendet, um analoge Spannungen zu simulieren.

Betrachten wir folgende Werte:

| **PWM** | **Mittlere Spannung** |
|---|---|
| 0 % | 0,0 V |
| 50 % | ca. 1,65 V |
| 100 % | ca. 3,3 V |

Mit einem Tiefpassfilter kann daraus eine nahezu analoge Spannung erzeugt werden.

## 35.18. PWM in der Praxis

PWM wird verwendet für:

- LED-Dimmer

- Motorsteuerungen
- Lüfterregelungen
- Servoansteuerungen
- Audioausgabe
- Netzteile

Es gehört zu den wichtigsten Techniken moderner Mikrocontroller.

# 35.19. Praxisprojekt: Dimmer mit Potentiometer

Nun verbinden wir die Kapitel 34 und 35.

Das Potentiometer liefert den Sollwert.

PWM steuert die Helligkeit.

# 35.20. Die Hardware

Benötigt werden:

- Potentiometer an GP40
- LED an GP0
- Vorwiderstand 330 Ω

```
GP40 -> Potentiometer

GP0 -> LED
```

# 35.21. Programm

```
SETPIN GP40,AIN
SETPIN GP0,PWM0A
```

DO

WERT=PIN(GP40)

HELLIGKEIT=WERT*100/3.3

PWM 0,1000,HELLIGKEIT

PAUSE 20

LOOP

Nun kann die Helligkeit der LED mit dem Potentiometer eingestellt werden.

## 35.22. Erweiterung: Anzeige der Helligkeit

```
SETPIN GP40,AIN
SETPIN GP0,PWM0A
```

DO

WERT=PIN(GP40)

HELLIGKEIT=WERT*100/3.3

PWM 0,1000,HELLIGKEIT

CLS

PRINT "Helligkeit:" PRINT INT(HELLIGKEIT);" %"

PAUSE 100

LOOP

Die aktuelle Helligkeit wird zusätzlich angezeigt.

## 35.23. Typische Fehler

Wenn die LED nicht reagiert:

1. Ist der richtige PWM-Pin verwendet?
2. Wurde SETPIN korrekt aufgerufen?
3. Ist die LED richtig angeschlossen?
4. Ist ein Vorwiderstand vorhanden?
5. Wurde der richtige PWM-Kanal verwendet?

## 35.24. Experimentiere!

Probiere folgende Änderungen aus:

andere PWM-Frequenzen mehrere LEDs unterschiedliche Helligkeitskurven langsamere oder schnellere Atemeffekte

## 35.25. Probier’s selbst!

Versuche folgende Aufgaben:

1. Erzeuge einen sanften Sonnenaufgang.
2. Lasse zwei LEDs gegeneinander dimmen.
3. Verwende zwei Potentiometer für zwei LEDs.
4. Baue eine einfache Instrumentenbeleuchtung.
5. Simuliere das Blinken einer Warnleuchte.

## 35.26. Zusammenfassung

In diesem Kapitel hast du gelernt:

was PWM ist wie ein Tastverhältnis funktioniert wie LEDs gedimmt werden können wie PWM analoge Spannungen simuliert wie ein Potentiometer eine LED-Helligkeit steuern kann

Damit beherrschst du nun die wichtigsten Ein- und Ausgabefunktionen eines Mikrocontrollers:

Digitale Eingänge Digitale Ausgänge Analoge Eingänge PWM-Ausgänge

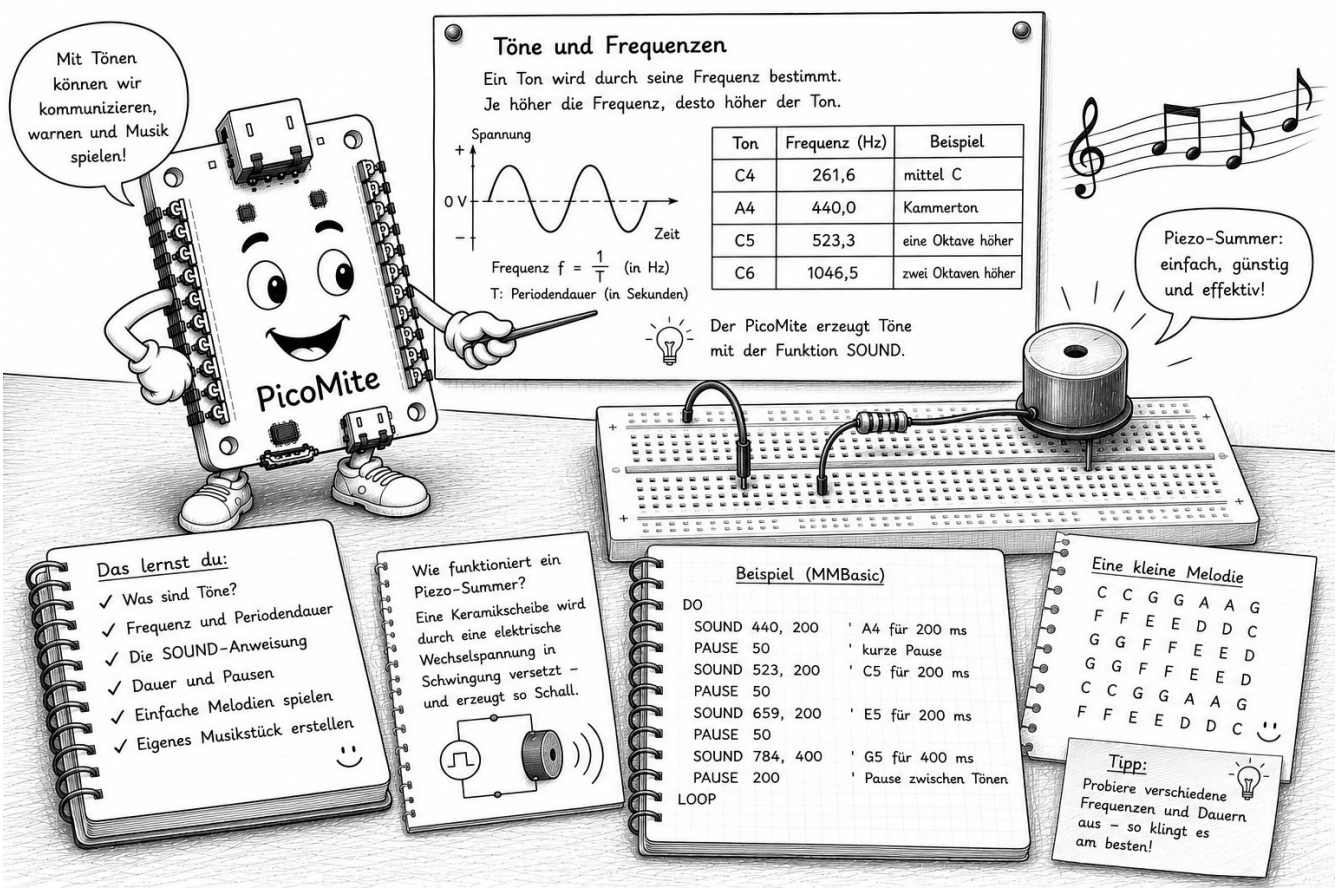
Im nächsten Kapitel werden wir Töne erzeugen und einen Piezo-Summer ansteuern.

## 35.27. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 35_led_dimmer.bas

# Kapitel 36. Töne und Piezo-Summer




## 36.1. Das lernst du

In diesem Kapitel lernst du:

- was ein Piezo-Summer ist
- den Unterschied zwischen aktiven und passiven Summern
- wie Töne erzeugt werden
- wie Tonhöhe und Frequenz zusammenhängen
- einfache Melodien zu programmieren
- Arrays für Notenfolgen zu verwenden

Am Ende des Kapitels kannst du eigene Melodien, Klingeltöne und Soundeffekte erzeugen.

## 36.2. Wenn Programme hörbar werden

Bisher konnten unsere Programme:

- Texte anzeigen
- Grafiken zeichnen
- LEDs steuern

- Taster auswerten

Nun kommt ein weiterer Sinn hinzu:

```
Hören
```

Zum ersten Mal erzeugt unser Programm Töne.

# 36.3. Was ist ein Piezo-Summer?

Ein Piezo-Summer ist ein kleines Bauteil zur Tonerzeugung.

Piezo-Summer werden häufig verwendet in:

- Alarmanlagen
- Küchengeräten
- Uhren
- Messgeräten
- Spielzeugen

# 36.4. Aktive und passive Summer

Es gibt zwei unterschiedliche Varianten.

## 36.4.1. Aktiver Summer

Ein aktiver Summer besitzt einen eingebauten Oszillator.

Wird er mit der vorgesehenen Spannung versorgt, erzeugt er selbstständig einen Ton. Die Tonhöhe ist dabei normalerweise fest vorgegeben.

Ein aktiver Summer eignet sich deshalb gut für einfache Signaltöne, aber nicht zum Spielen unterschiedlicher Noten.

## 36.4.2. Passiver Piezo-Schallwandler

Ein passiver Piezo-Schallwandler besitzt keinen eingebauten Tongenerator. Er benötigt ein wechselndes elektrisches Signal. Dessen Frequenz bestimmt die Tonhöhe.

Damit lassen sich unterschiedliche Töne und einfache Melodien erzeugen.

# 36.5. Die Audioausgabe von MMBasic

Für die folgenden Beispiele verwenden wir die Audioausgabe des jeweiligen MMBasic-Systems.

`PLAY TONE` gibt den Ton über die von MMBasic konfigurierte Audioausgabe aus. Der

Befehl erzeugt nicht einfach an einem beliebigen GPIO-Pin ein Tonsignal. Welche Anschlüsse für die Audioausgabe verwendet werden, hängt deshalb von der verwendeten PicoMite- oder Colour-Maximite-Hardware und deren Konfiguration ab.

Die Beispiele dieses Kapitels wurden auf einem **PicoMite HDMI/USB** überprüft.

Beim PicoMite wird ein Ton mit `PLAY TONE` erzeugt. Ein eigenständiger Befehl `TONE` ist nicht vorhanden.

## 36.6. Der `PLAY TONE`-Befehl

Zum Erzeugen von Tönen stellt MMBasic den Befehl `PLAY TONE` zur Verfügung.

Der allgemeine Aufbau lautet:

```
PLAY TONE FrequenzLinks [, FrequenzRechts [, Dauer [, Interrupt]]]
```

Die Parameter haben folgende Bedeutung:

| **Parameter** | **Bedeutung** |
|---|---|
| FrequenzLinks | Tonhöhe des linken Kanals in Hertz (Hz) |
| FrequenzRechts | Tonhöhe des rechten Kanals in Hertz (Hz) |
| Dauer | Spieldauer des Tons in Millisekunden (ms) |
| Interrupt | Optionale Subroutine, die nach Ablauf der Spieldauer aufgerufen wird |

Je höher die Frequenz ist, desto höher ist die Tonhöhe.

`PLAY TONE` arbeitet im Hintergrund. Das Programm läuft also weiter, während der Ton ausgegeben wird. Deshalb verwenden wir in vielen Beispielen zusätzlich `PAUSE`, wenn der nächste Programmschritt erst nach dem Ton ausgeführt werden soll.

Wird keine Dauer angegeben, läuft der Ton weiter, bis er beispielsweise mit

```
PLAY TONE 0,0,0
```

gestoppt oder das Programm beendet wird.

## 36.7. Der erste Ton

Mit MMBasic lässt sich ein Ton sehr einfach erzeugen:

```
PLAY PLAY TONE 1000,1000,100 : PAUSE 1000,500
```

```
PAUSE 500
```

Das bedeutet:

| Parameter | Bedeutung |
|---|---|
| erstes `1000` | Frequenz des linken Kanals in Hertz |
| zweites `1000` | Frequenz des rechten Kanals in Hertz |
| `500` | Dauer in Millisekunden |

Da beide Kanäle dieselbe Frequenz erhalten, wird auf beiden Kanälen derselbe Ton ausgegeben.

Die anschließende Pause sorgt dafür, dass unser Programm auf das Ende des Tons wartet.

## 36.8. Was ist eine Frequenz?

Die Frequenz gibt an, wie oft ein Signal pro Sekunde schwingt.

```
1000 Hz
=
1000 Schwingungen pro Sekunde
```

## 36.9. Tiefe und hohe Töne

Kleine Frequenz:

```
200 Hz
```

ergibt einen tiefen Ton.

Große Frequenz:

```
2000 Hz
```

ergibt einen hohen Ton.

## 36.10. Experimentiere!

Probiere verschiedene Frequenzen aus:

```
PLAY TONE 200,200,500 : PAUSE 500

PLAY TONE 500,500,500 : PAUSE 500
```

```
PLAY PLAY TONE 1000,1000,100 : PAUSE 1000,500 : PAUSE 500

PLAY TONE 2000,2000,500 : PAUSE 500
```

Wie verändert sich der Klang?

## 36.11. Eine kleine Tonleiter

Die folgenden Frequenzen entsprechen ungefähr einer Tonleiter.

```
PLAY TONE 262,262,500 : PAUSE 500
PLAY TONE 294,294,500 : PAUSE 500
PLAY TONE 330,330,500 : PAUSE 500
PLAY TONE 349,349,500 : PAUSE 500
PLAY TONE 392,392,500 : PAUSE 500
PLAY TONE 440,440,500 : PAUSE 500
PLAY TONE 494,494,500 : PAUSE 500
PLAY TONE 523,523,500 : PAUSE 500
```

Spiele das Programm ab.

Du hörst eine aufsteigende Tonfolge.

## 36.12. Töne und LEDs kombinieren

Wir können die LED aus Kapitel 31 mit einem Ton kombinieren.

```
SETPIN GP0,DOUT

PIN(GP0)=1

PLAY PLAY TONE 1000,1000,100 : PAUSE 1000,500 : PAUSE 500

PIN(GP0)=0
```

LED und Ton werden gleichzeitig aktiviert.

## 36.13. Eine Sirene

```
DO

  FOR F=500 TO 1500 STEP 20

    PLAY TONE F,F,10 : PAUSE 10

  NEXT F
```

```
  FOR F=1500 TO 500 STEP -20
    PLAY TONE F,F,10 : PAUSE 10
  NEXT F
LOOP WHILE INKEY$=""
```

Die Tonhöhe steigt und fällt, bis eine Taste gedrückt wird.

## 36.14. Eine einfache Klingel

Viele Türklingeln verwenden zwei Töne.

```
PLAY TONE 800,800,300 : PAUSE 300
PAUSE 50
PLAY TONE 600,600,500 : PAUSE 500
```

Das ergibt:

```
Ding-Dong
```

## 36.15. Wiederholte Signaltöne

```
FOR I=1 TO 5
  PLAY TONE 1000,1000,100 : PAUSE 100
  PAUSE 100
NEXT I
```

Das Programm erzeugt fünf kurze Signaltöne.

## 36.16. Melodien programmieren

Eine Melodie besteht aus vielen einzelnen Noten.

Jede Note besitzt:

- eine Frequenz

- eine Dauer

Beispiel:

```
PLAY TONE 262,262,500 : PAUSE 500

PLAY TONE 294,294,500 : PAUSE 500

PLAY TONE 330,330,500 : PAUSE 500
```

## 36.17. Arrays für Notenfolgen

Mit Arrays lassen sich Melodien übersichtlicher speichern.

```
DIM NOTE(8)
```

Nun füllen wir das Array:

```
NOTE(1)=262
NOTE(2)=294
NOTE(3)=330
NOTE(4)=349
NOTE(5)=392
NOTE(6)=440
NOTE(7)=494
NOTE(8)=523
```

## 36.18. Die Tonleiter mit einem Array

```
DIM NOTE(8)

NOTE(1)=262
NOTE(2)=294
NOTE(3)=330
NOTE(4)=349
NOTE(5)=392
NOTE(6)=440
NOTE(7)=494
NOTE(8)=523

FOR I=1 TO 8

  PLAY TONE NOTE(I),NOTE(I),400 : PAUSE 400

NEXT I
```

Das Ergebnis entspricht der vorherigen Tonleiter.

Der Quelltext ist jedoch wesentlich übersichtlicher.

## 36.19. Melodien mit Notennamen

Eine Melodie lässt sich noch übersichtlicher schreiben, wenn die einzelnen Noten Namen erhalten.

Dazu definieren wir zunächst Konstanten für die Positionen im Noten-Array. Die Notenfolge wird anschließend mit `DATA` beschrieben.

```
CONST C1 = 1
CONST D1 = 2
CONST E1 = 3
CONST F1 = 4
CONST G1 = 5
CONST A1 = 6
CONST H1 = 7
```

Nun kann eine Melodie wesentlich leichter gelesen werden:

```
DATA C1,D1,E1,F1,G1,G1,H1,H1,G1
```

Der eigentliche Frequenzwert wird weiterhin aus dem Array `NOTE()` entnommen. Dadurch lassen sich Melodien später einfacher ändern oder sogar auf andere Tonlagen übertragen.

## 36.20. Praxisprojekt: Alle meine Entchen

Nun spielen wir die ersten Takte eines bekannten Kinderlieds.

```
DIM NOTE(7)

NOTE(1)=262
NOTE(2)=294
NOTE(3)=330
NOTE(4)=349
NOTE(5)=392
NOTE(6)=440
NOTE(7)=494

PLAY TONE NOTE(1),NOTE(1),400 : PAUSE 400
PLAY TONE NOTE(2),NOTE(2),400 : PAUSE 400
PLAY TONE NOTE(3),NOTE(3),400 : PAUSE 400
PLAY TONE NOTE(4),NOTE(4),400 : PAUSE 400

PLAY TONE NOTE(5),NOTE(5),800 : PAUSE 800
```

```
PLAY TONE NOTE(5),NOTE(5),800 : PAUSE 800
```

Die Melodie ist noch nicht vollständig, aber bereits gut erkennbar.

## 36.21. Praxisprojekt: Elektronische Türklingel

```
DO

  PRINT "Taste ENTER"

  INPUT A$

  PLAY TONE 800,800,300 : PAUSE 300

  PAUSE 50

  PLAY TONE 600,600,500 : PAUSE 500

LOOP WHILE A$=CHR$(13)
```

Jedes Drücken der ENTER-Taste löst die Klingel aus.

## 36.22. Mehr als nur Pieptöne

`PLAY TONE` ist nur eine der Audiofunktionen von MMBasic.

Je nach MMBasic-System und verwendeter Hardware stehen weitere Möglichkeiten zur Verfügung, beispielsweise die Wiedergabe von Audiodateien oder andere Formen der Klangerzeugung.

Welche Audioausgänge und Funktionen zur Verfügung stehen, hängt vom jeweiligen Gerät ab. Deshalb sollte bei Audio-Projekten immer das Handbuch der verwendeten PicoMite- oder Colour-Maximite-Variante herangezogen werden.

Für dieses Kapitel genügt `PLAY TONE`, um die Grundlagen von Frequenz, Tonhöhe, Tondauer und einfachen Melodien kennenzulernen.

## 36.23. Wo werden Töne verwendet?

Piezo-Lautsprecher findet man in vielen Projekten:

- Alarmanlagen
- Wecker
- Küchen-Timer
- Türklingeln
- Messgeräte

- Spiele

Sie sind eine einfache Möglichkeit, dem Benutzer Rückmeldungen zu geben.

## 36.24. Typische Fehler

Wenn kein Ton hörbar ist:

1. Ist der Piezo richtig angeschlossen?
2. Wird ein passiver Piezo verwendet?
3. Ist die Lautstärke ausreichend?
4. Wird die richtige Frequenz verwendet?
5. Unterstützt das verwendete System `PLAY TONE` und ist die Audioausgabe richtig konfiguriert?

## 36.25. Experimentiere!

Probiere folgende Änderungen aus:

- andere Frequenzen
- andere Dauern
- eigene Tonfolgen
- verschiedene Sirenen

## 36.26. Probier’s selbst!

Versuche folgende Aufgaben:

1. Spiele eine eigene Melodie.
2. Erzeuge einen Alarmton.
3. Programmiere einen Wecker.
4. Erstelle eine Dreiklang-Folge.
5. Ergänze die fehlenden Takte von "Alle meine Entchen".

## 36.27. Ausblick

Nun beherrschst du:

- LEDs
- Taster
- Analoge Eingänge
- PWM
- Töne

Im nächsten Kapitel werden wir daraus ein vollständiges elektronisches Spiel entwickeln.

Der `PLAY TONE`-Befehl eignet sich nicht nur für Melodien. Kurze Signaltöne geben dem Benutzer eine akustische Rückmeldung, beispielsweise beim Drücken einer Taste, beim Erreichen eines Messwertes oder wenn ein Fehler aufgetreten ist. Schon ein einzelner kurzer Ton kann die Bedienung eines Programms deutlich angenehmer machen.

## 36.28. Zusammenfassung

In diesem Kapitel hast du gelernt:

- wie Töne erzeugt werden
- was Frequenzen bedeuten
- wie Melodien programmiert werden
- wie Arrays für Notenfolgen eingesetzt werden
- wie `PLAY TONE` zur Tonerzeugung verwendet wird

Damit hast du eine weitere wichtige MMBasic-Funktion kennengelernt, die wir für unser nächstes Projekt verwenden können.

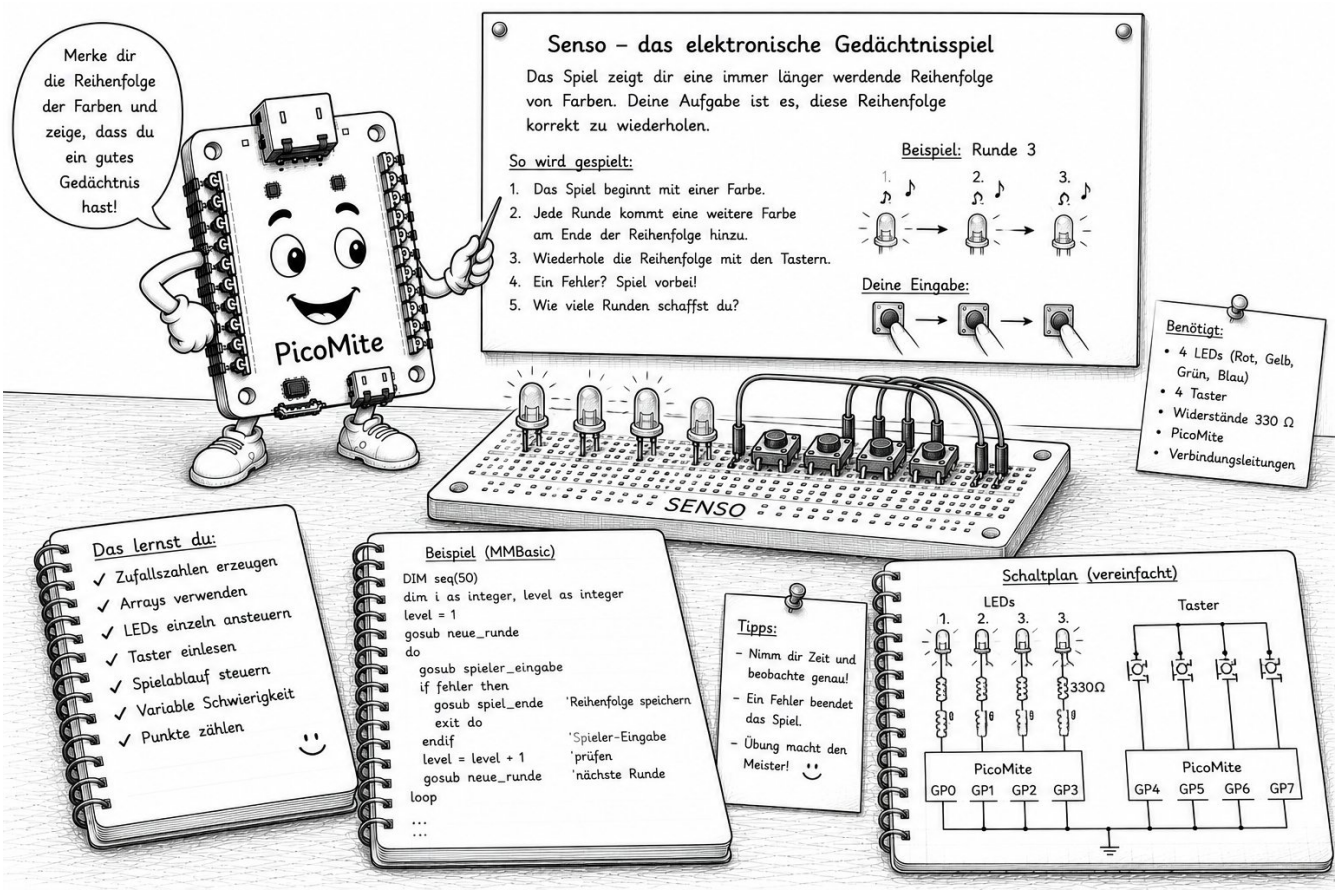
Im nächsten Kapitel entwickeln wir ein elektronisches Gedächtnisspiel nach dem Vorbild von Simon beziehungsweise Senso.

## 36.29. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 36_kleine_melodie.bas

# Kapitel 37. Senso - Das elektronische Gedächtnisspiel




## 37.1. Das lernst du

In diesem Kapitel lernst du:

- größere Programme zu strukturieren
- Arrays sinnvoll einzusetzen
- Zufallszahlen praktisch zu verwenden
- LEDs, Taster und Töne zu kombinieren
- Spielabläufe zu steuern
- Benutzereingaben über Taster und Tastatur auszuwerten
- eigene elektronische Spiele zu entwickeln

Am Ende des Kapitels hast du ein vollständiges elektronisches Gedächtnisspiel programmiert.

## 37.2. Das große Hardware-Projekt

In den letzten Kapiteln haben wir viele wichtige Techniken kennengelernt:

- Digitale Ausgänge

- LEDs
- Digitale Eingänge
- Taster
- Analoge Eingänge
- PWM
- Töne mit `PLAY TONE`

Bisher wurden diese Techniken meist einzeln betrachtet.

Nun wollen wir sie in einem größeren Hardware-Projekt kombinieren.

Dazu bauen wir ein elektronisches Gedächtnisspiel nach dem Vorbild von Simon beziehungsweise Senso.

## 37.3. Das Vorbild

Ende der 1970er Jahre erschien ein elektronisches Spiel mit vier farbigen Tasten.

Der Computer spielte eine Folge von Farben und Tönen vor.

Der Spieler musste diese Folge anschließend exakt wiederholen.

Nach jeder erfolgreichen Runde wurde die Folge um ein weiteres Element verlängert.

Mit jeder Runde wurde das Spiel schwieriger.

Das Spiel wurde unter verschiedenen Namen verkauft:

- Simon
- Senso

Bis heute zählt es zu den bekanntesten elektronischen Spielen.

## 37.4. Das Spielprinzip

Betrachten wir eine typische Spielrunde.

Runde 1:

```
Computer:
Rot

Spieler:
Rot
```

Runde 2:

```
Computer:
Rot Gelb

Spieler:
Rot Gelb
```

Runde 3:

```
Computer:
Rot Gelb Blau

Spieler:
Rot Gelb Blau
```

Nach jeder erfolgreichen Runde wird die Folge länger.

Irgendwann wird sie so lang, dass der Spieler einen Fehler macht.

Dann endet das Spiel.

## 37.5. Unsere Hardware

Wir benötigen:

- 4 LEDs
- 4 Taster
- passende Widerstände
- Steckbrett und Verbindungsdrähte
- eine für `PLAY TONE` konfigurierte Audioausgabe

Die Beispiele dieses Kapitels wurden auf einem **PicoMite HDMI/USB** überprüft.

`PLAY TONE` verwendet die von MMBasic konfigurierte Audioausgabe. Ein zusätzlicher Piezo an einem beliebigen GPIO-Pin ist für diese Beispiele nicht vorgesehen.

## 37.6. Anschlussbeispiel für den PicoMite

| **Funktion** | **GPIO** |
|---|---|
| LED Rot | GP0 |
| LED Gelb | GP1 |
| LED Grün | GP2 |
| LED Blau | GP3 |

| Funktion | GPIO |
|---|---|
| Taster Rot | GP4 |
| Taster Gelb | GP5 |
| Taster Grün | GP6 |
| Taster Blau | GP7 |

## 37.7. Die Farben

Intern verwenden wir Zahlen für die Farben.

| Wert | Farbe |
|---|---|
| 1 | Rot |
| 2 | Gelb |
| 3 | Grün |
| 4 | Blau |

Dadurch wird das Programm wesentlich einfacher.

## 37.8. Warum brauchen wir Arrays?

Der Computer muss sich die komplette Folge merken.

Beispielsweise:

```
Rot
Gelb
Blau
Gruen
Rot
Gelb
```

Eine einzelne Variable genügt dafür nicht.

Deshalb verwenden wir ein Array.

```
DIM FOLGE(100)
```

Damit können bis zu 100 Spielschritte gespeichert werden.

## 37.9. Eine neue Farbe erzeugen

Für jede neue Runde wird eine Zufallszahl erzeugt.

```
FOLGE(LEVEL)=INT(RND*4)+1
```

Mögliche Ergebnisse:

```
1
2
3
4
```

also:

```
Rot
Gelb
Gruen
Blau
```

## 37.10. Das Programm strukturieren

Unser Spiel besteht aus mehreren Aufgaben.

Deshalb teilen wir es in Unterprogramme auf.

```
Neue Farbe erzeugen

Folge anzeigen

Spielereingaben lesen

Fehler anzeigen

Spiel beenden
```

Dadurch bleibt das Programm übersichtlich.

## 37.11. Eine Farbe anzeigen

Zunächst erstellen wir ein Unterprogramm.

```
SUB ZeigeFarbe(FARBE)

END SUB
```

Dieses Unterprogramm erhält eine Zahl:

```
1
2
3
oder
4
```

und schaltet die passende LED ein.

## 37.12. Rot anzeigen

```
PIN(GP0)=1

PAUSE 500

PIN(GP0)=0
```

Das ist der erste Baustein unseres Spiels.

## 37.13. Unterschiedliche Töne

Jede Farbe erhält ihren eigenen Ton.

| **Farbe** | **Ton** |
|---|---|
| Rot | 262 Hz |
| Gelb | 330 Hz |
| Grün | 392 Hz |
| Blau | 523 Hz |

Dadurch erkennt der Spieler die Farben nicht nur optisch, sondern auch akustisch.

## 37.14. Das Unterprogramm ZeigeFarbe

Das Unterprogramm schaltet die passende LED ein und erzeugt gleichzeitig den zugehörigen Ton mit `PLAY TONE`.

```
SUB ZeigeFarbe(FARBE)
  SELECT CASE FARBE
  CASE 1
    PIN(GP0)=1
    PLAY TONE 262,262,250
    PAUSE 250
    PIN(GP0)=0
  CASE 2
    PIN(GP1)=1
```

```
    PLAY TONE 330,330,250
    PAUSE 250
    PIN(GP1)=0
  CASE 3
    PIN(GP2)=1
    PLAY TONE 392,392,250
    PAUSE 250
    PIN(GP2)=0
  CASE 4
    PIN(GP3)=1
    PLAY TONE 523,523,250
    PAUSE 250
    PIN(GP3)=0
  END SELECT

  PAUSE 300
END SUB
```

`PLAY TONE` arbeitet im Hintergrund. Die Pause von 250 ms sorgt deshalb dafür, dass die LED ungefähr so lange eingeschaltet bleibt wie der Ton ausgegeben wird. Die anschließende Pause von 300 ms trennt die einzelnen Farben der Folge voneinander.

## 37.15. Die komplette Folge anzeigen

Nun können wir eine gesamte Spielrunde ausgeben.

```
FOR I=1 TO LEVEL

  ZeigeFarbe FOLGE(I)

NEXT I
```

Damit wird die komplette Sequenz angezeigt.

## 37.16. Die Eingabe des Spielers

Der Spieler muss die angezeigte Folge exakt wiederholen.

Dazu verwenden wir die Funktion:

```
TASTE=HoleTaste()
```

Sie liefert eine Zahl von 1 bis 4 zurück.

Die Eingabe kann über die vier angeschlossenen Taster erfolgen. Zusätzlich können auf dem PicoMite HDMI/USB die Tasten 1 bis 4 auf der Tastatur verwendet werden. Das erleichtert das Testen des Programms, weil das Spiel auch ohne angeschlossene Taster ausprobiert werden kann.

# 37.17. Die Funktion HoleTaste

Die Funktion wartet so lange, bis ein Taster oder die entsprechende Zifferntaste gedrückt wird. Nach einer erkannten Eingabe beendet `EXIT FUNCTION` die Funktion sofort.

```
FUNCTION HoleTaste()
  DO
    t$=INKEY$

    IF PIN(GP4)=1 OR t$="1" THEN
      PRINT ".";
      DO : LOOP WHILE PIN(GP4)=1 OR KEYDOWN(0)>0
      HoleTaste=1
      ZeigeFarbe 1
      EXIT FUNCTION
    ENDIF

    IF PIN(GP5)=1 OR t$="2" THEN
      PRINT ".";
      DO : LOOP WHILE PIN(GP5)=1 OR KEYDOWN(0)>0
      HoleTaste=2
      ZeigeFarbe 2
      EXIT FUNCTION
    ENDIF

    IF PIN(GP6)=1 OR t$="3" THEN
      PRINT ".";
      DO : LOOP WHILE PIN(GP6)=1 OR KEYDOWN(0)>0
      HoleTaste=3
      ZeigeFarbe 3
      EXIT FUNCTION
    ENDIF

    IF PIN(GP7)=1 OR t$="4" THEN
      PRINT ".";
      DO : LOOP WHILE PIN(GP7)=1 OR KEYDOWN(0)>0
      HoleTaste=4
      ZeigeFarbe 4
      EXIT FUNCTION
    ENDIF

  LOOP
END FUNCTION
```

Nach einem erkannten Tastendruck wartet das Programm, bis der Hardware-Taster wieder losgelassen beziehungsweise keine Taste auf der Tastatur mehr gedrückt wird. Dadurch wird ein Tastendruck nicht mehrfach gewertet.

`KEYDOWN(0)` liefert dabei die Anzahl der momentan gedrückten Tasten. Solange dieser Wert größer

als 0 ist, wartet die Schleife weiter.

Nach der Eingabe ruft `HoleTaste()` außerdem `ZeigeFarbe` auf. Dadurch erhält der Spieler sofort eine optische und akustische Rückmeldung.

## 37.18. Die Eingabe prüfen

```
FOR I=1 TO LEVEL

  TASTE=HoleTaste()

  IF TASTE<>FOLGE(I) THEN

    FEHLER=1

    EXIT FOR

  ENDIF

NEXT I
```

Hier findet die eigentliche Spielprüfung statt.

## 37.19. Was passiert bei einem Fehler?

Wenn der Spieler einen Fehler macht, soll dies deutlich sichtbar und hörbar werden.

Dazu werden alle vier LEDs gemeinsam eingeschaltet und zwei unterschiedliche Töne ausgegeben.

## 37.20. Fehleranimation

```
SUB Fehleranimation(ANZAHL)
  PRINT
  PRINT "Falsche Taste!"
  PRINT

  FOR I=1 TO ANZAHL
    PIN(GP0)=1
    PIN(GP1)=1
    PIN(GP2)=1
    PIN(GP3)=1

    PLAY TONE 300,300,250
    PAUSE 200

    PIN(GP0)=0
    PIN(GP1)=0
```

```
    PIN(GP2)=0
    PIN(GP3)=0

    PLAY TONE 500,500,250
    PAUSE 200
  NEXT I
END SUB
```

Im Hauptprogramm wird die Animation dreimal ausgeführt:

```
Fehleranimation 3
```

Der Spieler erkennt dadurch sofort, dass die eingegebene Folge nicht korrekt war.

## 37.21. Der Spielablauf

Unser Spiel arbeitet nun nach folgendem Schema:

```
Neue Farbe erzeugen

Folge anzeigen

Spielereingaben lesen

Richtig?
 |
 +-- Ja --> Naechste Runde
 |
 +-- Nein --> Spielende
```

## 37.22. Das vollständige Programm

Nun setzen wir die einzelnen Bausteine zusammen.

Zu Beginn werden die vier LEDs als Ausgänge und die vier Taster als Eingänge konfiguriert:

```
SETPIN GP0,DOUT
SETPIN GP1,DOUT
SETPIN GP2,DOUT
SETPIN GP3,DOUT

SETPIN GP4,DIN
SETPIN GP5,DIN
SETPIN GP6,DIN
SETPIN GP7,DIN
```

```
DIM FOLGE(100)
```

Der eigentliche Spielablauf sieht vereinfacht so aus:

```
LEVEL=1

DO
  FOLGE(LEVEL)=INT(RND*4)+1

  FOR I=1 TO LEVEL
    ZeigeFarbe FOLGE(I)
  NEXT I

  FEHLER=0

  FOR I=1 TO LEVEL
    TASTE=HoleTaste()

    IF TASTE<>FOLGE(I) THEN
      FEHLER=1
      EXIT FOR
    ENDIF
  NEXT I

  IF FEHLER=1 THEN
    Fehleranimation 3
    EXIT DO
  ENDIF

  LEVEL=LEVEL+1
  PAUSE 500
LOOP
```

Das vollständige Programm enthält zusätzlich eine kurze Einführung der vier Tasten zu Beginn sowie die Möglichkeit, nach dem Spiel eine neue Runde zu starten.

## 37.23. Spielende

Nach einem Fehler wird die erreichte Runde ausgegeben.

```
PRINT

PRINT "Spiel beendet"

PRINT "Erreichte Runde:"

PRINT LEVEL
```

## 37.24. Erweiterung: Highscore

Eine interessante Erweiterung wäre ein Highscore.

```
IF LEVEL>BESTE_RUNDE THEN

  BESTE_RUNDE=LEVEL

ENDIF
```

Wird eine höhere Runde erreicht, kann das Programm dies als neue Bestleistung anzeigen. Die Variable `BESTE_RUNDE` bleibt während der Programmlaufzeit erhalten.

## 37.25. Erweiterung: Höhere Geschwindigkeit

Nach jeder Runde könnte die Anzeigegeschwindigkeit erhöht werden.

Das macht das Spiel deutlich schwieriger.

## 37.26. Erweiterung: Mehr Farben

Wer möchte, kann weitere LEDs und Taster hinzufügen.

Dadurch entstehen noch längere und anspruchsvollere Folgen.

## 37.27. Erweiterung: Grafikdarstellung

Besitzt dein System einen Bildschirm, könnten die Farben zusätzlich grafisch dargestellt werden.

Damit entsteht eine moderne Version des Spiels.

## 37.28. Warum dieses Projekt wichtig ist

In diesem Projekt kommen fast alle bisher erlernten Techniken zusammen.

Verwendet wurden:

- Variablen
- Schleifen
- Arrays
- Unterprogramme
- Zufallszahlen
- LEDs
- Taster

- Töne mit `PLAY TONE`

Genau deshalb eignet sich Senso hervorragend als Abschlussprojekt des Hardware-Teils.

## 37.29. Experimentiere!

Probiere folgende Erweiterungen aus:

- höhere Spielgeschwindigkeit
- Highscore
- zusätzliche Farben
- andere Tonhöhen
- grafische Darstellung

## 37.30. Probier’s selbst!

Versuche folgende Aufgaben:

1. Ergänze eine Startmelodie.
2. Speichere den Highscore in einer Datei.
3. Zeige die aktuelle Runde auf dem Bildschirm an.
4. Erhöhe die Schwierigkeit automatisch.
5. Entwickle eine eigene Variante des Spiels.

## 37.31. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Arrays praktisch einzusetzen
- größere Programme zu strukturieren
- LEDs, Taster und Töne zu kombinieren
- Spielabläufe zu programmieren
- Benutzereingaben auszuwerten

Du hast damit dein erstes vollständiges elektronisches Spiel entwickelt.

Herzlichen Glückwunsch!

Viele professionelle Softwareprojekte bestehen aus genau den Bausteinen, die du in diesem Kapitel verwendet hast.

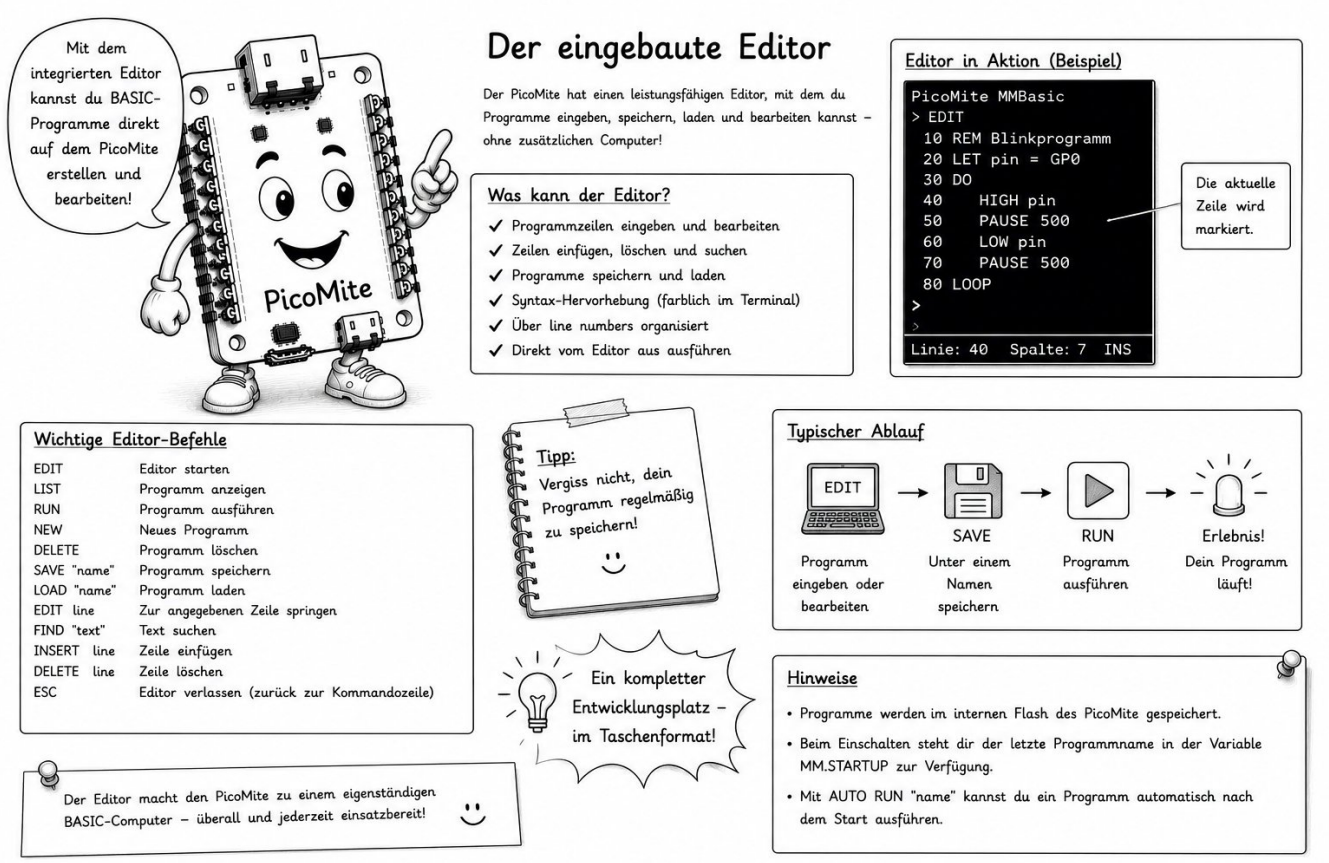
## 37.32. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 37_senso.bas

# Teil V – Werkzeuge und Entwicklung

# Kapitel 38. Der eingebaute Editor




## 38.1. Das lernst du

In diesem Kapitel lernst du:

- den eingebauten Vollbild-Editor von MMBasic kennen
- den Unterschied zwischen `EDIT` und `EDIT "DATEI.BAS"` verstehen
- Programme direkt auf dem PicoMite zu erstellen, zu speichern und auszuführen
- Text zu suchen, zu markieren, zu kopieren und zu ersetzen
- Dateien in den Editor zu importieren und markierten Text zu exportieren
- Programme mit **Beautify** automatisch zu formatieren
- die integrierte Hilfe zu verwenden

Am Ende des Kapitels kannst du den eingebauten Editor sicher für die Programmentwicklung direkt auf deinem PicoMite einsetzen.

Die Editorfunktionen in diesem Kapitel wurden auf einem **PicoMiteHDMIUSB mit MMBasic V6.03.01** praktisch überprüft. Bei älteren oder neueren Versionen können einzelne Funktionen oder Tastenbelegungen abweichen.

## 38.2. Programmieren direkt auf dem PicoMite

Beim PicoMite stehen BASIC-Interpreter und Vollbild-Editor unmittelbar auf dem Zielsystem zur Verfügung. Für kleine Programme, Experimente oder Änderungen wird deshalb kein zusätzlicher PC benötigt.

Du brauchst lediglich:

- einen PicoMite
- eine Tastatur
- einen Bildschirm

und kannst sofort mit dem Programmieren beginnen.

## 38.3. Den Editor starten

Mit `EDIT`:

```
EDIT
```

wird das aktuell im Programmspeicher des PicoMite befindliche BASIC-Programm im Vollbild-Editor geöffnet.

Ist noch kein Programm vorhanden, erscheint ein leeres Editorfenster.

Beispiel:

```
PRINT "Hallo Welt"

FOR I=1 TO 10
  PRINT I
NEXT I
```

## 38.4. Die Statuszeile des Editors

Am unteren Bildschirmrand zeigt der Editor die wichtigsten Funktionen an. Bei der getesteten Version V6.03.01 erscheint:

```
F1:Save F2:Run F3:Find F4:Mrk F5:Pst F9:In F10:Out ^A:Btfy
```

`^A` bedeutet `Ctrl+A`. Nicht alle verfügbaren Funktionstasten werden in dieser kurzen Statuszeile angezeigt.

## 38.5. Die Funktionstasten

| Taste | Funktion | Bedeutung |
|---|---|---|
| F1 | Save | Änderungen speichern |
| F2 | Run | Beim normalen `EDIT`: speichern und Programm ausführen |
| F3 | Find | Text suchen |
| F4 | Mark | Markiermodus einschalten |
| F5 | Paste | Inhalt der Zwischenablage an der Cursorposition einfügen |
| F6 | Find next | Nächste Fundstelle suchen |
| Shift+F3 | Find next | Wie F6: nächste Fundstelle suchen |
| F7 | Replace | Die zuvor gefundene Fundstelle ersetzen |
| F8 | Replace again | Weitere Fundstelle durch denselben Ersatztext ersetzen |
| F9 | Import | Inhalt einer Datei an der Cursorposition einfügen |
| F10 | Export | Inhalt der Zwischenablage in eine Datei schreiben |
| F12 | Beautify | Programm automatisch formatieren; alternativ `Ctrl+A` |

`F11` ist bei der getesteten Version nicht belegt.

## 38.6. Zwei unterschiedliche Arten zu editieren

Beim PicoMite ist es wichtig, zwischen:

```
EDIT
```

und:

```
EDIT "HALLO.BAS"
```

zu unterscheiden.

Die beiden Befehle sehen ähnlich aus, arbeiten aber nicht mit demselben Programmspeicher.

### 38.6.1. EDIT ohne Dateinamen

`EDIT` bearbeitet das aktuelle BASIC-Programm im Programmspeicher des PicoMite.

Mit `F1` werden die Änderungen dort gespeichert. Das Programm bleibt auch nach einem Reset erhalten.

Mit F2 wird das Programm gespeichert und anschließend sofort ausgeführt.

Damit entsteht ein sehr schneller Entwicklungszyklus:

```
Programm ändern
       
F2 drücken
       
Programm testen
       
EDIT
       
Programm weiter ändern
```

### 38.6.2. EDIT mit Dateinamen

Mit :

```
EDIT "HALLO.BAS"
```

wird dagegen direkt eine Datei auf dem aktuellen Laufwerk bearbeitet.

Existiert die Datei bereits, wird ihr Inhalt in den Editor geladen. Existiert sie noch nicht, beginnt der Editor mit einem leeren Fenster und die Datei wird beim Speichern angelegt.

EDIT "HALLO.BAS" lädt die Datei zum Bearbeiten in den Editor, ersetzt aber nicht das aktuelle BASIC-Programm im Programmspeicher.

Nach dem Speichern beziehen sich LIST und ein RUN ohne Dateinamen deshalb weiterhin auf das zuvor im Programmspeicher befindliche Programm.

Bei der Bearbeitung einer Datei speichern F1 und F2 die Datei. F2 führt in diesem Fall nicht wie beim normalen EDIT das gerade bearbeitete Dateiprogramm aus.

Soll die Datei anschließend ausgeführt werden, kann sie beispielsweise mit:

```
RUN "HALLO.BAS"
```

gestartet werden.

Danach befindet sich dieses Programm auch im aktuellen Programmkontext und ein EDIT ohne Dateinamen zeigt dieses Programm an.

Merke dir:

- EDIT bearbeitet das aktuelle Programm im Programmspeicher.

- `EDIT "DATEI.BAS"` bearbeitet eine Datei auf dem Laufwerk.
- `RUN "DATEI.BAS"` startet das Programm aus dieser Datei.

## 38.7. Auch andere Textdateien bearbeiten

Der Editor ist nicht auf BASIC-Dateien beschränkt. Auch andere Textdateien können direkt bearbeitet werden, zum Beispiel:

```
EDIT "INDEX.HTML"
EDIT "SPRITES.TXT"
EDIT "CONFIG.INI"
```

## 38.8. Cursor bewegen

| Taste | Funktion |
|---|---|
| Pfeiltasten | Cursor bewegen |
| HOME | Zum Zeilenanfang |
| END | Zum Zeilenende |
| PAGE UP | Eine Seite nach oben |
| PAGE DOWN | Eine Seite nach unten |
| INS | Einfüge- oder Überschreibmodus umschalten |
| DEL | Zeichen an der Cursorposition löschen |
| BACKSPACE | Zeichen links vom Cursor löschen |

Durch zweimaliges Drücken von `HOME` kann schnell zum Programmanfang gesprungen werden:

```
HOME HOME
```

Entsprechend führt:

```
END END
```

zum Programmende.

## 38.9. Text suchen

Mit `F3` wird die Suchfunktion gestartet. In der unteren Zeile erscheint:

```
Find (Use SHIFT-F3 to repeat):
```

Nach Eingabe des Suchbegriffs wird der Cursor auf die erste Fundstelle gesetzt.

Mit `F6` oder `Shift+F3` springst du anschließend zur nächsten Fundstelle. Nach der letzten Fundstelle beginnt die Suche wieder bei der ersten.

## 38.10. Suchen und Ersetzen

Suchen und Ersetzen baut auf der normalen Suche auf.

Zuerst suchst du mit `F3` nach dem gewünschten Text. Steht der Cursor auf einer Fundstelle, kannst du mit `F7` den Ersatzvorgang starten:

```
Replace:
```

Nach Eingabe des Ersatztextes wird die aktuelle Fundstelle ersetzt.

Mit `F8` kann anschließend eine weitere Fundstelle durch denselben Ersatztext ersetzt werden.

`F7` und `F8` setzen eine vorherige erfolgreiche Suche voraus. Gibt es keine aktuelle Fundstelle, meldet der Editor:

```
NOTHING TO REPLACE
```

## 38.11. Text markieren, kopieren und ausschneiden

Mit `F4` wird der Markiermodus aktiviert. Die Statuszeile zeigt dann:

```
MARK MODE ESC=Exit DEL:Delete F4:Cut F5:Copy F10:Export
```

Ausgehend von der aktuellen Cursorposition kann der gewünschte Bereich unter anderem mit den Pfeiltasten sowie `HOME`, `END`, `PAGE UP` und `PAGE DOWN` markiert werden.

Im Markiermodus stehen folgende Funktionen zur Verfügung:

| **Taste** | **Funktion** |
|---|---|
| ESC | Markiermodus verlassen |
| DEL | Markierten Text löschen |
| F4 | Markierten Text ausschneiden |
| F5 | Markierten Text in die Zwischenablage kopieren |
| F10 | Markierten Text bzw. den Inhalt der Zwischenablage in eine Datei exportieren |

## 38.12. Text einfügen

Mit `F5` wird der Inhalt der Zwischenablage an der aktuellen Cursorposition eingefügt.

Ist die Zwischenablage leer, erscheint:

```
CLIPBOARD IS EMPTY
```

## 38.13. Datei importieren

Mit `F9` erscheint die Eingabeaufforderung:

```
Import File:
```

Der Dateiname wird ohne Anführungszeichen eingegeben, beispielsweise:

```
TEST.BAS
```

Der Inhalt der Datei wird ab der aktuellen Cursorposition eingefügt. Anschließend meldet der Editor die Anzahl der importierten Zeichen, zum Beispiel:

```
IMPORTED 410 CHARS
```

## 38.14. Zwischenablage in eine Datei exportieren

Mit `F10` kann der Inhalt der Zwischenablage in einer Datei gespeichert werden.

Der Editor fragt nach dem Dateinamen:

```
Export to file:
```

Ist die Zwischenablage leer, erscheint:

```
CLIPBOARD IS EMPTY
```

## 38.15. Programme automatisch formatieren

MMBasic V6.03 besitzt im Editor die Funktion **Beautify**. Sie wird mit `F12` oder `Ctrl+A` aufgerufen. In der Statuszeile wird `Ctrl+A` als `^A:Btfy` angezeigt.

Beautify formatiert das gesamte Programm automatisch. Überflüssige führende Leerzeichen

werden entfernt und verschachtelte Programmblöcke einheitlich eingerückt.

Aus:

```
For i=1 To 3
Print "Durchlauf";i
If i=2 Then
Print "Zwei"
Else
Print "Nicht Zwei"
EndIf
Next i
```

wird:

```
For i=1 To 3
  Print "Durchlauf";i
  If i=2 Then
    Print "Zwei"
  Else
    Print "Nicht Zwei"
  EndIf
Next i
```

Ein erneuter Aufruf von Beautify führt zum gleichen Ergebnis. Auch zuvor manuell eingefügte zusätzliche Leerzeichen am Zeilenanfang werden auf die einheitliche Einrückung zurückgesetzt.

Beautify ist besonders bei verschachtelten `IF`-Anweisungen und Schleifen hilfreich. Eine saubere Einrückung macht die Programmstruktur wesentlich leichter erkennbar.

## 38.16. Hilfe zu Befehlen und Funktionen

Der PicoMite besitzt mit `HELP` eine praktische Hilfe zu MMBasic-Befehlen und -Funktionen.

Damit `HELP` verwendet werden kann, muss sich eine entsprechend formatierte Datei `help.txt` im Root-Verzeichnis von Laufwerk `A:` befinden. Die Hilfetexte sind nicht fest in der PicoMite-Firmware gespeichert.

Die allgemeine Syntax lautet:

```
HELP Suchtext
```

Bei der für dieses Tutorial getesteten Hilfedatei ist es sinnvoll, am Ende des Suchbegriffs ein * anzugeben:

```
HELP PRINT*
```

oder:

```
HELP PLAY*
```

* steht für beliebig viele Zeichen. ? steht für genau ein beliebiges Zeichen.

| **Platzhalter** | **Bedeutung** |
|---|---|
| * | Beliebig viele Zeichen oder auch kein Zeichen |
| ? | Genau ein beliebiges Zeichen |

Welche Suchbegriffe ohne * einen Treffer liefern, hängt vom Aufbau der verwendeten `help.txt` ab.

# 38.17. Farbsyntax

Der Editor hebt verschiedene Elemente des BASIC-Programms farbig hervor, zum Beispiel:

- BASIC-Schlüsselwörter
- Zahlen
- Kommentare
- Zeichenketten

Dadurch wird der Quelltext leichter lesbar.

Die Farbsyntax kann mit `OPTION COLOURCODE` ausgeschaltet werden:

```
OPTION COLOURCODE OFF
```

und wieder eingeschaltet werden:

```
OPTION COLOURCODE ON
```

# 38.18. Arbeiten mit der Maus

Der integrierte Editor unterstützt auf geeigneten PicoMite-Systemen auch eine USB-Maus. Sie kann insbesondere bei größeren Programmen die Navigation und das Markieren von Text erleichtern.

Aktiviert wird sie mit:

```
OPTION MOUSE ON
```

Deaktiviert wird sie mit:

```
OPTION MOUSE OFF
```

Die Bedienung mit der Tastatur steht unabhängig davon weiterhin zur Verfügung.

## 38.19. Automatische Sicherung

Beim Bearbeiten von Dateien kann MMBasic Sicherungskopien anlegen. Da das genaue Verhalten von der verwendeten MMBasic-Version und der Art des Editierens abhängen kann, solltest du dich bei wichtigen Programmen nicht allein darauf verlassen.

Vor größeren Änderungen empfiehlt es sich, eine zusätzliche Kopie des Programms unter einem anderen Dateinamen anzulegen.

## 38.20. Wann sollte man den eingebauten Editor verwenden?

Der Editor eignet sich besonders für:

- kleine und mittlere Programme
- Experimente
- Fehlersuche
- Unterricht und Lernen
- Änderungen direkt am PicoMite

Auf einem PicoMite mit HDMI oder VGA kann damit vollständig ohne PC programmiert werden.

## 38.21. Grenzen des eingebauten Editors

Bei größeren Projekten bieten externe Werkzeuge zusätzlichen Komfort, zum Beispiel:

- mehrere Dateien gleichzeitig bearbeiten
- Projektverwaltung
- Versionsverwaltung
- Git-Integration

Hier können Programme wie MMEdit oder Visual Studio Code sinnvoll sein.

## 38.22. Programme automatisch starten

Viele PicoMite-Projekte sollen nach dem Einschalten sofort einsatzbereit sein, zum Beispiel eine Uhr, ein Spiel oder ein Messgerät.

Mit `OPTION AUTORUN`:

```
OPTION AUTORUN ON
```

wird das aktuell gespeicherte Programm nach dem Einschalten oder Reset automatisch gestartet.

Während der Entwicklung ist es meist praktischer, diese Funktion zunächst ausgeschaltet zu lassen und das Programm manuell mit `RUN` zu starten.

Deaktivieren kannst du den automatischen Start mit:

```
OPTION AUTORUN OFF
```

## 38.23. Probier's selbst!

1. Starte den Editor mit `EDIT` und gib ein kleines BASIC-Programm ein.
2. Speichere es mit `F1`.
3. Ändere eine Zeile und starte das Programm mit `F2`.
4. Suche mit `F3` nach einem Wort und springe mit `F6` zu weiteren Fundstellen.
5. Ersetze eine Fundstelle mit `F7` und wiederhole das Ersetzen mit `F8`.
6. Markiere mit `F4` einen Textbereich und kopiere ihn im Markiermodus mit `F5`.
7. Füge den kopierten Text an einer anderen Stelle mit `F5` ein.
8. Importiere mit `F9` testweise eine Textdatei.
9. Formatiere ein absichtlich schlecht eingerücktes Programm mit `F12`.
10. Bearbeite mit `EDIT "TEST.BAS"` eine Datei und beobachte anschließend den Unterschied zwischen `LIST`, `RUN` und `RUN "TEST.BAS"`.
11. Rufe mit `HELP PRINT*` die Hilfe zum `PRINT`-Befehl auf.

## 38.24. Zusammenfassung

Der PicoMite besitzt einen leistungsfähigen Vollbild-Editor, mit dem BASIC-Programme direkt auf dem Zielsystem entwickelt werden können.

Besonders wichtig ist die Unterscheidung zwischen `EDIT` und `EDIT "DATEI.BAS"`: Der erste Befehl bearbeitet das aktuelle Programm im Programmspeicher, der zweite direkt eine Datei auf dem Laufwerk.

Neben den grundlegenden Funktionen zum Schreiben und Speichern bietet der Editor Suche und Ersetzen, eine Zwischenablage, Dateiimport und -export sowie mit MMBasic V6.03 die automatische Formatierung **Beautify**.

Im nächsten Kapitel lernst du mit **MMEdit** eine komfortable Entwicklungsumgebung für den PC

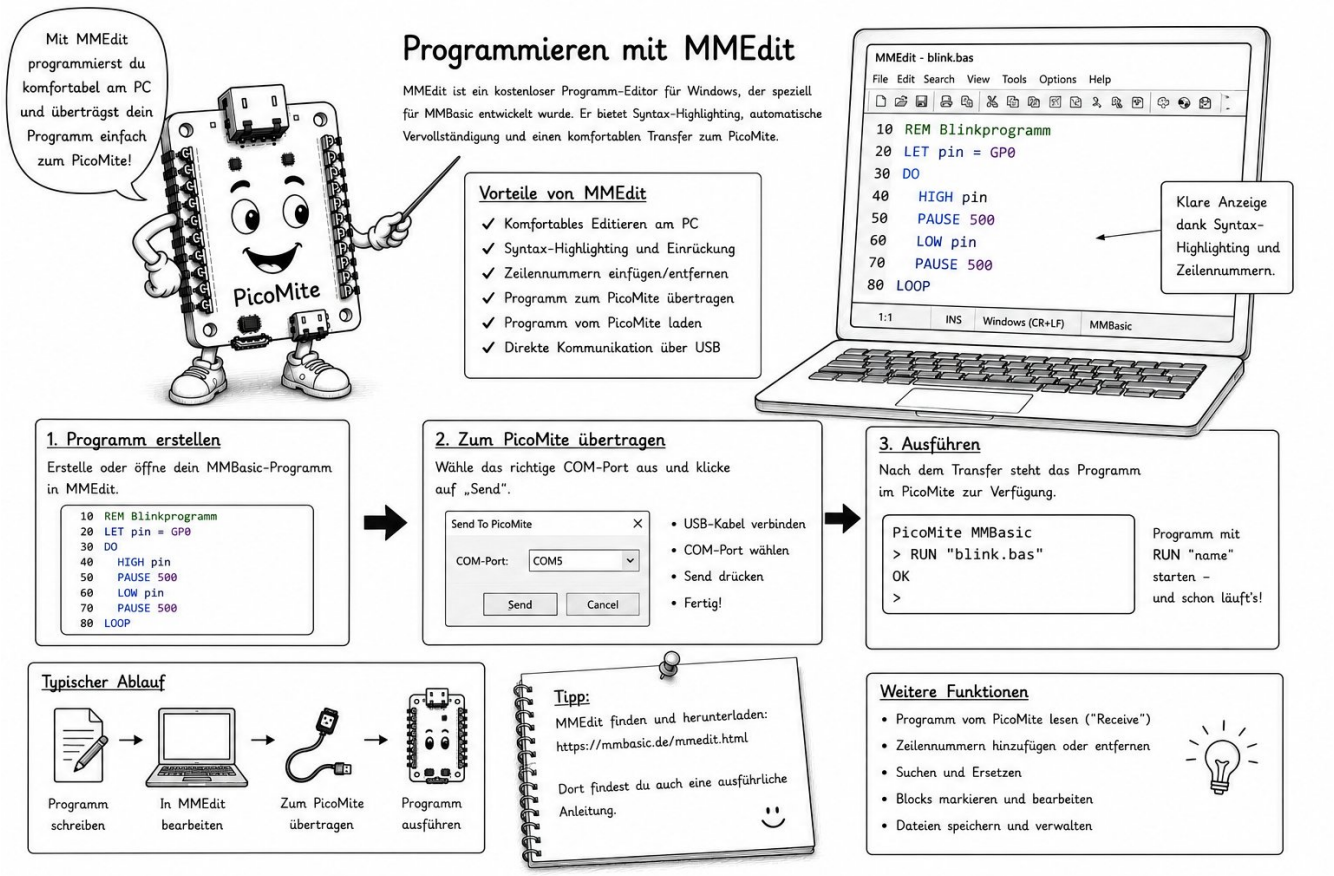
kennen, die den eingebauten Editor sinnvoll ergänzt.

## 38.25. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 38_editor_test.bas

# Kapitel 39. Programmieren mit MMEdit




## 39.1. Das lernst du

Im vorherigen Kapitel hast du den eingebauten Editor des PicoMite kennengelernt. Für kleine Programme und schnelle Änderungen direkt am Zielsystem ist er sehr praktisch.

Wenn Programme größer werden oder du den Quelltext lieber auf dem PC bearbeiten und verwalten möchtest, bietet sich **MMEdit** an.

In diesem Kapitel lernst du:

- was MMEdit und MMCC sind
- welche MMEdit-Version in diesem Tutorial verwendet wird
- wie die Verbindung zu einem PicoMite oder Colour Maximite hergestellt wird
- wie Programme auf dem PC erstellt, übertragen und getestet werden
- wie MMEdit und der eingebaute Editor gemeinsam verwendet werden können
- wie Programme nach einem Fehler direkt auf dem PicoMite korrigiert und erneut getestet werden können
- wie mehrere Dateien gleichzeitig bearbeitet werden können
- wie MMEdit bei größeren MMBasic-Projekten hilft

Am Ende des Kapitels kennst du einen typischen Entwicklungsablauf mit MMEdit und kannst die

Vorteile von MMEdit und dem eingebauten Editor miteinander kombinieren.

## 39.2. Was ist MMEdit?

MMEdit ist ein speziell für MMBasic entwickelter Programmeditor. Im Gegensatz zu einem allgemeinen Texteditor kennt MMEdit die Besonderheiten von MMBasic und unterstützt typische Arbeitsabläufe bei der Entwicklung.

Dazu gehören unter anderem:

- Syntax-Hervorhebung für MMBasic
- mehrere gleichzeitig geöffnete Dateien in Registerkarten
- Suchen und Ersetzen
- Übertragen und Starten von Programmen auf einem angeschlossenen MMBasic-System
- Zugriff auf Dateien des Zielsystems

MMEdit unterstützt verschiedene Mitglieder der MMBasic-Familie, darunter PicoMite, PicoMite VGA/HDMI und Colour Maximite 2.

## 39.3. MMEdit und MMCC

Seit MMEdit V5 ist die Kommunikation mit dem Zielsystem in ein eigenes Programm ausgelagert: das **Maximite Control Centre**, kurz **MMCC**.

Die Aufgaben sind damit klar getrennt:

```
MMEdit
  |
  +-- Programm schreiben und bearbeiten
  |
  +-- Load and Run
          |
          v
        MMCC
          |
          +-- serielle Verbindung
          +-- Terminal
          +-- Dateiübertragung
          |
          v
       PicoMite
```

MMEdit ist also in erster Linie der Editor. MMCC stellt die Verbindung zum PicoMite her und dient gleichzeitig als Terminal.

Im normalen Entwicklungsalltag arbeiten beide Programme eng zusammen. MMEdit kann MMCC bei Bedarf automatisch starten.

## 39.4. Welche Version wird verwendet?

Ursprünglich wurde dieses Tutorial mit folgender Version erstellt:

```
MMEdit V5.2.9
```

Inzwischen verwende ich:

```
MMEdit V5.4.3
```

Auch MMEdit V5.4.3 enthält eine portable Version. Diese verwende ich für die Beispiele und Tests in diesem Kapitel.

Die grundlegende Bedienung des Editors hat sich gegenüber V5.2.9 nicht wesentlich geändert. Die in diesem Kapitel beschriebenen Arbeitsschritte lassen sich daher auch mit älteren Versionen nachvollziehen.

MMEdit wird weiterhin entwickelt.

Ab MMEdit V5.3.7 wurde insbesondere die Übertragung von Programmen zum PicoMite deutlich beschleunigt. Bei kleinen Programmen fällt der Unterschied möglicherweise kaum auf, bei größeren Programmen kann die Zeitersparnis jedoch erheblich sein.

Wenn du MMEdit neu einrichtest, empfiehlt es sich daher, eine aktuelle Version zu verwenden.

## 39.5. Die portable Version

MMEdit kann auch ohne klassische Installation als portable Version verwendet werden. Eine solche portable Version ist auch in MMEdit V5.4.3 enthalten und wird in diesem Tutorial verwendet.

Die portable Version kann in ein beliebiges beschreibbares Verzeichnis kopiert und von dort gestartet werden. Dadurch lässt sie sich beispielsweise zusammen mit den übrigen Entwicklungswerkzeugen auf einem USB-Stick oder einer SD-Karte aufbewahren.

MMEdit und MMCC befinden sich dabei gemeinsam im Programmverzeichnis.

Die portable Version ist lediglich eine Möglichkeit, MMEdit zu verwenden. Sie ist keine Voraussetzung für den Betrieb eines PicoMite oder Colour Maximite und gehört nicht grundsätzlich zu MMBasic.

## 39.6. Der erste Start

Nach dem Start von MMEdit erscheint das Editorfenster. Hier kannst du BASIC-Programme öffnen, bearbeiten und speichern.

Eine besondere Rolle spielt die Symbolleiste. Für die tägliche Arbeit ist vor allem die Funktion **Load and Run** wichtig. Sie überträgt das aktuelle Programm über MMCC zum Zielsystem und startet es dort.

MMCC kann auch separat gestartet werden. Bei einer portablen Version befindet sich das Programm `MMCC.exe` im MMEdit-Verzeichnis.

## 39.7. Verbindung zum Zielsystem

Damit MMCC mit einem PicoMite oder Colour Maximite kommunizieren kann, wird eine serielle Verbindung benötigt.

Beim PicoMite erfolgt diese normalerweise über USB beziehungsweise über die vom jeweiligen System bereitgestellte serielle USB-Verbindung.

```
PC
 |
 +---- USB/seriell ---- PicoMite
```

Unter Windows erscheint diese Verbindung als COM-Port, beispielsweise:

```
COM3
COM5
COM8
```

Welche Nummer Windows vergibt, hängt vom Computer und vom angeschlossenen Gerät ab.

## 39.8. MMCC einrichten

Starte MMCC und wähle dort den passenden COM-Port und den richtigen Gerätetyp aus. Anschließend wird die Verbindung mit **Connect** hergestellt.

Ist die Verbindung erfolgreich, siehst du im Terminal die Ausgabe von MMBasic und schließlich die Eingabeaufforderung:

```
>
```

Nun kannst du Befehle direkt im Terminal eingeben.

Zum Testen eignet sich beispielsweise:

```
PRINT MM.INFO(VERSION)
```

Auf meinem PicoMite HDMI/USB mit MMBasic V6.03.01 erscheint beispielsweise:

```
6.0301
```

Damit weißt du, dass die Kommunikation funktioniert.

## 39.9. MMCC als Terminal

MMCC ist nicht nur für die Programmübertragung zuständig. Es kann auch als normales Terminal für MMBasic verwendet werden.

Hier kannst du:

- Befehle direkt eingeben
- Programme starten und abbrechen
- Ausgaben des Programms beobachten
- Fehlermeldungen lesen
- die Verbindung zum Zielsystem kontrollieren

Damit übernimmt MMCC eine ähnliche Aufgabe wie andere Terminalprogramme, ist aber direkt auf die Zusammenarbeit mit MMEdit abgestimmt.

## 39.10. Der typische Entwicklungszyklus

Der große Vorteil von MMEdit zeigt sich beim wiederholten Ändern und Testen eines Programms.

```
Programm in MMEdit schreiben
          |
          v
        Speichern
          |
          v
      Load and Run
          |
          v
MMCC überträgt das Programm
          |
          v
PicoMite startet das Programm
          |
          v
        Testen
          |
          v
       Ändern ...
```

Während der Entwicklung wird dieser Zyklus oft viele Male wiederholt.

## 39.11. Ein erstes Programm

Erstelle in MMEdit eine neue Datei und speichere sie beispielsweise als:

```
39_mmedit_hallo.bas
```

Der Inhalt ist bewusst einfach gehalten. Mit `OPTION EXPLICIT` verlangt MMBasic dabei die ausdrückliche Deklaration der verwendeten Variablen:

```
OPTION EXPLICIT

DIM INTEGER Zaehler

FOR Zaehler = 1 TO 10
  PRINT "Hallo MMBasic! Durchlauf"; Zaehler
  PAUSE 500
NEXT Zaehler

PRINT "Programm beendet."
```

Speichere das Programm und starte anschließend **Load and Run**.

MMEdit übergibt das Programm an MMCC. MMCC überträgt es zum PicoMite und startet es dort.

Im Terminal erscheinen nacheinander die Ausgaben des Programms.

Ändere nun beispielsweise:

```
PAUSE 500
```

in:

```
PAUSE 100
```

und führe erneut **Load and Run** aus.

Damit hast du bereits den wichtigsten MMEdit-Arbeitsablauf kennengelernt: **ändern, übertragen, testen**.

## 39.12. Syntax-Hervorhebung

MMEdit erkennt viele Elemente der MMBasic-Syntax und stellt sie mit Syntax-Hervorhebung unterschiedlich dar. Dazu gehören beispielsweise Schlüsselwörter, Kommentare, Zahlen und Zeichenketten.

Das erleichtert besonders bei längeren Programmen die Orientierung im Quelltext.

MMBasic und insbesondere der PicoMite werden ständig weiterentwickelt. Deshalb kann es vorkommen, dass sehr neue Befehle von einer älteren MMEdit-Syntaxdatei noch nicht farbig hervorgehoben werden.

Das bedeutet nicht automatisch, dass der Befehl auf dem PicoMite ungültig ist. Im Zweifelsfall ist das aktuelle PicoMite-Handbuch maßgeblich.

## 39.13. Programme auf dem PC speichern

MMEdit speichert BASIC-Programme als normale Textdateien. Für MMBasic-Programme wird üblicherweise die Dateiendung `.bas` verwendet.

Beispiele:

```
senso.bas
pong.bas
bahnhofsuhr.bas
```

Die Dateien können dadurch problemlos kopiert, gesichert, archiviert oder beispielsweise mit Git verwaltet werden.

## 39.14. Programme übertragen und starten

Für das normale Entwickeln ist **Load and Run** besonders praktisch.

Der aktuelle Quelltext wird dabei vom PC zum Zielsystem übertragen und anschließend ausgeführt. Dadurch musst du die Datei nicht nach jeder Änderung von Hand auf eine SD-Karte kopieren.

Das ist einer der größten Vorteile gegenüber einem Arbeitsablauf, bei dem Programme ausschließlich über Datenträger zwischen PC und PicoMite ausgetauscht werden.

## 39.15. MMEdit und den eingebauten Editor gemeinsam verwenden

MMEdit und der eingebaute Editor des PicoMite müssen nicht als zwei getrennte Arbeitsweisen betrachtet werden. Gerade bei der Fehlersuche können sich beide Werkzeuge sehr gut ergänzen.

Überträgst du ein Programm mit **Load and Run** aus MMEdit und MMBasic meldet beim Ausführen einen Fehler, befindet sich das Programm bereits im Speicher des PicoMite.

Drücke nun im MMCC-Terminal die Taste `F4`.

Damit öffnest du den eingebauten Editor des PicoMite. Die Schreibmarke steht dabei an der Stelle, an der MMBasic den Fehler erkannt hat.

Du kannst die betreffende Programmzeile nun unmittelbar korrigieren und das Programm anschließend mit F2 erneut starten.

Der Arbeitsablauf sieht dann beispielsweise so aus:

```
Programm in MMEdit bearbeiten
          |
          v
      Load and Run
          |
          v
     Fehler tritt auf
          |
          v
F4: eingebauten Editor öffnen
          |
          v
     Fehler korrigieren
          |
          v
    F2: erneut testen
```

Das ist besonders praktisch, wenn beim Testen nur eine kleine Änderung notwendig ist. Du musst das Programm nicht erst erneut in MMEdit ändern und übertragen.

Wenn du eine Änderung direkt im eingebauten Editor vorgenommen hast, befindet sich diese Änderung zunächst nur auf dem PicoMite.

Vergiss deshalb nicht, die erfolgreiche Korrektur anschließend auch in deiner Programmdatei auf dem PC nachzuführen.

Alternativ kannst du das geänderte Programm mit MMCC wieder vom PicoMite auf den PC übertragen.

Damit ergibt sich ein sehr praktischer Entwicklungsablauf:

```
       MMEdit auf dem PC
              |
              | Load and Run
              v
          PicoMite
              |
              | Fehler?
              v
     eingebauter Editor
              |
              | F4 bearbeiten
              | F2 testen
              v
```

```
   Korrektur funktioniert
             |
        +----+----+
        |         |
        v         v
 Änderung in   mit MMCC zurück
 MMEdit        auf den PC
 nachführen    übertragen
```

So kannst du die Vorteile beider Editoren kombinieren: MMEdit dient als komfortable Entwicklungsumgebung und Hauptablage des Quelltextes, während der eingebaute Editor schnelle Korrekturen direkt auf dem Zielsystem ermöglicht.

# 39.16. Dateien auf dem Zielsystem

MMCC besitzt zusätzlich einen Dateimanager. Damit können Dateien zwischen PC und MMBasic-System übertragen werden.

Das ist beispielsweise praktisch für:

- BASIC-Programme
- Datendateien
- Bilder
- Sounddateien
- Sicherungskopien

Beim PicoMite kennst du die Laufwerksbezeichnungen bereits aus Kapitel 41:

```
A:   interner Flash-Speicher
B:   SD-Karte, wenn konfiguriert
```

Damit ist die SD-Karte nur **eine** Möglichkeit, Dateien zu speichern. Für den normalen Editieren-Testen-Zyklus ist sie nicht erforderlich.

# 39.17. Programme vom Zielsystem holen

Über den Dateimanager können vorhandene Dateien auch vom Zielsystem auf den PC übertragen werden.

Das ist beispielsweise hilfreich, wenn ein Programm nur noch auf dem PicoMite oder auf dessen SD-Karte vorhanden ist und du es wieder auf dem PC bearbeiten möchtest.

Das gilt auch für Änderungen, die du während der Fehlersuche mit dem eingebauten Editor vorgenommen hast. Möchtest du diese Version übernehmen, kannst du das geänderte Programm wieder auf den PC übertragen.

Für eine saubere Projektverwaltung empfiehlt es sich trotzdem, den PC als Hauptablage für den Quelltext zu verwenden und wichtige Projekte zusätzlich zu sichern.

## 39.18. Arbeiten mit mehreren Dateien

Ein großer Vorteil gegenüber dem eingebauten MMBasic-Editor ist die Möglichkeit, mehrere Dateien gleichzeitig geöffnet zu halten.

Ein Projekt könnte beispielsweise aus folgenden Dateien bestehen:

```
senso.bas
sound.bas
anzeige.bas
test.bas
```

MMEdit zeigt geöffnete Dateien in Registerkarten an. Dadurch kannst du schnell zwischen verschiedenen Programmen wechseln und beispielsweise Programmteile von einer Datei in eine andere übernehmen.

## 39.19. Suchen und Ersetzen

Mit zunehmender Programmgröße werden Suchen und Ersetzen immer wichtiger.

Wenn eine Variable beispielsweise an vielen Stellen verwendet wird, kannst du nach ihrem Namen suchen:

```
Punktzahl
```

und die entsprechenden Stellen im Programm schnell finden.

Auch das Ersetzen eines Begriffs an mehreren Stellen ist wesentlich komfortabler als bei einer rein manuellen Bearbeitung.

## 39.20. Projekte sinnvoll organisieren

MMEdit schreibt dir keine bestimmte Projektstruktur vor. Trotzdem lohnt es sich, für jedes größere Projekt ein eigenes Verzeichnis anzulegen.

Beispiel:

```
LittleProfessor
|
+-- littleprofessor.bas
+-- fragen.dat
+-- bilder
|   +-- logo.jpg
```

```
|   +-- titel.jpg
+-- dokumentation
    +-- readme.txt
```

Damit liegen Programm, Daten und zusätzliche Dateien übersichtlich zusammen.

Bei größeren Projekten erleichtert eine solche Struktur nicht nur die Entwicklung, sondern auch Sicherungen und Versionsverwaltung.

# 39.21. MMEdit oder der eingebaute Editor?

Beide Werkzeuge haben ihre Berechtigung und können sich während der Entwicklung sehr gut ergänzen.

| **Aufgabe** | **MMBasic-Editor** | **MMEdit** |
|---|---|---|
| Kleine Änderungen direkt am Gerät | ✔ |  |
| Arbeiten ohne PC | ✔ |  |
| Mehrere Dateien gleichzeitig bearbeiten |  | ✔ |
| Große Programme bearbeiten |  | ✔ |
| Komfortables Suchen und Ersetzen |  | ✔ |
| Direkt auf dem Zielsystem arbeiten | ✔ |  |
| Programme schnell vom PC übertragen und testen |  | ✔ |
| Schnelle Korrektur nach einer Fehlermeldung | ✔ |  |

Für kurze Tests oder Änderungen direkt am PicoMite ist der eingebaute Editor hervorragend geeignet. Wenn du dagegen längere Programme auf dem PC entwickelst, ist MMEdit deutlich komfortabler.

Besonders praktisch ist die Kombination aus beiden: Das Programm wird in MMEdit entwickelt und übertragen. Tritt beim Testen ein Fehler auf, kann es direkt mit `F4` im eingebauten Editor geöffnet, korrigiert und mit `F2` erneut gestartet werden.

# 39.22. MMEdit oder Visual Studio Code?

Natürlich kannst du MMBasic-Programme auch mit einem allgemeinen Texteditor bearbeiten.

Beispiele sind:

- Visual Studio Code
- Notepad++
- Geany

MMEdit besitzt jedoch den Vorteil, speziell für MMBasic entwickelt worden zu sein. Die

Zusammenarbeit mit MMCC und damit das direkte Übertragen und Starten eines Programms ist bereits integriert.

Visual Studio Code bietet dagegen beispielsweise Vorteile bei Git, GitHub, Dokumentation oder der Bearbeitung vieler unterschiedlicher Dateitypen.

Es spricht nichts dagegen, beide Werkzeuge parallel zu verwenden.

## 39.23. Tipps aus der Praxis

- Lege für jedes größere Projekt ein eigenes Verzeichnis an.
- Verwende aussagekräftige Dateinamen.
- Sichere wichtige Projekte regelmäßig.
- Lass MMCC während der Entwicklung geöffnet.
- Teste Änderungen früh und häufig mit **Load and Run**.
- Nutze bei kleinen Korrekturen nach einer Fehlermeldung den eingebauten Editor mit `F4`.
- Teste die Korrektur direkt mit `F2`.
- Übernimm erfolgreiche Änderungen anschließend wieder in die PC-Version deines Programms.
- Bewahre die PC-Version deines Quelltextes als Hauptversion auf.
- Verwende bei Bedarf den MMCC-Dateimanager, um zusätzliche Dateien zum PicoMite oder zurück auf den PC zu übertragen.

## 39.24. Probier’s selbst!

1. Verbinde MMCC mit deinem PicoMite.
2. Prüfe die Verbindung mit `PRINT MM.INFO(VERSION)`.
3. Erstelle in MMEdit das Beispielprogramm aus diesem Kapitel.
4. Übertrage und starte es mit **Load and Run**.
5. Ändere die Pause von 500 auf 100 Millisekunden und teste erneut.
6. Baue testweise einen kleinen Fehler in das Programm ein und starte es erneut.
7. Öffne nach der Fehlermeldung mit `F4` den eingebauten Editor.
8. Korrigiere den Fehler und teste das Programm mit `F2`.
9. Übernimm die Korrektur anschließend auch in MMEdit.
10. Öffne zusätzlich ein zweites BASIC-Programm in einer weiteren Registerkarte.
11. Probiere die Suchfunktion von MMEdit aus.

## 39.25. Zusammenfassung

In diesem Kapitel hast du gelernt:

- MMEdit ist ein speziell für MMBasic entwickelter Programmeditor.
- MMCC übernimmt die Kommunikation mit dem Zielsystem und dient als Terminal.
- Dieses Tutorial wurde ursprünglich mit MMEdit V5.2.9 erstellt; inzwischen verwende ich MMEdit V5.4.3.
- Neuere MMEdit-Versionen bieten unter anderem eine schnellere Programmübertragung.
- Mit **Load and Run** entsteht ein schneller Zyklus aus Bearbeiten, Übertragen und Testen.
- Nach einer Fehlermeldung kannst du mit `F4` den eingebauten Editor öffnen, eine Korrektur vornehmen und das Programm mit `F2` erneut testen.
- Änderungen auf dem PicoMite sollten anschließend auch in die PC-Version übernommen oder mit MMCC zurückübertragen werden.
- Der MMCC-Dateimanager ermöglicht zusätzlich die Übertragung von Dateien zwischen PC und Zielsystem.
- Mehrere Registerkarten sowie Suchen und Ersetzen erleichtern die Arbeit an größeren Programmen.
- Der eingebaute MMBasic-Editor und MMEdit ergänzen sich: MMEdit eignet sich für die komfortable Entwicklung auf dem PC, der eingebaute Editor für schnelle Änderungen und Tests direkt auf dem Zielsystem.

Damit endet der Teil **Werkzeuge und Entwicklung**. Du kennst nun sowohl den eingebauten Editor des PicoMite als auch eine komfortable Entwicklungsumgebung für den PC – und weißt, wie beide Werkzeuge gemeinsam eingesetzt werden können.

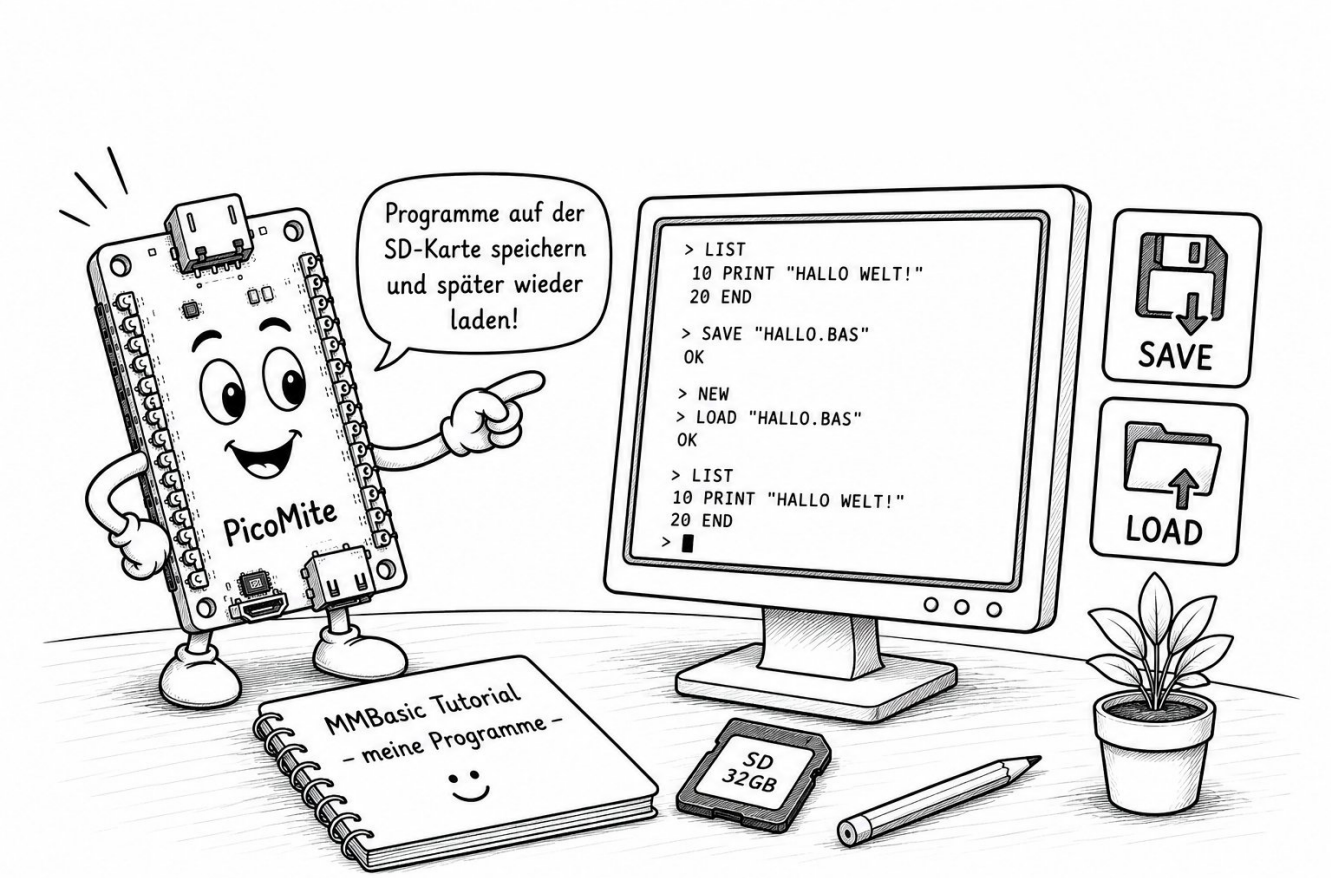
## 39.26. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 39_mmedit_hallo.bas

# Teil VI – Dateien und Projekte

# Kapitel 40. Programme speichern, laden und Libraries verwenden




## 40.1. Das lernst du

In diesem Kapitel vertiefst du den Umgang mit Programmen und Dateien.

Du lernst,

- die verschiedenen Speicherbereiche des PicoMite zu unterscheiden,
- Programme als Dateien zu speichern, zu laden und direkt zu starten,
- mehrere Programme sinnvoll zu organisieren,
- zusätzliche Programme in Flash-Slots abzulegen,
- wiederverwendbare Unterprogramme und Funktionen in einer Library zu speichern und
- einen bewährten Arbeitsablauf für größere Projekte.

## 40.2. Warum dieses Kapitel?

In Kapitel 8 hast du bereits gelernt, Programme mit `SAVE` als Datei zu speichern und mit `LOAD` wieder zu laden.

Für die bisherigen Beispiele war das vollkommen ausreichend.

Mit der Zeit entstehen jedoch mehrere Programme gleichzeitig. Vielleicht entwickelst du ein Spiel, arbeitest parallel an einem Messprogramm oder möchtest ältere Versionen deiner Programme aufbewahren.

Jetzt ist der richtige Zeitpunkt, den Umgang mit Programmen und Dateien etwas genauer kennenzulernen.

# 40.3. Wo speichert der PicoMite Programme?

Beim PicoMite begegnen dir mehrere Speicherbereiche. Sie erfüllen unterschiedliche Aufgaben und sollten nicht miteinander verwechselt werden.

| Speicherort | Inhalt | Bleibt erhalten? | Typischer Zugriff |
|---|---|---|---|
| Programmspeicher | Das aktuelle BASIC-Programm | Ja | `EDIT`, `RUN` |
| Flash-Dateisystem `A:` | Programme und andere Dateien | Ja | `SAVE`, `LOAD`, `RUN "…"` |
| SD-Karte `B:` | Programme und andere Dateien | Ja | `SAVE`, `LOAD`, `RUN "…"` |
| Flash-Slots 1 bis 3 | Zusätzliche BASIC-Programme | Ja | `FLASH SAVE`, `FLASH LOAD`, `FLASH RUN` |

## 40.3.1. Der Programmspeicher

Das aktuell bearbeitete BASIC-Programm liegt im **Programmspeicher** des PicoMite. Dieser befindet sich im Flash-Speicher und ist deshalb nichtflüchtig. Das Programm bleibt also auch erhalten, wenn du den PicoMite ausschaltest oder zurücksetzt.

Wenn du ein Programm mit `EDIT` bearbeitest und den Editor mit **F1** (`Save`) verlässt, befindet sich die bearbeitete Version im Programmspeicher. Mit

```
RUN
```

kannst du dieses Programm jederzeit starten.

Der Programmspeicher enthält immer nur **ein aktuelles Arbeitsprogramm**.

Du musst dieses Programm nicht mit `SAVE` als Datei sichern, damit es einen Neustart oder das Ausschalten übersteht. `SAVE` hat eine andere Aufgabe: Der Befehl erzeugt zusätzlich eine Programmdatei im Dateisystem.

## 40.3.2. Das Flash-Dateisystem A:

Zusätzlich zum Programmspeicher besitzt der PicoMite ein eigenes Flash-Dateisystem. Es wird als Laufwerk `A:` angesprochen und verhält sich ähnlich wie ein kleines Laufwerk auf einem Computer.

Hier kannst du viele Programme und andere Dateien ablegen.

Nach dem Start des PicoMite ist `A:` normalerweise das aktive Laufwerk. Deshalb entspricht

```
SAVE "HANGMAN.BAS"
```

in diesem Fall einer Speicherung auf Laufwerk `A:`. Ausführlich könntest du auch schreiben:

```
SAVE "A:/HANGMAN.BAS"
```

Das Programm befindet sich anschließend weiterhin im Programmspeicher und zusätzlich als Datei im Flash-Dateisystem.

Mit

```
FILES
```

kannst du dir die Dateien und Verzeichnisse des aktuellen Laufwerks anzeigen lassen.

### 40.3.3. Die SD-Karte B:

Wenn dein PicoMite eine konfigurierte SD-Karte besitzt, wird sie als Laufwerk `B:` angesprochen. Ein Programm kannst du beispielsweise direkt dort speichern:

```
SAVE "B:/HANGMAN.BAS"
```

Mit `DRIVE` lässt sich auch das aktive Laufwerk wechseln:

```
DRIVE "B:"
```

Danach beziehen sich Dateinamen ohne Laufwerksangabe auf die SD-Karte. Mit

```
DRIVE "A:"
```

wechselst du wieder zum Flash-Dateisystem zurück.

## 40.4. Programme speichern

Mit `SAVE` speicherst du das aktuelle Arbeitsprogramm zusätzlich als Datei:

```
SAVE "HANGMAN.BAS"
```

Das ist besonders wichtig, wenn du mehrere Programme aufbewahren, Sicherungskopien anlegen oder Programme auf einen anderen Rechner übertragen möchtest.

`SAVE` macht den Programmspeicher nicht erst dauerhaft. Der Programmspeicher ist bereits nichtflüchtig. `SAVE` legt eine **zusätzliche Datei** auf `A:` oder `B:` an.

## 40.5. Programme laden

Mit `LOAD` lädst du eine gespeicherte BASIC-Datei in den Programmspeicher:

```
LOAD "HANGMAN.BAS"
```

Das zuvor im Programmspeicher befindliche Arbeitsprogramm wird dadurch ersetzt. Anschließend kannst du das geladene Programm mit

```
RUN
```

starten oder mit `EDIT` weiter bearbeiten.

Mit der zusätzlichen Option `, R` kannst du ein Programm laden und anschließend sofort starten:

```
LOAD "HANGMAN.BAS", R
```

Noch kürzer geht es mit:

```
RUN "HANGMAN.BAS"
```

`RUN` mit einem Dateinamen lädt das Programm und führt es unmittelbar aus.

## 40.6. Programme direkt bearbeiten

Eine gespeicherte BASIC-Datei kannst du auch direkt mit dem Editor öffnen:

```
EDIT "HANGMAN.BAS"
```

MMBasic lädt die Datei dabei in den Programmspeicher und öffnet sie im Editor. So kannst du ein vorhandenes Programm bequem weiterbearbeiten.

## 40.7. Zusätzliche Programme in Flash-Slots

Neben dem normalen Programmspeicher und den Dateisystemen besitzt der PicoMite drei zusätzliche Flash-Slots. Dort kannst du beispielsweise andere Programme oder frühere Versionen deines aktuellen Programms ablegen.

Die wichtigsten Befehle sind `FLASH SAVE`, `FLASH LOAD`, `FLASH RUN`, `FLASH LIST` und `FLASH ERASE`:

```
FLASH SAVE 1
FLASH LOAD 1
FLASH RUN 1
FLASH LIST
FLASH ERASE 1
```

Die Nummer kann zwischen 1 und 3 liegen. `FLASH SAVE 1` kopiert beispielsweise das aktuelle Arbeitsprogramm in Flash-Slot 1. Mit `FLASH LOAD 1` holst du es wieder in den Programmspeicher.

Die Library verwendet Flash-Slot 3. Sobald du die Library-Funktion nutzt, stehen dir deshalb nur noch die Flash-Slots 1 und 2 für normale Programme zur Verfügung.

## 40.8. Programme organisieren

Mit zunehmender Erfahrung wächst auch deine Programmsammlung.

Eine gute Organisation hilft dir dabei, jederzeit den Überblick zu behalten.

### 40.8.1. Aussagekräftige Dateinamen

Verwende möglichst sprechende Dateinamen.

Gut geeignet sind beispielsweise:

- `HANGMAN.BAS`
- `LITTLEPROFESSOR.BAS`
- `REAKTIONSTEST.BAS`

Weniger hilfreich sind Namen wie:

- `TEST.BAS`
- `NEU.BAS`
- `SPIEL1.BAS`

Schon nach kurzer Zeit wirst du Programme anhand ihres Dateinamens schneller wiederfinden.

### 40.8.2. Sicherheitskopien anlegen

Vor größeren Änderungen empfiehlt es sich, eine Sicherheitskopie anzulegen.

Beispielsweise:

```
PONG_V1.BAS
PONG_V2.BAS
```

So kannst du jederzeit zu einer funktionierenden Version zurückkehren.

### 40.8.3. Ordner verwenden

Bei einer größeren Programmsammlung lohnt es sich, eigene Ordner anzulegen.

Zum Beispiel:

```
/Spiele
/Lernprogramme
/Elektronik
/Experimente
```

Dadurch bleiben deine Projekte dauerhaft übersichtlich.

## 40.9. Wiederverwendbare Programmteile

In größeren Projekten werden bestimmte Unterprogramme und Funktionen häufig mehrfach benötigt.

Vielleicht hast du beispielsweise Routinen geschrieben, die

- Texte auf dem Bildschirm positionieren,
- Messwerte umrechnen,
- Eingaben überprüfen oder
- wiederkehrende Berechnungen durchführen.

Du könntest diese Routinen in jedes neue Programm kopieren. Das wird jedoch schnell unübersichtlich. Außerdem müsstest du Fehlerkorrekturen anschließend in allen Kopien nachtragen.

MMBasic bietet dafür eine Library. Darin gespeicherte Routinen stehen allen Programmen zur Verfügung und können wie eingebaute Befehle oder Funktionen verwendet werden.

### 40.9.1. Eine Library-Routine erstellen

Library-Routinen werden zunächst wie gewöhnliche Bestandteile eines MMBasic-Programms geschrieben und getestet.

Ein einfaches Beispiel:

```
SUB Begruessung
  PRINT "Willkommen bei MMBasic!"
END SUB

FUNCTION Quadrat(Zahl)
  Quadrat = Zahl * Zahl
END FUNCTION
```

Teste die Routinen zunächst mit einem kleinen Hauptprogramm:

```
Begruessung
PRINT "Das Quadrat von 7 ist"; Quadrat(7)

SUB Begruessung
  PRINT "Willkommen bei MMBasic!"
END SUB

FUNCTION Quadrat(Zahl)
  Quadrat = Zahl * Zahl
END FUNCTION
```

Wenn alles fehlerfrei funktioniert, speicherst du den Quelltext zuerst als normale BASIC-Datei:

```
SAVE "MEINELIB.BAS"
```

Danach überträgst du den Inhalt des Programmspeichers mit `LIBRARY SAVE` in die Library.

```
LIBRARY SAVE
```

in die Library.

MMBasic komprimiert den Quelltext und speichert ihn in einem besonderen Bereich des Flash-Speichers. Anschließend wird der normale Programmspeicher geleert.

Speichere den Quelltext vor `LIBRARY SAVE` immer zusätzlich als normale BASIC-Datei.

Beim Übertragen in die Library entfernt MMBasic unter anderem Kommentare, Leerzeilen und überflüssige Leerzeichen. Die Library ist deshalb nicht als Ersatz für die ursprüngliche Quelltextdatei gedacht.

### 40.9.2. Library-Routinen verwenden

Nach dem Speichern stehen die Routinen allen BASIC-Programmen zur Verfügung.

Ein neues Programm kann sie direkt aufrufen:

```
Begruessung

PRINT "Das Quadrat von 12 ist"; Quadrat(12)
```

Die Definitionen von `Begruessung` und `Quadrat` müssen nicht mehr im Hauptprogramm enthalten sein.

Die Routinen bleiben auch dann erhalten, wenn du mit `NEW` den Programmspeicher löschst oder ein anderes Programm lädst.

Du kannst sie sogar direkt an der Eingabeaufforderung verwenden:

```
PRINT Quadrat(9)
```

MMBasic behandelt Library-Routinen damit fast wie zusätzliche Bestandteile der Programmiersprache.

### 40.9.3. Die Library erweitern

Du kannst `LIBRARY SAVE` mehrfach verwenden.

Bei jedem Aufruf wird der aktuelle Inhalt des Programmspeichers an die bereits vorhandene Library angehängt.

So kannst du deine Library schrittweise um weitere Routinen ergänzen.

Achte darauf, keine Routinen mit bereits verwendeten Namen erneut hinzuzufügen. Da die neuen Bestandteile an die vorhandene Library angehängt werden, solltest du deine Library sorgfältig planen und den vollständigen Quelltext aufbewahren.

### 40.9.4. Inhalt der Library anzeigen

Mit `LIBRARY LIST`:

```
LIBRARY LIST
```

kannst du dir den Inhalt der Library anzeigen lassen.

Bei einem normalen

```
LIST
```

werden die Library-Routinen dagegen nicht angezeigt. `LIST` zeigt nur das aktuell geladene Hauptprogramm.

### 40.9.5. Library sichern und wiederherstellen

Die gesamte Library kann mit `LIBRARY DISK SAVE` als Datei gesichert werden:

```
LIBRARY DISK SAVE "MEINELIB.LIB"
```

Mit `LIBRARY DISK LOAD`:

```
LIBRARY DISK LOAD "MEINELIB.LIB"
```

kannst du sie später wiederherstellen.

Diese Sicherung ist beispielsweise nützlich, wenn du die Firmware aktualisieren oder dieselbe Library auf einem anderen PicoMite verwenden möchtest.

### 40.9.6. Library löschen

Mit `LIBRARY DELETE`:

```
LIBRARY DELETE
```

wird die gesamte Library gelöscht.

Der dafür verwendete Flash-Speicher steht anschließend wieder für normale Programme zur Verfügung.

Die Library verwendet den Flash-Slot 3. Solange eine Library installiert ist, kann dieser Slot nicht zum Speichern eines normalen Programms verwendet werden.

### 40.9.7. Was kann eine Library enthalten?

Eine Library kann mehr als nur `SUB`- und `FUNCTION`-Routinen enthalten.

Möglich sind unter anderem:

- Unterprogramme,
- Funktionen,
- eingebettete C-Routinen,
- eingebettete Fonts und

- Initialisierungscode.

Für den Einstieg sind wiederverwendbare Unterprogramme und Funktionen jedoch am wichtigsten.

# 40.10. Automatisch ausgeführter Library-Code

Eine Library kann auch Anweisungen enthalten, die sich nicht innerhalb einer `SUB` oder `FUNCTION` befinden.

Solcher Code wird automatisch ausgeführt, bevor ein Programm mit `RUN` gestartet wird.

Damit können beispielsweise Konstanten bereitgestellt werden:

```
CONST TRUE = 1
CONST FALSE = 0
```

Nach dem Speichern in der Library stehen `TRUE` und `FALSE` allen Programmen zur Verfügung.

Initialisierungscode in der Library wird vor jedem mit `RUN` gestarteten Programm ausgeführt.

Er ist nicht mit der besonderen Routine `MM.STARTUP` zu verwechseln, die nur beim Einschalten oder Zurücksetzen des PicoMite ausgeführt wird.

## 40.10.1. Besondere Systemroutinen

MMBasic kennt einige besondere Namen für Unterprogramme.

**`MM.STARTUP`**

Wird einmal nach dem Einschalten oder einem Reset ausgeführt. Damit kann beispielsweise eine angeschlossene Echtzeituhr eingelesen werden.

```
SUB MM.STARTUP
  RTC GETTIME
END SUB
```

Verwende `MM.STARTUP` nicht zur allgemeinen Vorbereitung eines später mit `RUN` gestarteten Programms, beispielsweise zum Dimensionieren von Arrays oder zum Öffnen von Kommunikationskanälen.

Beim Start mit `RUN` setzt MMBasic den Zustand des Interpreters für einen neuen Programmlauf zurück. Solche Vorbereitungen gehören deshalb in den normalen Initialisierungsteil des Hauptprogramms oder in den automatisch vor `RUN` ausgeführten Library-Code.

`MM.PROMPT`

Wird anstelle der normalen Eingabeaufforderung ausgeführt. Damit lässt sich beispielsweise eine eigene Eingabeaufforderung anzeigen.

```
SUB MM.PROMPT
  PRINT TIME$ "> ";
END SUB
```

`MM.END`

Wird ausgeführt, wenn ein Programm durch `END` oder durch das Erreichen des Programmendes beendet wird.

Diese Routinen sind für besondere Anwendungen gedacht. Für gewöhnliche Initialisierungen innerhalb eines Programms ist weiterhin normaler Programmcode besser geeignet.

## 40.11. Ein typischer Arbeitsablauf

Beim Entwickeln arbeitest du zunächst mit dem aktuellen Programm im Programmspeicher:

```
EDIT
 
Programm entwickeln
 
F1 (Save)
 
RUN
```

Wenn du einen bestimmten Entwicklungsstand dauerhaft als eigene Datei aufbewahren möchtest, sicherst du ihn zusätzlich:

```
SAVE "MEINPROGRAMM.BAS"
```

Zu einem späteren Zeitpunkt kannst du diese Version wieder laden und weiterbearbeiten:

```
LOAD "MEINPROGRAMM.BAS"
 
EDIT
 
Programm erweitern
 
F1 (Save)
 
RUN
```

Der entscheidende Unterschied ist: **F1 speichert die Änderungen des Editors im aktuellen**

**Programmspeicher. `SAVE` legt zusätzlich eine Datei im Dateisystem an.**

Dieser Arbeitsablauf ist besonders praktisch, weil du während der Entwicklung schnell arbeiten kannst und wichtige Zwischenstände trotzdem als eigene Dateien aufbewahrst.

## 40.12. Profitipps

**Programme automatisch starten**

Soll dein Programm nach jedem Einschalten des PicoMite automatisch gestartet werden, kannst du dies mit folgendem Befehl aktivieren:

```
OPTION AUTORUN ON
```

Während der Entwicklung empfiehlt es sich, diese Funktion zunächst deaktiviert zu lassen und Programme manuell mit `RUN` zu starten.

Mit

```
OPTION AUTORUN OFF
```

wird der automatische Start wieder abgeschaltet.

**Programme regelmäßig sichern**

Speichere wichtige Programme zusätzlich auf deinem PC oder in einem Cloud-Speicher. Eine SD-Karte kann verloren gehen oder beschädigt werden.

**Ordnung spart Zeit**

Eine sinnvolle Ordnerstruktur und aussagekräftige Dateinamen helfen dir, auch bei vielen Projekten den Überblick zu behalten.

## 40.13. Zusammenfassung

Du kennst nun die wichtigsten Möglichkeiten, Programme auf dem PicoMite zu speichern. Das aktuelle Arbeitsprogramm liegt im nichtflüchtigen Programmspeicher und bleibt auch nach dem Ausschalten erhalten.

Mit `SAVE` legst du zusätzliche Programmdateien im Flash-Dateisystem `A:` oder auf der SD-Karte `B:` an. Mit `LOAD` kannst du sie wieder in den Programmspeicher laden, während `RUN "DATEI.BAS"` ein gespeichertes Programm direkt lädt und startet.

Die zusätzlichen Flash-Slots eignen sich zum Aufbewahren weiterer Programme. Außerdem hast du gelernt, wie wiederverwendbare `SUB`- und `FUNCTION`-Routinen in einer Library gespeichert werden und anschließend allen Programmen zur Verfügung stehen.

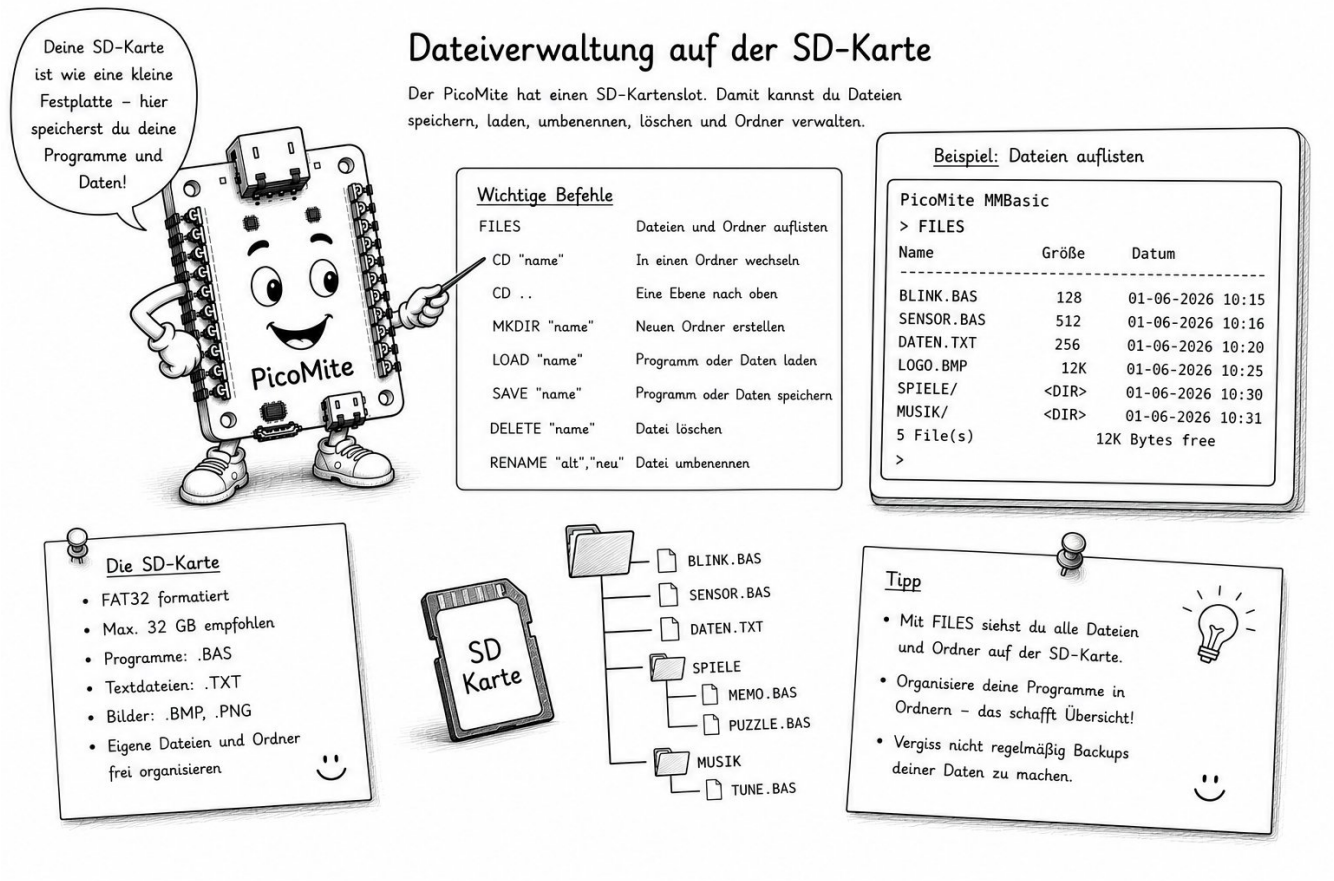
Mit aussagekräftigen Dateinamen, Ordnern und regelmäßigen Sicherungen behältst du auch bei größeren BASIC-Projekten den Überblick.

## 40.14. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 40_library_test.bas

# Kapitel 41. Dateiverwaltung auf der SD-Karte




## 41.1. Das lernst du

In diesem Kapitel lernst du:

- zwischen Flash-Dateisystem und SD-Karte zu wechseln
- Dateien und Verzeichnisse anzuzeigen
- Verzeichnisse anzulegen
- zwischen Verzeichnissen zu wechseln
- das aktuelle Arbeitsverzeichnis zu ermitteln
- Dateien umzubenennen und zu verschieben
- Dateien zu kopieren und zu löschen
- Verzeichnisse zu löschen
- Informationen über ein Laufwerk abzufragen
- eine sinnvolle Projektstruktur aufzubauen

Am Ende des Kapitels kannst du auch größere Programmsammlungen übersichtlich organisieren und verwalten.

## 41.2. Warum Verzeichnisse?

Im vorherigen Kapitel hast du gelernt, Programme sinnvoll zu organisieren und mit `SAVE` und `LOAD` zu verwalten.

Sobald mehrere Projekte entstehen, reicht es jedoch nicht mehr aus, alle Dateien in einem einzigen Verzeichnis abzulegen.

Mit Verzeichnissen kannst du deine Programme thematisch ordnen und behältst auch bei einer größeren Programmsammlung den Überblick.

Beispiel:

```
PONG.BAS
```

Mit zunehmender Erfahrung entstehen jedoch immer mehr Programme.

Beispielsweise:

```
PONG.BAS
SENSO.BAS
AMPEL.BAS
UHR.BAS
TEST.BAS
```

Irgendwann wird es schwierig, den Überblick zu behalten.

Deshalb unterstützt MMBasic Verzeichnisse und eine Reihe von Befehlen zur Dateiverwaltung.

## 41.3. Laufwerke beim PicoMite

Der PicoMite unterscheidet zwischen zwei Laufwerken:

| Laufwerk | Speicher | Hinweis |
|---|---|---|
| `A:` | Flash-Dateisystem | Ist im PicoMite selbst gespeichert. |
| `B:` | SD-Karte | Steht zur Verfügung, wenn eine SD-Karte konfiguriert ist. |

Das aktuell verwendete Laufwerk wird als **aktives Laufwerk** bezeichnet. Alle Dateibefehle ohne ausdrücklich angegebenes Laufwerk beziehen sich auf dieses aktive Laufwerk.

Mit `DRIVE` wechselst du das aktive Laufwerk.

Um beispielsweise auf die SD-Karte zu wechseln, verwendest du:

```
DRIVE "B:"
```

Zurück zum Flash-Dateisystem gelangst du mit:

```
DRIVE "A:"
```

Welches Laufwerk gerade aktiv ist, kannst du mit `MM.INFO(drive)` abfragen:

```
PRINT MM.INFO(drive)
```

Die Ausgabe lautet beispielsweise:

```
B:
```

Damit weißt du, dass sich die folgenden Dateibefehle auf die SD-Karte beziehen.

Die SD-Karte ist beim PicoMite das Laufwerk `B:`. Das Laufwerk muss jedoch entsprechend konfiguriert sein, damit es verwendet werden kann.

## 41.4. Dateien und Verzeichnisse anzeigen

Bevor du Dateien kopierst, umbenennst oder löschst, möchtest du normalerweise zuerst sehen, welche Dateien und Verzeichnisse vorhanden sind.

Dafür gibt es den bereits bekannten Befehl `FILES`. Neu ist hier die Verwendung von Wildcards für Dateimuster.

```
FILES
```

`FILES` durchsucht das aktuelle Verzeichnis des aktiven Laufwerks und zeigt die gefundenen Dateien und Unterverzeichnisse an.

Eine mögliche Ausgabe könnte beispielsweise so aussehen:

```
SPIELE
PROJEKTE
PONG.BAS
TEST.BAS
```

`FILES` kann auch mit einem Suchmuster verwendet werden. Möchtest du zum Beispiel nur BASIC-Programme anzeigen, kannst du schreiben:

```
FILES "*.BAS"
```

Das Sternchen * steht dabei als Platzhalter für eine beliebige Folge von Zeichen.

`FILES` kann nur an der MMBasic-Kommandozeile verwendet werden. Innerhalb eines laufenden BASIC-Programms steht der Befehl nicht zur Verfügung.

Soll ein Programm selbst nach Dateien suchen, gibt es dafür die Funktion `DIR$(fspec, type)`. Sie ermöglicht die Dateisuche aus einem BASIC-Programm heraus. Die genaue Verwendung von `DIR$()` wird später behandelt, wenn wir uns mit Dateizugriffen aus Programmen beschäftigen.

## 41.5. Inhalt einer Textdatei anzeigen

Mit `LIST` kannst du den Inhalt eines BASIC-Programms oder einer Textdatei auf der Konsole anzeigen.

Beispiel:

```
LIST "PONG.BAS"
```

Das ist praktisch, wenn du den Inhalt einer Datei kurz ansehen möchtest, ohne sie zuerst mit `LOAD` als aktuelles Programm zu laden.

## 41.6. Eine sinnvolle Projektstruktur

Ein möglicher Aufbau auf der SD-Karte könnte so aussehen:

```
/
 ├──── SPIELE
 │     ├──── PONG.BAS
 │     └──── SENSO.BAS
 │
 ├──── LERNPROGRAMME
 │     ├──── HANGMAN.BAS
 │     └──── LITTLEPROFESSOR.BAS
 │
 ├──── ELEKTRONIK
 │     ├──── AMPEL.BAS
 │     └──── UHR.BAS
 │
 └──── TEST
```

Dadurch bleiben Programme übersichtlich organisiert.

## 41.7. Verzeichnisse anlegen

Mit `MKDIR` wird ein neues Verzeichnis im aktuellen Verzeichnis des aktiven Laufwerks erstellt.

Beispiel:

```
MKDIR "SPIELE"
```

Mit `FILES` kannst du anschließend kontrollieren, ob das neue Verzeichnis angelegt wurde:

```
FILES
```

## 41.8. Verzeichnis wechseln

Mit `CHDIR` wird das aktuelle Arbeitsverzeichnis geändert.

Beispiel:

```
CHDIR "SPIELE"
```

Alle folgenden Dateibefehle ohne vollständige Pfadangabe beziehen sich nun auf dieses Verzeichnis.

## 41.9. Das aktuelle Verzeichnis anzeigen

Mit `CWD$` kannst du das aktuelle Arbeitsverzeichnis abfragen.

```
PRINT CWD$
```

Befindest du dich beispielsweise im Verzeichnis `SPIELE`, zeigt die Ausgabe den entsprechenden Pfad an.

Das ist besonders nützlich, wenn du zuvor mehrmals mit `CHDIR` zwischen Verzeichnissen gewechselt hast und nicht mehr sicher bist, in welchem Verzeichnis du dich gerade befindest.

## 41.10. Besondere Verzeichnisnamen

Bei `CHDIR` gibt es einige besondere Verzeichnisnamen:

| Eintrag | Bedeutung |
|---|---|
| `.` | Aktuelles Verzeichnis |
| `..` | Eine Verzeichnisebene nach oben |
| `\` | Wurzelverzeichnis des aktuellen Laufwerks |

Zum Wurzelverzeichnis des aktuellen Laufwerks gelangst du also mit:

```
CHDIR "\"
```

Eine Ebene nach oben wechselst du mit:

```
CHDIR ".."
```

Beispiel:

```
/
 └──── SPIELE
     └──── PONG.BAS
```

Befindest du dich im Verzeichnis `SPIELE`, führt:

```
CHDIR ".."
```

zurück in das übergeordnete Verzeichnis.

# 41.11. Dateien umbenennen

Mit `RENAME` kann eine Datei einen neuen Namen erhalten.

Beispiel:

```
RENAME "PONG.BAS" AS "PONG_V1.BAS"
```

Danach heißt die Datei:

```
PONG_V1.BAS
```

# 41.12. Dateien verschieben

`RENAME` kann gleichzeitig zum Verschieben einer Datei verwendet werden. Dazu gibst du als neuen Namen einen Pfad an.

Beispiel:

```
RENAME "PONG.BAS" AS "/SPIELE/PONG.BAS"
```

Die Datei wird dabei in das Verzeichnis `SPIELE` verschoben.

# 41.13. Dateien kopieren

Mit `COPY` wird eine Datei kopiert.

Beispiel:

```
COPY "PONG.BAS" TO "PONG_ALT.BAS"
```

Danach existieren beide Dateien:

```
PONG.BAS
PONG_ALT.BAS
```

Auch `COPY` unterstützt Pfadangaben. Dadurch kannst du eine Datei direkt in ein anderes Verzeichnis kopieren:

```
COPY "/SPIELE/PONG.BAS" TO "/TEST/PONG.BAS"
```

## 41.14. Warum Kopien sinnvoll sind

Vor größeren Änderungen kann eine Sicherheitskopie erstellt werden.

Beispiel:

```
COPY "SENSO.BAS" TO "SENSO_BAK.BAS"
```

Falls bei späteren Änderungen etwas schiefgeht, steht die ursprüngliche Version weiterhin zur Verfügung.

## 41.15. Dateien löschen

Mit `KILL` wird eine Datei im aktuellen Verzeichnis des aktiven Laufwerks gelöscht.

Beispiel:

```
KILL "TEST.BAS"
```

Danach ist die Datei nicht mehr vorhanden.

Gelöschte Dateien können in der Regel nicht wiederhergestellt werden.

Verwende `KILL` deshalb mit Vorsicht.

`KILL` unterstützt außerdem Platzhalter zum Löschen mehrerer Dateien. Damit können sehr schnell viele Dateien entfernt werden. Solche Löschvorgänge solltest du deshalb besonders sorgfältig prüfen.

## 41.16. Verzeichnisse löschen

Mit `RMDIR` wird ein Verzeichnis im aktuellen Verzeichnis des aktiven Laufwerks entfernt.

Beispiel:

```
RMDIR "TEST"
```

Das Verzeichnis muss leer sein. Befinden sich darin noch Dateien oder weitere Unterverzeichnisse, kann es nicht einfach mit `RMDIR` entfernt werden.

## 41.17. Prüfen, ob Dateien oder Verzeichnisse existieren

In einem BASIC-Programm ist es oft hilfreich, vor einem Dateizugriff zu prüfen, ob eine bestimmte Datei vorhanden ist.

Dafür gibt es `MM.INFO(exists file fname$)`.

Beispiel:

```
IF MM.INFO(exists file "PONG.BAS") THEN
  PRINT "PONG.BAS ist vorhanden."
ELSE
  PRINT "PONG.BAS wurde nicht gefunden."
ENDIF
```

Entsprechend kannst du mit `MM.INFO(exists dir dirname$)` prüfen, ob ein Verzeichnis existiert:

```
IF MM.INFO(exists dir "SPIELE") THEN
  PRINT "Das Verzeichnis SPIELE ist vorhanden."
ENDIF
```

Diese Funktionen sind besonders nützlich, wenn ein Programm mit mehreren Dateien oder Verzeichnissen arbeitet.

## 41.18. Speicherplatz des Laufwerks abfragen

MMBasic kann auch Informationen über das aktive Laufwerk liefern.

Mit:

```
PRINT MM.INFO(free space)
```

wird mit `MM.INFO(free space)` der noch freie Speicherplatz des aktiven Laufwerks ausgegeben.

Die Größe des Laufwerks erhältst du mit `MM.INFO(disk size)`:

```
PRINT MM.INFO(disk size)
```

Zusammen mit `MM.INFO(drive)` kannst du damit schnell feststellen, welches Laufwerk aktiv ist und wie viel Speicherplatz dort zur Verfügung steht.

# 41.19. Der Pfad des laufenden Programms

Mit `MM.INFO(path)` kann ein Programm ermitteln, aus welchem Verzeichnis es gestartet wurde.

```
PRINT MM.INFO(path)
```

Das ist vor allem bei größeren Projekten nützlich. Ein Programm kann damit Ressourcen wie Daten- oder Konfigurationsdateien relativ zu seinem eigenen Speicherort finden.

# 41.20. Beispielprojekt anlegen

Erstellen wir nun auf der SD-Karte eine kleine Projektstruktur.

Zuerst wechseln wir auf die SD-Karte:

```
DRIVE "B:"
```

Dann wechseln wir in das Wurzelverzeichnis:

```
CHDIR "\"
```

Nun legen wir einige Verzeichnisse an:

```
MKDIR "SPIELE"
MKDIR "PROJEKTE"
MKDIR "TEST"
```

Mit:

```
FILES
```

kannst du das Ergebnis kontrollieren.

Die Struktur sieht nun beispielsweise so aus:

```
/
 ├──── SPIELE
 ├──── PROJEKTE
 └──── TEST
```

## 41.21. Dateien organisieren

Wechseln wir zunächst in das Verzeichnis `SPIELE`:

```
CHDIR "SPIELE"
```

Mit:

```
PRINT CWD$
```

kannst du kontrollieren, ob du dich im gewünschten Verzeichnis befindest.

Nun speichern wir dort beispielsweise ein Programm:

```
SAVE "PONG.BAS"
```

Mit `FILES` kannst du anschließend prüfen, ob die Datei vorhanden ist.

## 41.22. Vollständige Pfadangaben

Dateibefehle können auch mit vollständigen Pfaden verwendet werden. Dadurch musst du nicht für jeden Zugriff zuerst mit `CHDIR` in ein anderes Verzeichnis wechseln.

Beispiel:

```
LOAD "/SPIELE/PONG.BAS"
```

oder:

```
COPY "/SPIELE/PONG.BAS" TO "/TEST/PONG.BAS"
```

Das ist besonders praktisch bei größeren Projekten.

# 41.23. Dateiverwaltung an der Kommandozeile und im Programm

Viele Befehle zur Dateiverwaltung können sowohl an der MMBasic-Kommandozeile als auch innerhalb eines BASIC-Programms verwendet werden.

Dazu gehören unter anderem:

- `DRIVE`
- `MKDIR`
- `CHDIR`
- `RMDIR`
- `COPY`
- `RENAME`
- `KILL`

Eine wichtige Ausnahme ist `FILES`: Dieser Befehl kann nur an der Kommandozeile verwendet werden.

Für die Arbeit mit Dateien innerhalb eines Programms stellt MMBasic zusätzliche Funktionen bereit. Dazu gehören beispielsweise `DIR$()`, `INPUT$()`, `EOF()`, `LOC()` und `LOF()`. Außerdem kann mit `SEEK` die aktuelle Position innerhalb einer geöffneten Datei verändert werden.

Diese Funktionen werden wichtig, wenn ein BASIC-Programm Dateien nicht nur organisieren, sondern deren Inhalte selbst lesen oder schreiben soll.

# 41.24. Tipps aus der Praxis

- Verwende für jedes Projekt ein eigenes Verzeichnis.
- Prüfe mit `MM.INFO(drive)`, welches Laufwerk gerade aktiv ist.
- Verwende `FILES`, bevor du Dateien umbenennst oder löschst.
- Kontrolliere bei Bedarf mit `CWD$`, in welchem Verzeichnis du dich befindest.
- Lege wiederverwendbare Dateien getrennt von deinen Projekten ab.
- Sichere wichtige Projekte regelmäßig auf deinem PC.
- Verwende möglichst keine Leerzeichen in Dateinamen.
- Halte Verzeichnisnamen möglichst kurz und eindeutig.
- Verwende aussagekräftige Dateinamen.
- Erstelle vor größeren Änderungen Sicherheitskopien.
- Lösche Dateien erst, wenn du sicher bist, dass sie nicht mehr benötigt werden.
- Halte die Verzeichnisstruktur möglichst einfach.

## 41.25. Probier’s selbst!

Versuche folgende Aufgaben direkt an deinem PicoMite:

1. Wechsle mit `DRIVE "B:"` auf die SD-Karte.
2. Zeige mit `FILES` den Inhalt des aktuellen Verzeichnisses an.
3. Erstelle ein Verzeichnis `SPIELE`.
4. Wechsle in dieses Verzeichnis.
5. Kontrolliere mit `CWD$` das aktuelle Arbeitsverzeichnis.
6. Speichere dort ein kleines Testprogramm.
7. Zeige mit `FILES "*.BAS"` nur die BASIC-Dateien an.
8. Erstelle mit `COPY` eine Sicherheitskopie des Programms.
9. Benenne die Kopie mit `RENAME` um.
10. Prüfe mit `MM.INFO(exists file …)`, ob die Datei vorhanden ist.
11. Lösche die Kopie wieder mit `KILL`.
12. Wechsle mit `CHDIR "\"` zurück ins Wurzelverzeichnis.

## 41.26. Zusammenfassung

In diesem Kapitel hast du gelernt:

- mit `DRIVE` zwischen Flash-Dateisystem und SD-Karte zu wechseln
- mit `FILES` Dateien und Verzeichnisse anzuzeigen
- mit `LIST` den Inhalt einer Programm- oder Textdatei anzuzeigen
- Verzeichnisse mit `MKDIR` anzulegen
- mit `CHDIR` zwischen Verzeichnissen zu wechseln
- mit `CWD$` das aktuelle Arbeitsverzeichnis zu ermitteln
- Dateien mit `COPY` zu kopieren
- Dateien mit `RENAME` umzubenennen oder zu verschieben
- Dateien mit `KILL` zu löschen
- Verzeichnisse mit `RMDIR` zu entfernen
- mit `MM.INFO()` Informationen über Laufwerke, Dateien und Verzeichnisse abzufragen

Damit stehen dir die wichtigsten Werkzeuge zur Verfügung, um deine MMBasic-Programme und Projekte übersichtlich auf Flash-Speicher und SD-Karte zu organisieren.

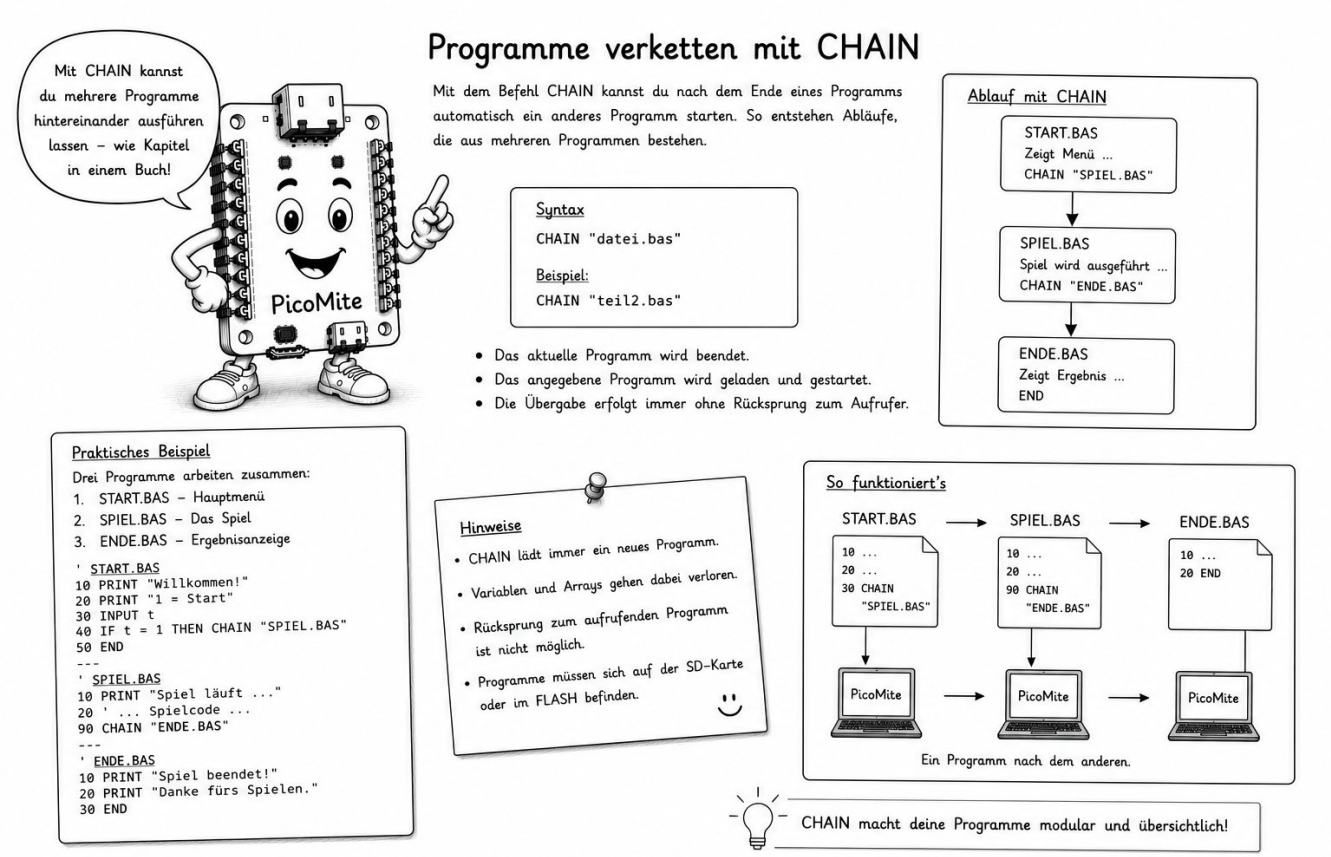
Im nächsten Kapitel lernst du mit `CHAIN` eine Möglichkeit kennen, mehrere Programme miteinander zu verbinden.

## 41.27. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 41_dateiverwaltung_demo.bas

# Kapitel 42. Programme verketten mit CHAIN




## 42.1. Das lernst du

In diesem Kapitel lernst du:

- was `CHAIN` macht,
- wie eine größere Anwendung auf mehrere BASIC-Programme aufgeteilt werden kann,
- warum die beteiligten Programme aufeinander abgestimmt sein müssen,
- wie Variablen beim Programmwechsel erhalten bleiben,
- welche Einschränkungen sich daraus ergeben,
- wie du mit `MM.CMDLINE$` zusätzliche Informationen an das nächste Programm übergibst,
- wie ein gemeinsames Menü innerhalb einer Anwendung aufgebaut werden kann und
- wann Daten zusätzlich in einer Datei gespeichert werden sollten.

Am Ende des Kapitels weißt du, wann `CHAIN` sinnvoll eingesetzt werden kann und worauf du achten musst, wenn mehrere BASIC-Programme gemeinsam eine Anwendung bilden.

## 42.2. Eine Anwendung aus mehreren Programmen

Bisher bestanden unsere Beispiele meistens aus einem einzelnen BASIC-Programm.

Bei größeren Projekten kann es jedoch sinnvoll sein, eine Anwendung bewusst auf mehrere zusammengehörende Programme aufzuteilen.

Eine Anwendung zur Verwaltung von Daten könnte beispielsweise aus folgenden Programmteilen bestehen:

```
DATENBANK
 │
 ├──── MENU.BAS
 ├──── EINGABE.BAS
 ├──── ANZEIGE.BAS
 ├──── SUCHE.BAS
 └──── STATISTIK.BAS
```

Alle diese Programme gehören zu derselben Anwendung. Sie werden gemeinsam geplant und verwenden teilweise dieselben Daten.

Das Menü könnte beispielsweise die Dateneingabe starten. Nach der Eingabe könnte wieder zum Menü zurückgekehrt oder ein anderer Programmteil aufgerufen werden.

Genau für solche Anwendungen kann der Befehl `CHAIN` verwendet werden.

Mit `CHAIN` kann ein laufendes BASIC-Programm ein anderes BASIC-Programm laden und sofort starten. Dabei bleibt der Variablenbereich erhalten.

Auf diese Weise kann eine größere Anwendung auf mehrere Programmdateien verteilt werden, ohne dass beim Wechsel zwischen den Programmteilen alle Daten verloren gehen.

## 42.3. Was macht CHAIN?

Die grundlegende Syntax lautet:

```
CHAIN fname$ [cmdline$]
```

`fname$` bezeichnet das BASIC-Programm, das geladen und gestartet werden soll.

Optional kann zusätzlich mit `cmdline$` eine Zeichenkette an das neue Programm übergeben werden.

Ein einfaches Beispiel:

```
CHAIN "ANZEIGE.BAS"
```

Das aktuell laufende Programm wird durch `ANZEIGE.BAS` ersetzt und dieses sofort gestartet.

Der wichtige Unterschied zu einem vollständigen Neustart ist: Der vorhandene Variablenbereich bleibt erhalten.

Beim Wechsel mit CHAIN bleiben die Variablen des bisherigen Programms erhalten.

Das ist einerseits der besondere Vorteil von CHAIN, führt andererseits aber auch zu wichtigen Einschränkungen, die bei der Entwicklung berücksichtigt werden müssen.

## 42.4. CHAIN erfordert gemeinsam geplante Programme

CHAIN ist nicht einfach ein Ersatz für RUN, mit dem beliebige BASIC-Programme zu einer Programmsammlung verbunden werden können.

Da beim Programmwechsel der Variablenbereich erhalten bleibt, müssen die beteiligten Programme aufeinander abgestimmt sein.

Programme, die mit CHAIN verbunden werden, sollten als Teile einer gemeinsamen Anwendung entwickelt werden.

Dabei müssen insbesondere gemeinsam verwendete Variablen in den beteiligten Programmen zueinander passen. Derselbe Variablenname darf beispielsweise nicht in einem Programm als numerische Variable und in einem anderen als Stringvariable verwendet werden.

Auch der Speicherbedarf der gesamten Anwendung muss berücksichtigt werden. Große Arrays, Framebuffer oder andere speicherintensive Daten können deshalb beim Entwurf einer Programmkette eine wichtige Rolle spielen.

CHAIN eignet sich deshalb besonders, wenn eine größere Anwendung bewusst in mehrere zusammengehörende Programmteile zerlegt wird.

Für eine Sammlung voneinander unabhängiger Programme ist CHAIN dagegen nicht automatisch die geeignete Lösung.

## 42.5. Variablen bleiben erhalten

Nehmen wir an, ein erstes Programm setzt einige Werte:

```
spieler$ = "Manfred"
punkte = 100

CHAIN "SPIEL.BAS"
```

Im Programm SPIEL.BAS können diese Variablen weiterverwendet werden:

```
PRINT "Spieler: "; spieler$
```

```
PRINT "Punkte: "; punkte
```

Die Ausgabe könnte so aussehen:

```
Spieler: Manfred
Punkte: 100
```

Damit eignet sich `CHAIN` dazu, eine größere Anwendung in mehrere Teile aufzuteilen und dabei gemeinsame Daten weiterzugeben.

Dabei müssen die Programme jedoch zueinander passen.

Angenommen, das erste Programm verwendet:

```
a% = 100
```

Ein anderes Programm derselben Programmkette darf `a%` nicht plötzlich als Stringvariable behandeln.

Die beteiligten Programme müssen deshalb gemeinsam geplant werden.

Bei größeren Programmkette lohnt es sich, gemeinsam verwendete Variablen bereits beim Entwurf der Anwendung festzulegen.

Notiere beispielsweise:

```
spieler$    Spielername
punkte%     Punktestand
level%      aktuelles Level
modus$      Spielmodus
```

Damit vermeidest du, dass dieselben Variablennamen in verschiedenen Programmteilen unterschiedlich verwendet werden.

# 42.6. Speicherbedarf beachten

Da `CHAIN` den Variablenbereich erhält, sollte auch der Speicherbedarf der gesamten Anwendung berücksichtigt werden.

Das wird besonders wichtig, wenn Programme größere Datenbereiche verwenden, beispielsweise:

- große Arrays,
- umfangreiche Zeichenketten,
- Grafikdaten oder
- Framebuffer.

Ein Programmteil sollte deshalb nicht unabhängig von den anderen Teilen entwickelt werden, wenn die Programme später über `CHAIN` zusammenarbeiten sollen.

`CHAIN` ist besonders dann sinnvoll, wenn die einzelnen Programmteile von Anfang an als Bestandteile derselben Anwendung geplant werden.

Bereits vorhandene, voneinander unabhängige Programme nachträglich mit `CHAIN` zu verbinden, kann dagegen zu Konflikten bei Variablen und Speicherbelegung führen.

## 42.7. Zusätzliche Parameter mit MM.CMDLINE$

Neben dem erhaltenen Variablenbereich kann `CHAIN` dem nächsten Programm noch eine zusätzliche Zeichenkette übergeben.

Beispiel:

```
CHAIN "SPIEL.BAS", "LEVEL=2"
```

Im gestarteten Programm steht diese Zeichenkette in `MM.CMDLINE$` zur Verfügung:

```
PRINT MM.CMDLINE$
```

Die Ausgabe lautet:

```
LEVEL=2
```

`MM.CMDLINE$` eignet sich beispielsweise für:

- Startoptionen,
- Dateinamen,
- einen gewünschten Spielmodus oder
- andere kurze Parameter, die das neue Programm beim Start auswerten soll.

Ein einfaches Beispiel wäre:

```
IF MM.CMDLINE$ = "SCHWER" THEN
  PRINT "Schwieriger Spielmodus"
ELSE
  PRINT "Normaler Spielmodus"
ENDIF
```

`MM.CMDLINE$` ist damit eine gute Möglichkeit, dem nächsten Programmteil gezielt mitzuteilen, warum oder in welchem Modus er gestartet wurde.

## 42.8. CHAIN oder RUN?

Auf den ersten Blick scheint `CHAIN` dem Befehl `RUN` ähnlich zu sein. Beide können ein BASIC-Programm starten, sie werden aber für unterschiedliche Zwecke eingesetzt.

`RUN` wird häufig an der MMBasic-Eingabeaufforderung verwendet, um ein Programm zu starten:

```
RUN "PONG.BAS"
```

`CHAIN` wird dagegen innerhalb eines laufenden Programms verwendet, wenn die Programmausführung mit einem anderen, zur selben Anwendung gehörenden BASIC-Programm fortgesetzt werden soll:

```
IF taste$ = "S" THEN
  CHAIN "SUCHE.BAS"
ENDIF
```

Der besondere Unterschied besteht darin, dass bei `CHAIN` der Variablenbereich beim Programmwechsel erhalten bleibt.

## 42.9. Ein gemeinsames Hauptmenü

Auch ein Menü kann Bestandteil einer mit `CHAIN` aufgebauten Anwendung sein.

Wichtig ist dabei, dass das Menü und die aufgerufenen Programme gemeinsam entwickelt wurden und zur selben Anwendung gehören.

Nehmen wir wieder unsere Datenbank:

```
DATENBANK
 │
 ├──── MENU.BAS
 ├──── EINGABE.BAS
 ├──── ANZEIGE.BAS
 ├──── SUCHE.BAS
 └──── STATISTIK.BAS
```

`MENU.BAS` könnte beispielsweise so aufgebaut sein:

```
CLS

PRINT "========================"
PRINT "       DATENBANK"
PRINT "========================"
PRINT
PRINT "1 - Daten eingeben"
```

```
PRINT "2 - Daten anzeigen"
PRINT "3 - Daten suchen"
PRINT "4 - Statistik"
PRINT
PRINT "0 - Ende"

INPUT "Auswahl"; wahl%

SELECT CASE wahl%

  CASE 1
    CHAIN "EINGABE.BAS", "MENU"

  CASE 2
    CHAIN "ANZEIGE.BAS", "MENU"

  CASE 3
    CHAIN "SUCHE.BAS", "MENU"

  CASE 4
    CHAIN "STATISTIK.BAS", "MENU"

  CASE 0
    END

  CASE ELSE
    PRINT "Ungültige Auswahl"

END SELECT
```

Wählt der Benutzer einen Menüpunkt aus, lädt `CHAIN` den entsprechenden Programmteil und startet ihn.

Zusätzlich erhält das gestartete Programm in diesem Beispiel über `MM.CMDLINE$` die Information, dass es aus dem Menü aufgerufen wurde.

## 42.10. Zurück zum Hauptmenü

Damit eine zusammenhängende Anwendung entsteht, können die aufgerufenen Programmteile wieder zum Hauptmenü zurückkehren.

Am Ende eines Programmteils könnte beispielsweise stehen:

```
PRINT
PRINT "Beliebige Taste für das Hauptmenü ..."

DO
LOOP UNTIL INKEY$ <> ""
```

```
CHAIN "MENU.BAS"
```

Nach dem Tastendruck wird das Hauptmenü wieder geladen und gestartet.

Da der Variablenbereich erhalten bleibt, können dabei gemeinsam verwendete Daten weiterhin zur Verfügung stehen.

## 42.11. CHAIN möglichst auf Programmebene verwenden

Das PicoMite-Handbuch empfiehlt, `CHAIN` auf der obersten Programmebene zu verwenden und möglichst nicht innerhalb einer Subroutine aufzurufen.

Verwende `CHAIN` nach Möglichkeit im Hauptablauf deines Programms und nicht aus einer `SUB` oder `FUNCTION` heraus.

Eine gute Struktur ist beispielsweise:

```
DO
  k$ = INKEY$
LOOP UNTIL k$ <> ""

IF k$ = "1" THEN CHAIN "EINGABE.BAS"
IF k$ = "2" THEN CHAIN "ANZEIGE.BAS"
```

So bleibt der Programmablauf übersichtlich und der Programmwechsel findet auf der obersten Ebene statt.

## 42.12. Wann ist CHAIN sinnvoll?

`CHAIN` kann sinnvoll sein, wenn:

- eine größere Anwendung bewusst auf mehrere BASIC-Programme verteilt wird,
- die beteiligten Programme gemeinsam entwickelt werden,
- Variablen zwischen den Programmteilen erhalten bleiben sollen,
- die Variablennamen und Datentypen aufeinander abgestimmt sind,
- der gemeinsame Speicherbedarf berücksichtigt wird und
- mit `MM.CMDLINE$` zusätzliche Startinformationen übergeben werden sollen.

Ein typisches Beispiel wäre eine größere Anwendung mit mehreren Funktionsbereichen:

```
ANWENDUNG
 │
 ├──── MENU.BAS
```

```
├──── EINGABE.BAS
├──── ANZEIGE.BAS
├──── SUCHE.BAS
└──── AUSWERTUNG.BAS
```

Alle Programmteile gehören dabei zu einem gemeinsamen Softwareprojekt.

Sollen mehrere Programme vor allem dieselben Unterprogramme oder Funktionen verwenden, ist häufig die MMBasic-Library die bessere Wahl.

## 42.13. Wann ist CHAIN weniger geeignet?

`CHAIN` ist weniger geeignet, wenn voneinander unabhängige Programme lediglich über ein gemeinsames Startmenü aufgerufen werden sollen.

Beispiele wären eine Sammlung verschiedener Spiele, Demonstrationsprogramme oder Werkzeuge, die unabhängig voneinander entwickelt wurden.

Solche Programme können:

- dieselben Variablennamen unterschiedlich verwenden,
- unterschiedliche Datentypen für gleichnamige Variablen besitzen,
- große eigene Datenbereiche anlegen oder
- unterschiedliche Annahmen über den verfügbaren Speicher treffen.

Eine nachträgliche Verkettung solcher Programme kann deshalb unnötige Abhängigkeiten und schwer nachvollziehbare Fehler verursachen.

`CHAIN` sollte nicht allein deshalb verwendet werden, weil mehrere Programme über ein gemeinsames Menü gestartet werden sollen.

Entscheidend ist, dass die Programme als Teile einer gemeinsamen Anwendung entwickelt wurden und ihre Variablen- und Speicherbelegung aufeinander abgestimmt ist.

## 42.14. Wann trotzdem eine Datei sinnvoll ist

Da `CHAIN` den Variablenbereich erhält, müssen Werte nicht allein wegen eines Programmwechsels in einer Datei gespeichert werden.

Eine Datei ist aber weiterhin sinnvoll, wenn Daten dauerhaft erhalten bleiben sollen, zum Beispiel:

- nach dem Ende der Programmkette,
- nach einem Neustart des PicoMite,
- nach dem Ausschalten oder
- wenn die Daten später von einem anderen Programm erneut geladen werden sollen.

### 42.14.1. Beispiel: Spielstand dauerhaft speichern

```
OPEN "PUNKTE.TXT" FOR OUTPUT AS #1
PRINT #1, punkte
CLOSE #1
```

Ein anderes Programm kann den gespeicherten Wert später wieder einlesen:

```
OPEN "PUNKTE.TXT" FOR INPUT AS #1
INPUT #1, punkte
CLOSE #1

PRINT "Punkte: "; punkte
```

Der Unterschied ist wichtig: `CHAIN` erhält Werte während der laufenden Programmkette. Eine Datei speichert sie dauerhaft auf dem Laufwerk.

## 42.15. CHAIN und Verzeichnisse

`CHAIN` kann auch mit Pfadangaben verwendet werden.

Beispiel:

```
CHAIN "A:/ANWENDUNG/ANZEIGE.BAS"
```

oder bei einer entsprechend eingerichteten SD-Karte:

```
CHAIN "B:/DATENBANK/SUCHE.BAS"
```

Dadurch lassen sich auch größere Anwendungen übersichtlich in verschiedenen Verzeichnissen organisieren.

Wie du Laufwerke und Verzeichnisse verwendest, hast du bereits im vorherigen Kapitel kennengelernt.

## 42.16. Historischer Hintergrund

Schon viele klassische BASIC-Systeme kannten Möglichkeiten, umfangreichere Anwendungen auf mehrere Programme aufzuteilen.

Auf Computern mit wenig Speicher war das besonders nützlich, weil nicht immer der gesamte Programmcode gleichzeitig im Speicher liegen musste.

MMBasic führt dieses Konzept mit `CHAIN` fort und erlaubt dabei, den Variablenbereich zwischen den Programmteilen zu erhalten.

# 42.17. Praxisprojekt: Eine kleine Programmkette

Zum Abschluss erstellen wir drei kleine Programme, mit denen du die wichtigsten Eigenschaften von `CHAIN` direkt ausprobieren kannst.

Alle drei Programme werden von Anfang an als Teile derselben Anwendung entwickelt.

Die Struktur sieht so aus:

```
42_chain_start.bas
        |
        v
42_chain_spiel.bas
        |
        v
42_chain_ende.bas
```

Dabei verwenden alle drei Programmteile dieselben gemeinsamen Variablen.

## 42.17.1. 42_chain_start.bas

```
CLS

spieler$ = "Manfred"
punkte = 100

PRINT "Startprogramm"
PRINT "Spieler: "; spieler$
PRINT "Punkte:  "; punkte
PRINT
PRINT "Weiter mit beliebiger Taste ..."

DO
LOOP UNTIL INKEY$ <> ""

CHAIN "42_chain_spiel.bas", "LEVEL=2"
```

Das Programm legt zwei Variablen an und startet anschließend `42_chain_spiel.bas`.

Zusätzlich wird mit `LEVEL=2` ein Parameter übergeben.

## 42.17.2. 42_chain_spiel.bas

```
CLS

PRINT "Spielprogramm"
PRINT "Spieler: "; spieler$
```

```
PRINT "Punkte:  "; punkte
PRINT "Parameter: "; MM.CMDLINE$
PRINT

punkte = punkte + 50

PRINT "50 Punkte gewonnen!"
PRINT "Neuer Punktestand: "; punkte
PRINT
PRINT "Weiter mit beliebiger Taste ..."

DO
LOOP UNTIL INKEY$ <> ""

CHAIN "42_chain_ende.bas"
```

Hier werden die Variablen des ersten Programms weiterverwendet. Der Punktestand wird verändert und bleibt auch beim nächsten `CHAIN` erhalten.

### 42.17.3. 42_chain_ende.bas

```
CLS

PRINT "Endprogramm"
PRINT
PRINT "Spieler: "; spieler$
PRINT "Endstand: "; punkte
PRINT
PRINT "Die Variablen wurden über die gesamte"
PRINT "Programmkette hinweg erhalten."
```

Wenn alles funktioniert, zeigt das letzte Programm weiterhin den Spielernamen und den inzwischen erhöhten Punktestand an.

Damit kannst du direkt beobachten, dass `CHAIN` den Variablenbereich nicht löscht, sondern an den nächsten Programmteil weitergibt.

## 42.18. Tipps aus der Praxis

- Verwende `CHAIN`, wenn eine Anwendung sinnvoll in mehrere zusammengehörende BASIC-Programme aufgeteilt werden kann.
- Plane die beteiligten Programme gemeinsam.
- Lege gemeinsam verwendete Variablennamen und Datentypen frühzeitig fest.
- Berücksichtige bei größeren Arrays, Grafikdaten oder Framebuffern den Speicherbedarf der gesamten Anwendung.
- Nutze den erhaltenen Variablenbereich, um Daten an den nächsten Programmteil

weiterzugeben.

- Verwende `MM.CMDLINE$`, wenn du dem nächsten Programm gezielt einen zusätzlichen Startparameter mitgeben möchtest.
- Verwende `CHAIN` möglichst auf der obersten Programmebene und nicht innerhalb einer Subroutine.
- Teste jeden Programmteil zunächst einzeln und anschließend im Zusammenspiel mit den anderen Teilen.
- Speichere Daten in einer Datei, wenn sie auch nach dem Ende der Programmkette oder nach einem Neustart erhalten bleiben sollen.
- Verwende `CHAIN` nicht automatisch als Startmechanismus für eine Sammlung voneinander unabhängiger Programme.

## 42.19. Probier's selbst!

1. Erstelle ein Startprogramm und ein zweites BASIC-Programm.
2. Lege im Startprogramm eine numerische Variable und eine Stringvariable an.
3. Starte das zweite Programm mit `CHAIN` und gib dort beide Variablen aus.
4. Übergib zusätzlich einen Textparameter und zeige ihn mit `MM.CMDLINE$` an.
5. Verändere im zweiten Programm einen Wert und starte mit `CHAIN` ein drittes Programm.
6. Prüfe, ob der geänderte Wert dort weiterhin vorhanden ist.
7. Überlege anschließend, welche Variablen alle drei Programme gemeinsam verwenden.
8. Erweitere die Programmkette um ein Menü, das zu den einzelnen Teilen der Anwendung wechseln kann.

## 42.20. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Programme mit `CHAIN` zu starten,
- eine größere Anwendung auf mehrere BASIC-Programme aufzuteilen,
- dass der Variablenbereich beim Programmwechsel erhalten bleibt,
- dass die beteiligten Programme deshalb aufeinander abgestimmt werden müssen,
- gemeinsam verwendete Variablennamen und Datentypen einheitlich zu planen,
- den gemeinsamen Speicherbedarf zu berücksichtigen,
- zusätzliche Parameter über `MM.CMDLINE$` zu übergeben,
- `CHAIN` möglichst auf der obersten Programmebene einzusetzen und
- wann Daten zusätzlich dauerhaft in einer Datei gespeichert werden sollten.

`CHAIN` startet nicht einfach nur ein anderes Programm. Der Befehl ermöglicht es, eine gemeinsam entwickelte Anwendung auf mehrere BASIC-Dateien aufzuteilen und dabei den Variablenbereich

weiterzuführen.

Gerade dieser erhaltene Variablenbereich ist sowohl die Stärke als auch die wichtigste Einschränkung von `CHAIN`.

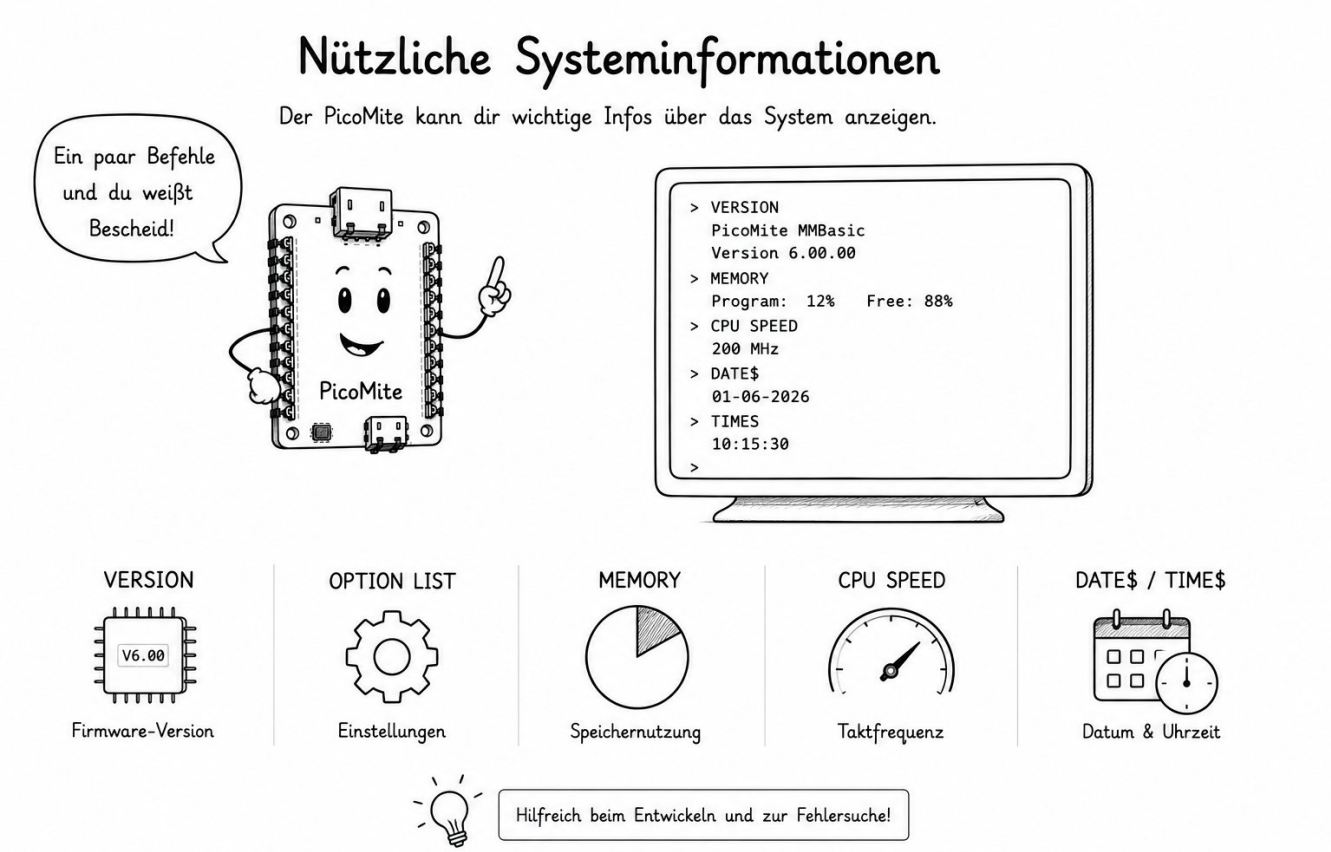
Deshalb eignet sich `CHAIN` besonders für Programmteile, die von Anfang an als Bestandteile derselben Anwendung geplant wurden. Für eine Sammlung unabhängig voneinander entwickelter Programme ist `CHAIN` dagegen nicht automatisch die richtige Wahl.

## 42.21. Programmbeispiel

Die vollständigen BASIC-Programme findest du hier:

- 42_chain_start.bas
- 42_chain_spiel.bas
- 42_chain_ende.bas

# Kapitel 43. Nützliche Systeminformationen




## 43.1. Das lernst du

In diesem Kapitel lernst du:

- wichtige Systeminformationen abzufragen
- Bildschirmgrößen automatisch zu erkennen
- Informationen über Speicher, Laufwerke und Dateien auszulesen
- die Funktionen `MM.INFO()` und `MM.INFO$()` zu verwenden
- Programme an unterschiedliche Hardware und Laufzeitbedingungen anzupassen

Am Ende des Kapitels kannst du Programme schreiben, die wichtige Eigenschaften ihrer Umgebung selbst ermitteln und darauf reagieren.

## 43.2. Warum Systeminformationen wichtig sind

In den bisherigen Kapiteln haben wir häufig feste Werte verwendet.

Beispielsweise:

```
CIRCLE 400, 300, 100
```

Das funktioniert zwar, setzt aber eine bestimmte Bildschirmauflösung voraus.

Eleganter ist es, Informationen direkt vom System abzufragen. Dadurch werden Programme flexibler und lassen sich leichter an unterschiedliche Geräte oder Einstellungen anpassen.

## 43.3. Bildschirmauflösung ermitteln

Eine der wichtigsten Informationen ist die aktuelle Bildschirmgröße.

MMBasic stellt dafür zwei schreibgeschützte Systemvariablen bereit:

```
PRINT MM.HRES
PRINT MM.VRES
```

Dabei bedeutet:

| **Variable** | **Bedeutung** |
|---|---|
| `MM.HRES` | Bildschirmbreite in Pixeln |
| `MM.VRES` | Bildschirmhöhe in Pixeln |

Je nach eingestelltem Grafikmodus können die Werte unterschiedlich sein.

Ein Programm kann dadurch automatisch auf die aktuelle Auflösung reagieren.

## 43.4. Objekte zentrieren

Anstatt feste Koordinaten zu verwenden:

```
CIRCLE 400, 300, 100
```

kann das Objekt automatisch zentriert werden:

```
CIRCLE MM.HRES / 2, MM.VRES / 2, 100
```

Dieses Verfahren haben wir bereits in mehreren Grafikprojekten verwendet.

## 43.5. Die aktuelle Position

Auch die aktuelle Ausgabe- beziehungsweise Grafikposition kann abgefragt werden.

Dafür gibt es:

```
PRINT MM.INFO(HPOS)
```

```
PRINT MM.INFO(VPOS)
```

`MM.INFO(HPOS)` liefert die aktuelle horizontale Position und `MM.INFO(VPOS)` die aktuelle vertikale Position in Pixeln. Die Werte beziehen sich auf die Position nach dem letzten Grafik- oder `PRINT` -Befehl.

# 43.6. Informationen mit MM.INFO()

Viele weitere Systeminformationen können mit `MM.INFO()` abgefragt werden.

Grundsätzlich gibt es zwei Schreibweisen:

```
MM.INFO(parameter)
MM.INFO$(parameter)
```

Beide Varianten können laut PicoMite-Handbuch grundsätzlich untereinander verwendet werden. Gute Programmierpraxis ist jedoch, die Variante passend zum zurückgegebenen Datentyp zu wählen:

- `MM.INFO()` für numerische Werte
- `MM.INFO$()` für Zeichenketten

Im weiteren Verlauf dieses Kapitels verwenden wir daher jeweils die passende Schreibweise.

# 43.7. Freien MMBasic-Speicher anzeigen

Ein häufig interessanter Wert ist der noch freie MMBasic-Heap.

```
PRINT MM.INFO(HEAP)
```

Der MMBasic-Heap wird unter anderem für Strings, Arrays und verschiedene interne Puffer verwendet. `MM.INFO(HEAP)` liefert den noch freien Speicher in Byte.

Daneben gibt es mit `MM.INFO(SYSTEM HEAP)` auch eine Abfrage für den freien System-Heap. Für normale BASIC-Programme ist jedoch meist `MM.INFO(HEAP)` die interessantere Information.

# 43.8. CPU-Geschwindigkeit anzeigen

Die aktuelle CPU-Geschwindigkeit kann mit `MM.INFO$(CPUSPEED)` abgefragt werden:

```
PRINT MM.INFO$(CPUSPEED)
```

Auf dem für dieses Tutorial verwendeten PicoMite HDMI/USB ergibt sich beispielsweise:

```
252000000
```

Das entspricht einer CPU-Geschwindigkeit von 252 MHz.

Obwohl die Ausgabe nur aus Ziffern besteht, liefert `MM.INFO$(CPUSPEED)` laut Handbuch eine Zeichenkette zurück.

## 43.9. Firmware-Version anzeigen

Mit `MM.INFO(VERSION)` lässt sich die installierte PicoMite-Firmwareversion ermitteln:

```
PRINT MM.INFO(VERSION)
```

Bei unserem Testsystem lautet die Ausgabe:

```
6.0301
```

Diese Information kann beispielsweise bei der Fehlersuche hilfreich sein, wenn sich Befehle oder Funktionen zwischen Firmwareversionen geändert haben.

## 43.10. Ursache des letzten Starts anzeigen

Interessant ist auch die Frage, warum der Pico zuletzt neu gestartet wurde. Dafür liefert `MM.INFO$(BOOT)` die Boot-Ursache.

```
PRINT MM.INFO$(BOOT)
```

Bei einem Neustart über den Reset-Taster erhielten wir:

```
Reset Switch
```

Je nach Ursache sind auch andere Rückgabewerte möglich, beispielsweise `Power On`, `Restart`, `S/W Watchdog`, `H/W Watchdog` oder `Firmware update`.

## 43.11. Laufzeit seit dem Start

Mit `MM.INFO(UPTIME)` kann die bisherige Laufzeit seit dem letzten Start abgefragt werden:

```
PRINT MM.INFO(UPTIME)
```

Der Wert wird in Sekunden als Fließkommazahl zurückgegeben.

Damit lässt sich beispielsweise feststellen, wie lange ein Programm oder ein PicoMite-System bereits läuft.

## 43.12. Aktives Laufwerk anzeigen

Das aktuell aktive Laufwerk wird mit `MM.INFO$(DRIVE)` ermittelt:

```
PRINT MM.INFO$(DRIVE)
```

Mögliche Rückgabewerte sind:

```
A:
```

oder:

```
B:
```

Wie wir in Kapitel 41 gesehen haben, steht `A:` beim PicoMite für das Flash-Dateisystem und `B:` für die SD-Karte, sofern diese eingerichtet ist.

## 43.13. Verfügbaren Speicherplatz anzeigen

Der freie Speicherplatz auf dem aktuell aktiven Laufwerk wird mit `MM.INFO(FREE SPACE)` ermittelt:

```
PRINT MM.INFO(FREE SPACE)
```

Der Rückgabewert wird in Byte angegeben.

## 43.14. Größe des Laufwerks anzeigen

Die Gesamtkapazität des aktiven Laufwerks liefert `MM.INFO(DISK SIZE)`:

```
PRINT MM.INFO(DISK SIZE)
```

Auch dieser Wert wird in Byte zurückgegeben.

Damit lassen sich beispielsweise freier und belegter Speicherplatz berechnen.

# 43.15. Existiert eine Datei?

Oft soll vor dem Öffnen einer Datei geprüft werden, ob sie tatsächlich vorhanden ist.

Dafür gibt es `MM.INFO(EXISTS FILE …)`:

```
IF MM.INFO(EXISTS FILE "PONG.BAS") = 1 THEN
  PRINT "Datei gefunden"
ELSE
  PRINT "Datei nicht gefunden"
ENDIF
```

Die Funktion liefert nicht nur wahr oder falsch, sondern unterscheidet drei Fälle:

| **Rückgabewert** | **Bedeutung** |
|---|---|
| `1` | Die Datei existiert. |
| `-1` | Der angegebene Name bezeichnet ein Verzeichnis. |
| `0` | Die Datei beziehungsweise das Verzeichnis wurde nicht gefunden. |

Deshalb prüfen wir im Beispiel ausdrücklich auf `= 1`.

# 43.16. Existiert ein Verzeichnis?

Auch Verzeichnisse können geprüft werden:

```
IF MM.INFO(EXISTS DIR "SPIELE") THEN
  PRINT "Verzeichnis vorhanden"
ENDIF
```

`MM.INFO(EXISTS DIR …)` liefert einen booleschen Wert zurück, der angibt, ob das Verzeichnis vorhanden ist.

# 43.17. Das aktuelle Verzeichnis

Das aktuelle Arbeitsverzeichnis lässt sich mit `CWD$` anzeigen:

```
PRINT CWD$
```

`CWD` steht für *Current Working Directory*.

Eine mögliche Ausgabe wäre beispielsweise:

```
/SPIELE
```

Das aktuelle Arbeitsverzeichnis ist besonders wichtig, wenn Programme mit relativen Dateinamen arbeiten.

## 43.18. Informationen über das aktuelle Programm

Der PicoMite kann auch Informationen über das aktuell geladene BASIC-Programm bereitstellen.

Mit `MM.INFO$(CURRENT)` wird der Name des aktuellen Programms abgefragt:

```
PRINT MM.INFO$(CURRENT)
```

`MM.INFO$(PATH)` liefert den Pfad, aus dem das Programm geladen wurde:

```
PRINT MM.INFO$(PATH)
```

Wurde kein Programm aus einer Datei geladen, können beide Funktionen `NONE` zurückgeben.

Bei unserem Test direkt nach einer Eingabe an der Eingabeaufforderung erhielten wir beispielsweise:

```
NONE
NONE
```

Diese Informationen können in größeren Projekten nützlich sein, wenn ein Programm wissen muss, aus welcher Datei beziehungsweise aus welchem Verzeichnis es gestartet wurde.

## 43.19. Beispiel: Systemübersicht

Mit den bisher kennengelernten Funktionen lässt sich bereits eine kleine Systemübersicht erstellen:

```
CLS

PRINT "Systeminformationen"
PRINT "-------------------"
PRINT
PRINT "Breite      : "; MM.HRES
PRINT "Hoehe       : "; MM.VRES
PRINT "CPU-Speed   : "; MM.INFO$(CPUSPEED)
PRINT "Version     : "; MM.INFO(VERSION)
PRINT "Boot        : "; MM.INFO$(BOOT)
PRINT "Uptime      : "; MM.INFO(UPTIME); " s"
PRINT "Heap frei   : "; MM.INFO(HEAP)
PRINT "Laufwerk    : "; MM.INFO$(DRIVE)
PRINT "Verzeichnis : "; CWD$
PRINT "Programm    : "; MM.INFO$(CURRENT)
```

```
PRINT "Pfad        : "; MM.INFO$(PATH)
```

Damit erhält man auf einen Blick einige der wichtigsten Informationen über das laufende PicoMite-System.

# 43.20. Programme automatisch anpassen

Mit Systeminformationen lassen sich Programme deutlich flexibler gestalten.

Ein einfaches Beispiel ist die Anpassung an die Bildschirmbreite:

```
IF MM.HRES >= 1024 THEN
  PRINT "Grosse Aufloesung"
ELSE
  PRINT "Kleine Aufloesung"
ENDIF
```

Dasselbe Prinzip kann für viele andere Eigenschaften verwendet werden.

Ein Programm könnte beispielsweise prüfen:

- welche Bildschirmauflösung aktiv ist
- welches Laufwerk verwendet wird
- ob eine benötigte Datei vorhanden ist
- wie viel Speicher noch frei ist
- welche Firmwareversion installiert ist

# 43.21. Warum schreibgeschützte Variablen?

Systemvariablen wie `MM.HRES` und `MM.VRES` werden von MMBasic bereitgestellt und enthalten Informationen über die aktuelle Umgebung.

Sie können vom BASIC-Programm gelesen, aber nicht verändert werden.

Folgender Befehl erzeugt deshalb einen Fehler:

```
MM.HRES = 800
```

Die Bildschirmauflösung wird nicht durch eine Zuweisung an `MM.HRES` geändert, sondern durch die dafür vorgesehenen MMBasic-Befehle und Optionen.

## 43.22. Nicht jede MM.INFO()-Abfrage gibt es auf jedem System

`MM.INFO()` enthält eine große Zahl unterschiedlicher Abfragen. Einige davon gelten nur für bestimmte PicoMite-Varianten oder bestimmte Hardware.

Beispiele sind Informationen über:

- WLAN beim WebMite
- Akku und Ladezustand beim PicoCalc
- PS/2-Schnittstellen
- Touch-Controller
- USB-Geräte
- PSRAM

Die verfügbaren `MM.INFO()`-Parameter können sich außerdem zwischen verschiedenen MMBasic-Plattformen unterscheiden. Die in diesem Kapitel beschriebenen Funktionen beziehen sich auf den PicoMite.

Bei anderen Systemen, beispielsweise dem Colour Maximite 2, sollte deshalb immer die jeweilige Dokumentation herangezogen werden.

## 43.23. Tipps aus der Praxis

- Verwende `MM.HRES` und `MM.VRES` statt unnötig vieler fester Bildschirmkoordinaten.
- Prüfe vor dem Öffnen wichtiger Dateien, ob sie existieren.
- Verwende `MM.INFO$()` für Rückgabewerte, die Zeichenketten sind.
- Nutze `CWD$`, `MM.INFO$(CURRENT)` und `MM.INFO$(PATH)`, wenn Programme mit mehreren Dateien arbeiten.
- Zeige bei der Fehlersuche Firmwareversion, Boot-Ursache und andere Systeminformationen an.
- Verwende nur die `MM.INFO()`-Parameter, die auf der jeweiligen PicoMite-Variante verfügbar sind.

## 43.24. Probier’s selbst!

Schreibe ein kleines Programm, das folgende Informationen ausgibt:

- Bildschirmbreite
- Bildschirmhöhe
- CPU-Geschwindigkeit
- Firmwareversion
- Boot-Ursache

- Laufzeit seit dem Start
- freien MMBasic-Heap
- aktuelles Laufwerk
- aktuelles Verzeichnis

Erweitere das Programm anschließend um eine Prüfung, ob eine bestimmte Datei vorhanden ist.

## 43.25. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Bildschirmgrößen mit `MM.HRES` und `MM.VRES` abzufragen
- numerische Systeminformationen mit `MM.INFO()` auszulesen
- Zeichenketten mit `MM.INFO$()` abzufragen
- den freien MMBasic-Heap mit `MM.INFO(HEAP)` zu ermitteln
- CPU-Geschwindigkeit, Firmwareversion, Boot-Ursache und Laufzeit auszulesen
- Dateien und Verzeichnisse auf ihre Existenz zu prüfen
- Laufwerk und aktuelles Verzeichnis zu ermitteln
- Informationen über das aktuell geladene Programm abzufragen
- Programme automatisch an ihre Umgebung anzupassen

Mit diesen Möglichkeiten können Programme ihre Laufzeitumgebung selbst untersuchen und dadurch robuster und flexibler reagieren.

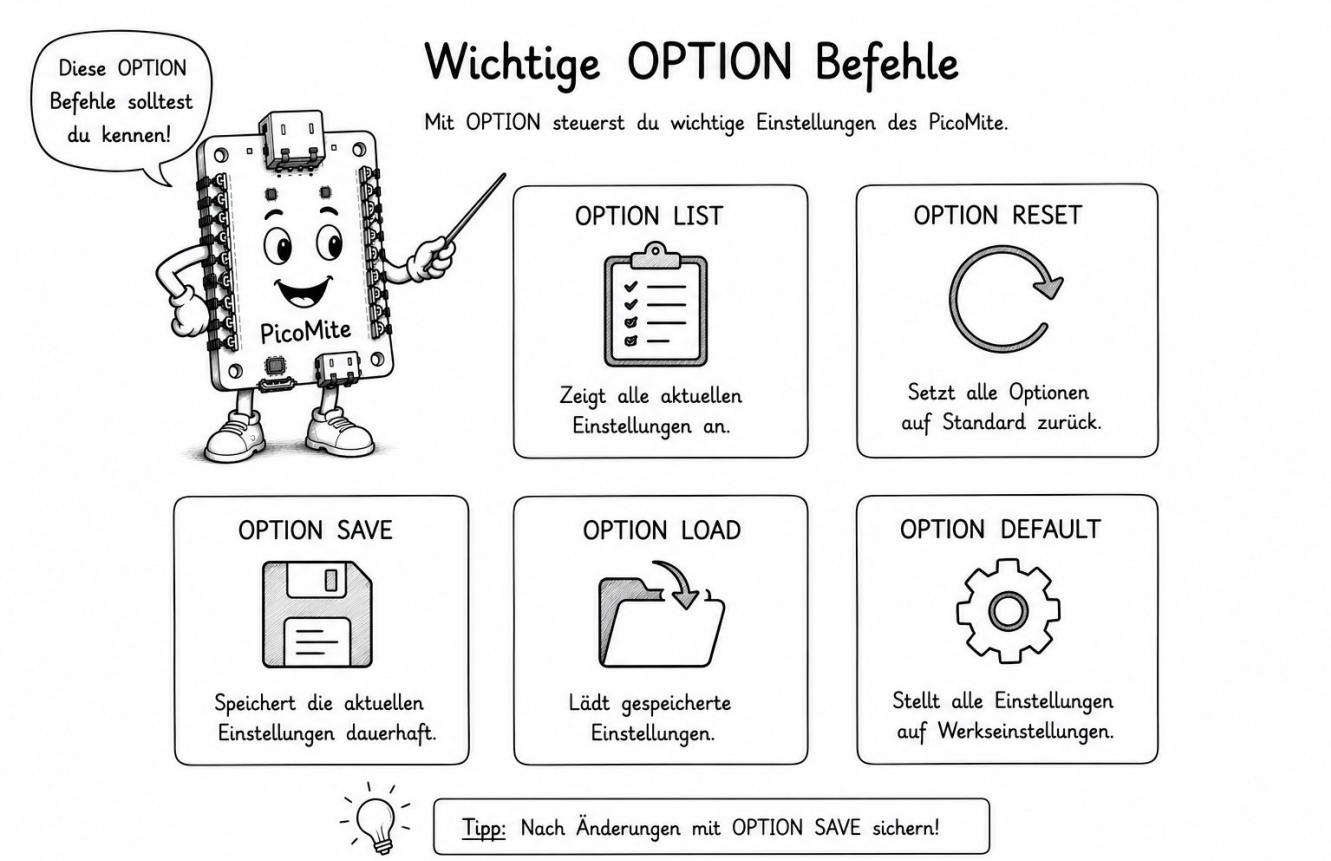
## 43.26. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 43_systembericht.bas

# Teil VII – Fortgeschrittene Techniken

# Kapitel 44. Wichtige OPTION-Befehle




## 44.1. Das lernst du

In diesem Kapitel lernst du:

- was `OPTION`-Befehle sind
- den Unterschied zwischen temporären und permanenten Optionen
- warum manche OPTION-Einstellungen vor dem Start eines Programms vorgenommen werden müssen
- wichtige `OPTION`-Befehle für Programme und die Systemkonfiguration einzusetzen
- Bildschirmauflösung und CPU-Takt zu konfigurieren
- Tastaturlayout und Tastenwiederholung einzustellen
- die aktuelle Konfiguration mit `OPTION LIST` zu untersuchen
- typische Fehler beim Umgang mit Optionen zu vermeiden

Am Ende des Kapitels kennst du einige der wichtigsten Möglichkeiten, das Verhalten von MMBasic an deine Bedürfnisse anzupassen.

## 44.2. Was sind OPTION-Befehle?

Bisher haben wir überwiegend Befehle kennengelernt, die unmittelbar etwas ausführen:

```
PRINT "Hallo Welt"
CLS
LINE 0,0,100,100
```

`OPTION`-Befehle funktionieren etwas anders. Sie verändern Einstellungen von MMBasic und können sowohl das Verhalten eines BASIC-Programms als auch die Konfiguration des PicoMite beeinflussen.

Dabei ist eine wichtige Unterscheidung zu beachten:

- **Temporäre bzw. programmbezogene Optionen** gelten nur vorübergehend und werden beispielsweise beim Start eines Programms oder beim Neustart wieder auf ihre Standardwerte gesetzt.
- **Permanente Optionen** werden im nichtflüchtigen Speicher abgelegt und nach einem Neustart automatisch wiederhergestellt.

Viele permanente `OPTION`-Befehle können einen Neustart der PicoMite-Firmware auslösen. Das im Programmspeicher befindliche BASIC-Programm geht dabei nicht verloren.

## 44.3. Warum müssen manche OPTION-Einstellungen vorher vorgenommen werden?

Vielleicht hast du dich schon gefragt, warum bestimmte OPTION-Einstellungen nicht einfach am Anfang eines BASIC-Programms stehen können.

Der Grund liegt im Unterschied zwischen programmbezogenen Optionen und den Grundeinstellungen der MMBasic-Umgebung.

Programmbezogene Optionen beeinflussen nur die Ausführung eines BASIC-Programms. Sie können deshalb direkt im Programm stehen. Beispiele dafür sind OPTION EXPLICIT, OPTION DEFAULT und OPTION BASE.

Andere OPTION-Befehle legen dagegen Grundeinstellungen der MMBasic-Umgebung oder der verwendeten Hardware fest. Damit wird beispielsweise bestimmt, welche Bildschirmkonfiguration verwendet wird, welche Pins für bestimmte Funktionen reserviert sind oder wie bestimmte Hardware-Komponenten genutzt werden.

Viele dieser Einstellungen werden dauerhaft gespeichert und beim nächsten Start von MMBasic wieder verwendet. Manche beeinflussen Ressourcen oder Hardware-Funktionen, die MMBasic bereits beim Start einrichtet. Eine Änderung kann deshalb einen Neustart erforderlich machen.

Eine einfache Unterscheidung hilft dabei:

Programmbezogene OPTION-Befehle legen fest, wie dein BASIC-Programm ausgeführt wird. Permanente OPTION-Befehle legen Grundeinstellungen fest, in welcher Umgebung dein BASIC-Programm ausgeführt wird. Deshalb können programmbezogene Optionen im BASIC-Programm stehen, während grundlegende System- und Hardwareoptionen normalerweise vorher an der

MMBasic-Eingabeaufforderung eingestellt werden.

## 44.4. Aktuelle Einstellungen anzeigen

Mit `OPTION LIST`:

```
OPTION LIST
```

kannst du die permanenten Optionen anzeigen, die gegenüber ihren Standardwerten geändert wurden.

Zusätzlich zeigt `OPTION LIST` die Versionsnummer und die geladene Firmware an.

`OPTION LIST` muss an der MMBasic-Eingabeaufforderung ausgeführt werden. Der Befehl kann nicht innerhalb eines BASIC-Programms verwendet werden.

## 44.5. Programmbezogene Optionen

Einige `OPTION`-Befehle beeinflussen unmittelbar die Ausführung eines BASIC-Programms. Dazu gehören beispielsweise `OPTION EXPLICIT`, `OPTION DEFAULT` und `OPTION BASE`.

Diese Einstellungen sind nicht dauerhaft gespeichert.

## 44.6. OPTION EXPLICIT

Eine besonders nützliche Option für Programmierer ist:

```
OPTION EXPLICIT
```

Ist diese Option aktiv, müssen alle Variablen vor ihrer Verwendung ausdrücklich mit `DIM`, `LOCAL` oder `STATIC` deklariert werden.

Beispiel:

```
OPTION EXPLICIT

DIM punkte

punkte = 100
```

Wird versehentlich eine andere Variable verwendet:

```
punkt = 100
```

meldet MMBasic einen Fehler, weil `punkt` nicht deklariert wurde.

Ohne `OPTION EXPLICIT` würde MMBasic automatisch eine neue Variable anlegen.

`OPTION EXPLICIT` muss vor der ersten Verwendung einer Variablen stehen. Beim Start eines Programms ist die Option standardmäßig deaktiviert und muss deshalb bei Bedarf im Programm selbst aktiviert werden.

## 44.7. Warum OPTION EXPLICIT sinnvoll ist

Betrachten wir folgendes Beispiel:

```
punkte = punkte + 10
PRINT punkte
```

Vertippt man sich:

```
punkte = punkt + 10
```

kann ohne `OPTION EXPLICIT` unbemerkt eine neue Variable mit dem Namen `punkt` entstehen.

Solche Fehler sind oft schwer zu finden. `OPTION EXPLICIT` hilft dabei, sie frühzeitig zu erkennen.

## 44.8. OPTION DEFAULT

Mit `OPTION DEFAULT` wird festgelegt, welchen Datentyp eine Variable erhält, wenn kein Typ ausdrücklich angegeben wurde.

Mögliche Einstellungen sind verschiedene Standard-Datentypen:

```
OPTION DEFAULT FLOAT
OPTION DEFAULT INTEGER
OPTION DEFAULT STRING
OPTION DEFAULT NONE
```

Beim Start eines Programms wird die Einstellung standardmäßig auf `FLOAT` gesetzt.

## 44.9. OPTION DEFAULT FLOAT

```
OPTION DEFAULT FLOAT
```

Variablen ohne ausdrücklich angegebenen Typ werden als Fließkommazahlen behandelt.

Dies ist die Standardeinstellung von MMBasic und dient der Kompatibilität mit Microsoft BASIC

und früheren MMBasic-Versionen.

## 44.10. OPTION DEFAULT INTEGER

```
OPTION DEFAULT INTEGER
```

Variablen ohne ausdrücklich angegebenen Typ werden als Ganzzahlen behandelt.

## 44.11. OPTION DEFAULT STRING

```
OPTION DEFAULT STRING
```

Variablen ohne ausdrücklich angegebenen Typ werden als Zeichenketten behandelt.

## 44.12. OPTION DEFAULT NONE

Die strengste Variante ist:

```
OPTION DEFAULT NONE
```

Nun muss für jede Variable ein Datentyp ausdrücklich angegeben werden. Andernfalls meldet MMBasic:

```
Variable type not specified
```

Beispiel:

```
DIM name$
DIM punkte%
DIM temperatur!
```

Die Typzeichen `$`, `%` und `!` legen dabei den jeweiligen Datentyp fest.

## 44.13. OPTION EXPLICIT und OPTION DEFAULT NONE

Die beiden Optionen prüfen unterschiedliche Dinge:

- `OPTION EXPLICIT` verlangt, dass Variablen deklariert werden.
- `OPTION DEFAULT NONE` verlangt, dass ihr Datentyp ausdrücklich festgelegt wird.

Beide Optionen können miteinander kombiniert werden:

```
OPTION EXPLICIT
OPTION DEFAULT NONE

DIM punkte%
DIM temperatur!
DIM name$
```

Dadurch können viele Schreib- und Typfehler bereits während der Entwicklung erkannt werden.

## 44.14. OPTION BASE

Mit `OPTION BASE` wird festgelegt, bei welchem Index Arrays beginnen.

Standard ist:

```
OPTION BASE 0
```

Beispiel:

```
DIM zahlen(10)
```

Die Elemente reichen dann von:

```
zahlen(0)
bis
zahlen(10)
```

Alternativ kann festgelegt werden:

```
OPTION BASE 1
```

Dann beginnt das Array bei `zahlen(1)`.

`OPTION BASE` muss verwendet werden, bevor das erste Array deklariert wird. Nach dem Einschalten des PicoMite ist der Standardwert wieder `0`.

In diesem Tutorial verwenden wir grundsätzlich `OPTION BASE 0`.

## 44.15. Permanente Optionen

Andere `OPTION`-Befehle verändern Einstellungen der MMBasic-Arbeitsumgebung oder der Hardwarekonfiguration und werden dauerhaft gespeichert.

Dazu gehören beispielsweise `OPTION TAB` und `OPTION COLOURCODE`. Auch `OPTION CPUSPEED` gehört bei den PicoMite-Versionen, die diesen Befehl unterstützen, zu den permanenten Optionen.

# 44.16. OPTION TAB

Diese Option legt den Abstand der Tabulatortaste fest.

Mögliche Werte sind:

```
OPTION TAB 2
OPTION TAB 3
OPTION TAB 4
OPTION TAB 8
```

Beispielsweise stellt:

```
OPTION TAB 4
```

den Tabulatorabstand auf vier Zeichenpositionen ein.

Der Standardwert ist `2`. Die Einstellung wird dauerhaft gespeichert.

# 44.17. OPTION COLOURCODE

Der eingebaute Editor kann BASIC-Programme farbig darstellen.

Aktivieren:

```
OPTION COLOURCODE ON
```

Deaktivieren:

```
OPTION COLOURCODE OFF
```

Schlüsselwörter werden beispielsweise in Cyan und Kommentare in Gelb dargestellt. Die Standardeinstellung ist `OFF`.

Alternativ akzeptiert MMBasic auch die amerikanische Schreibweise `OPTION COLORCODE`.

Die Farbdarstellung funktioniert bei VGA- und HDMI-Ausgabe sowie über die serielle Konsole mit einem VT100-kompatiblen Terminalprogramm.

`OPTION COLOURCODE` ist eine permanente Option und muss an der MMBasic-Eingabeaufforderung ausgeführt werden. Innerhalb eines BASIC-Programms ist der Befehl nicht zulässig.

# 44.18. CPU-Takt bei VGA und HDMI

Bei den VGA- und HDMI-Versionen wird die CPU-Taktfrequenz nicht mit `OPTION CPUSPEED` eingestellt. Hier hängt sie mit der verwendeten Videoauflösung zusammen.

Dafür gibt es `OPTION RESOLUTION`:

```
OPTION RESOLUTION nn [,cpuspeedinKhz]
```

Bei einer Auflösung von 640 × 480 nennt das PicoMite-Handbuch beispielsweise CPU-Taktfrequenzen von 252 MHz, 315 MHz oder 378 MHz, abhängig von der gewünschten Bildwiederholfrequenz.

Das erklärt auch, warum beispielsweise:

```
PRINT MM.INFO$(CPUSPEED)
```

auf einem PicoMite HDMI/USB einen Wert wie:

```
252000000
```

liefern kann. `MM.INFO$(CPUSPEED)` gibt die CPU-Taktfrequenz dabei in Hz zurück.

## 44.18.1. Bildwiederholrate und Monitor-Kompatibilität

Die mit `OPTION RESOLUTION` gewählte CPU-Taktfrequenz beeinflusst bei der Videoausgabe auch die Bildwiederholrate.

Das kann in der Praxis eine wichtige Rolle spielen. Beim Test des Pico Computer 3 lieferte beispielsweise folgende Einstellung an einem Monitor zunächst kein Bild:

```
OPTION RESOLUTION 640,315000
```

Ein anderer Monitor konnte diese Einstellung dagegen problemlos darstellen. Mit dieser Einstellung wird eine Bildwiederholrate von 75 Hz verwendet.

Durch die Umstellung auf:

```
OPTION RESOLUTION 640,252000
```

wurde die Bildwiederholrate auf 60 Hz reduziert. Damit funktionierte die Ausgabe auch am zuvor problematischen Monitor.

Dieses Beispiel darf jedoch nicht als allgemeine Empfehlung für 60 Hz verstanden werden. Die

tatsächlich unterstützten Bildwiederholraten hängen vom verwendeten Monitor oder Fernsehgerät ab.

Auch wenn 640 × 480 mit 60 Hz traditionell eine sehr weit verbreitete Bildschirmauflösung ist, unterstützen nicht alle modernen Displays diese Betriebsart. Es gibt Geräte, die 640 × 480 mit 60 Hz nicht darstellen, während 640 × 480 mit 75 Hz problemlos funktioniert.

Umgekehrt gibt es, wie der oben beschriebene Test zeigt, auch Monitore, bei denen 60 Hz funktionieren und 75 Hz nicht.

Wenn ein PicoMite mit VGA- oder HDMI-Ausgabe an einem Monitor kein Bild liefert, muss deshalb nicht unbedingt ein Hardwarefehler vorliegen.

Prüfe auch die mit `OPTION RESOLUTION` eingestellte Auflösung und Bildwiederholrate.

Bei 640 × 480 kann es sich lohnen, sowohl die Einstellung mit 60 Hz als auch die mit 75 Hz auszuprobieren:

```
OPTION RESOLUTION 640,252000
```

für 60 Hz beziehungsweise:

```
OPTION RESOLUTION 640,315000
```

für 75 Hz.

Welche Einstellung besser funktioniert, hängt vom verwendeten Display ab.

# 44.19. Tastatur konfigurieren

MMBasic geht standardmäßig von einer Tastatur mit US-amerikanischem Layout aus.

Mit `OPTION KEYBOARD` kann ein anderes Tastaturlayout eingestellt werden.

Die grundlegende Syntax lautet:

```
OPTION KEYBOARD language
```

Für eine deutsche Tastatur verwenden wir:

```
OPTION KEYBOARD GR
```

`GR` steht dabei für das deutsche Tastaturlayout.

Weitere unterstützte Tastaturlayouts sind beispielsweise `US`, `UK`, `FR`, `BE`, `IT`, `BR` und `ES`.

Bei nicht-amerikanischen Tastaturlayouts können zwar zusätzliche Sondertasten der jeweiligen Tastatur berücksichtigt werden. Nicht alle zugehörigen Sonderzeichen sind jedoch in den Standardfonts des PicoMite enthalten. In diesem Fall wird stattdessen ein entsprechendes ASCII-Zeichen dargestellt.

### 44.19.1. Caps Lock, Num Lock und Tastenwiederholung

Bei der Konfiguration der Tastatur können weitere Parameter angegeben werden:

```
OPTION KEYBOARD nn [,capslock] [,numlock] [,repeatstart] [,repeatrate]
```

`capslock` und `numlock` bestimmen den anfänglichen Zustand der beiden Tastaturfunktionen.

Die Standardwerte sind:

| Parameter | Standard | Bedeutung |
|---|---|---|
| `capslock` | `0` | Caps Lock ausgeschaltet |
| `numlock` | `1` | Num Lock eingeschaltet |

Die beiden weiteren Parameter bestimmen die Tastenwiederholung.

`repeatstart` legt fest, wie lange eine Taste gedrückt gehalten werden muss, bevor die automatische Wiederholung beginnt.

`repeatrate` bestimmt anschließend den zeitlichen Abstand zwischen den einzelnen Wiederholungen.

Bei einer USB-Tastatur werden beide Zeiten in Millisekunden angegeben:

| Parameter | Bereich | Bedeutung |
|---|---|---|
| `repeatstart` | 100–2000 ms | Verzögerung bis zur ersten Wiederholung |
| `repeatrate` | 25–2000 ms | Abstand zwischen den weiteren Wiederholungen |

Eine mögliche Einstellung für eine deutsche USB-Tastatur wäre beispielsweise:

```
OPTION KEYBOARD GR,0,1,250,50
```

Damit verwenden wir das deutsche Tastaturlayout, Caps Lock ist zunächst ausgeschaltet und Num Lock eingeschaltet. Eine gehaltene Taste beginnt nach 250 ms mit der Wiederholung und wird anschließend alle 50 ms erneut gemeldet.

Bei einer PS/2-Tastatur werden für `repeatstart` und `repeatrate` keine Millisekundenwerte angegeben. Stattdessen gelten die in der PS/2-Spezifikation

definierten Einstellwerte.

Der Standardwert für `repeatstart` ist 1, entsprechend 500 ms. Für `repeatrate` ist der Standardwert 12, entsprechend 100 ms.

### 44.19.2. Wiederholrate einer USB-Tastatur ändern

Bei einer bereits konfigurierten USB-Tastatur kann die Tastenwiederholung auch separat geändert werden.

Dafür gibt es `OPTION KEYBOARD REPEAT`:

```
OPTION KEYBOARD REPEAT repeatstart,repeatrate
```

Beispielsweise:

```
OPTION KEYBOARD REPEAT 250,50
```

Die Wiederholung beginnt nach 250 ms und erfolgt anschließend alle 50 ms.

Je kleiner `repeatrate` gewählt wird, desto schneller werden die Tastenwiederholungen erzeugt. Der kleinste zulässige Wert von 25 ms ergibt bei einer USB-Tastatur somit die höchste Wiederholrate.

Wenn sich die Tastatur bei der Eingabe zu träge anfühlt, kann eine kürzere Startverzögerung und insbesondere ein kleinerer Wert für `repeatrate` die Bedienung deutlich angenehmer machen.

Für eine kontinuierliche Tastatursteuerung in Spielen ist eine hohe Tastenwiederholrate trotzdem nicht die beste Lösung.

Dafür eignet sich `KEYDOWN()` besser. Damit kann ein Programm direkt feststellen, ob eine Taste gerade gedrückt gehalten wird. Die Steuerung ist dann unabhängig von der eingestellten Tastenwiederholrate.

Dieses Verfahren verwenden wir beispielsweise bei der Steuerung unseres Pong-Spiels.

`OPTION KEYBOARD` muss an der MMBasic-Eingabeaufforderung eingegeben werden und führt zu einem Neustart.

Die Tastaturkonfiguration kann wieder deaktiviert werden mit:

```
OPTION KEYBOARD DISABLE
```

# 44.20. Was tun, wenn kein Bild mehr erscheint?

Da viele `OPTION`-Einstellungen dauerhaft gespeichert werden, kann eine ungeeignete Einstellung auch nach einem Neustart erhalten bleiben.

Besonders bei VGA- und HDMI-Versionen kann es passieren, dass nach einer Änderung der Auflösung, Bildwiederholrate oder des Grafikmodus kein Bild mehr auf dem Monitor erscheint.

Ein schwarzer Bildschirm bedeutet nicht automatisch, dass der PicoMite nicht mehr funktioniert.

Oft läuft MMBasic weiterhin und lediglich die Videoausgabe ist so konfiguriert, dass der angeschlossene Monitor sie nicht darstellen kann.

## 44.20.1. Einen anderen Monitor ausprobieren

Wenn die Ursache eine nicht unterstützte Auflösung oder Bildwiederholrate ist, kann bereits ein anderer Monitor helfen.

So war es beispielsweise beim Pico Computer 3: Ein Monitor konnte die mit

```
OPTION RESOLUTION 640,315000
```

erzeugte Ausgabe mit 75 Hz darstellen, ein anderer dagegen nicht.

Über den funktionierenden Monitor konnte die Einstellung anschließend auf 60 Hz geändert werden:

```
OPTION RESOLUTION 640,252000
```

In diesem konkreten Fall war 60 Hz die kompatiblere Einstellung. Bei einem anderen Monitor oder Fernsehgerät kann es jedoch auch umgekehrt sein.

Falls bei 640 × 480 kein Bild erscheint, solltest du deshalb gegebenenfalls beide Bildwiederholraten ausprobieren:

```
OPTION RESOLUTION 640,252000
```

für 60 Hz und:

```
OPTION RESOLUTION 640,315000
```

für 75 Hz.

Gerade bei unterschiedlichen Generationen von Monitoren und Fernsehgeräten kann sich die

Unterstützung dieser Bildschirmmodi unterscheiden.

### 44.20.2. Die Konsole als Rettungsweg

Wenn über VGA oder HDMI keine brauchbare Ausgabe mehr erscheint, kann die MMBasic-Konsole einen zweiten Zugangsweg bieten.

Ist eine serielle Konsole konfiguriert und erreichbar, können dort die gespeicherten Einstellungen zunächst überprüft werden:

```
OPTION LIST
```

Anschließend kann die fehlerhafte Einstellung korrigiert werden.

Gerade beim Experimentieren mit permanenten `OPTION`-Einstellungen ist eine funktionierende Konsolenverbindung ein nützlicher Rettungsweg. Damit lässt sich der PicoMite häufig noch bedienen, obwohl die Bildschirmausgabe nicht mehr funktioniert.

### 44.20.3. Konfiguration zurücksetzen

Sind mehrere Einstellungen verstellt oder ist die Ursache nicht mehr bekannt, kann die Konfiguration zurückgesetzt werden.

Mit:

```
OPTION RESET LIST
```

werden zunächst die für die verwendete Firmware verfügbaren vordefinierten Konfigurationen angezeigt.

Anschließend kann eine passende Konfiguration geladen werden. Welche Namen zur Verfügung stehen, hängt von der verwendeten Firmware ab.

Bevor du `OPTION RESET` verwendest, solltest du wichtige Dateien des internen Flashlaufwerks `A:` sichern.

Je nach verwendeter Firmware und Reset-Konfiguration kann ein `OPTION RESET` nicht nur die gespeicherten Optionen zurücksetzen, sondern auch das Flashlaufwerk `A:` löschen oder neu initialisieren.

Kopiere deshalb eigene BASIC-Programme, Konfigurationsdateien und andere wichtige Daten vorher beispielsweise auf eine SD-Karte oder auf deinen Computer.

Prüfe außerdem nach Möglichkeit zunächst mit `OPTION LIST` die aktuelle Konfiguration und notiere wichtige Einstellungen.

`OPTION RESET` sollte deshalb als Rettungsweg erst verwendet werden, wenn sich die fehlerhafte Einstellung nicht gezielt korrigieren lässt.

### 44.20.4. Wenn auch die Konsole nicht mehr erreichbar ist

Als letzter Rettungsweg kann die PicoMite-Firmware neu aufgespielt werden. In besonders hartnäckigen Fällen kann zuvor auch der Flash-Speicher des Pico vollständig gelöscht und die Firmware anschließend neu installiert werden.

Das ist jedoch normalerweise erst notwendig, wenn sich die Konfiguration nicht mehr über die MMBasic-Konsole korrigieren lässt.

## 44.21. OPTION RESET

Mit `OPTION RESET`:

```
OPTION RESET
```

werden alle **gespeicherten** Optionen auf ihre Standardwerte zurückgesetzt.

`OPTION RESET` löscht die zuvor dauerhaft gespeicherten Anpassungen der PicoMite-Konfiguration. Verwende den Befehl daher mit Bedacht.

`OPTION RESET` muss an der MMBasic-Eingabeaufforderung ausgeführt werden und kann nicht innerhalb eines Programms verwendet werden.

Temporäre programmbezogene Einstellungen wie `OPTION EXPLICIT`, `OPTION DEFAULT` oder `OPTION BASE` gehören nicht zu den dauerhaft gespeicherten Optionen, die mit `OPTION RESET` zurückgesetzt werden.

## 44.22. Vordefinierte Konfigurationen

Zusätzlich unterstützt MMBasic vordefinierte Konfigurationen.

Welche Konfigurationen die verwendete Firmware kennt, lässt sich mit `OPTION RESET LIST` anzeigen:

```
OPTION RESET LIST
```

anzeigen.

Anschließend kann eine der angezeigten Konfigurationen mit:

```
OPTION RESET <cfg>
```

aktiviert werden. Dabei wird `<cfg>` durch den tatsächlich von `OPTION RESET LIST` angezeigten Namen ersetzt.

Die verfügbaren Konfigurationsnamen können von der verwendeten PicoMite-Firmware abhängen. Deshalb solltest du zuerst `OPTION RESET LIST` verwenden, anstatt einen Konfigurationsnamen aus einem anderen System zu übernehmen.

Solche vordefinierten Konfigurationen sind beispielsweise hilfreich, wenn ein PicoMite nach dem Flashen einer Firmware für eine bestimmte Hardware eingerichtet werden soll.

# 44.23. Wann sollte man OPTION-Befehle verwenden?

Das hängt von der Art der Option ab.

Programmbezogene Optionen wie:

```
OPTION EXPLICIT
OPTION DEFAULT NONE
OPTION BASE 0
```

können bewusst am Anfang eines BASIC-Programms stehen und dessen Verhalten festlegen.

Permanente Optionen wie `OPTION TAB`, `OPTION COLOURCODE` oder verschiedene Hardwareoptionen gehören dagegen zur Konfiguration des Systems und werden normalerweise an der MMBasic-Eingabeaufforderung eingerichtet.

# 44.24. Tipps aus der Praxis

- Verwende `OPTION EXPLICIT`, um Tippfehler bei Variablennamen frühzeitig zu erkennen.
- Mit `OPTION DEFAULT NONE` kannst du zusätzlich eine ausdrückliche Typangabe erzwingen.
- Setze `OPTION BASE` immer vor der ersten Array-Deklaration.
- Verwende `OPTION CPUSPEED` nur auf PicoMite-Versionen, die diesen Befehl unterstützen.
- Dokumentiere Änderungen an permanenten Systemoptionen.
- Mit `OPTION LIST` kannst du geänderte permanente Einstellungen kontrollieren.
- Verwende vor `OPTION RESET <cfg>` zunächst `OPTION RESET LIST`.

# 44.25. Probier’s selbst!

1. Gib an der MMBasic-Eingabeaufforderung `OPTION LIST` ein.
2. Schreibe ein kleines Programm mit `OPTION EXPLICIT` und einer absichtlich falsch geschriebenen Variablen.
3. Teste anschließend `OPTION DEFAULT NONE` und lasse bei einer Variablen die Typangabe weg.
4. Probiere `OPTION BASE 0` mit einem kleinen Array aus.

5. Falls dein System einen Bildschirm oder ein geeignetes Terminal verwendet, probiere `OPTION COLOURCODE ON` aus.

## 44.26. Zusammenfassung

In diesem Kapitel hast du gelernt:

- dass es temporäre und permanent gespeicherte `OPTION`-Einstellungen gibt
- warum manche OPTION-Einstellungen vor dem Start eines Programms vorgenommen werden müssen
- geänderte permanente Einstellungen mit `OPTION LIST` anzuzeigen
- `OPTION EXPLICIT` zur Kontrolle von Variablendeklarationen einzusetzen
- mit `OPTION DEFAULT` den Standard-Datentyp festzulegen
- mit `OPTION BASE` den Startindex von Arrays zu bestimmen
- `OPTION TAB` und `OPTION COLOURCODE` anzupassen
- dass `OPTION CPUSPEED` nicht bei VGA- und HDMI-Versionen zur Verfügung steht
- dass VGA- und HDMI-Versionen ihre CPU-Taktfrequenz im Zusammenhang mit der Videoauflösung und Bildwiederholrate festlegen
- dass eine ungeeignete Bildwiederholrate dazu führen kann, dass ein Monitor kein Bild anzeigt
- mit `OPTION KEYBOARD` das Tastaturlayout und weitere Tastatureinstellungen festzulegen
- mit `OPTION KEYBOARD REPEAT` die Tastenwiederholung einer USB-Tastatur anzupassen
- gespeicherte Einstellungen mit `OPTION RESET` zurückzusetzen
- verfügbare vordefinierte Konfigurationen mit `OPTION RESET LIST` anzuzeigen

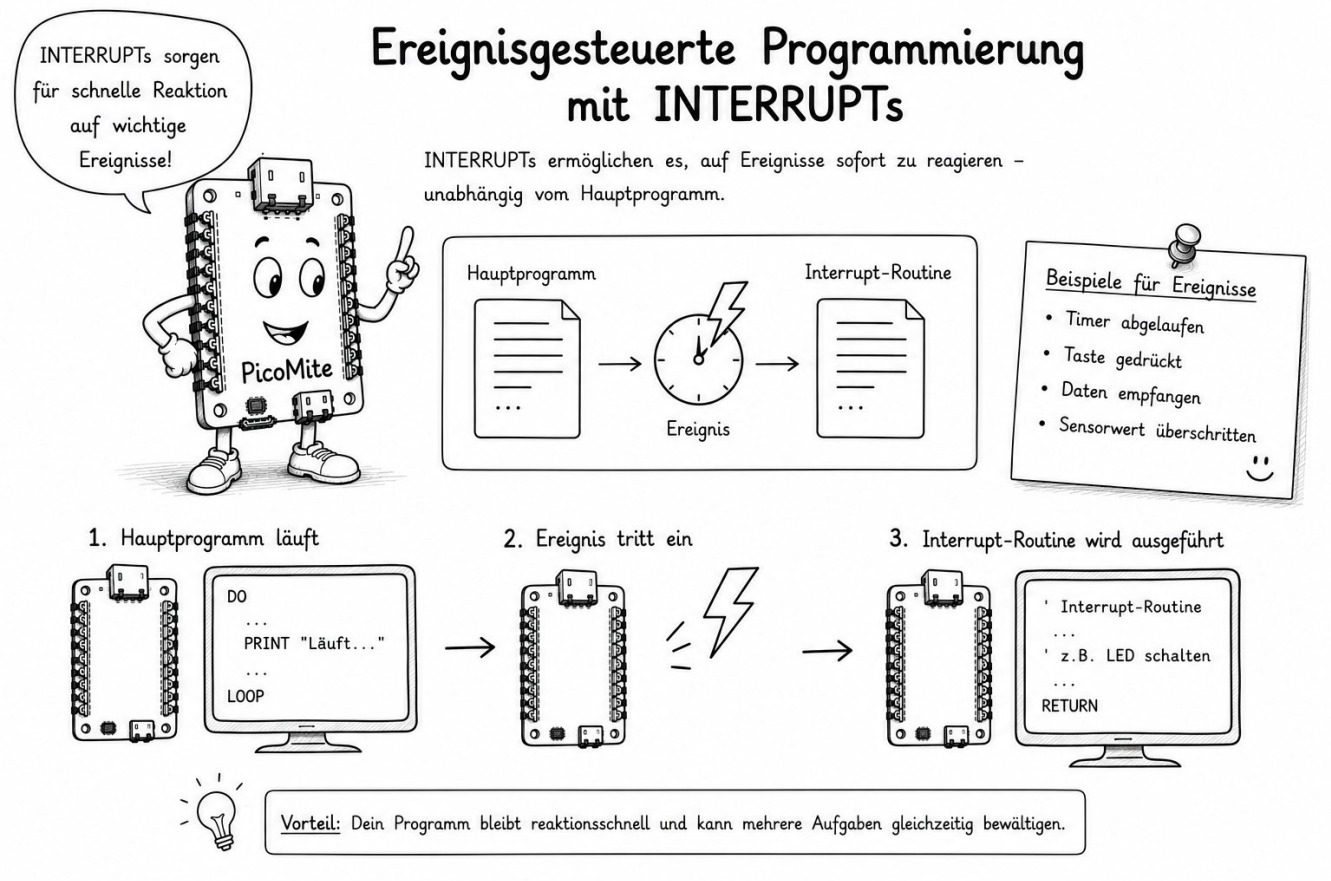
`OPTION`-Befehle sind damit sowohl ein Werkzeug für eine strengere und besser kontrollierbare BASIC-Programmierung als auch für die Anpassung des PicoMite-Systems an die verwendete Hardware und die eigenen Bedürfnisse.

## 44.27. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 44_option_informationen.bas

# Kapitel 45. Ereignisgesteuerte Programmierung mit Interrupts




## 45.1. Das lernst du

In diesem Kapitel lernst du:

- den Unterschied zwischen Polling und Interrupts kennen
- auf steigende und fallende Signalflanken zu reagieren
- Taster ohne ständige Abfrage auszuwerten
- Interrupt-Subroutinen zu schreiben
- Interrupts mit Flags sicher mit dem Hauptprogramm zu verbinden
- Taster bei der Verwendung von Interrupts zu entprellen
- Interrupts wieder zu deaktivieren

Am Ende des Kapitels kannst du Programme schreiben, die auf Ereignisse reagieren, ohne dass das Hauptprogramm ständig die Eingänge abfragen muss.

## 45.2. Was bedeutet ereignisgesteuert?

Bisher haben wir Taster meist in einer Schleife abgefragt.

Beispiel:

```
DO
  IF PIN(GP14) = 0 THEN
    PRINT "Taster gedrueckt"
  ENDIF
LOOP
```

Dieses Verfahren nennt man **Polling**.

Das Programm fragt ständig:

```
Taster gedrückt?
Taster gedrückt?
Taster gedrückt?
...
```

Das funktioniert, kann bei größeren Programmen aber schnell unübersichtlich werden.

## 45.3. Das Problem mit Polling

Stellen wir uns vor, unser Programm soll gleichzeitig:

- eine Uhr anzeigen
- Messwerte erfassen
- LEDs steuern
- auf einen Taster reagieren

Dann muss das Hauptprogramm regelmäßig prüfen, ob eines dieser Ereignisse eingetreten ist.

```
Taster gedrückt?
Messwert vorhanden?
Zeit abgelaufen?
...
```

Je größer das Programm wird, desto aufwendiger kann diese Vorgehensweise werden.

## 45.4. Die Idee eines Interrupts

Bei einem Interrupt muss das Hauptprogramm ein Ereignis nicht ständig selbst abfragen.

Anstatt immer wieder zu fragen:

```
"Taster gedrückt?"
```

können wir MMBasic sinngemäß sagen:

```
"Wenn der Taster gedrückt wird,
dann rufe diese Subroutine auf."
```

Tritt das konfigurierte Ereignis ein, unterbricht MMBasic kurz den normalen Programmablauf und ruft die angegebene Subroutine auf. Nach `END SUB` oder `EXIT SUB` wird das Hauptprogramm fortgesetzt.

## 45.5. Ein Vergleich aus dem Alltag

Polling ist vergleichbar mit einem Kind, das ständig fragt:

```
Sind wir schon da?
Sind wir schon da?
Sind wir schon da?
```

Ein Interrupt entspricht eher einer Türklingel.

Du musst nicht ständig zur Haustür laufen und nachsehen. Die Klingel meldet sich, wenn jemand den Taster betätigt.

## 45.6. Interrupts sind kein Multitasking

Ein häufiger Irrtum ist:

```
Interrupt = zweites Programm
```

Das stimmt nicht.

Ein Interrupt unterbricht das laufende Programm nur kurz:

```
Hauptprogramm
      
Interrupt-Subroutine
      
Hauptprogramm läuft weiter
```

Während die Interrupt-Subroutine ausgeführt wird, sind andere Interrupts gesperrt.

Die Interrupt-Subroutine läuft also nicht gleichzeitig mit dem Hauptprogramm, sondern unterbricht es vorübergehend. Deshalb sollte eine Interrupt-Subroutine immer möglichst kurz sein.

# 45.7. Steigende und fallende Flanken

Ein digitaler Eingang kennt die beiden Zustände Low (`0`) und High (`1`). Ein Interrupt kann ausgelöst werden, wenn sich dieser Zustand ändert.

MMBasic unterscheidet drei Möglichkeiten:

| Konfiguration | Signaländerung | Bedeutung |
|---|---|---|
| `INTH` | Low → High | Interrupt bei steigender Flanke |
| `INTL` | High → Low | Interrupt bei fallender Flanke |
| `INTB` | Low → High oder High → Low | Interrupt bei jeder Flanke |

Bei unserem Taster verwenden wir einen Pull-up-Widerstand. Im Ruhezustand liegt deshalb High an. Beim Drücken verbindet der Taster den Eingang mit GND:

```
Taster nicht gedrückt:  HIGH ──────────────────
                                │
Taster gedrückt:                └──────── LOW
                                ▯
                        fallende Flanke
```

Für das Drücken des Tasters benötigen wir deshalb `INTL`.

# 45.8. Einen Pin-Interrupt einrichten

Die grundlegende Syntax lautet:

```
SETPIN pin, cfg, target [, option]
```

`cfg` bestimmt, bei welcher Flanke der Interrupt ausgelöst wird. `target` ist der Name der Subroutine, die MMBasic beim Ereignis aufruft.

Für einen Taster an GP14, der gegen GND schaltet, können wir schreiben:

```
SETPIN GP14, INTL, TasterGedrueckt, PULLUP
```

Damit wird GP14 gleichzeitig als digitaler Eingang mit Pull-up konfiguriert. Ein zusätzliches `SETPIN GP14, DIN` ist nicht notwendig.

MMBasic erlaubt bis zu zehn gleichzeitig konfigurierte Pin-Interrupts.

## 45.9. Ein erster Interrupt

Unser erstes vollständiges Beispiel sieht so aus:

```
DIM ereignis = 0

SETPIN GP14, INTL, TasterGedrueckt, PULLUP

PRINT "Warte auf Taster..."

DO
  IF ereignis THEN
    ereignis = 0
    PRINT "Taster gedrueckt"
  ENDIF
LOOP

SUB TasterGedrueckt
  ereignis = 1
END SUB
```

Die Interrupt-Subroutine macht hier absichtlich sehr wenig. Sie setzt lediglich die Variable `ereignis` auf 1.

Die eigentliche Verarbeitung übernimmt anschließend das Hauptprogramm.

Dieses Vorgehen nennt man häufig das **Flag-Prinzip**. Eine Variable dient als Flag und zeigt dem Hauptprogramm an, dass ein Ereignis eingetreten ist.

Der Ablauf lässt sich vereinfacht so darstellen:

```
Taster
   |
   v
Interrupt-Subroutine
   |
   | ereignis = 1
   v
Flag
   |
   v
Hauptprogramm
   |
   v
eigentliche Verarbeitung
```

Einer Interrupt-Subroutine können keine Parameter übergeben werden. Innerhalb der Interrupt-Subroutine dürfen jedoch andere Subroutinen und

Funktionen aufgerufen werden.

## 45.10. Was passiert im Hintergrund?

Mit

```
SETPIN GP14, INTL, TasterGedrueckt, PULLUP
```

wird MMBasic sinngemäß mitgeteilt:

```
Wenn GP14 von HIGH nach LOW wechselt,
rufe SUB TasterGedrueckt auf.
```

Nach `END SUB` kehrt MMBasic zum normalen Programmablauf zurück.

Der aktuelle Zustand des Interrupt-Pins kann bei Bedarf weiterhin mit `PIN()` gelesen werden:

```
PRINT PIN(GP14)
```

## 45.11. Interrupt-Routinen kurz halten

Während eine Interrupt-Subroutine läuft, sind andere Interrupts gesperrt. Deshalb gilt eine besonders wichtige Regel:

Verwende niemals `PAUSE` innerhalb einer Interrupt-Subroutine.

Auch lange Schleifen oder aufwendige Berechnungen gehören nicht in eine Interrupt-Subroutine.

Ungünstig wäre beispielsweise:

```
SUB TasterGedrueckt
  PAUSE 50
  PRINT "Taster gedrueckt"
END SUB
```

Besser ist es, nur ein Flag zu setzen:

```
SUB TasterGedrueckt
  ereignis = 1
END SUB
```

Das Hauptprogramm kann anschließend alle zeitaufwendigeren Arbeiten erledigen.

## 45.12. Vorsicht beim Prellen

Im Kapitel über Taster haben wir bereits das sogenannte Prellen kennengelernt.

Ein einzelner Tastendruck kann mehrere schnelle Signalwechsel erzeugen. Bei Interrupts macht sich dieses Verhalten besonders bemerkbar.

Statt eines einzigen Ereignisses kann das Programm beispielsweise mehrere Interrupts erkennen.

```
1 Tastendruck
    
mehrere schnelle Flanken
    
mehrere Interrupts
```

Deshalb sollten mechanische Taster bei Bedarf entprellt werden.

## 45.13. Entprellen mit TIMER

Eine Möglichkeit besteht darin, den Zeitpunkt des letzten gültigen Interrupts zu speichern. Weitere Interrupts innerhalb einer kurzen Zeitspanne werden ignoriert.

```
DIM ereignis = 0
DIM letzteZeit = -100

SETPIN GP14, INTL, TasterGedrueckt, PULLUP

DO
  IF ereignis THEN
    ereignis = 0
    PRINT "Taster gedrueckt"
  ENDIF
LOOP

SUB TasterGedrueckt
  IF TIMER - letzteZeit < 50 THEN EXIT SUB

  letzteZeit = TIMER
  ereignis = 1
END SUB
```

Die Interrupt-Subroutine wartet dabei nicht. Sie prüft lediglich den Zeitabstand zum letzten gültigen Tastendruck und kehrt sofort zurück, wenn dieser zu klein ist.

Die `50` steht hier für eine Entprellzeit von 50 Millisekunden.

## 45.14. Praxisbeispiel: Ereigniszähler

Nun zählen wir, wie oft der Taster betätigt wurde. Auch hier verwenden wir die Entprellung mit `TIMER` und das zuvor kennengelernte Flag-Prinzip.

```
DIM anzahl = 0
DIM ereignis = 0
DIM letzteZeit = -100

SETPIN GP14, INTL, Zaehler, PULLUP

PRINT "Tastendruecke: 0"

DO
  IF ereignis THEN
    ereignis = 0
    anzahl = anzahl + 1
    PRINT "Tastendruecke: "; anzahl
  ENDIF
LOOP

SUB Zaehler
  IF TIMER - letzteZeit < 50 THEN EXIT SUB

  letzteZeit = TIMER
  ereignis = 1
END SUB
```

Die Interrupt-Subroutine erledigt nur die zeitkritische Erkennung und Entprellung des Tastendrucks. Anschließend setzt sie das Flag `ereignis`.

Das Hauptprogramm erkennt dieses Flag, erhöht den Zähler und übernimmt die Ausgabe. Damit bleibt die Interrupt-Subroutine kurz und das Beispiel folgt konsequent dem Flag-Prinzip.

## 45.15. Praxisbeispiel: LED umschalten

Eine LED an GP15 soll bei jedem Tastendruck ihren Zustand wechseln.

```
DIM ereignis = 0
DIM led = 0
DIM letzteZeit = -100

SETPIN GP14, INTL, Taster, PULLUP
SETPIN GP15, DOUT

PIN(GP15) = led
```

```
DO
  IF ereignis THEN
    ereignis = 0
    led = 1 - led
    PIN(GP15) = led
  ENDIF
LOOP

SUB Taster
  IF TIMER - letzteZeit < 50 THEN EXIT SUB

  letzteZeit = TIMER
  ereignis = 1
END SUB
```

Mit

```
led = 1 - led
```

wechselt `led` eindeutig zwischen `0` und `1`.

# 45.16. Interrupts wieder abschalten

Ein Interrupt sollte deaktiviert werden, wenn er nicht mehr benötigt wird.

Für GP14 geschieht das mit `SETPIN pin, OFF`:

```
SETPIN GP14, OFF
```

Hintergrund-Interrupts, die nicht mehr benötigt werden, können sonst später zu schwer nachvollziehbaren Programmfehlern führen.

# 45.17. Praxisprojekt: Reaktionstest

Nun bauen wir einen einfachen Reaktionstest.

Eine LED wird nach einer zufälligen Wartezeit eingeschaltet. Sobald sie leuchtet, muss der Benutzer den Taster möglichst schnell drücken.

# 45.18. Schaltung

- Taster zwischen GP14 und GND
- LED an GP15

## 45.19. Programm

```
DIM startZeit
DIM messung
DIM messungAktiv = 0
DIM tasteGedrueckt = 0
DIM letzteZeit = -100

SETPIN GP14, INTL, Reaktion, PULLUP
SETPIN GP15, DOUT

PIN(GP15) = 0

PRINT "Reaktionstest"
PRINT "Warte, bis die LED leuchtet..."

PAUSE 1000 + INT(RND * 4000)

startZeit = TIMER
messungAktiv = 1
PIN(GP15) = 1

DO
  IF tasteGedrueckt THEN
    tasteGedrueckt = 0
    PIN(GP15) = 0
    PRINT "Reaktionszeit: "; messung; " ms"
    EXIT DO
  ENDIF
LOOP

SETPIN GP14, OFF

SUB Reaktion
  IF TIMER - letzteZeit < 50 THEN EXIT SUB
  letzteZeit = TIMER

  IF messungAktiv = 0 THEN EXIT SUB

  messung = TIMER - startZeit
  messungAktiv = 0
  tasteGedrueckt = 1
END SUB
```

Die Variable `messungAktiv` sorgt dafür, dass ein Tastendruck nur dann als Reaktion gewertet wird, wenn die LED bereits eingeschaltet wurde.

Die Zeitmessung selbst findet direkt in der Interrupt-Subroutine statt. Das ist hier sinnvoll, weil der Zeitpunkt des Tastendrucks möglichst genau erfasst werden soll.

Alles, was nicht zeitkritisch ist, übernimmt dagegen das Hauptprogramm. Dazu gehören das Ausschalten der LED und die Ausgabe der gemessenen Reaktionszeit.

Nach dem gültigen Tastendruck wird `messungAktiv` sofort auf `0` gesetzt. Weitere Tastendrücke können die gemessene Zeit deshalb nicht mehr verändern.

Am Ende deaktiviert

```
SETPIN GP14, OFF
```

den nicht mehr benötigten Interrupt.

# 45.20. Können Interrupts verloren gehen?

Pin-Interrupts werden von MMBasic nach dem Abschluss eines Befehls geprüft und nicht von der Hardware zwischengespeichert.

Sehr kurze Impulse können deshalb unter Umständen auftreten und wieder verschwinden, bevor MMBasic sie erkennt.

Die meisten Befehle werden sehr schnell ausgeführt. Einige Hardwareoperationen können jedoch deutlich länger dauern. Während solcher Operationen kann sich die Reaktion auf einen Interrupt verzögern oder ein sehr kurzer Impuls übersehen werden.

Auch während einer `INPUT`-Anweisung sind Interrupts deaktiviert.

Für normale Taster ist das meist kein Problem, weil ein Tastendruck vergleichsweise lange dauert. Bei sehr kurzen elektrischen Impulsen muss dieser Punkt jedoch berücksichtigt werden.

# 45.21. Mehrere Interrupts

MMBasic kann nicht nur Pin-Interrupts verarbeiten. Auch Timer, serielle Schnittstellen, ADC, I2C und andere Funktionen können Interrupts auslösen.

In diesem Kapitel beschränken wir uns bewusst auf Pin-Interrupts. Andere Interruptquellen werden dort behandelt, wo die jeweiligen Funktionen verwendet werden.

Treten mehrere Interrupts gleichzeitig auf, verwendet MMBasic eine festgelegte Prioritätsreihenfolge. Pin-Interrupts werden dabei in der Reihenfolge ihrer Definition verarbeitet.

Für den Einstieg genügt es, sich zu merken:

- Interrupt-Subroutinen immer kurz halten.
- Während einer Interrupt-Subroutine sind andere Interrupts gesperrt.
- Aufwendige Arbeiten möglichst im Hauptprogramm erledigen.

## 45.22. Warum Interrupts so nützlich sind

Interrupts eignen sich besonders für:

- Taster
- Drehgeber
- Sensoren
- Lichtschranken
- Frequenzmessungen
- Kommunikation

Sie sind immer dann interessant, wenn ein Ereignis zu einem nicht genau vorhersehbaren Zeitpunkt auftreten kann.

## 45.23. Tipps aus der Praxis

- Halte Interrupt-Subroutinen möglichst kurz.
- Verwende niemals `PAUSE` innerhalb einer Interrupt-Subroutine.
- Vermeide lange Schleifen und aufwendige Berechnungen im Interrupt.
- Setze bei längeren Arbeiten ein Flag und erledige die Verarbeitung im Hauptprogramm.
- Denke bei mechanischen Tastern an das Prellen.
- Deaktiviere Interrupts, sobald sie nicht mehr benötigt werden.
- Verwende Interrupts nur dort, wo sie gegenüber Polling einen wirklichen Vorteil bieten.

## 45.24. Probier’s selbst!

1. Baue den Ereigniszähler nach.
2. Ändere `INTL` testweise in `INTH`. Wann wird der Interrupt jetzt ausgelöst?
3. Verwende `INTB` und beobachte, was beim Drücken und Loslassen des Tasters passiert.
4. Erweitere den Ereigniszähler auf zwei Taster.
5. Schalte mit einem Taster mehrere LEDs um.
6. Miss deine Reaktionszeit mehrmals.
7. Erweitere den Reaktionstest um die bisher beste Zeit.

## 45.25. Zusammenfassung

In diesem Kapitel hast du gelernt:

- was Interrupts sind und wie sie sich von Polling unterscheiden
- was steigende und fallende Flanken sind

- wie `INTH`, `INTL` und `INTB` verwendet werden
- wie Pin-Interrupts mit `SETPIN` eingerichtet werden
- warum Interrupt-Subroutinen möglichst kurz sein müssen
- wie das Flag-Prinzip funktioniert
- wie Taster ohne `PAUSE` in der Interrupt-Subroutine entprellt werden können
- warum zeitkritische Arbeiten von der übrigen Verarbeitung getrennt werden sollten
- dass sehr kurze Impulse unter bestimmten Umständen übersehen werden können
- wie ein Interrupt mit `SETPIN pin, OFF` wieder deaktiviert wird

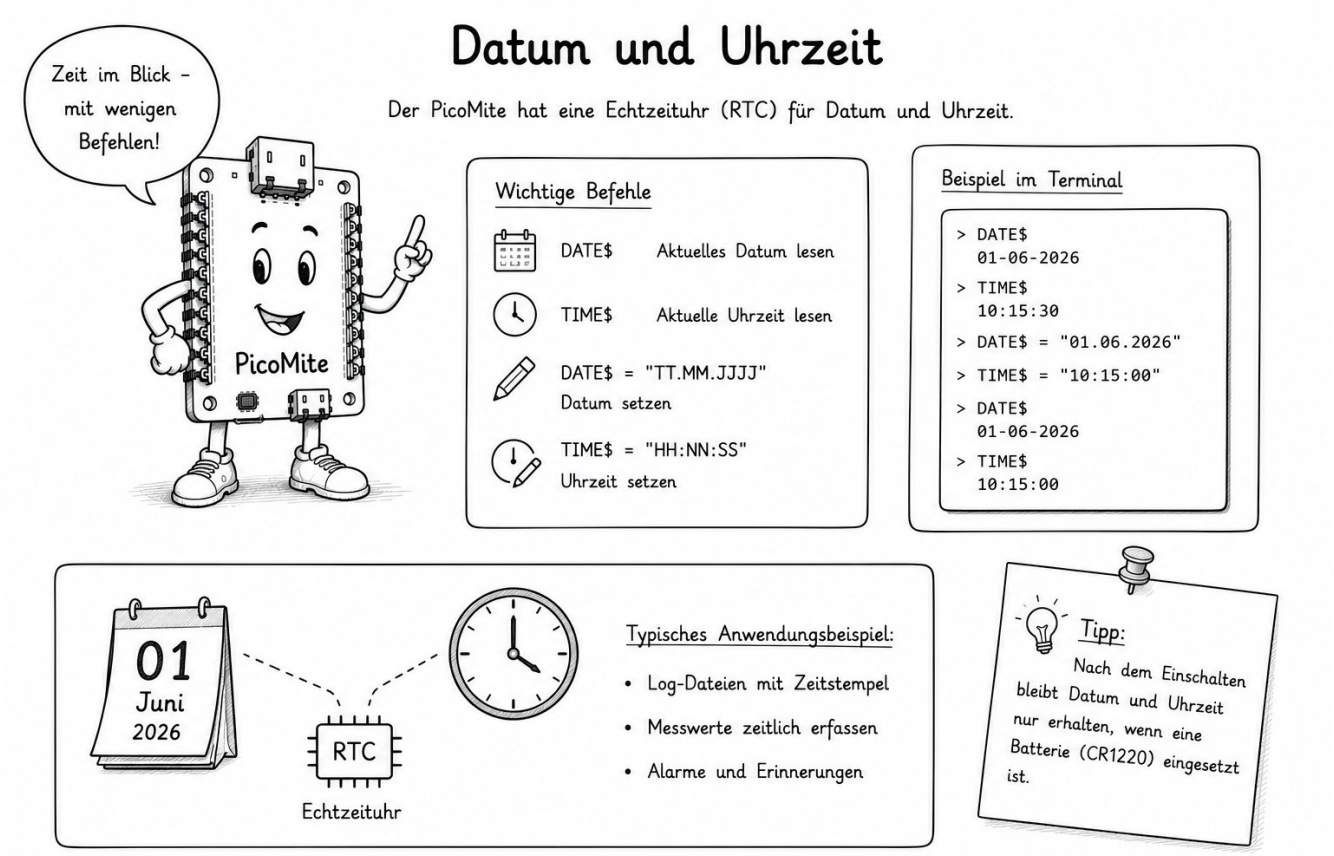
Interrupts gehören zu den wichtigsten Werkzeugen der Mikrocontroller-Programmierung. Richtig eingesetzt ermöglichen sie Programme, die schnell und übersichtlich auf Ereignisse reagieren.

## 45.26. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 45_interrupt_demo.bas

# Kapitel 46. Datum und Uhrzeit




## 46.1. Das lernst du

In diesem Kapitel lernst du:

- Datum und Uhrzeit auszulesen
- die interne Echtzeituhr (RTC) von MMBasic zu verwenden
- Zeitmessungen durchzuführen
- Datum und Uhrzeit auszulesen und zu setzen
- praktische Anwendungen mit Datum und Uhrzeit zu entwickeln

Am Ende des Kapitels kannst du Uhren, Stoppuhren und einfache Datenlogger programmieren.

## 46.2. Warum Datum und Uhrzeit wichtig sind

Viele Programme benötigen Informationen über die aktuelle Zeit.

Beispiele sind:

- Uhren
- Wecker
- Datenlogger

- Alarmanlagen
- Zeiterfassung
- Messwertaufzeichnungen

MMBasic stellt dafür verschiedene Funktionen bereit.

## 46.3. Die aktuelle Uhrzeit

Die aktuelle Uhrzeit kann mit:

```
TIME$
```

abgerufen werden.

Beispiel:

```
PRINT TIME$
```

Ausgabe:

```
21:37:15
```

## 46.4. Das aktuelle Datum

Für das Datum gibt es:

```
DATE$
```

Beispiel:

```
PRINT DATE$
```

Ausgabe:

```
17-07-2026
```

`DATE$` liefert das aktuelle Datum. Beim Setzen akzeptiert MMBasic verschiedene Schreibweisen, die im folgenden Abschnitt gezeigt werden.

## 46.5. Datum und Uhrzeit gemeinsam anzeigen

```
PRINT DATE$; " "; TIME$
```

Beispiel:

```
17-07-2026 21:37:15
```

## 46.6. Eine digitale Uhr

Mit wenigen Zeilen Code lässt sich eine laufende Uhr erstellen.

```
DO

  CLS

  PRINT TIME$

  PAUSE 1000

LOOP
```

Jede Sekunde wird die Anzeige aktualisiert.

## 46.7. Datum und Uhrzeit setzen

Datum und Uhrzeit können auch gesetzt werden.

Für das Datum akzeptiert MMBasic mehrere Schreibweisen:

```
DATE$ = "DD-MM-YY"
DATE$ = "DD-MM-YYYY"
DATE$ = "DD/MM/YY"
DATE$ = "DD/MM/YYYY"
DATE$ = "YYYY-MM-DD"
DATE$ = "YYYY/MM/DD"
```

Beispiel:

```
DATE$ = "17-07-2026"
```

Auch die Uhrzeit kann mit unterschiedlicher Genauigkeit gesetzt werden:

```
TIME$ = "21:45:30"
TIME$ = "21:45"
TIME$ = "21"
```

Werden Minuten oder Sekunden weggelassen, setzt MMBasic die fehlenden Werte auf null. So setzt beispielsweise

```
TIME$ = "14:30"
```

die Uhr auf `14:30:00`.

## 46.8. Die interne Uhr und die RTC

RTC steht für:

```
Real Time Clock
```

MMBasic führt intern Datum und Uhrzeit und stellt sie über `DATE$` und `TIME$` zur Verfügung. Ob nach dem Einschalten sofort das richtige Datum und die richtige Uhrzeit vorhanden sind, hängt beim PicoMite jedoch von der RTC-Konfiguration ab.

Mit `OPTION RTC AUTO ENABLE`

```
OPTION RTC AUTO ENABLE
```

übernimmt die PicoMite-Firmware beim Start `DATE$` und `TIME$` aus der konfigurierten RTC.

Ohne `OPTION RTC AUTO ENABLE` startet die Firmware nach dem Einschalten mit:

```
DATE$ = "01-01-2024"
TIME$ = "00:00:00"
```

`DATE$` und `TIME$` bedeuten daher nicht automatisch, dass der PicoMite nach dem Ausschalten die aktuelle Zeit kennt. Für eine dauerhaft korrekte Uhrzeit muss eine geeignete RTC vorhanden und entsprechend konfiguriert sein.

## 46.9. Die Funktion TIMER

Neben der Uhrzeit gibt es auch die Variable:

```
TIMER
```

`TIMER` ist ein fortlaufender Millisekundenzähler. Sein aktueller Wert kann gelesen werden.

Beispiel:

```
PRINT TIMER
```

Mögliche Ausgabe:

```
123456
```

Der Timer kann außerdem auf einen positiven Millisekundenwert gesetzt werden. Besonders praktisch ist das Zurücksetzen auf null:

```
TIMER = 0
```

Damit lassen sich einfache Zeitmessungen besonders übersichtlich durchführen.

### 46.9.1. Wie lange kann TIMER zählen?

Da `TIMER` fortlaufend Millisekunden zählt, stellt sich die Frage, ob der Zähler irgendwann überläuft.

Der Raspberry Pi Pico besitzt dafür einen 64-Bit-Hardware-Timer. Der zugrunde liegende Hardwarezähler arbeitet mit Mikrosekunden und kann einen sehr großen Wertebereich erfassen.

Ein 64-Bit-Zähler kann

```
18.446.744.073.709.551.616
```

verschiedene Werte darstellen.

Würde ein solcher Zähler in Mikrosekunden fortlaufend zählen, entspräche dies einer Zeitspanne von ungefähr 585.000 Jahren.

Für Zeitmessungen mit `TIMER` muss man sich deshalb in normalen MMBasic-Programmen praktisch keine Gedanken über einen Überlauf nach Tagen, Monaten oder Jahren machen.

Zum Vergleich: Ein Jahr entspricht ungefähr

```
31.536.000.000 ms
```

also rund 31,5 Milliarden Millisekunden.

Der Hardware-Timer des Pico arbeitet intern mit Mikrosekunden. `TIMER` stellt diesen Zeitbezug in MMBasic als Millisekundenwert zur Verfügung.

> Für die üblichen Anwendungen von TIMER, beispielsweise Stoppuhren, Laufzeitmessungen oder Zeitabstände zwischen Ereignissen, ist ein Überlauf daher praktisch nicht relevant.

## 46.10. Zeit messen

Mit TIMER können Zeitmessungen durchgeführt werden.

```
TIMER = 0

PAUSE 2000

PRINT TIMER
```

Ausgabe:

```
2000
```

Das Programm hat etwa zwei Sekunden gewartet.

Alternativ kann der aktuelle Wert von TIMER in einer Variablen gespeichert werden. Das ist sinnvoll, wenn der Timer nicht zurückgesetzt werden soll:

```
start = TIMER

PAUSE 2000

PRINT TIMER - start
```

## 46.11. Eine einfache Stoppuhr

```
start = TIMER

PRINT "Beliebige Taste druecken"

DO
LOOP UNTIL INKEY$ <> ""

PRINT
PRINT "Zeit: "; TIMER - start; " ms"
```

Die vergangene Zeit wird in Millisekunden angezeigt.

## 46.12. Minuten und Sekunden berechnen

Angenommen, wir haben eine Zeitdauer in Sekunden:

```
sekunden = 367
```

Dann können Minuten und Sekunden berechnet werden:

```
minuten = sekunden \ 60

rest = sekunden MOD 60

PRINT minuten; " Minuten"

PRINT rest; " Sekunden"
```

Ausgabe:

```
6 Minuten
7 Sekunden
```

## 46.13. Eine laufende Stoppuhr

```
start = TIMER

DO

  CLS

  PRINT "Zeit: "; (TIMER - start) / 1000

  PAUSE 100

LOOP
```

Die Anzeige wird zehnmal pro Sekunde aktualisiert.

## 46.14. Praxisprojekt: Digitale Uhr

```
DO

  CLS

  PRINT "Digitale Uhr"
```

```
  PRINT

  PRINT DATE$

  PRINT TIME$

  PAUSE 1000

LOOP
```

Dieses kleine Programm verwandelt PicoMite oder Colour Maximite in eine einfache Uhr.

# 46.15. Praxisprojekt: Monatskalender

Nachdem wir uns mit Datum und Uhrzeit beschäftigt haben, wollen wir nun ein etwas größeres Praxisprojekt umsetzen.

Wir schreiben einen einfachen Monatskalender.

Der Benutzer gibt Monat und Jahr ein, und das Programm erzeugt einen Kalender ähnlich wie man ihn aus Wandkalendern oder Terminplanern kennt.

Beispiel:

```
Kalender fuer Juli 2026

Mo Di Mi Do Fr Sa So

       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
```

Dieses Projekt zeigt sehr schön, wie viele der bisher gelernten Techniken zusammenarbeiten:

- Variablen
- IF-Abfragen
- Schleifen
- Funktionen
- Datumsberechnungen
- Textformatierung

### 46.15.1. Schaltjahre berücksichtigen

Bevor wir einen Kalender erzeugen können, müssen wir wissen, wie viele Tage ein Monat besitzt.

Besonders der Februar ist interessant.

Er besitzt:

- 28 Tage in normalen Jahren
- 29 Tage in Schaltjahren

Ein Schaltjahr liegt vor, wenn:

- das Jahr durch 4 teilbar ist
- aber nicht durch 100
- außer es ist zusätzlich durch 400 teilbar

### 46.15.2. Anzahl der Tage bestimmen

```
FUNCTION TageImMonat(monat, jahr)

  SELECT CASE monat

    CASE 1,3,5,7,8,10,12
      TageImMonat = 31

    CASE 4,6,9,11
      TageImMonat = 30

    CASE 2

      IF (jahr MOD 400 = 0) OR _
         ((jahr MOD 4 = 0) AND (jahr MOD 100 <> 0)) THEN

        TageImMonat = 29

      ELSE

        TageImMonat = 28

      ENDIF

  END SELECT

END FUNCTION
```

### 46.15.3. Wochentag des ersten Tages bestimmen

Für einen Kalender müssen wir außerdem wissen, auf welchen Wochentag der erste Tag des Monats fällt.

Für unser Beispiel berechnen wir den Wochentag mit einer eigenen Funktion:

```
FUNCTION ErsterWochentag(monat, jahr)

  LOCAL q, m, j, h

  q = 1

  IF monat < 3 THEN
    monat = monat + 12
    jahr = jahr - 1
  ENDIF

  m = monat
  j = jahr

  h = (q + INT((13*(m+1))/5) + _
       j + INT(j/4) - INT(j/100) + _
       INT(j/400)) MOD 7

  ErsterWochentag = (h + 5) MOD 7

END FUNCTION
```

Die Funktion liefert:

```
0 = Montag
1 = Dienstag
...
6 = Sonntag
```

### 46.15.4. Das Hauptprogramm

```
CLS

INPUT "Monat"; monat
INPUT "Jahr"; jahr

tage = TageImMonat(monat, jahr)

starttag = ErsterWochentag(monat, jahr)

PRINT
```

```
PRINT "Mo Di Mi Do Fr Sa So"
PRINT

FOR i = 1 TO starttag
  PRINT "   ";
NEXT i

FOR tag = 1 TO tage

  PRINT RIGHT$("  " + STR$(tag), 2); " ";

  IF (starttag + tag) MOD 7 = 0 THEN
    PRINT
  ENDIF

NEXT tag

PRINT
```

Für ein robustes Programm sollte die Eingabe zusätzlich geprüft werden. Ein gültiger Monat liegt zwischen 1 und 12.

### 46.15.5. Erweiterungen

Dieses Programm lässt sich leicht erweitern.

Mögliche Ideen sind:

- aktuelles Datum hervorheben
- Monatsnamen anzeigen
- Feiertage markieren
- Geburtstage eintragen
- mehrere Monate hintereinander ausgeben
- einen Jahreskalender erzeugen

### 46.15.6. Ein Blick zurück

Kalenderprogramme gehörten in den 1980er-Jahren zu den beliebtesten BASIC-Projekten auf Heimcomputern.

Sie waren anspruchsvoll genug, um interessante Programmiertechniken zu vermitteln, gleichzeitig aber praktisch im Alltag nutzbar.

Auch heute ist ein Kalender ein schönes Beispiel dafür, wie Mathematik und Programmierung zusammenarbeiten.

Versuche den Kalender so zu erweitern, dass automatisch der aktuelle Monat

angezeigt wird.

Verwende dazu DATE$ und werte Monat und Jahr aus dem aktuellen Datum aus.

## 46.16. Praxisprojekt: Laufzeitzähler

Mit `TIMER` lässt sich auch anzeigen, wie lange ein Programm bereits läuft.

Das folgende Beispiel ist ein einfacher Laufzeitzähler:

```
start = TIMER

DO

  CLS

  PRINT "Betriebszeit"

  PRINT

  PRINT INT((TIMER - start) / 1000)

  PRINT " Sekunden"

  PAUSE 1000

LOOP
```

Dies ist noch kein echter Betriebsstundenzähler. Ein solcher müsste seinen Zählerstand dauerhaft speichern und beim nächsten Einschalten wieder laden.

## 46.17. Praxisprojekt: Messwerte mit Zeitstempel

Stellen wir uns vor, ein Sensor liefert Messwerte.

Dann könnte jeder Datensatz mit Datum und Uhrzeit gespeichert werden. Voraussetzung für aussagekräftige Zeitstempel ist natürlich, dass Datum und Uhrzeit zuvor korrekt eingestellt oder aus einer RTC übernommen wurden.

```
OPEN "MESSUNG.TXT" FOR APPEND AS #1

PRINT #1, DATE$; ";"; TIME$; ";"; temperatur

CLOSE #1
```

Die Datei könnte später so aussehen:

```
17-07-2026;21:30:15;23.5
17-07-2026;21:35:15;23.7
17-07-2026;21:40:15;23.8
```

Dieses Verfahren wird in Datenloggern häufig eingesetzt.

## 46.18. Die analoge Bahnhofsuhr revisited

Erinnerst du dich an die Bahnhofsuhr aus Kapitel 24?

Damals haben wir die Zeiger mit Hilfe von Variablen bewegt. Mit `TIME$` können wir daraus nun eine echte Uhr machen.

Ein Ausschnitt aus der Hauptschleife könnte beispielsweise so aussehen:

```
Do
  If alteZeit$<>Time$ Then
    alteZeit$=Time$
    Berechne Time$
    ZeichneZifferblatt mitteX,mitteY,radius
    ZeichneText mitteX,mitteY,radius
    ZeichneStundenzeiger mitteX,mitteY,radius,stundenWinkel
    ZeichneMinutenzeiger mitteX,mitteY,radius,minutenWinkel
    ZeichneSekundenzeiger mitteX,mitteY,radius,sekundenWinkel
  EndIf
Loop While Inkey$=""
```

Die Variable `alteZeit$` enthält die zuletzt dargestellte Uhrzeit. Nur wenn sich `TIME$` geändert hat, wird die Uhr neu gezeichnet.

Da sich `TIME$` nur einmal pro Sekunde ändert, vermeiden wir damit unnötiges Neuzeichnen der Grafik.

Die Subroutine `Berechne` zerlegt `TIME$` in Stunden, Minuten und Sekunden und berechnet daraus die Winkel der drei Zeiger.

Dieses Verfahren ist auch für andere Anzeigen interessant. Statt mit `PAUSE` auf eine feste Zeit zu warten, kann ein Programm prüfen, ob sich der anzuzeigende Wert tatsächlich geändert hat, und die Anzeige nur dann aktualisieren.

## 46.19. Probier’s selbst!

1. Erstelle eine digitale Uhr.
2. Baue eine Stoppuhr.
3. Miss deine Reaktionszeit.

4. Erweitere die Bahnhofsuhr um die aktuelle Uhrzeit.
5. Speichere Datum und Uhrzeit in einer Datei.

## 46.20. Zusammenfassung

In diesem Kapitel hast du gelernt:

- Datum mit DATE$ auszulesen
- Uhrzeiten mit TIME$ anzuzeigen
- Zeitmessungen mit TIMER durchzuführen
- Stoppuhren zu programmieren
- Zeitstempel in Dateien zu speichern
- eigene Projekte um Datum und Uhrzeit zu erweitern

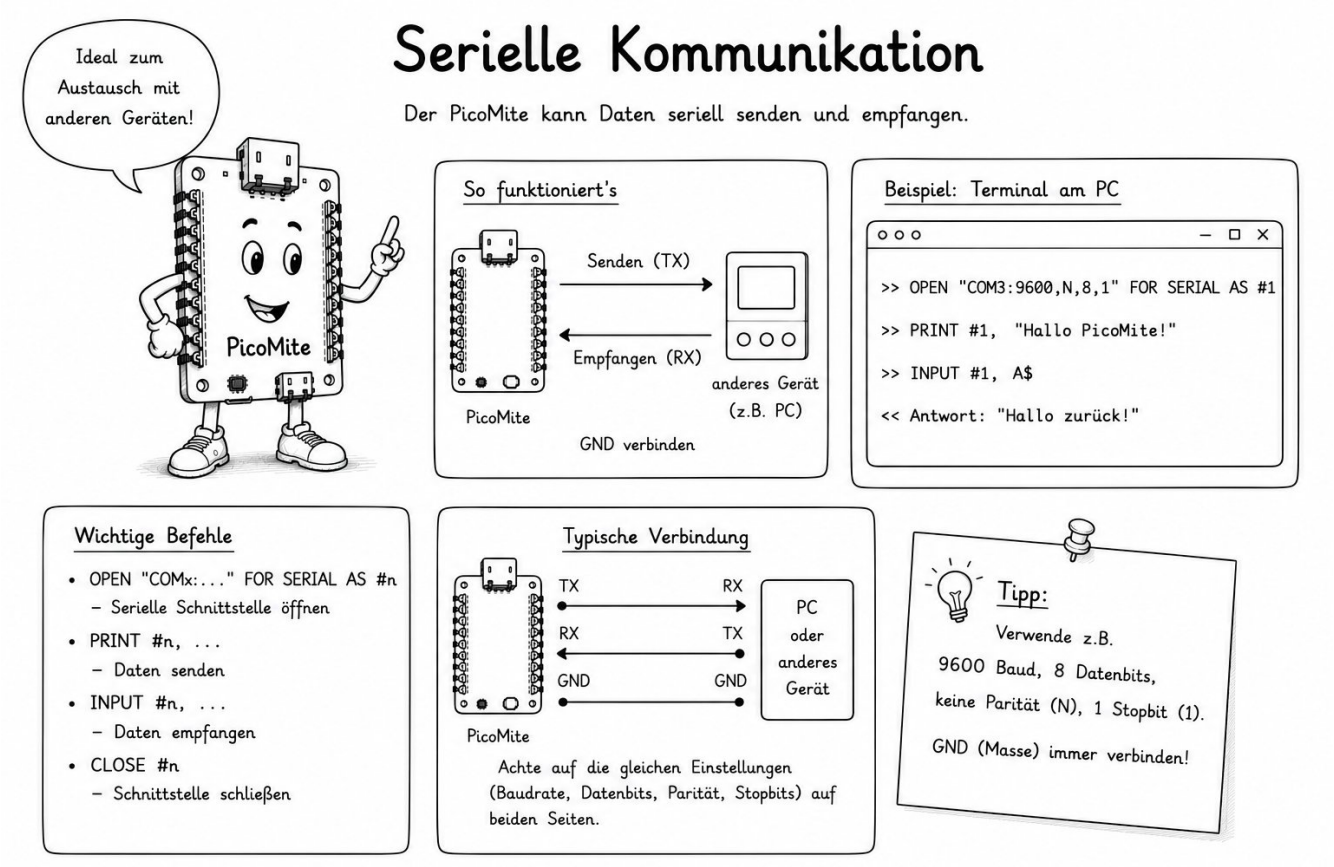
Datum und Uhrzeit gehören zu den wichtigsten Informationen in technischen Anwendungen und eröffnen viele neue Möglichkeiten für praktische Projekte.

## 46.21. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 46_datum_uhrzeit.bas

# Kapitel 47. Serielle Kommunikation




## 47.1. Das lernst du

In diesem Kapitel lernst du:

- was eine serielle Schnittstelle ist
- Daten über eine UART-Verbindung zu senden und zu empfangen
- einen PicoMite mit einem PC zu verbinden
- zwei MMBasic-Systeme miteinander kommunizieren zu lassen
- einfache Kommunikationsprotokolle zu entwickeln
- größere Datenmengen mit Long Strings zu verarbeiten

Am Ende des Kapitels kannst du eigene Geräte miteinander Daten austauschen lassen.

## 47.2. Eine der ältesten Computerschnittstellen

Die serielle Datenübertragung ist deutlich älter als der Personal Computer. Bereits Anfang des 20. Jahrhunderts wurde sie bei Fernschreibern eingesetzt. Das heute noch gebräuchliche Start-/Stopbit-Verfahren entstand bereits in den 1920er-Jahren.

Mit dem Aufkommen der Mikrocomputer setzte sich in den 1970er- und 1980er-Jahren vor allem die RS-232-Schnittstelle als Standard für die serielle Kommunikation zwischen Computern und

Peripheriegeräten durch.

Viele bekannte Systeme nutzten serielle Schnittstellen oder konnten entsprechend erweitert werden:

- Commodore C64
- Atari ST
- Apple II
- CP/M-Rechner
- Robotron-Computer
- Industriecomputer
- Terminals

Damals war die serielle Schnittstelle oft die wichtigste Verbindung zur Außenwelt.

Auch heute wird serielle Kommunikation noch in unzähligen Geräten eingesetzt.

## 47.3. Was bedeutet seriell?

Daten bestehen aus einzelnen Bits.

Bei einer seriellen Verbindung werden diese Bits nacheinander übertragen.

Beispiel:

```
H = 01001000
```

Die einzelnen Bits werden nacheinander über eine Leitung gesendet:

```
0 1 0 0 1 0 0 0
```

Daher stammt die Bezeichnung **seriell**.

## 47.4. UART

Die meisten Mikrocontroller besitzen eine UART.

UART steht für:

```
Universal Asynchronous
Receiver Transmitter
```

Die UART übernimmt das Senden und Empfangen der einzelnen Bits. Das BASIC-Programm kann dadurch mit Zeichen und Zeichenketten arbeiten, ohne sich um jedes einzelne Bit kümmern zu

müssen.

## 47.5. Wo begegnet uns UART heute?

Serielle Schnittstellen werden heute beispielsweise verwendet für:

- GPS-Empfänger
- Sensoren
- Funkmodule
- industrielle Steuerungen
- Retrocomputer
- andere Mikrocontroller

Sie gehören zu den wichtigsten Kommunikationsschnittstellen in der Elektronik.

## 47.6. Baudrate

Damit zwei Geräte miteinander kommunizieren können, müssen beide dieselbe Übertragungsgeschwindigkeit verwenden.

Bei einer UART wird diese üblicherweise als Baudrate angegeben. Bei der hier verwendeten UART-Übertragung entsprechen beispielsweise 9600 Baud einer Übertragung von 9600 Bits pro Sekunde auf der Leitung.

Typische Werte sind:

```
1200
2400
4800
9600
19200
38400
115200
```

Der PicoMite unterstützt für `COM1` und `COM2` Baudraten von 1200 bis 921600 Baud.

Für die ersten Versuche verwenden wir 9600 Baud.

## 47.7. Sender und Empfänger

Eine UART besitzt zwei Signale:

| **Signal** | **Bedeutung** |
|---|---|
| TX | Senden |
| RX | Empfangen |

Beim Verbinden zweier Geräte werden Sender und Empfänger gekreuzt verbunden:

```
TX → RX
RX → TX
```

Zusätzlich benötigen beide Geräte eine gemeinsame Masseverbindung.

## 47.8. UART ist nicht RS-232

Die Begriffe UART und RS-232 werden häufig miteinander vermischt, bezeichnen aber nicht dasselbe.

Eine UART beschreibt die serielle Datenübertragung. Die UART-Pins des PicoMite arbeiten mit den Logikpegeln der GPIO-Pins, also mit 3,3-V-Signalen.

Eine klassische RS-232-Schnittstelle verwendet dagegen andere elektrische Signalpegel. Sie darf deshalb **nicht direkt** mit den GPIO-Pins des PicoMite verbunden werden.

Für eine Verbindung zu einer echten RS-232-Schnittstelle wird ein geeigneter Pegelwandler benötigt, beispielsweise ein `MAX3232`.

Die UART-Pins des PicoMite sind keine RS-232-Anschlüsse. Eine klassische RS-232-Schnittstelle darf niemals direkt an die GPIO-Pins angeschlossen werden.

## 47.9. COM1 und COM2 den GPIO-Pins zuordnen

Der PicoMite besitzt zwei serielle Schnittstellen: `COM1` und `COM2`.

Bevor eine davon verwendet werden kann, müssen die GPIO-Pins für RX und TX mit `SETPIN` zugeordnet werden.

Die Syntax für `COM1` lautet:

```
SETPIN rx, tx, COM1
```

Für die Beispiele dieses Kapitels verwenden wir:

```
SETPIN GP1, GP0, COM1
```

Dabei gilt:

| Pin | Funktion |
|---|---|
| GP1 | COM1 RX – Daten zum PicoMite |
| GP0 | COM1 TX – Daten vom PicoMite |

MMBasic erkennt anhand der gewählten GPIO-Pins, welcher Pin RX und welcher TX ist. Deshalb dürfen die Pins bei `SETPIN` auch in umgekehrter Reihenfolge angegeben werden.

Das folgende Kommando ist daher ebenfalls gültig:

```
SETPIN GP0, GP1, COM1
```

Für `COM2` verwenden wir beispielsweise:

```
SETPIN GP5, GP4, COM2
```

Dabei ist GP5 der RX-Pin und GP4 der TX-Pin.

Beim PicoMite HDMI sind GP12 bis GP19 für die Videoausgabe reserviert. Bei Firmware-Versionen mit USB-Tastatur/Maus sind außerdem GP8 und GP9 für die serielle Konsole reserviert.

GP0 und GP1 eignen sich deshalb gut für `COM1` in den Beispielen dieses Tutorials.

## 47.10. Verbindung zweier Systeme

```
PicoMite A          PicoMite B

GP0 TX ------------ GP1 RX
GP1 RX ------------ GP0 TX
GND ---------------- GND
```

Die GPIO-Nummern beziehen sich immer auf das jeweilige Gerät.

Merke:

```
TX → RX
RX → TX
GND → GND
```

## 47.11. Das erste UART-Programm

Nachdem `COM1` den GPIO-Pins zugeordnet wurde, kann die serielle Schnittstelle geöffnet werden.

Das folgende Programm sendet den Text `Hallo Welt!` über `COM1`:

```
SETPIN GP1, GP0, COM1
```

```
OPEN "COM1:9600" AS #1

PRINT #1, "Hallo Welt!"

CLOSE #1
```

Der Ablauf besteht aus vier Schritten:

- Pins mit `SETPIN` zuordnen
- Schnittstelle mit `OPEN` öffnen
- Daten senden oder empfangen
- Schnittstelle mit `CLOSE` schließen

MMBasic behandelt eine serielle Schnittstelle dabei ähnlich wie eine Datei. Nach dem Öffnen erhält sie eine Dateinummer, die von Befehlen wie `PRINT`, `INPUT$`, `LOC` und `CLOSE` verwendet wird.

## 47.12. Die serielle Schnittstelle öffnen

Eine serielle Schnittstelle wird mit `OPEN` geöffnet:

```
OPEN "COM1:9600" AS #1
```

Die 1 ist die Dateinummer, unter der das Programm anschließend auf die Schnittstelle zugreift.

Die Voreinstellungen des PicoMite sind:

- 9600 Baud
- 8 Datenbits
- keine Parität
- 1 Stopbit
- 256 Byte Empfangspuffer
- 256 Byte Sendepuffer

Deshalb würde sogar folgende Schreibweise 9600 Baud verwenden:

```
OPEN "COM1:" AS #1
```

Im Tutorial geben wir die Baudrate trotzdem ausdrücklich an, damit sofort erkennbar ist, mit welcher Geschwindigkeit die Verbindung arbeitet.

Bei Bedarf kann auch ein größerer Empfangspuffer festgelegt werden:

```
OPEN "COM1:9600, 1024" AS #1
```

Damit wird ein Empfangspuffer von 1024 Byte verwendet.

Ein größerer UART-Empfangspuffer bedeutet nicht automatisch, dass die empfangenen Daten in eine normale Stringvariable passen. Normale Strings sind in MMBasic auf 255 Zeichen begrenzt.

Wie größere Datenmengen verarbeitet werden können, sehen wir später in diesem Kapitel bei den Long Strings.

## 47.13. Daten senden

Zum Senden kann `PRINT` mit der Dateinummer der seriellen Schnittstelle verwendet werden:

```
SETPIN GP1, GP0, COM1
OPEN "COM1:9600" AS #1

PRINT #1, "Hallo Welt"
```

Der Text wird zunächst in einem Sendepuffer abgelegt und anschließend von der UART übertragen. Das BASIC-Programm kann währenddessen normalerweise bereits weiterarbeiten.

## 47.14. Schnittstelle schließen

Nach der Verwendung sollte die serielle Schnittstelle geschlossen werden:

```
CLOSE #1
```

`CLOSE` wartet zunächst, bis noch vorhandene Daten aus dem Sendepuffer übertragen wurden. Anschließend werden die verwendeten Puffer freigegeben.

## 47.15. Daten empfangen

Empfangene Zeichen werden von MMBasic automatisch in einem Speicherpuffer gesammelt.

Mit `LOC()` kann festgestellt werden, wie viele Zeichen bereits im Empfangspuffer warten:

```
anzahl = LOC(#1)
```

Die Funktion `INPUT$()` liest Zeichen aus diesem Puffer. Sie wartet nicht auf neue Daten. Sind momentan keine Zeichen vorhanden, wird sofort ein leerer String zurückgegeben.

Ein einfaches Empfangsprogramm sieht so aus:

```
SETPIN GP1, GP0, COM1
```

```
OPEN "COM1:9600" AS #1

DO
  IF LOC(#1) > 0 THEN
    text$ = INPUT$(LOC(#1), #1)
    PRINT text$;
  ENDIF
LOOP
```

Durch das Semikolon bei `PRINT text$;` werden die Zeichen so ausgegeben, wie sie empfangen wurden.

Solange jeweils höchstens 255 Zeichen in `text$` übernommen werden, reicht dafür eine normale Stringvariable aus. Für größere zusammenhängende Datenmengen benötigen wir dagegen Long Strings.

## 47.16. Zeichenstrom statt fertiger Nachrichten

Eine UART überträgt zunächst nur einen Strom von Zeichen. Sie kennt von sich aus keine Befehle, Zeilen oder Nachrichten.

Wird beispielsweise

```
LEDON
```

gesendet, könnte das empfangende Programm bei einer frühen Abfrage zunächst nur

```
LE
```

und etwas später

```
DON
```

im Empfangspuffer finden.

Wenn ein Programm vollständige Befehle übertragen soll, benötigt es deshalb eine Vereinbarung darüber, wo eine Nachricht endet. Häufig wird dafür ein Zeilenende mit CR/LF verwendet.

Auf diesem Prinzip bauen wir später unsere einfachen Kommunikationsprotokolle auf.

## 47.17. Praxisprojekt: PicoMite und PC verbinden

Ein besonders nützliches Experiment ist die Verbindung mit einem PC.

## 47.18. Verdrahtung

Für dieses Beispiel verwenden wir einen USB-UART-Adapter mit 3,3-V-Logikpegeln:

```
PicoMite          USB-UART-Adapter

GP0 (TX) -------- RX
GP1 (RX) -------- TX
GND ------------- GND
```

Verwende einen USB-UART-Adapter, dessen Logikpegel für 3,3 V geeignet sind. Ein echter RS-232-Anschluss eines PCs darf nicht direkt mit GP0 und GP1 verbunden werden.

Anschließend wird `COM1` konfiguriert:

```
SETPIN GP1, GP0, COM1
```

## 47.19. Benötigte Software

Geeignete Terminalprogramme sind beispielsweise:

- Tera Term
- PuTTY
- MobaXterm

Diese Programme können serielle Daten anzeigen und senden.

Für unser Beispiel muss das Terminalprogramm auf 9600 Baud, 8 Datenbits, keine Parität und 1 Stopbit eingestellt werden.

## 47.20. Mehr als nur Text: VT100-Terminals

Viele Terminalprogramme unterstützen VT100- bzw. ANSI-Steuersequenzen. Ursprünglich wurde der VT100-Standard Ende der 1970er-Jahre für Computerterminals von Digital Equipment Corporation (DEC) entwickelt.

Dadurch kann ein Programm nicht nur Text ausgeben, sondern beispielsweise auch den Bildschirm löschen oder den Cursor positionieren.

MMBasic kann diese Steuersequenzen wie gewöhnlichen Text über die serielle Schnittstelle senden:

```
PRINT #1, CHR$(27) + "[2J"    ' Bildschirm löschen
PRINT #1, CHR$(27) + "[H"     ' Cursor nach links oben
```

```
PRINT #1, "Hallo VT100!"
```

Im Anhang findest du eine Übersicht der wichtigsten ASCII-Zeichen sowie der gebräuchlichsten VT100-/ANSI-Steuersequenzen.

## 47.21. Testprogramm

```
SETPIN GP1, GP0, COM1
OPEN "COM1:9600" AS #1

PRINT #1, "Hallo vom PicoMite"

CLOSE #1
```

Wird nun ein passend eingestelltes Terminalprogramm gestartet, erscheint die gesendete Nachricht auf dem Bildschirm.

## 47.22. Praxisprojekt: Zwei PicoMites verbinden

Nun verbinden wir zwei MMBasic-Systeme miteinander.

## 47.23. Sender

```
SETPIN GP1, GP0, COM1
OPEN "COM1:9600" AS #1

DO
  PRINT #1, "Hallo"
  PAUSE 1000
LOOP
```

`PRINT` fügt am Ende der Ausgabe ein Zeilenende hinzu. Dadurch können wir auf der Empfängerseite erkennen, wann die Nachricht vollständig ist.

## 47.24. Empfänger

Für vollständige Zeilen sammeln wir die empfangenen Zeichen zunächst in einer Variablen:

```
SETPIN GP1, GP0, COM1
OPEN "COM1:9600" AS #1

puffer$ = ""

DO
```

```
  IF LOC(#1) > 0 THEN
    frei = 255 - LEN(puffer$)
    IF frei > 0 THEN
      anzahl = LOC(#1)
      IF anzahl > frei THEN anzahl = frei
      puffer$ = puffer$ + INPUT$(anzahl, #1)
    ENDIF

    pos = INSTR(puffer$, CHR$(10))
    DO WHILE pos > 0
      text$ = LEFT$(puffer$, pos - 1)

      IF RIGHT$(text$, 1) = CHR$(13) THEN
        text$ = LEFT$(text$, LEN(text$) - 1)
      ENDIF

      PRINT text$
      puffer$ = MID$(puffer$, pos + 1)
      pos = INSTR(puffer$, CHR$(10))
    LOOP
  ENDIF
LOOP
```

Der Empfänger zeigt nun jede Sekunde:

```
Hallo
```

an.

Dieses Beispiel zeigt bereits einen wichtigen Unterschied zwischen einer UART und einem Nachrichtenprotokoll: Die UART liefert Zeichen, während unser Programm daraus vollständige, durch ein Zeilenende getrennte Nachrichten bildet.

Da `puffer$` eine normale Stringvariable ist, achten wir darauf, nie mehr als 255 Zeichen darin zu speichern. Die innere `DO WHILE`-Schleife verarbeitet außerdem alle bereits vollständig empfangenen Zeilen und nicht nur die erste.

Für kurze Befehle und Textzeilen ist dieses Verfahren gut geeignet. Bei größeren Datenblöcken verwenden wir später Long Strings.

# 47.25. Ein einfacher Chat

Interessanter wird es, wenn beide Seiten senden und empfangen können.

Auf einer Seite könnte beispielsweise folgender Sender laufen:

```
SETPIN GP1, GP0, COM1
OPEN "COM1:9600" AS #1
```

```
DO
  INPUT "Text"; text$
  PRINT #1, text$
LOOP
```

Die andere Seite kann die Nachrichten mit dem zuvor gezeigten Empfangsverfahren auslesen.

Für einen echten Chat müssten beide Programme gleichzeitig Tastatureingaben und serielle Daten behandeln. Das Grundprinzip bleibt jedoch dasselbe: gesendete Zeilen werden durch ein Zeilenende voneinander getrennt.

## 47.26. Nachrichten strukturieren

Bei größeren Projekten empfiehlt es sich, feste Nachrichtenformate zu verwenden.

Beispiele:

```
TEMP:23.5
LED:ON
LED:OFF
```

Das empfangende Programm kann anhand des Textes vor dem Doppelpunkt erkennen, welche Art von Nachricht empfangen wurde.

Solche einfachen Textprotokolle haben einen großen Vorteil: Sie lassen sich mit einem Terminalprogramm leicht beobachten und testen.

## 47.27. Praxisprojekt: Fernsteuerung einer LED

Der Sender überträgt einen Befehl:

```
SETPIN GP1, GP0, COM1
OPEN "COM1:9600" AS #1

PRINT #1, "LED:ON"
```

Beim Empfänger verwenden wir GP2 als Ausgang. Dieser Pin ist beim PicoMite HDMI nicht für die HDMI-Ausgabe reserviert.

```
SETPIN GP1, GP0, COM1
SETPIN GP2, DOUT
OPEN "COM1:9600" AS #1

puffer$ = ""
```

```
DO
  IF LOC(#1) > 0 THEN
    frei = 255 - LEN(puffer$)
    IF frei > 0 THEN
      anzahl = LOC(#1)
      IF anzahl > frei THEN anzahl = frei
      puffer$ = puffer$ + INPUT$(anzahl, #1)
    ENDIF

    pos = INSTR(puffer$, CHR$(10))
    DO WHILE pos > 0
      text$ = LEFT$(puffer$, pos - 1)

      IF RIGHT$(text$, 1) = CHR$(13) THEN
        text$ = LEFT$(text$, LEN(text$) - 1)
      ENDIF

      IF text$ = "LED:ON" THEN
        PIN(GP2) = 1
      ELSEIF text$ = "LED:OFF" THEN
        PIN(GP2) = 0
      ENDIF

      puffer$ = MID$(puffer$, pos + 1)
      pos = INSTR(puffer$, CHR$(10))
    LOOP
  ENDIF
LOOP
```

Damit kann ein Mikrocontroller einen anderen fernsteuern.

# 47.28. Wenn 255 Zeichen nicht mehr reichen

In Kapitel 12 haben wir bereits gesehen, dass normale Zeichenketten in MMBasic maximal 255 Zeichen enthalten können. Damals haben wir Long Strings angekündigt – nun gibt es einen praktischen Grund, sie einzusetzen.

Bei der seriellen Kommunikation können größere Datenmengen eintreffen. Ein UART-Empfangspuffer kann beispielsweise 1024 Byte groß sein. Sollen solche Daten als zusammenhängender Text gespeichert werden, reicht eine normale Stringvariable nicht mehr aus.

Für diesen Fall stellt MMBasic sogenannte **Long Strings** zur Verfügung.

Long Strings können wesentlich mehr als 255 Zeichen aufnehmen. Ihre maximale Größe wird im Wesentlichen durch den dafür reservierten Speicher und den verfügbaren RAM begrenzt.

### 47.28.1. Long Strings sind Integer-Arrays

Eine Besonderheit fällt sofort auf: Long Strings werden nicht als normale Stringvariablen mit `$` angelegt.

Stattdessen reservieren wir ein eindimensionales Integer-Array als Speicherbereich.

Für einen Long String mit maximal 2048 Zeichen können wir beispielsweise schreiben:

```
CONST MAXLEN=2048
DIM INTEGER EMPFANG(MAXLEN/8)
```

Ein Integer belegt in MMBasic acht Byte. Deshalb wird die gewünschte maximale Zeichenanzahl durch acht geteilt.

Der neu angelegte Long String ist zunächst leer.

Bei den Long-String-Befehlen wird der Arrayname mit leeren Klammern angegeben:

```
EMPFANG()
```

### 47.28.2. Text anhängen

Mit `LONGSTRING APPEND` kann ein normaler String an einen Long String angehängt werden:

```
LONGSTRING APPEND EMPFANG(),text$
```

Das ist für die serielle Kommunikation besonders praktisch. Kleine Datenblöcke können zunächst mit `INPUT$()` gelesen und anschließend an den Long String angehängt werden.

### 47.28.3. Länge eines Long Strings

Die Funktion `LLEN()` entspricht bei Long Strings ungefähr der bekannten Funktion `LEN()` für normale Strings:

```
PRINT LLEN(EMPFANG())
```

Sie liefert die aktuelle Länge des Long Strings.

### 47.28.4. Long String ausgeben

Mit `LONGSTRING PRINT` kann der komplette Inhalt ausgegeben werden:

```
LONGSTRING PRINT EMPFANG()
```

Ein Long String kann auch über eine geöffnete Datei oder Schnittstelle ausgegeben werden, indem die entsprechende Dateinummer angegeben wird.

### 47.28.5. Teile wieder als normalen String lesen

Oft soll nicht der gesamte Long String auf einmal verarbeitet werden.

Mit `LGETSTR$()` kann ein Teil daraus als normaler String gelesen werden:

```
teil$=LGETSTR$(EMPFANG(),1,100)
```

Hier werden 100 Zeichen ab Position 1 gelesen.

Da das Ergebnis wieder ein normaler String ist, darf der zurückgegebene Teil höchstens 255 Zeichen lang sein.

### 47.28.6. Weitere Long-String-Funktionen

MMBasic stellt noch zahlreiche weitere Befehle und Funktionen für Long Strings bereit. Dazu gehören beispielsweise:

| Befehl oder Funktion | Aufgabe |
|---|---|
| `LONGSTRING CLEAR` | Long String leeren |
| `LONGSTRING COPY` | Long String kopieren |
| `LONGSTRING CONCAT` | zwei Long Strings zusammenfügen |
| `LONGSTRING LEFT` | linken Teil übernehmen |
| `LONGSTRING MID` | Teil aus der Mitte übernehmen |
| `LONGSTRING RIGHT` | rechten Teil übernehmen |
| `LONGSTRING LCASE` | in Kleinbuchstaben umwandeln |
| `LONGSTRING UCASE` | in Großbuchstaben umwandeln |
| `LINSTR()` | Text innerhalb eines Long Strings suchen |
| `LGETBYTE()` | ein einzelnes Byte lesen |

Für unsere ersten Anwendungen benötigen wir davon nur wenige. Die vollständige Beschreibung findest du im PicoMite-Handbuch.

## 47.29. Praxisbeispiel: Einen großen Datenblock empfangen

Nun verbinden wir Long Strings mit der seriellen Kommunikation.

Das folgende Beispiel sammelt bis zu 2048 empfangene Zeichen in einem Long String. Die UART selbst verwendet dabei einen Empfangspuffer von 1024 Byte.

```
CONST MAXLEN=2048
DIM INTEGER EMPFANG(MAXLEN/8)

SETPIN GP1,GP0,COM1
OPEN "COM1:9600,1024" AS #1

PRINT "Warte auf serielle Daten ..."
PRINT "Beenden mit einer Taste."
PRINT

DO
  IF LOC(#1)>0 THEN
    anzahl=LOC(#1)

    DO WHILE anzahl>0
      block=anzahl
      IF block>255 THEN block=255

      text$=INPUT$(block,#1)

      IF LLEN(EMPFANG())+LEN(text$)<=MAXLEN THEN
        LONGSTRING APPEND EMPFANG(),text$
      ELSE
        PRINT "Long-String-Puffer ist voll."
        EXIT DO
      ENDIF

      anzahl=LOC(#1)
    LOOP
  ENDIF
LOOP WHILE INKEY$=""

CLOSE #1

PRINT
PRINT "Empfangen: ";LLEN(EMPFANG());" Zeichen"
PRINT
LONGSTRING PRINT EMPFANG()
```

Warum lesen wir die Daten in Blöcken von höchstens 255 Zeichen ein?

`INPUT$()` liefert einen normalen String zurück. Dieser kann maximal 255 Zeichen enthalten. Jeder kleine Block wird deshalb anschließend mit `LONGSTRING APPEND` an den großen Speicherbereich angehängt.

So können wir Datenmengen verarbeiten, die insgesamt deutlich größer als 255 Zeichen sind.

Der UART-Empfangspuffer und der Long String sind zwei verschiedene Dinge.

Der Empfangspuffer gehört zur seriellen Schnittstelle und nimmt Zeichen auf, bis das BASIC-Programm sie abholt. Der Long String ist dagegen ein Speicherbereich unseres Programms, in dem wir die abgeholten Daten anschließend sammeln und verarbeiten.

Long Strings sind nicht nur für serielle Schnittstellen interessant. Sie eignen sich besonders für größere Datenmengen, wie sie beispielsweise auch bei WLAN- und Internetanwendungen auftreten können.

## 47.30. Serielle Interrupts

Bei Programmen, die nicht ständig den Empfangspuffer abfragen sollen, kann eine serielle Schnittstelle auch einen Interrupt auslösen.

Die vollständige `OPEN`-Syntax erlaubt unter anderem die Angabe einer Interrupt-Subroutine und einer Anzahl empfangener Zeichen, ab der der Interrupt ausgelöst werden soll.

Ein Beispiel aus der PicoMite-Syntax ist:

```
OPEN "COM2:19200, 1024, ComInt, 256" AS #5
```

Hier wird ein Empfangspuffer von 1024 Byte angelegt. Sobald mindestens 256 Zeichen empfangen wurden, wird die Subroutine `ComInt` aufgerufen.

Wie wir in Kapitel 45 gesehen haben, sollte eine Interrupt-Subroutine möglichst kurz bleiben. Typischerweise setzt sie nur ein Flag; die eigentliche Auswertung der empfangenen Daten erfolgt anschließend im Hauptprogramm.

## 47.31. Retrocomputer und serielle Kommunikation

Viele klassische Computer besitzen serielle Schnittstellen oder können mit passenden Interfaces erweitert werden.

Beispiele:

- AC1
- Z1013
- C64 mit entsprechendem Interface
- Apple II
- CP/M-Rechner
- Siemens PGs
- Eigenbaucomputer

Ein PicoMite kann dadurch als moderne Erweiterung dienen.

## 47.32. Praktische Anwendungen

Beispielsweise:

- Programme übertragen
- Messdaten speichern
- SD-Karten-Zugriff ermöglichen
- Terminalfunktionen bereitstellen
- Sensorwerte anzeigen

Dadurch lassen sich moderne und historische Systeme miteinander verbinden.

## 47.33. Praxisprojekt: Zeitmessung für einen Commodore CBM

In einem Retrocomputer-Forum stellte ein Leser die folgende Frage:

> Ich brauche demnächst die Anbindung eines Pi Pico an einen Commodore CBM zur Zeiterfassung von RC-Fahrzeugen. Die Zeit soll mit einer Lichtschranke gemessen werden.
>
> Der CBM ist dafür eigentlich zu langsam, der Pi Pico kann dagegen problemlos Hundertstelsekunden oder sogar Millisekunden erfassen.
>
> Die Daten sollen anschließend über eine serielle Verbindung an den CBM übertragen werden.

Dieses Beispiel zeigt sehr schön, wie sich moderne Mikrocontroller und klassische Computer sinnvoll ergänzen können.

Der PicoMite übernimmt die zeitkritische Erfassung und Zeitmessung, während der Commodore CBM die Ergebnisse anzeigt, speichert oder weiterverarbeitet.

## 47.34. Aufgaben sinnvoll verteilen

Beide Computer besitzen unterschiedliche Stärken.

Der Commodore CBM bietet:

- komfortable BASIC-Programmierung
- Tastatur und Bildschirm
- einfache Datenauswertung

- Möglichkeiten zur Speicherung der Ergebnisse

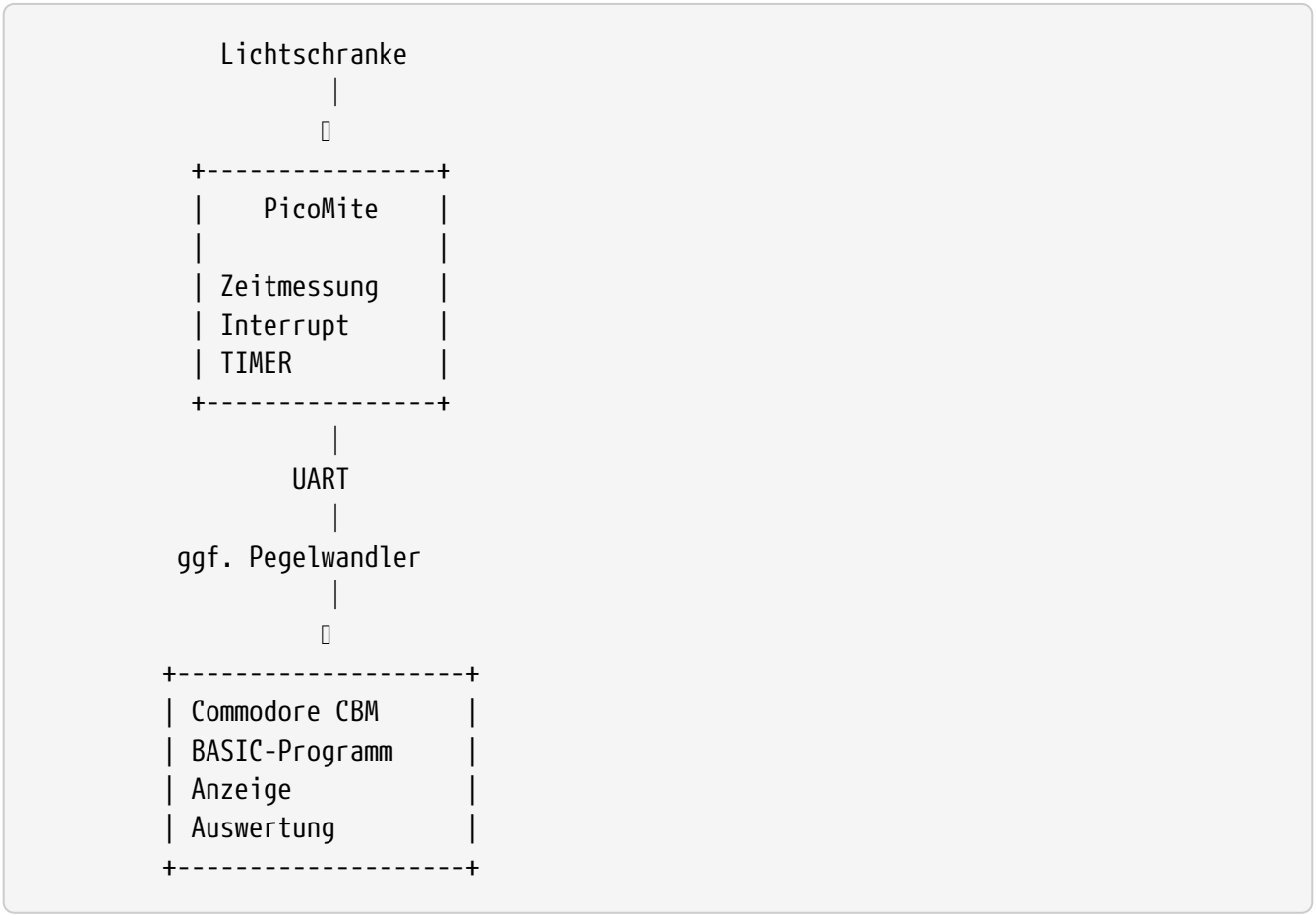
Der PicoMite bietet dagegen:

- einen schnellen Mikrocontroller
- präzise Timer
- Interrupts
- moderne Ein- und Ausgänge

Anstatt den CBM durch einen modernen Rechner zu ersetzen, übernimmt der PicoMite lediglich die Aufgabe, für die er besser geeignet ist.

# 47.35. Aufbau der Schaltung

Der prinzipielle Aufbau sieht folgendermaßen aus:




Sobald das Fahrzeug die Lichtschranke durchfährt, registriert der PicoMite das Ereignis und bestimmt die verstrichene Zeit.

Anschließend wird das Ergebnis über die serielle Verbindung an den Commodore übertragen.

# 47.36. Warum übernimmt der CBM die Zeitmessung nicht selbst?

Natürlich könnte man versuchen, die Zeit direkt im BASIC-Programm des CBM zu messen.

Allerdings arbeitet BASIC interpretiert. Während der Rechner gleichzeitig Tastatur, Bildschirm und Programm bearbeitet, kann zusätzliche und nicht immer gleich große Verzögerung entstehen.

Für viele Anwendungen reicht diese Genauigkeit völlig aus. Bei einer Rennstrecke oder Lichtschranke möchte man jedoch möglichst reproduzierbare Messwerte erhalten.

Der PicoMite kann ein Eingangssignal per Interrupt erfassen, ohne dass das Hauptprogramm den Eingang ständig abfragen muss. MMBasic prüft Interrupts jeweils nach Abschluss eines Befehls. Viele Befehle werden sehr schnell ausgeführt, bei länger dauernden Hardwareoperationen kann sich die Reaktion jedoch verzögern.

MMBasic-Interrupts sind nicht hardwareseitig zwischengespeichert. Sehr kurze Eingangssignale können deshalb unter ungünstigen Umständen verpasst werden. Ein Signal einer Lichtschranke sollte daher lange genug anliegen oder bei Bedarf hardwareseitig verlängert werden.

Die Interrupt-Subroutine sollte außerdem möglichst kurz bleiben. Typischerweise wird dort nur die Zeit erfasst und ein Merker gesetzt. Die eigentliche Verarbeitung erfolgt anschließend im Hauptprogramm.

# 47.37. Die serielle Verbindung zum Retrocomputer

Der RP2040 beziehungsweise RP2350 besitzt bereits integrierte UARTs. Für eine Verbindung zu einem anderen Gerät mit kompatiblen 3,3-V-Logikpegeln ist deshalb kein zusätzlicher UART-Baustein erforderlich.

Besitzt der Zielcomputer jedoch eine echte RS-232-Schnittstelle, müssen die unterschiedlichen elektrischen Pegel angepasst werden.

Dafür eignet sich beispielsweise ein `MAX3232`:

```
PicoMite (3,3-V-UART)
          |
          ▯
       MAX3232
          |
          ▯
        RS-232
          |
          ▯
     Retrocomputer
```

Verfügt der Retrocomputer stattdessen über eine Logikschnittstelle, beispielsweise an einem entsprechend beschalteten User-Port, muss die konkrete Schaltung geprüft werden. Entscheidend ist, dass Pegel und Signalpolarität zu den 3,3-V-UART-Signalen des PicoMite passen.

## 47.38. Ein einfaches Kommunikationsprotokoll

Der PicoMite muss dem Commodore lediglich die gemessene Zeit übertragen.

Eine einfache Textnachricht genügt bereits.

Beispiele:

```
TIME:2.37
```

oder

```
TIME:2371
```

Im zweiten Beispiel wird die Zeit in Millisekunden übertragen.

Das Zeilenende kennzeichnet jeweils das Ende einer Nachricht.

Textbasierte Protokolle besitzen den Vorteil, dass sie leicht lesbar und einfach mit einem Terminalprogramm zu testen sind.

## 47.39. Das BASIC-Programm auf dem Commodore

Der Commodore muss die empfangene Zeichenkette lediglich einlesen und auswerten.

Das Programm könnte anschließend beispielsweise folgende Ausgabe erzeugen:

```
Runde 1 : 2.371 s
Runde 2 : 2.294 s
Runde 3 : 2.415 s
```

Die eigentliche Zeitmessung erfolgt vollständig auf dem PicoMite.

## 47.40. Erweiterungsmöglichkeiten

Aus dieser einfachen Idee lassen sich viele interessante Projekte entwickeln.

Zum Beispiel:

- Rundenzähler
- Bestzeiten speichern

- mehrere Lichtschranken auswerten
- Fehlstarts erkennen
- Ziel- und Zwischenzeiten messen
- Ergebnisse auf SD-Karte speichern
- Live-Anzeige auf einem zweiten Display

## 47.41. Mehr als nur ein Commodore

Das gleiche Prinzip funktioniert selbstverständlich auch mit vielen anderen klassischen Computern.

Zum Beispiel:

- AC1
- Z1013
- KC85
- Apple II
- CP/M-Rechner
- Siemens PG
- Eigenbaucomputer mit serieller Schnittstelle

Der Retrocomputer übernimmt dabei die Benutzeroberfläche, während der PicoMite zeitkritische Aufgaben erledigt.

Viele Retrocomputer besitzen bereits eine serielle Schnittstelle oder können mit einem passenden Interface nachgerüstet werden.

Vor dem Anschluss muss immer geprüft werden, welche elektrischen Pegel die jeweilige Schnittstelle verwendet. Eine UART mit Logikpegeln ist nicht automatisch eine RS-232-Schnittstelle.

## 47.42. Tipps aus der Praxis

- Sender und Empfänger müssen dieselbe Baudrate und dasselbe Datenformat verwenden.
- TX und RX werden gekreuzt verbunden.
- Vergiss die gemeinsame Masse nicht.
- Beginne mit 9600 Baud.
- Verwende für den PicoMite nur geeignete 3,3-V-UART-Signale.
- Verwende einfache, klar begrenzte Textnachrichten.
- Mit `LOC()` kannst du prüfen, wie viele Zeichen im Empfangspuffer warten.
- Mit `INPUT$()` lassen sich die verfügbaren Zeichen auslesen, ohne auf neue Daten zu warten.

- Für Datenmengen über 255 Zeichen können Long Strings verwendet werden.

## 47.43. Probier’s selbst!

1. Verbinde PicoMite und PC über einen geeigneten USB-UART-Adapter.
2. Sende eine Textnachricht.
3. Lies empfangene Zeichen mit `LOC()` und `INPUT$()` ein.
4. Verbinde zwei PicoMites miteinander.
5. Übertrage einfache Befehle wie `LED:ON` und `LED:OFF`.
6. Entwickle ein eigenes kleines Textprotokoll.
7. Empfange einen Datenblock mit mehr als 255 Zeichen und speichere ihn in einem Long String.

## 47.44. Zusammenfassung

In diesem Kapitel hast du gelernt:

- wie serielle Kommunikation funktioniert
- was eine UART ist
- warum UART und RS-232 nicht dasselbe sind
- wie `COM1` und `COM2` mit `SETPIN` konfiguriert werden
- Daten über eine serielle Schnittstelle zu senden
- empfangene Daten mit `LOC()` und `INPUT$()` auszulesen
- einen PicoMite mit einem PC zu verbinden
- zwei MMBasic-Systeme miteinander kommunizieren zu lassen
- einfache Kommunikationsprotokolle zu entwickeln
- größere Datenmengen mit Long Strings zu speichern und zu verarbeiten
- wie ein PicoMite zeitkritische Aufgaben für einen Retrocomputer übernehmen kann

Serielle Schnittstellen gehören zu den wichtigsten Werkzeugen der Elektronik und ermöglichen den Datenaustausch zwischen unterschiedlichsten Geräten.

## 47.45. Ein Blick in die Zukunft

Serielle Kommunikation bildet die Grundlage vieler weiterer Anwendungen.

Dazu gehören beispielsweise:

- Bluetooth-Module
- WLAN-Module
- GPS-Empfänger

- Funkmodule
- industrielle Geräte

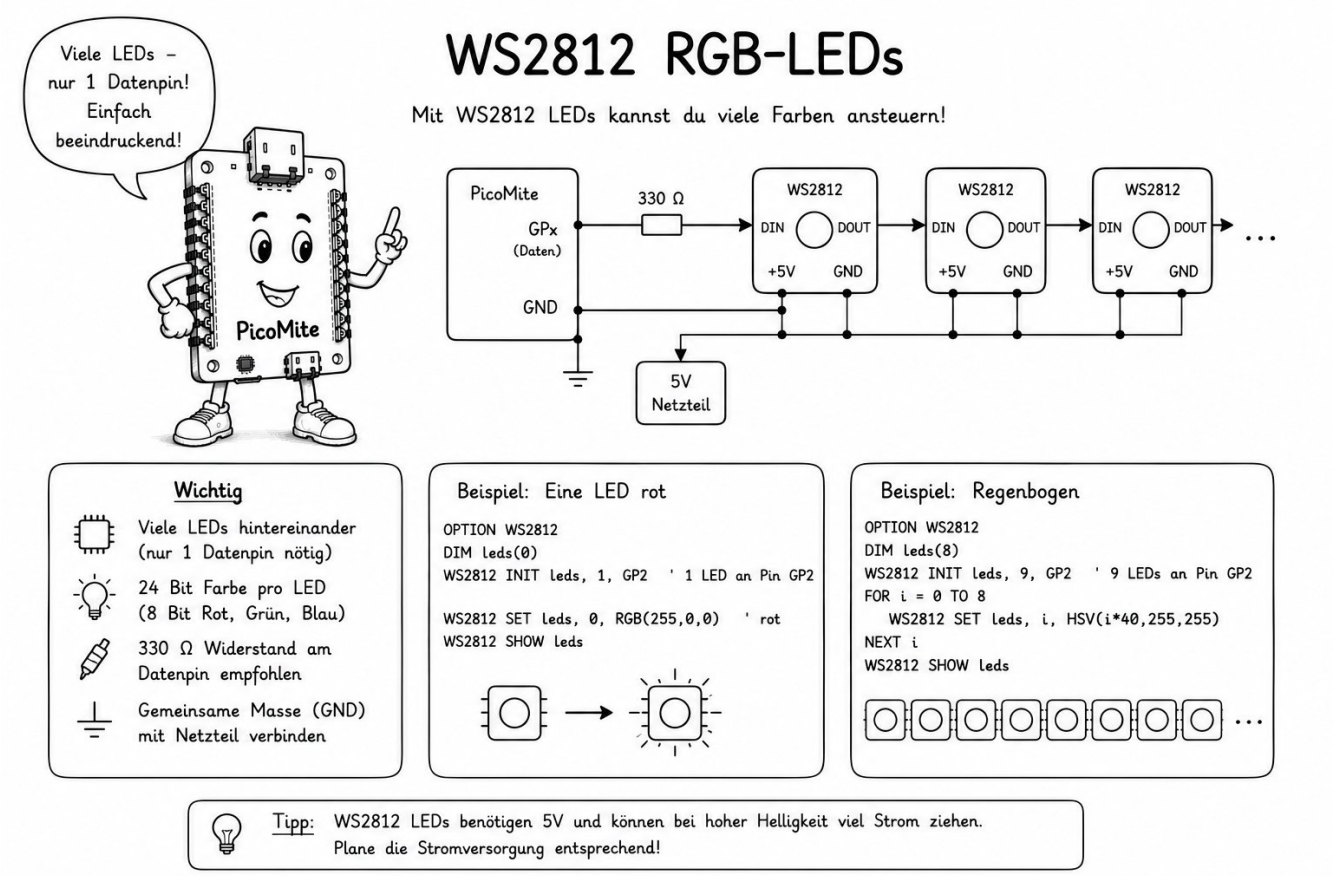
Wer UART verstanden hat, besitzt damit eine wichtige Grundlage für viele weitere Kommunikationsprojekte.

## 47.46. Programmbeispiel

Die vollständigen BASIC-Programme findest du hier:

- 47_seriell_senden.bas
- 47_seriell_empfangen.bas
- 47_longstring_empfangen.bas

# Kapitel 48. WS2812 RGB-LEDs




## 48.1. Das lernst du

WS2812-LEDs verbinden eine RGB-LED mit einem kleinen integrierten Controller. Dadurch lassen sich viele LEDs hintereinander schalten und trotzdem über nur eine einzige Datenleitung ansteuern.

Der PicoMite unterstützt diese LEDs direkt mit dem Befehl `WS2812`. Die zeitkritische Erzeugung des Datensignals übernimmt dabei die Firmware.

In diesem Kapitel lernst du:

- wie WS2812-LEDs aufgebaut sind
- wie mehrere LEDs zu einer Kette verbunden werden
- wie du eine WS2812-Kette an den PicoMite anschließt
- wie der Befehl `WS2812` verwendet wird
- wie Farben mit `RGB()` festgelegt werden
- wie mehrere LEDs über ein Integer-Array unterschiedliche Farben erhalten
- wie einfache Lichtanimationen entstehen
- worauf du bei Versorgungsspannung, Strombedarf und Signalpegel achten solltest

## 48.2. Was sind WS2812-LEDs?

Eine WS2812 ist mehr als eine gewöhnliche Leuchtdiode. In ihrem Gehäuse befinden sich:

- eine RGB-LED mit den Farbkanälen Rot, Grün und Blau
- ein integrierter Controller zur Auswertung und Weitergabe der Farbdaten

Jede LED übernimmt aus dem eintreffenden Datenstrom die für sie bestimmten Farbinformationen. Die restlichen Daten gibt sie an die nächste LED weiter.

Eine Kette sieht vereinfacht so aus:

```
PicoMite
    |
    v
  DIN
+------+ DOUT    DIN +------+ DOUT    DIN +------+
| LED1 |------------>| LED2 |------------>| LED3 | ---> ...
+------+             +------+             +------+
```

Dadurch benötigt der PicoMite für die komplette LED-Kette nur einen GPIO-Ausgang.

Der PicoMite unterstützt mit dem Befehl `WS2812` Ketten mit **1 bis 256 LEDs**.

## 48.3. Anschluss

Für die Beispiele in diesem Kapitel verwenden wir `GP5` als Datenleitung. Falls dieser Pin bei deinem Aufbau bereits verwendet wird, kannst du einen anderen geeigneten GPIO-Ausgang verwenden.

Ein einfacher Anschluss sieht so aus:

```
PicoMite                        WS2812 / WS2812B

GP5 --------------------------> DIN
GND --------------------------- GND
5-V-Versorgung ----------------- +5 V
```

Bei mehreren LEDs wird jeweils `DOUT` einer LED mit `DIN` der nächsten LED verbunden.

Bei wenigen LEDs kann die Versorgung je nach Aufbau aus einer vorhandenen 5-V-Versorgung erfolgen. Größere LED-Ketten benötigen dagegen ein ausreichend dimensioniertes externes Netzteil.

Achte auf die Richtung des Datenflusses. Viele LED-Streifen besitzen einen Pfeil, der von `DIN` nach `DOUT` zeigt. Das Datensignal des PicoMite muss am `DIN`-Anschluss der ersten LED eingespeist werden.

Wird für die LEDs ein separates 5-V-Netzteil verwendet, müssen dessen Masse und die Masse des PicoMite miteinander verbunden werden. Ohne eine gemeinsame Masse besitzt das Datensignal keinen eindeutigen Bezug.

## 48.4. Stromversorgung

WS2812-LEDs können bei hoher Helligkeit einen erheblichen Strom aufnehmen. Als konservative Abschätzung werden häufig bis zu etwa 60 mA pro RGB-LED bei voll eingeschaltetem Rot-, Grün- und Blaukanal angesetzt.

Für 100 LEDs ergibt sich damit theoretisch:

```
100 x 60 mA = 6000 mA = 6 A
```

In der Praxis hängt der tatsächliche Strom vom verwendeten LED-Typ, der eingestellten Helligkeit und den dargestellten Farben ab. Für die Planung der Stromversorgung ist eine großzügige Reserve dennoch sinnvoll.

Größere LED-Streifen sollten nicht einfach aus einem GPIO-Pin oder einer dafür nicht ausgelegten Versorgung des PicoMite gespeist werden. Verwende für größere LED-Ketten ein ausreichend dimensioniertes externes Netzteil und verbinde die Masse des Netzteils mit `GND` des PicoMite.

Als praktische Schutzmaßnahme wird bei längeren Leitungen häufig zusätzlich ein kleiner Serienwiderstand in der Datenleitung nahe der ersten LED verwendet. Auch ein ausreichend dimensionierter Stützkondensator an der Versorgung der LED-Kette kann Spannungsspitzen beim Einschalten abfangen.

## 48.5. 3,3-V-Datensignal und 5-V-LEDs

Der PicoMite liefert an seinen GPIO-Ausgängen ein 3,3-V-Logiksignal. WS2812-LEDs werden dagegen häufig mit 5 V versorgt.

Je nach verwendetem LED-Typ und Versorgungsspannung kann ein 3,3-V-Datensignal zu niedrig sein, um von der ersten LED zuverlässig als logisch HIGH erkannt zu werden. Das kann sich beispielsweise durch flackernde LEDs oder eine unzuverlässige Datenübertragung bemerkbar machen.

Das PicoMite-Handbuch weist ausdrücklich auf diese mögliche Pegelproblematik hin. Eine robuste Lösung ist ein geeigneter Pegelwandler zwischen PicoMite und der ersten LED. Manche WS2812B-Varianten arbeiten auch direkt zuverlässig mit einem 3,3-V-Datensignal.

Wenn eine LED-Kette mit 5 V versorgt wird und nicht zuverlässig reagiert, solltest du zuerst die gemeinsame Masse und die Stromversorgung prüfen. Bleibt das Problem bestehen, ist eine saubere Pegelanpassung des Datensignals die zuverlässigste Lösung.

## 48.6. Der Befehl WS2812

Die PicoMite-Firmware besitzt eine eingebaute Unterstützung für WS2812- und verwandte LEDs. Der Befehl lautet:

```
WS2812 type, pin, nbr, value%[()]
```

Die Parameter haben folgende Bedeutung:

| Parameter | Bedeutung |
|---|---|
| type | Typ der verwendeten LED |
| pin | GPIO-Pin, an dem der Dateneingang der ersten LED angeschlossen ist |
| nbr | Anzahl der LEDs in der Kette, von 1 bis 256 |
| value% bzw. value%() | Farbwert für eine einzelne LED oder Integer-Array mit den Farbwerten mehrerer LEDs |

Bevor WS2812 verwendet wird, muss der betreffende GPIO-Pin als digitaler Ausgang konfiguriert werden:

```
SETPIN GP5, DOUT
```

## 48.7. Unterstützte LED-Typen

Mit dem ersten Parameter des WS2812-Befehls wird der LED-Typ angegeben:

| Typ | Bedeutung |
|---|---|
| 0 | ursprünglicher WS2812 |
| B | WS2812B |
| S | SK6812 |
| W | SK6812W mit zusätzlichem Weißkanal (RGBW) |

Für einen WS2812B-Streifen lautet der Anfang eines Befehls also beispielsweise:

```
WS2812 B, GP5, ...
```

In den folgenden Beispielen verwenden wir deshalb den Typ B.

## 48.8. Farben mit RGB()

Für WS2812, WS2812B und SK6812 erwartet MMBasic die Farben im normalen RGB888-Format. Jeder der drei Farbkanäle besitzt einen Wert von 0 bis 255.

| RGB-Werte | Farbe |
|---|---|
| `255, 0, 0` | Rot |
| `0, 255, 0` | Grün |
| `0, 0, 255` | Blau |
| `255, 255, 0` | Gelb |
| `255, 0, 255` | Magenta |
| `0, 255, 255` | Cyan |
| `255, 255, 255` | Weiß |
| `0, 0, 0` | Aus |

Mit der bereits bekannten Funktion `RGB()` lassen sich daraus die passenden Integer-Farbwerte erzeugen:

```
rot% = RGB(255, 0, 0)
gruen% = RGB(0, 255, 0)
blau% = RGB(0, 0, 255)
```

MMBasic kennt außerdem Farbnamen:

```
rot% = RGB(RED)
gruen% = RGB(GREEN)
blau% = RGB(BLUE)
```

Obwohl die WS2812-Familie die Farbdaten intern in einer vom normalen RGB-Schema abweichenden Reihenfolge übertragen kann, musst du dich darum im BASIC-Programm nicht kümmern. Für den `WS2812`-Befehl werden die Farben in MMBasic ganz normal als RGB888-Werte angegeben.

# 48.9. Eine einzelne LED ansteuern

Bei nur einer LED wird kein Array benötigt. Ein einzelner Integer-Farbwert genügt.

Das folgende Programm schaltet eine WS2812B zuerst rot, dann grün und anschließend blau:

```
SETPIN GP5, DOUT

farbe% = RGB(RED)
WS2812 B, GP5, 1, farbe%
PAUSE 1000

farbe% = RGB(GREEN)
WS2812 B, GP5, 1, farbe%
PAUSE 1000
```

```
farbe% = RGB(BLUE)
WS2812 B, GP5, 1, farbe%
PAUSE 1000

farbe% = RGB(BLACK)
WS2812 B, GP5, 1, farbe%
```

Nach jedem Aufruf von `WS2812` bleibt die zuletzt übertragene Farbe erhalten. Die LED muss also nicht fortlaufend mit Daten versorgt werden.

Auch das Beenden des BASIC-Programms schaltet die LEDs nicht automatisch aus. Sollen die LEDs beim Programmende dunkel werden, müssen sie zuvor noch einmal mit `RGB(BLACK)` angesteuert werden.

## 48.10. Mehrere LEDs ansteuern

Sollen mehrere LEDs unterschiedliche Farben erhalten, werden die Farbwerte in einem Integer-Array abgelegt.

Für fünf LEDs benötigen wir fünf Array-Elemente:

```
DIM farbe%(4)
```

Bei der üblichen Array-Basis 0 sind das die Elemente `farbe%(0)` bis `farbe%(4)`.

Jetzt können wir jeder LED eine eigene Farbe zuweisen:

```
DIM farbe%(4)

farbe%(0) = RGB(RED)
farbe%(1) = RGB(GREEN)
farbe%(2) = RGB(BLUE)
farbe%(3) = RGB(YELLOW)
farbe%(4) = RGB(CYAN)

SETPIN GP5, DOUT
WS2812 B, GP5, 5, farbe%()
```

Mit nur einem `WS2812`-Befehl werden alle fünf Farbwerte an die komplette LED-Kette übertragen.

Dimensioniere das Array passend zur Anzahl der LEDs. Für fünf LEDs werden fünf Farbwerte benötigt, für zehn LEDs zehn Farbwerte usw.

## 48.11. Helligkeit über die Farbwerte steuern

WS2812-LEDs besitzen beim `WS2812`-Befehl keinen zusätzlichen Helligkeitsparameter. Die wahrgenommene Helligkeit wird deshalb über die RGB-Werte bestimmt.

Volles Rot:

```
farbe% = RGB(255, 0, 0)
```

Deutlich dunkleres Rot:

```
farbe% = RGB(40, 0, 0)
```

Das gilt genauso für gemischte Farben. Ein dunkleres Weiß könnte zum Beispiel so aussehen:

```
farbe% = RGB(40, 40, 40)
```

Kleinere Werte reduzieren nicht nur die Helligkeit, sondern normalerweise auch den Stromverbrauch der LED-Kette.

## 48.12. Praxisprojekt: Ein einfaches Lauflicht

Jetzt verwenden wir ein Array und eine Schleife, um einen einzelnen Lichtpunkt über fünf LEDs laufen zu lassen.

```
CONST LEDS = 5
DIM leds%(LEDS - 1)

SETPIN GP5, DOUT

DO
  FOR pos = 0 TO LEDS - 1

    ' Alle LEDs ausschalten
    FOR i = 0 TO LEDS - 1
      leds%(i) = RGB(BLACK)
    NEXT i

    ' Aktuelle LED einschalten
    leds%(pos) = RGB(40, 0, 0)

    ' Komplette Kette aktualisieren
    WS2812 B, GP5, LEDS, leds%()

    PAUSE 120
  NEXT pos
```

```
LOOP
```

Die äußere Schleife sorgt dafür, dass das Lauflicht immer wieder von vorne beginnt. In jedem Durchgang werden zunächst alle LEDs ausgeschaltet und danach nur die LED an der aktuellen Position eingeschaltet.

Mit dem Wert

```
RGB(40, 0, 0)
```

verwenden wir bewusst ein gedämpftes Rot.

# 48.13. Mehrere LEDs gleichzeitig verändern

Da das Array den vollständigen Zustand der LED-Kette enthält, lassen sich auch komplexere Muster vorbereiten, bevor sie mit einem einzigen `WS2812`-Befehl übertragen werden.

Zum Beispiel:

```
leds%(0) = RGB(40, 0, 0)
leds%(1) = RGB(30, 10, 0)
leds%(2) = RGB(20, 5, 0)
leds%(3) = RGB(5, 0, 0)
leds%(4) = RGB(BLACK)

WS2812 B, GP5, 5, leds%()
```

Damit könnte beispielsweise der Anfang eines Lauflichts mit einem schwächer werdenden Schweif erzeugt werden.

Das Grundprinzip für Animationen lautet immer:

1. gewünschte Farbwerte im Array berechnen
2. das komplette Array mit `WS2812` übertragen
3. kurz warten
4. die Farbwerte für das nächste Bild verändern

# 48.14. Sanfte Farbwechsel

Abrupte Farbwechsel sind nicht immer gewünscht. Werden die RGB-Werte schrittweise verändert, entsteht ein weicher Übergang, ein sogenannter *Fade*. Eine einzelne LED kann zum Beispiel langsam heller rot werden:

```
SETPIN GP5, DOUT
```

```
FOR hell = 0 TO 100
  farbe% = RGB(hell, 0, 0)
  WS2812 B, GP5, 1, farbe%
  PAUSE 20
NEXT hell
```

Das gleiche Prinzip lässt sich auf mehrere Farbkanäle und auf komplette LED-Ketten übertragen.

## 48.15. Animationen flüssig gestalten

Zwischen zwei Bildern einer Animation wird meist eine kurze Pause eingefügt:

```
PAUSE 20
```

Wie groß die Pause sein sollte, hängt vom gewünschten Effekt ab. Ein Lauflicht kann mit 100 bis 200 Millisekunden gut erkennbar sein, während weiche Farbverläufe oft mit deutlich kleineren Pausen angenehmer wirken.

Da der PicoMite die zeitkritische Übertragung des WS2812-Protokolls selbst übernimmt, muss das BASIC-Programm keine exakten Mikrosekunden-Impulse erzeugen.

## 48.16. RGBW mit SK6812W

Der Typ `W` ist für SK6812W-LEDs mit einem zusätzlichen Weißkanal vorgesehen.

Dabei verwendet jedes Array-Element einen 32-Bit-RGBW-Wert im Bereich von `0` bis `&HFFFFFFFF`.

```
WS2812 W, pin, nbr, value%()
```

RGBW-LEDs behandeln wir in diesem Kapitel nicht weiter. Wichtig ist zunächst nur, dass du für diese LEDs den Typ `W` und einen RGBW-Wert statt eines normalen RGB888-Werts verwendest.

## 48.17. Praxisprojekt: Wandernder Regenbogen

Zum Abschluss verbinden wir Arrays, Farben, Schleifen und den `WS2812`-Befehl zu einer kleinen Animation. Acht WS2812B-LEDs zeigen einen wandernden Regenbogen.

Zunächst legen wir die Anzahl der LEDs und die Anzahl der verwendeten Farben fest:

```
CONST LEDS = 8
CONST FARBEN = 6

DIM leds%(LEDS - 1)
DIM palette%(FARBEN - 1)
```

Das Array `leds%()` enthält den aktuellen Zustand der LED-Kette. Im Array `palette%()` speichern wir dagegen unsere sechs Regenbogenfarben:

```
palette%(0) = RGB(40, 0, 0)      ' Rot
palette%(1) = RGB(40, 20, 0)     ' Orange
palette%(2) = RGB(40, 40, 0)     ' Gelb
palette%(3) = RGB(0, 40, 0)      ' Gruen
palette%(4) = RGB(0, 0, 40)      ' Blau
palette%(5) = RGB(30, 0, 40)     ' Violett
```

Die Farbwerte sind bewusst relativ klein gewählt. Dadurch leuchten die LEDs nicht mit voller Helligkeit und der Strombedarf bleibt beim Experimentieren geringer.

### 48.17.1. Den Regenbogen verschieben

Der entscheidende Teil der Animation steckt in nur wenigen Zeilen:

```
FOR i = 0 TO LEDS - 1
  farbindex = (i + offset) MOD FARBEN
  leds%(i) = palette%(farbindex)
NEXT i
```

Jede LED erhält eine Farbe aus der Palette. Der Ausdruck

```
(i + offset) MOD FARBEN
```

sorgt dafür, dass der Farbindex immer im gültigen Bereich von `0` bis `5` bleibt.

Erreicht der Wert `FARBEN`, beginnt er durch `MOD` wieder bei `0`.

Beispielsweise ergibt sich bei sechs Farben:

```
0 MOD 6 = 0
1 MOD 6 = 1
...
5 MOD 6 = 5
6 MOD 6 = 0
7 MOD 6 = 1
```

Damit lässt sich eine zyklische Folge besonders einfach programmieren.

Nachdem alle Farben vorbereitet wurden, wird die komplette LED-Kette aktualisiert:

```
WS2812 B, GP5, LEDS, leds%()
```

Anschließend verschieben wir den Regenbogen um eine Position:

```
offset = (offset + 1) MOD FARBEN
```

Auch hier sorgt `MOD` dafür, dass nach dem letzten Farbversatz wieder bei `0` begonnen wird.

Mit

```
PAUSE 150
```

bestimmen wir die Geschwindigkeit der Animation.

### 48.17.2. Programm mit einer Taste beenden

Die Animation läuft so lange, bis eine Taste gedrückt wird:

```
LOOP UNTIL INKEY$ <> ""
```

Danach sollten wir die LEDs ausdrücklich ausschalten. Wie wir bereits gesehen haben, behalten WS2812-LEDs ihren zuletzt übertragenen Zustand auch nach dem Ende des BASIC-Programms bei.

Deshalb setzen wir alle Elemente des Arrays auf Schwarz und übertragen den neuen Zustand noch einmal:

```
FOR i = 0 TO LEDS - 1
  leds%(i) = RGB(BLACK)
NEXT i

WS2812 B, GP5, LEDS, leds%()
```

Damit endet das Programm mit ausgeschalteten LEDs.

Ändere einmal `PAUSE 150` oder die Werte in `palette%()`. Schon mit kleinen Änderungen entstehen deutlich andere Farbeindrücke und Animationsgeschwindigkeiten.

## 48.18. Ideen für eigene Experimente

Mit dem gleichen Grundprinzip lassen sich viele weitere Effekte programmieren:

- ein Knight-Rider-Lauflicht mit nachleuchtendem Schweif
- ein pulsierender Farbverlauf
- zufällig funkelnde Sterne
- Statusanzeigen mit Grün, Gelb und Rot

- Modellbau- und Fahrzeugbeleuchtungen
- VU-Meter
- Spiel- und Effektbeleuchtungen
- Weihnachtsdekorationen

Für all diese Effekte bleibt das Verfahren gleich: Die gewünschten Farben werden im Array vorbereitet und anschließend gemeinsam an die LED-Kette übertragen.

## 48.19. Zusammenfassung

Der PicoMite macht die Ansteuerung von WS2812-LEDs besonders einfach. Die Firmware übernimmt das genaue Timing des seriellen LED-Protokolls, während das BASIC-Programm nur die gewünschten Farben festlegt.

Die wichtigsten Punkte sind:

- Der GPIO-Pin muss zuerst mit `SETPIN pin, DOUT` als digitaler Ausgang konfiguriert werden.
- `WS2812 type, pin, nbr, value%[()]` überträgt die Farbwerte an die LED-Kette.
- Unterstützt werden WS2812, WS2812B, SK6812 und SK6812W.
- Eine Kette darf 1 bis 256 LEDs enthalten.
- Eine einzelne LED kann mit einer einfachen Integer-Variablen angesteuert werden.
- Für mehrere LEDs wird ein Integer-Array mit den Farbwerten verwendet.
- Mit `MOD` lassen sich zyklische Farbfolgen und Animationen besonders einfach erzeugen.
- Beim Programmende müssen die LEDs bei Bedarf ausdrücklich mit `RGB(BLACK)` ausgeschaltet werden.
- WS2812, WS2812B und SK6812 verwenden im BASIC-Programm RGB888-Farbwerte.
- Bei 5-V-LED-Ketten müssen Stromversorgung, gemeinsame Masse und der 3,3-V-Signalpegel beachtet werden.

Mit Arrays, Schleifen und `RGB()` lassen sich bereits mit wenigen BASIC-Zeilen Lauflichter, Farbverläufe und viele andere Lichteffekte erzeugen.

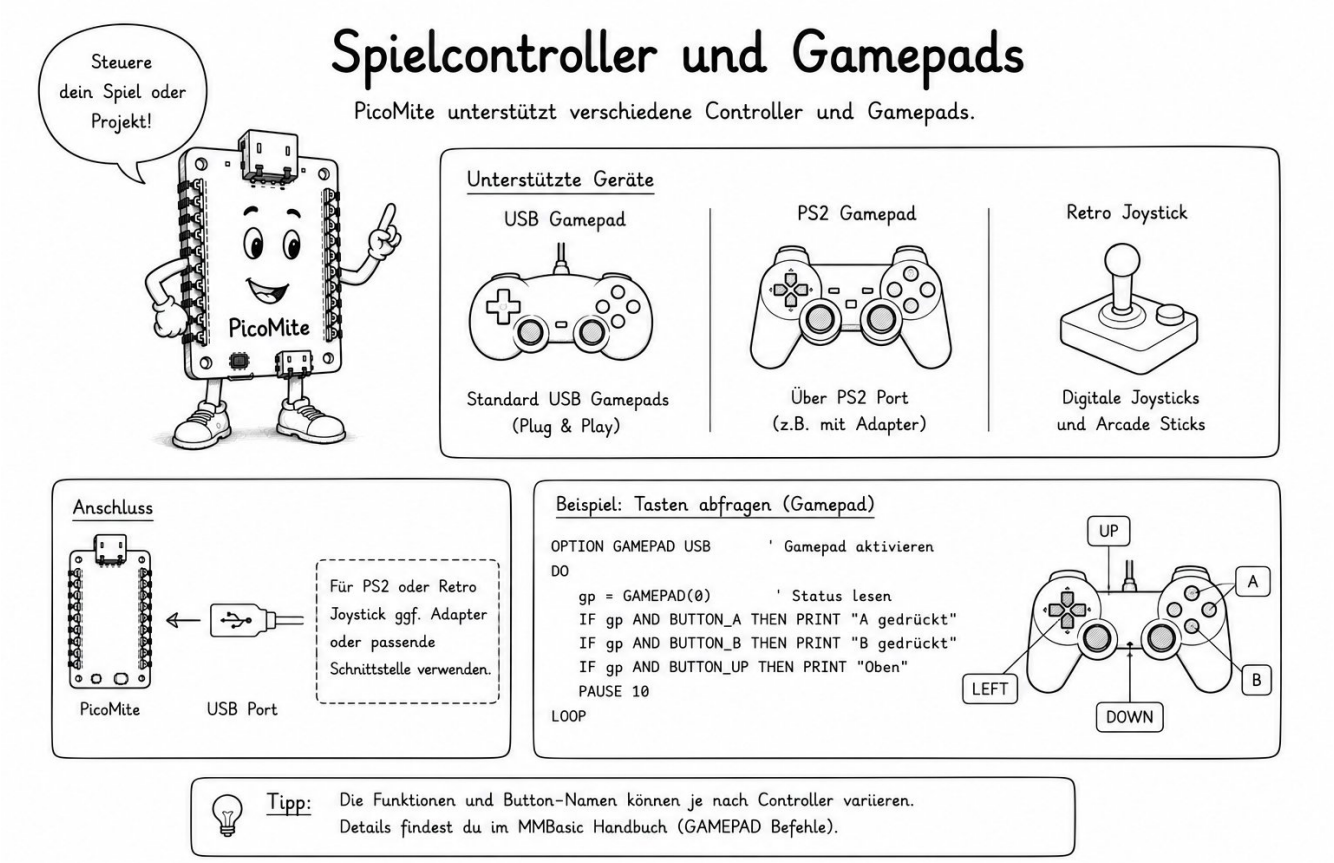
Im nächsten Kapitel verlassen wir für einen Moment die LEDs und beschäftigen uns mit einer ganz anderen Form der Ein- und Ausgabe: Spielcontrollern und Gamepads.

## 48.20. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 48_ws2812_regenbogen.bas

# Kapitel 49. Spielcontroller und Gamepads




## 49.1. Das lernst du

In den bisherigen Kapiteln wurden unsere Programme hauptsächlich über die Tastatur gesteuert. Das funktioniert für viele Anwendungen hervorragend, bei Spielen ist ein Gamepad jedoch oft die angenehmere Lösung.

Ein moderner Spielcontroller besitzt nicht nur Tasten, sondern meist auch Analogsticks, ein Steuerkreuz und Schultertasten. Einige Controller verfügen zusätzlich über Vibrationsmotoren, Bewegungssensoren oder eine farbige Leuchtanzeige.

In diesem Kapitel lernst du:

- welche Gamepads die USB-Versionen des PicoMite unterstützen
- wie USB-Geräte den Kanälen 1 bis 4 zugeordnet werden
- wie du mit `MM.INFO(USB n)` erkennst, welches Gerät angeschlossen ist
- wie du mit `DEVICE(GAMEPAD …)` Analogsticks, Trigger und Tasten ausliest
- wie eine Tastenbelegung als Bitmaske ausgewertet wird
- wie du eine Totzone für Analogsticks verwendest
- wie sich ein Gamepad für Grafikprogramme und Spiele einsetzen lässt

Im nächsten Kapitel lernen wir mit BLIT und Sprites weitere wichtige Techniken für bewegliche

Spielfiguren und grafische Objekte kennen.

## 49.2. Vom Joystick zum modernen Gamepad

Bereits in den 1980er-Jahren besaßen viele Heimcomputer einen Joystickanschluss.

Ein klassischer Joystick verfügte meist nur über vier Richtungen und einen Feuerknopf. Damit ließen sich erstaunlich viele Spiele bedienen.

Moderne Gamepads bieten deutlich mehr Möglichkeiten:

- Steuerkreuz
- zwei Analogsticks
- mehrere Aktionstasten
- Schultertasten und Trigger
- Start-, Select- oder Menütasten

Je nach Modell kommen weitere Funktionen wie Vibrationsmotoren, Gyroskop oder Beschleunigungssensor hinzu.

## 49.3. Gamepads am PicoMite

Die PicoMite-Firmware kann Tastatur und Maus entweder über PS/2 oder USB unterstützen. Welche Schnittstelle verfügbar ist, hängt von der verwendeten Firmware-Version ab.

Die Versionen mit USB-Unterstützung können zusätzlich USB-Gamepads verwenden. Laut PicoMite-Handbuch werden insbesondere folgende Controller unterstützt:

- PS3-Controller
- PS4-Controller
- SNES/Generic-USB-Gamepads

Ein USB-Gamepad muss im BASIC-Programm nicht geöffnet oder initialisiert werden. Wird es vom PicoMite erkannt, steht es unmittelbar zur Verfügung.

Bei Firmware-Versionen ohne USB-Unterstützung können alternativ Wii-Controller über I2C verwendet werden. Dafür gibt es die Befehle `WII CLASSIC` und `WII NUNCHUCK`. In diesem Kapitel beschäftigen wir uns jedoch mit USB-Gamepads.

## 49.4. USB-Kanäle

Der PicoMite verwaltet bis zu vier USB-Geräte. Jedes Gerät erhält einen Kanal von 1 bis 4.

Standardmäßig gilt folgende Zuordnung:

| Kanal | Gerät |
|---|---|
| 1 | Tastatur |
| 2 | Maus |
| 3 | erstes Gamepad |
| 4 | zweites Gamepad |

Diese Zuordnung ist die Voreinstellung. Werden mehrere Geräte gleicher Art angeschlossen, können freie Kanäle entsprechend anders belegt werden.

Mit einem USB-Hub lassen sich mehrere Geräte gleichzeitig anschließen, beispielsweise Tastatur, Maus und Gamepad. Insgesamt unterstützt die Firmware maximal vier USB-Geräte.

## 49.5. Angeschlossene USB-Geräte erkennen

Mit

```
MM.INFO(USB n)
```

kannst du feststellen, welches USB-Gerät Kanal `n` belegt.

Die Funktion liefert folgende Gerätecodes:

| Rückgabewert | Gerät |
|---|---|
| 0 | Kanal nicht belegt |
| 1 | Tastatur |
| 2 | Maus |
| 128 | PS4-Controller |
| 129 | PS3-Controller |
| 130 | SNES/Generic-Gamepad |

Um beispielsweise zu prüfen, was sich auf Kanal 3 befindet, kannst du an der Eingabeaufforderung eingeben:

```
PRINT MM.INFO(USB 3)
```

Wird dort beispielsweise `128` ausgegeben, hat der PicoMite einen PS4-Controller erkannt.

Das erste Gamepad befindet sich normalerweise auf Kanal 3. Trotzdem ist es sinnvoll, die Belegung mit `MM.INFO(USB n)` zu überprüfen, bevor du ein größeres Programm entwickelst.

Die folgenden Beispiele verwenden deshalb der Einfachheit halber Kanal 3. Falls

dein Controller einem anderen Kanal zugeordnet wurde, musst du die Konstante `Gamepad` entsprechend anpassen.

## 49.6. Controller mit GAMEPAD MONITOR untersuchen

Bevor wir ein eigenes BASIC-Programm schreiben, bietet der PicoMite eine sehr praktische Diagnosefunktion:

```
GAMEPAD MONITOR
```

Gib diesen Befehl ein, bevor du das Gamepad anschließt. Nach dem Anschließen zeigt der PicoMite Änderungen der vom Controller gelieferten Daten an, sobald du Tasten drückst oder Bedienelemente bewegst.

Das ist besonders hilfreich, wenn du einen neuen oder unbekannten Controller ausprobieren möchtest.

Für Gamepads, die nicht direkt von der Firmware unterstützt werden, existiert außerdem der Befehl `GAMEPAD CONFIGURE`. Damit lässt sich die Zuordnung der Daten im USB-Report zu den einzelnen Gamepad-Tasten konfigurieren. Da dieser Befehl viele Parameter benötigt, verwenden wir ihn in diesem Einsteigerkapitel nicht weiter.

## 49.7. Gamepad-Daten mit DEVICE() auslesen

Innerhalb eines BASIC-Programms werden die Daten des Controllers mit der Funktion

```
DEVICE(GAMEPAD channel, funct)
```

ausgelesen.

`channel` gibt den USB-Kanal des Gamepads an. Beim ersten Gamepad ist das normalerweise Kanal 3.

Mit `funct` bestimmst du, welcher Wert gelesen werden soll.

| Funktion | Bedeutung |
|---|---|
| `LX` | X-Achse des linken Analogsticks |
| `LY` | Y-Achse des linken Analogsticks |
| `RX` | X-Achse des rechten Analogsticks |
| `RY` | Y-Achse des rechten Analogsticks |

| Funktion | Bedeutung |
|---|---|
| `L` | linker analoger Trigger |
| `R` | rechter analoger Trigger |
| `B` | Bitmaske aller Tasten |
| `T` | ID-Code des Controllers |
| `GX`, `GY`, `GZ` | X-, Y- und Z-Achse des Gyroskops, sofern vom Controller unterstützt |
| `AX`, `AY`, `AZ` | X-, Y- und Z-Achse des Beschleunigungssensors, sofern vom Controller unterstützt |

So liest du beispielsweise den linken Analogstick aus:

```
lx = DEVICE(GAMEPAD 3, LX)
ly = DEVICE(GAMEPAD 3, LY)
```

Die Stellung des rechten Analogsticks erhältst du entsprechend mit:

```
rx = DEVICE(GAMEPAD 3, RX)
ry = DEVICE(GAMEPAD 3, RY)
```

## 49.8. Die Tasten als Bitmaske

Alle digitalen Tasten werden gemeinsam als Bitmaske geliefert:

```
buttons% = DEVICE(GAMEPAD 3, B)
```

Jedes Bit steht für eine bestimmte Taste. Ist ein Bit gesetzt, ist die zugehörige Taste gedrückt.

| Bit | Taste |
|---|---|
| 0 | R / R1 |
| 1 | Start / Options |
| 2 | Home |
| 3 | Select / Share |
| 4 | L / L1 |
| 5 | Steuerkreuz unten |
| 6 | Steuerkreuz rechts |
| 7 | Steuerkreuz oben |
| 8 | Steuerkreuz links |
| 9 | R2 |

| Bit | Taste |
|---|---|
| 10 | X / Triangle |
| 11 | A / Circle |
| 12 | Y / Square |
| 13 | B / Cross |
| 14 | L2 |
| 15 | Touchpad |

Die Bezeichnungen wirken auf den ersten Blick etwas ungewöhnlich. Bei einem PlayStation-Controller entsprechen die vier Aktionstasten den Symbolen Triangle, Circle, Square und Cross.

Um eine bestimmte Taste zu prüfen, wird der entsprechende Bitwert verwendet. Bit 13 besitzt beispielsweise den Wert `2^13`, also `8192`:

```
buttons% = DEVICE(GAMEPAD 3, B)

IF (buttons% AND 8192) <> 0 THEN
    PRINT "B/Cross gedrueckt"
ENDIF
```

Für Bit 1, also Start beziehungsweise Options, beträgt die Maske `2`:

```
IF (buttons% AND 2) <> 0 THEN
    PRINT "START/OPTIONS"
ENDIF
```

Dieses Verfahren kennst du bereits vom Arbeiten mit einzelnen Bits: Durch das logische `AND` bleiben nur die Bits übrig, die sowohl in `buttons%` als auch in der Maske gesetzt sind.

## 49.9. Praxisprojekt: Controller testen

Bevor wir einen Controller zur Steuerung eines Spiels verwenden, schauen wir uns zunächst seine Werte an.

Das folgende Programm zeigt die beiden Analogsticks, die Trigger und die Tasten-Bitmaske an:

```
Option EXPLICIT

Const Gamepad = 3

Const BtnR1%      = 1
Const BtnStart%   = 2
Const BtnHome%    = 4
Const BtnSelect%  = 8
```

```
Const BtnL1%      = 16
Const BtnDown%    = 32
Const BtnRight%   = 64
Const BtnUp%      = 128
Const BtnLeft%    = 256
Const BtnR2%      = 512
Const BtnTriangle%= 1024
Const BtnCircle%  = 2048
Const BtnSquare%  = 4096
Const BtnCross%   = 8192
Const BtnL2%      = 16384
Const BtnTouch%   = 32768

Dim usbType%
Dim lx, ly, rx, ry
Dim lTrigger, rTrigger
Dim buttons%

usbType% = MM.Info(USB Gamepad)

Print "Gamepad-Test"
Print "============"
Print

If usbType% = 0 Then
    Print "Kein USB-Geraet auf Kanal "; Gamepad; " gefunden."
    Print "Das erste Gamepad wird normalerweise Kanal 3 zugeordnet."
    End
EndIf

Print "USB-Kanal : "; Gamepad
Print "Geraetecode: "; usbType%;

Select Case usbType%
    Case 128
        Print "  (PS4-Controller)"
    Case 129
        Print "  (PS3-Controller)"
    Case 130
        Print "  (SNES/Generic-Gamepad)"
    Case Else
        Print
        Print "Achtung: Auf diesem Kanal wurde kein bekanntes Gamepad erkannt."
        End
End Select

Print
Print "Controller bewegen oder Tasten druecken."
Print "Abbruch mit Ctrl-C."
Print
```

```
Do
    lx = DEVICE(GAMEPAD Gamepad, LX)
    ly = DEVICE(GAMEPAD Gamepad, LY)
    rx = DEVICE(GAMEPAD Gamepad, RX)
    ry = DEVICE(GAMEPAD Gamepad, RY)

    lTrigger = DEVICE(GAMEPAD Gamepad, L)
    rTrigger = DEVICE(GAMEPAD Gamepad, R)
    buttons% = DEVICE(GAMEPAD Gamepad, B)

    Print @(0,150) "LX:"; lx; "  LY:"; ly; "  RX:"; rx; "  RY:"; ry; "    "
    Print @(0,165) "L :"; lTrigger; "  R :"; rTrigger; "  B:"; buttons%; "    "
    Print @(0,180) "Tasten: ";
    If buttons% <> 0 Then


        If (buttons% And BtnR1%)       <> 0 Then Print "R1 ";
        If (buttons% And BtnStart%)    <> 0 Then Print "START/OPTIONS ";
        If (buttons% And BtnHome%)     <> 0 Then Print "HOME ";
        If (buttons% And BtnSelect%)   <> 0 Then Print "SELECT/SHARE ";
        If (buttons% And BtnL1%)       <> 0 Then Print "L1 ";
        If (buttons% And BtnDown%)     <> 0 Then Print "DOWN ";
        If (buttons% And BtnRight%)    <> 0 Then Print "RIGHT ";
        If (buttons% And BtnUp%)       <> 0 Then Print "UP ";
        If (buttons% And BtnLeft%)     <> 0 Then Print "LEFT ";
        If (buttons% And BtnR2%)       <> 0 Then Print "R2 ";
        If (buttons% And BtnTriangle%) <> 0 Then Print "X/TRIANGLE ";
        If (buttons% And BtnCircle%)   <> 0 Then Print "A/CIRCLE ";
        If (buttons% And BtnSquare%)   <> 0 Then Print "Y/SQUARE ";
        If (buttons% And BtnCross%)    <> 0 Then Print "B/CROSS ";
        If (buttons% And BtnL2%)       <> 0 Then Print "L2 ";
        If (buttons% And BtnTouch%)    <> 0 Then Print "TOUCHPAD ";

        Print "                                                  "
    Else
        Print "keine                                             "
    EndIf

    Print
    Pause 100
Loop
```

Das Programm prüft zunächst, ob auf Kanal 3 tatsächlich ein unterstütztes Gamepad erkannt wurde. Die Gerätecodes `128`, `129` und `130` stehen für PS4-, PS3- beziehungsweise SNES/Generic-Gamepads.

Anschließend werden Analogsticks, Trigger und die komplette Tasten-Bitmaske laufend angezeigt. Für jedes gesetzte Bit gibt das Programm zusätzlich die zugehörige Tastenbezeichnung aus.

Bewege beide Analogsticks langsam in alle Richtungen und beobachte die ausgegebenen Werte.

Achte besonders darauf, welche Werte `LX` und `LY` liefern, wenn der linke Stick nicht berührt wird.

Diese Mittelstellung brauchen wir gleich für die Totzone.

## 49.10. Analogsticks und Totzone

Ein Analogstick liefert nicht nur die Zustände links, rechts, oben und unten. Seine Position wird als Zahlenwert ausgegeben.

Der genaue Wertebereich ist für unsere ersten Programme gar nicht entscheidend. Wichtig ist zunächst der Wert in der Mittelstellung.

Viele Analogsticks kehren mechanisch nicht immer exakt auf denselben Zahlenwert zurück. Ohne Gegenmaßnahme könnte sich deshalb eine Spielfigur langsam bewegen, obwohl der Stick gar nicht berührt wird.

Dagegen verwendet man eine sogenannte **Totzone**.

Wir merken uns zunächst die Mittelstellung:

```
centerX = DEVICE(GAMEPAD 3, LX)
centerY = DEVICE(GAMEPAD 3, LY)
```

Anschließend reagieren wir nur dann auf eine Bewegung, wenn die Abweichung von dieser Mittelstellung groß genug ist:

```
IF ABS(lx - centerX) > Deadzone THEN
    ' Stick wurde deutlich nach links oder rechts bewegt
ENDIF
```

Wie groß `Deadzone` sein sollte, hängt vom verwendeten Controller ab. Mit dem Testprogramm kannst du leicht feststellen, wie stark die Werte in Ruhestellung schwanken.

## 49.11. Praxisprojekt: Cursor mit dem Analogstick bewegen

Nun bewegen wir einen kleinen Kreis mit dem linken Analogstick.

Damit das Beispiel nicht von einem bestimmten Wertebereich des Controllers abhängt, benutzen wir zunächst nur die Richtung der Abweichung. Der Kreis bewegt sich also mit einer festen Geschwindigkeit.

```
'----------------------------------------
' Cursor mit Gamepad bewegen
'----------------------------------------
OPTION EXPLICIT
```

```
CONST Gamepad = 3
CONST Deadzone = 5
CONST Speed = 4

DIM x, y
DIM lx, ly
DIM centerX, centerY

IF MM.INFO(USB 3) = 0 THEN
    PRINT "Kein Gamepad auf USB-Kanal 3 gefunden."
    END
ENDIF

x = MM.HRES / 2
y = MM.VRES / 2

' Stick beim Programmstart nicht bewegen!
centerX = DEVICE(GAMEPAD Gamepad, LX)
centerY = DEVICE(GAMEPAD Gamepad, LY)

DO
    lx = DEVICE(GAMEPAD Gamepad, LX)
    ly = DEVICE(GAMEPAD Gamepad, LY)

    IF lx < centerX - Deadzone THEN x = x - Speed
    IF lx > centerX + Deadzone THEN x = x + Speed

    IF ly < centerY - Deadzone THEN y = y - Speed
    IF ly > centerY + Deadzone THEN y = y + Speed

    IF x < 8 THEN x = 8
    IF x > MM.HRES - 8 THEN x = MM.HRES - 8
    IF y < 8 THEN y = 8
    IF y > MM.VRES - 8 THEN y = MM.VRES - 8

    CLS
    CIRCLE x, y, 8

    PAUSE 20
LOOP
```

Beim Start des Programms sollte der Analogstick in Mittelstellung stehen. Die beiden Werte `centerX` und `centerY` werden unmittelbar nach dem Programmstart gemessen.

Falls sich der Kreis trotzdem ohne Berührung bewegt, erhöhe den Wert von `Deadzone` etwas. Ist die Reaktion dagegen zu unempfindlich, verkleinere ihn.

# 49.12. Praxisprojekt: Elektronisches Zeichenbrett

Mit wenigen Änderungen wird aus unserem Cursorprogramm ein kleines Zeichenbrett.

Der linke Analogstick bewegt den Zeichenstift. Mit B/Cross wird gezeichnet, mit A/Circle wechseln wir die Farbe und mit Y/Square löschen wir den Bildschirm.

```
'----------------------------------------
' Zeichenbrett mit Gamepad
'----------------------------------------
OPTION EXPLICIT

CONST Gamepad = 3
CONST Deadzone = 5
CONST Speed = 3

CONST BtnCircle% = 2048
CONST BtnSquare% = 4096
CONST BtnCross%  = 8192

DIM x, y
DIM lx, ly
DIM centerX, centerY
DIM buttons%, oldButtons%
DIM penColour%

IF MM.INFO(USB 3) = 0 THEN
    PRINT "Kein Gamepad auf USB-Kanal 3 gefunden."
    END
ENDIF

x = MM.HRES / 2
y = MM.VRES / 2
penColour% = RGB(WHITE)

centerX = DEVICE(GAMEPAD Gamepad, LX)
centerY = DEVICE(GAMEPAD Gamepad, LY)

CLS

DO
    lx = DEVICE(GAMEPAD Gamepad, LX)
    ly = DEVICE(GAMEPAD Gamepad, LY)
    buttons% = DEVICE(GAMEPAD Gamepad, B)

    IF lx < centerX - Deadzone THEN x = x - Speed
    IF lx > centerX + Deadzone THEN x = x + Speed
    IF ly < centerY - Deadzone THEN y = y - Speed
    IF ly > centerY + Deadzone THEN y = y + Speed
```

```
    IF x < 1 THEN x = 1
    IF x > MM.HRES - 1 THEN x = MM.HRES - 1
    IF y < 1 THEN y = 1
    IF y > MM.VRES - 1 THEN y = MM.VRES - 1

    ' B/Cross: zeichnen
    IF (buttons% AND BtnCross%) <> 0 THEN
        PIXEL x, y, penColour%
    ENDIF

    ' A/Circle: Farbe bei neuem Tastendruck wechseln
    IF (buttons% AND BtnCircle%) <> 0 THEN
        IF (oldButtons% AND BtnCircle%) = 0 THEN
            IF penColour% = RGB(WHITE) THEN
                penColour% = RGB(YELLOW)
            ELSE
                penColour% = RGB(WHITE)
            ENDIF
        ENDIF
    ENDIF

    ' Y/Square: Bildschirm bei neuem Tastendruck loeschen
    IF (buttons% AND BtnSquare%) <> 0 THEN
        IF (oldButtons% AND BtnSquare%) = 0 THEN
            CLS
        ENDIF
    ENDIF

    oldButtons% = buttons%

    PAUSE 10
LOOP
```

Bei den beiden Funktionen zum Farbwechseln und Löschen wird zusätzlich `oldButtons%` benutzt. Dadurch wird die Aktion nur beim Übergang von "nicht gedrückt" zu "gedrückt" ausgeführt. Würden wir lediglich den aktuellen Zustand prüfen, würde die Aktion während eines längeren Tastendrucks ständig wiederholt.

Hier unterscheiden wir zwischen **Zustand** und **Ereignis**:

- `buttons%` beschreibt, welche Tasten im Moment gedrückt sind.
- Der Vergleich mit `oldButtons%` erkennt, welche Taste gerade neu gedrückt wurde.

Dieses Prinzip ist immer dann nützlich, wenn eine Aktion pro Tastendruck nur einmal ausgeführt werden soll.

## 49.13. PS4-Controller: Farbe und Vibration

Bei einem PS4-Controller kann der PicoMite zusätzlich die Farbe der Leuchtanzeige verändern.

Die Syntax lautet:

```
GAMEPAD COLOUR channel, colour
```

Beispielsweise:

```
GAMEPAD COLOUR 3, RGB(RED)
```

Auch die beiden Vibrationsmotoren lassen sich ansteuern:

```
GAMEPAD HAPTIC channel, left, right
```

`left` und `right` dürfen Werte von `0` bis `255` besitzen. `0` schaltet den jeweiligen Motor aus, `255` entspricht der maximalen Stärke.

Ein kurzer Vibrationsimpuls könnte beispielsweise so aussehen:

```
GAMEPAD HAPTIC 3, 160, 160
PAUSE 200
GAMEPAD HAPTIC 3, 0, 0
```

Das lässt sich später sehr gut als Rückmeldung in einem Spiel verwenden, beispielsweise bei einer Kollision.

## 49.14. Gamepad-Interrupts

Bisher fragen unsere Programme den Controller ständig innerhalb der Hauptschleife ab. Für Tastenereignisse stellt MMBasic außerdem Gamepad-Interrupts zur Verfügung.

Die wichtigsten Befehle sind:

```
GAMEPAD INTERRUPT ENABLE channel, int [, mask]
```

```
GAMEPAD INTERRUPT DISABLE channel
```

Mit der optionalen `mask` kann festgelegt werden, welche Tasten einen Interrupt auslösen sollen. Wird keine Maske angegeben, können alle Tasten einen Interrupt auslösen.

Die Maske entspricht direkt der Bitbelegung von `DEVICE(GAMEPAD channel, B)`. Soll beispielsweise nur B/Cross, also Bit 13, einen Interrupt auslösen, kann die Maske `8192` verwendet werden:

```
GAMEPAD INTERRUPT ENABLE 3, GamepadTaste, 8192
```

Wie bei den Pin-Interrupts aus Kapitel 45 sollte die Interrupt-Subroutine möglichst kurz bleiben. Die eigentliche Verarbeitung kann anschließend im Hauptprogramm erfolgen.

Für unsere ersten Gamepad-Programme ist das regelmäßige Abfragen in einer Schleife einfacher und übersichtlicher. Interrupts werden interessant, wenn ein Programm unabhängig von seiner normalen Verarbeitung sofort auf bestimmte Tastendrücke reagieren soll.

## 49.15. Praxisprojekt: Pong mit Gamepad

Wenn ein vorhandenes Spiel bereits sauber zwischen Spiellogik und Eingabe unterscheidet, lässt sich die Tastatursteuerung leicht durch ein Gamepad ersetzen.

In Kapitel 29 haben wir die Pong-Steuerung mit `KEYDOWN()` realisiert, damit die Bewegung nicht von der Tastenwiederholrate der Tastatur abhängt.

Für eine Gamepad-Steuerung ersetzen wir lediglich die Ermittlung der Bewegungsrichtung. Mit dem linken Analogstick könnte sie beispielsweise so aussehen:

```
ly = DEVICE(GAMEPAD 3, LY)

IF ly < centerY - Deadzone THEN y = y - 4
IF ly > centerY + Deadzone THEN y = y + 4
```

Die eigentliche Spiellogik muss dafür nicht verändert werden. Nur die Ermittlung der gewünschten Bewegungsrichtung wird ausgetauscht.

Das zeigt einen wichtigen Grundsatz beim Programmieren: Eingabe, Spiellogik und Ausgabe sollten möglichst unabhängig voneinander bleiben.

## 49.16. Mehr als nur Spiele

Ein Gamepad eignet sich nicht nur für Computerspiele. Es ist ein preiswertes und komfortables Eingabegerät mit vielen Tasten und mehreren analogen Bedienelementen.

Damit lassen sich beispielsweise auch folgende Projekte steuern:

- Roboter

- Modellfahrzeuge
- Servomotoren
- Kameraschwenks
- Menüs
- Mess- und Prüfaufbauten

Gerade die beiden Analogsticks machen ein Gamepad für technische Experimente interessant, weil sich damit zwei Bewegungsachsen gleichzeitig bedienen lassen.

## 49.17. Noch ein Wort zur USB-Firmware

Bei den USB-Versionen der PicoMite-Firmware arbeitet die USB-Schnittstelle des Raspberry Pi Pico als USB-Host. Dadurch können Tastatur, Maus und Gamepad direkt beziehungsweise über einen USB-Hub angeschlossen werden.

Da der USB-Anschluss dann nicht als normale serielle USB-Konsole zur Verfügung steht, verwendet die Firmware standardmäßig folgende Pins für die serielle Konsole:

- Pin 11 / GP8: Tx
- Pin 12 / GP9: Rx

Die Baudrate beträgt standardmäßig 115200 Baud. Mit `OPTION SERIAL CONSOLE` können bei Bedarf andere Pins eingestellt werden.

Beim PicoMite HDMI/USB ist diese USB-Unterstützung bereits Bestandteil des vorgesehenen Systems. Der Hinweis ist insbesondere dann wichtig, wenn du eigene Hardware auf Basis der USB-Firmware aufbaust und GP8 oder GP9 anderweitig verwenden möchtest.

## 49.18. Retro-Ecke

Die klassischen Joysticks der Heimcomputerzeit waren elektrisch ausgesprochen einfach aufgebaut. Ein Competition Pro besaß im Wesentlichen vier Richtungsschalter und einen Feuerknopf.

Der Computer musste deshalb nur fünf digitale Signale auswerten.

Ein moderner USB-Controller ist intern erheblich komplexer. Er liefert digitale Tasten, analoge Achsen und je nach Modell sogar Bewegungsdaten. Dank der PicoMite-Firmware bleibt die Programmierung trotzdem angenehm einfach: Mit `DEVICE(GAMEPAD …)` können wir auf diese Informationen direkt aus MMBasic zugreifen.

Das Grundprinzip ist damit seit den Heimcomputerzeiten gleich geblieben: Der Spieler bewegt einen Controller, und das Programm setzt diese Eingabe in eine Aktion um.

## 49.19. Zusammenfassung

In diesem Kapitel hast du gelernt:

- wie USB-Gamepads vom PicoMite erkannt und USB-Kanälen zugeordnet werden
- wie du mit `MM.INFO(USB n)` den Gerätetyp eines USB-Kanals prüfst
- wie du mit `GAMEPAD MONITOR` einen Controller untersuchst
- wie `DEVICE(GAMEPAD …)` Analogsticks, Trigger und Tasten ausliest
- wie die Tasten eines Gamepads als Bitmaske ausgewertet werden
- wie eine Totzone ungewollte Bewegungen eines Analogsticks verhindert
- wie du zwischen einem gehaltenen Tastenzustand und einem neuen Tastendruck unterscheidest
- wie Gamepad-Interrupts mit einer Tastenmaske eingeschränkt werden können
- wie sich ein Gamepad als Eingabegerät für Grafikprogramme und Spiele verwenden lässt

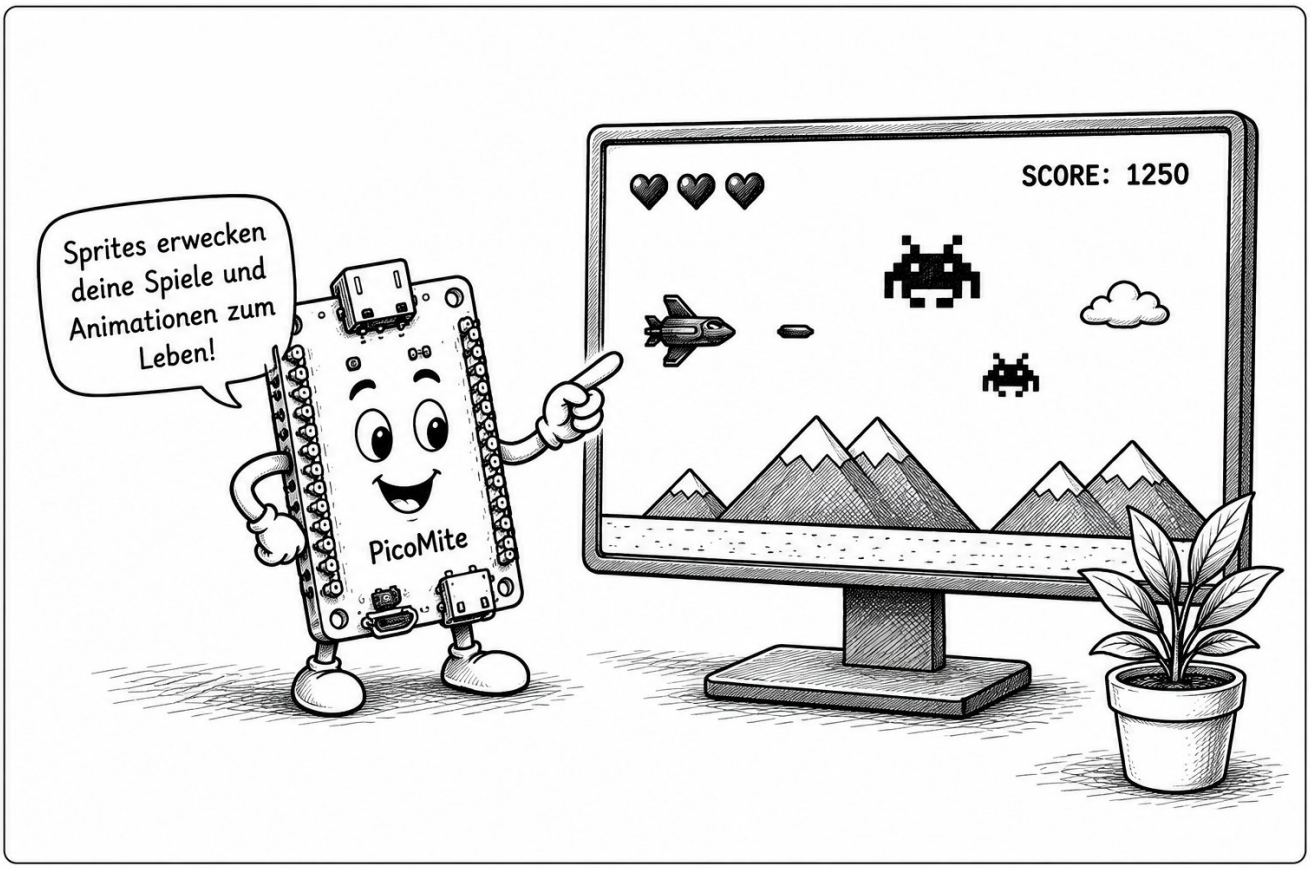
Damit kannst du USB-Gamepads nicht nur für Spiele, sondern auch als vielseitige Eingabegeräte für eigene PicoMite-Projekte einsetzen.

## 49.20. Programmbeispiel

Das vollständige BASIC-Programm findest du hier:

- 49_gamepad_test.bas

# Kapitel 50. Sprites und BLIT




## 50.1. Das lernst du

Fast jedes Computerspiel besteht aus denselben grundlegenden Bausteinen: Spielfiguren, Gegnern, Hindernissen, Hintergründen und Animationen. Viele dieser Dinge müssen unabhängig voneinander bewegt werden, ohne dass dabei der gesamte Bildschirm ständig neu aufgebaut werden muss.

Genau dafür stellt MMBasic auf dem PicoMite zwei wichtige Werkzeuggruppen bereit: `SPRITE` und `BLIT`.

In diesem Kapitel lernst du,

- was ein Sprite ist,
- wie der PicoMite Sprites über einem Hintergrund verwaltet,
- wie Sprites geladen, angezeigt, bewegt und ausgeblendet werden,
- wie mehrere Sprites mit Layern organisiert werden,
- wie Kollisionen zwischen Sprites und mit dem Bildschirmrand erkannt werden,
- wie sich Animationen mit mehreren Sprite-Bildern erzeugen lassen,
- worin sich `SPRITE` und `BLIT` unterscheiden
- und wie `BLIT` rechteckige Bildbereiche schnell kopiert.

Am Ende des Kapitels hast du damit wichtige Grundlagen kennengelernt, die wir im nächsten Kapitel zu einem vollständigen Spiel zusammensetzen können.

## 50.2. Warum reichen normale Grafikbefehle nicht immer aus?

Bisher haben wir Grafiken meist direkt auf den Bildschirm gezeichnet. Ein Kreis entsteht mit `CIRCLE`, eine Linie mit `LINE` und ein Rechteck mit `BOX`.

Soll sich eine Grafik bewegen, könnte man beispielsweise den gesamten Bildschirm löschen und anschließend alles an der neuen Position neu zeichnen:

```
x = 20
y = 100

DO
    CLS
    CIRCLE x, y, 10
    x = x + 2
LOOP
```

Bei einem einzelnen Kreis ist das noch kein großes Problem. In einem Spiel befinden sich jedoch schnell viele Elemente gleichzeitig auf dem Bildschirm:

- ein Hintergrund,
- eine Spielfigur,
- mehrere Gegner,
- Hindernisse,
- Schüsse,
- Punkteanzeigen
- und weitere Animationen.

Würde der gesamte Bildschirm bei jeder kleinen Bewegung gelöscht und vollständig neu aufgebaut, wäre das unnötig aufwendig. Außerdem können sichtbares Flackern oder ruckartige Bewegungen entstehen.

Für bewegliche Spielobjekte ist deshalb ein anderes Verfahren praktischer.

## 50.3. Eine Idee aus der klassischen Spieleprogrammierung

Schon frühe Heimcomputer und Spielkonsolen mussten viele bewegliche Objekte mit vergleichsweise geringer Rechenleistung darstellen.

Ein besonders bekanntes Beispiel ist der Commodore 64. Sein VIC-II-Grafikchip besitzt acht Hardware-Sprites. Diese kleinen Grafiken konnten unabhängig vom normalen Bildschirmhintergrund positioniert werden.

Auch andere Systeme verwendeten spezielle Unterstützung für bewegliche Grafiken. Der Amiga kombinierte Hardware-Sprites mit seinem leistungsfähigen Blitter. Andere Rechner, beispielsweise der Atari ST, besaßen dagegen keine vergleichbare klassische Sprite-Hardware und mussten bewegliche Spielgrafiken weitgehend per Software erzeugen.

Die technische Umsetzung war also von System zu System verschieden. Die Grundidee blieb jedoch gleich:

Eine kleine Grafik soll sich über einem Hintergrund bewegen können, ohne dass der Programmierer bei jeder Bewegung den kompletten Hintergrund neu zeichnen muss.

=== Was ist ein Sprite?

Ein Sprite ist eine Grafik, die als eigenständiges Objekt über einem Hintergrund dargestellt und bewegt werden kann.

Stell dir ein Landschaftsbild auf einem Blatt Papier vor. Darauf legst du einen kleinen Aufkleber mit einer Spielfigur.

Möchtest du die Figur bewegen, setzt du den Aufkleber einfach an eine andere Stelle. Die Landschaft darunter soll anschließend wieder so aussehen wie zuvor.

Genau diese Arbeit übernimmt die Sprite-Verwaltung des PicoMite für uns.

Ein Sprite besitzt dabei unter anderem

- eine Grafik,
- eine Breite und Höhe,
- eine Position,
- eine Layer-Zuordnung
- und einen gespeicherten Hintergrundbereich.

Zusätzlich kann der PicoMite erkennen, wenn sich die rechteckigen Begrenzungen von Sprites berühren oder ein Sprite einen Bildschirmrand erreicht.

=== `SPRITE` und `BLIT` sind nicht dasselbe

In älteren PicoMite-Versionen wurden die Begriffe `SPRITE` und `BLIT` teilweise für dieselbe Funktionalität verwendet.

Seit Firmware-Version 6.00.00 sind beide Befehlsfamilien klar voneinander getrennt.

### 50.3.1. `BLIT`

`BLIT` ist im Kern eine schnelle Kopieroperation für rechteckige Bildbereiche.

Damit können Daten beispielsweise

- vom Display in einen Speicherpuffer,
- aus einem Speicherpuffer auf das Display
- oder direkt von einer Bildschirmposition zu einer anderen

kopiert werden.

Ein BLIT-Puffer enthält RGB888-Farbdaten. Dadurch steht die volle Farbinformation zur Verfügung, allerdings wird entsprechend mehr Speicher benötigt.

### 50.3.2. SPRITE

SPRITE geht einen Schritt weiter.

Wird ein Sprite zum ersten Mal angezeigt, merkt sich die Firmware den Hintergrund, der von der Sprite-Grafik überdeckt wird. Wird das Sprite später verschoben, geschieht automatisch Folgendes:

1. Der alte Hintergrund wird wiederhergestellt.
2. Der Hintergrund an der neuen Position wird gespeichert.
3. Das Sprite wird an der neuen Position gezeichnet.
4. Eventuelle Kollisionen werden ermittelt.

Sprites werden intern besonders speichersparend als RGB121-Werte abgelegt. Zwei Pixel passen dabei in ein Byte.

Die beiden Befehlsfamilien erfüllen also unterschiedliche Aufgaben:

BLIT kopiert Bilddaten.

SPRITE verwaltet bewegliche Grafikobjekte einschließlich Hintergrund und Kollisionen.

=== Wo können Sprites verwendet werden?

Sprite-Funktionen benötigen Zugriff auf den Bildspeicher, damit der verdeckte Hintergrund gelesen und später wiederhergestellt werden kann.

Bei den VGA- und HDMI-Versionen des PicoMite können Sprites direkt mit dem Bildschirm verwendet werden. Außerdem stehen sie bei geeigneten In-Memory-Framebuffern zur Verfügung.

Auch die Möglichkeiten von BLIT READ hängen davon ab, ob der verwendete Bildspeicher gelesen werden kann.

Für unser Tutorial ist das besonders praktisch, weil der PicoMite HDMI/USB diese Voraussetzungen erfüllt.

## 50.4. Sprite-Nummern

Der PicoMite verwaltet Sprites über Nummern.

Es stehen die Sprite-Nummern 1 bis 64 zur Verfügung.

Ein geladenes Sprite kann deshalb beispielsweise die Nummer 1 erhalten:

```
SPRITE LOADPNG #1, "spieler.png"
```

Ein Gegner könnte anschließend als Sprite 2 geladen werden:

```
SPRITE LOADPNG #2, "gegner.png"
```

Die Sprite-Nummer ist danach die Kennung, über die wir das Objekt ansprechen.

`SPRITE LOADPNG` ist eine besonders komfortable Möglichkeit auf geeigneten RP2350-Versionen. Sprites können außerdem aus klassischen Sprite-Dateien, aus BMP-Dateien, aus Arrays oder direkt aus einem Bildbereich erzeugt werden.

=== Sprites laden

MMBasic stellt mehrere Möglichkeiten zur Verfügung, Sprite-Grafiken in den Speicher zu übernehmen.

### 50.4.1. Sprite-Datei laden

Mit

```
SPRITE LOAD fname$ [, start_sprite_number] [, mode]
```

kann eine Sprite-Datei geladen werden.

Beispiel:

```
SPRITE LOAD "figuren.spr", 1, 1
```

Mehrere Sprites können sich gemeinsam in einer solchen Datei befinden.

### 50.4.2. Sprite aus einer PNG-Datei laden

Auf unterstützten Systemen lässt sich ein Sprite direkt aus einer PNG-Datei laden:

```
SPRITE LOADPNG #1, "spieler.png"
```

Dabei wird die Erweiterung `.png` automatisch ergänzt, wenn sie nicht im Dateinamen angegeben wurde.

Optional können auch Transparenzparameter angegeben werden:

```
SPRITE LOADPNG #1, "spieler.png", 0, 20
```

### 50.4.3. Sprite aus einem Array erzeugen

Ein Sprite kann auch vollständig innerhalb eines BASIC-Programms erzeugt werden:

```
SPRITE LOADARRAY #n, w, h, array%()
```

Das ist besonders praktisch für kleine selbst erzeugte Grafiken, weil keine zusätzliche Bilddatei benötigt wird.

=== Das erste Sprite anzeigen

Nachdem ein Sprite geladen wurde, erscheint es noch nicht automatisch auf dem Bildschirm.

Dazu dient:

```
SPRITE SHOW #n, x, y, layer
```

`x` und `y` geben die Position der linken oberen Ecke des Sprites an.

Das folgende Beispiel zeigt Sprite 1 an Position 160/120:

```
x = 160
y = 120

SPRITE SHOW #1, x, y, 1
```

Der letzte Parameter ist der Layer. Darauf kommen wir gleich noch genauer zurück.

Beim ersten `SPRITE SHOW` speichert der PicoMite den Hintergrund unterhalb des Sprites und zeichnet anschließend die Sprite-Grafik.

## 50.5. Ein Sprite bewegen

Für eine einfache Bewegung benötigen wir keinen speziellen `MoveSprite()`-Befehl.

Ein bereits sichtbares Sprite wird durch einen weiteren Aufruf von `SPRITE SHOW` an die neue Position bewegt.

```
x = 160
y = 120

SPRITE SHOW #1, x, y, 1

DO
    x = x + 2
    SPRITE SHOW #1, x, y, 1
    PAUSE 20
LOOP
```

Dabei kümmert sich die Firmware selbst um den verdeckten Hintergrund.

Intern geschieht bei jeder Bewegung:

1. Der Hintergrund an der bisherigen Sprite-Position wird restauriert.
2. Der Hintergrund an der neuen Position wird gespeichert.
3. Das Sprite wird an der neuen Position gezeichnet.

Wir müssen den Bildschirm also nicht mit `CLS` löschen.

=== Ein Sprite ausblenden

Mit

```
SPRITE HIDE #1
```

wird Sprite 1 vom Bildschirm entfernt.

MMBasic zeichnet dabei automatisch den zuvor gespeicherten Hintergrund zurück.

Soll das Sprite später erneut erscheinen, kann es wieder mit `SPRITE SHOW` angezeigt werden.

=== Sprite-Speicher wieder freigeben

Wird ein Sprite nicht mehr benötigt, kann sein Speicher freigegeben werden:

```
SPRITE CLOSE #1
```

Alle Sprite-Speicher lassen sich gemeinsam freigeben:

```
SPRITE CLOSE ALL
```

`SPRITE HIDE` und `SPRITE CLOSE` erfüllen unterschiedliche Aufgaben.

`SPRITE HIDE` entfernt ein Sprite nur vom Bildschirm.

> `SPRITE CLOSE` gibt den für das Sprite belegten Speicher frei.

=== Layer – mehrere Ebenen für Sprites

Bei jedem `SPRITE SHOW` muss ein Layer angegeben werden.

Das Prinzip ähnelt mehreren transparenten Folien, die übereinanderliegen.

Beispielsweise könnten wir verwenden:

- Layer 0 für Objekte, die mit dem Hintergrund verbunden sind,
- Layer 1 für Spieler und Gegner,
- Layer 2 für besondere Spielelemente.

Layer 0 besitzt eine besondere Bedeutung.

Sprites kollidieren mit

- Sprites auf demselben Layer,
- Sprites auf Layer 0
- und dem Bildschirmrand.

Dadurch kann Layer 0 beispielsweise für Hindernisse verwendet werden, mit denen Objekte auf allen anderen Layern kollidieren sollen. Layer 0 ist damit eine Art gemeinsame Kollisionsebene.

Beim Scrollen bewegt sich Layer 0 zusammen mit dem Hintergrund. Sprites auf anderen Layern bleiben dagegen an ihrer Bildschirmposition.

=== Die Zeichenreihenfolge und das LIFO-Prinzip

Beim Anzeigen von Sprites muss der PicoMite berücksichtigen, in welcher Reihenfolge sie gezeichnet wurden.

Nehmen wir an, wir zeigen drei Sprites nacheinander an:

```
SPRITE SHOW #1, 100, 100, 1
SPRITE SHOW #2, 110, 110, 1
SPRITE SHOW #3, 120, 120, 1
```

Sprite 3 wurde zuletzt gezeichnet und liegt deshalb gegebenenfalls über Sprite 2. Sprite 2 wiederum kann über Sprite 1 liegen.

Diese Reihenfolge funktioniert nach dem Prinzip *Last In, First Out*, kurz LIFO.

Würde nun das zuerst gezeichnete Sprite 1 bewegt, könnten gespeicherte Hintergrundbereiche Teile der später darüber gezeichneten Sprites überschreiben.

Vereinfacht können wir uns die Zeichenreihenfolge so vorstellen:

```
Hintergrund
   |
   v
Sprite 1
   |
   v
Sprite 2
   |
   v
Sprite 3
```

Soll nun Sprite 1 sicher bewegt werden, müssen die darüberliegenden Sprites vorübergehend in umgekehrter Reihenfolge entfernt und anschließend wieder aufgebaut werden:

```
Sprite 3 entfernen
Sprite 2 entfernen
Sprite 1 bewegen
Sprite 2 wieder anzeigen
Sprite 3 wieder anzeigen
```

Genau dafür gibt es `SPRITE SHOW SAFE`.

=== `SPRITE SHOW SAFE`

Die sichere Variante lautet:

```
SPRITE SHOW SAFE #n, x, y, layer
```

Sie berücksichtigt automatisch Sprites, die das zu bewegende Sprite überdecken.

Beispiel:

```
SPRITE SHOW SAFE #1, 200, 120, 1
```

MMBasic entfernt dabei intern erforderliche darüberliegende Sprites in umgekehrter Reihenfolge, bewegt das gewünschte Sprite und stellt anschließend die anderen Sprites wieder her.

Das ist komfortabler, benötigt aber etwas mehr Rechenarbeit als das normale `SPRITE SHOW`.

Wenn du sicher weißt, dass sich die Sprites nicht problematisch überlagern, ist `SPRITE SHOW` die einfachere und schnellere Variante.

Bei überlappenden Sprites ist `SPRITE SHOW SAFE` dagegen oft die sicherere Wahl.

=== Transparenz

Sprite-Grafiken sind rechteckig. Die Spielfigur selbst füllt dieses Rechteck jedoch meistens nicht vollständig aus.

Damit der rechteckige Hintergrund der Grafik nicht sichtbar wird, kann eine Farbe als transparent definiert werden:

```
SPRITE SET TRANSPARENT 0
```

Die Farbnummer 0 ist standardmäßig die transparente Farbe.

Dadurch wird nur die eigentliche Figur sichtbar, während die transparenten Pixel den Hintergrund durchscheinen lassen.

## 50.6. Ein Sprite spiegeln

Der optionale Parameter von `SPRITE SHOW` kann verwendet werden, um die Grafik beim Anzeigen zu spiegeln.

Die Bits haben folgende Bedeutung:

| **Wert** | **Bedeutung** |
|---|---|
| `&B001` | horizontal spiegeln |
| `&B010` | vertikal spiegeln |
| `&B100` | schwarze Pixel nicht als transparent behandeln |

Eine Figur lässt sich dadurch beispielsweise ohne zweites Bild nach links und rechts schauen lassen:

```
' normale Richtung
SPRITE SHOW #1, x, y, 1, 0

' horizontal gespiegelt
SPRITE SHOW #1, x, y, 1, &B001
```

Das spart bei einfachen Animationen zusätzliche Grafikdateien.

=== Mehrere Sprites gemeinsam bewegen

Wenn mehrere Sprites praktisch gleichzeitig ihre Position ändern sollen, stellt MMBasic einen weiteren Mechanismus bereit.

Zunächst wird für jedes Sprite die nächste Position festgelegt:

```
SPRITE NEXT #1, PlayerX, PlayerY
SPRITE NEXT #2, EnemyX, EnemyY
```

```
SPRITE NEXT #3, ShotX, ShotY
```

Anschließend werden alle vorbereiteten Positionsänderungen gemeinsam ausgeführt:

```
SPRITE MOVE
```

Das ist etwas anderes als das erneute `SPRITE SHOW` eines einzelnen Sprites.

`SPRITE NEXT` zusammen mit `SPRITE MOVE` ist besonders interessant, wenn mehrere Objekte innerhalb derselben Spielrunde bewegt werden sollen. Die Kollisionen werden ausgewertet, nachdem alle vorbereiteten Bewegungen ausgeführt wurden.

=== Animationen aus mehreren Sprite-Bildern

Eine bewegte Spielfigur besteht häufig aus mehreren Einzelbildern.

Beispielsweise können vier Bilder verschiedene Phasen einer Laufbewegung darstellen:

```
Bild 1    Bild 2    Bild 3    Bild 4
  |         |         |         |
 Schritt   Schritt   Schritt   Schritt
```

Diese Einzelbilder werden als Animationsframes bezeichnet.

MMBasic kann ein sichtbares Sprite durch ein anderes Sprite ersetzen:

```
SPRITE SWAP #1, #2
```

Dabei müssen beide Sprites dieselbe Breite und Höhe besitzen.

Das neue Sprite übernimmt die Position und den gespeicherten Hintergrund des bisherigen Sprites.

Für eine Animation könnten beispielsweise mehrere gleich große Sprites vorbereitet werden:

```
' Sprite 1 ist sichtbar
SPRITE SHOW #1, x, y, 1

' später das Bild austauschen
SPRITE SWAP #1, #2
```

Durch regelmäßiges Wechseln mehrerer Sprite-Bilder entsteht eine Animation.

Oft reichen schon wenige gut gezeichnete Einzelbilder für eine überzeugende Bewegung. Entscheidend ist weniger die Anzahl der Bilder als ein passendes Zeitintervall zwischen den Frames.

# 50.7. Kollisionen

Sobald sich mehrere Spielobjekte bewegen, stellt sich eine wichtige Frage:

Was passiert, wenn sie sich berühren?

Diese Überprüfung nennt man *Kollisionserkennung*.

Der PicoMite besitzt dafür eine eingebaute Sprite-Kollisionserkennung. Es ist also nicht notwendig, für jedes Objekt selbst die Koordinatenbereiche miteinander zu vergleichen.

Die Kollisionsgrenze eines Sprites ist das umschließende Rechteck seiner Grafik.

Die Kollisionserkennung arbeitet mit den rechteckigen Sprite-Grenzen. Sie prüft nicht die Form einzelner sichtbarer Pixel.

=== Ein Collision-Interrupt

Besonders komfortabel ist die automatische Benachrichtigung über eine Kollision.

Mit

```
SPRITE INTERRUPT collision
```

wird die Subroutine `collision` als Interrupt-Routine registriert.

Tritt eine Sprite-Kollision auf, ruft MMBasic diese Subroutine auf.

Ein einfaches Gerüst sieht so aus:

```
SPRITE INTERRUPT collision

' Hauptprogramm
DO
    ' Spiel läuft hier weiter
LOOP

SUB collision
    PRINT "Kollision!"
END SUB
```

Wie bei den Interrupts aus Kapitel 45 sollte auch eine Sprite-Interrupt-Routine möglichst kurz bleiben. Bei umfangreicheren Spielen ist es sinnvoll, dort nur die Kollisionsinformationen zu erfassen oder ein Flag zu setzen und die eigentliche Spielreaktion anschließend in der Hauptschleife auszuführen.

In einem echten Spiel möchten wir natürlich wissen, welches Sprite beteiligt war.

Dafür gibt es die Funktion `SPRITE()`.

=== Welches Sprite verursachte die Kollision?

Mit

```
SPRITE(S)
```

erhalten wir die Nummer des Sprites, dessen Bewegung die Kollision ausgelöst hat.

Beispiel:

```
SUB collision
    LOCAL INTEGER s

    s = SPRITE(S)

    IF s <> 0 THEN
        PRINT "Kollision von Sprite "; s
    ENDIF
END SUB
```

Der Rückgabewert 0 hat eine besondere Bedeutung: Dann wurde die Kollision durch eine Scroll-Operation ausgelöst.

=== Wie viele Kollisionen gab es?

Mit

```
SPRITE(C, n)
```

kann abgefragt werden, wie viele aktuelle Kollisionen für Sprite `n` gespeichert sind.

Beispiel:

```
PRINT SPRITE(C, 1)
```

Sind mehrere Kollisionen gleichzeitig aufgetreten, können sie einzeln abgefragt werden:

```
FOR i = 1 TO SPRITE(C, 1)
    PRINT SPRITE(C, 1, i)
NEXT i
```

Damit kann ein Sprite in derselben Bewegung beispielsweise gleichzeitig einen Gegner und einen Bildschirmrand berühren.

# 50.8. Kollision mit dem Bildschirmrand

Die Kollisionsabfrage liefert für Bildschirmränder spezielle Werte:

| Wert | Bildschirmrand |
|---|---|
| `&HF1` | links |
| `&HF2` | oben |
| `&HF4` | rechts |
| `&HF8` | unten |

Eine Interrupt-Routine kann diese Werte direkt auswerten:

```
SUB collision
    LOCAL INTEGER s, i, hit

    s = SPRITE(S)

    IF s = 0 THEN EXIT SUB

    FOR i = 1 TO SPRITE(C, s)
        hit = SPRITE(C, s, i)

        SELECT CASE hit
            CASE &HF1
                PRINT "linker Rand"
            CASE &HF2
                PRINT "oberer Rand"
            CASE &HF4
                PRINT "rechter Rand"
            CASE &HF8
                PRINT "unterer Rand"
            CASE ELSE
                PRINT "Kollision mit Sprite "; hit
        END SELECT
    NEXT i
END SUB
```

Damit lassen sich beispielsweise Bälle, Gegner oder ein DVD-Logo am Bildschirmrand abprallen lassen.

=== Weitere Informationen über Sprites

Die Funktion `SPRITE()` kann noch weitere Eigenschaften liefern.

Besonders nützlich sind:

```
SPRITE(X, n)
```

```
SPRITE(Y, n)
SPRITE(W, n)
SPRITE(H, n)
SPRITE(N)
```

Damit lassen sich unter anderem Position, Größe und Anzahl aktiver Sprites abfragen.

Beispielsweise:

```
PRINT "X = "; SPRITE(X, 1)
PRINT "Y = "; SPRITE(Y, 1)
PRINT "Breite = "; SPRITE(W, 1)
PRINT "Hoehe = "; SPRITE(H, 1)
PRINT "Aktive Sprites = "; SPRITE(N)
```

Diese Funktionen sind besonders praktisch, wenn viele Objekte von der Firmware verwaltet werden.

=== Was bedeutet BLIT?

Der Begriff `BLIT` steht für *Bit Block Transfer*.

Dabei wird ein rechteckiger Bildbereich sehr schnell von einer Speicherposition an eine andere übertragen.

Stell dir wieder unser Blatt Papier mit einer Landschaft vor. Diesmal möchten wir aber nicht ein eigenständiges Objekt verwalten, sondern einfach einen rechteckigen Ausschnitt des Bildes kopieren.

Genau das ist die Aufgabe von `BLIT`.

=== Einen Bildbereich in einen BLIT-Puffer lesen

Mit

```
BLIT READ #b, x, y, w, h
```

wird ein rechteckiger Bereich des Bildschirms in einen Speicherpuffer kopiert.

Beispiel:

```
BLIT READ #1, 100, 80, 64, 64
```

Damit werden 64 × 64 Pixel ab Position 100/80 in BLIT-Puffer 1 gespeichert.

Die Puffer können die Nummern 1 bis 64 verwenden.

=== Einen BLIT-Puffer wieder anzeigen

Der gespeicherte Bildbereich kann später wieder auf den Bildschirm geschrieben werden:

```
BLIT WRITE #1, 200, 80
```

Der Inhalt von Puffer 1 erscheint nun ab Position 200/80.

Damit lässt sich ein Bildbereich zunächst sichern und später an einer anderen Position wieder ausgeben.

=== BLIT-Puffer aus einer Datei laden

Ein Bild kann auch direkt in einen BLIT-Puffer geladen werden:

```
BLIT LOAD #1, "bild.bmp", 0, 0, 64, 64
```

Die Parameter geben an, welcher Ausschnitt der Bilddatei geladen werden soll.

=== BLIT-Puffer freigeben

Wird der Puffer nicht mehr benötigt, sollte sein Speicher wieder freigegeben werden:

```
BLIT CLOSE #1
```

# 50.9. Direkt von einer Bildschirmposition zur anderen kopieren

Für viele Aufgaben wird gar kein zusätzlicher Puffer benötigt.

Mit

```
BLIT x1, y1, x2, y2, w, h
```

kann ein rechteckiger Bereich direkt an eine andere Bildschirmposition kopiert werden.

Beispiel:

```
BLIT 0, 0, 100, 100, 80, 60
```

Dabei wird der 80 × 60 Pixel große Bereich links oben ab Position 0/0 nach Position 100/100 kopiert.

Quell- und Zielbereich dürfen sich sogar überlappen. Der PicoMite führt die Kopie trotzdem korrekt aus.

=== BLIT zum Scrollen verwenden

Gerade diese direkte Form von BLIT eignet sich gut für scrollende Darstellungen.

Soll ein Bildschirmbereich beispielsweise um einen Pixel nach links verschoben werden, kann nahezu der gesamte vorhandene Bereich kopiert werden. Anschließend muss nur die neu entstandene Spalte am Rand gezeichnet werden.

Das Prinzip sieht so aus:

```
Vorher:

+-------------------------+
| vorhandener Bildbereich  |
+-------------------------+

nach links verschieben:

+-------------------------+?
| verschobener Bildbereich | neue Spalte
+-------------------------+?
```

Für Diagramme, Laufschriften oder scrollende Spielhintergründe kann dieses Verfahren sehr effizient sein.

MMBasic stellt für Sprite-Hintergründe zusätzlich eigene SPRITE SCROLL-Funktionen bereit.

=== Bildbereiche skalieren

Mit

```
BLIT RESIZE src, dst, sx, sy, sw, sh, dx, dy, dw, dh
```

kann ein rechteckiger Quellbereich auf eine andere Größe skaliert werden.

Die Skalierung verwendet ein schnelles Nearest-Neighbour-Verfahren.

Dadurch eignet sich BLIT RESIZE beispielsweise zum schnellen Vergrößern von Pixelgrafiken.

Für den Einstieg genügt es jedoch, zunächst die normalen BLIT-, BLIT READ- und BLIT WRITE-Befehle zu beherrschen.

=== Weitere BLIT-Funktionen

Der PicoMite besitzt außerdem fortgeschrittene Varianten wie

- BLIT MEMORY,
- BLIT COMPRESSED,
- BLIT FRAMEBUFFER
- und BLIT MERGE.

Diese Befehle sind vor allem bei umfangreicheren Grafik- und Spieleprojekten interessant. Für unsere ersten Sprite-Programme benötigen wir sie noch nicht.

## 50.10. SPRITE oder BLIT?

Beide Techniken sind nützlich, aber für unterschiedliche Aufgaben.

| | SPRITE | BLIT |
|---|---|---|
| Hauptaufgabe | bewegliche Grafikobjekte verwalten | rechteckige Bildbereiche kopieren |
| Hintergrund sichern | automatisch beim Anzeigen und Bewegen | muss über passende Kopieroperationen organisiert werden |
| Kollisionserkennung | eingebaut | nein |
| Layer | ja | nein |
| Farbspeicherung | RGB121 | RGB888 |
| Typische Verwendung | Spieler, Gegner, Schüsse, Objekte | Bildkopien, Scrollen, Puffer, Skalierung |

Für eine Spielfigur ist normalerweise SPRITE die bessere Wahl.

Wenn dagegen einfach ein rechteckiger Bildbereich kopiert oder verschoben werden soll, ist BLIT meist das passendere Werkzeug.

## 50.11. Von der Spielfigur zur Spielschleife

Nahezu jedes Computerspiel arbeitet mit einer sich ständig wiederholenden Spielschleife. Typischerweise werden dabei immer wieder dieselben Aufgaben ausgeführt:

1. Eingaben lesen
2. Positionen berechnen
3. Sprites bewegen
4. Animationen aktualisieren
5. Kollisionen auswerten
6. Spielregeln und Punkte aktualisieren
7. kurze Pause beziehungsweise Bildwechsel

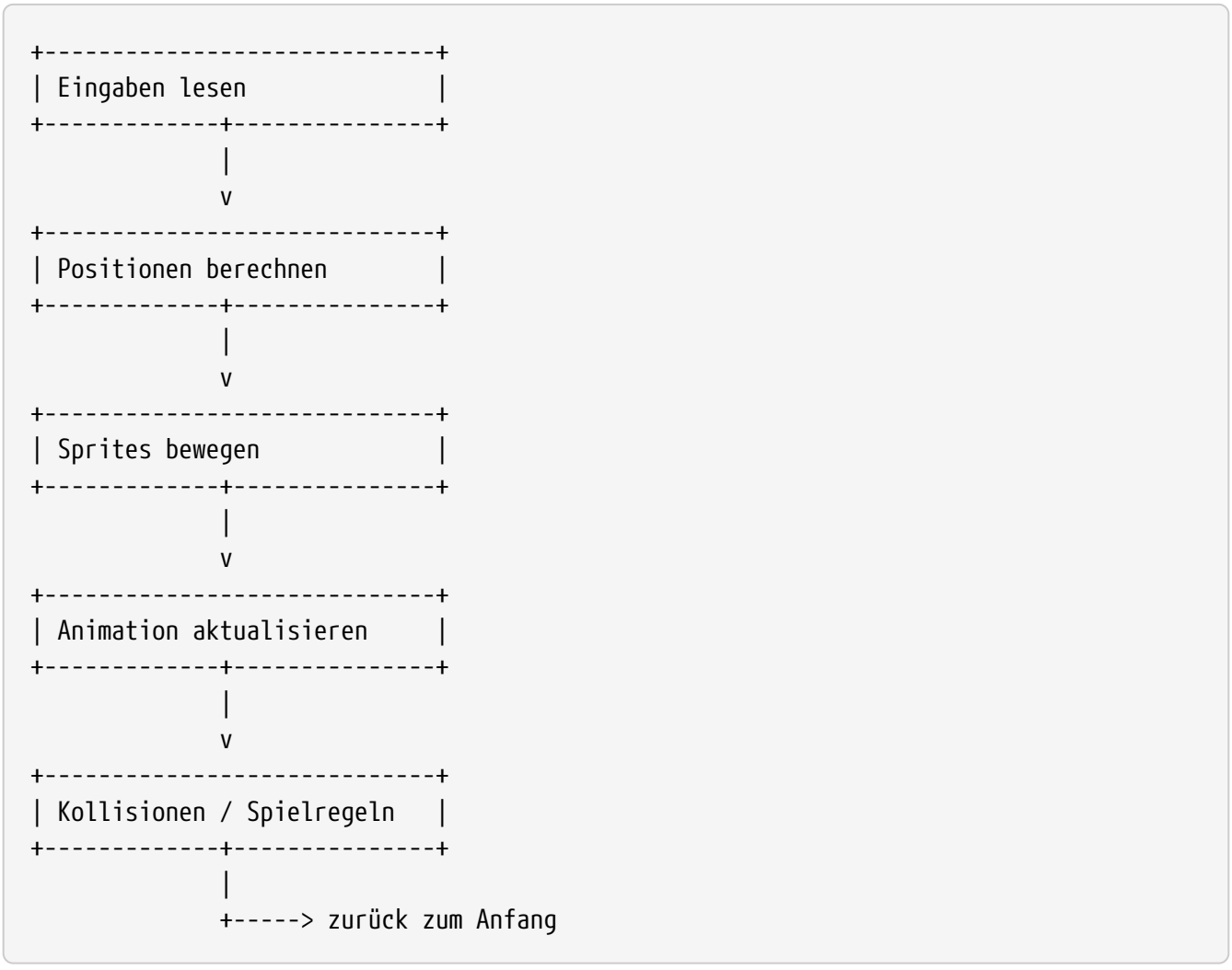
Ein vereinfachter Ablauf könnte so aussehen:

```
DO
    ' Eingaben lesen

    ' neue Positionen berechnen
    SPRITE NEXT #1, PlayerX, PlayerY
```

```
    SPRITE NEXT #2, EnemyX, EnemyY

    ' vorbereitete Bewegungen gemeinsam ausfuehren
    SPRITE MOVE

    ' Animation und Spielregeln aktualisieren

    PAUSE 20
LOOP
```

Mit `SPRITE NEXT` und `SPRITE MOVE` können mehrere Bewegungen für dieselbe Spielrunde vorbereitet und anschließend gemeinsam ausgeführt werden.

Kollisionen können parallel dazu über `SPRITE INTERRUPT` erfasst werden. Damit müssen wir uns im Hauptprogramm nicht mehr selbst um das Löschen und Restaurieren jedes einzelnen bewegten Bildbereichs kümmern.




Je klarer diese Aufgaben voneinander getrennt sind, desto leichter lässt sich das Spiel später erweitern.

# 50.12. Praxisprojekt: Zwei Sprites in Bewegung

Zum Abschluss verbinden wir die wichtigsten Sprite-Techniken dieses Kapitels in einem kleinen Demoprogramm.

Das Programm erzeugt zwei Sprite-Grafiken direkt im BASIC-Programm. Es sind also keine zusätzlichen Bilddateien erforderlich.

Dazu werden zwei Integer-Arrays vorbereitet:

```
DIM INTEGER BallPixels%(255)
DIM INTEGER BoxPixels%(399)
```

Das erste Array enthält die 16 x 16 Pixel eines Balls, das zweite die 20 x 20 Pixel eines Quadrats. Die Subroutine `CreateSprites` füllt beide Arrays mit den gewünschten Farben.

Anschließend werden daraus zwei Sprites erzeugt:

```
SPRITE LOADARRAY #1, 16, 16, BallPixels%()
SPRITE LOADARRAY #2, 20, 20, BoxPixels%()
```

Mit

```
SPRITE SET TRANSPARENT 0
```

wird der Farbwert 0 transparent dargestellt.

## 50.12.1. Beide Sprites gemeinsam bewegen

Die beiden Objekte erhalten jeweils eine Position und eine Bewegungsrichtung. In jeder Runde der Spielschleife werden zunächst die neuen Koordinaten berechnet.

Danach bereiten wir beide Bewegungen vor:

```
SPRITE NEXT #1, x1%, y1%
SPRITE NEXT #2, x2%, y2%
```

Erst

```
SPRITE MOVE
```

führt beide vorbereiteten Bewegungen gemeinsam aus. Anschließend übernimmt das Programm die tatsächlichen Sprite-Positionen wieder von der Firmware:

```
x1% = SPRITE(X, 1)
y1% = SPRITE(Y, 1)
x2% = SPRITE(X, 2)
y2% = SPRITE(Y, 2)
```

Das ist besonders nach einer Kollision nützlich.

### 50.12.2. Kollisionen auswerten

Vor Beginn der Spielschleife wird die Kollisionsroutine registriert:

```
SPRITE INTERRUPT Collision
```

Das Demoprogramm berücksichtigt dabei einen wichtigen Sonderfall von `SPRITE MOVE`: Werden mehrere vorbereitete Sprite-Bewegungen gemeinsam ausgeführt, kann `SPRITE(S)` den Wert `0` liefern. Dann werden die von der Bewegung betroffenen Sprites über

```
SPRITE(C, 0)
SPRITE(C, 0, i%)
```

ermittelt.

Für jedes betroffene Sprite wertet `ProcessCollision` anschließend die gespeicherten Kollisionen aus. Bei einem Bildschirmrand wird nur die entsprechende Bewegungsrichtung umgekehrt:

```
CASE &HF1
    dx1% = ABS(dx1%)

CASE &HF4
    dx1% = -ABS(dx1%)
```

Entsprechend werden oberer und unterer Rand über `dy1%` beziehungsweise `dy2%` behandelt.

Bei einer Kollision der beiden Sprites wird die Bewegungsrichtung ebenfalls umgekehrt. Dadurch prallen die Objekte voneinander ab.

Die Sprite-Kollisionserkennung arbeitet mit den umschließenden Rechtecken. Die sichtbaren Formen von Ball und Quadrat werden dabei nicht pixelgenau geprüft.

### 50.12.3. Am Programmende aufräumen

Mit `Q` wird die Schleife beendet. Danach entfernt das Programm alle Sprites vom Bildschirm und gibt den belegten Speicher wieder frei:

```
SPRITE HIDE ALL
```

```
SPRITE CLOSE ALL
```

Damit führt das Praxisprojekt noch einmal mehrere wichtige Techniken des Kapitels zusammen:

- `SPRITE LOADARRAY`
- Transparenz
- Layer
- `SPRITE NEXT`
- `SPRITE MOVE`
- `SPRITE INTERRUPT`
- `SPRITE()`
- Rand- und Sprite-Kollisionen
- `SPRITE HIDE ALL`
- `SPRITE CLOSE ALL`

# 50.13. Zusammenfassung

In diesem Kapitel hast du zwei wichtige Grafikwerkzeuge des PicoMite kennengelernt.

Du weißt nun,

- dass `SPRITE` und `BLIT` seit PicoMite V6.00.00 getrennte Befehlsfamilien sind,
- dass Sprites ihren verdeckten Hintergrund automatisch sichern und restaurieren,
- dass ein bereits sichtbares Sprite mit einem weiteren `SPRITE SHOW` bewegt werden kann,
- dass `SPRITE SHOW SAFE` überlappende Sprites sicher berücksichtigt,
- dass Layer festlegen, welche Sprites miteinander kollidieren und Layer 0 dabei eine besondere Rolle besitzt,
- dass `SPRITE INTERRUPT` und `SPRITE()` eine eingebaute Kollisionserkennung bereitstellen,
- dass `SPRITE SWAP` für Animationen verwendet werden kann,
- wie mehrere Sprites mit `SPRITE NEXT` und `SPRITE MOVE` gemeinsam bewegt werden,
- dass `BLIT` rechteckige Bildbereiche kopiert
- und dass BLIT-Puffer im Gegensatz zu Sprites RGB888-Farbdaten verwenden.

Damit besitzen wir nun fast alle Bausteine für ein kleines Computerspiel.

# 50.14. Ausblick auf Kapitel 51

Im nächsten Kapitel setzen wir die bisherigen Techniken zu einem vollständigen Spiel zusammen.

Dabei treffen viele Themen aus dem Buch wieder aufeinander:

- Variablen
- Schleifen
- Funktionen und Subroutinen
- Grafik
- Sound
- Zufallszahlen
- Timer
- Gamepad-Steuerung
- Sprites
- Animationen
- Kollisionen

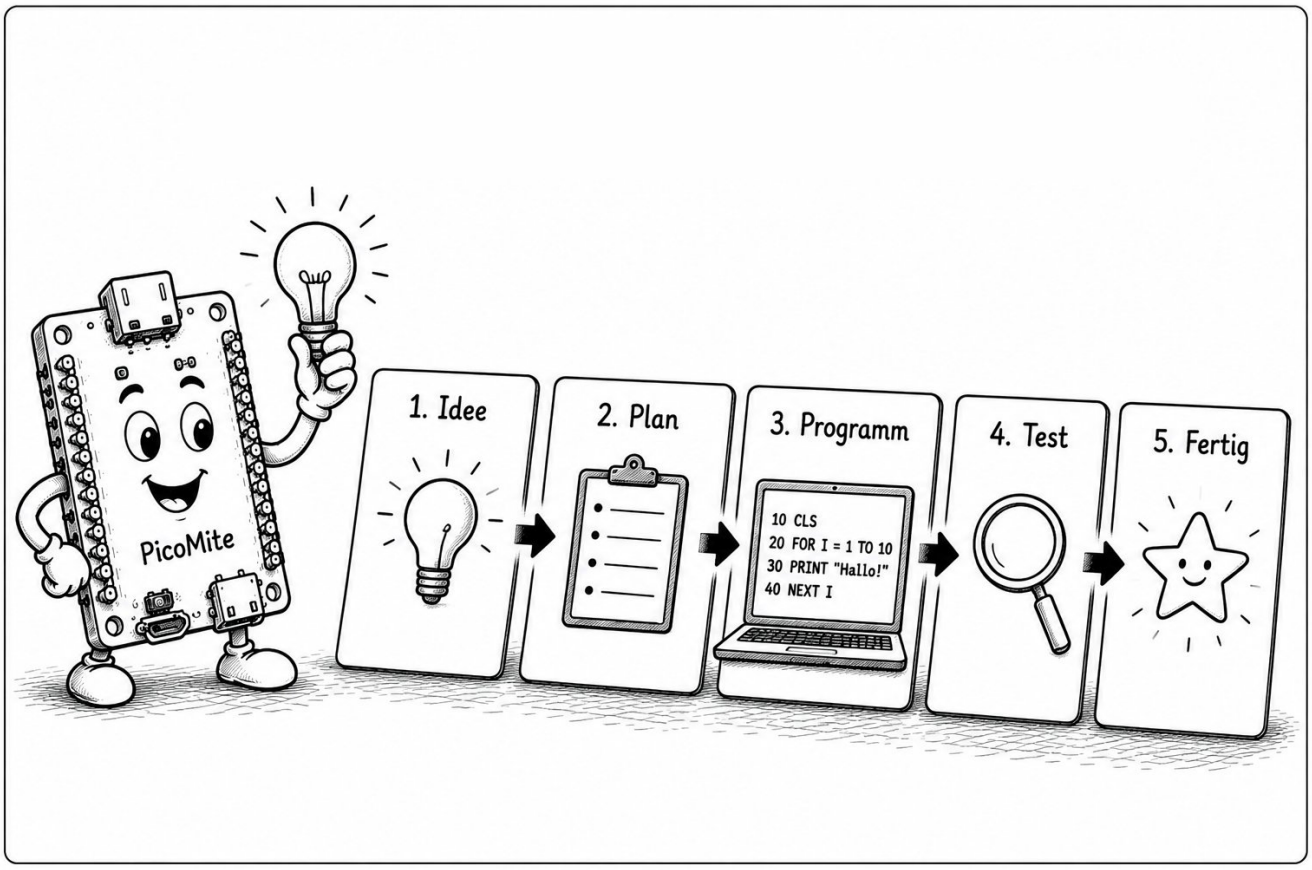
Aus vielen kleinen BASIC-Bausteinen entsteht dann Schritt für Schritt ein spielbares Programm.

## 50.15. Programmbeispiel

Das vollständige BASIC-Programm zu diesem Kapitel findest du hier:

- 50_sprite_demo.bas

# Kapitel 51. Von der Idee zum fertigen Spiel




## 51.1. Willkommen zum Finale

Herzlichen Glückwunsch!

Wenn du bis hierher gekommen bist, hast du bereits einen langen Weg zurückgelegt.

Du hast gelernt,

- Variablen und Konstanten zu verwenden,
- Entscheidungen mit `IF` und `SELECT CASE` zu treffen,
- Schleifen zu programmieren,
- Arrays einzusetzen,
- eigene `SUB`-Programme und `FUNCTION`-Funktionen zu schreiben,
- Grafiken und Texte auszugeben,
- Sound zu erzeugen,
- Dateien zu verwenden,
- Timer einzusetzen,
- Cursortasten, Maus und Gamepads auszulesen
- und Sprites über den Bildschirm zu bewegen.

Jedes dieser Themen war zunächst nur ein kleiner Baustein.

Jetzt setzen wir diese Bausteine zu einem vollständigen Programm zusammen.

In diesem Kapitel entwickeln wir ein kleines Computerspiel. Dabei geht es nicht darum, möglichst viele neue Befehle kennenzulernen. Stattdessen verwenden wir bewusst das, was wir in den vergangenen Kapiteln gelernt haben.

Genau darin liegt der wichtigste Schritt beim Programmieren: Aus einzelnen Befehlen, Funktionen und Ideen entsteht ein eigenes Programm.

## 51.2. Die Spielidee

Unser Abschlussprojekt soll überschaubar bleiben und trotzdem typische Elemente eines kleinen Arcade-Spiels enthalten.

Der Spieler steuert eine Figur über das Spielfeld und sammelt Münzen ein. Gleichzeitig bewegen sich mehrere Gegner über den Bildschirm.

Berührt der Spieler eine Münze, erhält er Punkte. Berührt ihn dagegen ein Gegner, verliert er ein Leben.

Sind alle Münzen eingesammelt, beginnt das nächste Level. Die Münzen werden neu verteilt und die Gegner werden etwas schneller.

Das Spiel endet, sobald keine Leben mehr übrig sind.

Damit besitzt unser Spiel bereits viele typische Bestandteile eines klassischen Computerspiels:

- Eingabe,
- Bewegung,
- Sprites,
- Kollisionen,
- Punktestand,
- mehrere Level,
- Soundeffekte
- und eine ständig wiederholte Spielschleife.

## 51.3. Spielregeln

Die Regeln unseres Spiels sind schnell erklärt.

- Zu Beginn wählt der Spieler die gewünschte Steuerung: Cursortasten, Maus, Gamepad-Tasten oder den linken Analogstick eines USB-Gamepads.
- Jede eingesammelte Münze bringt Punkte.
- Mehrere Gegner bewegen sich selbstständig über den Bildschirm.

- Berührt ein Gegner den Spieler, geht ein Leben verloren.
- Sind alle Münzen eingesammelt, beginnt das nächste Level.
- Mit jedem Level bewegen sich die Gegner schneller.
- Sind keine Leben mehr vorhanden, endet das Spiel.
- Nach `GAME OVER` kann der Spieler entscheiden, ob eine neue Runde gestartet werden soll.

Mehr Regeln benötigen wir zunächst nicht.

Gerade einfache Spielideen eignen sich hervorragend, um sich auf die Programmstruktur zu konzentrieren.

# 51.4. Das Spielfeld

Bevor wir mit dem Programmieren beginnen, überlegen wir uns den Aufbau des Spielfeldes.

```
+-------------------------------------------------+
| Punkte: 120          Level: 2          Leben: 3 |
+-------------------------------------------------+
|
|        [M]
|
|                        [G]
|
|   [S]
|
|                [M]
|
|                                [G]
|
|          [M]
|
+-------------------------------------------------+
```

Die Darstellung ist hier nur schematisch.

`[S]` steht für den Spieler, `[M]` für eine Münze und `[G]` für einen Gegner. Im eigentlichen Programm verwenden wir dafür kleine Sprite-Grafiken.

Am oberen Bildschirmrand befindet sich zusätzlich eine Statuszeile mit

- dem aktuellen Punktestand,
- dem aktuellen Level
- und den verbleibenden Leben.

## 51.5. Für welches System schreiben wir das Spiel?

Das Beispiel in diesem Kapitel ist auf einen PicoMite mit VGA- oder HDMI-Ausgabe ausgelegt. Besonders gut passt es damit zu unserem PicoMite HDMI/USB.

Die verbesserte Version des Spiels ist bei der Eingabe deutlich flexibler: Sie kann mit Tastatur, USB-Maus, Gamepad-Tasten oder dem linken Analogstick eines USB-Gamepads gesteuert werden.

Die Sprite-Befehle arbeiten bei VGA und HDMI in den dafür vorgesehenen Grafikmodi. Für unser Spiel verwenden wir `MODE 2`.

Andere PicoMite-Varianten können ebenfalls Sprite-Funktionen besitzen, wenn ein geeigneter Framebuffer verwendet wird. Die genaue Verfügbarkeit hängt jedoch von der jeweiligen Firmware-Variante und der Display-Konfiguration ab.

Das Abschlussprogramm in diesem Kapitel konzentriert sich deshalb bewusst auf VGA/HDMI. Für Maus und Gamepad wird eine PicoMite-Variante mit entsprechender USB-Unterstützung benötigt.

## 51.6. Planung vor dem Programmieren

Auch bei einem kleinen Spiel lohnt sich eine kurze Planung.

Unser Programm muss mehrere Aufgaben erledigen:

- Grafikmodus vorbereiten,
- Sprite-Grafiken anlegen,
- Spielvariablen initialisieren,
- Münzen platzieren,
- Gegner platzieren,
- gewünschte Steuerung auswählen,
- Cursortasten, Maus oder Gamepad einlesen,
- Spieler bewegen,
- Gegner bewegen,
- Sprites gemeinsam verschieben,
- Kollisionen prüfen,
- Punktestand und Leben aktualisieren,
- das nächste Level starten,
- Soundeffekte zeitgesteuert beenden,
- das Spielende anzeigen
- und auf Wunsch eine neue Runde starten.

Diese Aufgaben werden wir nicht alle direkt in das Hauptprogramm schreiben.

Stattdessen zerlegen wir das Spiel in kleine SUB-Programme und Funktionen.

Das macht den Programmcode wesentlich leichter verständlich.

## 51.7. Die Programmstruktur

Der grundsätzliche Ablauf sieht so aus:

```
Programmstart

    |
    v

Grafikmodus einstellen

    |
    v

Steuerung auswählen

    |
    v

Sprites erzeugen

    |
    v

Spiel initialisieren

    |
    v

+-------------------------+
|       GAME LOOP         |
|                         |
|  Eingabe lesen          |
|  Spieler bewegen        |
|  Gegner bewegen         |
|  Sprites verschieben    |
|  Kollisionen prüfen     |
|  Level prüfen           |
|  Status anzeigen        |
|  Sound aktualisieren    |
+-------------------------+

    |
    v
```

```
GAME OVER

    |
    v

Nochmal spielen?
```

Die zentrale Spielschleife bleibt dadurch relativ kurz. Die eigentliche Arbeit übernehmen die Unterprogramme.

## 51.8. Schritt 1: Welche Daten benötigen wir?

Bevor wir die erste Programmzeile schreiben, überlegen wir, welche Informationen während des Spiels gespeichert werden müssen.

Für den Spieler benötigen wir beispielsweise

- die X-Position,
- die Y-Position,
- den Punktestand
- und die Anzahl der Leben.

Für jeden Gegner benötigen wir

- eine X-Position,
- eine Y-Position,
- eine Bewegungsrichtung in X-Richtung
- und eine Bewegungsrichtung in Y-Richtung.

Für jede Münze benötigen wir

- eine X-Position,
- eine Y-Position
- und die Information, ob sie noch sichtbar ist.

Eine mögliche Übersicht sieht so aus:

| **Information** | **Bedeutung** |
|---|---|
| `PlayerX` | X-Koordinate des Spielers |
| `PlayerY` | Y-Koordinate des Spielers |
| `Score` | Aktueller Punktestand |
| `Lives` | Verbleibende Leben |
| `Level` | Aktuelles Spiellevel |

| Information | Bedeutung |
|---|---|
| `EnemyX()` | X-Koordinaten der Gegner |
| `EnemyY()` | Y-Koordinaten der Gegner |
| `EnemyDX()` | Bewegung der Gegner in X-Richtung |
| `EnemyDY()` | Bewegung der Gegner in Y-Richtung |
| `CoinX()` | X-Koordinaten der Münzen |
| `CoinY()` | Y-Koordinaten der Münzen |
| `CoinVisible()` | Gibt an, ob eine Münze noch vorhanden ist |

### 51.8.1. Warum Arrays?

Wir besitzen mehrere Gegner und mehrere Münzen.

Natürlich könnten wir für jeden Gegner eigene Variablen anlegen:

```
Enemy1X = 100
Enemy2X = 180
Enemy3X = 250
Enemy4X = 320
```

Das wird jedoch schnell unübersichtlich.

Mit einem Array können wir dieselben Informationen wesentlich eleganter speichern:

```
EnemyX(1) = 100
EnemyX(2) = 180
EnemyX(3) = 250
EnemyX(4) = 320
```

Der große Vorteil zeigt sich anschließend in einer Schleife:

```
FOR i = 1 TO MaxEnemies
    EnemyX(i) = EnemyX(i) + EnemyDX(i)
NEXT i
```

Genau für solche Aufgaben sind Arrays gedacht.

## 51.9. Schritt 2: Konstanten festlegen

Viele Werte unseres Spiels ändern sich während der Programmausführung nicht.

Dazu gehören beispielsweise

- die Anzahl der Gegner,
- die Anzahl der Münzen,
- die Startanzahl der Leben,
- der Punktewert einer Münze
- und die Sprite-Nummern.

Solche Werte legen wir als Konstanten fest.

```
CONST MaxEnemies = 4
CONST MaxCoins = 10
CONST StartLives = 3
CONST CoinPoints = 10

CONST PlayerSpeed = 5
CONST EnemyStartSpeed = 1

CONST PlayerSprite = 1
CONST FirstEnemySprite = 2
CONST FirstCoinSprite = 6

CONST SpriteSize = 12
CONST PlayfieldTop = 32
CONST MouseChannel = 2
CONST GamepadChannel = 3

CONST DeadZoneLow = 110
CONST DeadZoneHigh = 146
```

Damit sind die Sprite-Nummern eindeutig vergeben:

| **Sprite-Nummer** | **Verwendung** |
|---|---|
| 1 | Spieler |
| 2 bis 5 | vier Gegner |
| 6 bis 15 | zehn Münzen |

Sprechende Konstantennamen sind fast immer besser als fest eingetragene Zahlen.

`FirstCoinSprite + i - 1` ist beispielsweise leichter nachvollziehbar als eine scheinbar zufällige Zahl irgendwo im Programm.

### 51.9.1. Keine unnötigen magischen Zahlen

Die Level unseres Spiels beginnen nicht bei einem bestimmten Punktestand, sondern immer dann, wenn alle Münzen eingesammelt wurden.

Wir benötigen deshalb keine Konstante wie

```
CONST NextLevelScore = 500
```

Der Levelwechsel ergibt sich direkt aus der Spielregel.

## 51.10. Schritt 3: Grafikmodus vorbereiten

Sprites benötigen bei VGA und HDMI einen geeigneten Grafikmodus.

Wir verwenden:

```
MODE 2
CLS RGB(BLACK)
```

Anschließend stehen uns `MM.HRES` und `MM.VRES` für die aktuelle Bildschirmbreite und Bildschirmhöhe zur Verfügung.

```
ScreenWidth = MM.HRES
ScreenHeight = MM.VRES
```

Damit müssen wir die Bildschirmgröße nicht als feste Zahl in das Programm schreiben.

Trotzdem ist unser Programm nicht automatisch auf jeder beliebigen PicoMite-Ausgabe lauffähig. Entscheidend ist, dass die verwendete Firmware und der Grafikmodus die Sprite-Funktionen unterstützen.

## 51.11. Schritt 4: Die Sprites erzeugen

Im vorherigen Kapitel haben wir die Sprite-Befehle kennengelernt.

Für unser Abschlussprojekt könnten wir die Grafiken aus Dateien laden. Dann wären neben dem BASIC-Programm jedoch zusätzliche Grafikdateien erforderlich.

Wir wählen deshalb einen anderen Weg: Die kleinen Spielfiguren werden direkt im BASIC-Programm erzeugt.

Dafür eignet sich

```
SPRITE LOADARRAY #n, Breite, Hoehe, array%()
```

Das Array enthält die RGB888-Farbwerte der einzelnen Pixel.

Für einen kleinen 12 x 12 Pixel großen Spieler könnte die Vorbereitung beispielsweise so beginnen:

```
DIM INTEGER PlayerImage(12 * 12 - 1)
```

```
FOR i = 0 TO 12 * 12 - 1
    PlayerImage(i) = RGB(GREEN)
NEXT i

SPRITE LOADARRAY #PlayerSprite, 12, 12, PlayerImage()
```

Im vollständigen Programm gestalten wir Spieler, Gegner und Münze etwas schöner als einfarbige Rechtecke.

### 51.11.1. Ein Bild, mehrere Sprites

Alle vier Gegner sehen gleich aus. Wir müssen die Grafik deshalb nicht viermal im Speicher ablegen.

Zunächst erzeugen wir das erste Gegner-Sprite. Anschließend können wir es kopieren:

```
SPRITE COPY #FirstEnemySprite, #(FirstEnemySprite + 1), MaxEnemies - 1
```

Dasselbe Prinzip lässt sich für die Münzen verwenden.

Dadurch teilen sich mehrere Sprites dieselbe Grafik, besitzen aber trotzdem jeweils eine eigene Position und einen eigenen sichtbaren Zustand.

## 51.12. Schritt 5: Das Spiel initialisieren

Bevor die Game Loop beginnen kann, setzen wir alle Spielvariablen auf ihre Startwerte.

```
Score = 0
Lives = StartLives
Level = 1
EnemySpeedNow = EnemyStartSpeed

PlayerX = MM.HRES \ 2 - SPRITE(W, PlayerSprite) \ 2
PlayerY = MM.VRES \ 2 - SPRITE(H, PlayerSprite) \ 2
```

Mit `SPRITE(W, n)` und `SPRITE(H, n)` können wir Breite und Höhe eines geladenen Sprites abfragen.

Dadurch müssen wir die Sprite-Größe nicht an mehreren Stellen im Programm wiederholen.

### 51.12.1. Brauchen wir `RANDOMIZE`?

Für die Positionen unserer Münzen und Gegner verwenden wir `RND`.

Auf dem RP2350 liefert `RND` Zufallswerte direkt aus der dafür vorgesehenen Hardware. Ein vorheriges `RANDOMIZE` ist dort nicht erforderlich.

Für unser PicoMite-HDMI/USB-System mit RP2350 können wir daher direkt mit `RND` arbeiten.

```
x = INT(RND * 200)
```

Bei älteren RP2040-Systemen arbeitet `RND` als Pseudozufallszahlengenerator. Dort kann `RANDOMIZE` verwendet werden, um den Generator neu zu initialisieren.

## 51.13. Schritt 6: Gegner platzieren

Die Gegner erscheinen zu Beginn an zufälligen Stellen des Spielfeldes.

Dabei müssen wir darauf achten, dass sie nicht in der Statuszeile liegen und nicht über den rechten oder unteren Bildschirmrand hinausragen.

```
FOR i = 1 TO MaxEnemies
    EnemyX(i) = INT(RND * (MM.HRES - SPRITE(W, FirstEnemySprite)))
    EnemyY(i) = PlayfieldTop + _
                INT(RND * (MM.VRES - PlayfieldTop - _
                SPRITE(H, FirstEnemySprite)))
NEXT i
```

Zusätzlich erhält jeder Gegner eine Bewegungsrichtung.

```
IF RND < 0.5 THEN
    EnemyDX(i) = -EnemySpeedNow
ELSE
    EnemyDX(i) = EnemySpeedNow
ENDIF

IF RND < 0.5 THEN
    EnemyDY(i) = -EnemySpeedNow
ELSE
    EnemyDY(i) = EnemySpeedNow
ENDIF
```

Damit bewegen sich die Gegner nicht alle in dieselbe Richtung.

## 51.14. Schritt 7: Münzen verteilen

Auch die Münzen werden zufällig verteilt.

```
FOR i = 1 TO MaxCoins
    CoinX(i) = INT(RND * (MM.HRES - SPRITE(W, FirstCoinSprite)))
    CoinY(i) = PlayfieldTop + _
               INT(RND * (MM.VRES - PlayfieldTop - _
               SPRITE(H, FirstCoinSprite)))
```

```
    CoinVisible(i) = 1
NEXT i
```

Wird eine Münze eingesammelt, setzen wir den zugehörigen Eintrag später auf `0`.

## 51.15. Schritt 8: Sprites anzeigen

Die Startpositionen stehen nun fest.

Jetzt können wir die Sprites zum ersten Mal anzeigen.

```
SPRITE SHOW #PlayerSprite, PlayerX, PlayerY, 1

FOR i = 1 TO MaxEnemies
    SpriteNo = FirstEnemySprite + i - 1
    SPRITE SHOW #SpriteNo, EnemyX(i), EnemyY(i), 1
NEXT i

FOR i = 1 TO MaxCoins
    SpriteNo = FirstCoinSprite + i - 1
    SPRITE SHOW #SpriteNo, CoinX(i), CoinY(i), 1
NEXT i
```

Wir verwenden für alle Spielfiguren denselben Layer. Dadurch können sie miteinander kollidieren.

## 51.16. Schritt 9: Die Steuerung auswählen und einlesen

Die überarbeitete Version unseres Spiels ist nicht mehr auf den Analogstick eines Gamepads beschränkt. Direkt nach `MODE 2` fragt das Programm mit `GameControl()` ab, wie der Spieler steuern möchte:

```
Cursortasten....: 1
Maus............: 2
Gamepad-Button.: 3
Gamepad-Stick..: 4
```

Die Auswahl wird in der Variablen `Control` gespeichert. In der Game Loop genügt anschließend ein Aufruf von `ReadControl`.

```
SUB ReadControl
    IF Control = 2 THEN
        ReadMouse
    ELSEIF Control = 3 THEN
        ReadGamepadButtons
    ELSEIF Control = 4 THEN
        ReadGamepadStick
```

```
    ELSE
        ReadKeyboard
    ENDIF
END SUB
```

Damit bleibt die eigentliche Spielsteuerung vom restlichen Programm getrennt.

`ReadControl` bildet hier eine kleine Abstraktionsschicht zwischen Eingabegerät und Spiel. Tastatur, Maus und Gamepad funktionieren technisch sehr unterschiedlich. Für den Rest des Spiels liefern sie aber immer dieselben beiden Werte: `MoveX` und `MoveY`.

Dadurch müssen `MovePlayer`, die Spielregeln und die Game Loop überhaupt nicht wissen, welches Eingabegerät der Spieler gewählt hat. Dieses Prinzip ist auch bei größeren Programmen sehr nützlich.

### 51.16.1. Cursortasten

Für die Tastatur verwenden wir die vier Cursortasten. Statt `INKEY$` kommt hier `KEYDOWN()` zum Einsatz. Das hat einen wichtigen Vorteil: `KEYDOWN(0)` liefert die Anzahl der gleichzeitig gedrückten Tasten und mit `KEYDOWN(i)` können wir diese einzeln abfragen.

Damit sind auch diagonale Bewegungen möglich. Werden beispielsweise `Cursor hoch` und `Cursor rechts` gleichzeitig gedrückt, erhält der Spieler sowohl eine Bewegung in X- als auch in Y-Richtung.

Die Cursortasten liefern dabei die Codes 128 bis 131:

- 128 = Cursor hoch
- 129 = Cursor runter
- 130 = Cursor links
- 131 = Cursor rechts

```
SUB ReadKeyboard
    LOCAL INTEGER i, key

    MoveX = 0
    MoveY = 0

    FOR i = 1 TO KEYDOWN(0)
        key = KEYDOWN(i)

        SELECT CASE key
            CASE 128
                MoveY = -PlayerSpeed

            CASE 129
                MoveY = PlayerSpeed
```

```
            CASE 130
                MoveX = -PlayerSpeed

            CASE 131
                MoveX = PlayerSpeed
        END SELECT
    NEXT i
END SUB
```

Durch die gemeinsamen Variablen `MoveX` und `MoveY` unterscheidet sich der restliche Programmablauf nicht von Maus- oder Gamepad-Steuerung.

`KEYDOWN()` ist für eine Spielsteuerung besser geeignet als `INKEY$`, wenn mehrere Tasten gleichzeitig erkannt werden sollen. Genau das benötigen wir für diagonale Bewegungen mit den Cursortasten.

### 51.16.2. Maus

Bei der Maus vergleichen wir die aktuelle Position mit der Position des vorherigen Schleifendurchlaufs. Entscheidend ist also nicht die absolute Mausposition, sondern die Bewegungsrichtung.

```
x = DEVICE(MOUSE MouseChannel, X)
dx = x - MouseXPos

IF dx < 0 THEN
    MoveX = -PlayerSpeed
ELSEIF dx > 0 THEN
    MoveX = PlayerSpeed
ELSE
    MoveX = 0
ENDIF
```

Für die Y-Richtung funktioniert das genauso.

### 51.16.3. Gamepad-Tasten

Auch die Tasten eines Gamepads können zur Steuerung verwendet werden. `DEVICE(GAMEPAD …, B)` liefert dabei ein Bitmuster. Mit `AND` prüfen wir, welche Richtungstaste gedrückt ist.

```
buttons% = DEVICE(GAMEPAD GamepadChannel, B)

IF (buttons% AND BtnLeft%) <> 0 THEN MoveX = -PlayerSpeed
IF (buttons% AND BtnRight%) <> 0 THEN MoveX = PlayerSpeed
IF (buttons% AND BtnUp%) <> 0 THEN MoveY = -PlayerSpeed
IF (buttons% AND BtnDown%) <> 0 THEN MoveY = PlayerSpeed
```

Die Konstanten `BtnDown%`, `BtnRight%`, `BtnUp%` und `BtnLeft%` enthalten die zu unserem Gamepad passenden Bitwerte.

### 51.16.4. Gamepad-Analogstick

Beim PicoMite HDMI/USB kann ein unterstütztes USB-Gamepad direkt über `DEVICE()` abgefragt werden.

Für den linken Analogstick interessieren uns die beiden Achsen

- `LX` für links/rechts
- und `LY` für oben/unten.

```
StickX = DEVICE(GAMEPAD GamepadChannel, "LX")
StickY = DEVICE(GAMEPAD GamepadChannel, "LY")
```

Die Achswerte liegen typischerweise zwischen `0` und `255`. Die Mittelstellung befindet sich ungefähr bei `128`.

Direkt mit diesen Werten können wir noch nichts anfangen. Wir wandeln sie deshalb in eine Bewegungsrichtung um.

```
MoveX = 0
MoveY = 0

IF StickX < 110 THEN MoveX = -PlayerSpeed
IF StickX > 146 THEN MoveX = PlayerSpeed

IF StickY < 110 THEN MoveY = -PlayerSpeed
IF StickY > 146 THEN MoveY = PlayerSpeed
```

Der Bereich zwischen `110` und `146` bildet eine kleine Totzone um die Mittelstellung.

Ohne diese Totzone könnte ein nicht perfekt zentrierter Analogstick dazu führen, dass sich die Spielfigur langsam von selbst bewegt.

Welchen USB-Kanal ein Gamepad erhält, hängt von den angeschlossenen USB-Geräten ab. Bei gleichzeitig verwendeter USB-Tastatur und USB-Maus ist Kanal 3 ein typischer Wert.

Falls dein Gamepad auf einem anderen Kanal erkannt wurde, passt du lediglich `GamepadChannel` an. Informationen über die USB-Geräte lassen sich mit den entsprechenden `MM.INFO(USB n)`-Abfragen ermitteln.

## 51.17. Schritt 10: Spieler bewegen

Unabhängig von der gewählten Eingabemethode liefern die vier Leseroutinen am Ende nur zwei

Werte: `MoveX` und `MoveY`.

Aus diesen Bewegungswerten berechnen wir die neue Spielerposition.

```
PlayerX = PlayerX + MoveX
PlayerY = PlayerY + MoveY
```

Anschließend begrenzen wir die Position auf das Spielfeld.

```
IF PlayerX < 0 THEN PlayerX = 0
IF PlayerY < PlayfieldTop THEN PlayerY = PlayfieldTop

IF PlayerX > MM.HRES - SPRITE(W, PlayerSprite) THEN
    PlayerX = MM.HRES - SPRITE(W, PlayerSprite)
ENDIF

IF PlayerY > MM.VRES - SPRITE(H, PlayerSprite) THEN
    PlayerY = MM.VRES - SPRITE(H, PlayerSprite)
ENDIF
```

Damit kann der Spieler weder den sichtbaren Bildschirm verlassen noch in die Statuszeile fahren.

## 51.18. Schritt 11: Gegner bewegen

Die Gegner ändern ihre Position automatisch.

```
FOR i = 1 TO MaxEnemies
    EnemyX(i) = EnemyX(i) + EnemyDX(i)
    EnemyY(i) = EnemyY(i) + EnemyDY(i)
NEXT i
```

Erreicht ein Gegner einen Rand des Spielfeldes, kehren wir die entsprechende Bewegungsrichtung um.

```
IF EnemyX(i) <= 0 THEN
    EnemyX(i) = 0
    EnemyDX(i) = ABS(EnemyDX(i))
ENDIF

IF EnemyX(i) >= MM.HRES - EnemyWidth THEN
    EnemyX(i) = MM.HRES - EnemyWidth
    EnemyDX(i) = -ABS(EnemyDX(i))
ENDIF
```

Für die vertikale Bewegung verwenden wir dasselbe Prinzip.

## 51.19. Schritt 12: Mehrere Sprites gemeinsam bewegen

Wir könnten jede neue Position sofort mit `SPRITE SHOW` anzeigen.

Bei mehreren bewegten Sprites ist jedoch `SPRITE NEXT` zusammen mit `SPRITE MOVE` die bessere Lösung.

Zunächst teilen wir MMBasic die neuen Positionen mit:

```
SPRITE NEXT #PlayerSprite, PlayerX, PlayerY

FOR i = 1 TO MaxEnemies
    SpriteNo = FirstEnemySprite + i - 1
    SPRITE NEXT #SpriteNo, EnemyX(i), EnemyY(i)
NEXT i
```

Erst danach führen wir die gesamte Bewegung aus:

```
SPRITE MOVE
```

Damit werden alle vorgemerkten Sprite-Positionen in einer gemeinsamen Operation aktualisiert.

Das ist nicht nur sauberer, sondern besonders für Spiele wichtig: Die Kollisionsprüfung erfolgt nach dieser gemeinsamen Bewegung.

## 51.20. Schritt 13: Die eingebaute Kollisionserkennung verwenden

Im ersten Entwurf dieses Kapitels hätten wir eine eigene Funktion schreiben können, die zwei Rechtecke miteinander vergleicht.

Das ist grundsätzlich möglich. In Kapitel 50 haben wir jedoch gesehen, dass der PicoMite bereits eine Kollisionserkennung für Sprites besitzt.

Genau diese verwenden wir hier.

```
SPRITE(C, PlayerSprite)
```

liefert die Anzahl der momentan erkannten Kollisionen des Spieler-Sprites.

Mit

```
SPRITE(C, PlayerSprite, i)
```

können wir anschließend feststellen, womit der Spieler kollidiert ist.

Das ist für unser Spiel besonders praktisch, weil die Sprite-Nummern bereits eindeutig vergeben sind.

In Kapitel 50 haben wir außerdem `SPRITE INTERRUPT` kennengelernt. Unser Abschlussspiel verwendet bewusst keine eigene Sprite-Interrupt-Routine. Stattdessen führt `SPRITE MOVE` die vorbereiteten Bewegungen aus und `CheckCollisions` fragt anschließend die dabei gespeicherten Kollisionsinformationen mit `SPRITE()` ab.

Für die überschaubare Game Loop dieses Spiels ist dieses Verfahren besonders leicht nachvollziehbar. Bei komplexeren Spielen kann ein Kollisionsinterrupt sinnvoll sein.

Die Kollisionsdaten gehören immer zur zuletzt ausgeführten Sprite-Operation. Deshalb erfolgt `CheckCollisions` in unserer Game Loop unmittelbar nach `MoveSprites`.

## 51.21. Schritt 14: Münzen einsammeln

Die Münzen besitzen die Sprite-Nummern 6 bis 15.

Wenn der Spieler mit einem Sprite aus diesem Bereich kollidiert, wurde eine Münze eingesammelt.

Der prinzipielle Ablauf sieht so aus:

```
HitCount = SPRITE(C, PlayerSprite)

FOR i = 1 TO HitCount
    HitList(i) = SPRITE(C, PlayerSprite, i)
NEXT i

FOR i = 1 TO HitCount
    HitSprite = HitList(i)

    IF HitSprite >= FirstCoinSprite AND _
       HitSprite < FirstCoinSprite + MaxCoins THEN

        CoinIndex = HitSprite - FirstCoinSprite + 1

        IF CoinVisible(CoinIndex) THEN
            CoinVisible(CoinIndex) = 0
            SPRITE HIDE SAFE HitSprite
            Score = Score + CoinPoints
            StartCoinSound
        ENDIF
    ENDIF
NEXT i
```

Damit übernimmt die Sprite-Verwaltung einen großen Teil der Arbeit für uns.

Neu ist hier `SPRITE HIDE SAFE`. Außerdem ist `HitList()` neu: Wir kopieren die aktuelle Kollisionsliste zunächst in ein eigenes Array. Das ist wichtig, weil `SPRITE HIDE SAFE` beim Einsammeln einer Münze den Sprite-Zustand verändert. Würden wir währenddessen immer wieder die ursprüngliche Kollisionsliste abfragen, könnte sich deren Inhalt bereits verändert haben.

Wir müssen weder Breite und Höhe beider Objekte vergleichen noch selbst berechnen, ob sich zwei Rechtecke überschneiden.

### 51.21.1. Ein kurzer Soundeffekt

Beim Einsammeln einer Münze soll der Spieler eine akustische Rückmeldung erhalten.

Dazu können wir mit `PLAY SOUND` einen kurzen Ton abspielen.

Welche Audioausgabe zur Verfügung steht, hängt von der verwendeten PicoMite-Variante und ihrer Konfiguration ab.

Im vollständigen Programm verwenden wir deshalb einfache Soundeffekte, die zum HDMI/USB-Aufbau des Tutorials passen.

Die Töne werden nicht mit einer zusätzlichen `PAUSE` beendet. Stattdessen merken wir uns mit `Timer`, wann ein Ton ausgeschaltet werden soll. `UpdateSound` wird in jedem Durchlauf der Game Loop aufgerufen.

```
SUB StartCoinSound
    PLAY SOUND 1, B, Q, 1200, 10
    SoundActive = 1
    SoundOffAt = TIMER + 60
END SUB

SUB UpdateSound
    IF SoundActive THEN
        IF TIMER >= SoundOffAt THEN
            PLAY SOUND 1, B, 0
            SoundActive = 0
        ENDIF
    ENDIF
END SUB
```

So blockiert der kurze Soundeffekt die Spielschleife nicht.

Das ist ein wichtiges Muster für ereignisgesteuerte Programme: `StartCoinSound` startet nur den Vorgang und merkt sich mit `SoundOffAt`, wann er beendet werden soll. Die Game Loop läuft währenddessen weiter. `UpdateSound` prüft später, ob der Zeitpunkt erreicht ist.

Eine zusätzliche `PAUSE 60` direkt in `StartCoinSound` wäre zwar kürzer, würde aber während dieser Zeit Eingabe, Bewegung und Kollisionserkennung anhalten.

# 51.22. Schritt 15: Kollision mit einem Gegner

Die Gegner besitzen die Sprite-Nummern 2 bis 5.

Entsprechend leicht können wir erkennen, ob der Spieler einen Gegner berührt hat.

```
IF EnemyHit THEN
    Lives = Lives - 1
    StartHitSound

    IF Lives > 0 THEN
        ResetPlayer
        SPRITE SHOW SAFE PlayerSprite, PlayerX, PlayerY, 1
        PAUSE 300
    ENDIF

    EXIT SUB
ENDIF
```

Nach einer Kollision setzen wir den Spieler zurück in die Bildschirmmitte.

```
SUB ResetPlayer
    PlayerX = MM.HRES \ 2 - SPRITE(W, PlayerSprite) \ 2
    PlayerY = MM.VRES \ 2 - SPRITE(H, PlayerSprite) \ 2
END SUB
```

`ResetPlayer` berechnet nur die neue Startposition. Das erneute Anzeigen mit `SPRITE SHOW SAFE` und die kurze Pause erfolgen in `CheckCollisions`.

Außerdem haben Gegnerkollisionen Vorrang vor Münzen. Berührt der Spieler in demselben Moment einen Gegner und eine Münze, verliert er zuerst ein Leben und die Kollisionsroutine wird beendet. Dadurch bleibt das Verhalten eindeutig.

Die kurze Pause verhindert, dass direkt mehrere Leben durch dieselbe Berührung verloren gehen. Anders als unsere Soundsteuerung ist diese `PAUSE 300` bewusst blockierend: Für einen kurzen Moment steht das Spiel still.

Für ein größeres Spiel wäre eine zeitgesteuerte Unverwundbarkeitsphase mit `TIMER` die elegantere Lösung. Für unser Abschlussprojekt ist die kurze Pause leichter verständlich und erfüllt ihren Zweck.

# 51.23. Schritt 16: Sind noch Münzen vorhanden?

Für den Levelwechsel müssen wir wissen, ob noch Münzen auf dem Spielfeld liegen.

Dafür schreiben wir eine kleine Funktion:

```
FUNCTION CoinsRemaining()
    LOCAL INTEGER i, Count

    Count = 0

    FOR i = 1 TO MaxCoins
        IF CoinVisible(i) THEN Count = Count + 1
    NEXT i

    CoinsRemaining = Count
END FUNCTION
```

In der Game Loop genügt anschließend:

```
IF CoinsRemaining() = 0 THEN
    StartNextLevel
ENDIF
```

## 51.24. Schritt 17: Das nächste Level

Sind alle Münzen eingesammelt, erhöhen wir die Levelnummer.

```
Level = Level + 1
EnemySpeedNow = EnemyStartSpeed + Level - 1
```

Anschließend werden die Münzen neu verteilt und wieder angezeigt.

Auch die Bewegungswerte der Gegner werden an die neue Geschwindigkeit angepasst.

Damit wird das Spiel mit jedem Level etwas schwieriger.

Wir erhöhen die Geschwindigkeit bewusst nur langsam. Bei Pixelbewegungen kann bereits eine Erhöhung um einen einzigen Pixel pro Schleifendurchlauf deutlich spürbar sein.

## 51.25. Schritt 18: Die Statusanzeige

Am oberen Bildschirmrand zeigen wir Punktestand, Level und Leben an.

```
BOX 0, 0, MM.HRES, PlayfieldTop - 1, 0, RGB(BLACK), RGB(BLACK)

TEXT 4, 8, "Punkte: " + STR$(Score), L, 1, 1, RGB(WHITE)
TEXT MM.HRES \ 2, 8, "Level: " + STR$(Level), C, 1, 1, RGB(WHITE)
TEXT MM.HRES - 4, 8, "Leben: " + STR$(Lives), R, 1, 1, RGB(WHITE)
```

Nur die Statuszeile wird neu gezeichnet.

Das Spielfeld selbst bleibt erhalten. Dadurch vermeiden wir, den kompletten Bildschirm in jedem Schleifendurchlauf löschen und neu aufbauen zu müssen.

## 51.26. Schritt 19: Die Game Loop

Jetzt besitzen wir alle wichtigen Bestandteile.

Die zentrale Spielschleife kann deshalb erstaunlich kompakt bleiben:

```
DO WHILE Lives > 0

    ReadControl
    MovePlayer
    MoveEnemies
    MoveSprites
    CheckCollisions

    IF Lives > 0 THEN
        IF CoinsRemaining() = 0 THEN StartNextLevel
    ENDIF

    UpdateStatus
    UpdateSound

    PAUSE 16
LOOP
```

Diese Schleife liest sich fast wie eine Aufgabenliste.

Genau das ist Absicht.

Die Details befinden sich in den einzelnen SUB-Programmen. Dadurch bleibt sofort erkennbar, was in jedem Durchlauf geschieht.

## 51.27. Warum PAUSE 16?

Ohne Pause würde die Schleife so schnell laufen, wie der PicoMite sie ausführen kann.

Mit

```
PAUSE 16
```

wartet das Programm nach jedem Schleifendurchlauf ungefähr 16 Millisekunden.

Rein rechnerisch entsprechen 16 Millisekunden

```
1000 ms / 16 ms = 62,5
```

Durchläufen pro Sekunde.

Das bedeutet jedoch nicht, dass unser Spiel tatsächlich exakt mit 62,5 Bildern pro Sekunde läuft.

Auch

- das Einlesen des gewählten Eingabegeräts,
- die BASIC-Berechnungen,
- die Sprite-Bewegungen,
- die Kollisionsprüfung
- und die Textausgabe

benötigen Zeit.

Die tatsächliche Bildrate liegt deshalb niedriger.

Für unser kleines Spiel ist diese einfache Lösung vollkommen ausreichend.

Dabei ist auch wichtig: `PlayerSpeed` und `EnemySpeedNow` geben in unserem Programm Pixel **pro Schleifendurchlauf** an. Ändert sich die Laufzeit der Game Loop deutlich, ändert sich deshalb auch die wahrgenommene Bewegungsgeschwindigkeit.

`PAUSE 16` ist keine exakte Bildratenregelung.

Bei anspruchsvolleren Spielen sollte die Bewegung anhand der tatsächlich vergangenen Zeit berechnet werden. Für unser Lernprojekt würde das die zentrale Idee jedoch unnötig kompliziert machen.

# 51.28. Schritt 20: Game Over

Sobald `Lives` den Wert `0` erreicht, endet die Game Loop.

Danach blenden wir die Sprites aus und zeigen den Endbildschirm an.

```
SPRITE HIDE ALL

CLS RGB(BLACK)

TEXT MM.HRES \ 2, MM.VRES \ 2 - 28, _
     "GAME OVER", C, 2, 2, RGB(RED)

TEXT MM.HRES \ 2, MM.VRES \ 2 + 12, _
     "Punkte: " + STR$(Score), C, 1, 1, RGB(WHITE)

TEXT MM.HRES \ 2, MM.VRES \ 2 + 40, _
```

```
     "Level: " + STR$(Level), C, 1, 1, RGB(WHITE)
```

Damit besitzt das Spiel einen vollständigen Ablauf:

```
Initialisieren

    |
    v

Spiel starten

    |
    v

Game Loop

    |
    +--> Eingabe
    +--> Bewegung
    +--> Kollision
    +--> Punkte
    +--> Level

    |
    v

Game Over
```

# 51.29. Schritt 21: Noch einmal spielen?

Die verbesserte Version endet nach `GAME OVER` nicht sofort. Mit `PlayAgain$()` fragen wir, ob eine weitere Runde gestartet werden soll.

```
AGAIN$ = PlayAgain$()

LOOP WHILE AGAIN$ = "J"
```

`PlayAgain$()` akzeptiert nur `J` oder `N`. Bei `J` beginnt die äußere Schleife erneut. Dabei werden alte Sprite-Puffer vorsichtshalber mit `SPRITE CLOSE ALL` freigegeben, der Bildschirm gelöscht und das Spiel vollständig neu initialisiert.

Die einmal gewählte Steuerungsart bleibt dabei erhalten. Sie wird nur beim Programmstart abgefragt.

# 51.30. Was haben wir dabei verwendet?

Unser kleines Abschlussprojekt verbindet eine erstaunlich große Zahl der Themen aus diesem Buch.

| Thema | Verwendung im Spiel |
|---|---|
| Variablen | Positionen, Punkte, Leben und Level |
| Konstanten | Anzahl der Objekte, Geschwindigkeiten und Sprite-Nummern |
| Arrays | Positionen und Zustände mehrerer Gegner und Münzen |
| `IF` | Grenzen, Eingabeauswertung und Spielregeln |
| `FOR` | Bearbeitung mehrerer Gegner und Münzen |
| `DO ⋯ LOOP` | Zentrale Game Loop |
| `SUB` | Aufteilung des Spiels in überschaubare Aufgaben |
| `FUNCTION` | Steuerung auswählen, Münzen zählen und Neustart abfragen |
| `RND` | Zufällige Positionen und Bewegungsrichtungen |
| `KEYDOWN()` und `DEVICE()` | Auslesen von Cursortasten, Maus und USB-Gamepad |
| Grafikbefehle | Statuszeile und Endbildschirm |
| Sprites | Spieler, Gegner, Münzen, Bewegung und Kollisionen |
| Sound und `TIMER` | Akustische Rückmeldung ohne blockierende Wartezeit |

# 51.31. Vom ersten Programm zum eigenen Spiel

Wenn du an die ersten Kapitel dieses Buches zurückdenkst, bestand ein BASIC-Programm vielleicht noch aus wenigen Zeilen wie

```
PRINT "Hallo Welt"
```

Jetzt arbeiten in einem einzigen Programm

- mehrere Arrays,
- zahlreiche Variablen,
- verschiedene Unterprogramme,
- vier Eingabemöglichkeiten,
- Grafiken,
- Sprites,
- Kollisionen,
- Sound
- und eine komplette Spielschleife

zusammen.

Der entscheidende Unterschied liegt jedoch nicht in der Anzahl der verwendeten Befehle.

Du hast gelernt, ein Problem in kleinere Teilaufgaben zu zerlegen.

Genau das ist Programmieren.

## 51.32. Wie könnte es weitergehen?

Unser Spiel ist bewusst einfach gehalten. Es bietet deshalb viele Möglichkeiten für eigene Erweiterungen.

Du könntest beispielsweise

- verschiedene Gegnertypen hinzufügen,
- die Zahl der Gegner mit dem Level erhöhen,
- mehrere Münzsorten mit unterschiedlichen Punktwerten einbauen,
- Bonusgegenstände ergänzen,
- eine Highscore-Datei speichern,
- ein Startmenü programmieren,
- einen Pausenmodus hinzufügen,
- eigene Sprite-Grafiken aus PNG-Dateien laden,
- unterschiedliche Spielfelder gestalten
- oder einen zweiten Spieler hinzufügen.

Dabei musst du nicht alles auf einmal ändern.

Nimm dir eine Idee vor, überlege welche Daten und Unterprogramme du dafür benötigst und erweitere das Spiel Schritt für Schritt.

So entstehen auch größere Programme.

## 51.33. Fazit

Wir haben dieses Buch mit kleinen BASIC-Programmen begonnen.

Im Laufe der Kapitel kamen Variablen, Bedingungen, Schleifen, Grafik, Eingabegeräte, Dateien, Unterprogramme und viele weitere Möglichkeiten hinzu.

Im Abschlussprojekt haben wir diese Bausteine zu einem vollständigen Spiel zusammengesetzt.

Dabei war nicht ein einzelner MMBasic-Befehl entscheidend.

Entscheidend war die Struktur:

- eine Idee entwickeln,

- die benötigten Daten bestimmen,
- das Problem in Teilaufgaben zerlegen,
- passende Werkzeuge auswählen,
- klare Schnittstellen zwischen den Teilaufgaben schaffen,
- die Teile programmieren
- und sie anschließend zusammenfügen.

Wenn du dieses Prinzip verstanden hast, kannst du weit über die Beispiele dieses Buches hinausgehen.

Dann beginnt der interessanteste Teil:

Deine eigenen Programme.

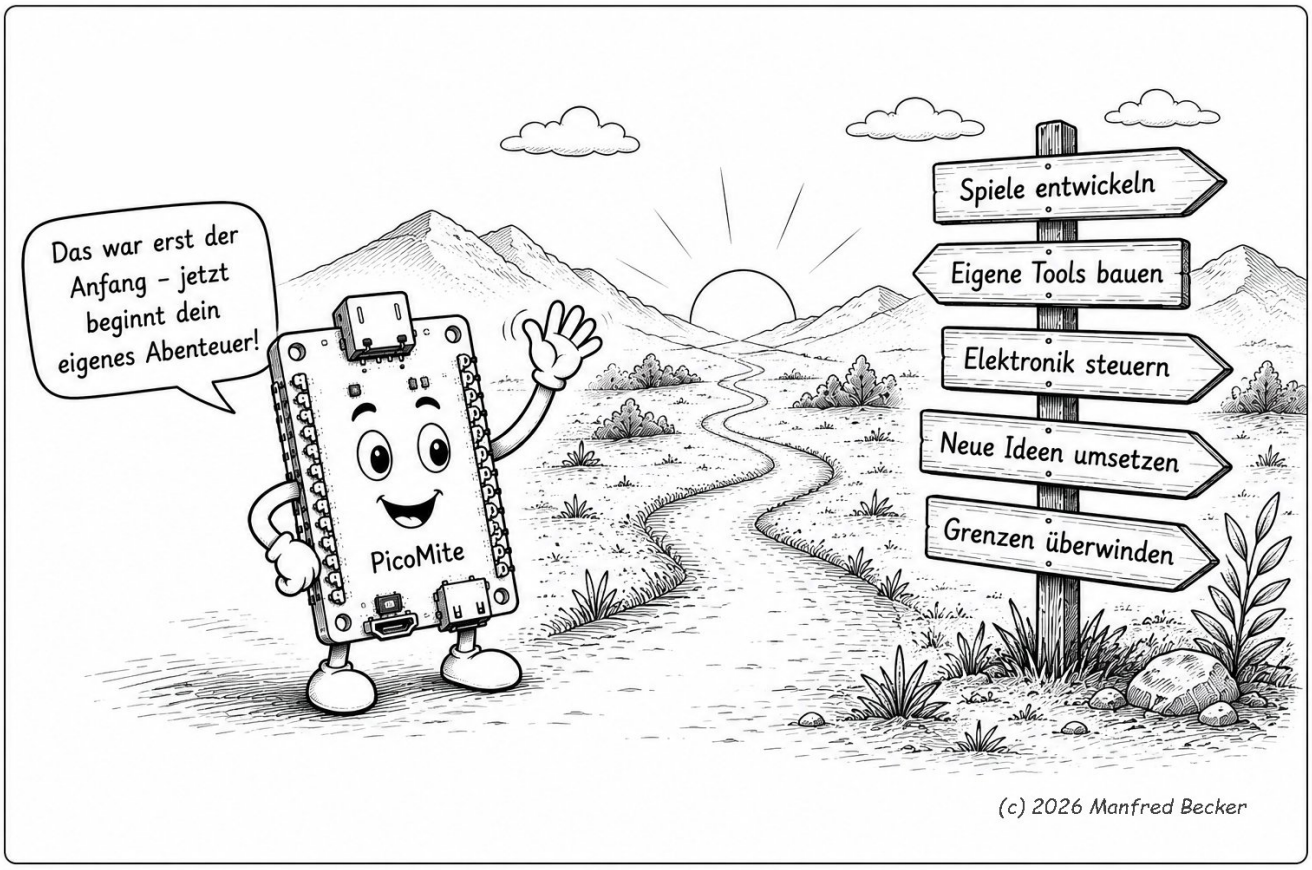
## 51.34. Programmbeispiel

Das vollständige BASIC-Programm zu diesem Kapitel findest du hier:

- 51_abschlussspiel.bas

# Epilog

# Kapitel 52. Wie geht es jetzt weiter?




## 52.1. Herzlichen Glückwunsch!

Wenn du dieses Buch bis hierhin durchgearbeitet hast, hast du einen wichtigen Meilenstein erreicht.

Du kannst jetzt eigene Programme in MMBasic schreiben, mit Variablen und Arrays arbeiten, Entscheidungen treffen, Schleifen verwenden, Unterprogramme und Funktionen erstellen, Grafiken zeichnen, Sounds erzeugen, Dateien verwalten, Eingabegeräte abfragen und sogar eigene kleine Computerspiele entwickeln.

Vor allem aber hast du etwas viel Wichtigeres gelernt:

Du hast gelernt, Probleme in kleine, überschaubare Schritte zu zerlegen und daraus funktionierende Programme zu entwickeln.

Genau darum geht es beim Programmieren.

Natürlich wirst du noch längst nicht jede Aufgabe sofort lösen können. Auch erfahrene Programmierer können längst nicht jede Aufgabe sofort lösen.

Sie schlagen Befehle nach, lesen Dokumentationen, testen Ideen und suchen nach neuen Lösungswegen. Das gehört zum Programmieren einfach dazu.

Lass dich deshalb nicht entmutigen, wenn einmal etwas nicht auf Anhieb funktioniert.

Fehlermeldungen gehören zum Programmieren dazu. Sie geben dir Hinweise darauf, wo du weitersuchen musst.

Fast jedes Programm entsteht durch Ausprobieren, Testen und Verbessern – oft viele Male hintereinander.

## 52.2. Jetzt bist du dran

Das Abschlussprojekt aus dem vorherigen Kapitel war das letzte große Beispiel dieses Buches. Für dich muss es aber kein Abschluss sein.

Vielleicht hast du beim Lesen bereits eigene Ideen entwickelt. Jetzt ist der richtige Zeitpunkt, sie auszuprobieren.

Du könntest zum Beispiel

- das Abschlussspiel um weitere Gegner, Level oder Soundeffekte erweitern,
- einen Highscore dauerhaft in einer Datei speichern,
- eigene Spritegrafiken entwerfen,
- ein Jump-and-Run, Geschicklichkeits- oder Denkspiel programmieren,
- Messwerte über die Ein- und Ausgänge des PicoMite erfassen,
- LEDs, Taster oder andere Hardware in ein eigenes Projekt einbauen,
- ein kleines Werkzeug schreiben, das dir im Alltag nützlich ist,
- oder einfach mit einem bisher unbekannten MMBasic-Befehl experimentieren.

Dein nächstes Projekt muss dabei weder groß noch besonders kompliziert sein. Oft entstehen die interessantesten Programme aus einer ganz einfachen Idee.

## 52.3. Nutze die Dokumentation

Dieses Buch konnte nur einen Teil der Möglichkeiten von MMBasic zeigen. Im offiziellen PicoMite User Manual findest du viele weitere Befehle, Funktionen und Optionen.

Du musst sie nicht alle kennen – und schon gar nicht auswendig lernen.

Wenn du eine neue Aufgabe lösen möchtest, überlege zuerst, was dein Programm dafür benötigt, und suche dann gezielt nach den passenden Befehlen. Genau so arbeiten auch erfahrene Entwickler.

Auf dem PicoMite kann außerdem der Befehl `HELP` eine praktische Unterstützung sein. Voraussetzung dafür ist eine passende Datei `help.txt` im Hauptverzeichnis von Laufwerk `A:`. Wie du die Hilfe verwendest, hast du bereits im Kapitel über den Editor kennengelernt.

Und noch etwas ist wichtig:

MMBasic und die PicoMite-Firmware werden weiterentwickelt. Wenn sich ein Befehl anders verhält als erwartet, lohnt sich deshalb immer ein Blick in das Handbuch der tatsächlich

verwendeten Firmware-Version. Achte darauf, dass Handbuch und Firmware-Version zusammenpassen.

## 52.4. Tausche dich mit anderen aus

Programmieren macht noch mehr Spaß, wenn man seine Projekte mit anderen teilt.

Im Internet gibt es Foren und Communities rund um MMBasic, den PicoMite und den Colour Maximite 2. Dort findest du Ideen, Beispielprogramme, Hinweise zu neuen Firmware-Versionen und natürlich andere Menschen, die sich für dieselben Systeme interessieren.

Zeige deine Projekte, stelle Fragen und teile deine Erfahrungen. Vielleicht kannst du schon bald selbst jemandem weiterhelfen, der gerade seine ersten Schritte mit MMBasic macht.

## 52.5. Dein nächstes Programm

Nach all den Beispielprogrammen in diesem Buch gibt es an dieser Stelle kein weiteres fertiges Projekt mehr.

Denn das nächste Programm soll von dir kommen.

Als kleine Starthilfe findest du im Repository noch eine einfache Vorlage:

- 52_dein_naechstes_programm.bas

Sie enthält absichtlich fast nichts:

```
OPTION EXPLICIT

CLS

PRINT "Dein naechstes Programm beginnt hier."
PRINT
PRINT "Was moechtest du als Naechstes programmieren?"
PRINT
PRINT "Vielleicht ..."
PRINT "  - ein kleines Spiel?"
PRINT "  - ein nuetzliches Werkzeug?"
PRINT "  - ein Experiment mit Grafik oder Sound?"
PRINT "  - ein Projekt mit Ein- und Ausgaengen?"
PRINT
PRINT "Die erste Zeile ist schon geschrieben."
PRINT
PRINT "Jetzt bist du dran!"

' ------------------------------------------------------------
' Deine Idee beginnt hier ...
' ------------------------------------------------------------
```

Du kannst alles löschen, verändern oder ergänzen. Die Datei soll dir nur eines sagen:

Jetzt beginnt dein eigenes Programm.

## 52.6. Zum Schluss noch ein persönlicher Gedanke

Ich programmiere seit vielen Jahrzehnten. In dieser Zeit habe ich viele Programmiersprachen, Betriebssysteme und Computer kennengelernt.

Doch eines hat sich nie geändert:

Der schönste Moment ist immer der, wenn ein eigenes Programm zum ersten Mal genau das tut, was man sich vorgestellt hat.

Dieses Gefühl verliert man nie.

Ich wünsche dir, dass du noch viele solcher Momente erleben wirst.

Bleib neugierig. Experimentiere. Hab keine Angst vor Fehlern.

Und vor allem:

Hab Spaß am Programmieren.

Auch dein nächstes Programm beginnt wieder ganz einfach – mit der ersten Zeile.

```
PRINT "Hallo Welt"
```

Vielen Dank, dass ich dich auf deinem Einstieg in die Welt von MMBasic begleiten durfte.

Ich wünsche dir viel Erfolg – und vor allem viel Freude – bei deinen eigenen Projekten.

**Auf Wiedersehen …**

**Dies ist nicht das Ende. Es ist der Anfang deiner eigenen Programme.**

# Anhang

# Anhang A: MMBasic-Befehlsreferenz

Diese Referenz enthält die MMBasic-Befehle in alphabetischer Reihenfolge.

Zu jedem Befehl werden die Syntax, die Behandlung im Buch und die unterstützten Plattformen angegeben.

Die vollständige Beschreibung der Syntax und aller Parameter findest du im jeweiligen MMBasic-Handbuch.

*Tabelle 1. Legende der Plattformen*

| Abkürzung | Plattform | Abkürzung | Plattform | Abkürzung | Plattform | Abkürzung | Plattform |
|---|---|---|---|---|---|---|---|
| P | PicoMite | V | PicoMite VGA | H | PicoMite HDMI | W | WebMite |
| K | PicoCalc | | | | | | |

## A.1. Sonderzeichen

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| ' | ' (single quotation mark) | - | X | X | X | X | X |
| *file | *file | - | X | X | X | X | X |
| /* … */ | /* | - | X | X | X | X | X |
| ? | ? (question mark) | - | X | X | X | X | X |

## A.2. A

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| A: / B: | A: or B: | - | X | X | X | X | X |
| ADC | ADC | 307 | X | X | X | X | X |
| ADC CLOSE | ADC CLOSE | - | X | X | X | X | X |
| ADC FREQUENCY | ADC FREQUENCY freq | - | X | X | X | X | X |
| ADC OPEN | ADC OPEN freq, n_channels [,interrupt] | - | X | X | X | X | X |
| ADC RUN | ADC RUN array1%(),array2%) | - | X | X | X | X | X |
| ADC START | ADC START array1!() [,array2!()] [,array3!()] [,array4!()] [,C1min] [,C1max] [,C2min] [,C2max] [,C3min] | - | X | X | X | X | X |
| ARC | ARC x, y, r1, [r2], a1, a2 [, c] | 152 | X | X | X | - | X |
| ARRAY ADD | ARRAY ADD in(), value ,out() | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| ARRAY INSERT | ARRAY INSERT targetarray(), [d1] [,d2] [,d3] [,d4] [,d5] , sourcearray() | - | X | X | X | X | X |
| ARRAY SET | ARRAY SET value, array() | - | X | X | X | X | X |
| ARRAY SLICE | ARRAY SLICE sourcearray(), [d1] [,d2] [,d3] [,d4] [,d5] destinationarray() | - | X | X | X | X | X |
| AUTOSAVE | AUTOSAVE or | - | X | X | X | X | X |
| AUTOSAVE APPEND | AUTOSAVE APPEND or | - | X | X | X | X | X |
| AUTOSAVE CRUNCH | AUTOSAVE CRUNCH or | - | X | X | X | X | X |
| AUTOSAVE N | AUTOSAVE N | - | X | X | X | X | X |

# A.3. B

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| BACKLIGHT | BACKLIGHT n [,DEFAULT] BACKLIGHT n [,FreqInHz] | - | - | - | - | - | X |
| BEZIER | BEZIER x%(),y%() [,n] [,colour] | - | X | X | X | - | X |
| BIT() | BIT(var%, bitno) = value | - | X | X | X | X | X |
| BITSTREAM | BITSTREAM pin1, count1, array1() [, mode1] [,pin2, [,logic] | - | X | X | X | X | X |
| BLIT | BLIT BLIT LOAD[BMP] [#]b, BLIT x1, y1, x2, y2, w, h | 506 | X | X | X | - | X |
| BLIT CLOSE | BLIT CLOSE [#]b | - | X | X | X | - | X |
| BLIT COMPRESSED | BLIT COMPRESSED | - | X | X | X | - | X |
| BLIT FLASH | BLIT FLASH from, to, xin, [,colour] | - | X | X | X | - | X |
| BLIT FRAMEBUFFER | BLIT FRAMEBUFFER from, to, xin, yin, xout, yout, width, height [,colour] | - | X | X | X | - | X |
| BLIT MEMORY | BLIT MEMORY address, x, y [,col] | - | X | X | X | - | X |
| BLIT MERGE | BLIT MERGE colour, x, y, w, h | - | X | X | X | - | X |
| BLIT READ | BLIT READ [#]b, x, y, w, h | - | X | X | X | - | X |
| BLIT WRITE | BLIT WRITE [#]b, x, y [,mode] | - | X | X | X | - | X |
| BOX | BOX x, y, w, h [, lw] [,c] [,fill] | 141 | X | X | X | - | X |
| BYTE() | BYTE(var$, byteno)=value | - | X | X | X | X | X |

# A.4. C

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| CALL | CALL usersubname$ [,usersubparameters,..] | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| CAMERA | CAMERA | - | X | X | X | X | X |
| CAMERA CAPTURE | CAMERA CAPTURE [scale, [x , y]] | - | X | X | X | X | X |
| CAMERA CHANGE | CAMERA CHANGE ,y]] | - | X | X | X | X | X |
| CAMERA CLOSE | CAMERA CLOSE | - | X | X | X | X | X |
| CAMERA OPEN | CAMERA OPEN XLKpin, | - | X | X | X | X | X |
| CAMERA REGISTER | CAMERA REGISTER reg%, | - | X | X | X | X | X |
| CAMERA TEST | CAMERA TEST tnum | - | X | X | X | X | X |
| CHAIN | CHAIN fname$ [cmdline$] | 405 | X | X | X | X | X |
| CHDIR | CHDIR dir$ | 395 | X | X | X | X | X |
| CIRCLE | CIRCLE x, y, r [,lw] [, a] [, | 140, 141 | X | X | X | - | X |
| CLEAR | CLEAR | - | X | X | X | X | X |
| CLOSE | CLOSE []fnbr [,[]fnbr] … | 114 | X | X | X | X | X |
| CLS | CLS [colour] | 132 | X | X | X | X | X |
| CMM2 LOAD | CMM2 LOAD or | - | X | X | X | X | X |
| CMM2 RUN | CMM2 RUN | - | X | X | X | X | X |
| COLOR | COLOR fore [, back] | - | X | X | X | - | X |
| COLOUR | COLOUR fore [, back] or | 131 | X | X | X | - | X |
| COLOUR MAP | COLOUR MAP inarray%(), | - | X | X | X | - | X |
| CONFIGURE | CONFIGURE cfg | - | X | X | X | X | X |
| CONTINUE | CONTINUE | - | X | X | X | X | X |
| CONTINUE DO | CONTINUE DO or | - | X | X | X | X | X |
| CONTINUE FOR | CONTINUE FOR | - | X | X | X | X | X |
| COPY | COPY fname1$ TO fname2$ COPY fname$ TO dirname$ | 396 | X | X | X | X | X |
| CPU RESTART | CPU RESTART | - | X | X | X | X | X |
| CPU SLEEP | CPU SLEEP n | - | X | X | X | X | X |
| CSUB | CSUB name [type [, type] hex [[ hex[…] hex [[ hex[…] | - | X | X | X | X | X |

# A.5. D

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| DATA | DATA constant[,constant].. | 335 | X | X | X | X | X |
| DATE$ | DATE$ = "DD-MM-YY[YY]" or DATE$ = "DD/MM/YY[YY]" or DATE$ =□YYYY-MM-DD□ or DATE$=□YYYY/MM/DD□ | 87 | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| DEFINEFONT | DEFINEFONT #Nbr hex [[ hex[…] hex [[ hex[…] | - | X | X | X | X | X |
| DEVICE BITSTREAM | DEVICE BITSTREAM | - | X | X | X | X | X |
| DEVICE CAMERA | DEVICE CAMERA | - | X | X | X | X | X |
| DEVICE KEYPAD | DEVICE KEYPAD | - | X | X | X | X | X |
| DEVICE LCD | DEVICE LCD | - | X | X | X | X | X |
| DEVICE MOUSE | DEVICE MOUSE | - | X | X | X | X | X |
| DEVICE SERIALRX | DEVICE SERIALRX pinno, [,terminators$] | - | X | X | X | X | X |
| DEVICE SERIALTX | DEVICE SERIALTX pinno, | - | X | X | X | X | X |
| DEVICE WII | DEVICE WII | - | X | X | X | X | X |
| DEVICE WS2812 | DEVICE WS2812 | - | X | X | X | X | X |
| DIM | DIM [type] decl [,decl].. DIM nbr(50) DIM a(5,5,5), b(1000) DIM strn$(200) LENGTH 20 DIM a = 1234, b = 345 … | 72 | X | X | X | X | X |
| DIM INTEGER | DIM INTEGER nbr(50) | - | X | X | X | X | X |
| DIM STRING | DIM STRING strn(200) DIM STRING strn = "text" | - | X | X | X | X | X |
| DO | DO | 58 | X | X | X | X | X |
| DO WHILE | DO WHILE expression | - | X | X | X | X | X |
| DRAW3D | DRAW3D | - | X | X | X | - | X |
| DRIVE | DRIVE drive$ | 380 | X | X | X | X | X |

# A.6. E

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| EDIT | EDIT or EDIT fname$ or | 355 | X | X | X | X | X |
| EDIT FILE | EDIT FILE fname$ | - | X | X | X | X | X |
| ELSE | ELSE | 35 | X | X | X | X | X |
| ELSE IF | ELSE IF expression THEN | - | X | X | X | X | X |
| ELSEIF | ELSEIF expression THEN or | 36 | X | X | X | X | X |
| END | END cmd$ | - | X | X | X | X | X |
| END CSUB | END CSUB | - | X | X | X | X | X |
| END DEFINEFONT | END DEFINEFONT | - | X | X | X | X | X |
| END FUNCTION | END FUNCTION | 66 | X | X | X | X | X |
| END IF | END IF | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| END SELECT | END SELECT | - | X | X | X | X | X |
| END SUB | END SUB | 65 | X | X | X | X | X |
| END TYPE | END TYPE | - | X | X | X | X | X |
| ENDIF | ENDIF or ENDIF | - | X | X | X | X | X |
| ERASE | ERASE variable [,variable].. | - | X | X | X | X | X |
| ERROR | ERROR [error_msg$] | - | X | X | X | X | X |
| EXECUTE | EXECUTE command$ | - | X | X | X | X | X |
| EXIT FOR | EXIT FOR | - | X | X | X | X | X |
| EXIT FUNCTION | EXIT FUNCTION | 346 | X | X | X | X | X |
| EXIT SUB | EXIT SUB | - | X | X | X | X | X |

# A.7. F

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| FILES | FILES [fspec$] [,sort] | 51 | X | X | X | X | X |
| FILL | FILL x, y, fillcolour [,bordercolour] | - | X | X | X | X | X |
| FLAG() | FLAG(n%)=value | - | X | X | X | X | X |
| FLASH | FLASH | - | X | X | X | X | X |
| FLASH CHAIN | FLASH CHAIN n | - | X | X | X | X | X |
| FLASH DISK LOAD | FLASH DISK LOAD n, | - | X | X | X | X | X |
| FLASH ERASE | FLASH ERASE n | - | X | X | X | X | X |
| FLASH ERASE ALL | FLASH ERASE ALL | - | X | X | X | X | X |
| FLASH LIST | FLASH LIST FLASH LIST n [,all] | - | X | X | X | X | X |
| FLASH LOAD IMAGE | FLASH LOAD IMAGE n, | - | X | X | X | X | X |
| FLASH OVERWRITE | FLASH OVERWRITE n | - | X | X | X | X | X |
| FLASH SAVE | FLASH SAVE n | 382 | X | X | X | X | X |
| FLUSH | FLUSH [#]fnbr | - | X | X | X | X | X |
| FONT | FONT [#]font-number, scaling | 133 | X | X | X | - | X |
| FOR | FOR counter = start TO finish [STEP increment] | - | X | X | X | X | X |
| FRAME | FRAME | - | X | X | X | X | X |
| FRAMEBUFFER | FRAMEBUFFER | - | X | X | X | - | X |
| FRAMEBUFFER CLOSE | FRAMEBUFFER CLOSE [which] | 226 | X | X | X | - | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| FRAMEBUFFER COPY | FRAMEBUFFER COPY | 223 | X | X | X | - | X |
| FRAMEBUFFER CREATE | FRAMEBUFFER CREATE FRAMEBUFFER CREATE 2 | 222 | X | X | X | - | X |
| FRAMEBUFFER LAYER | FRAMEBUFFER LAYER FRAMEBUFFER LAYER [colour] | 231 | X | X | X | - | X |
| FRAMEBUFFER MERGE | FRAMEBUFFER MERGE [colour] [,mode] [,updaterate] | - | X | X | X | - | X |
| FRAMEBUFFER SYNC | FRAMEBUFFER SYNC | - | X | X | X | - | X |
| FRAMEBUFFER WAIT | FRAMEBUFFER WAIT | - | X | X | X | - | X |
| FRAMEBUFFER WRITE | FRAMEBUFFER WRITE | 222 | X | X | X | - | X |
| FUNCTION | FUNCTION xxx (arg1 [,arg2, ⋯]) [AS <type>} | 66 | X | X | X | X | X |

# A.8. G

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| GAMEPAD COLOUR | GAMEPAD COLOUR | - | X | X | X | - | X |
| GAMEPAD CONFIGURE | GAMEPAD CONFIGURE | 493 | X | X | X | - | X |
| GAMEPAD HAPTIC | GAMEPAD HAPTIC channel | - | X | X | X | - | X |
| GAMEPAD INTERRUPT | GAMEPAD INTERRUPT | - | X | X | X | - | X |
| GAMEPAD MONITOR | GAMEPAD MONITOR | 493 | X | X | X | - | X |
| GOTO | GOTO target | - | X | X | X | X | X |
| GUI BITMAP | GUI BITMAP x, y, bits [, | - | X | X | X | - | X |
| GUI CALIBRATE | GUI CALIBRATE or GUI CALIBRATE a,b,c,d,d | - | X | X | X | - | X |
| GUI RESET LCDPANEL | GUI RESET LCDPANEL | - | X | X | X | - | X |
| GUI TEST LCDPANEL | GUI TEST LCDPANEL | - | X | X | X | - | X |
| GUI TEST TOUCH | GUI TEST TOUCH | - | X | X | X | - | X |

# A.9. H

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| HEARTBEAT | HEARTBEAT | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| HELP | HELP searchtext | 361 | X | X | X | X | X |
| HUMID | HUMID pin, tvar, hvar [,DHT11] | - | X | X | X | X | X |

# A.10. I

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| I2C | I2C | - | X | X | X | X | X |
| I2C CHECK | I2C CHECK addr | - | X | X | X | X | X |
| I2C CLOSE | I2C CLOSE | - | X | X | X | X | X |
| I2C OPEN | I2C OPEN speed, timeout | - | X | X | X | X | X |
| I2C SLAVE | I2C SLAVE | - | X | X | X | X | X |
| I2C WRITE | I2C WRITE addr, option, | - | X | X | X | X | X |
| I2C2 | I2C2 | - | X | X | X | X | X |
| I2CLCD | I2CLCD I2CLCD line, position, string$ I2CLCD line, Cn, string$ | - | X | X | X | X | X |
| I2CLCD BACKLIGHT | I2CLCD BACKLIGHT state | - | X | X | X | X | X |
| I2CLCD CLEAR | I2CLCD CLEAR | - | X | X | X | X | X |
| I2CLCD CLOSE | I2CLCD CLOSE | - | X | X | X | X | X |
| I2CLCD CMD | I2CLCD CMD byte [, byte ⋯] | - | X | X | X | X | X |
| I2CLCD CREATECHAR | I2CLCD CREATECHAR | - | X | X | X | X | X |
| I2CLCD CURSOR | I2CLCD CURSOR state [, BLINK] | - | X | X | X | X | X |
| I2CLCD DATA | I2CLCD DATA byte [, byte | - | X | X | X | X | X |
| I2CLCD INIT | I2CLCD INIT address | - | X | X | X | X | X |
| IF | IF expr THEN stmt [: stmt] or IF expr THEN stmt ELSE stmt IF expression THEN [ELSEIF expression THEN [ELSE | 34 | X | X | X | X | X |
| IMAGE | IMAGE file$ [,x, y, w, h] | - | X | X | X | - | X |
| INC | INC var [,increment] | - | X | X | X | X | X |
| INPUT | INPUT ["prompt$";] var1 [,var2 [, var3 [, etc]]] INPUT #nbr, | 114, 28 | X | X | X | X | X |
| IR | IR dev, key , int or | - | X | X | X | X | X |
| IR CLOSE | IR CLOSE | - | X | X | X | X | X |
| IR SEND | IR SEND pin, dev, key | - | X | X | X | X | X |

# A.11. K

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| KEYPAD | KEYPAD var, int, r1, r2, r3, or KEYPAD keymapmap!(), | - | X | X | X | X | X |
| KEYPAD CLOSE | KEYPAD CLOSE | - | X | X | X | X | X |
| KILL | KILL file$ [,all] | 397 | X | X | X | X | X |

# A.12. L

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| LCD | LCD line, pos, text$ or | - | X | X | X | - | X |
| LCD CLEAR | LCD CLEAR or | - | X | X | X | - | X |
| LCD CLOSE | LCD CLOSE | - | X | X | X | - | X |
| LCD DATA | LCD DATA d1 [, d2 [, etc]] | - | X | X | X | - | X |
| LCD INIT | LCD INIT d4, d5, d6, d7, rs, or | - | X | X | X | - | X |
| LET | LET variable = expression | - | X | X | X | X | X |
| LIBRARY DELETE | LIBRARY DELETE or | 386 | X | X | X | X | X |
| LIBRARY DISK LOAD | LIBRARY DISK LOAD | 386 | X | X | X | X | X |
| LIBRARY DISK SAVE | LIBRARY DISK SAVE or | 386 | X | X | X | X | X |
| LIBRARY LIST | LIBRARY LIST or | 385 | X | X | X | X | X |
| LIBRARY LIST ALL | LIBRARY LIST ALL or | - | X | X | X | X | X |
| LIBRARY SAVE | LIBRARY SAVE or | 384 | X | X | X | X | X |
| LINE | LINE x1, y1, x2, y2 [,[-] LW [, C]] | 138 | X | X | X | - | X |
| LINE GRAPH | LINE GRAPH x(), y(), colour | - | X | X | X | - | X |
| LINE INPUT | LINE INPUT [prompt$,] LINE INPUT #nbr, | - | X | X | X | - | X |
| LINE PLOT | LINE PLOT ydata() [,nbr] [,xstart] [,xinc] [,ystart] [,yinc] [,colour] | - | X | X | X | - | X |
| LIST | LIST [fname$] or | 394 | X | X | X | X | X |
| LIST ALL | LIST ALL [fname$] | - | X | X | X | X | X |
| LIST COMMANDS | LIST COMMANDS or | - | X | X | X | X | X |
| LIST FUNCTIONS | LIST FUNCTIONS | - | X | X | X | X | X |
| LIST PINS | LIST PINS | - | X | X | X | X | X |
| LIST SYSTEM I2C | LIST SYSTEM I2C | - | X | X | X | X | X |
| LIST VARIABLES | LIST VARIABLES [s%()] | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| LMID() | LMID(array%(),start [,num])=string$ | - | X | X | X | X | X |
| LOAD | LOAD IMAGE/BMP fname$ [,x] [,y] [,mode] [,ximage] [,yimage] | 52 | X | X | X | X | X |
| LOAD CONTEXT | LOAD CONTEXT [KEEP] | - | X | X | X | X | X |
| LOAD DATA | LOAD DATA fname$, address | - | X | X | X | X | X |
| LOAD JPG | LOAD JPG file$ [, x] [, y] [,mode] [,ximage] [,yimage] | - | X | X | X | X | X |
| LOAD PNG | LOAD PNG fname$ [, x] [, y] [,transparent] [,alphacut] | - | X | X | X | X | X |
| LOCAL | LOCAL variable [, variables] | - | X | X | X | X | X |
| LOCATION | LOCATION | - | X | X | X | X | X |
| LONGSTRING | LONGSTRING | - | X | X | X | X | X |
| LONGSTRING AES128 | LONGSTRING AES128 | - | X | X | X | X | X |
| LONGSTRING APPEND | LONGSTRING APPEND array%(), string$ | - | X | X | X | X | X |
| LONGSTRING BASE64 | LONGSTRING BASE64 | - | X | X | X | X | X |
| LONGSTRING CLEAR | LONGSTRING CLEAR array%() | - | X | X | X | X | X |
| LONGSTRING CONCAT | LONGSTRING CONCAT | - | X | X | X | X | X |
| LONGSTRING COPY | LONGSTRING COPY | - | X | X | X | X | X |
| LONGSTRING LEFT | LONGSTRING LEFT | - | X | X | X | X | X |
| LONGSTRING LOAD | LONGSTRING LOAD array%(), nbr, string$ | - | X | X | X | X | X |
| LONGSTRING MID | LONGSTRING MID dest%(), | - | X | X | X | X | X |
| LONGSTRING PRINT | LONGSTRING PRINT [#n,] | - | X | X | X | X | X |
| LONGSTRING REPLACE | LONGSTRING REPLACE array%() , string$, start | - | X | X | X | X | X |
| LONGSTRING RESIZE | LONGSTRING RESIZE | - | X | X | X | X | X |
| LONGSTRING RIGHT | LONGSTRING RIGHT | - | X | X | X | X | X |
| LONGSTRING SETBYTE | LONGSTRING SETBYTE | - | X | X | X | X | X |
| LONGSTRING TRIM | LONGSTRING TRIM array%(), nbr | - | X | X | X | X | X |
| LONGSTRING UCASE | LONGSTRING UCASE array%() | - | X | X | X | X | X |
| LOOP | LOOP LOOP [UNTIL expression] | 58 | X | X | X | X | X |
| LOOP UNTIL | LOOP UNTIL expression | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| LOOP WHILE | LOOP WHILE expression | - | X | X | X | X | X |

# A.13. M

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| MAP | MAP | - | X | X | X | - | X |
| MAP GREYSCALE | MAP GREYSCALE | - | X | X | X | - | X |
| MAP MAXIMITE | MAP MAXIMITE | - | X | X | X | - | X |
| MAP RESET | MAP RESET | - | X | X | X | - | X |
| MAP SET | MAP SET | - | X | X | X | - | X |
| MAP()= | MAP( n ) = rgb% | - | X | X | X | - | X |
| MATH ADD | MATH ADD in(), num ,out() | - | X | X | X | X | X |
| MATH AES128 | MATH AES128 | - | X | X | X | X | X |
| MATH C_ADD | MATH C_ADD array1(), array2(), | - | X | X | X | X | X |
| MATH C_AND | MATH C_AND array1(), array2(), | - | X | X | X | X | X |
| MATH C_DIV | MATH C_DIV array1(), array2(), | - | X | X | X | X | X |
| MATH C_MUL | MATH C_MUL array1(), array2(), | - | X | X | X | X | X |
| MATH C_OR | MATH C_OR array1(), array2(), | - | X | X | X | X | X |
| MATH C_SUB | MATH C_SUB array1(), array2(), | - | X | X | X | X | X |
| MATH C_XOR | MATH C_XOR array1(), array2(), | - | X | X | X | X | X |
| MATH FFT | MATH FFT signalarray!(), | - | X | X | X | X | X |
| MATH FFT INVERSE | MATH FFT INVERSE | - | X | X | X | X | X |
| MATH FFT MAGNITUDE | MATH FFT MAGNITUDE () | - | X | X | X | X | X |
| MATH FFT PHASE | MATH FFT PHASE | - | X | X | X | X | X |
| MATH INSERT | MATH INSERT targetarray(), [d1] [,d2] [,d3] [,d4] [,d5] , | - | X | X | X | X | X |
| MATH INTERPOLATE | MATH INTERPOLATE in1(), | - | X | X | X | X | X |
| MATH M_INVERSE | MATH M_INVERSE array!(), | - | X | X | X | X | X |
| MATH M_MULT | MATH M_MULT in1(), in2(), | - | X | X | X | X | X |
| MATH M_PRINT | MATH M_PRINT array() | - | X | X | X | X | X |
| MATH M_TRANSPOSE | MATH M_TRANSPOSE in(), | - | X | X | X | X | X |
| MATH PID INIT | MATH PID INIT channel, | - | X | X | X | X | X |
| MATH PID START | MATH PID START channel | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| MATH PID STOP | MATH PID STOP channel | - | X | X | X | X | X |
| MATH POWER | MATH POWER inarray(), | - | X | X | X | X | X |
| MATH Q_CREATE | MATH Q_CREATE theta, x, | - | X | X | X | X | X |
| MATH Q_EULER | MATH Q_EULER yaw, pitch, | - | X | X | X | X | X |
| MATH Q_INVERT | MATH Q_INVERT inQ(), | - | X | X | X | X | X |
| MATH Q_ROTATE | MATH Q_ROTATE , RQ(), | - | X | X | X | X | X |
| MATH Q_VECTOR | MATH Q_VECTOR x, y, z, | - | X | X | X | X | X |
| MATH RANDOMIZE | MATH RANDOMIZE [n] | - | X | X | X | X | X |
| MATH SCALE | MATH SCALE in(), scale ,out() | - | X | X | X | X | X |
| MATH SENSORFUSION | MATH SENSORFUSION [,p2] | - | X | X | X | X | X |
| MATH SET | MATH SET nbr, array() | - | X | X | X | X | X |
| MATH SHIFT | MATH SHIFT inarray%(), | - | X | X | X | X | X |
| MATH SINC | MATH SINC x_in(), y_in(), n , m, window, freq, x_out(), MATH SINC x_in(), y_in(), n, | - | X | X | X | X | X |
| MATH SLICE | MATH SLICE sourcearray(), [d1] [,d2] [,d3] [,d4] [,d5] , destinationarray() | - | X | X | X | X | X |
| MATH V_CROSS | MATH V_CROSS inV1(), | - | X | X | X | X | X |
| MATH V_MULT | MATH V_MULT matrix(), | - | X | X | X | X | X |
| MATH V_NORMALISE | MATH V_NORMALISE | - | X | X | X | X | X |
| MATH V_PRINT | MATH V_PRINT array() [,hex] | - | X | X | X | X | X |
| MATH V_ROTATE | MATH V_ROTATE x, y, a, | - | X | X | X | X | X |
| MATH WINDOW | MATH WINDOW in(), | - | X | X | X | X | X |
| MEMORY | MEMORY | - | X | X | X | X | X |
| MEMORY COPY | MEMORY COPY [,sourceincrement][,destination | - | X | X | X | X | X |
| MEMORY COPY FLOAT | MEMORY COPY FLOAT [,sourceincrement][,destination | - | X | X | X | X | X |
| MEMORY COPY INTEGER | MEMORY COPY INTEGER [,sourceincrement][,destination | - | X | X | X | X | X |
| MEMORY INPUT | MEMORY INPUT [#]fnbr , | - | X | X | X | X | X |
| MEMORY PACK | MEMORY PACK | - | X | X | X | X | X |
| MEMORY PRINT | MEMORY PRINT #]fnbr , | - | X | X | X | X | X |
| MEMORY SET BYTE | MEMORY SET BYTE | - | X | X | X | X | X |
| MEMORY SET FLOAT | MEMORY SET FLOAT ,numberofloats [,increment] | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| MEMORY SET INTEGER | MEMORY SET INTEGER ,numberofintegers [,increment] | - | X | X | X | X | X |
| MEMORY SET SHORT | MEMORY SET SHORT | - | X | X | X | X | X |
| MEMORY SET WORD | MEMORY SET WORD | - | X | X | X | X | X |
| MID$()= | MID$( str$, start, num) = str2$ | - | X | X | X | X | X |
| MKDIR | MKDIR dir$ | 394 | X | X | X | X | X |
| MODE | MODE 1 or MODE 2 MODE 3 (RP2350 only) MODE n MODE 400 or ⋯ | 123 | X | X | X | - | X |
| MOUSE | MOUSE | - | X | X | X | - | X |
| MOUSE CLOSE | MOUSE CLOSE channel | - | X | X | X | - | X |
| MOUSE INTERRUPT | MOUSE INTERRUPT | - | X | X | X | - | X |
| MOUSE OPEN | MOUSE OPEN channel, | - | X | X | X | - | X |
| MOUSE SET | MOUSE SET channel, x- | - | X | X | X | - | X |

# A.14. N

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| NEW | NEW | - | X | X | X | X | X |

# A.15. O

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| ON ERROR ABORT | ON ERROR ABORT or | - | X | X | X | X | X |
| ON ERROR CLEAR | ON ERROR CLEAR | - | X | X | X | X | X |
| ON ERROR IGNORE | ON ERROR IGNORE or | - | X | X | X | X | X |
| ON ERROR RESTART | ON ERROR RESTART | - | X | X | X | X | X |
| ON ERROR SKIP | ON ERROR SKIP [nn] or | - | X | X | X | X | X |
| ON KEY | ON KEY target or ON KEY ASCIIcode, target | - | X | X | X | X | X |
| ON PS2 | ON PS2 target | - | X | X | X | - | X |
| ONESHOT | ONESHOT triggerPin, sense, | - | X | X | X | X | X |
| ONESHOT DISABLE | ONESHOT DISABLE | - | X | X | X | X | X |
| ONESHOT R | ONESHOT R triggerPin, | - | X | X | X | X | X |
| ONEWIRE RESET | ONEWIRE RESET pin or | - | X | X | X | X | X |
| ONEWIRE WRITE | ONEWIRE WRITE pin, flag, or | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| OPEN | OPEN fname$ FOR mode AS []fnbr OPEN comspec$ AS []fnbr OPEN comspec$ AS GPS [,timezone_offset] [,monitor] | 113 | X | X | X | X | X |
| OPTION | OPTION | 427 | X | X | X | X | X |

# A.16. P

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| PAUSE | PAUSE delay | 142 | X | X | X | X | X |
| PIN()= | PIN( pin ) = value | 265 | X | X | X | X | X |
| PIO | PIO PIO assemble pio,linedata$ | - | X | X | X | X | X |
| PIO CLEAR | PIO CLEAR pio | - | X | X | X | X | X |
| PIO CONFIGURE | PIO CONFIGURE pio, sm, [,sidesetbase] [,sidesetno] [,sidesetout] [,setbase] [,setno] [,setout] [,outbase] [,outno] [,outout] [,inbase] [,jmppin] [,wraptarget] [,wrap] | - | X | X | X | X | X |
| PIO DMA RX | PIO DMA RX pio, sm, nbr, [,transfersize] [,loopbackcount] | - | X | X | X | X | X |
| PIO DMA RX OFF | PIO DMA RX OFF | - | X | X | X | X | X |
| PIO DMA TX | PIO DMA TX pio, sm, nbr, [,transfersize] [,loopbackcount] | - | X | X | X | X | X |
| PIO DMA TX OFF | PIO DMA TX OFF | - | X | X | X | X | X |
| PIO INIT MACHINE | PIO INIT MACHINE pio%, [,pinctrl] [,execctrl] [,shiftctrl] [,startinstruction] [,sideout [,setout] [,outout] | - | X | X | X | X | X |
| PIO INTERRUPT | PIO INTERRUPT pio, sm [,RXinterrupt] [,TXinterrupt] | - | X | X | X | X | X |
| PIO PROGRAM | PIO PROGRAM pio PIO PROGRAM pio,array%() | - | X | X | X | X | X |
| PIO PROGRAM LINE | PIO PROGRAM LINE pio, | - | X | X | X | X | X |
| PIO READ | PIO READ pio, | - | X | X | X | X | X |
| PIO SET BASE | PIO SET BASE 0/16 | - | X | X | X | X | X |
| PIO START | PIO START pio, statemachine | - | X | X | X | X | X |
| PIO STOP | PIO STOP pio, statemachine | - | X | X | X | X | X |
| PIO SYNC | PIO SYNC pio, statemachines, [,prevstatemachines] [,nextstatemachines] | - | X | X | X | X | X |
| PIO WRITE | PIO WRITE pio, [,data1..] | - | X | X | X | X | X |
| PIO WRITEFIFO | PIO WRITEFIFO a,b,c,d | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| PIXEL | PIXEL x, y [,c] | 137 | X | X | X | - | X |
| PLAY ARRAY | PLAY ARRAY l%(), r%(), [,terminationinterrupt] | - | X | X | X | X | X |
| PLAY CONTINUE | PLAY CONTINUE track$ | - | X | X | X | X | X |
| PLAY FLAC | PLAY FLAC file$ [, interrupt] | - | X | X | X | X | X |
| PLAY HALT | PLAY HALT | - | X | X | X | X | X |
| PLAY LOAD SOUND | PLAY LOAD SOUND array%() | - | X | X | X | X | X |
| PLAY MIDI | PLAY MIDI | - | X | X | X | X | X |
| PLAY MIDI CMD | PLAY MIDI CMD cmd%, | - | X | X | X | X | X |
| PLAY MIDIFILE | PLAY MIDIFILE file$ [, | - | X | X | X | X | X |
| PLAY MODFILE | PLAY MODFILE file$ [,interrupt] | - | X | X | X | X | X |
| PLAY MODSAMPLE | PLAY MODSAMPLE | - | X | X | X | X | X |
| PLAY MP3 | PLAY MP3 file$ [, interrupt] | - | X | X | X | X | X |
| PLAY NEXT | PLAY NEXT | - | X | X | X | X | X |
| PLAY NOTE OFF | PLAY NOTE OFF channel%, | - | X | X | X | X | X |
| PLAY NOTE ON | PLAY NOTE ON channel%, | - | X | X | X | X | X |
| PLAY PAUSE | PLAY PAUSE | - | X | X | X | X | X |
| PLAY PREVIOUS | PLAY PREVIOUS | - | X | X | X | X | X |
| PLAY RELEASE | PLAY RELEASE | - | X | X | X | X | X |
| PLAY RESUME | PLAY RESUME | - | X | X | X | X | X |
| PLAY SAMPLE | PLAY SAMPLE left%(), or | - | X | X | X | X | X |
| PLAY SOUND | PLAY SOUND soundno, [,volume] | 541 | X | X | X | X | X |
| PLAY STOP | PLAY STOP | - | X | X | X | X | X |
| PLAY STREAM | PLAY STREAM buffer%(), | - | X | X | X | X | X |
| PLAY TONE | PLAY TONE left, right [,dur] [,interrupt] | 330 | X | X | X | X | X |
| PLAY WAV | PLAY WAV file$ [, interrupt] | - | X | X | X | X | X |
| POKE BYTE | POKE BYTE addr%, byte or | - | X | X | X | X | X |
| POKE DISPLAY | POKE DISPLAY command [,data1] [,data2] [,datan] | - | X | X | X | X | X |
| POKE DISPLAY HRES | POKE DISPLAY HRES n | - | X | X | X | X | X |
| POKE DISPLAY VRES | POKE DISPLAY VRES n | - | X | X | X | X | X |
| POKE FLOAT | POKE FLOAT addr%, float! or | - | X | X | X | X | X |
| POKE INTEGER | POKE INTEGER addr%, int% or | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| POKE SHORT | POKE SHORT addr%, short% or | - | X | X | X | X | X |
| POKE VAR | POKE VAR var, offset, byte or | - | X | X | X | X | X |
| POKE VARTBL | POKE VARTBL, offset, byte | - | X | X | X | X | X |
| POKE WORD | POKE WORD addr%, word% or | - | X | X | X | X | X |
| POLYGON | POLYGON n, xarray%(), POLYGON n(), xarray%(), | - | X | X | X | - | X |
| PORT()= | PORT(start, nbr [,start, nbr]⋯) | - | X | X | X | X | X |
| PRINT | PRINT expression [[,; ]expression] ⋯ etc PRINT #nbr, expression [[,; ]expression] ⋯ etc PRINT #GPS, expression [[,; ]expression] ⋯ etc PRINT @(x [, y]) expression or PRINT @(x, [y], m) | 15, 17 | X | X | X | X | X |
| PULSE | PULSE pin, width | - | X | X | X | X | X |
| PWM | PWM channel, frequency, [dutyA] [,dutyB][,phase][,defer] PWM channel, OFF | 319 | X | X | X | X | X |
| PWM SYNC | PWM SYNC s0 [,s1][,s2][,s3][,s4][,s5][,s6][,s7 | - | X | X | X | X | X |

# A.17. R

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| RAM | RAM | - | X | X | X | X | X |
| RAM CHAIN | RAM CHAIN n | - | X | X | X | X | X |
| RAM ERASE | RAM ERASE n | - | X | X | X | X | X |
| RAM ERASE ALL | RAM ERASE ALL | - | X | X | X | X | X |
| RAM FILE LOAD | RAM FILE LOAD n, | - | X | X | X | X | X |
| RAM LIST | RAM LIST RAM LIST n [,all] | - | X | X | X | X | X |
| RAM LOAD | RAM LOAD n | - | X | X | X | X | X |
| RAM OVERWRITE | RAM OVERWRITE n | - | X | X | X | X | X |
| RAM RUN | RAM RUN n | - | X | X | X | X | X |
| RAM SAVE | RAM SAVE n | - | X | X | X | X | X |
| RANDOMIZE | RANDOMIZE nbr | - | X | X | X | X | X |
| RAY | RAY | - | X | X | X | - | X |
| RBOX | RBOX x, y, w, h [, r] [,c] [,fill] | 150 | X | X | X | - | X |
| READ | READ variable[, variable].. | - | X | X | X | X | X |
| READ RESTORE | READ RESTORE | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| READ SAVE | READ SAVE or | - | X | X | X | X | X |
| REDIM | REDIM [PRESERVE] array1(dimensions) [, array2(dimensions]…[,arrayn( | - | X | X | X | X | X |
| REFRESH | REFRESH | - | X | X | X | X | X |
| REM | REM string | - | X | X | X | X | X |
| RESTORE | RESTORE [line] | - | X | X | X | X | X |
| RMDIR | RMDIR dir$ | 398 | X | X | X | X | X |
| RTC GETREG | RTC GETREG reg, var | - | X | X | X | X | X |
| RTC GETTIME | RTC GETTIME | - | X | X | X | X | X |
| RTC SETREG | RTC SETREG reg, value | - | X | X | X | X | X |
| RTC SETTIME | RTC SETTIME year, month, | - | X | X | X | X | X |
| RUN | RUN or RUN [file$] [, cmdline$] | - | X | X | X | X | X |

# A.18. S

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| SAVE | SAVE file$ | 51 | X | X | X | X | X |
| SAVE COMPRESSED | SAVE COMPRESSED | - | X | X | X | X | X |
| SAVE CONTEXT | SAVE CONTEXT [CLEAR] | - | X | X | X | X | X |
| SAVE IMAGE | SAVE IMAGE file$ [,x, y, w, h] or | - | X | X | X | X | X |
| SAVE PERSISTENT | SAVE PERSISTENT n% | - | X | X | X | X | X |
| SEEK | SEEK [#]fnbr, pos | - | X | X | X | X | X |
| SELECT CASE | SELECT CASE value | 297 | X | X | X | X | X |
| SERVO | SERVO channel [positionA] [,positionB] | - | X | X | X | X | X |
| SETPIN | SETPIN pin, cfg [, option] SETPIN pin, cfg, target [, SETPIN p1[, p2 [, p3]], device SETPIN rx, tx, COM1 SETPIN rx, tx, COM2 … | 265 | X | X | X | X | X |
| SETTICK | SETTICK period, target [, nbr] | - | X | X | X | X | X |
| SETTICK PAUSE | SETTICK PAUSE, target [, nbr] or | - | X | X | X | X | X |
| SETTICK RESUME | SETTICK RESUME, target [, nbr] | - | X | X | X | X | X |
| SPI CLOSE | SPI CLOSE | - | X | X | X | X | X |
| SPI OPEN | SPI OPEN speed, mode, bits or | - | X | X | X | X | X |
| SPI READ | SPI READ nbr, array() or | - | X | X | X | X | X |
| SPI WRITE | SPI WRITE nbr, data1, data2, SPI WRITE nbr, array() or | - | X | X | X | X | X |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| SPI2 | SPI2 | - | X | X | X | X | X |
| SPRITE | SPRITE | 506 | X | X | X | - | X |
| STAR | STAR | - | X | X | X | X | X |
| STATIC | STATIC variable [, variables] | - | X | X | X | X | X |
| STEPPER | STEPPER | - | X | X | X | X | X |
| STRUCT | STRUCT | - | X | X | X | X | X |
| SUB | SUB xxx (arg1 [,arg2, ⋯]) | 65 | X | X | X | X | X |
| SYNC | SYNC time% [,period] or SYNC | - | X | X | X | X | X |

# A.19. T

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| TEMPR START | TEMPR START pin [, | - | X | X | X | X | X |
| TILE | TILE x, y [,foreground] [,background] [,nbr_tiles_wide] [,nbr_tiles_high] | - | X | X | X | - | X |
| TILE HEIGHT | TILE HEIGHT n | - | X | X | X | - | X |
| TILEMAP | TILEMAP | - | X | X | X | - | X |
| TIME$ | TIME$ = "HH:MM:SS" or TIME$ = "HH:MM" or TIME$ = "HH" | 87 | X | X | X | X | X |
| TIMER | TIMER = msec | 290 | X | X | X | X | X |
| TRACE LIST | TRACE LIST nn | - | X | X | X | X | X |
| TRACE OFF | TRACE OFF or | - | X | X | X | X | X |
| TRIANGLE RESTORE | TRIANGLE RESTORE [#]n | - | X | X | X | - | X |
| TRIANGLE SAVE | TRIANGLE SAVE [#]n, | - | X | X | X | - | X |
| TRIANGLE X1 Y1 X2 Y2 | TRIANGLE X1, Y1, X2, Y2, | - | X | X | X | - | X |
| TURTLE | TURTLE | - | X | X | X | - | X |
| TYPE | TYPE struct-name | - | X | X | X | X | X |

# A.20. U

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| UPDATE FIRMWARE | UPDATE FIRMWARE | - | X | X | X | X | X |

# A.21. V

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| VAR CLEAR | VAR CLEAR | - | X | X | X | X | X |
| VAR RESTORE | VAR RESTORE or | - | X | X | X | X | X |
| VAR SAVE | VAR SAVE var [, var]⋯ or | - | X | X | X | X | X |

# A.22. W

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| WATCHDOG | WATCHDOG timeout or | - | X | X | X | X | X |
| WATCHDOG HW | WATCHDOG HW timeout or | - | X | X | X | X | X |
| WATCHDOG HW OFF | WATCHDOG HW OFF | - | X | X | X | X | X |
| WATCHDOG OFF | WATCHDOG OFF or | - | X | X | X | X | X |
| WEB | WEB | - | - | - | - | X | - |
| WEB CLOSE TCP CLIENT | WEB CLOSE TCP CLIENT | - | - | - | - | X | - |
| WEB CONNECT | WEB CONNECT [ssid$, [,ipaddress$, mask$, | - | - | - | - | X | - |
| WEB MQTT CLOSE | WEB MQTT CLOSE | - | - | - | - | X | - |
| WEB MQTT CONNECT | WEB MQTT CONNECT | - | - | - | - | X | - |
| WEB MQTT PUBLISH | WEB MQTT PUBLISH | - | - | - | - | X | - |
| WEB MQTT SUBSCRIBE | WEB MQTT SUBSCRIBE | - | - | - | - | X | - |
| WEB MQTT UNSUBSCRIBE | WEB MQTT UNSUBSCRIBE | - | - | - | - | X | - |
| WEB NTP | WEB NTP [timeoffset [, | - | - | - | - | X | - |
| WEB OPEN TCP CLIENT | WEB OPEN TCP CLIENT | - | - | - | - | X | - |
| WEB OPEN TCP STREAM | WEB OPEN TCP STREAM | - | - | - | - | X | - |
| WEB SCAN | WEB SCAN [array%()] | - | - | - | - | X | - |
| WEB TCP CLIENT STREAM | WEB TCP CLIENT STREAM | - | - | - | - | X | - |
| WEB TCP CLOSE | WEB TCP CLOSE cb% | - | - | - | - | X | - |
| WEB TCP INTERRUPT | WEB TCP INTERRUPT | - | - | - | - | X | - |
| WEB TCP READ | WEB TCP READ cb%, | - | - | - | - | X | - |
| WEB TCP SEND | WEB TCP SEND cb%, | - | - | - | - | X | - |

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| WEB TRANSMIT CODE | WEB TRANSMIT CODE | - | - | - | - | X | - |
| WEB TRANSMIT FILE | WEB TRANSMIT FILE cb%, | - | - | - | - | X | - |
| WEB UDP INTERRUPT | WEB UDP INTERRUPT | - | - | - | - | X | - |
| WEB UDP SEND | WEB UDP SEND addr$, port, | - | - | - | - | X | - |
| WII | WII [CLASSIC] OPEN [,interrupt] WII [CLASSIC] CLOSE | - | X | X | X | - | X |
| WII NUNCHUCK CLOSE | WII NUNCHUCK CLOSE | - | X | X | X | - | X |
| WII NUNCHUCK OPEN | WII NUNCHUCK OPEN [,interrupt] | - | X | X | X | - | X |
| WS2812 | WS2812 type, pin, nbr, | 480 | X | X | X | X | X |

# A.23. X

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| XMODEM CRUNCH | XMODEM CRUNCH | - | X | X | X | X | X |
| XMODEM RECEIVE | XMODEM RECEIVE or XMODEM RECEIVE file$ or | - | X | X | X | X | X |
| XMODEM SEND | XMODEM SEND or XMODEM SEND file$ or | - | X | X | X | X | X |

# A.24. Y

| Befehl | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| YMODEM CRUNCH | YMODEM CRUNCH | - | X | X | X | X | X |
| YMODEM RECEIVE | YMODEM RECEIVE or YMODEM RECEIVE file$ or | - | X | X | X | X | X |
| YMODEM SEND | YMODEM SEND or YMODEM SEND file$ or | - | X | X | X | X | X |

# Anhang B: MMBasic-Funktionsreferenz

Diese Referenz enthält die MMBasic-Funktionen in alphabetischer Reihenfolge.

Zu jeder Funktion werden die Syntax, die Behandlung im Buch und die unterstützten Plattformen angegeben.

Die vollständige Beschreibung der Syntax und aller Parameter findest du im jeweiligen MMBasic-Handbuch.

*Tabelle 2. Legende der Plattformen*

| Abkürzung | Plattform | Abkürzung | Plattform | Abkürzung | Plattform | Abkürzung | Plattform |
|---|---|---|---|---|---|---|---|
| P | PicoMite | V | PicoMite VGA | H | PicoMite HDMI | W | WebMite |
| K | PicoCalc | | | | | | |

## B.1. A

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| ABS() | ABS(number) | 20 | X | X | X | X | X |
| ACOS() | ACOS(number) | - | X | X | X | X | X |
| ASC() | ASC(string$) | - | X | X | X | X | X |
| ASIN() | ASIN(number) | - | X | X | X | X | X |
| ATN() | ATN(number) | - | X | X | X | X | X |
| ATAN2() | ATAN2(y, x) | - | X | X | X | X | X |

## B.2. B

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| BIN$() | BIN$(number[, chars]) | - | X | X | X | X | X |
| BIN2STR$() | BIN2STR$(type, value[, BIG]) | - | X | X | X | X | X |
| BIT() | BIT(var%, bitno) | - | X | X | X | X | X |
| BOUND() | BOUND(array()[, dimension]) | - | X | X | X | X | X |
| BYTE() | BYTE(var$, byteno) | - | X | X | X | X | X |

## B.3. C

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| CALL() | CALL(userfunname$[, userfunparameters,…]) | - | X | X | X | X | X |
| CHOICE() | CHOICE(condition, ExpressionIfTrue, ExpressionIfFalse) | - | X | X | X | X | X |
| CHR$() | CHR$(number) | - | X | X | X | X | X |
| CINT() | CINT(number) | - | X | X | X | X | X |
| COS() | COS(number) | 185 | X | X | X | X | X |
| CWD$ | CWD$ | 395 | X | X | X | X | X |

# B.4. D

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| DATE$ | DATE$ | 87 | X | X | X | X | X |
| DATETIME$() | DATETIME$(n) | - | X | X | X | X | X |
| DAY$() | DAY$(date$) | - | X | X | X | X | X |
| DEG() | DEG(radians) | - | X | X | X | X | X |
| DEVICE(GAMEPAD) | DEVICE(GAMEPAD channel, funct) | - | - | X | X | - | X |
| DEVICE(MOUSE) | DEVICE(MOUSE channel, funct) | - | - | X | X | - | X |
| DEVICE(WII CLASSIC) | DEVICE(WII [CLASSIC] funct) | - | X | X | X | X | X |
| DEVICE(NUNCHUCK) | DEVICE(NUNCHUCK funct) | - | X | X | X | X | X |
| DIR$() | DIR$(fspec[, type]) | - | X | X | X | X | X |
| DISTANCE() | DISTANCE(trigger, echo) / DISTANCE(trig-echo) | - | X | X | X | X | X |

# B.5. E

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| EOF() | EOF([#]fnbr) | - | X | X | X | X | X |
| EPOCH() | EPOCH(datetime$) | - | X | X | X | X | X |
| EVAL() | EVAL(string$) | - | X | X | X | X | X |
| EXP() | EXP(number) | - | X | X | X | X | X |

# B.6. F

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| FIELD$() | FIELD$(string1, nbr, string2[, string3…]) | - | X | X | X | X | X |
| FIX() | FIX(number) | - | X | X | X | X | X |
| FLAG() | FLAG(n%) | - | X | X | X | X | X |
| FORMAT$() | FORMAT$(nbr[, fmt$]) | - | X | X | X | X | X |

## B.7. G

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| GPS() | GPS() | - | X | X | X | X | X |

## B.8. H

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| HEX$() | HEX$(number[, chars]) | - | X | X | X | X | X |

## B.9. I

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| INPUT$() | INPUT$(nbr, [#]fnbr) | 467 | X | X | X | X | X |
| INSTR() | INSTR([start-position,] string-searched$, string-pattern$) | 86 | X | X | X | X | X |
| INT() | INT(number) | - | X | X | X | X | X |

## B.10. J

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| JSON$() | JSON$(array%(), string$) | - | X | X | X | X | X |

## B.11. K

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| KEYDOWN() | KEYDOWN(n) | 346 | - | X | X | - | X |

## B.12. L

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| LCASE$() | LCASE$(string$) | 84 | X | X | X | X | X |

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| LCOMPARE() | LCOMPARE(array1%(), array2%()) | - | X | X | X | X | X |
| LEFT$() | LEFT$(string$, nbr) | 83 | X | X | X | X | X |
| LEN() | LEN(string$) | 83 | X | X | X | X | X |
| LGETBYTE() | LGETBYTE(array%(), n) | - | X | X | X | X | X |
| LGETSTR$() | LGETSTR$(array%(), start, length) | - | X | X | X | X | X |
| LINSTR() | LINSTR(array%(), search$[, start][, size]) | - | X | X | X | X | X |
| LLEN() | LLEN(array%()) | - | X | X | X | X | X |
| LINPUT() | LINPUT(array%(), fnbr, nbr) | - | X | X | X | X | X |
| LOC() | LOC([#]fnbr) | 467 | X | X | X | X | X |
| LOF() | LOF([#]fnbr) | - | X | X | X | X | X |
| LOG() | LOG(number) | - | X | X | X | X | X |

# B.13. M

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| MAP() | MAP(n) | - | X | X | X | - | X |
| MATH(ATAN3) | MATH(ATAN3 x, y) | - | X | X | X | X | X |
| MATH(COSH) | MATH(COSH a) | - | X | X | X | X | X |
| MATH(LOG10) | MATH(LOG10 a) | - | X | X | X | X | X |
| MATH(SINH) | MATH(SINH a) | - | X | X | X | X | X |
| MATH(TANH) | MATH(TANH a) | - | X | X | X | X | X |
| MATH(CRCn) | MATH(CRCn data[, length]) | - | X | X | X | X | X |
| MATH(RAND) | MATH(RAND) | - | X | X | X | X | X |
| MATH(CHI) | MATH(CHI a()) | - | X | X | X | X | X |
| MATH(CHI_P) | MATH(CHI_p a()) | - | X | X | X | X | X |
| MATH(CROSSING) | MATH(CROSSING array(), level) | - | X | X | X | X | X |
| MATH(CORREL) | MATH(CORREL a(), b()) | - | X | X | X | X | X |
| MATH(MAX) | MATH(MAX a()[, index%]) | - | X | X | X | X | X |
| MATH(MEAN) | MATH(MEAN a()) | - | X | X | X | X | X |
| MATH(MEDIAN) | MATH(MEDIAN a()) | - | X | X | X | X | X |
| MATH(MIN) | MATH(MIN a()[, index%]) | - | X | X | X | X | X |
| MATH(SD) | MATH(SD a()) | - | X | X | X | X | X |
| MATH(SUM) | MATH(SUM a()) | - | X | X | X | X | X |

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| MATH(MAGNITUDE) | MATH(MAGNITUDE v()) | - | X | X | X | X | X |
| MATH(DOTPRODUCT) | MATH(DOTPRODUCT v1(), v2()) | - | X | X | X | X | X |
| MATH(M_DETERMINANT) | MATH(M_DETERMINANT matrix()) | - | X | X | X | X | X |
| MATH(PID) | MATH(PID channel, ⋯) | - | X | X | X | X | X |
| MATH(BASE64) | MATH(BASE64 ENCODE/DECODE ⋯) | - | X | X | X | X | X |
| MAX() | MAX(arg1[, arg2[, ⋯]]) | - | X | X | X | X | X |
| MIN() | MIN(arg1[, arg2[, ⋯]]) | - | X | X | X | X | X |
| MID$() | MID$(string$, start[, nbr]) | 84 | X | X | X | X | X |

# B.14. O

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| OCT$() | OCT$(number[, chars]) | - | X | X | X | X | X |

# B.15. P

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| PEEK(BYTE) | PEEK(BYTE addr%) | - | X | X | X | X | X |
| PEEK(SHORT) | PEEK(SHORT addr%) | - | X | X | X | X | X |
| PEEK(WORD) | PEEK(WORD addr%) | - | X | X | X | X | X |
| PEEK(INTEGER) | PEEK(INTEGER addr%) | - | X | X | X | X | X |
| PEEK(FLOAT) | PEEK(FLOAT addr%) | - | X | X | X | X | X |
| PEEK(VARADDR) | PEEK(VARADDR var) | - | X | X | X | X | X |
| PEEK(VARHEADER) | PEEK(VARHEADER var) | - | X | X | X | X | X |
| PEEK(CFUNADDR) | PEEK(CFUNADDR cfun) | - | X | X | X | X | X |
| PEEK(VAR) | PEEK(VAR var, ±offset) | - | X | X | X | X | X |
| PEEK(VARTBL) | PEEK(VARTBL, ±offset) | - | X | X | X | X | X |
| PEEK(PROGMEM) | PEEK(PROGMEM, ±offset) | - | X | X | X | X | X |
| PEEK(BP/SP/WP) | PEEK(BP n%) / PEEK(SP n%) / PEEK(WP n%) | - | X | X | X | X | X |
| PIN() | PIN(pin) | 265 | X | X | X | X | X |
| PIN(BOOTSEL) | PIN(BOOTSEL) | - | X | X | X | X | X |
| PIN(TEMP) | PIN(TEMP) | - | X | X | X | X | X |
| PIO(DMA POINTER) | PIO(DMA RX/TX POINTER) | - | X | X | X | X | X |

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| PIO(SHIFTCTRL) | PIO(SHIFTCTRL …) | - | X | X | X | X | X |
| PIO(PINCTRL) | PIO(PINCTRL …) | - | X | X | X | X | X |
| PIO(EXECCTRL) | PIO(EXECCTRL …) | - | X | X | X | X | X |
| PIO(READFIFO) | PIO(READFIFO a,b,c) | - | X | X | X | X | X |
| PIO(FDEBUG) | PIO(FDEBUG pio) | - | X | X | X | X | X |
| PIO(FSTAT) | PIO(FSTAT pio) | - | X | X | X | X | X |
| PIO(FLEVEL) | PIO(FLEVEL pio[, sm, dir]) | - | X | X | X | X | X |
| PIO(.WRAP) | PIO(.WRAP) / PIO(.WRAP TARGET) | - | X | X | X | X | X |
| PIO(NEXT LINE) | PIO(NEXT LINE) | - | X | X | X | X | X |
| PIXEL() | PIXEL(x, y) | - | X | X | X | - | X |
| PORT() | PORT(start, nbr[, start, nbr]…) | - | X | X | X | X | X |
| PULSIN() | PULSIN(pin, polarity[, t1[, t2]]) | - | X | X | X | X | X |

# B.16. R

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| RAD() | RAD(degrees) | - | X | X | X | X | X |
| RGB() | RGB(red, green, blue) / RGB(shortcut) | 127 | X | X | X | - | X |
| RIGHT$() | RIGHT$(string$, number-of-chars) | 83 | X | X | X | X | X |
| RND() | RND(number) | 48 | X | X | X | X | X |

# B.17. S

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| SGN() | SGN(number) | - | X | X | X | X | X |
| SIN() | SIN(number) | 185 | X | X | X | X | X |
| SPACE$() | SPACE$(number) | - | X | X | X | X | X |
| SPI() | SPI(data) | - | X | X | X | X | X |
| SPI2() | SPI2(data) | - | X | X | X | X | X |
| SPRITE() | SPRITE() | 506 | X | X | X | - | X |
| SQR() | SQR(number) | 20 | X | X | X | X | X |
| STR$() | STR$(number[, m[, n[, c$]]]) | 86 | X | X | X | X | X |
| STR2BIN() | STR2BIN(type, string$[, BIG]) | - | X | X | X | X | X |
| STRING$() | STRING$(nbr, ascii\|string$) | - | X | X | X | X | X |
| STRUCT() | STRUCT(arg, …) | - | X | X | X | X | X |

## B.18. T

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| TAB() | TAB(number) | - | X | X | X | X | X |
| TAN() | TAN(number) | - | X | X | X | X | X |
| TEMPR() | TEMPR(pin[, timeout]) | - | X | X | X | X | X |
| TOUCH(X/Y) | TOUCH(X) / TOUCH(Y) | - | X | - | - | - | X |
| TOUCH(X2/Y2) | TOUCH(X2) / TOUCH(Y2) | - | X | - | - | - | X |
| TRIM$() | TRIM$(source$[, mask$][, where]) | - | X | X | X | X | X |

## B.19. U

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| UCASE$() | UCASE$(string$) | 84 | X | X | X | X | X |

## B.20. V

| Funktion | Syntax | Im Buch | P | V | H | W | K |
|---|---|---|---|---|---|---|---|
| VAL() | VAL(string$) | 86 | X | X | X | X | X |

# Stichwortverzeichnis


**A**

Abprallwinkel, 313
ABS(), 26, 26
ADC, 376
AIN, 379
Aktiver Summer, 398
Aktives Laufwerk, 463
Ampelsteuerung, 363
  Anforderung speichern, 369
  Fußgängeranforderung, 369
Analog-Digital-Wandler, 376
Analoge Eingänge, 375
  Spannungsmessung, 376
Analogsticks, 587
Anforderung speichern, 369
Animation, 278, 605
  Bildschirmflackern, 286
  Kollisionserkennung, 274
  Nachzieheffekt, 280
Anode, 341
ANSI-Steuersequenzen, 551
APPEND, 152
ARAW, 377
ARC, 220
Array-Elemente, 97
Arrays, 97
  Mehrdimensionale Arrays, 107
  Textarrays, 100
Audio
  Frequenz, 400
  Melodie, 402
  Tonhöhe, 399
Audioausgabe, 398
Automatisierung, 368
  Zustandssteuerung, 366

**B**

Baudrate, 548
Beautify, 429
Bildschirmauflösung, 180, 180
Bildschirmflackern, 286
Bildschirmschoner, 278
Bit Block Transfer, 609
Bitmaske, 583
BLIT, 597
  CLOSE, 610
  COMPRESSED, 611
  FRAMEBUFFER, 611
  LOAD, 610
  MEMORY, 611
  MERGE, 611
  READ, 609
  RESIZE, 611
  Scrollen, 610
  WRITE, 609
BLIT-Puffer, 609
Body-Mass-Index, 93
Boot-Ursache, 492

**C**

CASE, 367
CHAIN, 476
  Pfadangaben, 484
  Programmverkettung, 476
  RUN im Vergleich, 480
  Startparameter, 479
  Variablenbereich, 476
CHDIR, 466
CIRCLE, 208, 209
  Seitenverhältnis, 220
CLOSE, 151
CLS, 193
COLOUR, 193
COM-Port, 438
Community, 648
Computergesteuerter Gegner, 311
CONST, 102, 280, 280
COPY, 467
CR/LF, 550
CTRL+C, 77
CWD$, 466

**D**

DATA, 404
DATE$, 118, 118, 530
Dateibefehle
  CLOSE, 151
  LINE INPUT #, 151
  LIST, 465
  OPEN, 150

PRINT #, 150
READ #, 153
WRITE #, 153
Dateien, 150
Dateimanager, 443
Dateimodi, 154
APPEND, 152
INPUT, 151
OUTPUT, 150
RANDOM, 154
Dateisystem
Flash-Dateisystem, 450
SD-Karte, 451
Dateiverwaltung
Aktives Laufwerk, 463
CHDIR, 466
COPY, 467
CWD$, 466
DRIVE, 451
FILES, 70
KILL, 468
LOAD, 71
MKDIR, 465
Pfadangaben, 471
RENAME, 467
RMDIR, 469
SAVE, 70
Verzeichnisse, 463
Wildcards, 464
Datenlogger
Zeitstempel, 540
Datentyp, 31
Datentypen
Standard-Datentyp, 503
Datum und Uhrzeit
DATE$, 118
TIME$, 118
DEVICE()
GAMEPAD, 582
DEVICE(GAMEPAD), 582
Digitale Ein- und Ausgänge, 333
Digitale Eingänge
Flankenerkennung, 359
Digitale Signale
HIGH, 333
LOW, 333
DIM, 33, 97
DIN, 335
Division
ganzzahlige Ergebnisse, 136
DO, 77
Dokumentation, 647
DOUT, 334
DRIVE, 451

**E**

Ebenentrennung, 302
Echtzeituhr, 532
EDIT, 49, 424
EDIT mit Dateinamen, 426
Editor
Beautify, 429
Dateiexport, 429
Dateiimport, 429
EDIT, 424
EDIT mit Dateinamen, 426
Farbsyntax, 431
HELP, 430
Markiermodus, 428
Maus, 131
Suchen, 427
Suchen und Ersetzen, 428
Vollbild-Editor, 424
Eingabefolge prüfen, 416
Eingabemethoden, 629
Ellipsen, 220
ELSE, 51
ELSEIF, 53
END FUNCTION, 87
END SELECT, 367
END SUB, 86
Endlosschleife, 77
Entfernungsberechnung, 227
Entprellung, 358
EXIT FOR, 63
EXIT FUNCTION, 415

**F**

Fade, 574
FALSE, 102
Farben
Farbkonstanten, 192
RGB, 192
RGB888, 570
RGBW, 575
Farbkonstanten, 192

Farbsyntax, 431, 431
Farbtiefe, 191
Fehleranalyse, 147
FILES, 70
Firmware-Version, 14, 647
Flackerfreie Grafik, 289
Flag-Prinzip, 520
Flankenerkennung, 359
FLASH SAVE, 453, 453, 453, 453, 453
Flash-Dateisystem, 450
Flash-Slots, 453
Floating Input, 353
FONT, 194
FOR, 57
Foren, 648
Framebuffer, 257, 598
  Flackerfreie Grafik, 289
FRAMEBUFFER CLOSE, 294
FRAMEBUFFER COPY, 291
FRAMEBUFFER COPY B, 316
FRAMEBUFFER CREATE, 290
FRAMEBUFFER LAYER, 299
FRAMEBUFFER WRITE, 290
FRAMEBUFFER WRITE L, 299
FRAMEBUFFER WRITE N, 299
Freilaufdiode, 345
Frequenz, 400
FUNCTION, 87
Fußgängeranforderung, 369

**G**

Game Over, 640
Gamepad, 579
  Analogsticks, 587
  Bitmaske, 583
  Totzone, 587
  Vibration, 591
GAMEPAD COLOUR, 591
GAMEPAD CONFIGURE, 582
GAMEPAD HAPTIC, 591
GAMEPAD INTERRUPT DISABLE, 591
GAMEPAD INTERRUPT ENABLE, 591
GAMEPAD MONITOR, 582
Gamepad-Interrupts, 591
Gedächtnisspiel, 409
Geometrische Muster, 263
  Rosetten, 265
  Spirograph, 266
GPIO-Befehle
  PIN(), 334
  SETPIN, 334
GPIO-Pins, 333
  Ausgang, 333
  Eingang, 333
  Floating Input, 353
  GPx, 334
  Pull-Down-Widerstand, 353
  Pull-Up-Widerstand, 353
  Treiberschaltung, 345
GPx, 334
Grafik
  Bildaustastlücke, 316
  Bildschirmauflösung, 180
  Farbtiefe, 191
  Framebuffer, 257
  Koordinatensystem, 179
  Layerbuffer, 295
  Pixel, 179
  Schriftarten, 194
  Turtle-Grafik, 211
Grafikbefehle
  ARC, 220
  BOX, 209
  CIRCLE, 208
  CLS, 193
  COLOUR, 193
  FONT, 194
  LINE, 206
  MODE, 181
  PIXEL, 205
  RBOX, 218
  TEXT, 221
Grafikfunktionen
  RGB(), 185
Grundrechenarten, 24

**H**

Haptisches Feedback, 591
Hardware-Projekt, 409
HELP, 430
HIGH, 333
Highscore, 237

**I**

IF, 50
Index, 97

INPUT, 39
INPUT (Dateimodus), 151
INPUT$(), 549
INSTR, 117
INT, 66
INTB, 519
Interrupt-Konfiguration
  INTB, 519
  INTH, 519
  INTL, 519
  OFF, 524
Interrupt-Subroutine, 521
Interrupts, 517
  Entprellung, 522, 523
  Flag-Prinzip, 520
  Gamepad-Interrupts, 591
  INPUT, 526
  Interrupt-Subroutine, 521
  Multitasking, 518
  Pin-Interrupt, 519
  Polling, 517
  Priorität, 526
  Serielle Interrupts, 559
  Signalflanken, 519
  verlorene Interrupts, 526
INTH, 519
INTL, 519

**K**

Kalender
  Monatskalender, 536
  Schaltjahr, 537
  Wochentag, 538
Kathode, 341
KEYDOWN(), 415
KILL, 468
Klammern
  Rechenausdrücke, 25
Kollisionserkennung, 274, 597
Konvertierungsfunktionen
  STR$, 117
  VAL, 117
Koordinatensystem, 179
Kreisberechnung, 263

**L**

Lauflicht, 573
Laufzeitzähler, 540
Layer, 602
Layerbuffer, 295
  Bewegliche Objekte, 298
  Statischer Hintergrund, 298
LCASE$, 115
LED, 341
  Anode, 341
  Dimmen, 392
  Fade, 574
  Helligkeitssteuerung, 573
  Kathode, 341
  Lauflicht, 573
  Regenbogen, 575
  SK6812, 570
  SK6812W, 570
  Vorwiderstand, 342
  WS2812, 567
  WS2812B, 570
LED-Dimmer, 392
LED-Kette, 568
LEFT$, 114
LEN, 114
Levelwechsel, 638
LGETSTR$(), 557
Library, 454
LIBRARY DELETE, 457
LIBRARY DISK LOAD, 457
LIBRARY DISK SAVE, 457
LIBRARY LIST, 456
LIBRARY SAVE, 455
LIFO, 602
LINE, 206
LINE INPUT #, 151
LIST, 465
LLEN(), 556
LOAD, 71
Load and Run, 438
LOC(), 549
Long Strings, 112, 555
LONGSTRING APPEND, 556
LONGSTRING PRINT, 556
LOOP, 77
Lottozahlen, 102
LOW, 333

**M**

Magische Zahlen, 625
Markiermodus, 428

Mathematische Funktionen
ABS(), 26
COS(), 253
INT, 66
SIN(), 253
SQR(), 26
Mathematische Konstanten
PI, 253
Maus
MOUSE(), 240
Maus im Editor, 131
MAX3232, 562
Maximite Control Centre, 436
Mehrdimensionale Arrays, 107
Melodie, 402
Messwerte
Glättung, 383
Messwertglättung, 383
MID$, 115
MKDIR, 465
MM.CMDLINE$, 479
MM.DEVICE$, 255
MM.END, 459
MM.FONTWIDTH, 278, 278
MM.HRES, 180
MM.INFO, 491
MM.INFO$(), 491
MM.INFO$(BOOT), 492
MM.INFO$(CPUSPEED), 491
MM.INFO$(CURRENT), 495
MM.INFO(), 491
MM.INFO(disk size), 470
MM.INFO(drive), 464
MM.INFO(exists dir), 469
MM.INFO(exists file), 469
MM.INFO(FONTHEIGHT), 182
MM.INFO(FONTWIDTH), 182
MM.INFO(free space), 470
MM.INFO(HEAP), 491
MM.INFO(HPOS), 491, 491
MM.INFO(path), 470
MM.INFO(PINNO), 380
MM.INFO(SYSTEM HEAP), 491
MM.INFO(UPTIME), 492
MM.INFO(USB n), 579
MM.INFO(VERSION), 492
MM.PROMPT, 459
MM.STARTUP, 458
MM.VRES, 180
MMCC, 436
COM-Port, 438
Dateimanager, 443
Serielle Verbindung, 438
Terminal, 439
MMEdit, 436
Eingebauter Editor, 441
Load and Run, 438
Registerkarten, 444
Syntax-Hervorhebung, 440
MOD, 61
MODE, 181
Monatskalender, 536
MOUSE(), 240
MOUSE(L), 240
MOUSE(X), 240
MOUSE(Y), 240

**N**

Nachzieheffekt, 280
Neustart, 641
NEW, 49
NEXT, 57
NPN-Transistor, 345, 345

**O**

OPEN, 150
OPTION, 501
OPTION AUTORUN, 433
OPTION BASE, 280
OPTION DEFAULT, 503
OPTION EXPLICIT, 33, 440
OPTION KEYBOARD, 508
OPTION KEYBOARD REPEAT, 510
OPTION LIST, 502
OPTION MOUSE ON, 131
OPTION RESET, 513
OPTION RESET LIST, 513
OPTION RESOLUTION, 507
OPTION RTC AUTO ENABLE, 532
OPTION SERIAL CONSOLE, 593
OPTION TAB, 506
OUTPUT, 150

**P**

Parameter, 87
Passiver Piezo-Schallwandler, 398

PAUSE, 210
Pegelwandler
  MAX3232, 562
Pegelwandlung, 569
Pfadangaben, 471
PI, 253, 253, 253, 253
PicoMite User Manual, 647
Piezo-Summer, 398
  Aktiver Summer, 398
  Passiver Piezo-Schallwandler, 398
PIN(), 334
Pin-Interrupt, 519
Pin-Modi
  AIN, 379
  ARAW, 377
  DIN, 335
  DOUT, 334
  PULLUP, 354
  PWM0A, 390
PIXEL, 205
Pixel, 179
Plattform, 13
PLAY SOUND, 636
PLAY TONE, 399
Polling, 517
Potentiometer, 377
  Spannungsteiler, 378
Potenzierung, 25
Prellen, 358
PRINT, 21, 21, 23, 29
PRINT #, 150
PRINT @(), 183
Problemzerlegung, 643
Programmierung
  Magische Zahlen, 625
  Problemzerlegung, 643
  Testen und Verbessern, 647
Programmplanung, 621
Programmspeicher, 450
Programmstruktur, 622
Programmverkettung, 476
Projektorganisation, 444
Pseudozufallszahlen, 67
Pull-Up-Widerstand, 353, 353
PULLUP, 354
Pulsweitenmodulation, 388
Punktesystem, 232
Punkteverwaltung, 314
PWM, 388
  Duty Cycle, 389
  PWM-Kanal, 390
  Tastverhältnis, 389
  Tiefpassfilter, 393
PWM (Befehl), 390
PWM OFF, 391
PWM0A, 390

**R**

RANDOM, 154
RANDOMIZE, 68
RBOX, 218
READ #, 153
Reaktionstest, 524
Rechenarten, 134
Regenbogen, 575
Registerkarten, 444
RENAME, 467
RGB(), 185
RGB121, 597
RGB888, 570, 597
RGBW, 575
RIGHT$, 114
RMDIR, 469
RND, 66
Rosetten, 265
RS-232, 562
RTC, 532
RUN, 49
RX, 547
Rückgabewert, 88

**S**

SAVE, 70
Schalter, 352
Schaltjahr, 537
Schleifen, 57
Schleifenbedingung, 77
Schleifenvariable, 60
Schlüsselwörter, 30
Schreibgeschützte Systemvariablen, 496
Schriftarten, 194
Schwierigkeitsstufen, 233
SD-Karte, 451
SELECT CASE, 366
Serielle Interrupts, 559
Serielle Kommunikation, 547

CR/LF, 550
INPUT$(), 549
LOC(), 549
Textprotokoll, 554
Serielle Schnittstellen, 546, 546, 546
COM1, 546
COM2, 546
Serielle Verbindung, 438
SETPIN, 334
OFF, 524
Signalflanken, 519
Spannungsmessung, 376
Spannungsteiler, 378
Speicherbereiche, 450
Spieleentwicklung
Programmplanung, 621
Programmstruktur, 622
Spielidee, 619
Spielregeln, 619
Spieleprogrammierung
Abprallwinkel, 313
Computergesteuerter Gegner, 311
Ebenentrennung, 302
Eingabefolge prüfen, 416
Eingabemethoden, 629
Game Over, 640
Gedächtnisspiel, 409
Highscore, 237
Kollisionserkennung, 312
Levelwechsel, 638
Neustart, 641
Punktesystem, 232
Punkteverwaltung, 314
Schwierigkeitsstufen, 233
Spielschleife, 310, 639
Spielidee, 619
Spielregeln, 619
Spielschleife, 310
Spirograph, 266
Überlagerung von Kreisbewegungen, 267
SPRITE, 597
CLOSE, 601
HIDE, 601
INTERRUPT, 606
LOAD, 599
LOADARRAY, 599, 614
LOADPNG, 599
MOVE, 604
NEXT, 604
SET TRANSPARENT, 603
SHOW, 600
SHOW SAFE, 603
SWAP, 605
Sprite, 597
SPRITE COPY, 627
SPRITE HIDE SAFE, 636
SPRITE()
Positions- und Größenabfragen, 608
SPRITE(C, n), 607
SPRITE(S), 607
Sprite-Demo, 614
SQR(), 26, 26
Standard-Datentyp, 503
Startparameter, 479
STEP, 59
Stoppuhr, 534
STR$, 41, 117
String, 31
String-Funktionen
INSTR, 117
LCASE$, 115
LEFT$, 114
LEN, 114
MID$, 115
RIGHT$, 114
UCASE$, 115
String-Variablen, 31
Strings
Long Strings, 555
SUB, 86, 86
Syntax-Hervorhebung, 440
Systemroutinen
MM.END, 459
MM.PROMPT, 459
MM.STARTUP, 458
Systemvariablen
MM.CMDLINE$, 479
MM.DEVICE$, 255
MM.FONTHEIGHT, 278
MM.FONTWIDTH, 278
MM.HRES, 180
MM.VRES, 180
Schreibgeschützt, 496


**T**


Tastatur

KEYDOWN(), 415
Tastendruck auswerten, 415
Taster, 352
Entprellung, 358
Prellen, 358
Tastendruck auswerten, 415
Temporäre Optionen, 501, 501
Terminal, 439
VT100, 551
TEXT, 221
Ausrichtung, 222
Drehung, 223
Schriftart, 222, 222
Transparenter Hintergrund, 223
Textarrays, 100
Textprotokoll, 554
TIME$, 532
TIMER, 359, 522
Tonhöhe, 399
Totzone, 587
Transparenz, 603
Treiberschaltung, 345
Freilaufdiode, 345
MOSFET, 345
NPN-Transistor, 345
TRUE, 102
TURTLE FORWARD, 212
TURTLE RESET, 212
TURTLE RIGHT, 213, 213
Turtle-Grafik, 211
TX, 547

**U**

UART, 547
Baudrate, 548
TX und RX, 547
USB-UART-Adapter, 551
Zeichenstrom, 550
UCASE$, 115
UNTIL, 78
USB
Kanäle, 580
USB-Host, 593
USB-Kanäle, 580
USB-UART-Adapter, 551

**V**

VAL, 40, 117
Variablen, 28
Variablenbereich, 476
Variablennamen, 30
Vergleichsoperatoren, 52
Verschachtelte Entscheidungen, 54
verschachtelte Schleifen, 63
Verzeichnisse, 463
Vollbild-Editor, 424
Vorwiderstand, 342
VT100, 551

**W**

WHILE, 78
Wildcards, 464
Winkel
Bogenmaß, 253
Wochentag, 538
WRITE #, 153
WS2812, 567
Helligkeitssteuerung, 573
LED-Kette, 568
Pegelwandlung, 569
Stromversorgung, 569
WS2812 (Befehl), 567

**Z**

Zeichenketten, 112
Zeichenstrom, 550
Zeilennummern, 51
Zeitmessung, 534
Laufzeitzähler, 540
Reaktionstest, 524
Stoppuhr, 534
TIMER, 359
Zeitstempel, 540
Zufallsfunktionen
RND, 66
Zufallszahlen, 65
Zustandssteuerung, 366
Zähler, 35
Zählvariable, 57